

A SELF-STUDY BOOK FOR THE CORE-SUBJECTS ROUND

CS Fundamentals for the Technical Round

Operating Systems · Databases · Networks · OOP and dev fundamentals

WHAT THIS BOOK IS

The round where an interviewer puts the resume down and asks core-subject questions one after another. Every topic here is a **question, a crisp answer, and then the worked example or diagram that makes the answer stick**. Service companies ask these heavily — often more than DSA. Product companies ask fewer, and go deeper.

- TIER 1** Indian service companies — TCS NQT, Accenture, Infosys, Wipro, Capgemini, Cognizant
- TIER 2** Singapore & Thailand — Grab, Sea/Shopee, GIC, DBS, Agoda, SCB, LINE MAN, Razer
- TIER 3** Global product companies — Google, Amazon, Microsoft, Goldman Sachs, D. E. Shaw, Adobe, Uber

12 chapters · diagrams · runnable JavaScript and real SQL · rapid-fire one-liners · one-page revision cards

AN HONEST NOTE ON WHERE THIS MATERIAL COMES FROM

Every question, answer, example, table and diagram in this book is **original**. Each one was written for this book.

Nothing here is copied, quoted or paraphrased from any published book, course, question bank, documentation page or website.

No company's actual interview, test paper or question list is reproduced here. Real hiring questions are private property, and they are not public documents.

When an item is labelled, for example, "TCS NQT · pattern" or "Grab · pattern", that label means one thing only: the item was written to match the **publicly documented shape** of that kind of round — its topic mix, its usual depth, and the follow-up it tends to attract. The label is never a claim that this question, or anything resembling it, was asked at a real company.

Company names appear only to describe that shape. This book is not affiliated with, endorsed by, or connected to any company named in it. All trademarks belong to their owners.

Every JavaScript and Python listing was executed before it was printed, and every SQL query was checked by hand against the sample tables shown beside it. Where the text claims an output, that output came from a real run. Numbers such as seek times, page sizes and round-trip times are stated as **illustrative** values for the arithmetic — your machine and your network will differ, and the method is the part that transfers.

Contents

How to use this book	4
Chapter 1 · OS: Processes, Threads & Scheduling	6
Chapter 2 · OS: Synchronisation & Deadlock	33
Chapter 3 · OS: Memory Management	61
Chapter 4 · OS: File Systems, I/O & Storage	88
Chapter 5 · DBMS: Relational Model, Keys & SQL	114
Chapter 6 · DBMS: Normalisation & Schema Design	143
Chapter 7 · DBMS: Transactions, Concurrency & Recovery	169
Chapter 8 · DBMS: Indexing & Query Performance	196
Chapter 9 · Networks: Layers, IP & Routing	227
Chapter 10 · Networks: TCP, UDP & HTTP	254
Chapter 11 · OOP: Principles & Language Mechanics	284
Chapter 12 · Compilers, Git & Dev Fundamentals	321
Appendix · Quick Reference	358

How to use this book

Two pages. Read them once, then start Chapter 1.

1 • What the tiers mean in this book

Inside every chapter the same ladder repeats. Climb it in order — a Tier 3 answer is reachable only because you already wrote the Tier 1 and Tier 2 ones.

Tier	Who asks like this	What the question feels like	Answer
0 Warm-up	nobody — it is you	Bare mechanics, one idea, no story. Fill the table, translate the address, name the state.	1–2 lines
1 Service	TCS NQT & Digital, Accenture, Infosys, Wipro, Capgemini, Cognizant	Definitions and standard comparisons, rapid-fire. “Process vs thread.” “DELETE vs TRUNCATE.” Recall and speed, not insight.	3–6 lines
2 SE Asia	Grab, Sea/Shopee, GIC, DBS, Agoda, SCB, LINE MAN, Razer	The same theory in a business suit — a slow orders query, a double-charged wallet, a laggy API. Find the concept under the story.	8–15 lines
3 Product	Google, Amazon, Microsoft, Goldman Sachs, D. E. Shaw, Adobe, Uber	One genuine insight, then follow-ups: “now at 10x traffic”, “now across two data centres”, “what breaks first”. Naming the trade-off is the answer.	discussion

2 • What is in every chapter, and what to do with it

Formulas box	Every definition and law the chapter needs, in one block. Cover it and write it out — this is the part you must be able to recite.
Diagrams	Process states, memory layout, the handshake, a B+ tree. Redraw each on blank paper; interviewers ask you to draw , not to describe.
Worked Q → A	Say your answer out loud before reading the solution, then mark the gap. Reading first teaches nothing.
TRAP boxes	The classic wrong answer and why it is wrong. These are the exact lines that lose marks. Read every one twice.
Code and SQL	Type it and run it — <code>node f.js</code> , <code>python3 f.py</code> , or a real SQL shell. A concept you have executed is one you can defend.
Rapid fire	20–30 one-line answers per chapter. Last-week material, not first-week material.
Revision card	One page at the chapter end. Write it from memory when you finish the chapter; the gaps are your revision list.

3 • The four rules of this round

1	Answer in one sentence, then stop. Give the definition and wait. Interviewers ask the follow-up themselves; rambling turns one question into a hole.
2	Every comparison needs a “because”. “TRUNCATE is faster” is half an answer. “...because it deallocates pages instead of logging each row” is the whole one.
3	Draw it. Page tables, the handshake, a deadlock cycle, a B+ tree. If you can draw it, you can explain it under pressure.
4	Say “I do not know” cleanly, then reason. “I have not used it, but from first principles it would have to...” scores far above a confident guess.

TRAP

Three answers sink this round more than any others. Threads do **not** share the stack — code, data and heap only. Deadlock needs **all four** conditions at once, not any one of them. And an index does **not** always make things faster — it slows every write.

4 • The 6-week plan

Two chapters a week, about two hours a day, six days a week. Day 7 is the checkpoint — blank paper, nothing open.

Week	Chapters	Daily target	Checkpoint (day 7)
1	1, 2	Ch 1 on days 1–3: state diagram, then every scheduling algorithm with the Gantt arithmetic done in full. Ch 2 on days 4–6.	Draw the process state diagram and the four deadlock conditions from memory.
2	3, 4	Ch 3 days 1–3 — paging, TLB, and every page-replacement algorithm worked by hand. Ch 4 days 4–6, including the disk-seek arithmetic.	Translate three virtual addresses, then run one FIFO, one LRU and one Optimal trace.
3	5, 6	Ch 5 with a real SQL shell open — type every query. Ch 6 days 4–6: attribute closure until it is automatic, then 1NF to BCNF decompositions.	Find all candidate keys of a fresh relation, then normalise it to BCNF.
4	7, 8	Ch 7: the four isolation levels and exactly which anomaly each one stops. Ch 8: B+ tree structure, EXPLAIN reading, and when an index hurts.	Write the isolation-versus-anomaly grid from memory, then justify three index choices.
5	9, 10	Ch 9: subnetting until you no longer need the table. Ch 10: handshake, congestion control with the numbers, HTTP/1.1 vs 2 vs 3.	Subnet one block four ways, then draw the handshake and the close, labelling every state.
6	11, 12	Ch 11: the four pillars in Java/C++ vocabulary as well as the JS reality. Ch 12: compilation stages, memory layout, git internals.	Overloading vs overriding, abstract vs interface, merge vs rebase — out loud, under a minute each.

5 • The 10-day crash plan

Use this only if the interview is inside two weeks. It cuts practice volume, not topics. About four hours a day. Each day: read the formulas box, work the Tier 1 items, read every TRAP box, then the rapid-fire list.

Day	Ch	Priority inside the chapter	Must be able to do by tonight
1	1	Process vs thread, state diagram, all scheduling algorithms.	Average waiting time under FCFS, SJF, SRTF and RR.
2	2	Race condition, mutex vs semaphore, the four deadlock conditions.	Banker's algorithm on a small table.
3	3	Paging, TLB, page faults, the replacement algorithms.	Effective access time, and an LRU trace.
4	4	Inodes, hard vs soft link, the disk-scheduling algorithms.	Total head movement for SCAN and C-SCAN.
5	5	Keys, all join types, GROUP BY vs HAVING, NULL logic.	Write a three-table join with an aggregate, first try.
6	6	Attribute closure, candidate keys, 2NF / 3NF / BCNF.	Normalise a given relation and say which rule broke.
7	7, 8	ACID and the isolation grid; then B+ trees and selectivity.	Name the anomaly each level stops, and when an index hurts.
8	9	OSI vs TCP/IP, subnetting, ARP, DNS resolution order.	Subnet a /24 into six usable networks.
9	10	Handshake, TCP vs UDP, HTTP methods and status codes, TLS.	Draw the handshake and explain why TIME_WAIT exists.
10	11, 12	Four pillars, overloading vs overriding; memory layout, git.	Read the whole Quick Reference appendix at the back of this book.

The last hour before the interview: the Quick Reference appendix only, plus the revision cards of your two weakest chapters. Nothing new. Nothing long.

OS: Processes, Threads & Scheduling

The CPU has one seat. This chapter is about who sits in it, and for how long.

What this round actually looks like

You sit down. The interviewer has your resume in one hand and a list in the other. Then:

- “What is the difference between a process and a thread?”
- “Draw the process state diagram.”
- “Here are four processes with arrival and burst times. Give me the average waiting time under SJF.”
- “Why is Round Robin fair but slow?”

Three of those four are **definition** questions. One is **arithmetic**. Neither needs cleverness. Both need practice. That is what this chapter gives you: the words to say, and the tables to fill in without a single slip.

EVERY FORMULA THIS CHAPTER NEEDS

The five per-process numbers. Let AT be arrival time, BT be burst time (how much CPU the process needs in total), CT be completion time.

$$\text{Turnaround Time (TAT)} = \text{CT} - \text{AT}$$

$$\text{Waiting Time (WT)} = \text{TAT} - \text{BT}$$

$$\text{Response Time (RT)} = (\text{time the CPU first runs it}) - \text{AT}$$

Read them in English:

- TAT = total time inside the system, from the moment it walked in to the moment it left.
- WT = the part of that time it spent **not** on the CPU.
- RT = how long before the user saw **anything** happen.

The two system-wide numbers.

$$\text{Throughput} = \frac{\text{number of processes finished}}{\text{total time}}$$

$$\text{CPU utilisation} = \frac{\text{time CPU was doing useful work}}{\text{total time}} \times 100\%$$

Averages. Add the column, divide by the number of processes. Nothing more.

CPU utilisation with n processes in memory, each blocked on I/O a fraction p of the time:

$$\text{utilisation} = 1 - p^n$$

Amdahl's law — the ceiling on what more cores can buy you. If a fraction f of the work can run in parallel and the rest cannot:

$$\text{speedup}(n) = \frac{1}{(1-f) + \frac{f}{n}} \quad \text{and as } n \rightarrow \infty, \quad \text{speedup} \rightarrow \frac{1}{1-f}$$

Next-burst prediction (exponential average), used by practical SJF:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n \quad \text{with } 0 \leq \alpha \leq 1$$

Response ratio, used by HRRN:

$$\text{RR} = \frac{\text{waiting time} + \text{burst time}}{\text{burst time}}$$

Part 1 — The process

Program vs process

A **program** is a file on disk. Bytes. It does nothing.

A **process** is that program **in execution**: the code, plus its current memory, plus the value of every CPU register, plus its open files. It is **active**.

SHORTCUT

One-line answer to memorise: “A program is passive — it is a file. A process is active — it is a program that has been loaded into memory and given resources by the OS. One program can produce many processes: three browser tabs, three processes, one chrome binary.”

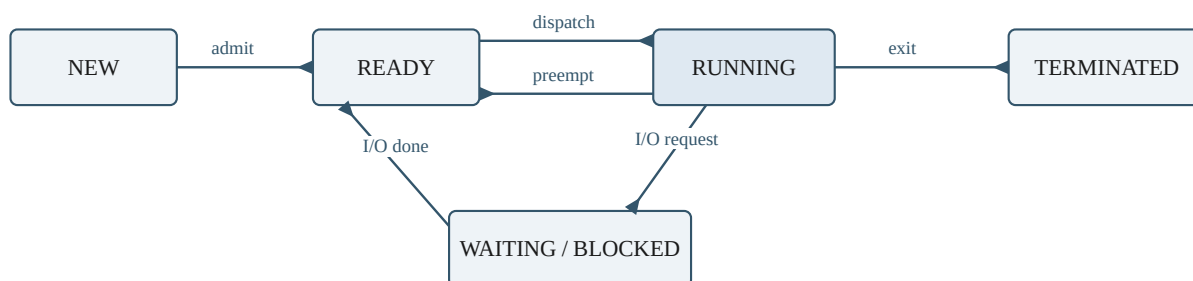
The Process Control Block (PCB)

The PCB is the OS’s record card for one process. When the interviewer asks “what does the OS store about a process?”, say these seven:

Field	Why it exists
PID (process id)	a unique number to name it
Process state	new / ready / running / waiting / terminated
Program counter	which instruction to resume at
CPU registers	the values that were in the registers when it was thrown off the CPU
Scheduling info	priority, how long it has waited, which queue it is in
Memory info	page table or base and limit registers
I/O and accounting	open file table, CPU time used, parent PID

The PCB is **in kernel memory**, not in the process’s own memory. That is on purpose — if a buggy process could overwrite its own PCB it could give itself any priority it liked.

The five states



The five-state process model. Only ONE arrow enters Running, and it comes from Ready.

Read the diagram out loud, because that is exactly how you answer it:

1. **New** — the PCB has been created, the process is not yet in memory.
2. **Ready** — in memory, has everything it needs, waiting only for the CPU.
3. **Running** — actually on the CPU. On a single-core machine, exactly one process is here.

4. **Waiting / Blocked** — it asked for something slow (a disk read, a network packet, a keypress) and cannot use the CPU until that arrives.
5. **Terminated** — finished; the OS is reclaiming its resources.

TRAP

The single most common wrong answer in this round: “**Waiting goes back to Running.**” It does not. A blocked process whose I/O completes goes to **READY**, and then must be **dispatched** like everybody else. There is no shortcut back onto the CPU.

Also: there is **no arrow from Ready to Waiting**. A process cannot block on I/O without first running the instruction that asks for the I/O — and to run an instruction it must be Running.

Example 1 [warm-up]

A process is in the RUNNING state. Its time quantum expires. Which state does it move to?

- (a) New (b) Ready (c) Waiting (d) Terminated

SOLUTION

A quantum expiry means “your turn is over”, not “you are waiting for a device”. The process still has everything it needs. It goes back into the queue of things that could run.

Answer: (b) Ready

Example 2 [Infosys · pattern]

A process issues a `read()` on a file that is not in the page cache, and later the disk delivers the data. Write the full sequence of states from just before the `read()` to just after the process resumes using the data.

SOLUTION

Step by step:

1. Before the call the process is on the CPU: **RUNNING**.
2. `read()` is a system call. The data is not ready, so the kernel cannot return. It marks the process **WAITING** and puts its PCB on the disk device’s wait queue.
3. The scheduler now picks somebody else. Our process is off the CPU.
4. The disk finishes and raises an interrupt. The interrupt handler moves our process from the device queue to the ready queue: **READY**.
5. Some time later the scheduler dispatches it: **RUNNING** again, resuming at the instruction after `read()`.

Answer: RUNNING → WAITING → READY → RUNNING

Notice step 4 and step 5 are separate. Being unblocked is not the same as being scheduled.

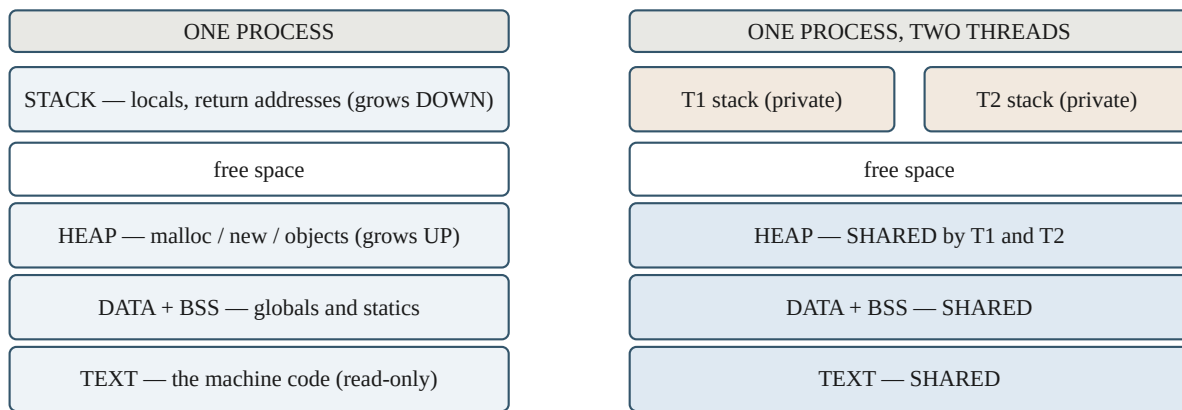
Two more states you should name

Some textbooks and most real systems add **suspended** states, reached when the OS swaps a process out to disk to free memory:

- **Ready-suspended**: was ready, now swapped out to disk.
- **Blocked-suspended**: was blocked, now swapped out to disk.

Say this only if asked “are there any other states?” — otherwise five is the expected answer.

Part 2 — What a process looks like in memory



Left: the address space of one process. Right: two threads inside one process — everything is shared except the stack (and the registers).

Segment	Holds	Lifetime
Text	compiled instructions	whole run; usually read-only and shareable
Data	initialised globals, e.g. <code>int n = 5;</code>	whole run
BSS	uninitialised globals, e.g. <code>int n;</code>	whole run; zero-filled at load
Heap	memory you asked for at run time	until you free it
Stack	one frame per function call: parameters, locals, return address	until the function returns

TRAP

“Stack overflow” and “heap overflow” are different bugs. **Stack overflow** = too many nested calls (usually runaway recursion) so the stack grows down into the heap’s space. **Heap exhaustion** = you keep allocating and never free, so allocation fails. In Node the first shows as `RangeError: Maximum call stack size exceeded` and the second as `JavaScript heap out of memory`.

Example 3 [TCS NQT · pattern]

Where does each of these live? (i) a `const` array created with `new Array(1000)` inside a function, (ii) the variable that holds the reference to it, (iii) the function’s compiled code.

SOLUTION

In a language with a heap and a stack — which is every language you will be asked about:

- (i) The 1000-element array itself is an object created at run time. Objects go on the **heap**.
- (ii) The variable is a local of the function, so the **reference** (a pointer-sized value) lives in that function’s **stack frame**.
- (iii) The instructions are fixed at compile time: **text segment**.

Answer: heap / stack / text

This is why “pass by reference vs pass by value” gets confusing: JS always copies the **value in the stack slot**, and for an object that value is a reference to the heap.

Part 3 — Making processes: fork, exec, wait

This is one place where the language policy of this series bends, on purpose. `fork()` is a C idea and interviewers ask it in C. Learn the shape; the Node version comes right after.

The classic fork shape — what the interviewer expects to see

C

```

pid_t pid = fork();           /* ONE call, TWO returns */

if (pid < 0) {
    /* fork failed */
} else if (pid == 0) {
    /* CHILD: fork() returned 0 here */
    execlp("ls", "ls", "-l", NULL); /* replaces this process image */
} else {
    /* PARENT: fork() returned the child's PID here */
    wait(NULL);                 /* sleep until the child exits; reaps the zombie */
}

```

Four facts, and each one is a question on its own:

1. **fork() returns twice.** Once in the parent (returning the child's PID, a positive number) and once in the child (returning 0). That is how each one knows who it is.
2. **The child is a copy** of the parent: same code, same data, same open file descriptors, different PID. Modern systems use **copy-on-write** — the pages are shared read-only until one side writes, and only then is a real copy made.
3. **exec() does not create a process.** It **replaces** the current process image with a new program. Same PID, brand new text/data/heap/stack. On success exec never returns, because there is no longer any code to return to.
4. **wait()** blocks the parent until a child exits and collects the child's exit status.

Zombie and orphan — the two words you must get right

	Zombie	Orphan
Who died	the child died	the parent died
What is wrong	parent never called <code>wait()</code> , so the exit status is still sitting in the process table	child is still running but has no parent to report to
What it costs	one process-table entry (a PID) — leaks if repeated	nothing
Who fixes it	the parent, by calling <code>wait()</code> ; if the parent dies the zombie is re-parented to <code>init</code> which reaps it	<code>init</code> (PID 1) adopts it

TRAP

A zombie is **not** consuming CPU or memory. It is a dead process whose **table entry** has not been removed. The damage is PID exhaustion, not slowness. Saying “a zombie eats CPU” is the classic wrong answer.

Example 4 [Shopee · pattern]

How many **new** processes does this create, and how many lines does it print?

```

int main() {
    fork();
    fork();
    fork();
    printf("hi\n");
}

```

SOLUTION

Track the count after each `fork()`. Every existing process splits into two.

After	Processes alive	Reason
start	1	the original
1st fork()	2	1×2

2nd fork()	4	2×2 — the child forks too
3rd fork()	8	4×2

All 8 reach the printf.

Answer: $2^3 = 8$ processes total, so 7 new children, and 8 lines printed

General rule: n unconditional fork() calls in a row give 2^n processes, i.e. $2^n - 1$ new ones.

Example 5 [Amazon · pattern]

Same question, but now the calls are:

```
fork();
if (fork() == 0) { fork(); }
printf("hi\n");
```

How many lines are printed?

SOLUTION

Do it in layers. Do not guess.

After line 1 (fork()): 2 processes. Call them A and B.

Line 2 (if (fork() == 0)): each of A and B forks, giving 4 processes:

- A (parent side, fork returned a $PID \neq 0$) — condition false
- A's child (fork returned 0) — condition **true**
- B (parent side) — condition false
- B's child — condition **true**

Line 3: only the two processes with a true condition run the inner fork(). Each becomes two. So those 2 become 4, and the 2 false-side processes stay as they are.

Total = $2 + 4 = 6$.

Answer: 6 lines

SHORTCUT

Method that never fails: draw a tree. Each fork() node has two children — the “parent return” branch and the “child return” branch. Count the **leaves** that reach the print.

The Node equivalent

Node cannot fork() in the Unix sense. It gives you two different tools, and the difference between them is the process-vs-thread difference:

Threads share memory. Processes do not. Run with: node s2.js

JS

```
const { Worker, isMainThread, workerData } = require('worker_threads');
const { fork } = require('child_process');

if (isMainThread && !process.env.CHILD) {
  // 1) two THREADS, one shared buffer
  const shared = new Int32Array(new SharedArrayBuffer(4));
  shared[0] = 100;
  const w = new Worker(__filename, { workerData: shared });
  w.on('exit', () => {
    console.log('THREAD : parent sees shared[0] =', shared[0]);
    // 2) a child PROCESS gets a COPY, not the same memory
    const child = fork(__filename, [], { env: { ...process.env, CHILD: '1' } });
  });
}
```

```

let myCounter = 100;
child.on('message', (m) => {
  console.log('PROCESS : child set its counter to', m.counter);
  console.log('PROCESS : parent still sees myCounter =', myCounter);
  child.kill();
});
});
} else if (!isMainThread) {
  workerData[0] = 999;          // writes the PARENT's memory
} else {
  let myCounter = 100;
  myCounter = 999;              // writes only THIS process's copy
  process.send({ counter: myCounter });
}

```

Actual output

TEXT

```

THREAD : parent sees shared[0] = 999
PROCESS : child set its counter to 999
PROCESS : parent still sees myCounter = 100

```

That output is the whole lesson. The thread changed the parent's number. The process could not, and had to **send a message** instead.

Part 4 — Threads

A **thread** is one flow of execution inside a process. Every process starts with one thread. A multithreaded process has several, all inside the same address space.

The table the interviewer wants

	Process	Thread
Address space	its own	shared with its siblings
Code / data / heap	private	shared
Stack and registers	private	private — each thread has its own
Creation cost	high (new page tables, new PCB)	low
Context switch cost	high — must switch page tables, flush the TLB	low — same page tables
Communication	IPC: pipes, sockets, shared-memory segments, message queues	just read the same variable
If one crashes	the others survive	the whole process usually dies
Protection	OS isolates them	none between threads — they can corrupt each other

SHORTCUT

Compress the whole table into one sentence you can say in five seconds: **“Threads share the address space; processes do not. So threads are cheap to create and cheap to switch between, but one bad thread can corrupt the others; processes are expensive and isolated.”**

TRAP

“Threads share the stack” — wrong, and it is the answer that ends the round. Each thread gets its **own** stack, because each thread is in the middle of its own chain of function calls. What they share is text, data and **heap**.

User threads vs kernel threads

- **Kernel-level threads:** the OS knows about each thread and schedules each one. Real parallelism on multiple cores. Slower to create (a system call).
- **User-level threads:** a library in user space multiplexes many threads onto fewer kernel threads. Very fast to create and switch (no system call). **But** if one user thread makes a blocking system call, the kernel blocks the whole kernel thread, and every user thread on it stops.

The three mapping models:

Model	Meaning	Main weakness
Many-to-one	many user threads on 1 kernel thread	one blocking call freezes all; no multi-core use
One-to-one	each user thread has its own kernel thread	creating threads is expensive, so the OS caps the count
Many-to-many	m user threads on n kernel threads, $n \leq m$	complex to implement

Example 6 [Grab · pattern]

A service uses a many-to-one threading library. One of its 50 threads calls a synchronous file read that takes 200 ms. How long are the other 49 threads stalled, and why?

SOLUTION

1. In many-to-one, all 50 user threads are multiplexed onto **one** kernel thread.
2. A synchronous read is a **blocking system call**. The kernel does not know about user threads; it only sees the one kernel thread that made the call.
3. So the kernel moves that kernel thread to the WAITING state.
4. The user-level scheduler lives **inside** that kernel thread. With the kernel thread blocked, nothing in user space runs — not even the thread scheduler.

Answer: All 49 stall for the full 200 ms.

The fix in an interview answer: use one-to-one (or many-to-many) mapping, or use non-blocking / asynchronous I/O so the call never blocks the kernel thread. That second option is exactly what Node does.

Why Node is “single-threaded” and what that really means

Your JavaScript runs on **one** thread. I/O does not — libuv keeps a pool of worker threads (4 by default) for file system work, and the kernel handles sockets asynchronously.

Consequence: a long synchronous loop blocks **everything**.

One thread means one blocker stops the world. Run with: `node s3.js`

JS

```
const t0 = Date.now();
const stamp = (msg) => console.log(`t=${String(Date.now() - t0).padStart(4)} ms
${msg}`);

setTimeout(() => stamp('timer that asked for 0 ms finally ran'), 0);
stamp('main line starts a 600 ms busy loop');
const end = Date.now() + 600;
while (Date.now() < end) { /* burn CPU - no yield */ }
stamp('busy loop finished');
```

Actual output

TEXT

```

t= 0 ms  main line starts a 600 ms busy loop
t= 603 ms  busy loop finished
t= 603 ms  timer that asked for 0 ms finally ran

```

The timer asked for 0 ms and waited 603 ms. Nothing was broken; the one thread was busy. This is cooperative scheduling: Node cannot preempt your loop the way the OS preempts a process.

Amdahl's law — the answer to “just add more cores”

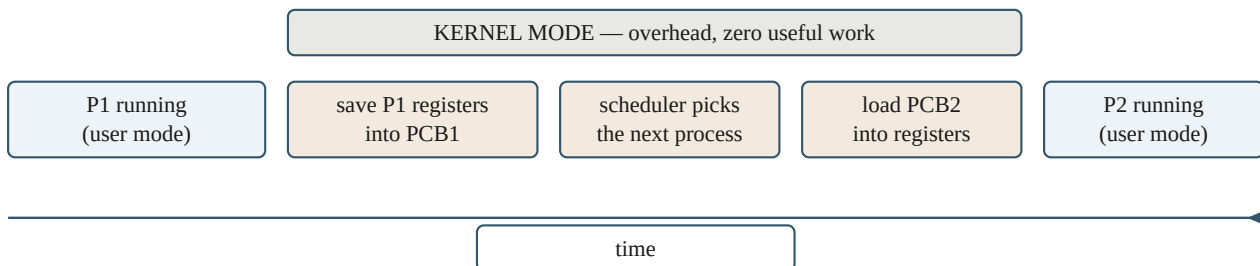
Suppose 75% of a job can be split across cores and 25% cannot.

Cores n	Speedup	Working
1	1.00	$1/(0.25 + 0.75/1)$
2	1.60	$1/(0.25 + 0.375) = 1/0.625$
4	2.29	$1/(0.25 + 0.1875) = 1/0.4375$
8	2.91	$1/(0.25 + 0.09375)$
16	3.37	$1/(0.25 + 0.046875)$
1000	3.99	$1/(0.25 + 0.00075)$

A thousand cores buys you less than $4\times$. The ceiling is $\frac{1}{1-f} = \frac{1}{0.25} = 4$.

Answer: If 25% is serial, you can never beat $4\times$, no matter how many cores you buy.

Part 5 — The context switch



A context switch. Everything between the two user-mode boxes is pure overhead: the CPU is busy but no user work is done.

What gets saved and restored: program counter, all general registers, the stack pointer, the processor status word, and memory-management state (the page-table base register).

What it costs: typically 1 to 10 microseconds of direct cost. The **indirect** cost is worse: the new process finds the CPU caches and the TLB full of the old process's data, so it runs slowly until they refill. This is called **cache pollution**.

Example 7 [Agoda · pattern]

A context switch costs 5 microseconds. Compare the CPU cost at 1,000 switches per second and at 100,000 switches per second.

SOLUTION

- At 1,000 per second: $5 \mu\text{s} \times 1000 = 5000 \mu\text{s} = 5 \text{ ms}$ of every 1000 ms. $\frac{5}{1000} = 0.5\%$ of the CPU.
- At 100,000 per second: $5 \mu\text{s} \times 100000 = 500000 \mu\text{s} = 0.5 \text{ s}$ of every 1 s. = 50% of the CPU.

Answer: 0.5% vs 50%. Switching $100\times$ more often costs half the machine.

That is the whole argument for not making the Round Robin quantum tiny.

TRAP

A **context switch** (between processes) is not the same as a **mode switch** (user mode to kernel mode and back, which happens on every system call). A mode switch is much cheaper and does not change which process is running. Every context switch involves mode switches; not every mode switch is a context switch.

Part 6 — Who schedules what

Scheduler	Decides	Runs	Controls
Long-term (job)	which jobs enter the ready queue at all	rarely (seconds/minutes)	the degree of multiprogramming
Medium-term	which processes to swap out to disk	sometimes	memory pressure
Short-term (CPU)	which ready process gets the CPU next	very often (milliseconds)	responsiveness

The **dispatcher** is not a scheduler. The scheduler **chooses**; the dispatcher **does the handover** — switch context, switch to user mode, jump to the right instruction. The time it takes is **dispatch latency**.

Preemptive vs non-preemptive

- **Non-preemptive**: once a process has the CPU it keeps it until it finishes or blocks. Simple. A long job can hold everyone up.
- **Preemptive**: the OS can take the CPU away (timer interrupt, or a higher-priority process arrives). Responsive. Costs more switches, and opens the door to race conditions on shared kernel data.

Scheduling decisions happen at four moments. Two are unavoidable, two are optional:

Moment	Transition	Preemptive only?
Process blocks on I/O	Running → Waiting	no
Process terminates	Running → Terminated	no
Quantum expires / higher priority arrives	Running → Ready	yes
I/O completes for some other process	Waiting → Ready	yes

SHORTCUT

If **only** the first two cause a scheduling decision, the scheme is non-preemptive.

Part 7 — The algorithms, with the arithmetic done in full

Everything from here is the part you will actually be asked to **compute**. One worked set runs through all of them so you can see the numbers move.

THE MASTER SET USED BELOW

Process	P1	P2	P3	P4
Arrival time	0	1	2	3
Burst time	8	4	9	5

Total burst = $8 + 4 + 9 + 5 = 26$. Nobody is idle, so **every** algorithm finishes at $t = 26$. Only the order inside changes.

7.1 FCFS — First Come First Served

Rule: run in order of arrival. Non-preemptive. One queue, no thinking.

FCFS Gantt chart										TEXT
	P1		P2		P3		P4			
0		8	12		21		26			

Now fill the table. Take one row at a time and show the subtraction.

P	AT	BT	CT	TAT = CT - AT	WT = TAT - BT
P1	0	8	8	$8 - 0 = 8$	$8 - 8 = 0$
P2	1	4	12	$12 - 1 = 11$	$11 - 4 = 7$
P3	2	9	21	$21 - 2 = 19$	$19 - 9 = 10$
P4	3	5	26	$26 - 3 = 23$	$23 - 5 = 18$

Averages: $TAT = \frac{8+11+19+23}{4} = \frac{61}{4} = 15.25$ and $WT = \frac{0+7+10+18}{4} = \frac{35}{4} = 8.75$.

Response time equals waiting time here, because FCFS never preempts: the first time a process touches the CPU is the only time it starts.

Answer: FCFS: avg TAT = 15.25, avg WT = 8.75, avg RT = 8.75

The convoy effect

One long job first — AT/BT: P1(0,20), P2(1,3), P3(2,3)						TEXT
	P1		P2		P3	
0		20	23	26		

$WT = \frac{0+19+21}{3} = \frac{40}{3} = 13.33$. Two three-unit jobs waited about 20 units each behind one 20-unit job. That is the **convoy effect**: short jobs stuck behind a long one, like cars behind a truck on a single-lane road.

TRAP

FCFS is the only algorithm with **no starvation at all** — everyone eventually reaches the front of one queue. Do not confuse “bad average waiting time” with “starvation”. FCFS is bad but fair; Priority is good but can starve.

7.2 SJF — Shortest Job First (non-preemptive)

Rule: when the CPU is free, among the processes that **have arrived**, pick the smallest burst. Once started, a process runs to completion.

Walk the clock:

- $t = 0$: only P1 has arrived. Run P1 for its full 8. CPU busy until $t = 8$.
- $t = 8$: arrived and unfinished are P2 (4), P3 (9), P4 (5). Smallest is **P2** (4). Runs to $t = 12$.
- $t = 12$: P3 (9) and P4 (5) remain. Smallest is **P4**. Runs to $t = 17$.
- $t = 17$: only **P3**. Runs to $t = 26$.

SJF (non-preemptive) Gantt chart										TEXT
	P1		P2		P4		P3			
0		8	12	17		26				

P	AT	BT	CT	TAT	WT
P1	0	8	8	8	0
P2	1	4	12	11	7
P3	2	9	26	24	15
P4	3	5	17	14	9

avg TAT = $\frac{57}{4} = 14.25$, avg WT = $\frac{31}{4} = 7.75$.

Answer: SJF: avg TAT = 14.25, avg WT = 7.75

Better than FCFS (8.75 → 7.75) but not by much, because P1's 8 units still blocked everyone at the start.

TRAP

“SJF is optimal” is true **only** for non-preemptive scheduling of a fixed set of jobs available at the same time, and only for **average waiting time**. In this example SJF gives 7.75 and preemptive SRTF gives 6.50 — SRTF is the optimal one when arrivals are staggered. Say: **“SJF gives the minimum average waiting time among non-preemptive schemes; SRTF is its preemptive version and does better when jobs arrive at different times.”**

The real problem with SJF: you do not know the burst

The OS cannot see the future. It **predicts** the next burst from the previous ones with an exponential average:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$$

With $\alpha = 0.5$ and a starting guess $\tau_0 = 10$:

Burst no.	Actual t_n	New guess τ_{n+1}	Working
1	6	8.000	$0.5(6) + 0.5(10) = 3 + 5$
2	4	6.000	$0.5(4) + 0.5(8) = 2 + 4$
3	6	6.000	$0.5(6) + 0.5(6) = 3 + 3$
4	13	9.500	$0.5(13) + 0.5(6) = 6.5 + 3$
5	13	11.250	$0.5(13) + 0.5(9.5)$
6	13	12.125	$0.5(13) + 0.5(11.25)$

Notice row 4: the guess jumps only halfway towards the new reality, then creeps closer. That is the point of α .

- $\alpha = 0$: $\tau_{n+1} = \tau_n$. History never changes the guess.
- $\alpha = 1$: $\tau_{n+1} = t_n$. Only the most recent burst matters.

7.3 SRTF — Shortest Remaining Time First (preemptive SJF)

Rule: at **every** arrival, compare the new arrival's burst with the **remaining** time of the running process. Smaller remaining wins.

Walk the clock. This is where students lose marks, so every tick is shown.

1. $t = 0$: only P1 (rem 8). Run P1.
2. $t = 1$: P2 arrives with 4. P1 has $8 - 1 = 7$ left. $4 < 7$, so **preempt**. Run P2.
3. $t = 2$: P3 arrives with 9. P2 has $4 - 1 = 3$ left. $9 > 3$, no preemption. P2 continues.
4. $t = 3$: P4 arrives with 5. P2 has 2 left. $5 > 2$, no preemption. P2 continues.
5. $t = 5$: P2 finishes. Remaining now: P1 = 7, P3 = 9, P4 = 5. Smallest is **P4** (5).
6. $t = 10$: P4 finishes. Remaining: P1 = 7, P3 = 9. Smallest is **P1**.
7. $t = 17$: P1 finishes. Only **P3** left; it runs to 26.

SRTF Gantt chart

TEXT

P1	P2	P4	P1	P3
0	1	5	10	17
				26

P	AT	BT	CT	TAT	WT	RT
P1	0	8	17	17	9	0
P2	1	4	5	4	0	0
P3	2	9	26	24	15	15
P4	3	5	10	7	2	2

Check P1 by hand: it ran 0–1 and 10–17, which is $1 + 7 = 8$ units. Correct. $TAT = 17 - 0 = 17$, $WT = 17 - 8 = 9$. RT is 0 because it started at $t = 0$.

avg $TAT = \frac{52}{4} = 13.00$, avg $WT = \frac{26}{4} = 6.50$, avg $RT = \frac{17}{4} = 4.25$.

Answer: SRTF: avg TAT = 13.00, avg WT = 6.50 — the best of all five here

TRAP

Response time is **not** waiting time once preemption exists. P1 above has $RT = 0$ and $WT = 9$. If a question says “response time” and you hand in waiting time, you lose the mark.

7.4 Priority scheduling

Rule: smallest priority number = highest priority (this is the usual convention; **state your convention in the interview**, because some books reverse it).

New data set — AT/BT/priority: P1(0, 4, 3), P2(1, 3, 1), P3(2, 5, 4), P4(3, 2, 2).

Non-preemptive:

- $t = 0$: only P1. Run it to completion, $t = 4$.
- $t = 4$: P2(1), P3(4), P4(2) waiting. Best is P2. Run to $t = 7$.
- $t = 7$: P3(4), P4(2). Best is P4. Run to $t = 9$.
- $t = 9$: P3 runs to $t = 14$.

Priority, non-preemptive

TEXT

P1	P2	P4	P3
0	4	7	9
			14

P	prio	BT	CT	TAT	WT
P1	3	4	4	4	0
P2	1	3	7	6	3
P3	4	5	14	12	7
P4	2	2	9	6	4

avg $TAT = \frac{28}{4} = 7.00$, avg $WT = \frac{14}{4} = 3.50$.

Preemptive, same data:

- $t = 0$: P1 (prio 3) runs.
- $t = 1$: P2 (prio 1) arrives. $1 < 3$: preempt. P1 has 3 left.
- $t = 2$: P3 (prio 4) arrives. $4 > 1$: no.
- $t = 3$: P4 (prio 2) arrives. $2 > 1$: no. P2 keeps going.
- $t = 4$: P2 done. Waiting: P1(3, rem 3), P3(4, rem 5), P4(2, rem 2). Best is P4.

6. $t = 6$: P4 done. P1(3) beats P3(4). P1 runs its remaining 3 to $t = 9$.
 7. $t = 9$: P3 runs to $t = 14$.

Priority, preemptive						TEXT
P1	P2	P4	P1	P3		
0	1	4	6	9	14	

P	prio	BT	CT	TAT	WT	RT
P1	3	4	9	9	5	0
P2	1	3	4	3	0	0
P3	4	5	14	12	7	7
P4	2	2	6	3	1	1

avg TAT = $\frac{27}{4} = 6.75$, avg WT = $\frac{13}{4} = 3.25$, avg RT = $\frac{8}{4} = 2.00$.

Answer: Preemptive priority beats non-preemptive here: WT 3.25 vs 3.50, RT 2.00 vs 3.50

Starvation and aging

If high-priority work keeps arriving, a low-priority process may **never** run. That is **starvation** (also called indefinite blocking).

The fix is **aging**: increase a process's priority by 1 for every fixed period it spends waiting. A priority-10 process that waits long enough becomes priority 0 and must run.

Example 8 [DBS · pattern]

A batch job has priority 15 (lowest). The scheduler ages waiting processes by one level every 30 seconds. Highest priority is 0. What is the longest the job can wait before it is **guaranteed** to be the highest-priority process in the queue?

SOLUTION

1. It must climb from 15 to 0. That is 15 steps.
2. Each step takes 30 seconds.
3. $15 \times 30 = 450$ seconds = 7.5 minutes.

Answer: 7.5 minutes to reach priority 0

Careful reading: reaching priority 0 makes it **tied** for highest, not automatically first. It is guaranteed to run only if ties are broken by waiting time (FCFS within a level), which is what real agers do. Say that out loud — it is the follow-up question.

7.5 Round Robin

Rule: FCFS with a time limit. Each process gets at most one **quantum** q . If it is not finished, it goes to the **back** of the ready queue.

The one rule people get wrong: **when a process's quantum expires at the same instant another process arrives, the arriving process joins the queue FIRST**, then the preempted one. Say your convention; most textbooks and most papers use this one.

Back to the master set, $q = 2$. Here is the queue after **every** slice. Copy this habit — write the queue down, do not keep it in your head.

Time	Runs	Its rem.	Arrives during the slice	Ready queue after
0–2	P1	6	P2 at 1, P3 at 2	P2, P3, P1
2–4	P2	2	P4 at 3	P3, P1, P4, P2

4–6	P3	7	—	P1, P4, P2, P3
6–8	P1	4	—	P4, P2, P3, P1
8–10	P4	3	—	P2, P3, P1, P4
10–12	P2	0	—	P3, P1, P4 (P2 done, CT 12)
12–14	P3	5	—	P1, P4, P3
14–16	P1	2	—	P4, P3, P1
16–18	P4	1	—	P3, P1, P4
18–20	P3	3	—	P1, P4, P3
20–22	P1	0	—	P4, P3 (P1 done, CT 22)
22–23	P4	0	—	P3 (P4 done, CT 23; only 1 unit left)
23–26	P3	0	—	empty (P3 done, CT 26)

Two things to notice. At $t = 2$, P3 arrives at exactly the moment P1's quantum ends, and our convention puts the **arrival** into the queue before the **preempted** process — that is why the queue reads “P2, P3, P1” and not “P2, P1, P3”. And at $t = 22$, P4 has only 1 unit left, so its slice is shorter than the quantum. A process never runs longer than it needs.

Collapsed into a chart:

Round Robin, $q = 2$														TEXT
P1	P2	P3	P1	P4	P2	P3	P1	P4	P3	P1	P4	P3		
0	2	4	6	8	10	12	14	16	18	20	22	23	26	

P	AT	BT	CT	TAT	WT	RT
P1	0	8	22	22	14	0
P2	1	4	12	11	7	1
P3	2	9	26	24	15	2
P4	3	5	23	20	15	5

avg TAT = $\frac{77}{4} = 19.25$, avg WT = $\frac{51}{4} = 12.75$, avg RT = $\frac{8}{4} = 2.00$.

Answer: RR ($q = 2$): worst average waiting time (12.75) but the best average response time (2.00)

That trade is the entire point of Round Robin. Nobody finishes early; everybody starts early.

Choosing the quantum

Same data, three quantum values:

Quantum	avg TAT	avg WT	avg RT	Comment
$q = 2$	19.25	12.75	2.00	snappiest start, most switching
$q = 3$	20.00	13.50	3.00	middle
$q = 4$	18.25	11.75	4.50	fewer switches, slower first response
$q \geq 9$	15.25	8.75	8.75	identical to FCFS

SHORTCUT

Two limits to memorise: $q \rightarrow \infty$ turns Round Robin into FCFS. (Nobody is ever preempted.) $q \rightarrow 0$ turns it into “processor sharing” — in theory everybody progresses at $\frac{1}{n}$ speed; in practice the machine dies of context-switch overhead.

Here is that overhead, computed. Four processes, 10 units of burst each (40 units of useful work), context switch cost $s = 1$ unit:

q	slices	switches	total time	efficiency
1	40	39	79	50.6%
2	20	19	59	67.8%
5	8	7	47	85.1%
10	4	3	43	93.0%

Working for $q = 2$: each process needs $\lceil \frac{10}{2} \rceil = 5$ slices, so $4 \times 5 = 20$ slices and 19 switches between them. Useful work 40, overhead $19 \times 1 = 19$, total 59. Efficiency = $\frac{40}{59} = 67.8\%$.

Answer: Rule of thumb: pick q so that about 80% of bursts finish inside one quantum. Real systems use 10–100 ms.

7.6 HRRN — Highest Response Ratio Next

Non-preemptive. At each decision point compute, for every waiting process,

$$RR = \frac{\text{waiting so far} + \text{burst}}{\text{burst}}$$

and run the largest. Short jobs get a big ratio immediately (good, like SJF); long jobs get a big ratio eventually (good, no starvation).

New data — AT/BT: P1(0, 6), P2(2, 2), P3(3, 1), P4(5, 4).

- $t = 0$: only P1. Run 0–6.
- $t = 6$: P2 waited $6 - 2 = 4$, P3 waited $6 - 3 = 3$, P4 waited $6 - 5 = 1$.
 - P2: $\frac{4+2}{2} = 3.000$
 - P3: $\frac{3+1}{1} = 4.000 \leftarrow$ largest
 - P4: $\frac{1+4}{4} = 1.250$

Run P3, 6–7.

- $t = 7$: P2: $\frac{5+2}{2} = 3.500$; P4: $\frac{2+4}{4} = 1.500$. Run P2, 7–9.
- $t = 9$: only P4. Runs 9–13.

HRRN Gantt chart

TEXT

```
| P1 | P3 | P2 | P4 |
0      6   7   9   13
```

P	AT	BT	CT	TAT	WT
P1	0	6	6	6	0
P2	2	2	9	7	5
P3	3	1	7	4	3
P4	5	4	13	8	4

avg TAT = $\frac{25}{4} = 6.25$, avg WT = $\frac{12}{4} = 3.00$.

Answer: HRRN: avg WT = 3.00, and no process can starve

7.7 Multilevel Queue and Multilevel Feedback Queue

Multilevel Queue (MLQ). Split processes permanently into classes, each with its own queue and its own algorithm. For example: system processes (RR, $q = 8$), interactive (RR, $q = 16$), batch (FCFS). Scheduling **between** queues is usually fixed priority — nothing in the batch queue runs while anything interactive is ready. A process **cannot change queue**. Risk: starvation of the lowest queue.

Multilevel Feedback Queue (MLFQ). Same idea, but a process **can move between queues**. That one word — feedback — is the whole difference, and it is the answer to “what is the difference between MLQ and MLFQ?”.

The usual rules:

1. A new process enters the **highest** queue.
2. If it uses its whole quantum, it is demoted one level (it looks CPU-bound).
3. If it blocks for I/O before the quantum ends, it stays (it looks interactive).
4. Every so often, everything is promoted back to the top (that is aging — it prevents starvation).

Example 9 [Microsoft · pattern]

An MLFQ has three levels: Q0 (RR, $q = 4$), Q1 (RR, $q = 8$), Q2 (FCFS). A process needs 20 units of CPU and never does I/O. Trace which queues it visits, and how much CPU it gets in each.

SOLUTION

1. **Enters Q0.** Runs 4 units (its whole quantum). Remaining $20 - 4 = 16$. Demoted to Q1.
2. **In Q1.** Runs 8 units. Remaining $16 - 8 = 8$. Demoted to Q2.
3. **In Q2.** FCFS, no quantum. It runs its last 8 units to completion whenever it reaches the front.

Answer: Q0: 4 units, Q1: 8 units, Q2: 8 units. Total 20.

The follow-up: “**why is this good?**” Because a CPU-bound job sinks to the bottom after only $4 + 8 = 12$ units of measurement, and from then on interactive jobs — which block quickly and stay in Q0 — get the CPU almost immediately. MLFQ **learns** which jobs are interactive without being told.

Second follow-up: “**can that job starve?**” Yes, in Q2, if interactive work keeps arriving. The standard fix is periodic promotion of every process back to Q0.

7.8 The comparison table — memorise this shape

All five algorithms on the same master set (P1 0/8, P2 1/4, P3 2/9, P4 3/5):

Algorithm	avg TAT	avg WT	avg RT	Character
FCFS	15.25	8.75	8.75	simple, convoy effect
SJF (non-pre.)	14.25	7.75	7.75	good average, needs burst estimate
SRTF (pre.)	13.00	6.50	4.25	best averages, can starve long jobs
RR $q = 2$	19.25	12.75	2.00	fairest, best response, most overhead
RR $q = 4$	18.25	11.75	4.50	less overhead, slower response

SHORTCUT

The sentence that answers most comparison questions: “**SRTF minimises average waiting time. Round Robin minimises response time. FCFS minimises implementation effort. You pick based on whether the machine serves batch jobs or humans.**”

A scheduler you can run

Round-Robin simulator. Run with: `node s1.js`

JS

```
function roundRobin(procs, q) {
  const p = procs.map(x => ({ ...x, rem: x.bt, ct: 0, first: -1 }));
  const byArrival = [...p].sort((a, b) => a.at - b.at);
  const gantt = [], queue = [];
  let t = 0, i = 0, doneCount = 0;
  while (i < byArrival.length && byArrival[i].at <= t) queue.push(byArrival[i++]);
  while (doneCount < p.length) {
    if (queue.length === 0) { // CPU idle: jump to next arrival
      t = byArrival[i].at;
      while (i < byArrival.length && byArrival[i].at <= t) queue.push(byArrival[i++]);
    }
    continue;
  }
  const cur = queue.shift();
  if (cur.first < 0) cur.first = t; // first touch -> response time
  const run = Math.min(q, cur.rem);
  gantt.push([cur.id, t, t + run]);
  t += run; cur.rem -= run;
  // arrivals during this slice enter the queue BEFORE the preempted process
  while (i < byArrival.length && byArrival[i].at <= t) queue.push(byArrival[i++]);
  if (cur.rem > 0) queue.push(cur); else { cur.ct = t; doneCount++; }
}
return { gantt, p };
}

const procs = [
  { id: 'P1', at: 0, bt: 8 }, { id: 'P2', at: 1, bt: 4 },
  { id: 'P3', at: 2, bt: 9 }, { id: 'P4', at: 3, bt: 5 },
];
const { gantt, p } = roundRobin(procs, 2);
console.log('Gantt:', gantt.map(([id, s, e]) => `${id}[${s}-${e}]`).join(' '));
let sumT = 0, sumW = 0, sumR = 0;
for (const x of p) {
  const tat = x.ct - x.at, wt = tat - x.bt, rt = x.first - x.at;
  sumT += tat; sumW += wt; sumR += rt;
  console.log(`${x.id} CT=${x.ct} TAT=${tat} WT=${wt} RT=${rt}`);
}
const n = p.length;
console.log(`avg TAT=${(sumT/n).toFixed(2)} avg WT=${(sumW/n).toFixed(2)} avg RT=${(sumR/n).toFixed(2)}`);
```

Actual output — matches the table above

TEXT

```
P1 CT=22 TAT=22 WT=14 RT=0
P2 CT=12 TAT=11 WT=7 RT=1
P3 CT=26 TAT=24 WT=15 RT=2
P4 CT=23 TAT=20 WT=15 RT=5
avg TAT=19.25 avg WT=12.75 avg RT=2.00
```

TRAP

Line 3 of that program: `[...p].sort((a, b) => a.at - b.at)`. Two JS traps in one line. **(1)** `sort()` with no comparator sorts **lexicographically**, so arrival times `[10, 9, 1]` would come out as `[1, 10, 9]`. Numbers always need `(a, b) => a - b`. **(2)** `.sort()` sorts **in place**. Without the `[...p]` copy you would silently reorder the array you are also reporting from.

Part 8 — I/O bound vs CPU bound

- **CPU-bound** process: long CPU bursts, few I/O requests. A video encoder.
- **I/O-bound** process: short CPU bursts, many I/O requests. A text editor, a web server.

A good mix matters. If all processes are CPU-bound the I/O devices sit idle. If all are I/O-bound the CPU sits idle. The **long-term scheduler** exists to keep the mix balanced.

The utilisation formula makes this concrete. If each process is blocked on I/O 20% of the time ($p = 0.2$):

Processes in memory n	CPU utilisation	Working
1	80.00%	$1 - 0.2^1$
2	96.00%	$1 - 0.04$
3	99.20%	$1 - 0.008$
4	99.84%	$1 - 0.0016$

Diminishing returns arrive fast. Going from 3 to 4 processes buys 0.64%.

TRAP

$1 - p^n$ assumes the processes block **independently**. If they all wait on the same slow disk, they are not independent and the real number is far worse. If an interviewer pushes, say that: it shows you know it is a model, not a law.

Part 9 — Inter-process communication (IPC)

Threads talk by sharing a variable. Processes cannot — their address spaces are separate. So the OS provides channels. Know these five and one sentence each.

Mechanism	Shape	Good at	Catch
Pipe (anonymous)	byte stream, one direction	parent-to-child, e.g. a shell	only between related processes
Named pipe (FIFO)	byte stream with a filename	unrelated processes on one machine	still one machine, still a stream
Message queue	discrete messages with types	structured, no ordering worries	kernel copies every message — slower
Shared memory	a memory region mapped into both	the fastest IPC — no copying	you must add your own locking
Socket	byte stream or datagrams	works across machines	slowest; needs serialising

TRAP

“Shared memory is the fastest IPC” is the expected answer — but the follow-up is “**why?**” Because every other mechanism copies the data through the kernel (user → kernel → user). Shared memory copies nothing: both processes map the **same** physical pages, so a write by one is instantly visible to the other. The price is that the kernel gives you no synchronisation at all; you must add a semaphore or mutex yourself. That is Chapter 2.

Two models, one sentence each

- **Message passing**: `send(destination, message)` and `receive(source, message)`. Easy to reason about, works over a network, the kernel does the copying.
- **Shared memory**: both processes agree on a region; after setup the kernel is not involved at all.

Blocking vs non-blocking matters here:

	Blocking (synchronous)	Non-blocking (asynchronous)
send	sender waits until the message is received	sender returns immediately
receive	receiver waits until a message arrives	receiver gets a message or nothing

A **rendezvous** is blocking send plus blocking receive: neither side moves until both are ready.

Example 10 [Sea/Shopee · pattern]

Two Node processes on the same server must exchange 500 MB of image data 100 times a second. An engineer proposes `child.send(buffer)` over the built-in IPC channel. Why is that a bad choice, and what should be used instead?

SOLUTION

1. `child.send()` is **message passing**. Node serialises the buffer, the kernel copies it from the sender's address space into kernel space, then copies it again into the receiver's.
2. That is two full copies of 500 MB, 100 times a second = 100 GB/s of pure memory copying, before any real work.
3. Shared memory copies nothing. Both processes map the same physical pages.

In Node the concrete tool is a `SharedArrayBuffer` passed to a `Worker` (threads, same process) or a memory-mapped file for separate processes.

Answer: Use shared memory; message passing costs two copies per transfer.

Follow-up you should volunteer: “**and then I need locking, because shared memory gives no mutual exclusion.**” That sentence is what separates a memorised answer from an understood one.

Part 10 — System calls, modes and interrupts

Two modes

The CPU runs in one of two privilege levels:

- **User mode:** your code. Cannot touch I/O devices, cannot change page tables, cannot disable interrupts.
- **Kernel mode (supervisor mode):** the OS. Can do everything.

A single bit in a status register says which mode you are in. Your program cannot set that bit — if it could, the whole protection scheme would be decoration.

The system call

A **system call** is the only legal door from user mode into kernel mode. The sequence:

1. Your code puts a call number and arguments in registers.
2. It executes a special instruction (`syscall` on x86-64, `svc` on ARM).
3. The CPU switches to kernel mode and jumps to a **fixed** address — the `syscall` handler. Fixed, because the user must not choose where the kernel starts running.
4. The kernel checks the call number, does the work, puts a result in a register.
5. It switches back to user mode and returns.

The five families to name: **process control** (`fork`, `exec`, `exit`, `wait`), **file management** (`open`, `read`, `write`, `close`), **device management**, **information maintenance** (`getpid`, `time`), and **communication** (`pipe`, `socket`, `shmget`).

Interrupt vs trap vs fault

	Caused by	Timing	Example
Interrupt	hardware, outside the CPU	asynchronous — any time	disk finished, key pressed, timer tick

Trap (software interrupt)	the running instruction, on purpose	synchronous	a system call, a debugger breakpoint
Fault / exception	the running instruction, by accident	synchronous	divide by zero, page fault, bad pointer

SHORTCUT

One-line separator: “An interrupt comes from outside and is a surprise. A trap comes from inside and is deliberate. A fault comes from inside and is a mistake.”

TRAP

A **page fault** is not an error. It is a normal, expected event: the page you touched is not in RAM right now, so the kernel fetches it and restarts your instruction. Calling it a crash is a common mistake, and Chapter 3 spends a whole section on it.

The **timer interrupt** deserves its own line, because it is what makes preemption possible at all. The hardware timer fires every few milliseconds, forces a trip into kernel mode, and gives the scheduler a chance to say “your quantum is over”. Without it, a process that never makes a system call could hold the CPU forever.

Part 11 — More than one CPU, and hard deadlines

Multiprocessor scheduling

With m cores, two designs:

- **Asymmetric**: one core runs all kernel code and scheduling; the others run user code only. Simple, no locking on kernel data, but that one core is a bottleneck.
- **Symmetric (SMP)**: every core schedules itself. This is what everything real uses.

SMP brings two new words:

- **Processor affinity**: keep a process on the core whose cache is already warm with its data. **Soft affinity** = the OS tries; **hard affinity** = you pin it and it cannot move.
- **Load balancing**: keep every run queue busy. **Push migration** — a periodic task moves work off an overloaded core. **Pull migration** — an idle core steals work from a busy one.

These two goals **fight each other**. Migrating a process balances load but throws away its warm cache. Naming that tension is the answer the interviewer is fishing for.

Real-time scheduling

Hard real-time: missing a deadline is a failure (an airbag controller). **Soft real-time**: missing a deadline degrades quality (a video player drops a frame).

Each periodic task i has period p_i and worst-case execution time t_i . Its CPU share is $\frac{t_i}{p_i}$, and the total utilisation is

$$U = \sum_{i=1}^n \frac{t_i}{p_i}$$

Rate-Monotonic (RM) gives the highest static priority to the **shortest period**. It is guaranteed to work if

$$U \leq n(2^{1/n} - 1)$$

n tasks	RM bound
1	1.0000
2	0.8284
3	0.7798

4	0.7568
5	0.7435
$\rightarrow \infty$	$\ln 2 = 0.6931$

Earliest-Deadline-First (EDF) is dynamic: whoever’s deadline is nearest runs. Its bound is simply $U \leq 1$ — it can use the whole CPU.

Example 11 [D. E. Shaw · pattern]

Three periodic tasks on one core: T1 (period 20 ms, needs 6 ms), T2 (period 30 ms, needs 8 ms), T3 (period 50 ms, needs 10 ms). Is this schedulable under Rate-Monotonic? Under EDF? Now T3’s work grows to 15 ms — redo both answers.

SOLUTION

Step 1 — utilisation.

$$U = \frac{6}{20} + \frac{8}{30} + \frac{10}{50} = 0.3000 + 0.2667 + 0.2000 = 0.7667$$

Step 2 — the RM bound for $n = 3$.

$$3(2^{1/3} - 1) = 3(1.2599 - 1) = 3 \times 0.2599 = 0.7798$$

Step 3 — compare. $0.7667 \leq 0.7798$ ✓ — RM is **guaranteed** to meet every deadline. Priorities: T1 highest (shortest period 20), then T2, then T3.

Step 4 — EDF. $0.7667 \leq 1$ ✓ — schedulable, with room to spare.

Now $t_3 = 15$ ms.

$$U = 0.3000 + 0.2667 + \frac{15}{50} = 0.3000 + 0.2667 + 0.3000 = 0.8667$$

- RM bound is still 0.7798. $0.8667 > 0.7798$, so the RM **guarantee fails**.
- EDF bound is 1. $0.8667 \leq 1$ ✓, so EDF still meets every deadline.

Answer: Both work at $U = 0.7667$. At $U = 0.8667$ only EDF is guaranteed.

TRAP

The RM test is **sufficient, not necessary**. Failing it does not prove the task set misses a deadline — it only means RM cannot promise. Some sets above the bound still run fine, and you would have to simulate the whole hyperperiod to know. Say “the guarantee fails”, never “it will miss a deadline”.

Why use RM at all if EDF is better on paper? Because RM priorities are **fixed**, so the runtime is trivial and cheap; EDF has to re-sort by deadline constantly, and when EDF is overloaded it degrades badly — one late task can cascade into everything missing.

Linux in one paragraph

Modern Linux uses **CFS**, the Completely Fair Scheduler. It has no fixed quantum. It tracks each task’s **virtual runtime** — CPU time used, weighted by the task’s nice value — and always runs the task with the smallest virtual runtime, keeping them in a red-black tree so “smallest” is $O(\log n)$. A lower nice number (down to -20) means a larger weight, so virtual runtime grows more slowly, so the task is picked more often. Real-time tasks (SCHED_FIFO, SCHED_RR) sit in a separate class that always beats CFS.

Answer: One line to say: “CFS approximates giving every runnable task an equal share of the CPU, by always running whichever task is furthest behind on its fair share.”

Practice

WARM-UP

build the reflex

PRACTICE · WARM-UP target: 8 min

1. Name the five process states.
2. $TAT = 14$, $BT = 6$. Find WT .
3. $AT = 3$, $CT = 19$. Find TAT .
4. Which scheduler decides the degree of multiprogramming?
5. Which is cheaper to create: a process or a thread?
6. True or false: two threads of the same process share the stack.
7. A process makes a system call. Is that a context switch?
8. In Round Robin, what happens when q is larger than every burst time?

ANSWER KEY

1. New, Ready, Running, Waiting/Blocked, Terminated.
2. $WT = TAT - BT = 14 - 6 = 8$.
3. $TAT = CT - AT = 19 - 3 = 16$.
4. The long-term (job) scheduler.
5. A thread — no new address space or page tables.
6. False. Shared: text, data, heap. Private: stack and registers.
7. No. It is a **mode** switch (user $\rightarrow$ kernel). The same process continues.
8. It behaves exactly like FCFS.

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

PRACTICE · TIER 1 target: 20 min

1. Processes A(AT 0, BT 4), B(AT 1, BT 3), C(AT 2, BT 5). Draw the FCFS Gantt chart and give average waiting time.
2. Same data under non-preemptive SJF. Average waiting time?
3. Same data under Round Robin with $q = 2$. Average turnaround time?
4. Explain a zombie process in three lines, and say how to prevent one.
5. Three `fork()` calls run one after another with no conditions. How many processes exist at the end?
6. A process in WAITING has its I/O complete. Which state does it enter, and why not RUNNING?
7. Give one advantage and one disadvantage of preemptive scheduling.

ANSWER KEY

1. FCFS order A, B, C. A: 0–4, B: 4–7, C: 7–12. $WT: A = 0; B = (7 - 1) - 3 = 3; C = (12 - 2) - 5 = 5$. Average = $\frac{8}{3} = 2.67$.
2. At $t = 0$ only A has arrived, so A runs 0–4 regardless. At $t = 4$, B (3) and C (5) are waiting; B is shorter. B: 4–7, C: 7–12. Same chart as FCFS here. Average $WT = 2.67$. (A good reminder: SJF is not always different.)
3. $q = 2$. A 0–2 (2 left). B arrives at 1, C at 2. Queue after A's slice: [B, C, A]. B 2–4 (1 left) $\rightarrow$ queue [C, A, B]. C 4–6 (3 left) $\rightarrow$ [A, B, C]. A 6–8 (done, CT 8). B 8–9 (done, CT 9). C 9–11 (1 left) $\rightarrow$ C 11–12 (done, CT 12). $TAT: A = 8, B = 9 - 1 = 8, C = 12 - 2 = 10$. Average = $\frac{26}{3} = 8.67$.
4. A zombie is a child that has terminated but whose parent has not called `wait()`, so its exit status and PID still occupy a slot in the process table. It uses no CPU and no memory — only a PID. Prevent it by having the parent call `wait()` / `waitpid()`, or by handling `SIGCHLD`.

5. $2^3 = 8$.

6. READY. The CPU may be busy with someone else; completing I/O only makes the process **eligible** to run. The dispatcher still has to choose it.

7. Advantage: a single long process cannot monopolise the CPU, so response time stays low. Disadvantage: extra context-switch overhead, and shared kernel data can be left inconsistent mid-update, which forces locking.

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

PRACTICE · TIER 2 target: 25 min

- Four jobs: J1(0, 7), J2(2, 4), J3(4, 1), J4(5, 4) as (AT, BT). Compute average waiting time under SRTF, showing every preemption point.
- The same four jobs under Round Robin $q = 2$. Which algorithm gives the better average response time, and by how much?
- A web server handles requests that each spend 5 ms on CPU and 45 ms waiting on the database. Using $1 - p^n$, how many concurrent requests are needed to keep the CPU above 95% busy?
- A team replaces 8 OS processes with 8 threads in one process. Name two things that get better and two new risks.

ANSWER KEY

1. SRTF on J1(0,7), J2(2,4), J3(4,1), J4(5,4).

- $t = 0$: only J1. Runs.
- $t = 2$: J2 arrives with 4. J1 has $7 - 2 = 5$ left. $4 < 5 \rightarrow$ preempt. J2 runs.
- $t = 4$: J3 arrives with 1. J2 has 2 left. $1 < 2 \rightarrow$ preempt. J3 runs 4–5 and finishes.
- $t = 5$: J4 arrives with 4. Remaining: J1 = 5, J2 = 2, J4 = 4. Smallest is J2. J2 runs 5–7, finishes.
- $t = 7$: J1 = 5, J4 = 4. J4 runs 7–11, finishes.
- $t = 11$: J1 runs its last 5, 11–16.

Chart: | J1 | J2 | J3 | J2 | J4 | J1 | with boundaries 0, 2, 4, 5, 7, 11, 16.

CT: J1 = 16, J2 = 7, J3 = 5, J4 = 11. TAT: 16, 5, 1, 6. WT: $16 - 7 = 9$, $5 - 4 = 1$, $1 - 1 = 0$, $6 - 4 = 2$. Average WT = $\frac{12}{4} = 3.00$. Average TAT = $\frac{28}{4} = 7.00$.

2. RR $q = 2$ on the same jobs. Chart: | J1 | J2 | J1 | J3 | J2 | J4 | J1 | J4 | J1 | with boundaries 0, 2, 4, 6, 7, 9, 11, 13, 15, 16. CT: J1 = 16, J2 = 9, J3 = 7, J4 = 15. TAT: 16, 7, 3, 10 $\rightarrow$ average = $\frac{36}{4} = 9.00$. WT: 9, 3, 2, 6 $\rightarrow$ average = $\frac{20}{4} = 5.00$. First CPU touch: J1 at 0, J2 at 2, J3 at 6, J4 at 9. RT: $0 - 0 = 0$, $2 - 2 = 0$, $6 - 4 = 2$, $9 - 5 = 4$ $\rightarrow$ average = $\frac{6}{4} = 1.50$. For SRTF, first touches are J1 at 0, J2 at 2, J3 at 4, J4 at 7, so RT = 0, 0, 0, 2 $\rightarrow \frac{2}{4} = 0.50$.

Answer: SRTF wins on response time here, 0.50 vs 1.50 — a difference of 1.00 unit. (SRTF also wins on waiting time, 3.00 vs 5.00. RR's response-time advantage shows up when bursts are long and unequal, not when several jobs are already tiny.)

3. Each request is on the CPU 5 ms out of 50 ms, so it is blocked $p = \frac{45}{50} = 0.9$ of the time. Solve $1 - 0.9^n \geq 0.95$, i.e. $0.9^n \leq 0.05$.

- $0.9^{10} = 0.3487$
- $0.9^{20} = 0.1216$
- $0.9^{28} = 0.0523$
- $0.9^{29} = 0.0471 \leq 0.05 \checkmark$

Answer: **29 concurrent requests**. (In practice you would round to 30 and check memory.)

4. Better: (i) creation and context switching get much cheaper — no page-table switch, no TLB flush; (ii) sharing data is free, no serialisation through pipes or sockets. New risks: (i) no memory isolation — one thread's bad pointer or unhandled exception can take down all 8; (ii) every shared variable is now a potential race condition, so you need locks, and locks bring deadlock and priority inversion.

PRACTICE · TIER 3 target: 30 min

1. Prove, in words an interviewer will accept, that SRTF gives the minimum possible average waiting time for a given set of arrivals and bursts. Then give one reason a real OS still does not use it.
2. A 16-core machine runs a job that is 92% parallelisable. Management asks for a $10\times$ speedup. Is it possible with any number of cores? Give the number.
3. Design a scheduler for a food-delivery dispatch service: thousands of short “assign a rider” tasks and a few long “recompute all zones” tasks. Which algorithm, which quantum, and what stops the long tasks from starving?
4. Round Robin with quantum q and n processes. Give the worst-case time a process waits before its **first** CPU slice, including a context-switch cost s . State your assumptions.

ANSWER KEY

1. Exchange argument. Suppose at some decision point the scheduler runs job X when job Y is available and Y has a strictly smaller remaining time. Swap them: run Y first, then X. Y now finishes earlier by the length of X's run, and X finishes no later than Y did before, so the **sum** of completion times does not increase — and it strictly decreases whenever the remaining times differ. Since average waiting time = $\frac{\text{sum of CT} - \text{sum of AT} - \text{sum of BT}}{n}$ and only the sum of CT can change, always picking the smallest remaining time minimises it.

Why no real OS uses it: the OS cannot know the remaining time. It can only estimate it, and a wrong estimate makes the guarantee worthless. SRTF also starves long jobs, and it requires a comparison at **every** arrival, which costs CPU.

2. $f = 0.92$, so the ceiling is $\frac{1}{1-0.92} = \frac{1}{0.08} = 12.5$.

• 10 is below 12.5, so it **is** possible.

• Solve $\frac{1}{0.08 + \frac{0.92}{n}} = 10 \rightarrow 0.08 + \frac{0.92}{n} = 0.1 \rightarrow \frac{0.92}{n} = 0.02 \rightarrow n = 46$.

• Check: $\frac{1}{0.08 + \frac{0.92}{46}} = \frac{1}{0.08 + 0.02} = \frac{1}{0.1} = 10 \checkmark$

• At 16 cores you only get $\frac{1}{0.08 + 0.0575} = 7.27$.

Answer: yes, with **46 cores**. Ask for $12\times$ and it needs 276 cores; ask for $13\times$ and no number of cores will do it.

3. Use a **multilevel feedback queue** with two or three levels.

- New tasks enter Q0 with a short quantum (say 10 ms). A rider assignment finishes inside one quantum and leaves; it never gets demoted, so the common case is nearly FCFS with almost no preemption cost.
- A zone recomputation uses its whole quantum, gets demoted to Q1 (quantum 50 ms) and then to Q2 (FCFS). It stops competing with the short tasks.
- Anti-starvation: promote every process in Q1 and Q2 back to Q0 every, say, 2 seconds; or reserve a fixed share (e.g. 10% of slices) for the bottom queue.
- Quantum choice: pick it so most assignment tasks finish in one slice. If an assignment costs 2–8 ms of CPU, $q = 10$ ms means about 95% of them never get preempted.
- Trade-off to name out loud: this optimises response time for the many, at the cost of turnaround time for the few long jobs. If the zone recomputation has a deadline, it needs a reserved share, not just aging.

4. Assumptions: all n processes are already in the ready queue, our process is last, each of the others uses a full quantum, and one context switch of cost s happens before each slice.

- $n - 1$ processes run ahead of us, each taking q of CPU plus s of switching.
- Then one more switch brings us in.
- Worst-case wait = $(n - 1)(q + s) + s$.
- Sanity check with $n = 1$: wait = s , just the switch onto the CPU. Correct.
- If you ignore switch cost, the familiar textbook form is $(n - 1)q$.

Rapid fire — one-line answers

Question	Answer
----------	--------

Program vs process?	Program = passive file on disk; process = program in execution with memory and resources.
What is stored in a PCB?	PID, state, program counter, registers, scheduling info, memory info, open files.
Five process states?	New, Ready, Running, Waiting, Terminated.
Can Waiting go straight to Running?	No — it must pass through Ready.
Process vs thread in one line?	Threads share the address space; processes do not.
What do threads NOT share?	Stack, registers, program counter, thread ID.
What do threads share?	Text, data, heap, open files, signals.
What does <code>fork()</code> return?	0 in the child, the child's PID in the parent, negative on failure.
Does <code>exec()</code> create a process?	No. It replaces the current process image; the PID stays the same.
What is a zombie?	A terminated child whose parent has not called <code>wait()</code> ; its table entry remains.
What is an orphan?	A running process whose parent died; <code>init</code> (PID 1) adopts it.
Context switch vs mode switch?	Context switch changes which process runs; mode switch only changes privilege level.
What is saved on a context switch?	PC, registers, stack pointer, status word, memory-management registers.
Which scheduler runs most often?	The short-term (CPU) scheduler — every few milliseconds.
What is the dispatcher?	The module that actually hands the CPU over; its cost is dispatch latency.
TAT formula?	$CT - AT$.
WT formula?	$TAT - BT$.
RT formula?	First time on CPU minus arrival time.
Which algorithm has the convoy effect?	FCFS.
Which minimises average waiting time?	SRTF (preemptive SJF); SJF is optimal among non-preemptive schemes.
Which minimises response time?	Round Robin with a small quantum.
What is starvation?	A process never gets the CPU because higher-priority work keeps arriving.
What is aging?	Raising a waiting process's priority over time so it cannot starve forever.
RR with a huge quantum behaves like?	FCFS.
MLQ vs MLFQ?	MLQ queues are fixed; in MLFQ a process can move between queues (feedback).
Preemptive vs non-preemptive, one line?	Preemptive can take the CPU away mid-burst; non-preemptive waits for a block or an exit.
CPU-bound vs I/O-bound?	CPU-bound has long CPU bursts; I/O-bound has short bursts and many I/O waits.
CPU utilisation with n processes?	$1 - p^n$, where p is the fraction of time each is blocked.
Amdahl's ceiling?	$\frac{1}{1-f}$ where f is the parallel fraction.
Why is Node called single-threaded?	Your JS runs on one thread; libuv and the kernel do I/O on others.

ONE-PAGE REVISION CARD

States. New → Ready → Running → (Waiting → Ready) → Terminated. Only Ready → Running exists as an entry to the CPU.

Formulas. $TAT = CT - AT$; $WT = TAT - BT$; $RT = \text{first CPU} - AT$; $\text{utilisation} = 1 - p^n$;
 $\text{speedup} = \frac{1}{(1-f) + \frac{f}{n}}$.

Process vs thread. Shared by threads: text, data, heap, files. Private to each thread: stack, registers, PC. Threads are cheap and unprotected; processes are costly and isolated.

fork/exec. fork = one call two returns (0 = child). exec = same PID, new program. Zombie = dead child, unrealed. Orphan = live child, dead parent.

Algorithms.

Name	Preempt?	Best at	Weakness
FCFS	no	simplicity	convoy effect
SJF	no	avg waiting	needs burst estimate
SRTF	yes	avg waiting (optimal)	starves long jobs
Priority	either	importance	starvation — fix with aging
Round Robin	yes	response time	overhead; worst avg TAT
HRRN	no	balance	non-preemptive only
MLFQ	yes	learns job type	many knobs to tune

Master numbers to remember the shape of (P1 0/8, P2 1/4, P3 2/9, P4 3/5): FCFS WT 8.75 · SJF 7.75 · SRTF 6.50 · RR($q=2$) 12.75 but RT only 2.00.

Three traps. (1) Waiting goes to Ready, never straight to Running. (2) Threads do NOT share the stack. (3) Response time $\neq$ waiting time once preemption exists.

OS: Synchronisation & Deadlock

Two workers, one shared thing, and every way that can go wrong.

What this round actually looks like

Chapter 1 gave the CPU to one process at a time. The moment two of them touch the same variable, a new class of bug appears — one that passes every test on your laptop and fails once a week in production. Interviewers love this topic because the answers are short, checkable, and reveal instantly whether you have actually written concurrent code.

The four questions that come up almost every time:

- “What is a race condition? Give an example.”
- “Difference between a mutex and a semaphore?”
- “What are the four necessary conditions for deadlock?”
- “Explain the Banker’s algorithm.” (then: “run it on this table”)

EVERYTHING THIS CHAPTER NEEDS, IN ONE BOX

Race condition. Two or more threads access shared data at the same time, and the final result depends on the **order** in which they happen to run. The bug is the **order dependence**, not the sharing.

Critical section. The part of the code that touches the shared data. The structure is always the same four parts:

entry section → **critical section** → exit section → remainder

Any correct solution must satisfy all three:

1. **Mutual exclusion** — at most one process inside the critical section at a time.
2. **Progress** — if nobody is inside and somebody wants in, the choice of who enters cannot be postponed forever, and a process in its remainder section must not take part in the choice.
3. **Bounded waiting** — there is a limit on how many times others can enter ahead of you after you have asked.

Semaphore. An integer S with two atomic operations:

wait(S) (also called P / down / acquire): block while $S \leq 0$, then $S = S - 1$

signal(S) (also called V / up / release): $S = S + 1$, wake one waiter

The four necessary conditions for deadlock. All four must hold at once:

1. **Mutual exclusion** — at least one resource is non-shareable.
2. **Hold and wait** — a process holds one resource while waiting for another.
3. **No preemption** — a resource can only be released voluntarily by its holder.
4. **Circular wait** — a cycle $P_1 \rightarrow P_2 \rightarrow \dots \rightarrow P_n \rightarrow P_1$ where each waits for the next.

Banker’s algorithm.

$$\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$$

$$\text{Available}[j] = \text{Total}[j] - \sum_i \text{Allocation}[i][j]$$

A state is **safe** if some ordering of all processes exists in which each can get its Need from Available plus what the earlier ones release.

Part 1 — The race condition

The smallest possible example

Two threads run `counter = counter + 1` on a shared counter that starts at 5. That single line is really three machine instructions:

One line of source, three machine steps			TEXT
LOAD	R1, counter	; read	
ADD	R1, 1	; modify	
STORE	counter, R1	; write	

If the scheduler switches threads between the LOAD and the STORE, both threads read 5, both compute 6, both store 6. Two increments, one result.

step	t1	t2	t3	t4	t5
Thread A	read 100	—	—	write 100-100	—
Thread B	—	read 100	—	—	write 100-100
balance after	100	100	100	0	0

The lost update. Both threads read 100, both decide they can afford 100, and the account pays twice.

You can reproduce it in Node — twice, two different ways

Way 1: one thread, but `await` yields control. People assume JavaScript is safe because it is single-threaded. It is not. Every `await` is a place where the scheduler can switch to another task.

A real lost update in single-threaded Node. Run: <code>node c1.js</code>	JS
<pre> let balance = 100; const sleep = (ms) => new Promise(r => setTimeout(r, ms)); async function withdraw(name, amount) { const seen = balance; // READ await sleep(10); // <-- the scheduler can switch here if (seen >= amount) { balance = seen - amount; // MODIFY + WRITE, using a STALE read console.log(`\${name}: took \${amount}, balance now \${balance}`); } else { console.log(`\${name}: refused, only \${seen} available`); } } (async () => { await Promise.all([withdraw('A', 100), withdraw('B', 100)]); console.log('final balance =', balance); })(); </pre>	

Actual output	TEXT
<pre> A: took 100, balance now 0 B: took 100, balance now 0 final balance = 0 </pre>	

The account had 100. It paid out 200. The balance says 0 and nobody notices.

Way 2: real OS threads, real lost writes. Four worker threads each increment a shared 32-bit integer 200,000 times.

Lost updates across real threads, then two fixes. Run: `node c7.js`

JS

```
const { Worker, isMainThread, workerData } = require('worker_threads');
const ITER = 200000, WORKERS = 4;

if (isMainThread) {
  const run = (mode) => new Promise(res => {
    const sab = new SharedArrayBuffer(8);
    const a = new Int32Array(sab);           // a[0] = counter, a[1] = lock flag
    let left = WORKERS;
    for (let i = 0; i < WORKERS; i++) {
      const w = new Worker(__filename, { workerData: { sab, mode } });
      w.on('exit', () => { if (--left === 0) res(a[0]); });
    }
  });
  (async () => {
    console.log('expected      =', ITER * WORKERS);
    console.log('plain a[0]++   =', await run('plain'));
    console.log('Atomsics.add    =', await run('atomic'));
    console.log('spinlock (CAS)  =', await run('lock'));
  })();
} else {
  const { sab, mode } = workerData;
  const a = new Int32Array(sab);
  for (let i = 0; i < ITER; i++) {
    if (mode === 'plain') { a[0] = a[0] + 1; }
    else if (mode === 'atomic') { Atomics.add(a, 0, 1); }
    else {
      while (Atomics.compareExchange(a, 1, 0, 1) !== 0) { /* spin */ } // test-and-set
      a[0] = a[0] + 1; // critical section
      Atomics.store(a, 1, 0); // release
    }
  }
}
```

Actual output on the machine this book was written on

TEXT

```
expected      = 800000
plain a[0]++   = 500453
Atomsics.add    = 800000
spinlock (CAS)  = 800000
```

Almost 300,000 increments vanished. Run it again and you get a different number — that non-determinism **is** the definition of a race condition.

TRAP

“JavaScript is single-threaded so it cannot have race conditions.” Wrong on both halves. Your JS **code** runs on one thread, but (a) `await` and callbacks interleave tasks on that one thread, so a read-check-write split across

an `await` races with itself, and (b) `worker_threads` plus `SharedArrayBuffer` gives you genuine parallel threads on different cores. The output above is the proof.

Example 1 [warm-up]

counter starts at 5. Two threads each run `counter = counter + 1`. List every possible final value.

SOLUTION

1. If thread A completes all three steps (load, add, store) before B starts: A stores 6, B loads 6, stores 7. Final = 7.
2. Same if B goes completely first. Final = 7.
3. If they interleave — both load 5 before either stores — both compute 6 and both store 6. Final = 6.

Answer: 6 or 7. The correct answer is 7; 6 is the lost update.

Part 2 — Solving the critical-section problem

The three requirements, restated in plain words

1. **Mutual exclusion** — “only one at a time.” Easy to get.
2. **Progress** — “if the room is empty, somebody who wants in gets in, and soon.” A solution that makes two processes strictly alternate fails this: if A finishes and never comes back, B can only enter once more and then waits forever, even though the room is empty.
3. **Bounded waiting** — “no one waits forever while others cycle through.” This rules out starvation.

TRAP

The classic wrong answer is “mutual exclusion is enough.” A solution that simply says “nobody ever enters” gives perfect mutual exclusion and is useless. **Progress** is the requirement most broken solutions fail, and it is the one interviewers probe.

Attempt 1: a single turn variable — fails progress

Strict alternation — correct but not usable

C

```
/* shared */ int turn = 0;

/* process i */
while (turn != i) ;      /* entry: busy wait */
/* critical section */
turn = 1 - i;           /* exit */
```

Mutual exclusion: yes, `turn` can only hold one value. Progress: **no**. Process 0 must go, then 1, then 0, strictly. If process 1 never wants to enter again, process 0 is stuck forever at its second attempt even though the critical section is empty.

Attempt 2: a flag array — fails progress differently

Both raise a flag — this deadlocks

C

```
/* shared */ bool flag[2] = { false, false };

/* process i */
flag[i] = true;      /* "I want in" */
while (flag[1-i]) ;  /* wait while the other also wants in */
```

```
/* critical section */
flag[i] = false;
```

If both set their flag before either checks, both loop forever. That is a **deadlock**, and deadlock is a progress failure.

Peterson's solution — flag AND turn together

Peterson's algorithm for two processes — what the interviewer wants to see

C

```
/* shared */ bool flag[2] = { false, false };
int turn;

/* process i, with the other one called j */
flag[i] = true;           /* 1. I want in           */
turn = j;                 /* 2. but YOU go first       */
while (flag[j] && turn == j) ; /* 3. wait only if you want in AND it is your
turn */
/* critical section */
flag[i] = false;          /* 4. I am done             */
```

Why it works, in the three words the interviewer is listening for:

- **Mutual exclusion:** for both to be inside we would need `flag[0] && flag[1]` true and `turn` equal to both 0 and 1 at once. `turn` holds one value. Impossible.
- **Progress:** if process *j* does not want in, `flag[j]` is false and process *i* walks straight through. No alternation is forced.
- **Bounded waiting:** after *j* leaves it sets `flag[j] = false`; if it immediately tries again it must set `turn = i`, which lets *i* in. So *i* waits at most one turn.

SHORTCUT

The whole trick is line 2: `turn = j`, not `turn = i`. Each process **politely gives the turn away**. If both are polite at once, the last writer's value decides, and exactly one of them loses the tie. Writing `turn = i` breaks mutual exclusion, and it is the single most common slip.

TRAP

Peterson's algorithm assumes writes become visible in program order. Real CPUs **reorder** stores, so on modern hardware Peterson's needs memory fences to be correct. Say this if pushed — and say that in practice nobody hand-rolls it, because the hardware gives us atomic instructions instead.

The hardware answer: test-and-set and compare-and-swap

The CPU provides instructions that do read-and-write **as one indivisible step**.

What the hardware instruction does, written as if it were code

C

```
/* All of this happens atomically - no thread can get in between. */
bool test_and_set(bool *target) {
    bool old = *target;
    *target = true;
    return old;
}

int compare_and_swap(int *word, int expected, int newval) {
    int old = *word;
```

```

    if (old == expected) *word = newval;
    return old;           /* caller checks: old == expected means I won */
}

```

A spinlock built on it:

The simplest possible lock C

```

/* shared */ bool lock = false;

while (test_and_set(&lock)) ; /* spin until we are the one who flipped it */
/* critical section */
lock = false;

```

In JavaScript that exact instruction is `Atomics.compareExchange` — used in the 'lock' branch of the program above, and it produced the correct 800000.

TRAP

A spinlock gives mutual exclusion but **not** bounded waiting: an unlucky thread can lose the race every single time. A real lock adds a queue. Also: spinning burns CPU. Spinlocks are right only when the critical section is **shorter than a context switch** (a few hundred nanoseconds) — otherwise use a blocking lock that puts the waiter to sleep.

Part 3 — Mutex and semaphore

The definitions

Mutex (mutual exclusion lock): a lock with an **owner**. `lock()` and `unlock()`. Only the thread that locked it may unlock it.

Semaphore: an integer with `wait()` and `signal()`, and **no owner**. Any thread may signal.

Semaphore operations — the blocking version, which is what real systems use C

```

wait(S) {
    /* P, down, acquire */
    S->value--;
    if (S->value < 0) {
        add this process to S->queue;
        block();           /* go to WAITING state - no busy waiting */
    }
}

signal(S) {
    /* V, up, release */
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->queue;
        wakeup(P);         /* move P to READY */
    }
}

```

In this version a **negative** value has a meaning: $|S|$ is the number of processes waiting in the queue. If $S = -3$, three processes are blocked. That is a favourite follow-up question.

Mutex vs binary semaphore — the difference that gets marks

A binary semaphore is a semaphore whose value is only 0 or 1. It **looks** like a mutex. It is not.

	Mutex	Binary semaphore
Ownership	yes — the locker must be the unlocker	none — anyone may signal
Purpose	mutual exclusion	mutual exclusion or signalling between threads
Can it be released by another thread?	no (an error)	yes, and that is the point
Counting	no	a counting semaphore can go above 1
Priority inheritance	usually supported	usually not
Typical use	protect a shared variable	“the data is ready” signal; limit to N users

SHORTCUT

Say it in one line: “A mutex is a lock — it has an owner and the owner must release it. A semaphore is a counter — it has no owner, so one thread can wait and a completely different thread can signal. Use a mutex to protect data; use a semaphore to signal an event or to cap concurrency at N .”

A mutex you can read

A promise-based mutex, and the race from Part 1 fixed. Run: node c2.js

JS

```
class Mutex {
  constructor() { this.locked = false; this.waiters = []; }
  lock() {
    if (!this.locked) { this.locked = true; return Promise.resolve(); }
    return new Promise(resolve => this.waiters.push(resolve));
  }
  unlock() {
    const next = this.waiters.shift();
    if (next) next(); // hand the lock straight to the next waiter
    else this.locked = false;
  }
}

let balance = 100;
const m = new Mutex();
const sleep = (ms) => new Promise(r => setTimeout(r, ms));

async function withdraw(name, amount) {
  await m.lock(); // ENTRY
  try { // ---- critical section ----
    const seen = balance;
    await sleep(10);
    if (seen >= amount) { balance = seen - amount; console.log(`${name}: took ${amount}, balance now ${balance}`); }
    else { console.log(`${name}: refused, only ${seen} available`); }
  } finally { m.unlock(); } // EXIT - finally, so a throw cannot leak the lock
}

(async () => {
  await Promise.all([withdraw('A', 100), withdraw('B', 100)]);
})
```

```
    console.log('final balance =', balance);
  })();
```

Actual output — now correct

TEXT

```
A: took 100, balance now 0
B: refused, only 0 available
final balance = 0
```

TRAP

`try { ... } finally { m.unlock(); }` is not decoration. If the critical section throws and you unlock outside a `finally`, the lock is never released and **every** future caller blocks forever. That is a self-inflicted deadlock, and it is the number one concurrency bug in real code.

A counting semaphore you can read

Cap concurrency at 2. Run: `node c3.js`

JS

```
class Semaphore {
  constructor(count) { this.count = count; this.waiters = []; }
  async wait() { // P / down / acquire
    if (this.count > 0) { this.count--; return; }
    await new Promise(resolve => this.waiters.push(resolve));
  }
  signal() { // V / up / release
    const next = this.waiters.shift();
    if (next) next(); // do NOT raise count: pass the permit on
    else this.count++;
  }
}

const sem = new Semaphore(2);
const sleep = (ms) => new Promise(r => setTimeout(r, ms));
let inside = 0, peak = 0;

async function job(id) {
  await sem.wait();
  inside++; peak = Math.max(peak, inside);
  console.log(`job ${id} entered (inside now ${inside})`);
  await sleep(30);
  inside--;
  console.log(`job ${id} left (inside now ${inside})`);
  sem.signal();
}

(async () => {
  await Promise.all([1, 2, 3, 4, 5].map(job));
  console.log(`peak concurrent = ${peak} (limit was 2)`);
})();
```

Actual output (trimmed)

TEXT


```

job 1 entered  (inside now 1)
job 2 entered  (inside now 2)
job 1 left     (inside now 1)
job 3 entered  (inside now 2)
...
peak concurrent = 2  (limit was 2)

```

Note the line in `signal()`: when a waiter exists we hand it the permit directly instead of incrementing the count. Incrementing **and** waking would let a third job slip in. That single line is the difference between a working semaphore and a subtly broken one.

Example 2 [Capgemini · pattern]

A counting semaphore is initialised to 3. Then 5 `wait()` calls happen, followed by 2 `signal()` calls. What is the value of the semaphore, and how many processes are blocked?

SOLUTION

Use the version where the value can go negative.

Operation	Value after	Blocked	Why
start	3	0	three permits free
wait 1	2	0	$3 - 1$, still ≥ 0 , so it proceeds
wait 2	1	0	proceeds
wait 3	0	0	proceeds — permits now exhausted
wait 4	-1	1	value went below 0, so this one blocks
wait 5	-2	2	blocks
signal 1	-1	1	value ≤ 0 , so wake one waiter
signal 2	0	0	wake the other

Answer: Value = 0, and 0 processes are blocked.

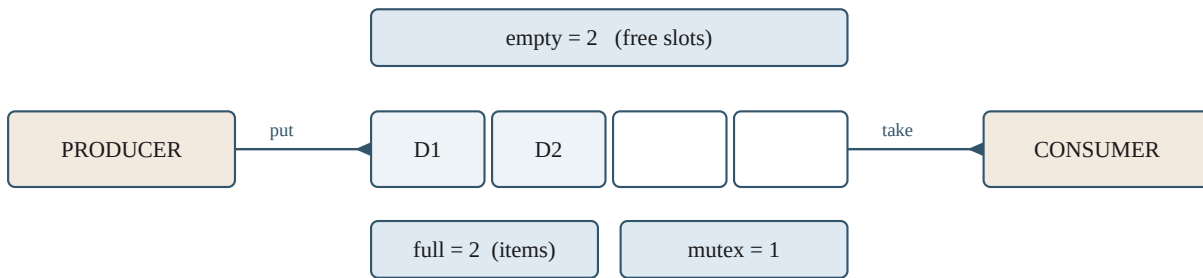
Follow-up they will ask: “what would it be after only the five waits?” -2, with 2 blocked — and the absolute value of a negative semaphore is always the number of waiters.

Part 4 — The three classic problems

Every OS syllabus has the same three. Know the shape of each and the failure mode of each.

4.1 Producer–Consumer (bounded buffer)

A producer adds items to a buffer of N slots; a consumer removes them. The producer must not write to a full buffer; the consumer must not read from an empty one.



Bounded buffer with three semaphores. `empty` counts free slots, `full` counts items, `mutex` protects the buffer itself.

The standard solution — memorise this shape

C

```
semaphore empty = N; /* free slots */
semaphore full = 0; /* used slots */
semaphore mutex = 1; /* protects the buffer */

producer() {
    while (true) {
        item = produce();
        wait(empty); /* is there room? */
        wait(mutex);
        insert(item);
        signal(mutex);
        signal(full); /* one more item */
    }
}

consumer() {
    while (true) {
        wait(full); /* is there anything? */
        wait(mutex);
        item = remove();
        signal(mutex);
        signal(empty); /* a slot is free now */
        consume(item);
    }
}
```

TRAP

Swap the order of the two waits and you get a deadlock. If the producer does `wait(mutex)` before `wait(empty)` and the buffer is full, the producer holds `mutex` and sleeps on `empty`. The consumer needs `mutex` to remove an item, so it blocks too. Nobody can ever signal `empty`. Both sleep forever.

The rule to state out loud: **always acquire the counting semaphore before the mutex, and release in the reverse order.** Never sleep while holding a lock.

The same solution, runnable. Run: `node c4.js`

JS

```
class Semaphore {
    constructor(c) { this.count = c; this.waiters = []; }
    async wait() { if (this.count > 0) { this.count--; return; }
        await new Promise(r => this.waiters.push(r)); }
    signal() { const n = this.waiters.shift(); if (n) n(); else this.count++; }
}

const N = 3, buf = [];
const empty = new Semaphore(N); // how many free slots
const full = new Semaphore(0); // how many filled slots
const mutex = new Semaphore(1); // protects buf itself
const sleep = ms => new Promise(r => setTimeout(r, ms));

async function producer(items) {
    for (const it of items) {
        await empty.wait(); await mutex.wait();
```

```

    buf.push(it); console.log(`produced ${it}  buffer=[${buf}] size=${buf.length}`);
    mutex.signal(); full.signal();
    await sleep(5);
  }
}
async function consumer(n) {
  for (let i = 0; i < n; i++) {
    await full.wait(); await mutex.wait();
    const it = buf.shift(); console.log(`          consumed ${it}  buffer=[${buf}]
size=${buf.length}`);
    mutex.signal(); empty.signal();
    await sleep(20);
  }
}
(async () => {
  await Promise.all([producer([1,2,3,4,5,6]), consumer(6)]);
  console.log('done. buffer never exceeded', N);
})();

```

Actual output — the producer is faster, so the buffer fills to 3 and stops

TEXT

```

produced 1  buffer=[1] size=1
      consumed 1  buffer=[] size=0
produced 2  buffer=[2] size=1
produced 3  buffer=[2,3] size=2
produced 4  buffer=[2,3,4] size=3
      consumed 2  buffer=[3,4] size=2
produced 5  buffer=[3,4,5] size=3
      consumed 3  buffer=[4,5] size=2
produced 6  buffer=[4,5,6] size=3
      consumed 4  buffer=[5,6] size=2
      consumed 5  buffer=[6] size=1
      consumed 6  buffer=[] size=0
done. buffer never exceeded 3

```

The producer pushes every 5 ms, the consumer pulls every 20 ms, yet the buffer never grows past 3. The empty semaphore forced the producer to wait. That is back-pressure, and it is the same mechanism a message queue uses at scale.

4.2 Readers–Writers

Many readers may read at once. A writer needs exclusive access — no other writer, no readers.

First readers-writers solution (reader preference) — shared state

C

```

semaphore rw_mutex  = 1; /* exclusive access to the data */
semaphore mutex     = 1; /* protects read_count only    */
int        read_count = 0; /* how many readers are inside */

```

The writer — the simple half

C

```

writer() {
  wait(rw_mutex);
  /* ... write the data ... */
}

```

```
    signal(rw_mutex);
}
```

The reader — all the cleverness is here

C

```
reader() {
    wait(mutex);
    read_count++;
    if (read_count == 1)
        wait(rw_mutex);    /* the FIRST reader locks writers out */
    signal(mutex);

    /* ... read the data ... */

    wait(mutex);
    read_count--;
    if (read_count == 0)
        signal(rw_mutex);  /* the LAST reader lets writers back in */
    signal(mutex);
}
```

The key idea: only the **first** reader takes `rw_mutex` and only the **last** reader releases it. Readers in between just adjust the counter.

Variant	Rule	Who starves
First (reader preference)	a reader never waits unless a writer is already writing	writers — a steady stream of readers blocks them forever
Second (writer preference)	once a writer is waiting, no new reader may start	readers
Third (fair)	an extra queue semaphore serves requests in arrival order	nobody

Example 3 [GIC · pattern]

A price table is read by 200 client threads every few milliseconds and written by one updater thread once a second. The team uses the first readers-writers solution. After a week, the updater has applied almost no updates. Explain exactly why, and give the smallest fix.

SOLUTION

1. In the first solution, `rw_mutex` is held from the moment the **first** reader arrives until the moment the **last** reader leaves.
2. With 200 readers arriving every few milliseconds, `read_count` essentially never returns to 0.
3. So `signal(rw_mutex)` is essentially never executed, and the writer's `wait(rw_mutex)` never returns.
4. This is **starvation**, not deadlock: the system is making progress — readers are being served — just never the writer.

Smallest fix: switch to the **second** (writer-preference) variant, so that as soon as the writer is waiting, newly arriving readers queue behind it. The 200 readers already inside finish, `read_count` reaches 0, and the writer gets in.

Answer: Reader preference starves the writer. Use writer preference, or a fair queue.

The follow-up: “**what does writer preference cost?**” Read latency spikes once a second while the writer drains the readers. Naming that trade is the point.

TRAP

Starvation is not deadlock. In deadlock, nothing at all progresses and no amount of waiting helps. In starvation, the system is busy and productive — one unlucky process just never gets served. Deadlock is fixed by breaking a cycle; starvation is fixed by adding fairness (aging, FIFO queues, priority boosts).

4.3 Dining philosophers

Five philosophers sit around a table. Between each pair is one fork. To eat, a philosopher needs the fork on the left **and** the fork on the right.

The naive solution — everybody picks up the left fork, then the right — deadlocks the moment all five pick up their left fork at the same instant. Every fork is held; every philosopher waits for a neighbour.

The deadlock and the one-line fix. Run: node c8.js

JS

```
const N = 5, fork = Array.from({length:N}, () => new Mutex());

async function philosopher(i, asymmetric) {
  const left = i, right = (i + 1) % N;
  let [a, b] = [left, right];
  if (asymmetric && i === N - 1) [a, b] = [right, left]; // the one lefty
  for (let k = 0; k < 3; k++) {
    await fork[a].lock(); await sleep(1); await fork[b].lock();
    meals++; // eating
    await sleep(1);
    fork[b].unlock(); fork[a].unlock();
    await sleep(1);
  }
}
```

Actual output

TEXT

```
everyone grabs left first : STUCK after 0 meals
one philosopher reversed : finished all 15 meals
```

Four accepted fixes — know all four, they are separate questions:

Fix	How	Which condition it breaks
Allow only 4 at the table	a semaphore initialised to $N - 1$ guards the table	circular wait — with 4 philosophers and 5 forks, someone always gets both
Pick up both or neither	test both forks inside one mutex; if both free, take both	hold and wait
Asymmetry	odd-numbered philosophers take right first, even take left first	circular wait
Global ordering	number the forks; always take the lower number first	circular wait

SHORTCUT

Global lock ordering is the fix you should name **first** in a real-world question, because it generalises: give every lock in the system a rank, and require every thread to acquire locks in increasing rank. A cycle needs somebody to go “down” in rank, and nobody is allowed to.

Part 5 — Deadlock

The four necessary conditions

Condition	Meaning	How you would break it
Mutual exclusion	at least one resource cannot be shared	make it shareable — e.g. a spooler so nobody holds the printer
Hold and wait	a process holds one resource and waits for another	require all resources to be requested at once; or release everything before asking for more
No preemption	a resource is only released voluntarily	let the OS take it back — works for CPU and memory, not for a half-written file
Circular wait	a cycle of waiting processes	impose a global ordering on resource types and require increasing order

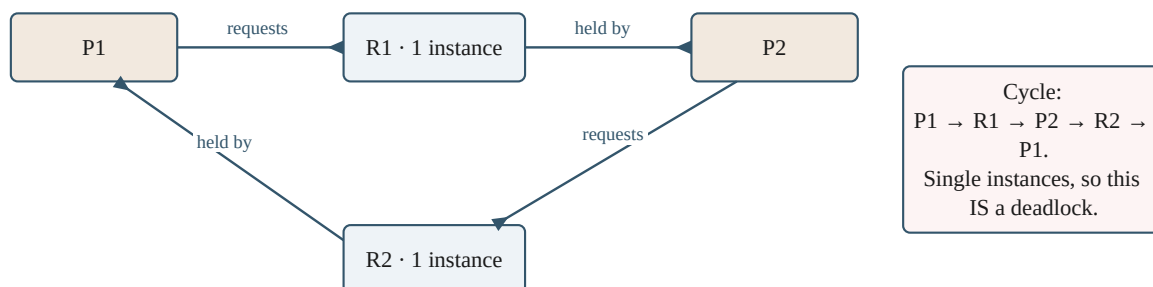
TRAP

Two mistakes, both fatal:

(1) “Deadlock needs all four conditions, so if any one is missing there is no deadlock.” The first half is right, the second half is the **useful** half — but students often state the reverse: “if all four hold, deadlock happens.” That is **false**. The four conditions are **necessary, not sufficient**. All four can hold and the system may still run fine; it only **can** deadlock. The RAG cycle test is what makes it sufficient in the single-instance case.

(2) Circular wait is the easiest one to attack in practice, and the **only** one most real systems bother with. Breaking mutual exclusion is usually impossible.

The resource allocation graph



A resource allocation graph. Process-to-resource = a request. Resource-to-process = an assignment. This graph has a cycle.

The rule has two halves, and the second half is the one that gets marks:

- **Single instance of every resource type:** a cycle in the graph means deadlock, and no cycle means no deadlock. Cycle $\Leftrightarrow$ deadlock.
- **Multiple instances of some type:** a cycle is only a **possibility** of deadlock. You must run the detection algorithm to be sure.

Example 4 [Wipro · pattern]

A resource allocation graph has a cycle. Can you conclude the system is deadlocked? Explain in two lines.

SOLUTION

Only if **every resource type in the cycle has exactly one instance**. With one instance each, a cycle means each process is waiting for a resource that can only be freed by the next process in the cycle, which is itself waiting — nothing can move.

If some resource type in the cycle has two or more instances, the extra instance might be held by a process **outside** the cycle that will finish and release it, breaking the cycle.

Answer: No — a cycle is necessary but sufficient only when all resources in it are single-instance.

A deadlock you can run, and the fix

Two locks, two orders. Run: node c5.js

JS

```
const A = new Mutex('A'), B = new Mutex('B');

async function taskAB() {           // takes A, then B
  await A.lock(); console.log('T1 holds A');
  await sleep(20);
  console.log('T1 wants B ...');
  await B.lock(); console.log('T1 holds B');
  B.unlock(); A.unlock();
}

async function taskBA() {           // takes B, then A <-- the opposite order
  await B.lock(); console.log('T2 holds B');
  await sleep(20);
  console.log('T2 wants A ...');
  await A.lock(); console.log('T2 holds A');
  A.unlock(); B.unlock();
}
```

Actual output

TEXT

```
T1 holds A
T2 holds B
T1 wants B ...
T2 wants A ...
--- 300 ms later: nothing moved. DEADLOCK. ---
A.locked = true  waiters: 1
B.locked = true  waiters: 1
```

All four conditions are visible in those six lines: the mutexes are exclusive (1), each task holds one and waits for the other (2), neither can be forced to let go (3), and T1 waits on T2 which waits on T1 (4).

The fix: a global ranking of locks. Run: node c6.js

JS

```
const ORDER = { A: 1, B: 2 };           // one global ranking of every lock

async function withBoth(who, m1, m2, work) {
  const lower = ORDER[m1.name] < ORDER[m2.name];
  const first  = lower ? m1 : m2;       // always take the lower rank first
  const second = lower ? m2 : m1;

  await first.lock();
  console.log(`${who} holds ${first.name}`);
  await sleep(20);
  await second.lock();
  console.log(`${who} holds ${second.name}`);

  await work();
}
```

```
second.unlock(); // release in reverse order
first.unlock();
console.log(`${who} released both`);
}
```

Actual output — T2 asked for B then A, but the helper reordered it

TEXT

```
T1 holds A
T1 holds B
T1 released both
T2 holds A
T2 holds B
T2 released both
both finished - circular wait was impossible
```

The four ways an OS can handle deadlock

Strategy	When it acts	Cost	Used by
Prevention	before anything runs — design the rules so a condition cannot hold	low resource utilisation; awkward to program	real-time and embedded systems
Avoidance	at each request — grant only if the state stays safe	needs each process to declare its maximum need in advance	almost nothing in practice
Detection + recovery	after the fact — run a check, then kill or roll back	the check costs CPU; recovery loses work	database engines
Ignore it (the “ostrich algorithm”)	never	a rare hang; reboot	Linux, Windows, macOS for user processes

Answer: If asked “what does Linux do about deadlock between user processes?” the correct answer is **nothing**. It is cheaper to reboot once a year than to slow every allocation down.

Banker’s algorithm — deadlock avoidance, worked in full

The idea: before granting a request, **pretend** to grant it and check whether the resulting state is **safe**. Safe means “there exists an order in which all processes can finish”. If the pretend-state is unsafe, do not grant; make the process wait.

THE STATE WE WILL WORK WITH

Four transactions T1–T4 compete for three pools: **A** = database connections (8 total), **B** = file locks (6 total), **C** = memory blocks (9 total).

	Allocation			Max		
	A	B	C	A	B	C
T1	1	2	2	4	3	3
T2	2	0	1	5	2	4
T3	3	1	3	4	3	5
T4	1	1	1	3	2	3

Step 1 — compute Need = Max – Allocation, cell by cell.

Process	A	B	C
T1	$4 - 1 = 3$	$3 - 2 = 1$	$3 - 2 = 1$
T2	$5 - 2 = 3$	$2 - 0 = 2$	$4 - 1 = 3$
T3	$4 - 3 = 1$	$3 - 1 = 2$	$5 - 3 = 2$
T4	$3 - 1 = 2$	$2 - 1 = 1$	$3 - 1 = 2$

Step 2 — compute Available = Total – (sum of the Allocation column).

- A: allocated = $1 + 2 + 3 + 1 = 7$. Available = $8 - 7 = 1$.
- B: allocated = $2 + 0 + 1 + 1 = 4$. Available = $6 - 4 = 2$.
- C: allocated = $2 + 1 + 3 + 1 = 7$. Available = $9 - 7 = 2$.

Answer: Available = (1, 2, 2)

Step 3 — the safety check. Keep a running vector Work, starting at Available. Walk the list; whenever Need $\leq$ Work, that process can finish, so add its Allocation to Work.

Try	Need vs Work	Verdict	Work becomes
T1	(3, 1, 1) vs (1, 2, 2) — A: $3 > 1$	blocked	(1, 2, 2)
T2	(3, 2, 3) vs (1, 2, 2) — A: $3 > 1$	blocked	(1, 2, 2)
T3	(1, 2, 2) vs (1, 2, 2) — all $\leq$	finishes , releases (3, 1, 3)	(4, 3, 5)
T4	(2, 1, 2) vs (4, 3, 5) — all $\leq$	finishes , releases (1, 1, 1)	(5, 4, 6)
T1	(3, 1, 1) vs (5, 4, 6) — all $\leq$	finishes , releases (1, 2, 2)	(6, 6, 8)
T2	(3, 2, 3) vs (6, 6, 8) — all $\leq$	finishes , releases (2, 0, 1)	(8, 6, 9)

Work ends at (8, 6, 9), which is exactly Total — every resource is back. All four finished.

Answer: The state is SAFE. Safe sequence: T3 → T4 → T1 → T2

Notice that **T3 is the only process that can start**. Availability (1, 2, 2) matches T3's need exactly. If T3 had needed one more of anything, the state would have been unsafe.

Example 5 [Grab · pattern]

From the state above, T2 requests (1, 1, 1). Should the banker grant it?

SOLUTION

Three checks, in this order. Skipping the order is how students lose the mark.

Check 1 — is the request within what T2 declared? Request = (1, 1, 1), Need[T2] = (3, 2, 3). $1 \leq 3$, $1 \leq 2$, $1 \leq 3$ ✓. Legal.

Check 2 — is there enough available right now? Available = (1, 2, 2). $1 \leq 1$, $1 \leq 2$, $1 \leq 2$ ✓. Physically possible.

Check 3 — pretend to grant it, then test safety.

- Available becomes $(1, 2, 2) - (1, 1, 1) = (0, 1, 1)$.
- Allocation[T2] becomes $(2, 0, 1) + (1, 1, 1) = (3, 1, 2)$.
- Need[T2] becomes $(3, 2, 3) - (1, 1, 1) = (2, 1, 2)$.

Now run the safety check with Work = (0, 1, 1):

Try	Need vs Work = (0, 1, 1)	Verdict
T1	(3, 1, 1) — A: $3 > 0$	blocked
T2	(2, 1, 2) — A: $2 > 0$	blocked

T3	(1, 2, 2) — A: $1 > 0$	blocked
T4	(2, 1, 2) — A: $2 > 0$	blocked

No process can proceed. Work never grows. The state is **unsafe**.

Answer: DENY. T2 must wait, even though the resources are physically available.

That last clause is the whole lesson of the Banker's algorithm and the follow-up question you should answer before it is asked: **the banker refuses requests it could physically satisfy, because satisfying them would leave no guaranteed way out.**

Example 6 [SCB · pattern]

Back to the original state. Now T3 requests (0, 2, 2). Grant or deny?

SOLUTION

Check 1 — within Need? $\text{Need}[T3] = (1, 2, 2)$. $0 \leq 1, 2 \leq 2, 2 \leq 2$ ✓.

Check 2 — available? $\text{Available} = (1, 2, 2)$. $0 \leq 1, 2 \leq 2, 2 \leq 2$ ✓.

Check 3 — pretend and test.

- $\text{Available} = (1, 2, 2) - (0, 2, 2) = (1, 0, 0)$.
- $\text{Allocation}[T3] = (3, 1, 3) + (0, 2, 2) = (3, 3, 5)$.
- $\text{Need}[T3] = (1, 2, 2) - (0, 2, 2) = (1, 0, 0)$.

Safety check with $\text{Work} = (1, 0, 0)$:

Try	Need vs Work	Verdict	Work becomes
T1	(3, 1, 1) vs (1, 0, 0)	blocked	(1, 0, 0)
T2	(3, 2, 3) vs (1, 0, 0)	blocked	(1, 0, 0)
T3	(1, 0, 0) vs (1, 0, 0) — equal	finishes , releases (3, 3, 5)	(4, 3, 5)
T4	(2, 1, 2) vs (4, 3, 5)	finishes , releases (1, 1, 1)	(5, 4, 6)
T1	(3, 1, 1) vs (5, 4, 6)	finishes , releases (1, 2, 2)	(6, 6, 8)
T2	(3, 2, 3) vs (6, 6, 8)	finishes , releases (2, 0, 1)	(8, 6, 9)

Answer: GRANT. Safe sequence T3 → T4 → T1 → T2.

Compare with Example 5: T2's request took away the last unit of A and left nobody able to move. T3's request took B and C but **left A alone** — and A was the scarce one.

Example 7 [TCS NQT · pattern]

Back to the original state. T1 requests (2, 0, 0). What happens?

SOLUTION

Check 1 — within Need? $\text{Need}[T1] = (3, 1, 1)$. $2 \leq 3$ ✓.

Check 2 — available? $\text{Available} = (1, 2, 2)$. Is $2 \leq 1$? **No**.

The check fails at step 2. We never reach the safety test.

Answer: T1 must WAIT — not because the state would be unsafe, but because the resources are not there right now.

TRAP

“Wait because unsafe” and “wait because unavailable” are different answers, and examiners check which one you gave. Step 2 failing means **“come back later”**. Step 3 failing means **“I have the resources but I refuse”**. Only the second one is the Banker's algorithm doing anything interesting.

Deadlock detection — when there is no Max matrix

If processes never declare a maximum, avoidance is impossible. Instead, let deadlock happen and detect it. The algorithm is the safety check with Request in place of Need.

Four transactions, three resource pools with totals $A = 4$, $B = 3$, $C = 3$:

	Allocation			Request (outstanding)		
	A	B	C	A	B	C
Q1	1	1	0	2	0	1
Q2	1	0	2	0	1	0
Q3	1	1	0	2	0	1
Q4	1	0	1	0	0	0

Available: $A = 4 - 4 = 0$, $B = 3 - 2 = 1$, $C = 3 - 3 = 0$, so Available = (0, 1, 0).

Case 1 — the table as shown.

Try	Request vs Work	Verdict	Work becomes
Q1	(2, 0, 1) vs (0, 1, 0)	stuck	(0, 1, 0)
Q2	(0, 1, 0) vs (0, 1, 0)	can finish , releases (1, 0, 2)	(1, 1, 2)
Q3	(2, 0, 1) vs (1, 1, 2) — A: $2 > 1$	stuck	(1, 1, 2)
Q4	(0, 0, 0) vs (1, 1, 2)	can finish , releases (1, 0, 1)	(2, 1, 3)
Q1	(2, 0, 1) vs (2, 1, 3)	can finish , releases (1, 1, 0)	(3, 2, 3)
Q3	(2, 0, 1) vs (3, 2, 3)	can finish , releases (1, 1, 0)	(4, 3, 3)

All four completed.

Answer: No deadlock. Completion order Q2, Q4, Q1, Q3.

Case 2 — identical table, except Q2 now requests (0, 3, 0) instead of (0, 1, 0).

Try	Request vs Work	Verdict	Work becomes
Q1	(2, 0, 1) vs (0, 1, 0)	stuck	(0, 1, 0)
Q2	(0, 3, 0) vs (0, 1, 0) — B: $3 > 1$	stuck	(0, 1, 0)
Q3	(2, 0, 1) vs (0, 1, 0)	stuck	(0, 1, 0)
Q4	(0, 0, 0) vs (0, 1, 0)	can finish , releases (1, 0, 1)	(1, 1, 1)
Q1	(2, 0, 1) vs (1, 1, 1) — A: $2 > 1$	stuck	(1, 1, 1)
Q2	(0, 3, 0) vs (1, 1, 1) — B: $3 > 1$	stuck	(1, 1, 1)
Q3	(2, 0, 1) vs (1, 1, 1) — A: $2 > 1$	stuck	(1, 1, 1)

Work stops growing at (1, 1, 1) and three processes never finish.

Answer: DEADLOCKED: Q1, Q2 and Q3. Only Q4 completed.

One extra unit of demand from Q2 turned a working system into a three-way deadlock. That is how sudden real deadlocks are.

Recovery

Once detected, something has to give:

1. **Abort every deadlocked process.** Always works, maximum damage.

2. **Abort one at a time** until the cycle breaks, re-running detection after each. Cheaper, slower. Choose the victim by: lowest priority, least CPU used so far, fewest resources held, least work lost.
3. **Preempt resources** and roll a process back to a safe checkpoint. This is what database engines do — they pick the transaction that has done the least work and abort it with a “deadlock victim” error, and the application retries.

TRAP

Rolling back can cause **starvation**: the same process is chosen as victim every time, is rolled back every time, and never completes. Real systems fix this by including “how many times have you already been the victim” in the victim-selection cost.

Part 6 — Monitors and condition variables

Semaphores work, but they are **easy to get wrong**. Miss a `signal` and the system hangs. Miss a `wait` and mutual exclusion is gone. Swap two lines and you deadlock. All of those bugs are invisible in a code review, because a semaphore is just an integer and nothing connects it to the data it is supposed to protect.

A **monitor** fixes that by construction. It is a language feature: a bundle of shared data plus the procedures that operate on it, with the rule that **only one thread can be executing inside the monitor at any time**. The compiler inserts the locking; you cannot forget it.

The shape of a monitor

C

```
monitor Account {
    int balance = 0;           /* data is PRIVATE to the monitor */
    condition enough;         /* a condition variable */

    void deposit(int n) {
        balance += n;
        signal(enough);       /* wake one waiter, if any */
    }

    void withdraw(int n) {
        while (balance < n)    /* while, NOT if - see the trap */
            wait(enough);      /* release the monitor and sleep */
        balance -= n;
    }
}
```

Condition variables — two operations, and they are not semaphores

- `wait(c)` — release the monitor lock and sleep on `c`. The thread is suspended.
- `signal(c)` — wake **one** thread waiting on `c`. If nobody is waiting, **nothing happens**.

	Semaphore	Condition variable
Has a value?	yes — an integer	no — it is only a waiting list
<code>signal</code> with no waiter	increments; the next <code>wait</code> passes straight through	is lost forever — nothing is remembered
Used with a lock?	it is the lock	always used inside a monitor / with a mutex
What it means	“there are N permits”	“a condition may have become true”

TRAP

Two mistakes, both extremely common:

(1) **if instead of while.** After `wait(c)` returns, the condition you were waiting for may already be false again — another thread could have run between the signal and your wake-up, or the wake-up could be spurious. Always re-check in a loop: `while (!condition) wait(c);` Writing `if` produces a bug that appears once in ten thousand runs.

(2) **“A condition variable is a binary semaphore.”** No. A signal on a condition variable with no waiter is **thrown away**. A signal on a semaphore is **remembered** as a count. That difference is exactly why condition variables must always be paired with a re-checked predicate.

Signal-and-wait vs signal-and-continue

When thread A signals and thread B was waiting, who holds the monitor next?

- **Hoare (signal-and-wait):** B runs immediately; A steps aside and waits. B is guaranteed the condition still holds. Harder to implement, costs an extra context switch.
- **Mesa (signal-and-continue):** A keeps running; B becomes merely **ready**. Simpler and faster — and it is why the condition can be false again by the time B wakes, and why the while loop is mandatory.

Every real system (Java’s `synchronized/wait/notify`, POSIX condition variables, C# monitors) uses **Mesa** semantics.

Example 8 [LINE MAN · pattern]

A job queue uses a monitor. A worker does:

```
if (queue.isEmpty()) wait(notEmpty);
job = queue.removeFirst();
```

With one worker it is fine. With eight workers, `removeFirst()` sometimes throws “empty queue”. Explain and fix.

SOLUTION

1. A producer adds one job and calls `signal(notEmpty)`.
2. Under Mesa semantics the producer **keeps running**; the woken worker W1 only becomes **READY**.
3. Before W1 is scheduled, another worker W2 — which arrived fresh and never waited — enters the monitor, sees a non-empty queue, and takes the job.
4. W1 finally runs. Its `wait` has returned, the `if` is not re-evaluated, and the queue is empty again. `removeFirst()` throws.

With one worker there is no W2, so the bug never shows.

Fix — one character of meaning, three of text:

```
while (queue.isEmpty()) wait(notEmpty);
job = queue.removeFirst();
```

Answer: Mesa semantics mean the condition can go false between signal and wake-up. Use while, never if.

Follow-up to volunteer: **“and this is also why signal versus broadcast matters”** — if several waiters are waiting for different conditions on the same variable, `signal` may wake the wrong one, and it goes back to sleep while the right one is never woken. Use `broadcast (notifyAll)` when waiters are not interchangeable.

JavaScript has no monitors, but the `Mutex` class from Part 3 plus a `while` re-check is exactly the same pattern. In Java the keywords are `synchronized`, `wait()`, `notify()` and `notifyAll()`, and the standard advice — “always call `wait()` inside a loop” — is this rule.

Part 7 — Three more hazards with similar names

Name	What is happening	Are threads making progress?
Deadlock	a cycle of waiting; nobody can move	no — CPU idle or doing other work

Livelock	threads keep changing state in response to each other but never advance	yes — they are busy, and achieving nothing
Starvation	the system works, but one process never gets served	yes, for everyone else

Livelock in one image: two people meet in a corridor, both step left, both step right, both step left. They are moving. They are not getting past. In code it appears with “try-lock and back off” schemes where both threads back off by the same amount, retry at the same moment, and collide again. The fix is **randomised** back-off.

Priority inversion

Three threads, priorities High, Medium, Low:

1. **L** (low) acquires a lock on a shared resource.
2. **H** (high) wakes up, tries to take the same lock, and blocks behind L. So far, correct.
3. **M** (medium) wakes up. It needs no lock. It preempts L, because M outranks L.
4. Now M runs, L cannot run, so L cannot release the lock, so H cannot run.

The result: a **medium**-priority thread has effectively blocked a **high**-priority one. That is priority inversion, and in a real-time system it is a missed deadline.

Fix	How it works
Priority inheritance	while H is blocked on L’s lock, L temporarily runs at H’s priority. M can no longer preempt L. L finishes fast, releases, and drops back down.
Priority ceiling	every lock is tagged with the highest priority of any thread that may ever take it. A thread holding the lock immediately runs at that ceiling.

Example 9 [Goldman Sachs · pattern]

A trading system has a high-priority risk-check thread, a medium-priority logging thread, and a low-priority metrics thread. Risk checks occasionally take 400 ms instead of the usual 2 ms. Metrics and risk both use a shared configuration object behind a mutex. Explain the mechanism, and give two fixes with their trade-offs.

SOLUTION

Mechanism — priority inversion.

1. The metrics thread (low) takes the config mutex.
2. A risk check (high) starts, needs the config, and blocks on that mutex.
3. The logging thread (medium) becomes runnable. It does not need the mutex, and it outranks metrics, so it preempts metrics.
4. Metrics cannot run, so it cannot release the mutex, so risk stays blocked — for as long as logging keeps running. The 400 ms is the logging burst, not the risk check itself.

The tell-tale sign is that the delay is **bimodal**: usually 2 ms, occasionally a fixed large value that matches some other thread’s work, never anything in between.

Fix 1 — priority inheritance on that mutex. While risk is blocked on it, metrics runs at risk’s priority, so logging cannot preempt it. Trade-off: the delay drops to the length of the metrics critical section, but it is no longer possible to reason about the priority of a thread by reading its declaration — priorities change at runtime, and that makes the system harder to analyse and test.

Fix 2 — remove the sharing. Give the risk thread its own immutable snapshot of the configuration, swapped atomically by a single pointer write when config changes. Readers take no lock at all. Trade-off: a risk check may briefly use a configuration that is one version old, and memory use doubles while both versions are live. For a risk check running every few milliseconds, one stale version is almost always acceptable — and this fix removes the whole class of bug rather than softening it.

Answer: Priority inversion; fix with priority inheritance, or better, by removing the shared mutable state.

The sentence that wins this question: “**The correct long-term fix is to not share mutable state across priority levels. Priority inheritance treats the symptom.**”

Practice

WARM-UP

build the reflex

PRACTICE · WARM-UP target: 8 min

1. Define a race condition in one sentence.
2. Name the three requirements of a correct critical-section solution.
3. A counting semaphore starts at 1. What is it usually called?
4. Which semaphore operation decreases the value?
5. Name the four necessary conditions for deadlock.
6. A binary semaphore is at 0. A thread calls `wait()`. What happens to it?
7. True or false: a spinlock puts the waiting thread to sleep.
8. In the bounded-buffer solution, what does the `full` semaphore count?

ANSWER KEY

1. Two or more threads access shared data concurrently and the final result depends on the order in which they run.
2. Mutual exclusion, progress, bounded waiting.
3. A binary semaphore (used as a lock).
4. `wait()` — also called P, down, or acquire.
5. Mutual exclusion, hold and wait, no preemption, circular wait.
6. It blocks — it is added to the semaphore's queue and moved to the WAITING state.
7. False. A spinlock **busy-waits**, burning CPU. A blocking lock sleeps.
8. The number of **filled** slots, i.e. items available to consume.

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

PRACTICE · TIER 1 target: 20 min

1. Give three differences between a mutex and a semaphore.
2. A semaphore starts at 4. Operations happen in this order: wait, wait, signal, wait, wait, wait, wait. Give the final value and the number of blocked processes.
3. Explain why the producer must call `wait(empty)` before `wait(mutex)` and not the other way round.
4. Which of the four deadlock conditions does “request all your resources at once” break?
5. Why is starvation not the same as deadlock?
6. Write the entry and exit sections of Peterson's algorithm from memory.
7. A system has 1 printer, 1 scanner. P1 holds the printer and wants the scanner; P2 holds the scanner and wants the printer. Draw the RAG and say whether this is a deadlock.

ANSWER KEY

1. (i) A mutex has an **owner** — only the locking thread may unlock; a semaphore has none. (ii) A mutex is binary; a counting semaphore can hold any non-negative count. (iii) A mutex is for mutual exclusion only; a semaphore is also used for **signalling** between threads and for capping concurrency at N . (Bonus fourth: mutexes usually support priority inheritance; semaphores usually do not.)
2. Start 4. wait $\rightarrow$ 3, wait $\rightarrow$ 2, signal $\rightarrow$ 3, wait $\rightarrow$ 2, wait $\rightarrow$ 1, wait $\rightarrow$ 0, wait $\rightarrow$ -1. Final value = -1, so **1 process is blocked**.

3. If the producer takes `mutex` first and the buffer is full, it will then block on `wait(empty)` while still holding the `mutex`. The consumer needs the `mutex` to remove an item and free a slot, so it blocks too. Neither can proceed — deadlock. The general rule: never go to sleep while holding a lock.

4. Hold and wait. (If you get everything at once, you never hold one thing while waiting for another.)

5. In deadlock nothing progresses and waiting will never help, because every process in the cycle waits on another in the cycle. In starvation the system is progressing — other processes are being served — one particular process just keeps losing. Deadlock needs the cycle broken; starvation needs fairness added.

6. Entry: `flag[i] = true; turn = j; while (flag[j] && turn == j) ;` Exit: `flag[i] = false;`

7. Graph: $P1 \rightarrow \text{scanner}, \text{scanner} \rightarrow P2, P2 \rightarrow \text{printer}, \text{printer} \rightarrow P1$. That is a cycle, and each resource has exactly one instance. **Yes, it is a deadlock.**

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

PRACTICE · TIER 2 target: 25 min

1. Given Allocation, Max and Total below, compute Need and Available, then find a safe sequence or show none exists. Total = (6, 5, 7).

	Alloc A	B	C	Max A	B	C
X1	1	1	2	3	2	4
X2	2	1	1	4	2	3
X3	1	1	2	2	3	4

2. From that state, X1 requests (1, 0, 1). Grant or deny? Show all three checks.

3. A logging library takes lock L then lock M. A metrics library takes M then L. Both are called from the same request handler. Describe the failure and give two fixes.

4. Explain how a database engine handles deadlock, and why it does not use the Banker's algorithm.

ANSWER KEY

1. Need = Max - Alloc:

- X1: $(3 - 1, 2 - 1, 4 - 2) = (2, 1, 2)$
- X2: $(4 - 2, 2 - 1, 3 - 1) = (2, 1, 2)$
- X3: $(2 - 1, 3 - 1, 4 - 2) = (1, 2, 2)$

Allocated totals: $A = 1 + 2 + 1 = 4$; $B = 1 + 1 + 1 = 3$; $C = 2 + 1 + 2 = 5$. Available = $(6 - 4, 5 - 3, 7 - 5) = (2, 2, 2)$.

Safety check, Work = (2, 2, 2):

- X1: Need $(2, 1, 2) \leq (2, 2, 2)$ ✓ → finishes, releases $(1, 1, 2) \rightarrow \text{Work} = (3, 3, 4)$.
- X2: Need $(2, 1, 2) \leq (3, 3, 4)$ ✓ → finishes, releases $(2, 1, 1) \rightarrow \text{Work} = (5, 4, 5)$.
- X3: Need $(1, 2, 2) \leq (5, 4, 5)$ ✓ → finishes, releases $(1, 1, 2) \rightarrow \text{Work} = (6, 5, 7) = \text{Total}$ ✓

Safe. Sequence X1 → X2 → X3. (X3 → X1 → X2 also works — any valid sequence scores.)

2. Request (1, 0, 1) from X1.

- Check 1: Need[X1] = (2, 1, 2). $1 \leq 2, 0 \leq 1, 1 \leq 2$ ✓.
- Check 2: Available = (2, 2, 2). $1 \leq 2, 0 \leq 2, 1 \leq 2$ ✓.
- Check 3: pretend. Available = (1, 2, 1); Alloc[X1] = (2, 1, 3); Need[X1] = (1, 1, 1).
 - X1: $(1, 1, 1) \leq (1, 2, 1)$ ✓ → releases (2, 1, 3) → Work = (3, 3, 4).
 - X2: $(2, 1, 2) \leq (3, 3, 4)$ ✓ → releases (2, 1, 1) → Work = (5, 4, 5).
 - X3: $(1, 2, 2) \leq (5, 4, 5)$ ✓ → releases (1, 1, 2) → Work = (6, 5, 7) ✓

GRANT. Safe sequence X1 → X2 → X3.

3. Two threads in the same handler can take **L** and **M** in opposite orders. Thread 1 holds **L** and waits for **M**; thread 2 holds **M** and waits for **L**. Circular wait, and the handler hangs — usually under load, when the two libraries are called close together.

Fix 1: **global lock ordering**. Rank every lock in the process and require increasing order; wrap both libraries in a helper that acquires in rank order. Cost: you must know every lock in every dependency, which is hard with third-party code.

Fix 2: **lock with a timeout** (`try_lock` with a deadline). On timeout, release everything you hold, back off a **random** interval, and retry. Cost: this converts a hang into a retry storm unless the back-off is randomised, and it turns deadlock into possible livelock.

(A third, best answer: do not hold two library locks at once — copy what you need out from under the first lock before taking the second.)

4. A database engine uses **detection plus recovery**. It maintains a wait-for graph of transactions and periodically looks for a cycle (or simply uses a lock-wait timeout). When it finds one, it picks a **victim** — usually the transaction that has done the least work or holds the fewest locks — aborts it, rolls it back, and returns a “deadlock detected” error so the application can retry.

It cannot use the Banker’s algorithm because the Banker’s algorithm requires each process to declare its **maximum** resource claim in advance. A transaction does not know in advance which rows it will lock; that depends on the data it reads. Without a Max matrix, avoidance is impossible, so detection is the only option left.

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

PRACTICE · TIER 3 target: 30 min

1. Prove that a safe state can never be a deadlocked state, and give an example of an **unsafe** state that does not deadlock.
2. Design the locking for a transfer function `transfer(from, to, amount)` used by thousands of threads over a million accounts. It must be deadlock-free and must not serialise all transfers behind one global lock. State the failure mode your design still has.
3. A service uses a bounded work queue with `empty/full/mutex` semaphores. Under load, CPU is 8% and throughput has collapsed, but nothing is deadlocked. Give three possible causes and how you would tell them apart.
4. Why does the Banker’s algorithm see almost no use in real operating systems? Give three separate reasons.

ANSWER KEY

1. Safe means there is an ordering $P_1, P_2, \dots, P_n$ in which each P_i can obtain its Need from Available plus everything released by $P_1 \dots P_{i-1}$. Deadlock means a set of processes exists where none can ever proceed. If the state is safe, P_1 can proceed right now with what is available, so at least one process is not blocked — so no process is in a permanent wait cycle. A safe state therefore cannot be deadlocked.

Unsafe that does not deadlock: three processes each hold 2 units of a 7-unit pool and each **may** eventually need 4 more, so Available = 1 and no single process is guaranteed to finish — the state is unsafe. But if one of them happens to never request its full maximum and instead finishes and releases its 2 units, everything resolves. Unsafe means “no guarantee”, not “certain failure”.

2. Per-account locks, acquired in a **global order** derived from the account id.

`transfer(from, to, amount):`

`if from.id == to.id: return`

`first, second = (from, to) if from.id < to.id else (to, from)`

`lock(first); lock(second)`

`try: if from.balance >= amount: from -= amount; to += amount`

`finally: unlock(second); unlock(first)`

- Circular wait is impossible: every thread takes the lower id first, so a cycle would need some thread to go from a higher id to a lower one.
- Contention stays low: two transfers touching four different accounts never meet.
- The `from.id == to.id` guard matters — without it a self-transfer locks the same mutex twice and self-deadlocks.

Remaining failure mode: **hot accounts**. If one account (a merchant, an exchange float) is in half of all transfers, its lock serialises those transfers, and the per-account scheme buys nothing for them. The fix is to shard that one account's balance into k sub-balances and pick one at random, summing them when a total is needed — which trades exact instantaneous balance readability for throughput. Say that trade out loud.

3. Low CPU plus low throughput means threads are **blocked**, not busy.

- **(a) The queue is empty** — producers are the bottleneck. Check: the `full` semaphore sits at 0 and consumers are all parked in `wait(full)`. Look upstream.
- **(b) The queue is full** — consumers are the bottleneck, and `empty` sits at 0 with producers parked. Check whether consumers are blocked on something downstream (a slow database, an exhausted connection pool). Symptom: queue depth pinned at N .
- **(c) Convoying on the mutex** — a consumer is holding `mutex` across a slow operation (logging, I/O, a JSON parse), so everyone queues behind it. Check: `mutex` is held far longer than it should be; thread dumps show many threads waiting on the same lock and one thread inside it doing something slow.

How to tell them apart in one measurement: **sample the queue depth**. Pinned at 0 $\rightarrow$ (a). Pinned at $N \rightarrow$ (b). Oscillating with threads stacked on `mutex` $\rightarrow$ (c).

4. Three reasons:

- **It needs a Max declaration.** A process must state its maximum need for every resource type before it starts. Real programs do not know this — memory use depends on input, file handles depend on user actions.
- **It assumes a fixed resource count.** Devices are plugged in and removed, memory is added to a VM, processes are created and destroyed constantly. The Banker's model assumes a static set of processes and resources.
- **It is too slow and too conservative.** The safety check is $O(n^2m)$ for n processes and m resource types, and it must run on **every single request**. Worse, it refuses requests it could safely have granted, so utilisation drops — for a problem that, on a desktop or server, happens rarely enough that a reboot is cheaper.

Where the idea does survive: systems with a small, fixed, known set of tasks — avionics, industrial controllers — which is exactly the environment the algorithm was designed for.

Rapid fire — one-line answers

Question	Answer
What is a race condition?	Shared data accessed concurrently, where the result depends on the order of execution.
What is a critical section?	The part of a program that accesses shared data and must not run concurrently with itself.
Three requirements of a CS solution?	Mutual exclusion, progress, bounded waiting.
The key line in Peterson's?	<code>turn = j</code> — you give the turn to the other process.
Test-and-set vs compare-and-swap?	Both are atomic. Test-and-set sets a flag to true and returns the old value; CAS writes only if the current value matches the expected one.
Spinlock vs blocking lock?	Spinlock busy-waits (good for very short sections); a blocking lock sleeps.
What does a negative semaphore value mean?	Its absolute value is the number of blocked processes.
Mutex vs semaphore in one line?	A mutex has an owner and is for locking; a semaphore is a counter with no owner and is also for signalling.
In producer-consumer, what does <code>empty</code> count?	Free slots. <code>full</code> counts items.
What breaks if you take <code>mutex</code> before <code>empty</code> ?	Deadlock — the producer sleeps holding the mutex.
Who starves in the first readers-writers solution?	Writers.
Dining philosophers: the simplest fix?	Make one philosopher pick up the right fork first (asymmetry).
Four necessary conditions for deadlock?	Mutual exclusion, hold and wait, no preemption, circular wait.
Are the four conditions sufficient?	No — necessary only. All four can hold without a deadlock occurring.

When does a RAG cycle prove deadlock?	Only when every resource type in the cycle has a single instance.
Four strategies for handling deadlock?	Prevention, avoidance, detection and recovery, ignore it.
What does Linux do about deadlock?	Nothing — the ostrich algorithm.
What does the Banker's algorithm need in advance?	Each process's maximum claim for every resource type.
Need formula?	$\text{Need} = \text{Max} - \text{Allocation}$.
Available formula?	$\text{Available} = \text{Total} - \sum \text{Allocation}$.
What is a safe state?	One where some ordering lets every process finish.
Is every unsafe state a deadlock?	No — unsafe means no guarantee, not certain failure.
What is a monitor?	Shared data plus its procedures, with automatic mutual exclusion — only one thread inside at a time.
Condition variable vs semaphore?	A semaphore remembers a signal as a count; a condition variable throws it away if nobody is waiting.
Why <code>while</code> and not <code>if</code> around <code>wait()</code> ?	Mesa semantics — the condition can be false again by the time you wake up.
Deadlock vs livelock?	In deadlock threads are blocked; in livelock they keep running and keep undoing each other.
Deadlock vs starvation?	Deadlock: nobody progresses. Starvation: everybody but one progresses.
What is priority inversion?	A low-priority thread holding a lock blocks a high-priority one, and a medium one preempts the low.
Fix for priority inversion?	Priority inheritance (or a priority ceiling).
How do databases handle deadlock?	Detect a cycle in the wait-for graph, abort a victim transaction, let the client retry.

ONE-PAGE REVISION CARD

Race condition = shared data + concurrent access + order-dependent result. Fix by making the read-modify-write atomic.

Critical section needs all three: mutual exclusion, **progress**, bounded waiting. Progress is the one broken solutions fail.

Peterson's: `flag[i] = true; turn = j; while (flag[j] && turn == j) ; → CS → flag[i] = false;`

Semaphore: wait decrements and may block; signal increments and wakes one. A negative value means that many waiters. **Mutex vs semaphore**: mutex has an owner and is a lock; semaphore has no owner and is a counter and a signal.

Producer-consumer order: `wait(empty); wait(mutex); ... signal(mutex); signal(full);`
Reverse the two waits and you deadlock.

Readers-writers: reader preference starves writers; writer preference starves readers; a fair queue starves nobody.

Dining philosophers fixes: at most $N - 1$ at the table · both-or-neither · asymmetry · global fork ordering.

Monitor = data + procedures + automatic mutual exclusion. Condition variables inside it: `wait` sleeps and releases, `signal` wakes one and is **lost** if nobody waits. Everything real uses Mesa semantics, so the predicate goes in a while loop, never an if.

Deadlock — four necessary conditions: mutual exclusion, hold and wait, no preemption, circular wait. Necessary, **not** sufficient. Circular wait is the one you attack.

RAG: cycle $\Leftrightarrow$ deadlock only for single-instance resources.

Banker's: $\text{Need} = \text{Max} - \text{Alloc}$; $\text{Available} = \text{Total} - \sum \text{Alloc}$; then repeatedly find a process with $\text{Need} \leq \text{Work}$ and add its Allocation to Work. Three checks on a request: within Need? within Available? still safe after a pretend grant?

The worked state to remember the shape of: Total (8, 6, 9), Available (1, 2, 2), safe sequence $T3 \rightarrow T4 \rightarrow T1 \rightarrow T2$. $T2$ asking for (1, 1, 1) makes it unsafe — deny. $T3$ asking for (0, 2, 2) keeps it safe — grant.

Three lookalikes. Deadlock: blocked cycle. Livelock: busy, no progress. Starvation: others progress, you do not.

Priority inversion: low holds the lock, high blocks on it, medium preempts low. Fix with priority inheritance — or by not sharing mutable state across priority levels.

OS: Memory Management

How one small RAM pretends to be many big ones

The one idea

THE WHOLE CHAPTER IN ONE BOX

Your program says “give me byte 5000”. RAM has no byte 5000 reserved for you. Something in the middle must **translate** that request into a real RAM address, and must do it for every running program at the same time, without letting any program read another’s data.

That translator is **memory management**. Three jobs, always:

1. **Translate** — turn a logical (virtual) address into a physical address.
2. **Protect** — stop process A from touching process B’s bytes.
3. **Stretch** — let the total memory all programs think they have be much larger than the RAM you actually bought.

Definitions you must be able to say without pausing:

- **Logical / virtual address** — the address the CPU produces while running your program.
- **Physical address** — the address that goes on the memory bus to the RAM chip.
- **MMU (Memory Management Unit)** — the hardware that converts one to the other.
- **Page** — a fixed-size block of the logical address space (typically 4 KB).
- **Frame** — a fixed-size block of physical memory, the **same size** as a page.
- **Page table** — the per-process array that says “page i lives in frame f ”.
- **Page fault** — the page you asked for is not in RAM right now.

The arithmetic everything rests on. If page size is 2^d bytes:

$$\text{page number} = \left\lfloor \frac{\text{logical address}}{2^d} \right\rfloor, \quad \text{offset} = \text{logical address} \bmod 2^d$$

$$\text{physical address} = \text{frame number} \times 2^d + \text{offset}$$

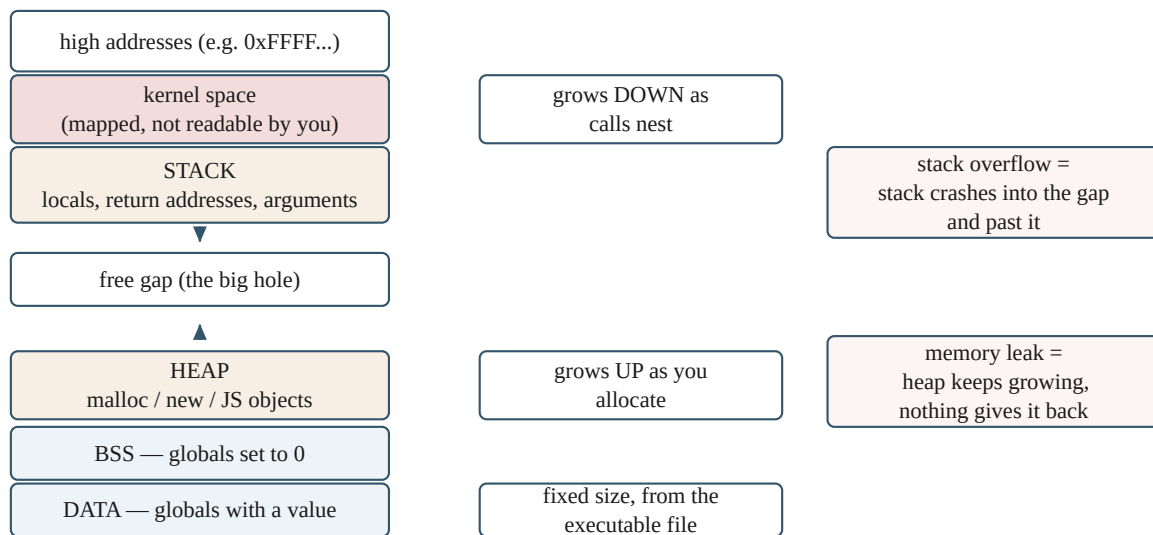
In binary this is free: the low d bits **are** the offset, the rest **are** the page number. Nothing is added or shifted for the offset — it is copied through untouched.

SHORTCUT

Offset bits = $\log_2(\text{page size})$. Page-number bits = (address bits) — (offset bits). Number of entries in a flat page table = $2^{\text{page-number bits}}$. Three facts, and half the numerical questions in this chapter fall out.

Where a running program’s bytes live

Before translation, know what is being translated. Every process gets one flat virtual address space, carved into regions.



The virtual address space of one process. Heap grows up, stack grows down; the gap in the middle is why one process can look huge and use little.

TEXT (the machine code) sits at the very bottom, below DATA, and is marked read-only and executable. That is why writing through a bad pointer into code space gives a segmentation fault instead of silently rewriting your program.

WARM-UP

build the reflex

Example 1 [warm-up]

A process's virtual address space is 4 GB but the machine has 8 GB of RAM and 40 processes are running. Total virtual memory asked for is 160 GB. Why does nothing break?

SOLUTION

Because almost none of those 4 GB are **used**. The address space is a promise, not a reservation. Only pages that are actually touched get a physical frame. The rest of the space is the “free gap” in the picture above — no frame, no page-table entry, no cost.

Answer: Virtual space is address range, not allocated RAM. Frames are handed out only on first touch.

Example 2 [warm-up]

Which of these is produced by the CPU while executing an instruction: logical address or physical address?

SOLUTION

The CPU produces a **logical** address. The MMU sits between the CPU and the bus and turns it into a physical one.

Answer: Logical (virtual). Physical addresses only exist after the MMU.

Before paging: contiguous allocation

The oldest scheme. Each process gets one continuous block of RAM. The hardware needs only two registers.

BASE AND LIMIT

- **Base register** — where this process's block starts in physical memory.
- **Limit register** — how long the block is.

On every access the hardware checks:

$$0 \leq \text{logical address} < \text{limit}$$

If it passes: physical = base + logical. If it fails: trap to the OS (segmentation fault). Two comparisons, done in hardware, on every single memory access.

Choosing which hole to use

Free memory becomes a list of **holes**. When a process of size n arrives, which hole do we cut it from?

Strategy	Rule	Character
First fit	Scan from the start, take the first hole that is big enough.	Fastest to run. Good enough in practice.
Best fit	Take the smallest hole that fits.	Must scan the whole list. Leaves a litter of tiny useless slivers.
Worst fit	Take the largest hole that fits.	Must scan the whole list. Destroys your big holes first. Usually worst overall.
Next fit	Like first fit, but resume scanning from where you stopped last time.	Spreads use around; slightly better than first fit on long runs.

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 3 [TCS NQT · pattern]

Free holes, in memory order, are 120K, 60K, 300K, 180K, 90K. Requests arrive in this order: 210K, 55K, 290K, 130K. Show what first fit, best fit and worst fit each do.

SOLUTION

Work request by request. A request that fits nowhere waits.

First fit — scan left to right, stop at the first hole that is big enough.

Request	Goes into	Holes after
210K	hole 3 (300K) — 120 and 60 are too small	120, 60, 90 , 180, 90
55K	hole 1 (120K) — first one big enough	65 , 60, 90, 180, 90
290K	nothing fits — must wait	65, 60, 90, 180, 90
130K	hole 4 (180K)	65, 60, 90, 50 , 90

Best fit — smallest hole that fits.

Request	Goes into	Holes after
210K	hole 3 (300K) — only one that fits	120, 60, 90 , 180, 90
55K	hole 2 (60K) — tightest fit	120, 5 , 90, 180, 90
290K	nothing fits — wait	120, 5, 90, 180, 90
130K	hole 4 (180K)	120, 5, 90, 50 , 90

Worst fit — largest hole that fits.

Request	Goes into	Holes after
210K	hole 3 (300K)	120, 60, 90 , 180, 90
55K	hole 4 (180K) — biggest	120, 60, 90, 125 , 90
290K	nothing fits — wait	120, 60, 90, 125, 90

130K	nothing fits — wait	120, 60, 90, 125, 90
------	---------------------	----------------------

Score: first fit and best fit each placed 3 of 4 requests. Worst fit placed only 2, because it spent its 180K hole on a 55K request.

Answer: First fit: holes 65, 60, 90, 50, 90 · Best fit: 120, 5, 90, 50, 90 · Worst fit: 120, 60, 90, 125, 90 with two requests still waiting.

fits.js — run it and change the numbers yourself

JS

```
function allocate(holes, requests, strategy) {
  const h = [...holes];
  const log = [];
  for (const r of requests) {
    let pick = -1;
    for (let i = 0; i < h.length; i++) {
      if (h[i] < r) continue;
      if (pick === -1) { pick = i; if (strategy === 'first') break; continue; }
      if (strategy === 'best' && h[i] < h[pick]) pick = i;
      if (strategy === 'worst' && h[i] > h[pick]) pick = i;
    }
    if (pick === -1) { log.push(`${r}K -> WAIT (no hole fits)`); continue; }
    log.push(`${r}K -> hole ${pick + 1} (was ${h[pick]}K, leftover ${h[pick] - r}K)`);
    h[pick] -= r;
  }
  return { log, holes: h };
}

const holes = [120, 60, 300, 180, 90];
const reqs = [210, 55, 290, 130];
for (const s of ['first', 'best', 'worst']) {
  const r = allocate(holes, reqs, s);
  console.log(`--- ${s} fit ---`);
  r.log.forEach(l => console.log(' ' + l));
  console.log(' holes now', r.holes.join(','));
}
```

Output (verified with node fits.js): worst fit ends with holes 120,60,90,125,90 and leaves both 290K and 130K waiting. First and best fit both end with 355K total free, worst fit with 485K free — and yet worst fit served **fewer** requests. More free bytes is not the same as more usable free bytes.

The two fragmentations

INTERNAL VS EXTERNAL FRAGMENTATION

- **External fragmentation** — enough total free memory exists, but it is split into scattered holes, none big enough. Belongs to **contiguous** allocation and to **segmentation**.
- **Internal fragmentation** — the block you were given is bigger than what you asked for, and the unused tail inside it is wasted. Belongs to **paging** and to fixed-size blocks.

Average internal fragmentation with paging $\approx$ half a page per process (the last page is on average half empty).

Cure for external fragmentation: **compaction** (slide everything together) — only possible if binding is dynamic, and it is expensive. The real cure is paging: stop requiring contiguity at all.

TRAP

“Paging removes fragmentation.” **Wrong.** Paging removes **external** fragmentation. It **creates** internal fragmentation, because the last page of a process is almost never full. Say exactly which one you mean, every time.

Example 4 [Infosys · pattern]

Page size is 4 KB. A process needs 19,430 bytes. How many pages? How much internal fragmentation?

SOLUTION

Step 1 — pages needed:

$$\frac{19430}{4096} = 4.744... \rightarrow \text{round up} \rightarrow 5 \text{ pages}$$

Step 2 — memory actually given:

$$5 \times 4096 = 20480 \text{ bytes}$$

Step 3 — waste:

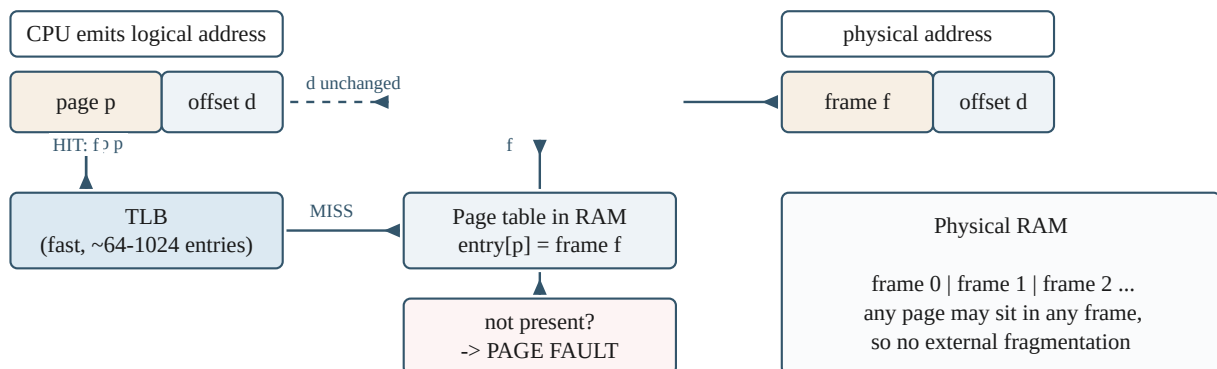
$$20480 - 19430 = 1050 \text{ bytes}$$

Answer: 5 pages; 1050 bytes of internal fragmentation.

The waste is always in the **last** page only, and is always less than one page size.

Paging — the real scheme

Break the logical address space into fixed-size **pages**. Break physical memory into same-size **frames**. Any page can go into any free frame. Contiguity in virtual space no longer means contiguity in physical space.



Paging. The offset is copied straight through; only the page number is looked up. The TLB is a small cache in front of the page table.

Reading a page-table entry (PTE)

A PTE is not just a frame number. Know the bits:

Field	What it is for
Frame number	Where the page actually is in RAM.
Valid / present bit	0 means “not in RAM” -> page fault on access.
Protection bits (R/W/X)	Write to a read-only page -> trap. This is how <code>const</code> pages and code pages are enforced.
Dirty (modified) bit	Set by hardware on a write. If 0 at eviction time, the frame can be dropped without writing to disk.

Referenced (accessed) bit	Set by hardware on any access. The OS clears it periodically; this is the raw material for LRU approximations.
User/supervisor bit	Whether user-mode code may touch this page.
Caching bits	Mark memory-mapped device pages as uncacheable.

SHORTCUT

The **dirty bit** is the cheapest optimisation in the whole chapter. Evicting a clean page costs 0 disk writes. Evicting a dirty page costs 1. So modern replacement policies prefer clean victims when quality is otherwise equal.

Example 5 [Wipro · pattern]

Logical addresses are 16 bits. Page size is 1 KB. (a) How many bits of offset? (b) How many pages can a process have? (c) Logical address 5000 with page 4 mapped to frame 9 — what is the physical address?

SOLUTION

(a) $1\text{KB} = 1024 = 2^{10}$, so the offset needs **10 bits**.

(b) Page-number bits = $16 - 10 = 6$, so $2^6 = 64$ **pages**.

(c) Split 5000:

$$\text{page} = \left\lfloor \frac{5000}{1024} \right\rfloor = 4, \quad \text{offset} = 5000 - 4 \times 1024 = 5000 - 4096 = 904$$

Page 4 is in frame 9:

$$\text{physical} = 9 \times 1024 + 904 = 9216 + 904 = 10120$$

Answer: (a) 10 bits (b) 64 pages (c) physical address 10120.

translate.js — the MMU in nine lines

JS

```
const OFFSET_BITS = 10;
const PAGE_SIZE = 1 << OFFSET_BITS;           // 1024 bytes
// page -> frame. 16 valid pages; anything else is not mapped.
const pageTable = [5, 9, 1, 7, 9, 2, 0, 3, 11, 6, 8, 4, 13, 2, 10, 12];

function translate(logical) {
  const page = logical >> OFFSET_BITS;         // same as Math.floor(logical/1024)
  const offset = logical & (PAGE_SIZE - 1);    // same as logical % 1024
  const frame = pageTable[page];
  if (frame === undefined) return { page, offset, fault: true };
  return { page, offset, frame, physical: frame * PAGE_SIZE + offset };
}
console.log(translate(5000));
console.log(translate(13580));
console.log(translate(70000));
```

Real output from node translate.js:

```
{ page: 4, offset: 904, frame: 9, physical: 10120 }
{ page: 13, offset: 268, frame: 2, physical: 2316 }
{ page: 68, offset: 368, fault: true }
```

Notice the shift and the mask. $\gg 10$ and $\& 1023$ are exactly what the wires in an MMU do — the page number and the offset are just two slices of the same bit string.

Example 6 [Capgemini · pattern]

Virtual addresses are 32 bits, page size 4 KB, and each page-table entry is 4 bytes. How big is one process's flat (single-level) page table?

SOLUTION

Step 1 — offset bits: $4\text{KB} = 2^{12}$, so 12 bits.

Step 2 — number of pages: $2^{32-12} = 2^{20} = 1,048,576$ pages, so the table needs 2^{20} entries.

Step 3 — size: $2^{20} \times 4 \text{ bytes} = 4 \text{ MB}$.

Answer: 4 MB per process — and that is per process, so 100 processes cost 400 MB of page tables alone.

That number is the whole reason multi-level page tables exist.

TLB: making translation not cost double

Without help, every memory access becomes two: one to read the page table, one to read the data. The TLB (Translation Lookaside Buffer) is a tiny fully-associative cache of recent page-to-frame mappings, inside the MMU.

EFFECTIVE ACCESS TIME WITH A TLB

Let α = TLB hit ratio, t = TLB lookup time, m = one main-memory access time.

$$\text{EAT} = \alpha(t + m) + (1 - \alpha)(t + m + m)$$

Read it as: **always** pay the TLB lookup. On a hit, one memory access (the data). On a miss, two (the page table, then the data).

For an L -level page table the miss branch costs $L \times m$ table accesses plus m for the data:

$$\text{EAT} = \alpha(t + m) + (1 - \alpha)(t + Lm + m)$$

Example 7 [Accenture · pattern]

TLB lookup 20 ns, main memory access 100 ns, TLB hit ratio 80%, single-level page table. Find the effective access time and the percentage slowdown versus raw memory.

SOLUTION

Hit branch (80%): 20 ns TLB + 100 ns data = 120 ns.

Miss branch (20%): 20 ns TLB + 100 ns page table + 100 ns data = 220 ns.

$$\text{EAT} = 0.8 \times 120 + 0.2 \times 220 = 96 + 44 = 140 \text{ ns}$$

Raw memory with no translation at all would be 100 ns.

$$\text{slowdown} = \frac{140 - 100}{100} = 40\%$$

Answer: EAT = 140 ns, a 40% slowdown over raw memory.

Compare: with **no** TLB, every access costs $100 + 100 = 200$ ns, a 100% slowdown. The TLB cut the penalty from 100% to 40%.

Example 8 [Grab · pattern]

A 64-bit server uses a **three-level** page table. TLB lookup is 10 ns, memory access 100 ns, TLB hit ratio 98%. Find EAT. Then explain why the hit ratio matters far more here than in Example 7.

SOLUTION

Hit branch (98%): $10 + 100 = 110$ ns.

Miss branch (2%): $10 + 3 \times 100$ (three table levels) + 100 (data) = 410 ns.

$$\text{EAT} = 0.98 \times 110 + 0.02 \times 410 = 107.8 + 8.2 = 116 \text{ ns}$$

Why the hit ratio matters more: the miss is now 410 ns against a 110 ns hit — a penalty of 300 ns per miss instead of 100 ns. Drop the hit ratio to 90% and EAT becomes $0.9 \times 110 + 0.1 \times 410 = 99 + 41 = 140$ ns, a 21% jump from one 8-point drop in hit ratio.

Answer: EAT = 116 ns. Deeper page tables make every TLB miss much more expensive, so hit ratio dominates.

This is why real systems use **huge pages** (2 MB or 1 GB instead of 4 KB) for large databases: one TLB entry then covers 512 times more memory, so the hit ratio climbs.

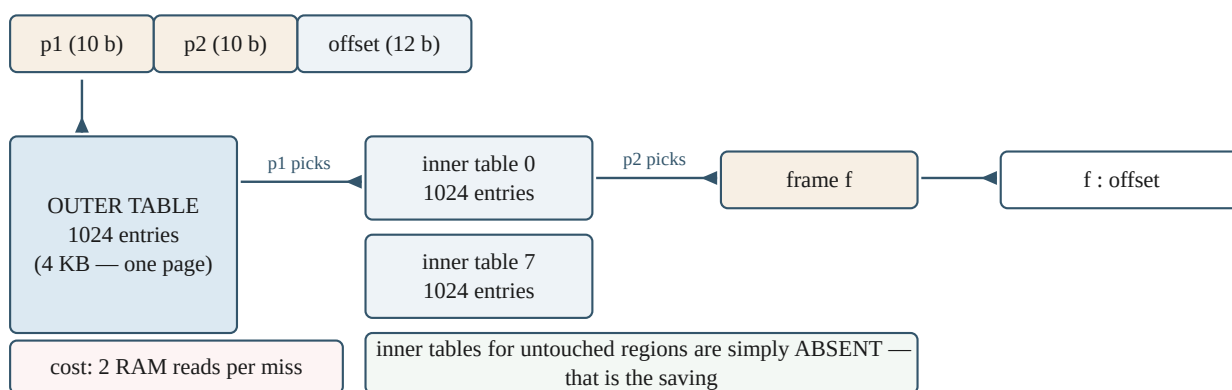
TRAP

“A context switch flushes the TLB.” Half true, and the half matters. On old hardware, yes — which made context switches expensive. Modern CPUs tag each TLB entry with an **address space identifier** (ASID / PCID), so entries from different processes coexist and no flush is needed. If you are asked, say both: “classically a flush; modern CPUs use ASIDs to avoid it.”

Making page tables affordable

Multi-level (hierarchical) paging

Split the page number itself into pieces. The outer table indexes inner tables; inner tables hold frames. Pages of the address space that are never touched need **no inner table at all** — and that is where the saving comes from.



Two-level paging on a 32-bit address with 4 KB pages: 10 outer bits, 10 inner bits, 12 offset bits.

Example 9 [Shopee · pattern]

32-bit addresses, 4 KB pages, 4-byte entries, two-level paging with a 10/10/12 split. A process actually uses only 12 MB: 8 MB of code+data at the bottom of the address space and 4 MB of stack at the top. Compare the page-table memory used by the flat scheme and the two-level scheme.

SOLUTION

Flat scheme — always 2^{20} entries $\times$ 4 B = 4 MB, whether you use 12 MB or 4 GB.

Two-level scheme — count what is actually needed.

Step 1 — outer table: 1024 entries $\times$ 4 B = 4096 B = 4 KB. Always present.

Step 2 — how much does one inner table cover?

$1024 \text{ entries} \times 4 \text{ KB per page} = 4 \text{ MB of address space per inner table}$

Step 3 — inner tables needed:

- 8 MB of code+data = 2 inner tables.
- 4 MB of stack = 1 inner table.
- Total 3 inner tables $\times$ 4 KB each = 12 KB.

Step 4 — total: 4 KB + 12 KB = 16 KB.

Answer: Flat: 4 MB. Two-level: 16 KB — 256 times smaller. The price is one extra memory read on every TLB miss.

This is the trade-off sentence the interviewer wants: **hierarchical paging trades time (more levels to walk) for space (unused regions cost nothing).**

Inverted page tables

Flip the direction. Instead of one table per process indexed by page, keep **one** table for the whole machine, with one entry per **physical frame**, holding (pid, page number).

Good	Bad
Size depends on RAM, not on the number of processes or the width of the address space. One table, period.	You cannot index it by page number — you must search it. Fixed with a hash table, which adds another memory access.
A 64-bit address space costs nothing extra.	Sharing memory between processes is awkward: one frame, but it needs to appear under two different (pid, page) keys.

Segmentation

Paging splits memory into equal blocks that mean nothing to the programmer. Segmentation splits it into blocks that **do** mean something: code, globals, heap, stack, one array. Segments have **different sizes**.

SEGMENTATION ARITHMETIC

A logical address is a pair: $(s, d) = (\text{segment number}, \text{offset within segment})$.

The segment table holds, for each s , a **base** and a **limit**.

if $d < \text{limit}[s]$: $\text{physical} = \text{base}[s] + d$
else: trap (segmentation fault)

Note the difference from paging: here the offset **is added**. In paging it is concatenated. That is because segments have arbitrary sizes and arbitrary bases; pages and frames are aligned to a power of two.

Example 10 [Cognizant · pattern]

Segment table:

Segment	Base	Limit
0	1400	600
1	6300	250
2	4300	1100
3	3200	580

Translate (0, 430), (2, 88), (3, 579) and (1, 250).

SOLUTION

(0, 430): is $430 < 600$? Yes. $1400 + 430 = 1830$.

(2, 88): is $88 < 1100$? Yes. $4300 + 88 = 4388$.

(3, 579): is $579 < 580$? Yes, by one. $3200 + 579 = 3779$.

(1, 250): is $250 < 250$? **No** — the test is strictly less than. Offsets run 0 to 249. Trap.

Answer: 1830, 4388, 3779, and a trap (segmentation fault) for (1, 250).

TRAP

The limit check is $d < \text{limit}$, never $d \leq \text{limit}$. Offset 250 in a 250-byte segment is the 251st byte. Exam writers love this off-by-one.

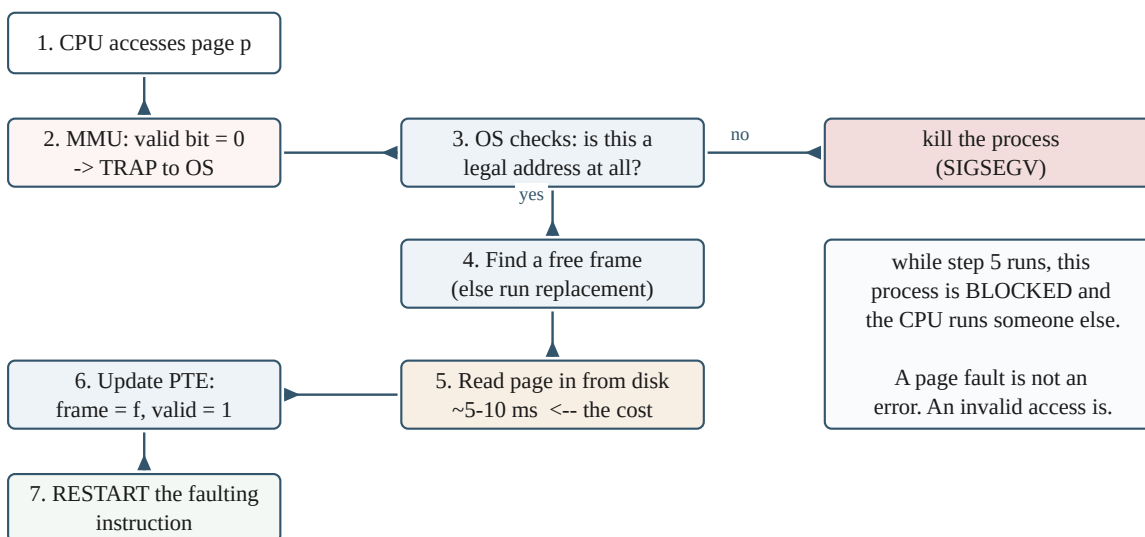
Paging vs segmentation — the comparison table

	Paging	Segmentation
Block size	Fixed (e.g. 4 KB)	Variable — as big as the logical unit needs
Chosen by	The OS / hardware	The programmer or compiler
Address form	One number, split into page + offset	A visible pair (segment, offset)
Offset combined by	Concatenation (no adder needed)	Addition to the base
External fragmentation	None	Yes — variable sizes leave holes
Internal fragmentation	Yes — last page half empty	None (segment fits exactly)
Protection granularity	Per page — meaningless units	Per segment — “this whole array is read-only” is natural
Sharing	Awkward: share page by page	Natural: share a whole code segment

Segmented paging (what x86 historically did, and what “segment + page table” means): first use the segment number to find a **per-segment page table**, then page within it. You get segmentation’s meaningful units and protection, plus paging’s freedom from external fragmentation. Modern 64-bit operating systems mostly ignore segmentation and use flat paging only.

Virtual memory and demand paging

Now the stretch. Do not load a page until it is touched. Keep the rest on disk in the **swap space** (or in the executable file itself).



What happens on a page fault. Steps 3 and 5 are the expensive ones — disk I/O, measured in milliseconds.

TRAP

“Page fault means the program crashed.” **No.** A page fault is a normal, planned event — every program takes thousands of them while starting up. The crash case is a **segmentation fault**: an access to an address with no legal mapping at all. Different thing, step 3 of the diagram.

EFFECTIVE ACCESS TIME UNDER DEMAND PAGING

Let p = page-fault rate ($0 \leq p \leq 1$), m = memory access time, F = page-fault service time (dominated by disk I/O).

$$\text{EAT} = (1 - p) \times m + p \times F$$

Because F is measured in milliseconds and m in nanoseconds, F is roughly **40,000 times** larger than m . So even a tiny p dominates the sum.

Example 11 [Agoda · pattern]

Memory access is 200 ns. A page fault takes 8 ms to service. (a) Find EAT when 1 access in 1000 faults. (b) What fault rate keeps the slowdown under 10%?

SOLUTION

First put both times in the same unit: 8 ms = 8,000,000 ns.

(a) $p = 0.001$.

$$\begin{aligned}\text{EAT} &= (1 - 0.001) \times 200 + 0.001 \times 8,000,000 \\ &= 199.8 + 8000 = 8199.8 \text{ ns}\end{aligned}$$

Slowdown = $\frac{8199.8}{200} = 41.0$ times. One fault per thousand accesses makes the machine **41 times slower**.

(b) “Under 10% slowdown” means $\text{EAT} \leq 220$ ns.

$$\begin{aligned}(1 - p) \times 200 + p \times 8,000,000 &\leq 220 \\ 200 - 200p + 8,000,000p &\leq 220 \\ 7,999,800p &\leq 20 \\ p &\leq \frac{20}{7,999,800} = 2.5 \times 10^{-6}\end{aligned}$$

That is about **1 fault in 400,000 accesses**.

Answer: (a) EAT = 8199.8 ns, a 41x slowdown. (b) $p < 2.5 \times 10^{-6}$, i.e. fewer than 1 fault per 400,000 accesses.

The lesson to say out loud: demand paging only works because the fault rate in real programs is far below one in a million, thanks to locality.

The same arithmetic, tabulated. Memory 200 ns, fault 8 ms:

Fault rate p	EAT	Slowdown
0	200 ns	1.0x
10^{-6}	208 ns	1.04x
10^{-5}	280 ns	1.4x
10^{-4}	1000 ns	5.0x
10^{-3}	8200 ns	41x
10^{-2}	80,198 ns	401x

Each factor of 10 in p costs you roughly a factor of 8 in speed once p passes 10^{-5} .

Page replacement algorithms

RAM is full and a page must come in. Which page leaves? This is the most-asked numerical topic in the whole OS syllabus, so we will do it slowly and completely.

THE RULES FOR TRACING

1. A **reference string** is the sequence of page numbers touched.
 2. Every miss is a **page fault**. Count them.
 3. The first F distinct pages always fault (cold start) — these are **compulsory** faults.
 4. **FIFO**: evict the page that arrived earliest. A hit does **not** change its position.
 5. **LRU**: evict the page whose last **use** is oldest. A hit **does** refresh it.
 6. **Optimal (OPT / MIN)**: evict the page whose **next use** is farthest in the future; a page never used again is the best victim. Needs the future, so it is only a benchmark.
 7. **LFU**: evict the least frequently used. **MFU**: evict the most frequently used (argument: the page with the smallest count just arrived and is probably still needed).
 8. **Clock / second chance**: FIFO plus a reference bit. Give a page one pardon before evicting it.
- Always: faults + hits = length of the reference string. Use this to check yourself.

We will use one reference string throughout so the algorithms can be compared fairly:

5, 2, 1, 3, 4, 3, 2, 5, 3, 1, 2, 1 with 3 frames

FIFO, step by step

Example 12 [TCS Digital · pattern]

Trace FIFO on the string above with 3 frames. Count the faults.

SOLUTION

Write the frames as a queue: the **left** end is the oldest and the next victim.

#	Ref	Frames (old -> new)	Fault?	Evicted
1	5	5	F	—
2	2	5, 2	F	—
3	1	5, 2, 1	F	—
4	3	2, 1, 3	F	5
5	4	1, 3, 4	F	2
6	3	1, 3, 4	hit	—
7	2	3, 4, 2	F	1
8	5	4, 2, 5	F	3
9	3	2, 5, 3	F	4
10	1	5, 3, 1	F	2
11	2	3, 1, 2	F	5
12	1	3, 1, 2	hit	—

Count the F rows: 1, 2, 3, 4, 5, 7, 8, 9, 10, 11 — that is **10 faults** and 2 hits. Check: $10 + 2 = 12$ = length of the string. Correct.

Answer: FIFO with 3 frames: 10 page faults, 2 hits.

Look at row 6. Page 3 is used, but its place in the queue does not move. That is the whole weakness of FIFO — it measures **age**, not **usefulness**.

LRU, step by step

Example 13 [TCS Digital · pattern]

Same string, 3 frames, LRU.

SOLUTION

Now the list is ordered by **last use**: left = least recently used = next victim. A hit moves that page to the right end.

#	Ref	Frames (LRU -> MRU)	Fault?	Evicted
1	5	5	F	—
2	2	5, 2	F	—
3	1	5, 2, 1	F	—
4	3	2, 1, 3	F	5
5	4	1, 3, 4	F	2
6	3	1, 4, 3	hit	—
7	2	4, 3, 2	F	1
8	5	3, 2, 5	F	4
9	3	2, 5, 3	hit	—
10	1	5, 3, 1	F	2
11	2	3, 1, 2	F	5
12	1	3, 2, 1	hit	—

Faults: rows 1, 2, 3, 4, 5, 7, 8, 10, 11 = **9 faults**, 3 hits. Check $9 + 3 = 12$.

Answer: LRU with 3 frames: 9 page faults.

Compare row 6 with FIFO's row 6. FIFO left page 3 at the front of the queue and evicted it at step 8. LRU moved it to the back, so at step 9 it was still resident — one extra hit.

Optimal, step by step

Example 14 [Sea/Shopee · pattern]

Same string, 3 frames, Optimal. Show how you pick each victim.

SOLUTION

Index the string so you can look ahead:

pos	1	2	3	4	5	6	7	8	9	10	11	12
page	5	2	1	3	4	3	2	5	3	1	2	1

Steps 1-3: cold faults, frames = {5, 2, 1}.

Step 4, need 3. Next uses after position 4: page 5 at position 8; page 2 at position 7; page 1 at position 10. Farthest is page **1** (position 10). Evict 1. Frames {5, 2, 3}.

Step 5, need 4. Next uses after position 5: page 5 -> 8; page 2 -> 7; page 3 -> 6. Farthest is page **5** (position 8). Evict 5. Frames {4, 2, 3}.

Step 6, page 3 — hit. **Step 7, page 2** — hit.

Step 8, need 5. Next uses after 8: page 4 -> never again; page 2 -> 11; page 3 -> 9. A page never used again is the perfect victim. Evict **4**. Frames {5, 2, 3}.

Step 9, page 3 — hit.

Step 10, need 1. Next uses after 10: page 5 -> never; page 2 -> 11; page 3 -> never. Two candidates never used again; take either, say **5**. Frames {1, 2, 3}.

Steps 11 and 12 — page 2 hit, page 1 hit.

Faults at steps 1, 2, 3, 4, 5, 8, 10 = **7 faults**.

Answer: Optimal with 3 frames: 7 page faults — the lower bound no real algorithm can beat.

All three side by side

Frames	FIFO	LRU	Optimal
3	10	9	7
4	8	7	6

SHORTCUT

In an exam, compute **Optimal first**. It is the floor. If your FIFO or LRU number comes out **below** the optimal number, you have made an arithmetic mistake — go back. $\text{Optimal} \leq \text{LRU}$ and $\text{Optimal} \leq \text{FIFO}$, always.

replace.js — all three, so you can check any string yourself

JS

```
function fifo(ref, frames) {
  const mem = []; // queue: oldest first
  let faults = 0;
  for (const p of ref) {
    if (mem.includes(p)) continue;
    if (mem.length === frames) mem.shift();
    mem.push(p);
    faults++;
  }
  return faults;
}

function lru(ref, frames) {
  const mem = new Map(); // Map keeps insertion order -> first key = least recent
  let faults = 0;
  for (const p of ref) {
    if (mem.has(p)) { mem.delete(p); mem.set(p, true); continue; }
    if (mem.size === frames) mem.delete(mem.keys().next().value);
    mem.set(p, true);
    faults++;
  }
  return faults;
}

function opt(ref, frames) {
  const mem = [];
  let faults = 0;
  for (let i = 0; i < ref.length; i++) {
    const p = ref[i];
    if (mem.includes(p)) continue;
    if (mem.length === frames) {
      let victim = 0, best = -1;
      for (let j = 0; j < mem.length; j++) {
        let next = ref.indexOf(mem[j], i + 1);
        if (next === -1) next = Infinity; // never used again -> perfect victim
        if (next > best) { best = next; victim = j; }
      }
      mem[victim] = p;
    } else mem.push(p);
  }
}
```

```

    faults++;
  }
  return faults;
}

const ref = [5, 2, 1, 3, 4, 3, 2, 5, 3, 1, 2, 1];
for (const f of [3, 4])
  console.log(`frames=${f}  FIFO=${fifo(ref, f)}  LRU=${lru(ref, f)}  OPT=${opt(ref, f)}`);

```

Output from node replace.js:

```

frames=3  FIFO=10  LRU=9  OPT=7
frames=4  FIFO=8   LRU=7  OPT=6

```

matching the hand traces above. The `lru` function is worth memorising separately: **a JS Map gives you an LRU cache almost for free**, because `Map` preserves insertion order and `mem.keys().next().value` is the oldest key. This exact trick is a common coding-round question too.

Clock (second chance)

True LRU needs a timestamp update on every memory access — far too expensive in hardware. Clock is the practical approximation every real OS uses.

THE CLOCK ALGORITHM

Frames are arranged in a circle. A hand points at the next candidate. Each frame has a **reference bit R**, set to 1 by hardware whenever the page is accessed.

On a fault, repeat:

- If R of the frame under the hand is **0** -> evict this page, put the new page here with R = 0, advance the hand, stop.
- If R is **1** -> set R to 0 (that is the “second chance”), advance the hand, look again.

A page that keeps being used keeps having R reset to 1 before the hand comes round again, so it survives. A page nobody touches meets the hand with R = 0 and dies.

Enhanced clock uses the pair (R, D) where D is the dirty bit, and prefers victims in this order: (0,0) best, then (0,1), then (1,0), then (1,1) worst.

Example 15 [DBS · pattern]

Trace clock on 5, 2, 1, 3, 4, 3, 2, 5, 3, 1, 2, 1 with 3 frames. Start with the hand at frame 0 and all frames empty.

SOLUTION

Write frames as [f0 f1 f2], R bits under them, and mark where the hand stops.

#	Ref	Frames	R bits	Hand after	Result
1	5	5, –, –	0, 0, 0	1	F — empty frame
2	2	5, 2, –	0, 0, 0	2	F — empty frame
3	1	5, 2, 1	0, 0, 0	0	F — empty frame
4	3	3, 2, 1	0, 0, 0	1	F — f0 had R=0, evict 5
5	4	3, 4, 1	0, 0, 0	2	F — f1 had R=0, evict 2
6	3	3, 4, 1	1, 0, 0	2	hit — set R of 3
7	2	3, 4, 2	1, 0, 0	0	F — f2 had R=0, evict 1
8	5	3, 5, 2	0, 0, 0	2	F — f0 R=1, pardon it, f1 R=0, evict 4

9	3	3, 5, 2	1, 0, 0	2	hit
10	1	3, 5, 1	1, 0, 0	0	F — f2 R=0, evict 2
11	2	3, 2, 1	0, 0, 0	2	F — f0 R=1, pardon, f1 R=0, evict 5
12	1	3, 2, 1	0, 0, 1	2	hit

Look carefully at step 8. The hand is at frame 0, which holds page 3 with $R = 1$. That is the second chance: set R to 0, move on, do **not** evict. Frame 1 holds page 4 with $R = 0$, so page 4 dies.

Faults: steps 1, 2, 3, 4, 5, 7, 8, 10, 11 = **9 faults**.

Answer: Clock with 3 frames: 9 faults — exactly matching LRU here, at a fraction of LRU's hardware cost.

That match is the point of the algorithm. Clock is not always equal to LRU, but it is usually close, and it costs one bit per frame instead of a timestamp per frame.

Belady's anomaly

BELADY'S ANOMALY

More frames can cause MORE page faults. It happens with FIFO. It can never happen with LRU or Optimal, because those are **stack algorithms**: the set of pages held with n frames is always a subset of the set held with $n + 1$ frames, so adding a frame can only help.

Example 16 [Microsoft · pattern]

Find the fault counts for FIFO on 2, 5, 3, 6, 2, 5, 4, 2, 5, 3, 6, 4 with 3 frames and with 4 frames. Then do LRU on the same string with 3 and 4 frames. Explain what you see.

SOLUTION

FIFO, 3 frames:

#	Ref	Frames (old->new)	Result
1	2	2	F
2	5	2, 5	F
3	3	2, 5, 3	F
4	6	5, 3, 6	F, out 2
5	2	3, 6, 2	F, out 5
6	5	6, 2, 5	F, out 3
7	4	2, 5, 4	F, out 6
8	2	2, 5, 4	hit
9	5	2, 5, 4	hit
10	3	5, 4, 3	F, out 2
11	6	4, 3, 6	F, out 5
12	4	4, 3, 6	hit

Faults = **9**.

FIFO, 4 frames:

#	Ref	Frames (old->new)	Result
1	2	2	F

2	5	2, 5	F
3	3	2, 5, 3	F
4	6	2, 5, 3, 6	F
5	2	2, 5, 3, 6	hit
6	5	2, 5, 3, 6	hit
7	4	5, 3, 6, 4	F, out 2
8	2	3, 6, 4, 2	F, out 5
9	5	6, 4, 2, 5	F, out 3
10	3	4, 2, 5, 3	F, out 6
11	6	2, 5, 3, 6	F, out 4
12	4	5, 3, 6, 4	F, out 2

Faults = **10**.

More memory, more faults. That is Belady's anomaly.

LRU on the same string: 3 frames gives 10 faults, 4 frames gives 8 faults — it went **down**, as it must.

Answer: FIFO: 9 faults with 3 frames but 10 with 4 — Belady's anomaly. LRU: 10 with 3 frames, 8 with 4 — never anomalous, because LRU is a stack algorithm.

Why FIFO can do this. At step 7 with 4 frames, page 2 is evicted even though it was used at step 5 — FIFO only looks at arrival order, and with 4 frames page 2's arrival is still the oldest. With 3 frames, page 2 had already been **re-loaded** at step 5, resetting its arrival time, so it survived longer. Extra memory changed the arrival order in a way that hurt.

TRAP

“Belady's anomaly proves FIFO is broken.” It shows FIFO is not a stack algorithm. It does **not** mean FIFO always gets worse with more frames — on average more frames still help. The correct exam sentence: **“FIFO can suffer Belady's anomaly; LRU and Optimal cannot, because they are stack algorithms.”**

LFU and MFU, briefly

Policy	Evicts	Its failure mode
LFU	Lowest access count	A page used heavily at start-up keeps a huge count forever and never leaves, even when it is dead. Fixed with ageing : halve every count periodically.
MFU	Highest access count	Rarely used. The argument is that a low count means “just arrived, probably still needed”, but it throws out genuinely hot pages.

How many frames, and thrashing

Allocating frames between processes

Scheme	Rule
Equal allocation	m frames, n processes $\rightarrow$ each gets m/n . Unfair to big processes.
Proportional allocation	Process i of virtual size s_i gets $\frac{s_i}{\sum s_j} \times m$ frames.
Local replacement	A faulting process may only steal from its own frames. Its fault rate depends only on itself. Cannot exploit spare frames elsewhere.

Global replacement	A faulting process may steal any frame. Better throughput, but one bad process can now wreck another's performance.
---------------------------	--

Example 17 [Infosys · pattern]

There are 62 free frames and 3 processes of virtual sizes 20 KB, 60 KB and 120 KB. Page size is 1 KB. Use proportional allocation.

SOLUTION

Total virtual size = 20 + 60 + 120 = 200 KB, so 200 pages.

$$P1 = \frac{20}{200} \times 62 = 0.10 \times 62 = 6.2 \rightarrow 6 \text{ frames}$$

$$P2 = \frac{60}{200} \times 62 = 0.30 \times 62 = 18.6 \rightarrow 18 \text{ frames}$$

$$P3 = \frac{120}{200} \times 62 = 0.60 \times 62 = 37.2 \rightarrow 37 \text{ frames}$$

Check: 6 + 18 + 37 = 61. One frame is left over (we rounded down each time); give it to the largest process or keep it in a free pool.

Answer: 6, 18 and 37 frames. Always round down and account for the leftovers — never hand out more frames than exist.

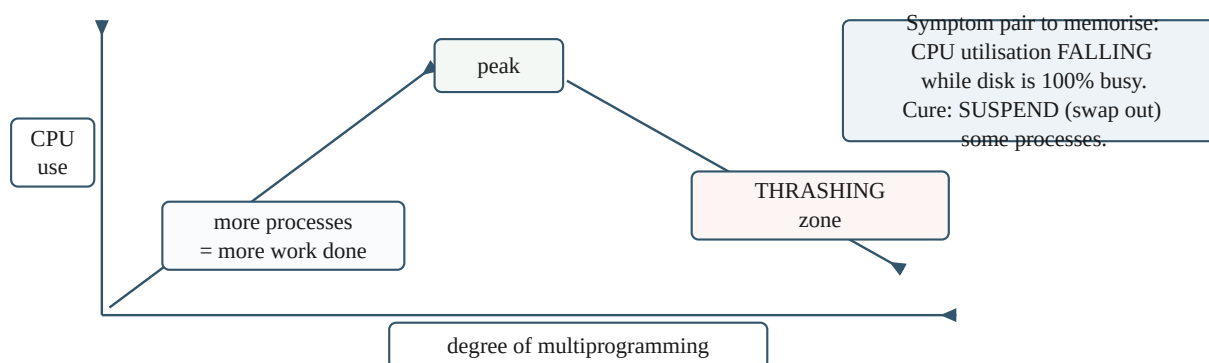
Thrashing

THRASHING

Thrashing = a process spends more time paging than executing. The chain of events:

1. Degree of multiprogramming rises, so each process gets fewer frames.
2. A process drops below the frames it needs for its current working set.
3. Its fault rate explodes; it blocks on disk constantly.
4. CPU utilisation **falls**.
5. The scheduler sees idle CPU and — wrongly — **admits more processes**.
6. Go to step 1. The system collapses.

The killer detail: **CPU utilisation goes down while the disk stays 100% busy**. That pair of symptoms is the exam answer.



CPU utilisation against degree of multiprogramming. Past the peak, adding processes makes throughput collapse.

The working set model

WORKING SET

The **working set** $W(t, \Delta)$ is the set of distinct pages referenced in the last Δ references ending at time t . $|W|$ is the **working set size** (WSS).

Rule: give each process enough frames to hold its working set. If $\sum_i \text{WSS}_i > \text{total frames}$, thrashing is guaranteed — suspend a process.

Δ too small: misses parts of the real locality. Δ too big: carries dead pages.

Example 18 [GIC · pattern]

Reference string 5, 2, 1, 3, 4, 3, 2, 5, 3, 1, 2, 1, window $\Delta = 5$. Compute the working set size at every step. What is the peak, and what does that tell you?

SOLUTION

At each position t , take the last 5 references (fewer at the start) and count **distinct** pages.

t	Last 5 references	Distinct set	WSS
1	5	{5}	1
2	5 2	{5,2}	2
3	5 2 1	{5,2,1}	3
4	5 2 1 3	{5,2,1,3}	4
5	5 2 1 3 4	{5,2,1,3,4}	5
6	2 1 3 4 3	{2,1,3,4}	4
7	1 3 4 3 2	{1,3,4,2}	4
8	3 4 3 2 5	{3,4,2,5}	4
9	4 3 2 5 3	{4,3,2,5}	4
10	3 2 5 3 1	{3,2,5,1}	4
11	2 5 3 1 2	{2,5,3,1}	4
12	5 3 1 2 1	{5,3,1,2}	4

Answer: Peak WSS = 5 at $t = 5$; it settles at 4. Give this process 5 frames and it will barely fault after the cold start; give it 3 and it will thrash.

Cross-check with our earlier traces: with 3 frames LRU took 9 faults, with 4 frames only 7. The working set says 4-5 — and indeed that is where the faults stop falling steeply.

ws.js — working set size over time

JS

```
function workingSet(ref, delta, t) {
  return new Set(ref.slice(Math.max(0, t - delta + 1), t + 1)).size;
}
const ref = [5, 2, 1, 3, 4, 3, 2, 5, 3, 1, 2, 1];
console.log('WSS(delta=5):', ref.map((_, t) => workingSet(ref, 5, t)).join(' '));
// prints: WSS(delta=5): 1 2 3 4 5 4 4 4 4 4 4 4
```

Page-fault frequency (PFF) — the practical control

Working sets are expensive to track exactly. PFF does the same job with a thermostat:

- Measure each process's fault rate.
- Rate above the upper bound -> the process needs more frames; give it one.

- Rate below the lower bound -> it has spare frames; take one away.
- No frames left to give -> suspend a process and free all of its frames.

The clever tricks built on paging

Copy-on-write (COW)

`fork()` should duplicate the parent's whole address space. Copying 2 GB to then immediately `exec()` a new program would be absurd. So:

1. Parent and child share every page, and every shared page is marked **read-only**.
2. Reads are free — both processes read the same frames.
3. The first **write** to a shared page traps. The OS copies just that one page, gives the writer a private writable copy, and restarts the instruction.

So only pages actually modified are ever copied. A `fork()` followed by `exec()` copies almost nothing.

Memory-mapped files

`mmap()` maps a file's bytes directly into the address space. Reading memory reads the file; writing memory dirties the page and the OS writes it back later.

Why it helps	What it costs
No <code>read()</code> syscall per block; no copy from kernel buffer to user buffer.	Page faults are now hidden inside ordinary loads — an innocent <code>arr[i]</code> can block for milliseconds.
Two processes mapping the same file share the same physical frames — that is how shared libraries and shared memory work.	File must fit in the address space; on 32-bit systems that was a real limit.

Demand zero and lazy allocation

`malloc(1 GB)` usually succeeds instantly even on a small machine, because nothing is allocated yet — the kernel just records the mapping. Frames appear on first touch, filled with zeros. This is why a program can “successfully allocate” more memory than exists and then be killed later when it actually writes to it (the Linux OOM killer).

TRAP

“`malloc` returning non-NULL means the memory is yours.” On Linux with default overcommit settings, **no**. The promise is only checked when you touch the page. This is why some servers run with `vm.overcommit_memory = 2`.

Locality: why any of this works at all

Every trick in this chapter — TLB, demand paging, LRU, working sets — is a bet on **locality**. If programs touched pages at random, none of it would work.

THE TWO LOCALITIES

- **Temporal locality** — a page touched now is likely to be touched again soon. (Loops, hot functions, a counter variable.) This is what LRU and clock exploit.
- **Spatial locality** — if you touch byte *x*, you will soon touch bytes near *x*. (Arrays, structs, sequential file reads.) This is why pages are 4 KB and not 8 bytes.

Code you write can destroy locality. The classic destroyer: walking a 2D array down the **columns** when it is stored by **rows**.

Example 19 [Google · pattern]

An array `int a[1024][1024]` of 4-byte integers is stored in row-major order. Page size is 4 KB, and the process has 100 frames. Compare the page faults for these two loops:

A: `for (i = 0..1023) for (j = 0..1023) a[i][j] = 0;`
B: `for (j = 0..1023) for (i = 0..1023) a[i][j] = 0;`

SOLUTION

Step 1 — how big is one row?

$$1024 \text{ elements} \times 4 \text{ bytes} = 4096 \text{ bytes} = \text{exactly one page}$$

So row i sits entirely in page i . The array spans 1024 pages.

Step 2 — loop A walks along a row before moving to the next row. Each row is one page, so each page is brought in once and all 1024 of its elements are used before you leave it.

$$\text{faults}_A = 1024$$

Step 3 — loop B walks **down a column**. The elements `a[0][j]`, `a[1][j]`, `a[2][j]`... are in 1024 **different** pages. The process only has 100 frames, so by the time the loop comes back to page 0 for column $j + 1$, page 0 has long been evicted. Every single access faults.

$$\text{faults}_B = 1024 \times 1024 = 1,048,576$$

Step 4 — ratio: $1048576/1024 = 1024$.

Answer: Loop A: 1024 faults. Loop B: 1,048,576 faults — 1024 times more, for identical output.

Put a number on it: at 8 ms per fault, loop A takes about 8 seconds of paging and loop B takes about 8,400 seconds — over two hours. **Same answer, same instructions, different order.**

The follow-up an interviewer will ask: **“how many frames would loop B need to be fast?”** Answer: 1024 — the whole array — because its working set for one pass of the outer loop is every page. That is why the fix is to change the loop, not to buy RAM.

SHORTCUT

If a question gives you an array, a page size and a loop, the first thing to compute is **how many array elements fit in one page**. Everything else follows from that one number.

Inside the kernel: buddy and slab allocators

Paging hands out whole frames. But the kernel itself constantly needs **small** objects — a 200-byte task structure, a 64-byte file handle. Two allocators sit on top of frames.

The buddy system

Memory is a power-of-two block. A request is rounded up to a power of two, and blocks are split in half repeatedly until a block of the right size exists. The two halves of a split are **buddies**; when both are free they merge back instantly.

Example 20 [Razer · pattern]

A 1 MB region is managed by a buddy allocator with a 64 KB minimum block. Requests arrive: 70 KB, then 35 KB, then 80 KB. Show the splits and compute the total internal fragmentation.

SOLUTION

Request 1: 70 KB. Round up to the next power of two: 128 KB.

Splits needed, starting from 1024 KB:

$$1024 \rightarrow 512 + 512 \quad 512 \rightarrow 256 + 256 \quad 256 \rightarrow 128 + 128$$

Allocate one 128 KB block. Free list now holds: 128, 256, 512. Waste inside it: $128 - 70 = 58$ KB.

Request 2: 35 KB. Round up to 64 KB. Split the free 128 into 64 + 64; allocate one. Free list: 64, 256, 512. Waste: $64 - 35 = 29$ KB.

Request 3: 80 KB. Round up to 128 KB. No free 128 exists, so split the 256 into 128 + 128; allocate one. Free list: 64, 128, 512. Waste: $128 - 80 = 48$ KB.

Total internal fragmentation:

$$58 + 29 + 48 = 135 \text{ KB}$$

Answer: Free blocks left: 64 KB, 128 KB, 512 KB. Internal fragmentation = 135 KB out of 272 KB allocated — about 50% wasted.

The trade-off sentence: the buddy system merges free blocks in $O(1)$ by just checking one buddy address, which makes it fast and keeps external fragmentation low — but rounding every request to a power of two can waste up to **just under 50%** of each allocation.

The slab allocator

The buddy system's waste is unacceptable for objects the kernel allocates a million times. The slab allocator fixes it:

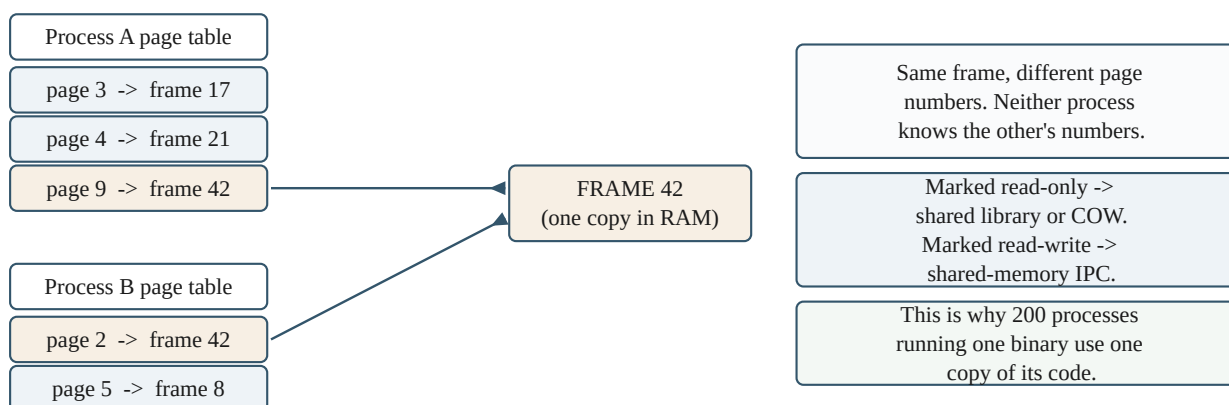
1. For each **type** of kernel object, keep a cache.
2. Each cache holds **slabs** — one or more contiguous frames, pre-carved into slots of exactly that object's size.
3. Allocation = pop a free slot. Free = push it back. No searching, no rounding.
4. Objects are kept **pre-initialised**, so reuse skips the constructor work.

Buddy	Slab
General purpose, any size	One cache per object type
Rounds to a power of two -> big internal waste	Slots are exactly the object size -> near-zero waste
Splitting and merging on every call	Pop and push a free list
Used for page-sized and larger requests	Used for the kernel's small fixed-size objects

Sharing and protection

How two processes share one page

Two page tables, two different page numbers, **the same frame number**. That is all sharing is.



Shared memory and shared libraries: different virtual pages in different processes, pointing at one physical frame.

Example 21 [SCB · pattern]

A server runs 150 copies of the same 8 MB program. Of the 8 MB, 6 MB is read-only code and 2 MB is per-process data. Page size 4 KB. How much RAM is used with sharing, and how much without?

| SOLUTION

Without sharing: $150 \times 8 \text{ MB} = 1200 \text{ MB} \approx 1.17 \text{ GB}$.

With sharing: code is identical and read-only, so one copy suffices.

$$6 \text{ MB (shared code)} + 150 \times 2 \text{ MB (private data)} = 6 + 300 = 306 \text{ MB}$$

Saving: $1200 - 306 = 894 \text{ MB}$, about a 75% reduction.

In frames: $306 \text{ MB} / 4 \text{ KB} = 78,336$ frames instead of 307,200.

Answer: 306 MB with sharing versus 1200 MB without — the code pages are mapped once and appear in 150 page tables.

TLB reach — the number nobody computes

TLB REACH

$$\text{TLB reach} = \text{number of TLB entries} \times \text{page size}$$

If your working set is bigger than the TLB reach, you will miss the TLB constantly no matter how good your replacement policy is.

Example 22 [D. E. Shaw · pattern]

A CPU has 64 data-TLB entries. An in-memory analytics job has a 100 MB working set. (a) What is the TLB reach with 4 KB pages? (b) With 2 MB huge pages? (c) What does this do to the effective access time if a TLB miss costs a 4-level walk at 80 ns per level?

SOLUTION

(a) $64 \times 4 \text{ KB} = 256 \text{ KB}$. The working set is 100 MB — that is **400 times** the reach. Essentially every access to new data misses the TLB.

(b) $64 \times 2 \text{ MB} = 128 \text{ MB}$. Now the whole 100 MB working set fits inside the TLB's reach, so once warm, almost everything hits.

(c) A miss costs $4 \times 80 = 320 \text{ ns}$ of walking, plus the data access.

With 4 KB pages, take the hit ratio as roughly 0 for streaming new data:

$$\text{EAT} \approx 320 + 80 = 400 \text{ ns per access}$$

With 2 MB pages, take the hit ratio as roughly 0.99:

$$\text{EAT} \approx 0.99 \times 80 + 0.01 \times 400 = 79.2 + 4 = 83.2 \text{ ns}$$

Answer: (a) 256 KB (b) 128 MB (c) roughly 400 ns per access falling to about 83 ns — a 4.8x speed-up from changing page size alone, with no code change.

The cost you must mention: 2 MB pages waste up to 2 MB per mapping, and the kernel must find physically contiguous 2 MB regions, which gets hard on a long-running fragmented machine.

Swapping, and what “swap” really means

Term	What it actually is
Swapping (classical)	Moving an entire process out to disk and back. Coarse, old, still used to relieve thrashing by suspending whole processes.
Paging out	Moving individual pages to disk. This is what actually happens minute to minute.
Swap space	The disk area holding paged-out anonymous memory (heap, stack). Code pages do not need it — they can be re-read from the executable.
Page in	Bringing a page back. Cost = one disk read.
Clean vs dirty eviction	Clean page: just drop it, 0 writes. Dirty page: write it out first, 1 write.

TRAP

“All evicted pages get written to swap.” No — only **dirty** pages, and only **anonymous** ones. A clean code page is simply discarded and re-read from the program file when needed again. This is why read-heavy processes evict almost for free.

Practice

PRACTICE · WARM-UP target: 6 min

1. Page size 2 KB. How many bits of offset?
2. A 14-bit logical address with 512-byte pages. How many pages?
3. Which fragmentation does paging cause?
4. Name the bit that tells the OS a page must be written back before eviction.
5. Reference string 1, 2, 3, 1, 2, 3 with 3 frames, FIFO. How many faults?

ANSWER KEY

1. $2\text{KB} = 2^{11} \rightarrow$ **11 bits**.
2. Offset bits = $\log_2 512 = 9$. Page bits = $14 - 9 = 5$, so $2^5 =$ **32 pages**.
3. **Internal** fragmentation (last page partly empty). It removes external fragmentation.
4. The **dirty** (modified) bit.
5. 3 cold faults, then everything hits: **3 faults**.

PRACTICE · TIER 1 target: 14 min

1. TLB access 15 ns, memory access 120 ns, hit ratio 90%, one-level page table. Find EAT.
2. 32-bit addresses, 8 KB pages, 4-byte PTEs. Size of a flat page table?
3. A process of 42,000 bytes with 8 KB pages. Pages needed and internal fragmentation?
4. Trace LRU on 3, 1, 4, 1, 5, 9, 2, 6, 5, 3 with 3 frames. Faults?
5. Segment table: seg 5 has base 2900, limit 420. Translate (5, 419) and (5, 420).

ANSWER KEY

1. Hit: $15 + 120 = 135$. Miss: $15 + 120 + 120 = 255$. $\text{EAT} = 0.9(135) + 0.1(255) = 121.5 + 25.5 = 147$ ns.
2. Offset bits = $\log_2 8192 = 13$. Entries = $2^{32-13} = 2^{19} = 524288$. Size = $524288 \times 4 = 2$ MB (**2 MB**).
3. $42000/8192 = 5.13$, round up to **6 pages**. Given $6 \times 8192 = 49152$ bytes. Waste = $49152 - 42000 = 7152$, so **7152 bytes** of internal fragmentation.
4. Trace: 3(F) 1(F) 4(F) 1(hit) 5(F, out 3) 9(F, out 4) 2(F, out 1) 6(F, out 5) 5(F, out 9) 3(F, out 2) = **9 faults**, 1 hit. Check $9 + 1 = 10$.
5. (5, 419): $419 < 420$, so $2900 + 419 = 3319 \rightarrow$ **3319**. (5, 420): $420 < 420$ is false $\rightarrow$ **trap**.

PRACTICE · TIER 2 target: 18 min

1. A 48-bit virtual address with 4 KB pages and 8-byte PTEs. How many levels of page table are needed if every table must fit in exactly one page?
2. Memory access 100 ns, page fault service 6 ms. What fault rate gives $\text{EAT} = 300$ ns?
3. Trace **clock** on 1, 2, 3, 4, 1, 2, 5, 1, 2, 3 with 3 frames, hand starting at frame 0.
4. Two processes each `fork()` and immediately `exec()`. Explain in three lines why COW makes this nearly free, and name the one case where COW costs **more** than a plain copy.

ANSWER KEY

1. Offset = 12 bits, leaving $48 - 12 = 36$ page-number bits. One page holds $4096/8 = 512 = 2^9$ entries, so each level consumes 9 bits. $36/9 = 4$, so **4 levels**.
2. $6 \text{ ms} = 6,000,000 \text{ ns}$. $(1 - p)100 + 6000000p = 300$, so $100 + 5999900p = 300$, giving $p = 200/5999900 \approx 3.33 \times 10^{-5}$ — **about 1 fault in 30,000 accesses**.
3. 1(F) 2(F) 3(F) [frames 1,2,3 R=0,0,0, hand at 0] 4(F, f0 R=0, evict 1 -> 4,2,3) 1(F, f1 R=0, evict 2 -> 4,1,3) 2(F, f2 R=0, evict 3 -> 4,1,2) 5(F, f0 R=0, evict 4 -> 5,1,2) 1(hit, R of 1 -> 1) 2(hit, R of 2 -> 1) 3(F: f1 R=1 pardon, f2 R=1 pardon, f0 R=0 evict 5 -> 3,1,2). Total **8 faults**.
4. COW shares all pages read-only; `exec()` then throws the whole address space away, so **zero** pages were ever copied. COW costs more when the child writes to nearly every page anyway — you then pay one trap per page **plus** the copy, instead of one bulk copy.

PRACTICE · TIER 3 target: 22 min

1. A database server shows 100% disk utilisation and 15% CPU utilisation, and adding worker processes makes both worse. Diagnose it, and give two fixes: one immediate, one structural.
2. Explain why huge pages (2 MB) help a large in-memory database but can hurt a machine running many small processes.
3. Prove that LRU cannot suffer Belady's anomaly. (Hint: show the resident set with n frames is always contained in the resident set with $n + 1$ frames.)
4. You must design a replacement policy for a phone, where evicting a dirty page costs a flash write and flash has limited write cycles. What do you change about clock, and what does it cost you?

ANSWER KEY

1. **Diagnosis:** thrashing. The pair “CPU falling, disk saturated, more processes makes it worse” is decisive — the combined working sets exceed RAM. **Immediate:** reduce the degree of multiprogramming — suspend or cap worker processes (lower the connection-pool size). **Structural:** add RAM, or shrink each worker's working set (smaller buffers, smaller result sets, better indexes so fewer pages are touched per query).
2. **Helps:** one TLB entry covers 2 MB instead of 4 KB, so a 64-entry TLB reaches 128 MB instead of 256 KB — TLB misses collapse, and page-table walks get shorter. **Hurts:** internal fragmentation is now up to 2 MB per mapping, so many small processes waste enormous memory, and the OS must find **physically contiguous** 2 MB regions, which fragmenting memory makes hard.
3. Let $S_n(t)$ be the set of pages resident after t references with n frames under LRU. LRU keeps exactly the n most recently used distinct pages. The n most recently used pages are, by definition, a subset of the $n + 1$ most recently used pages. So $S_n(t) \subseteq S_{n+1}(t)$ for every t . Therefore any reference that hits with n frames also hits with $n + 1$ frames, so faults with $n + 1$ frames $\leq$ faults with n frames. No anomaly is possible. The same argument works for Optimal (the n pages with nearest next use are a subset of the $n + 1$ such pages).
4. Use **enhanced clock:** prefer victims in the order $(R=0,D=0)$, $(R=0,D=1)$, $(R=1,D=0)$, $(R=1,D=1)$ — i.e. always take a clean page first, and give dirty pages an extra pardon. Add background **write-back** so pages are cleaned while the device is idle, turning them into cheap victims later. **Cost:** you sometimes evict a clean page that was more useful than a dirty one, so your hit rate drops slightly; you have traded a little CPU performance for flash lifetime. Also, the scan may now go round the clock twice, so eviction takes longer.

Rapid fire: one-line answers

SAY THESE IN ONE BREATH

1. **Logical vs physical address?** Logical is what the CPU produces; physical is what reaches RAM. The MMU converts.
2. **Page vs frame?** Page is a block of virtual memory; frame is a block of physical memory. Same size.
3. **What splits a logical address?** Low $\log_2(\text{page size})$ bits are the offset; the rest is the page number.
4. **Physical address formula?** frame $\times$ page size + offset.
5. **What is in a page-table entry?** Frame number, valid bit, protection bits, dirty bit, referenced bit.

6. **What does the valid bit 0 cause?** A page fault.
7. **Page fault vs segmentation fault?** Page fault is normal and recoverable; segfault is an illegal address and kills the process.
8. **What is a TLB?** A small fast cache of recent page-to-frame translations, inside the MMU.
9. **EAT with a TLB?** $\alpha(t + m) + (1 - \alpha)(t + 2m)$ for one-level tables.
10. **Does a context switch flush the TLB?** Classically yes; modern CPUs avoid it with ASID/PCID tags.
11. **Internal fragmentation?** Waste **inside** an allocated block — paging's last page.
12. **External fragmentation?** Free memory exists but is split into unusable holes — contiguous allocation and segmentation.
13. **Which fit is usually fastest?** First fit. Worst fit is usually the worst overall.
14. **Why multi-level page tables?** A flat 32-bit table costs 4 MB per process; hierarchy lets untouched regions cost nothing.
15. **What is an inverted page table?** One table for the whole machine, one entry per frame, keyed by (pid, page). Small, but needs hashing to search.
16. **Paging vs segmentation in one line?** Paging = fixed blocks chosen by the OS, no external fragmentation; segmentation = variable meaningful blocks chosen by the program, natural protection and sharing.
17. **What is demand paging?** Bring a page in only when it is first referenced.
18. **EAT under demand paging?** $(1 - p)m + pF$, where F is the fault service time.
19. **Optimal page replacement?** Evict the page whose next use is farthest away. Unimplementable; used as a lower bound.
20. **Why does LRU beat FIFO?** FIFO measures age; LRU measures recency of use, which tracks locality.
21. **Belady's anomaly?** More frames giving more faults. FIFO can; LRU and Optimal cannot.
22. **Why can LRU never be anomalous?** It is a stack algorithm — the n most recent pages are a subset of the $n + 1$ most recent.
23. **What is the clock algorithm?** FIFO plus a reference bit; a page with $R=1$ gets one pardon, R reset to 0, hand moves on.
24. **What is enhanced clock?** Clock using (referenced, dirty) pairs; prefers clean victims to avoid a disk write.
25. **What is thrashing?** Paging more than executing. Symptom: CPU utilisation falls while the disk stays saturated.
26. **Cure for thrashing?** Lower the degree of multiprogramming — suspend processes; or add RAM.
27. **What is a working set?** The distinct pages touched in the last Δ references. Give a process that many frames.
28. **What is copy-on-write?** Share pages read-only after fork; copy a page only when someone writes to it.
29. **Why do huge pages help?** Each TLB entry covers far more memory, so the TLB hit ratio rises and walks shorten.
30. **Why does malloc of 1 GB succeed on a small machine?** Lazy allocation — frames appear on first touch, not at allocation.

ONE-PAGE REVISION CARD

Address arithmetic. offset bits = $\log_2(\text{page size})$; page bits = address bits — offset bits; entries = $2^{\text{page bits}}$; physical = frame $\times$ page size + offset.

Two EATs, do not mix them up.

- With a TLB: $\alpha(t + m) + (1 - \alpha)(t + Lm + m)$ — L = page-table levels.
- With demand paging: $(1 - p)m + pF$ — F is the disk service time in the **same units**.

Fragmentation. Paging -> internal only. Segmentation and contiguous -> external. Average internal waste with paging $\approx$ half a page per process.

Replacement, ranked on our string 5, 2, 1, 3, 4, 3, 2, 5, 3, 1, 2, 1 with 3 frames: OPT 7 < LRU 9 = Clock 9 < FIFO 10. Compute OPT first; nothing may beat it.

Belady's anomaly. FIFO only. LRU and OPT are stack algorithms and are immune.

Thrashing. CPU utilisation down + disk 100% busy + adding processes makes it worse. Cure = fewer processes, or more RAM. Model = working set; controller = page-fault frequency.

Three sentences that win marks.

1. "Paging removes external fragmentation but creates internal fragmentation."
2. "Hierarchical page tables trade time for space: unused regions cost nothing, but every TLB miss now costs one memory read per level."
3. "A page fault is normal; a segmentation fault is a bug."

OS: File Systems, I/O & Storage

Turning a spinning platter into a name you can type

The one idea

THE WHOLE CHAPTER IN ONE BOX

A disk is a very long numbered array of fixed-size **blocks**. It has no idea what a file is, what a folder is, or what a name is. Everything you think of as a file system is software that maintains three mappings on top of that array:

1. **name** → **inode number** — this is what a **directory** is. A directory is just a file whose contents are (name, inode number) pairs.
2. **inode number** → **inode** — a fixed-size record holding the file's metadata and the addresses of its data blocks.
3. **inode** → **list of block numbers** — the actual bytes.

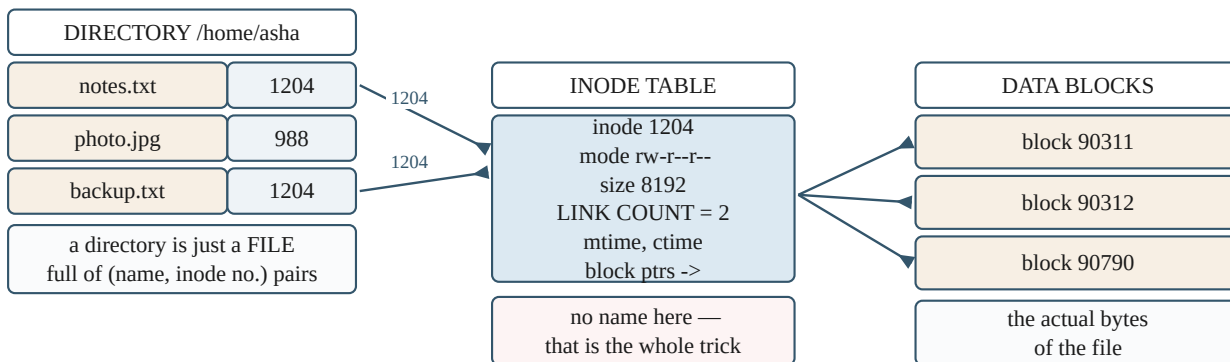
Vocabulary you must be able to say instantly:

- **Block (cluster)** — the smallest unit the file system allocates. Typically 4 KB.
- **Sector** — the smallest unit the **hardware** reads or writes. 512 B or 4096 B.
- **Inode** — the metadata record: size, owner, permissions, timestamps, link count, and block pointers. **The file name is NOT in the inode.**
- **Directory entry (dentry)** — a (name, inode number) pair inside a directory file.
- **Superblock** — the record describing the whole file system: block size, counts, where the inode table starts.
- **Mount** — grafting one file system's root onto a directory of another.

The single most-asked fact in this chapter: **the file name lives in the directory, not in the inode**. Every question about hard links, renames and permissions falls out of that.

SHORTCUT

Whenever a file-system question confuses you, ask: “which of the three mappings is this question about — name to inode, inode to metadata, or inode to blocks?” Almost every question is about exactly one of them.



The three mappings. Follow the arrows to open /home/asha/notes.txt. Notice that the NAME appears only on the left; the inode never learns it.

Two names, `notes.txt` and `backup.txt`, point at inode 1204. That is a **hard link**, and it is why the inode carries a **link count** of 2. Delete either name and the count drops to 1; the bytes survive. Delete both and the count reaches 0 and the blocks are freed.

A file, and what the OS keeps about it

File attributes (what is in an inode)

Attribute	Note
Type	regular file, directory, symlink, device, pipe, socket
Size in bytes	Not the same as blocks used — see sparse files below
Owner UID, group GID	Used by the permission check
Permission bits	rwX for user, group, other
Link count	How many directory entries point here. Reaches 0 -> the file is really deleted
Timestamps	atime (accessed), mtime (contents modified), ctime (inode changed)
Block pointers	Direct, single, double and triple indirect — the heart of it

TRAP

“ctime is the creation time.” **No.** On Unix ctime is **inode change time** — it moves when you chmod a file, even though the contents did not change. Classic Unix has no creation time at all; modern file systems add btime/crtime as an extra.

The open-file tables

When a process calls `open()`, three tables come into play. Interviewers ask this because it explains why `fork()` makes a child share the parent’s file **position**.

Table	One per	Holds
File descriptor table	process	fd number -> pointer into the open-file table
Open-file table (system-wide)	<code>open()</code> call	current offset , access mode, pointer to the inode
In-memory inode table	file	one cached copy of the inode, with a reference count

Two independent `open()` calls on the same file get **two** open-file entries, so they have **two separate offsets** but **one** inode. A `fork()` or `dup()` copies the **descriptor**, so parent and child point at the **same** open-file entry and therefore **share one offset** — when the child reads 10 bytes, the parent’s next read starts 10 bytes later. That surprise is the whole reason the middle table exists.

Directories and path resolution

A directory is a file. Its contents are entries. Resolving `/home/asha/notes.txt` is a loop, and you should be able to narrate it.

Example 1 [Infosys · pattern]

Walk through exactly what the kernel does to open `/home/asha/notes.txt`. How many disk reads at minimum, assuming nothing is cached?

SOLUTION

Step 1 — start at the root inode. Its number is fixed (2 on ext-family file systems), so no lookup is needed. **Read the root inode.** (1)

Step 2 — read the root directory’s data block. Search it for the name `home`. Get an inode number, say 431. (2)

Step 3 — read inode 431. Check it is a directory and that you have execute (x) permission on it. (3)

Step 4 — read inode 431’s data block. Search for `asha`. Get inode number 902. (4)

Step 5 — read inode 902, check permission, read its data block, search for `notes.txt`. (5, 6)

Step 6 — read the inode of `notes.txt`. Check read permission. (7)

Now, and only now, can data blocks be read.

Answer: 7 disk reads before a single byte of the file is touched — one inode read plus one directory-block read per path component. This is why the dentry cache exists, and why deep paths cost more than shallow ones.

The permission rule worth stating: you need **execute** permission on every directory in the path (that is what `x` means on a directory — “may traverse”), and then the appropriate permission on the file itself.

Directory structures, oldest to newest

Structure	Problem it has, or fixes
Single level — one flat namespace	Two users cannot both have <code>notes.txt</code> .
Two level — one directory per user	Fixes name clashes; no grouping within a user.
Tree	What everyone uses. One parent per file; a path is unique.
Acyclic graph (hard links)	A file can have several names. Deletion needs a link count .
General graph (allows cycles)	Needs garbage collection to find unreachable loops. Avoided; this is why Unix forbids hard links to directories.

Hard links and soft links

THE DIFFERENCE IN ONE LINE EACH

- A **hard link** is another directory entry pointing at the **same inode**. It increments the inode’s link count. All hard links are equal — there is no “original”.
- A **soft link** (symbolic link) is a tiny **separate file** whose contents are a **path string**. It has its own inode. It does not affect the target’s link count.

Consequences that get asked:

- Delete the “original”: hard link still works (link count only dropped from 2 to 1). Soft link **breaks** (dangling).
- Hard links cannot cross file systems (inode numbers are only meaningful within one file system). Soft links can, because they store a path.
- Hard links to directories are forbidden (they would create cycles). Soft links to directories are fine.
- `ls -li` shows the inode number: two hard links show the **same** number.

Example 2 [TCS NQT · pattern]

Run this and predict the output before you read it:

```
echo "hello" > original.txt
ln    original.txt hard.txt      # hard link
ln -s original.txt soft.txt     # soft link
ls -li
rm original.txt
ls -li
cat hard.txt
cat soft.txt
```

SOLUTION

Real output from a Linux shell:

```
8132533 -rw-rw-r-- 2 zayed zayed 6 hard.txt
8132533 -rw-rw-r-- 2 zayed zayed 6 original.txt
8132535 lrwxrwxrwx 1 zayed zayed 12 soft.txt -> original.txt
```

Read the columns. `hard.txt` and `original.txt` share inode **8132533** and both show a link count of **2**. `soft.txt` has its own inode **8132535**, link count 1, and a size of **12 bytes** — which is exactly the length of the string `original.txt`. That is all a symlink contains.

After `rm original.txt`:

```
8132533 -rw-rw-r-- 1 zayed zayed 6 hard.txt
8132535 lrwxrwxrwx 1 zayed zayed 12 soft.txt -> original.txt
```

The link count on inode 8132533 fell from 2 to 1. The inode and its data blocks are still alive.

```
$ cat hard.txt
hello
$ cat soft.txt
cat: soft.txt: No such file or directory
```

Answer: The hard link still reads “hello”; the soft link is now dangling and fails. `rm` does not delete a file — it deletes a name and decrements the link count. The file dies only when the count reaches 0 and no process still has it open.

TRAP

“`rm` frees the disk space.” Only if the link count hits 0 **and** no process has the file open. A running server whose log file you deleted keeps the blocks alive until it closes the file — which is why `df` can show a full disk while `du` shows very little. The fix is to restart the process or truncate the file, not to delete harder.

How a file’s blocks are found

This is the core of the chapter. Three schemes, and you must be able to compare them on four axes: sequential speed, random access speed, space overhead, and external fragmentation.

1. Contiguous allocation

Store the file in consecutive blocks. The directory entry holds (`start block`, `length`).

Good	Bad
Best possible sequential read — one seek, then stream.	External fragmentation; needs compaction.
$O(1)$ random access: block k is at <code>start + k</code> .	You must know the final size at creation, or the file cannot grow.
Tiny metadata: two numbers.	Deleting files leaves holes of awkward sizes.

2. Linked allocation

Each block stores a pointer to the next block. The directory entry holds the first (and often the last) block number.

Good	Bad
No external fragmentation; any free block works.	Random access is $O(k)$ — to read block 500 you must read 500 blocks.
Files grow freely.	The pointer eats space inside every data block, so block size is no longer a power of two for data.
	One corrupted pointer loses the rest of the file.

Example 3 [Wipro · pattern]

Block size is 4 KB. A pointer is 4 bytes. A file is exactly 2 MB. How many blocks does it need under (a) contiguous, (b) linked allocation?

SOLUTION

(a) Contiguous. Every byte of a block is data.

$$2 \text{ MB} = 2 \times 1024 \times 1024 = 2,097,152 \text{ bytes}$$

$$2097152/4096 = 512 \text{ blocks}$$

(b) Linked. Each block gives up 4 bytes to the “next” pointer:

$$\text{usable per block} = 4096 - 4 = 4092 \text{ bytes}$$

$$2097152/4092 = 512.5 \rightarrow \text{round up} \rightarrow 513 \text{ blocks}$$

Answer: (a) 512 blocks (b) 513 blocks — one extra, purely to carry pointers.

One extra block sounds harmless. The real cost is not space, it is **time**: to read byte 2,000,000 under linked allocation you must first read 488 earlier blocks. Under contiguous allocation you compute the block number and seek there once.

3. FAT — linked allocation with the links pulled out

Keep all the “next block” pointers in one table at the front of the disk, indexed by block number. Following a chain now walks the table in memory, not the disk.

Example 4 [Capgemini · pattern]

A FAT looks like this (EOF marks the end of a chain, 0 marks a free block):

block	5	9	12	17	23	30	31	44
next	44	23	0	9	5	0	EOF	31

A directory entry says `report.doc` starts at block 17. List its blocks. Then say how big the FAT is for a 500 GB disk with 4 KB blocks and 4-byte entries, and why that is a problem.

SOLUTION

Chain: start at 17.

$$17 \rightarrow 9 \rightarrow 23 \rightarrow 5 \rightarrow 44 \rightarrow 31 \rightarrow \text{EOF}$$

So the file occupies blocks **17, 9, 23, 5, 44, 31** — six blocks. Blocks 12 and 30 have `next = 0`, meaning free.

FAT size:

$$\text{blocks on disk} = \frac{500 \times 1024^3}{4096} = 131,072,000 \text{ blocks}$$

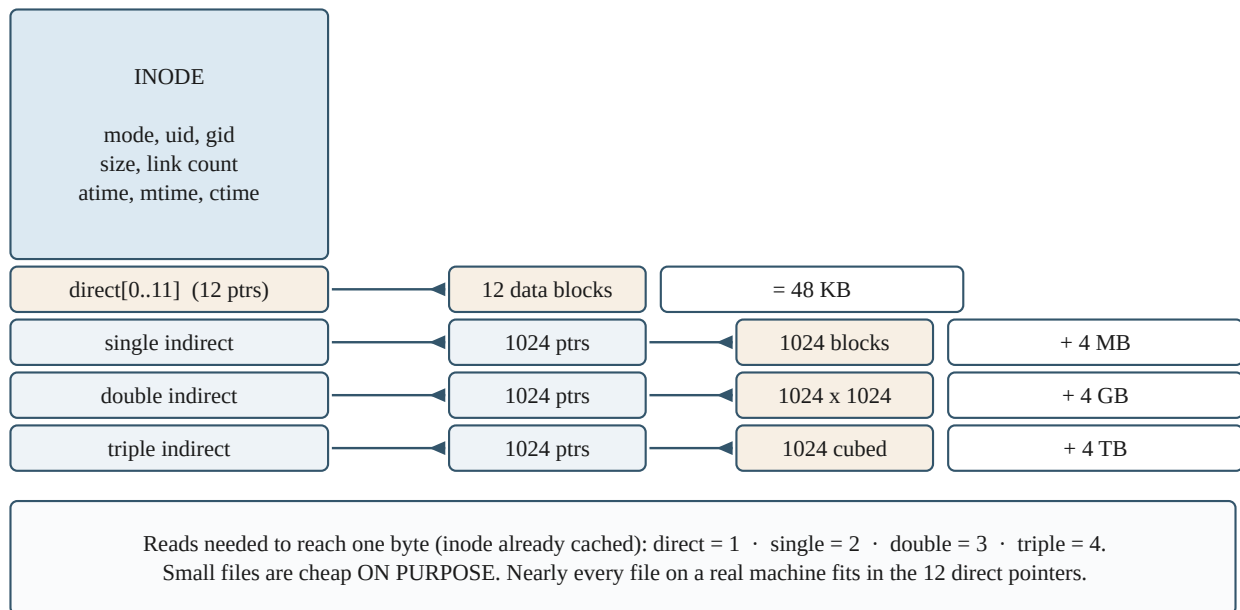
$$\text{FAT size} = 131072000 \times 4 \text{ bytes} = 524,288,000 \text{ bytes} = 500 \text{ MB}$$

Answer: Blocks 17, 9, 23, 5, 44, 31. The FAT itself would be 500 MB — and to be useful for random access it must be held in RAM, which is why FAT does not scale to large disks.

Note the pattern: FAT size grows linearly with disk size, and it is a **whole-disk** structure. An inode, by contrast, is **per file** — you only load the inodes of files you actually open. That is the structural reason inodes won.

4. Indexed allocation — the inode

Each file gets an **index block** holding pointers to its data blocks. Random access is one extra read, and there is no external fragmentation. The only question is what to do when a file is bigger than one index block can describe — and the answer is **indirection**.



A Unix inode. 12 direct pointers reach the first 48 KB; three levels of indirection reach 4 TB. Small files cost one read; huge files cost four.

Example 5 [Grab · pattern]

Block size 4 KB, pointer size 4 bytes, 12 direct pointers, one single, one double and one triple indirect pointer. (a) What is the largest file? (b) How many disk reads to fetch the byte at offset 100,000,000, assuming the inode is cached?

SOLUTION

Step 1 — pointers per block.

$$4096 \frac{\text{bytes}}{4} \text{ bytes per pointer} = 1024 \text{ pointers}$$

Step 2 — blocks reachable at each level.

Level	Blocks reached	Bytes added
12 direct	12	$12 \times 4\text{KB} = 48 \text{ KB}$
single indirect	1024	$1024 \times 4\text{KB} = 4 \text{ MB}$
double indirect	$1024^2 = 1,048,576$	$\approx 4 \text{ GB}$
triple indirect	$1024^3 = 1,073,741,824$	$\approx 4 \text{ TB}$

Step 3 — total.

$$12 + 1024 + 1048576 + 1073741824 = 1,074,791,436 \text{ blocks} \\ \times 4096 \text{ bytes} = 4,402,345,721,856 \text{ bytes} \approx 4.0039 \text{ TiB}$$

(b) Where does offset 100,000,000 live?

$$\text{block number} = \left\lfloor \frac{100000000}{4096} \right\rfloor = 24414$$

Is $24414 < 12$? No. Subtract: $24414 - 12 = 24402$. Is $24402 < 1024$? No. Subtract: $24402 - 1024 = 23378$. Is $23378 < 1024^2 = 1048576$? **Yes** — so it is under the **double** indirect pointer.

Reads: double-indirect block (1), then the inner index block (2), then the data block (3).

Answer: (a) about 4 TiB — exactly 4,402,345,721,856 bytes. (b) 3 reads: double index, inner index, data.

```
function inodeReach(blockSize, ptrSize, direct = 12) {
  const ppb = blockSize / ptrSize; // pointers per block
  const blocks = direct + ppb + ppb ** 2 + ppb ** 3;
  return { ppb, blocks, maxBytes: blocks * blockSize };
}

for (const [bs, ps] of [[1024, 4], [4096, 4], [8192, 8]]) {
  const r = inodeReach(bs, ps);
  console.log(`block=${bs}B ptr=${ps}B ppb=${r.ppb} maxBytes=${r.maxBytes} ` +
    `= ${r.maxBytes / 1024 ** 4).toFixed(4)} TiB`);
}

// which pointer level holds byte `off`, and how many block reads it costs
function locate(off, blockSize = 4096, ptrSize = 4, direct = 12) {
  const ppb = blockSize / ptrSize;
  let b = Math.floor(off / blockSize);
  if (b < direct) return { level: 'direct', reads: 1 };
  b -= direct;
  if (b < ppb) return { level: 'single', reads: 2 };
  b -= ppb;
  if (b < ppb * ppb) return { level: 'double', reads: 3 };
  b -= ppb * ppb;
  if (b < ppb ** 3) return { level: 'triple', reads: 4 };
  return { level: 'too big', reads: -1 };
}

for (const off of [100, 50_000, 5_000_000, 100_000_000, 40_000_000_000])
  console.log(off, locate(off));
```

Real output from node inode.js:

```
block=1024B ptr=4B ppb=256 maxBytes=17247252480 = 0.0157 TiB
block=4096B ptr=4B ppb=1024 maxBytes=4402345721856 = 4.0039 TiB
block=8192B ptr=8B ppb=1024 maxBytes=8804691443712 = 8.0078 TiB
100 { level: 'direct', reads: 1 }
50000 { level: 'single', reads: 2 }
5000000 { level: 'double', reads: 3 }
100000000 { level: 'double', reads: 3 }
40000000000 { level: 'triple', reads: 4 }
```

Look at the middle row versus the last: **doubling the block size to 8 KB and the pointer to 8 bytes doubles the maximum file size**, because pointers per block stayed at 1024 while each block got twice as big.

TRAP

“Bigger blocks are always better.” Bigger blocks give faster sequential I/O and reach bigger files — but internal fragmentation grows with them. With 4 KB blocks a 100-byte file wastes 3996 bytes; with 64 KB blocks it wastes 65,436 bytes. On a disk with a million tiny files that is 60 GB of pure waste. State both halves.

The four schemes, side by side

	Contiguous	Linked	FAT	Indexed (inode)
Sequential read	Best	Good	Good	Good
Random access to block k	$O(1)$	$O(k)$ disk reads	$O(k)$ in-RAM steps	1 to 4 reads

External fragmentation	Yes	No	No	No
Can the file grow?	Hard	Freely	Freely	Freely
Metadata cost	2 numbers	4 B per block, inside the data	One table for the whole disk	One inode per file
Where you see it	CD-ROM, some real-time systems	Teaching only	FAT32, SD cards, USB sticks	ext4, XFS, APFS, NTFS (variant)

Free space management

The file system must also know which blocks are free.

Method	How	Verdict
Bit vector (bitmap)	One bit per block: 1 = free, 0 = used.	Simple; finding a run of consecutive free blocks is just a scan for consecutive 1 bits, which hardware does fast. Must be kept in RAM to be quick.
Linked list of free blocks	Each free block points to the next free block.	Zero extra space, but finding n contiguous blocks is impossible without walking the chain.
Grouping	The first free block holds the addresses of the next n free blocks.	Finds many free blocks in one read.
Counting	Store (first free block, run length) pairs.	Very compact when free space is clustered — which it usually is.

Example 6 [Accenture · pattern]

A 500 GB disk uses 4 KB blocks. How large is the free-space bitmap? Would you keep it in RAM?

SOLUTION

Step 1 — number of blocks (using $1\text{ GB} = 1024^3$ bytes):

$$\frac{500 \times 1024^3}{4096} = 131,072,000 \text{ blocks}$$

Step 2 — one bit per block:

$$131072000 \frac{\text{bits}}{8} = 16,384,000 \text{ bytes}$$

Step 3 — in MB:

$$\frac{16384000}{1024^2} = 15.625 \text{ MB}$$

Answer: About 15.6 MB. Yes, keep it in RAM — 15 MB is nothing on a modern machine, and a bitmap is useless if every free-block query needs a disk read.

Compare with the FAT for the same disk: 500 MB. The bitmap is 32 times smaller because it stores 1 bit per block instead of a 4-byte pointer.

Crash consistency and journaling

WHY A CRASH IS DANGEROUS

Appending one block to a file needs **three** separate writes:

1. mark the block as used in the free bitmap,
2. write the data into the block,
3. update the inode (new block pointer, new size).

A crash between any two of them leaves the disk inconsistent. The two bad outcomes:

- **Lost block** — bitmap says used, no inode points at it. Wasted space; harmless.
- **Double allocation or garbage in a file** — inode points at a block the bitmap still calls free, so it may be handed to another file. **Data corruption.** This one is serious.

Journaling fixes it: before touching the real structures, write a description of the whole change to a sequential log, then a **commit** record. Only then apply the change to the real locations. After a crash, replay committed transactions and discard uncommitted ones. Recovery becomes reading a small log instead of scanning the entire disk (fsck).

Journal mode	What is logged	Trade-off
Writeback	Metadata only; data may be written any time.	Fastest. After a crash, the metadata is consistent but a file may contain stale garbage.
Ordered (the usual default)	Metadata only, but data is forced to disk before the metadata that points at it commits.	Good balance: you never see another file's old bytes.
Journal (full)	Metadata and data.	Safest and slowest — every byte is written twice.

Example 7 [Amazon · pattern]

Full journaling writes every byte twice, so you would expect it to halve write throughput. On a real system it often costs much less than half. Explain why, then explain the one workload where it really does cost about half.

SOLUTION

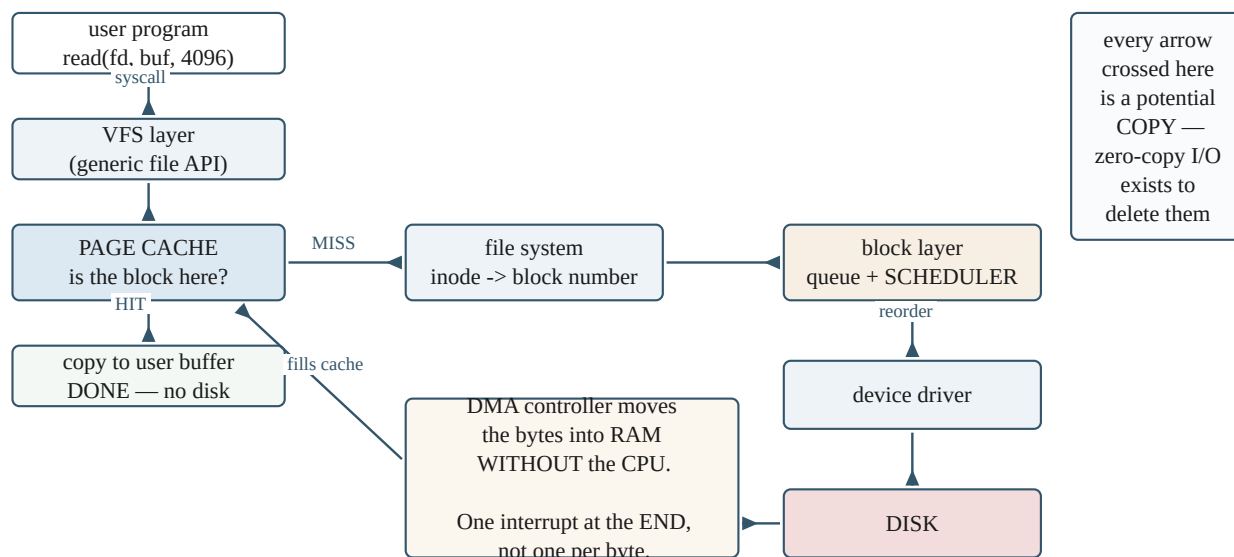
Why it costs less than half. The journal is written **sequentially** to one contiguous region. The real update is **random** — inode here, bitmap there, data block somewhere else. On a spinning disk, a sequential write costs roughly a transfer, while a random write costs a seek plus a rotation plus a transfer. From our disk model that is about 0.04 ms versus about 9.2 ms. So the “extra” write is nearly free compared with the one it protects.

There is a second effect: journal entries are batched. Many small updates commit together in one sequential burst, and the real writes can then be applied lazily and re-ordered into a better seek order. The journal effectively acts as a write-combining buffer.

Where it really does cost half. Large **sequential** writes of bulk data — streaming a 10 GB video file. Here the real write was already sequential, so the journal copy is the same cost as the real one, and you genuinely pay 2x bandwidth. This is exactly why `data=ordered` (metadata journaling only) is the default: it protects the structures without doubling bulk data.

Answer: The journal is sequential and batched, so its cost is tiny next to the random writes it protects — except for bulk sequential writes, where it truly doubles the I/O.

I/O: how bytes actually move



The path of one read(). The page cache is why the second read of the same file costs nothing, and DMA is why a 4 KB transfer costs one interrupt instead of thousands.

Three ways to talk to a device

Technique	How	Cost for a 4 KB transfer
Programmed I/O (polling)	CPU loops reading a status register until the device is ready, then moves one word itself.	CPU is 100% busy the whole time. Simple, and genuinely best for very fast devices where an interrupt would cost more than the wait.
Interrupt-driven	CPU issues the request and does something else; the device interrupts when a word is ready.	One interrupt per word — up to 4096 interrupts. Each interrupt costs a context save/restore.
DMA	CPU tells a DMA controller “move 4096 bytes from device to address X” and forgets about it.	One interrupt, at the end. The CPU is free for the whole transfer. This is what every real disk and NIC uses.

Example 8 [Sea/Shopee · pattern]

A device delivers one byte at a time. Handling one interrupt costs 2 microseconds of CPU time. Compare the CPU cost of transferring 4 KB with interrupt-driven I/O versus DMA (DMA setup 5 microseconds, one completion interrupt).

SOLUTION

Interrupt-driven:

$$4096 \text{ bytes} \times 1 \text{ interrupt each} = 4096 \text{ interrupts}$$

$$4096 \times 2 \text{ us} = 8192 \text{ us} = 8.192 \text{ ms of pure CPU time}$$

DMA:

$$5 \text{ us (setup)} + 2 \text{ us (one completion interrupt)} = 7 \text{ us}$$

Ratio:

$$\frac{8192}{7} \approx 1170$$

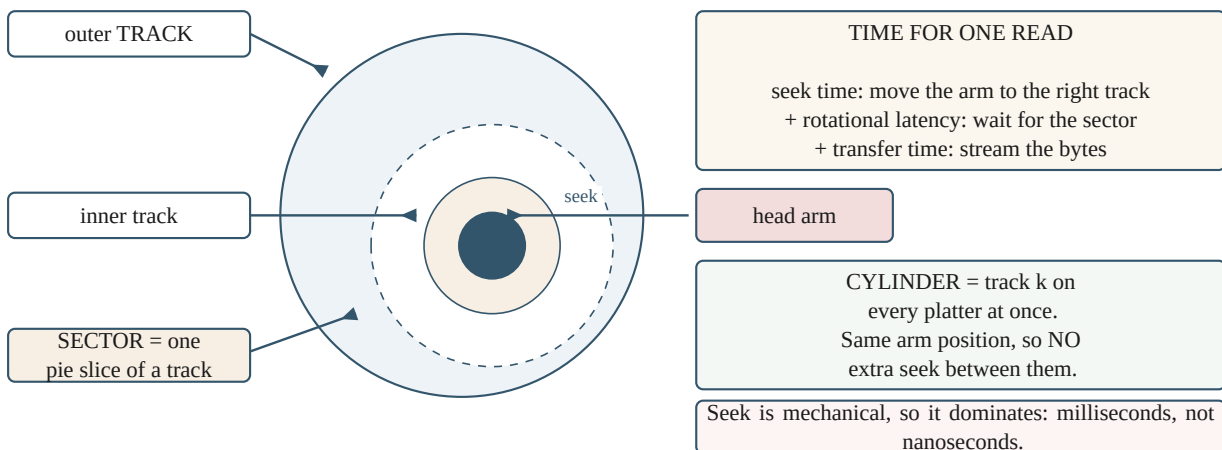
Answer: 8.192 ms of CPU versus 7 microseconds — DMA uses about 1170 times less CPU. And that is for a single 4 KB read.

Say the follow-up before you are asked: DMA does not make the **disk** faster. The transfer still takes the same milliseconds. DMA frees the **CPU** during those milliseconds, which is what raises whole-system throughput.

Buffering, caching, spooling — three different words

Buffering	Holding data in memory while a transfer is in progress — to smooth speed mismatches, to allow a different transfer size, and to keep a stable copy while I/O runs.
Caching	Keeping a copy of data you already fetched, in the hope of reusing it. The page cache is the big one.
Spooling	Queueing whole jobs for a device that cannot interleave them — a printer. Jobs go to disk first and are fed one at a time.

Disk geometry and the time it really takes



Disk geometry. A cylinder is the same track number on every platter — reachable with no extra seek, which is why file systems try to keep an inode and its blocks in one cylinder group.

DISK ACCESS TIME

$$T = T_{\text{seek}} + T_{\text{rotation}} + T_{\text{transfer}}$$

- T_{seek} — move the arm. Quoted as an **average**; 4 to 10 ms on a hard disk.
- T_{rotation} — wait for the sector to come under the head. On average **half a revolution**:

$$T_{\text{rotation}} = \frac{1}{2} \times \frac{60}{\text{RPM}} \text{ seconds} = \frac{30000}{\text{RPM}} \text{ milliseconds}$$

- T_{transfer} — bytes divided by the transfer rate.

Memorise two: 7200 RPM -> one revolution is 8.33 ms, so average latency **4.17 ms**. 15,000 RPM -> one revolution is 4 ms, so average latency **2 ms**.

Example 9 [TCS NQT · pattern]

A disk spins at 7200 RPM, has an average seek of 5 ms, and transfers at 100 MB/s. Find the time for one random 4 KB read, and the IOPS (random reads per second).

SOLUTION

Step 1 — rotational latency. One revolution:

$$\frac{60}{7200} = 0.008333 \text{ s} = 8.333 \text{ ms}$$

Average latency is half of that:

$$\frac{8.333}{2} = 4.167 \text{ ms}$$

Step 2 — transfer time for 4096 bytes at 100 MB/s:

$$\frac{4096}{100 \times 10^6} = 0.00004096 \text{ s} = 0.041 \text{ ms}$$

Step 3 — add them up:

$$T = 5 + 4.167 + 0.041 = 9.208 \text{ ms}$$

Step 4 — IOPS:

$$\frac{1000}{9.208} = 108.6$$

Answer: About 9.21 ms per random 4 KB read, so roughly 109 IOPS.

Now look at where the time went: 5 ms seek, 4.17 ms rotation, 0.04 ms of actual data transfer. **Over 99.5% of the time was spent waiting, not reading.** That single ratio is the justification for every disk-scheduling algorithm in the next section, and for why databases obsess about sequential access.

Example 10 [Agoda · pattern]

Same disk. Compare the **effective** throughput when reading 1000 separate 4 KB blocks scattered randomly, versus reading one contiguous 4 MB file.

SOLUTION

Random — 1000 separate reads:

$$1000 \times 9.208 \text{ ms} = 9208 \text{ ms} = 9.21 \text{ seconds}$$

Data moved: $1000 \times 4 \text{ KB} = 4 \text{ MB}$.

$$\text{throughput} = 4 \frac{\text{MB}}{9.21} \text{ s} = 0.43 \text{ MB/s}$$

Sequential — one 4 MB read: one seek, one rotation, then stream.

$$\text{transfer} = \frac{4 \times 1048576}{100 \times 10^6} = 0.04194 \text{ s} = 41.94 \text{ ms}$$

$$T = 5 + 4.167 + 41.94 = 51.11 \text{ ms}$$

$$\text{throughput} = 4 \frac{\text{MB}}{0.05111} \text{ s} = 78.3 \text{ MB/s}$$

Ratio:

$$\frac{78.3}{0.43} \approx 182$$

Answer: 0.43 MB/s random versus 78.3 MB/s sequential — the same disk, the same 4 MB, 182 times slower. Same bytes, different order.

This is **the** number to quote when an interviewer asks why a database adds an index, why logs are append-only, or why a full table scan can beat an index on a large range query.

Disk scheduling

Requests pile up in a queue. Reordering them to reduce arm movement is nearly free and pays enormously, because seek time dominates.

We will use one problem for all six algorithms so they can be compared fairly.

THE SIX ALGORITHMS

- **FCFS** — serve in arrival order. Fair, no starvation, terrible seek totals.
- **SSTF** (shortest seek time first) — always go to the nearest pending request. Good totals, but **can starve** a request at the far end of the disk.
- **SCAN** (elevator) — sweep in one direction servicing everything, **go all the way to the end of the disk**, reverse, sweep back.
- **C-SCAN** — sweep up servicing everything, go to the end, then **jump straight back to block 0** without servicing, and sweep up again. Gives more uniform waiting times.
- **LOOK** — like SCAN, but **turn around at the last request**, not at the edge of the disk.
- **C-LOOK** — like C-SCAN, but jump back to the **lowest request**, not to block 0.

The jump in C-SCAN and C-LOOK **is counted** as head movement unless the question says otherwise. State your assumption in the answer.

Example 11 [TCS Digital · pattern]

Disk cylinders are numbered 0 to 199. The head is at **95**. The pending queue, in arrival order, is:

86, 147, 22, 91, 177, 40, 128

The head is currently moving **towards higher** cylinder numbers. Compute the total head movement for FCFS, SSTF, SCAN, C-SCAN, LOOK and C-LOOK.

SOLUTION

First, sort the queue once — every algorithm except FCFS needs it:

22, 40, 86, 91, 128, 147, 177

Requests **above** 95: 128, 147, 177. Requests **below** 95: 22, 40, 86, 91.

FCFS — arrival order, no thinking:

from	95	86	147	22	91	177	40
to	86	147	22	91	177	40	128
move	9	61	125	69	86	137	88

$$9 + 61 + 125 + 69 + 86 + 137 + 88 = 575$$

SSTF — always the nearest pending request:

95 → 91(4) → 86(5) → 128(42) → 147(19) → 177(30) → 40(137) → 22(18)

$$4 + 5 + 42 + 19 + 30 + 137 + 18 = 255$$

Notice the 137 jump near the end. SSTF picked off everything nearby and left 40 and 22 stranded — with a steady stream of new requests near 90, those two could wait forever. That is **starvation**.

SCAN — go up to 199 first, then come back down:

95 → 128(33) → 147(19) → 177(30) → 199(22) → 91(108) → 86(5) → 40(46) → 22(18)

$$33 + 19 + 30 + 22 + 108 + 5 + 46 + 18 = 281$$

C-SCAN — up to 199, jump to 0, then up again:

95 → 128(33) → 147(19) → 177(30) → 199(22) → 0(199) → 22(22) → 40(18) → 86(46) → 91(5)

$$33 + 19 + 30 + 22 + 199 + 22 + 18 + 46 + 5 = 394$$

LOOK — SCAN without touching 199:

95 → 128(33) → 147(19) → 177(30) → 91(86) → 86(5) → 40(46) → 22(18)

$$33 + 19 + 30 + 86 + 5 + 46 + 18 = 237$$

C-LOOK — C-SCAN without touching 199 or 0:

95 → 128(33) → 147(19) → 177(30) → 22(155) → 40(18) → 86(46) → 91(5)

$$33 + 19 + 30 + 155 + 18 + 46 + 5 = 306$$

Summary:

Algorithm	Total movement	Comment
LOOK	237	Best here. No wasted travel to the disk edges.
SSTF	255	Close behind, but it can starve far-away requests.
SCAN	281	Pays 22 cylinders to reach 199 for nothing.
C-LOOK	306	Pays the jump, buys fair waiting times.
C-SCAN	394	Pays the jump and both edges.
FCFS	575	Worst, by a factor of 2.4 over LOOK.

Answer: FCFS 575 · SSTF 255 · SCAN 281 · C-SCAN 394 · LOOK 237 · C-LOOK 306 cylinders.

diskscan.js — all six, so you can check any question

JS

```
const head = 95, MIN = 0, MAX = 199;
const queue = [86, 147, 22, 91, 177, 40, 128];

const travel = (path) =>
  path.slice(1).reduce((sum, x, i) => sum + Math.abs(x - path[i]), 0);

const sorted = [...queue].sort((a, b) => a - b); // numeric sort, NOT plain
sort()
const up = sorted.filter(x => x >= head);
const down = sorted.filter(x => x < head);

const paths = {
  FCFS: [head, ...queue],
  SSTF: (() => {
    const left = [...queue]; let cur = head; const out = [head];
    while (left.length) {
      let bi = 0;
      for (let i = 1; i < left.length; i++)
        if (Math.abs(left[i] - cur) < Math.abs(left[bi] - cur)) bi = i;
      cur = left[bi]; out.push(cur); left.splice(bi, 1);
    }
    return out;
  })(),
  SCAN: [head, ...up, MAX, ...[...down].reverse()],
  'C-SCAN': [head, ...up, MAX, MIN, ...down],
  LOOK: [head, ...up, ...[...down].reverse()],
  'C-LOOK': [head, ...up, ...down],
};
for (const [name, p] of Object.entries(paths))
  console.log(name.padEnd(7), travel(p).toString().padStart(4), p.join(' -> '));
```

Real output from node diskscan.js:

```
FCFS      575 95 -> 86 -> 147 -> 22 -> 91 -> 177 -> 40 -> 128
SSTF      255 95 -> 91 -> 86 -> 128 -> 147 -> 177 -> 40 -> 22
SCAN      281 95 -> 128 -> 147 -> 177 -> 199 -> 91 -> 86 -> 40 -> 22
C-SCAN    394 95 -> 128 -> 147 -> 177 -> 199 -> 0 -> 22 -> 40 -> 86 -> 91
LOOK      237 95 -> 128 -> 147 -> 177 -> 91 -> 86 -> 40 -> 22
C-LOOK    306 95 -> 128 -> 147 -> 177 -> 22 -> 40 -> 86 -> 91
```

TRAP

`[...queue].sort()` without a comparator sorts **lexicographically**: `[86, 147, 22]` becomes `[147, 22, 86]`, because `"147" < "22"` as strings. In a disk-scheduling question that silently produces a wrong answer. Always write `.sort((a, b) => a - b)`.

TRAP

“C-SCAN is better than SCAN because it moves less.” It usually moves **more** — it pays for the return jump. What C-SCAN buys is **fairness**: in SCAN, a cylinder just behind the head waits for a full sweep out and back, while the edges get served twice per cycle. C-SCAN gives every cylinder the same wait. Say “uniform waiting time”, not “less movement”.

Example 12 [Google · pattern]

Your fleet moves from hard disks to SSDs. The storage team wants to keep the same LOOK scheduler because “it was the best one”. What do you tell them, and what would you use instead?

SOLUTION

What changed. LOOK exists to minimise **arm movement**. An SSD has no arm. Its access time to logical block 5 and logical block 5,000,000 is essentially identical — there is no seek, no rotation, just a flash-page read of roughly 0.1 ms. So the entire objective function that LOOK optimises has become a constant.

What LOOK now costs you. It still **reorders** requests, so it adds latency and CPU work, and it can delay a request that was already ready. Worse, it destroys the request order that the SSD’s own internal parallelism would exploit — a good SSD wants **many requests in flight at once** spread across its independent flash channels, and a sorting scheduler serialises them into a neat sequence that uses one channel at a time.

What to use instead.

- none / noop — a simple FIFO queue. Correct default for fast NVMe devices; the device does its own scheduling far better than the OS can.
- mq-deadline — FIFO plus a deadline per request, so nothing starves, and reads are prioritised over writes. Use when you need latency guarantees.
- Raise the **queue depth** so the device can work on many requests in parallel. This matters far more than the ordering.

The number that settles it. Our hard disk did about 109 random 4 KB IOPS. A consumer NVMe SSD does hundreds of thousands. When a single request costs 0.1 ms instead of 9.2 ms, a scheduler that spends even 0.05 ms deciding has eaten half your latency.

Answer: Disk scheduling optimises seek time, and SSDs have no seek. Switch to none for NVMe or mq-deadline where fairness matters, and increase queue depth so the device’s internal parallelism is used.

SSDs — a different set of rules

WHAT FLASH FORCES ON YOU

1. **Read and write in pages** (e.g. 4 KB), but **erase in blocks** (e.g. 256 pages together).
2. A page cannot be overwritten in place. To change one page you must erase the whole block it is in — so the controller instead writes the new version **somewhere else** and marks the old page invalid.
3. Each block tolerates a limited number of erase cycles.

Three consequences you must be able to name:

- **FTL (flash translation layer)** — the controller’s own map from logical block address to physical page. An SSD is running its own little file system inside itself.
- **Wear levelling** — spread erases evenly so no block dies early.

- **Write amplification** — writing 4 KB can cause far more than 4 KB of physical writes, because valid pages must be copied out before a block is erased.
- **TRIM** — the file system telling the SSD “these blocks hold deleted data”. Without TRIM the SSD faithfully copies dead data around during garbage collection forever.

	Hard disk (7200 RPM)	NVMe SSD
Random 4 KB read	≈ 9.2 ms	≈ 0.1 ms
Random 4 KB IOPS	≈ 109	100,000+
Sequential throughput	≈ 80 MB/s	2000+ MB/s
Random vs sequential gap	≈ 180 ×	small, but not zero
Wears out from	mechanical failure	erase cycles
Best OS scheduler	LOOK / C-LOOK	none or mq-deadline

RAID

RAID IN ONE BOX

Combine n disks so they look like one, for **speed**, **redundancy**, or both.

- **RAID 0 — striping.** Data split across all disks. Capacity $n \times c$. Speed: great. Redundancy: **none** — one failure loses everything. (Strictly, not redundant at all; the “R” is a lie here.)
- **RAID 1 — mirroring.** Every disk has a twin. Capacity $nc/2$. Reads can be served by either twin; writes go to both. Survives one failure per mirror pair.
- **RAID 5 — striping with distributed parity.** One disk’s worth of capacity goes to parity, spread across all disks. Capacity $(n - 1)c$. Survives **one** failure.
- **RAID 6 — two parity blocks.** Capacity $(n - 2)c$. Survives **two** failures. Used because rebuilding a big RAID 5 array takes so long that a second failure during the rebuild is a real risk.
- **RAID 10 — mirror, then stripe.** Capacity $nc/2$. Fast, safe, expensive. The usual choice for databases.

Parity is XOR. $P = D_0 \oplus D_1 \oplus D_2$. If D_1 dies, $D_1 = D_0 \oplus D_2 \oplus P$. That is the whole mathematics of RAID 5.

	Disk 0	Disk 1	Disk 2	Disk 3	
stripe 0	A0	A1	A2	P(A)	P(A) = A0 xor A1 xor A2 Lose disk 1? A1 = A0 xor A2 xor P(A)
stripe 1	B0	B1	P(B)	B2	
stripe 2	C0	P(C)	C1	C2	SMALL WRITE = 4 I/Os read old data, read old parity, write data, write parity
stripe 3	P(D)	D0	D1	D2	

Capacity with 4 disks of size c : RAID 0 = $4c$ (no safety) · RAID 5 = $3c$ (survives 1) · RAID 6 = $2c$ (survives 2) · RAID 10 = $2c$ (fast + safe)

RAID 5 with 4 disks. Parity rotates so no single disk becomes a write bottleneck. Any one column can be deleted and rebuilt from the other three.

```

const D0 = 0b1011, D1 = 0b0110, D2 = 0b1100;
const P = D0 ^ D1 ^ D2;
const bin = n => n.toString(2).padStart(4, '0');
console.log('D0', bin(D0), 'D1', bin(D1), 'D2', bin(D2), '=> P', bin(P));

// disk 1 dies. Rebuild D1 from everything that survived.
const rebuilt = D0 ^ D2 ^ P;
console.log('rebuilt D1 =', bin(rebuilt), '| matches:', rebuilt === D1);

// and the whole stripe still XORs to zero
console.log('D0^D1^D2^P =', bin(D0 ^ D1 ^ D2 ^ P));

```

Real output from node raid.js:

```

D0 1011 D1 0110 D2 1100 => P 0001
rebuilt D1 = 0110 | matches: true
D0^D1^D2^P = 0000

```

The third line is the invariant a RAID controller checks: **data XOR parity is always zero**.

Example 13 [DBS · pattern]

Eight disks of 4 TB each. Give the usable capacity and the number of simultaneous disk failures survived for RAID 0, 1, 5, 6 and 10. Then say which you would choose for a write-heavy transactional database, and why.

SOLUTION

Level	Usable capacity	Survives	Small-write cost
RAID 0	$8 \times 4 = 32$ TB	0 failures	1 write
RAID 1 (4 mirrored pairs)	$8 \times 4/2 = 16$ TB	1 per pair	2 writes
RAID 5	$(8 - 1) \times 4 = 28$ TB	1 failure	4 I/Os
RAID 6	$(8 - 2) \times 4 = 24$ TB	2 failures	6 I/Os
RAID 10	$8 \times 4/2 = 16$ TB	1 per mirror (up to 4 if lucky)	2 writes

Why RAID 5's small write costs 4 I/Os. To change one data block you must:

1. read the **old** data block,
2. read the **old** parity block,
3. compute $P_{\text{new}} = P_{\text{old}} \oplus D_{\text{old}} \oplus D_{\text{new}}$,
4. write the new data block,
5. write the new parity block.

That is 2 reads + 2 writes for one logical write. This is the **RAID 5 write penalty**. RAID 6 has two parity blocks, so it is 3 reads + 3 writes = 6 I/Os.

Choice for a write-heavy transactional database: RAID 10. You give up half your raw capacity, but a small write costs 2 writes with no read-modify-write cycle, and a rebuild copies one disk instead of recomputing parity across seven. On this array that is 16 TB usable instead of 28 TB — you are paying 12 TB for latency.

Answer: RAID 0: 32 TB / 0 · RAID 1: 16 TB / 1 per pair · RAID 5: 28 TB / 1 · RAID 6: 24 TB / 2 · RAID 10: 16 TB / 1 per mirror. Pick RAID 10 for write-heavy OLTP because it avoids the 4-I/O parity write penalty.

TRAP

“RAID is a backup.” **No.** RAID protects against a **disk** dying. It does not protect against DROP TABLE, ransomware, a bad deployment, fire, or theft — every mirror faithfully copies your mistake instantly. Backups are separate, offline, and tested.

Access methods, directories and permissions

How a program reads a file

Method	How it works	Where you see it
Sequential	Read or write from the current position; the position advances. rewind to start again.	Almost everything: text files, logs, streams, compilers.
Direct (random)	Treat the file as numbered records; jump to record n with seek .	Databases, index files, anything with fixed-size records.
Indexed sequential	A separate index file maps a key to a record position; look up the index, then jump.	Old ISAM systems; the idea survives as the database index.

How a directory is searched

A directory search happens on **every** path component of **every** `open()`, so its cost matters enormously in a directory with many files.

Example 14 [LINE MAN · pattern]

A directory holds 10,000 files. Each entry averages 40 bytes. Block size is 4 KB. Compare the disk reads for a name lookup under (a) an unsorted linear list, (b) a hashed directory, (c) a B-tree (H-tree) directory.

SOLUTION

Step 1 — how many blocks does the directory occupy?

$$10000 \times 40 = 400,000 \text{ bytes}$$

$$\frac{400000}{4096} = 97.66 \rightarrow 98 \text{ blocks}$$

(a) Linear list. You must scan entries until you find the name.

- Average (name present): half the blocks = 49 reads.
- Worst case, and **every** failed lookup: all **98** reads.

Note that a failed lookup is the **expensive** case, and failed lookups happen on every file **creation** (the OS must check the name is not taken). Creating 10,000 files in one directory therefore costs on the order of $10000 \times 49 \approx 490,000$ block reads in total. This is exactly why “too many files in one directory” is slow on old file systems.

(b) Hashed directory. Hash the name to a bucket; read that bucket.

- 1 read, plus one more if the bucket overflowed. Call it **1 to 2 reads**.
- Cost: the hash function fixes the number of buckets, so growing the directory means rehashing everything.

(c) B-tree / H-tree. One index block at the top, leaf blocks below.

- With 98 leaf blocks, one index block can hold pointers to all of them.
- 1 read for the index + 1 read for the leaf = **2 reads**.
- And unlike hashing, it stays balanced as the directory grows, and it keeps entries in sorted order.

Answer: Linear: 49 reads average, 98 for any miss. Hashed: 1-2. B-tree: 2, and it scales. Modern file systems (ext4's H-tree, XFS, NTFS) all use the tree.

Permissions, umask and the three special bits

THE OCTAL PERMISSION MODEL

Nine bits, three groups of three: **user**, **group**, **other**; each group is **read (4)**, **write (2)**, **execute (1)**.

$$7 = \text{rwx} \quad 6 = \text{rw-} \quad 5 = \text{r-x} \quad 4 = \text{r--} \quad 0 = \text{---}$$

On a **directory** the letters mean something different:

- **r** — you may **list** the names inside.

- w — you may **create or delete** entries inside.
- x — you may **traverse** it (use it in a path). Without x you cannot reach anything below, even if you know the exact name.

umask is a mask of bits to **remove**:

$$\text{final mode} = \text{base mode} \wedge \neg \text{umask}$$

Base is 666 for a new file (never executable) and 777 for a new directory.

Example 15 [Cognizant · pattern]

(a) What does mode 754 allow? (b) With `umask 022`, what mode does a newly created file get, and a newly created directory? (c) A directory is mode 711. Can another user read a file inside it called `secret.txt` if that file is mode 644?

SOLUTION

(a) Split 754 into three digits: 7, 5, 4.

- user = 7 = 4 + 2 + 1 = `rwX`
- group = 5 = 4 + 0 + 1 = `r-X`
- other = 4 = 4 + 0 + 0 = `r--`

So `rwXr-Xr--`: the owner can do everything, the group can read and execute but not modify, everyone else can only read.

(b) `umask 022` removes write permission for group and other.

- File: 666 AND NOT 022. Digit by digit: 6 − 0 = 6, 6 − 2 = 4, 6 − 2 = 4. Result **644** (`rw-r--r--`).
- Directory: 777 AND NOT 022 = 7, 5, 5. Result **755** (`rwXr-Xr-X`).

(c) Directory mode 711 is `rwX--X--X`. Other users have x but not r.

- No r means they **cannot list** the directory — `ls` fails.
- But x means they **can traverse** it. If they already know the exact name, they can open `secret.txt`, and 644 lets them read it.

Answer: (a) `rwXr-Xr--` (b) files 644, directories 755 (c) Yes — they cannot list the directory but they can still open a file whose name they know. Mode 711 hides names, not contents.

Special bit	On a file	On a directory
setuid (4000)	The program runs with the file owner's privileges, not the caller's. This is how <code>passwd</code> can edit a root-owned file.	No effect on Linux.
setgid (2000)	Runs with the file's group.	New files inside inherit the directory's group — used for shared project folders.
sticky bit (1000)	Historical (keep the program in swap); now ignored.	Only the owner of a file may delete it , even if everyone can write to the directory. This is what makes <code>/tmp</code> (mode 1777) safe.

TRAP

`/tmp` is world-writable, so anyone can delete my file there.” Not with the sticky bit set (`drwxrwxrwt` — note the final `t`). Everyone may **create** files, but only the file's owner may **remove** theirs. Without the sticky bit, a world-writable directory lets any user delete any other user's files, because deletion is a **directory** write, not a **file** write.

Sparse files: size is not space

A file's size field says where the last byte is. It does not say how many blocks were allocated. Skip forward with seek and write, and the skipped region becomes a **hole**: no blocks are allocated, and reads of the hole return zeros.

Example 16 [Sea/Shopee · pattern]

Run this, then explain the two different numbers:

```
dd if=/dev/zero of=sparse.dat bs=1 count=1 seek=1073741824
ls -l sparse.dat
du --block-size=1 sparse.dat
stat -c '%n size=%s blocks=%b blocksize=%B' sparse.dat
```

SOLUTION

Real output from a Linux shell:

```
-rw-rw-r-- 1 zayed zayed 1073741825 Sep 15 22:22 sparse.dat
4096      sparse.dat
sparse.dat size=1073741825 blocks=8 blocksize=512
```

Read the three lines carefully.

- `ls -l` reports **1,073,741,825** bytes — that is 1 GiB plus the one byte we wrote. This is the **size** field in the inode: the offset of the last byte, plus one.
- `du` reports **4096** bytes — the actual disk space used. One 4 KB block.
- `stat` explains it: `blocks=8` in units of `blocksize=512`, and $8 \times 512 = 4096$.

So the file **claims** 1 GiB and **occupies** 4 KB. The gigabyte in between is a hole: the inode simply has no pointer for those blocks, and a read there returns zeros without any disk access at all.

Answer: Size 1,073,741,825 bytes, space used 4096 bytes. Size is where the last byte sits; space is how many blocks the inode actually points at.

Two practical consequences to mention:

1. `cp` a sparse file carelessly and you get a **dense** 1 GiB copy — the zeros get written out for real. Use `cp --sparse=always`.
2. Virtual-machine disk images and database files are usually sparse. That is why `df` and `du` often disagree with what an application reports.

The page cache, and what `fsync` really costs

WRITE-BACK, READ-AHEAD, AND DURABILITY

Every read and write goes through the **page cache** — RAM holding recently used file blocks.

- **Read hit** — no disk access at all.
- **Read-ahead** — on detecting sequential reads, the kernel fetches the **next** blocks before you ask. This is why sequential reading is so much faster than the disk numbers suggest.
- **Write-back** (the default) — a `write()` only marks a page dirty in RAM and returns immediately. The kernel flushes it later. Fast, but a crash loses the data.
- **Write-through** — every write goes to disk before returning. Safe, slow.
- **`fsync(fd)`** — force this file's dirty pages **and** its metadata to durable storage, and do not return until the device confirms. This is the only way an application gets a real durability guarantee.

Example 17 [Goldman Sachs · pattern]

A service appends a 4 KB record to a log file and calls `fsync()` after each one. On a 7200 RPM disk with 4 ms average seek it manages about 100 records per second. Explain that number from first principles, and then explain why removing `fsync()` gives roughly 50,000 records per second — and what is lost.

SOLUTION

Where 100/second comes from. Each `fsync()` forces a real trip to the platter, and there are **two** structures to update: the journal commit record, and the file's data plus inode.

One durable write costs, at minimum:

$$\begin{aligned} T &= T_{\text{seek}} + T_{\text{rotation}} + T_{\text{transfer}} \\ &= 4 + \frac{30000}{7200} + \frac{4096}{100 \times 10^6} \times 1000 \\ &= 4 + 4.167 + 0.041 = 8.21 \text{ ms} \end{aligned}$$

With the journal commit needing its own rotation, roughly 9-10 ms per record:

$$1000 \frac{\text{ms}}{10} = 100 \text{ records per second}$$

The number is not about bandwidth at all — the disk moved 4 KB in 0.04 ms. It is about **waiting for the platter to come round**, twice.

Why removing `fsync()` gives 50,000/second. Without it, `write()` copies 4 KB into the page cache and returns. That is a memory copy of a few microseconds. The kernel later flushes thousands of accumulated pages in a handful of large sequential writes, so the per record disk cost collapses.

What is lost. Durability. The application has been told “written” for records that exist only in RAM. A power cut or kernel panic loses up to 30 seconds of “successful” writes. Note carefully: a **process** crash loses nothing (the page cache belongs to the kernel), but a **machine** crash does. Interviewers like that distinction.

The engineering answer is neither extreme: **group commit**. Collect records for a few milliseconds, then `fsync()` once for the whole batch.

$$\text{throughput} \approx 100 \text{ fsyncs/second} \times \text{batch size}$$

With a batch of 200 records you get about 20,000 durable records per second, and the worst case latency for any single record is only the batching window plus 10 ms.

Answer: 100/s is two rotational waits per record (10 ms), not a bandwidth limit. Dropping `fsync()` buys 500x by giving up durability on machine crash. Group commit gets most of the speed while keeping durability, at the cost of a few milliseconds of latency.

SHORTCUT

Any time an interview question says “slow, but the data is small”, stop thinking about bandwidth and start counting **round trips**: seeks, rotations, `fsync` calls, network round trips. Small-and-slow is always a latency problem, never a throughput one.

Mounting and the VFS

HOW SEVERAL FILE SYSTEMS BECOME ONE TREE

Mounting attaches the root of one file system onto a directory (the **mount point**) of another. After `mount /dev/sdb1 /data`, a path starting `/data/...` is resolved by the **second** file system's code, using **its** inode numbers and **its** superblock.

Anything that was previously inside `/data` is **hidden**, not deleted — it reappears when you unmount.

The **VFS (virtual file system)** is the kernel layer that makes this possible: it defines a common interface (`open`, `read`, `write`, `lookup`, ...) that every file system implements. `ext4`, `XFS`, `NTFS`, a network file system, and even `/proc` (which has no disk at all) all plug into the same interface. That is why one `read()` call works on all of them.

Layer	What it decides
-------	-----------------

Application	Which file, which bytes.
System call / VFS	Which file system owns this path; permission checks; the page cache.
File system (ext4, XFS, ...)	Inode to block-number translation; allocation; journaling.
Block layer	Queueing, merging adjacent requests, the I/O scheduler.
Device driver	Turning a block request into device commands; DMA setup.
Hardware	The actual read.

Say this when asked “what happens when I run `cat file.txt`”: name the layers from the top down and say one sentence for each. Six sentences, in order, is a complete answer — and far better than a long story about one layer.

Practice

PRACTICE · WARM-UP target: 6 min

1. Where is a file’s name stored — in the inode or in the directory?
2. A disk spins at 6000 RPM. What is the average rotational latency?
3. Which allocation method gives $O(1)$ random access but suffers external fragmentation?
4. You delete a file that is still open by a running process. Is the space freed?
5. Which RAID level gives no redundancy at all?

ANSWER KEY

1. In the **directory**, as a (name, inode number) entry. The inode holds everything else.
2. One revolution = $60/6000 = 10$ ms. Average latency is half: **5 ms**.
3. **Contiguous** allocation.
4. **No**. The link count reached 0, but the open-file reference has not. Space is freed when the process closes it or exits.
5. **RAID 0** — pure striping.

PRACTICE · TIER 1 target: 15 min

1. Block size 8 KB, pointer size 4 bytes, 10 direct pointers, plus one single and one double indirect. Largest file?
2. A 2 TB disk with 4 KB blocks. Size of the free-space bitmap?
3. Head at 60, queue 25, 140, 78, 12, 190, 95, cylinders 0-199, moving upward. Total movement for FCFS and for SSTF.
4. A disk has 10 ms average seek, 5400 RPM, 80 MB/s transfer. Time for one random 8 KB read?
5. Explain in two lines why a hard link cannot cross a file system boundary.

ANSWER KEY

1. Pointers per block = $8192/4 = 2048$. Blocks = $10 + 2048 + 2048^2 = 10 + 2048 + 4194304 = 4196362$. Bytes = $4196362 \times 8192 = 34376277504$, which is about **32 GiB**.
2. Blocks = $2 \times 1024^4 / 4096 = 536870912$. Bits to bytes: $/8 = 67108864$ bytes, so **64 MB**.
3. **FCFS**: $60 \rightarrow 25(35) \rightarrow 140(115) \rightarrow 78(62) \rightarrow 12(66) \rightarrow 190(178) \rightarrow 95(95)$; total = $35 + 115 + 62 + 66 + 178 + 95 = 551$ — **551**. **SSTF**: $60 \rightarrow 78(18) \rightarrow 95(17) \rightarrow 140(45) \rightarrow 190(50) \rightarrow 25(165) \rightarrow 12(13)$; total = $18 + 17 + 45 + 50 + 165 + 13 = 308$ — **308**.
4. Rotation: $60/5400 = 11.11$ ms, half = **5.56 ms**. Transfer: $8192/(80 \times 10^6) = 0.000102$ s = **0.10 ms**. Total = $10 + 5.56 + 0.10 = 15.66$ ms — **15.66 ms**.
5. A hard link is an inode **number**, and inode numbers are only unique **within one file system**. Inode 431 on the root file system and inode 431 on a mounted USB stick are different files, so the link would be ambiguous. A soft link stores a **path** string, which the kernel re-resolves from the root, so it crosses boundaries fine.

PRACTICE · TIER 2 target: 20 min

1. A file system uses 4 KB blocks. Compare the space wasted by 1,000,000 files averaging 3 KB each under 4 KB blocks and under 64 KB blocks.
2. A disk does 7200 RPM, 4 ms average seek, 150 MB/s. A query reads 200 rows scattered across 200 different blocks of 8 KB. How long? How long if the same 200 blocks were contiguous?
3. Head at 110, queue 34, 118, 21, 165, 92, 180, 55, cylinders 0-199, moving upward. Compute LOOK and C-LOOK.
4. Explain why data=ordered journaling never shows you another file's old bytes, while data=writeback can.

ANSWER KEY

1. **4 KB blocks:** each 3 KB file takes 1 block, wasting $4096 - 3072 = 1024$ B. Total = $10^6 \times 1024 = 1.024 \times 10^9$ B, about **0.95 GiB**. **64 KB blocks:** each file takes 1 block of 65,536 B, wasting $65536 - 3072 = 62464$ B. Total = 6.2464×10^{10} B, about **58.2 GiB**. Sixty-one times more waste, for the same files.
2. Rotation = $30000/7200 = 4.167$ ms. Transfer of 8 KB = $8192/(150 \times 10^6) = 0.0546$ ms. Per random read = $4 + 4.167 + 0.055 = 8.222$ ms; $\times 200 = 1644$ ms, about **1.64 s**. Contiguous: one seek + one rotation + transfer of $200 \times 8192 = 1638400$ B = 10.92 ms; total = $4 + 4.167 + 10.92 = 19.09$ ms — **19.09 ms**. Ratio: about **86 times**.
3. Sorted: 21, 34, 55, 92, 118, 165, 180. Above 110: 118, 165, 180. Below: 92, 55, 34, 21. **LOOK:** $110 \rightarrow 118(8) \rightarrow 165(47) \rightarrow 180(15) \rightarrow 92(88) \rightarrow 55(37) \rightarrow 34(21) \rightarrow 21(13)$; total = $8 + 47 + 15 + 88 + 37 + 21 + 13 = 229$ — **229**. **C-LOOK:** $110 \rightarrow 118(8) \rightarrow 165(47) \rightarrow 180(15) \rightarrow 21(159) \rightarrow 34(13) \rightarrow 55(21) \rightarrow 92(37)$; total = $8 + 47 + 15 + 159 + 13 + 21 + 37 = 300$ — **300**.
4. In **ordered** mode the file system **forces the data blocks to disk before** committing the metadata that points at them. So if a crash happens, either the metadata is not there (the block is still free — you see nothing) or it is there and the data is already correct. In **writeback** mode only metadata is ordered: the inode can commit, saying “block 5000 belongs to this file, size 4 KB”, while block 5000 still contains whatever the **previous** owner left there. After recovery you read that old content — possibly another user's deleted data. That is a security issue, not just a correctness one.

PRACTICE · TIER 3 target: 25 min

1. A log-structured file system writes **everything** — data and metadata — as one continuous append-only stream. Give two reasons this is fast, and the one problem it creates. How is that problem solved?
2. A 12-disk RAID 5 array of 8 TB disks is rebuilding after one failure. Estimate the rebuild time if the array can sustain 150 MB/s of rebuild traffic, and explain why this number is the main argument for RAID 6.
3. Your service writes a 4 KB record and calls `fsync()` after every write. Throughput is 100 writes/second on a hard disk. Explain the number, then give three different ways to make it faster and what each one risks.
4. Design the metadata layout for a file system that must store 100 million files averaging 2 KB. Say what you choose for block size and inode strategy, and justify both with numbers.

ANSWER KEY

1. **Fast because:** (1) every write is sequential, so seek and rotation are paid once per segment instead of once per update — from our numbers that is 9.2 ms down to about 0.04 ms per 4 KB. (2) Many small updates are batched into one large segment write, so a burst of tiny random writes becomes one big sequential one. **Problem:** old versions of blocks are scattered through the log as dead space, and the disk fills with garbage. **Solution:** a **cleaner** (garbage collector) reads old segments, copies out the still-live blocks, and frees the segment — which costs read and write bandwidth, and is why log-structured designs struggle when the disk is nearly full.
2. Rebuild must read all 11 surviving disks and write one full 8 TB disk. At 150 MB/s the write alone is $8 \times 10^{12}/(150 \times 10^6) = 53333$ s ≈ 14.8 **hours**, and in practice longer because production traffic competes. For nearly 15 hours the array has **no** redundancy, and it is doing a full read of 11 aging disks — exactly the stress that triggers a second failure. RAID 6 survives that second failure, which is why large arrays stopped using RAID 5.
3. `fsync()` forces the data to durable media, so the write cannot be absorbed by the page cache. Each one costs at least a seek + rotation + transfer, plus a second write for the journal: about 9-10 ms, giving roughly **100 per second**. Three fixes: (1) **Batch** — group many records into one `fsync()`; risk: you lose the whole batch on a crash, and latency for the first record in a

batch rises. (2) **Move the log to an SSD or a battery-backed write cache**; risk: cost, and a non-battery-backed cache that lies about durability will lose data silently. (3) **Relax durability** — `fdatasync()` instead of `fsync()` (skips the metadata flush) or accept “committed to OS, not to disk”; risk: a power failure loses acknowledged writes. Only (1) and (2) are safe; (3) is a deliberate trade.

4. **Block size**: 100 M files of 2 KB. With 4 KB blocks each file wastes about 2 KB, so $100 \times 10^6 \times 2048 = 2.05 \times 10^{11}$ B $\approx$ 190 GiB wasted; useful data is only 190 GiB, so **half the disk is waste**. Choose **1 KB blocks**: a 2 KB file uses 2 blocks with near-zero waste, at the cost of more pointers and slower sequential I/O — acceptable, because these files are tiny and never read sequentially in bulk. Better still, **inline small files inside the inode itself** (ext4 and NTFS both do this): a 2 KB file that fits in the inode costs **zero** data blocks and **one** read instead of two. **Inode strategy**: pre-allocate at least 100 M inodes — at 256 B each that is 25.6 GB of inode table, so it must be spread across cylinder groups so an inode sits near its data. Direct pointers matter here and indirect ones do not: with 1 KB blocks, 12 direct pointers already reach 12 KB, which covers essentially every file in this workload.

Rapid fire: one-line answers

SAY THESE IN ONE BREATH

1. **Where is a file’s name stored?** In the directory entry, not the inode.
2. **What is an inode?** A fixed-size metadata record: type, size, owner, permissions, timestamps, link count and block pointers.
3. **Block vs sector?** Sector is the hardware’s smallest unit (512 B or 4 KB); block is the file system’s allocation unit (usually 4 KB).
4. **Hard link vs soft link?** Hard link is another name for the same inode; soft link is a small file containing a path.
5. **Can a hard link cross file systems?** No — inode numbers are only unique within one file system. A soft link can.
6. **Why no hard links to directories?** They would create cycles, and the tree would need garbage collection.
7. **What does rm actually do?** Removes a directory entry and decrements the link count. The file dies at count 0 with no open handles.
8. **Contiguous allocation — one strength, one weakness?** $O(1)$ random access; external fragmentation and no growth.
9. **Linked allocation — one strength, one weakness?** No external fragmentation; random access to block k costs k reads.
10. **What is FAT?** Linked allocation with all “next” pointers moved into one table at the front of the disk.
11. **Why 12 direct pointers?** Almost every real file is small; direct pointers make small files cost one read.
12. **Reads to reach a byte through double indirect?** Three: double index, inner index, data block.
13. **Max file size formula?** $(\text{direct} + \text{ppb} + \text{ppb}^2 + \text{ppb}^3) \times \text{block size}$, where $\text{ppb} = \text{block size} / \text{pointer size}$.
14. **Bitmap vs linked free list?** Bitmap makes finding contiguous runs easy and is small; a free list finds one block easily but not a run.
15. **What is journaling for?** Crash consistency — write the intent to a sequential log and commit, so recovery replays a log instead of scanning the disk.
16. **What is a sparse file?** A file with unwritten holes; the size is large but no blocks are allocated for the holes.
17. **Disk access time formula?** Seek + rotational latency + transfer.
18. **Average rotational latency?** Half a revolution: $30000/\text{RPM}$ milliseconds.
19. **Why is sequential I/O so much faster?** One seek and one rotation are amortised over a large transfer instead of paid per block.
20. **SSTF’s weakness?** Starvation of requests far from the head.
21. **SCAN vs LOOK?** SCAN travels to the disk edge; LOOK turns around at the last request.
22. **Why C-SCAN, if it moves more?** Uniform waiting time — every cylinder is served once per sweep, in one direction.
23. **Best scheduler for an SSD?** none (FIFO) or mq-deadline — there is no seek to optimise.

24. **Polling vs interrupt vs DMA?** Polling burns the CPU waiting; interrupts cost one per word; DMA moves the whole block and interrupts once.
25. **Buffering vs caching vs spooling?** Buffering smooths a transfer in progress; caching keeps a reusable copy; spooling queues whole jobs for a serial device.
26. **RAID 0 / 1 / 5 / 6 / 10 in one word each?** Striping / mirroring / single parity / double parity / mirrored stripes.
27. **RAID 5 write penalty?** Four I/Os per small write: read old data, read old parity, write data, write parity.
28. **Is RAID a backup?** No. It survives disk failure, not deletion, corruption or fire.
29. **What is write amplification on an SSD?** Writing a little causes a lot of physical writes, because erase happens in large blocks and valid pages must be relocated first.
30. **What does TRIM do?** Tells the SSD which blocks hold deleted data so garbage collection stops copying them.

ONE-PAGE REVISION CARD

The three mappings. name → inode number (the directory) → inode (the metadata) → block numbers (the data). Every question is about one of them.

The inode formula. pointers per block = block size / pointer size. Max file = (direct + ppb + ppb² + ppb³) × block size. With 4 KB blocks and 4-byte pointers: 1024 ppb, and about **4 TiB**. Reads to a byte: direct 1, single 2, double 3, triple 4.

Links. Hard link = same inode, bumps link count, one file system only. Soft link = its own inode holding a path string, breaks if the target goes, crosses file systems.

Disk time. $T = T_{\text{seek}} + 30000/\text{RPM} + \text{bytes}/\text{rate}$. Memorise: 7200 RPM → 4.17 ms average latency; a random 4 KB read on such a disk is about **9.2 ms**, so about **109 IOPS**.

The sequential-versus-random number. Same disk, same 4 MB: **0.43 MB/s scattered versus 78 MB/s contiguous — about 180 times**. Quote this whenever indexes, logs or batching come up.

Disk scheduling, on head 95 with queue 86, 147, 22, 91, 177, 40, 128: LOOK 237 < SSTF 255 < SCAN 281 < C-LOOK 306 < C-SCAN 394 < FCFS 575. SSTF starves; C-SCAN and C-LOOK buy fairness, not distance.

RAID. 0 = speed, no safety. 1 = mirror. $5 = (n - 1)c$, survives 1, **4 I/Os per small write**. $6 = (n - 2)c$, survives 2. $10 = nc/2$, fast and safe. RAID is not a backup.

Three sentences that win marks.

1. “The file name is in the directory entry; the inode has everything except the name.”
2. “Over 99% of a random disk read is seek and rotation, which is why scheduling and sequential layout matter so much.”
3. “Disk scheduling optimises arm movement, so on an SSD it optimises nothing — use FIFO or a deadline scheduler and raise the queue depth.”

DBMS: Relational Model, Keys & SQL

Tables, keys, joins, NULLs and window functions — the half of the technical round that is pure vocabulary plus one query.

The DBMS round has two halves.

The first half is **vocabulary**. What is a candidate key? What is referential integrity? What is the difference between DELETE and TRUNCATE? These are 10-second answers. You either know them or you do not. There is no thinking time.

The second half is **one query**. The interviewer writes two small tables on the board and asks for the second-highest salary, or the departments with no employees, or the top earner per department. You get five minutes.

This chapter gives you both. Every SQL query in this chapter was run against a real database before it was printed, and the output you see is the real output.

How to read this chapter. Every worked example carries a coloured tier badge — grey for warm-up mechanics, teal for Tier 1 (TCS NQT, Accenture, Infosys, Wipro, Capgemini), amber for Tier 2 (Grab, Shopee, GIC, DBS, Agoda, SCB) and dark red for Tier 3 (Google, Amazon, Microsoft, Goldman Sachs, D. E. Shaw, Adobe). The practice sets at the end climb the same four steps in order, so do them in order.

WHAT YOU NEED TO KNOW

Relational vocabulary

- **Relation** = table. **Tuple** = row. **Attribute** = column. **Domain** = the set of legal values for one attribute.
- **Degree** = number of attributes (columns). **Cardinality** = number of tuples (rows).
- **Schema** = the design (names and types). **Instance** = the rows sitting in it right now.

Keys

- **Super key**: any attribute set whose value is different in every row.
- **Candidate key**: a super key with no useless attribute inside it (minimal).
- **Primary key**: the one candidate key you choose. Never NULL, never repeated.
- **Alternate key**: every candidate key you did not choose.
- **Foreign key**: an attribute set that must match a primary key somewhere (or be NULL).
- If one attribute A is known to be unique in a table of n attributes, the number of super keys containing A is 2^{n-1} .

Three-valued logic — SQL has TRUE, FALSE and UNKNOWN.

- Any comparison with NULL gives UNKNOWN. $\text{NULL} = \text{NULL}$ is UNKNOWN, not true.
- WHERE keeps only TRUE. It throws away FALSE and UNKNOWN.
- Test for null with IS NULL / IS NOT NULL only.

Logical order of a SELECT (not the written order): FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT.

Relational algebra → SQL

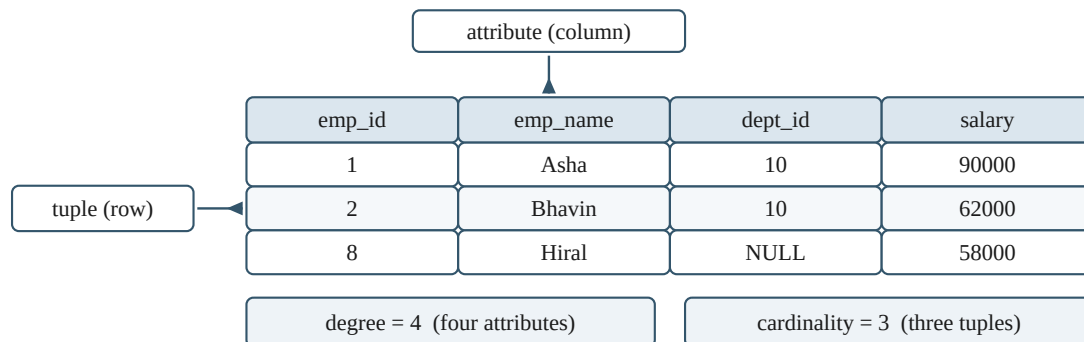
Operator	Name	SQL
$\sigma_c(R)$	select (rows)	WHERE c
$\pi_{a,b}(R)$	project (columns)	SELECT DISTINCT a, b
$R \times S$	cartesian product	FROM R, S
$R \bowtie_c S$	theta join	FROM R JOIN S ON c
$R \cup S$	union	UNION
$R \cap S$	intersection	INTERSECT
$R - S$	difference	EXCEPT

$R \div S$	division	double NOT EXISTS
------------	----------	-------------------

Relational algebra removes duplicates by definition. SQL does not, unless you say **DISTINCT** or use **UNION** instead of **UNION ALL**.

5.1 The relational model, in one picture

A relation is a table with rules. The rules are what the interviewer is testing.



Anatomy of a relation. Degree counts columns, cardinality counts rows.

The four rules of a relation

1. **Every value is atomic.** One cell holds one value. No lists, no “Hindi, English” in a single cell. (This is 1NF — Chapter 6.)
2. **No two rows are identical.** A true relation is a **set** of tuples. Real SQL tables break this rule if you forget a primary key.
3. **Row order means nothing.** Without **ORDER BY**, the database may hand rows back in any order it likes.
4. **Column order means nothing** to the model. Always name your columns; never rely on **SELECT *** position.

TRAP

“A table and a relation are the same thing.” Almost — but a SQL table **allows duplicate rows** and **allows nulls**, and a mathematical relation allows neither. Say: “A relation is a set of tuples; a SQL table is a multiset, so it can hold duplicates unless a key stops it.”

WARM-UP

build the reflex

Example 1 [warm-up]

A table **Book** has columns **isbn**, **title**, **author**, **price** and holds 500 rows. State its degree and cardinality.

SOLUTION

Degree counts **columns**: **isbn**, **title**, **author**, **price** $\Rightarrow$ 4. Cardinality counts **rows** $\Rightarrow$ 500.

Answer: degree 4, cardinality 500

Example 2 [warm-up]

Student(**roll**, **name**, **email**) has 0 rows. What is its degree and cardinality?

SOLUTION

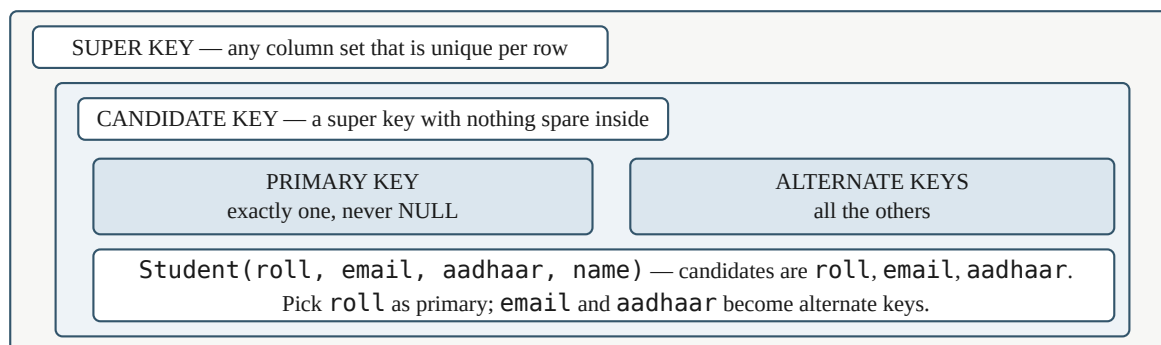
Degree is a property of the **schema**, so it stays 3 even with no data. Cardinality is a property of the **instance**, so it is 0.

Answer: degree 3, cardinality 0

Memory hook: “degree” and “design” both start with **de** — degree comes from the schema. “Cardinality” and “cards” both start with **card** — cardinality counts the cards (rows) in the deck right now.

5.2 Keys — the whole family

This is the single most asked DBMS definition set. Learn it as a nesting, not as a list.



Every candidate key is a super key. The primary key is one candidate key you pick; the rest become alternate keys.

Definitions you must be able to fire back in 10 seconds

Key	Meaning
Super key	Any set of attributes whose combined value never repeats. {roll} is one; so is {roll, name}; so is every column together.
Candidate key	A minimal super key. Drop any attribute and it stops being unique.
Primary key	The candidate key the designer chose. Unique + NOT NULL. One per table.
Alternate key	Every candidate key that was not chosen as primary.
Composite key	A key made of two or more attributes, e.g. {emp_id, proj_id}.
Foreign key	An attribute set in table A that must equal some primary key value in table B, or be entirely NULL.
Surrogate key	A meaningless number the system generates (AUTO_INCREMENT id). No business meaning, never changes.
Natural key	A key made of real business data (email, aadhaar, isbn). Meaningful, but can change.
Unique key	A uniqueness constraint that does allow NULL (usually many nulls).

TRAP

Three classic mix-ups:

1. “Primary key is the same as unique key.” No. Primary key forbids NULL; a unique constraint normally allows nulls — and in most engines it allows **many** nulls, because NULL != NULL.
2. “A candidate key must be a single column.” No. {emp_id, proj_id} in an assignment table is a perfectly good candidate key.
3. “A foreign key must point to a different table.” No. A self-referencing foreign key (manager_id → emp_id) points at the same table.

Counting super keys — the exam arithmetic

If you know that attribute *A* alone is unique, then **any** set that contains *A* is also unique. So count the ways to pick the other attributes.

Example 3 [TCS NQT · pattern]

R(A, B, C, D) and A is a candidate key (the only one). How many super keys does R have?

SOLUTION

Every super key must contain A. The remaining attributes are B, C, D — three of them, and each is independently in or out.

Number of subsets of $\{B, C, D\}$ is $2^3 = 8$.

List them to be sure: A, AB, AC, AD, ABC, ABD, ACD, ABCD — 8 sets.

Answer: $2^3 = 8$

Example 4 [Infosys · pattern]

R(P, Q, R, S, T) has **two** candidate keys: P and Q. How many super keys?

SOLUTION

A super key is any set that contains P **or** contains Q. Use complement counting.

Total subsets of 5 attributes = $2^5 = 32$.

Subsets that contain **neither** P nor Q = subsets of $\{R, S, T\} = 2^3 = 8$.

Super keys = $32 - 8 = 24$.

Check by inclusion–exclusion instead: sets containing P = $2^4 = 16$; containing Q = 16; containing both = $2^3 = 8$. So $16 + 16 - 8 = 24$. Same answer.

Answer: 24

SHORTCUT

Two candidate keys K_1, K_2 made of single attributes, n attributes total:

$$\text{super keys} = 2^n - 2^{n-2}$$

because the only non-super-keys are the subsets that avoid both key attributes. For $n = 5$: $32 - 8 = 24$. ✓

Example 5 [Shopee · pattern]

A table Ride(ride_id, driver_id, rider_id, started_at, fare) is used by a taxi app. Business rules: a ride id is globally unique; and a driver cannot start two rides at the same instant. List the candidate keys.

SOLUTION

Rule 1 gives $\{\text{ride_id}\} \rightarrow \text{everything}$. ride_id is unique, so $\{\text{ride_id}\}$ is a candidate key. Is it minimal? It is one attribute, so yes.

Rule 2 says $\{\text{driver_id}, \text{started_at}\}$ never repeats. So it is a super key. Is it minimal?

- Drop started_at: a driver has many rides, so $\{\text{driver_id}\}$ is not unique. ✗
- Drop driver_id: two drivers can start at the same instant, so $\{\text{started_at}\}$ is not unique. ✗

Neither half works alone, so $\{\text{driver_id}, \text{started_at}\}$ is minimal — a candidate key.

rider_id and fare repeat freely, so they add nothing.

Answer: $\{\text{ride_id}\}$ and $\{\text{driver_id}, \text{started_at}\}$. Pick ride_id as primary; the other is an alternate key.

Notice how the answer came only from the **business rules**, never from the sample rows. Sample data can only **disprove** a key (“look, this value repeats”); it can never prove one. Say this in the interview — it is a mark of someone who has actually designed a schema.

5.3 Integrity constraints

A constraint is a promise the database enforces for you, so that bad data cannot get in even if the application has a bug.

Constraint	What it guarantees
Domain	Each value belongs to the column's type and CHECK rule. <code>salary INTEGER CHECK (salary > 0)</code> .
Entity integrity	No part of a primary key is ever NULL. A row must be identifiable.
Referential integrity	A foreign key value either matches an existing primary key, or is fully NULL.
Key	No two rows share the same value of a PRIMARY KEY / UNIQUE column set.

The schema used for the rest of this chapter — this exact DDL was run

SQL

```
CREATE TABLE department (
  dept_id  INTEGER PRIMARY KEY,
  dept_name TEXT NOT NULL UNIQUE,
  city     TEXT
);

CREATE TABLE employee (
  emp_id    INTEGER PRIMARY KEY,
  emp_name  TEXT NOT NULL,
  dept_id   INTEGER REFERENCES department(dept_id),
  manager_id INTEGER REFERENCES employee(emp_id), -- self reference
  salary    INTEGER NOT NULL,
  joined_on TEXT
);

CREATE TABLE project (
  proj_id   INTEGER PRIMARY KEY,
  proj_name TEXT NOT NULL,
  dept_id   INTEGER REFERENCES department(dept_id)
);

CREATE TABLE assignment (
  emp_id   INTEGER REFERENCES employee(emp_id),
  proj_id  INTEGER REFERENCES project(proj_id),
  hours    INTEGER,
  PRIMARY KEY (emp_id, proj_id) -- composite primary key
);
```

The rows, once and for all:

emp_id	emp_name	dept_id	manager_id	salary	joined_on
1	Asha	10	NULL	90000	2019-03-01
2	Bhavin	10	1	62000	2020-07-15
3	Chetan	10	1	62000	2021-01-10
4	Divya	20	NULL	78000	2018-11-05
5	Eshan	20	4	45000	2022-02-20
6	Farida	30	4	52000	2021-09-01
7	Gopal	30	6	41000	2023-04-12
8	Hiral	NULL	NULL	58000	2023-06-01

dept_id	dept_name	city
10	Engineering	Pune

20	Sales	Mumbai
30	Support	Pune
40	Research	Bengaluru

proj_id	proj_name	dept_id
100	Atlas	10
101	Beacon	10
102	Comet	20
103	Delta	40

assignment(emp_id, proj_id, hours) holds: (1,100,120), (2,100,80), (3,101,150), (1,101,40), (4,102,200), (5,102,60), (6,100,30).

Three deliberate landmines in this data, because interviewers plant exactly these: **Hiral** has no department (dept_id is NULL); **Research** (40) has no employees; **Bhavin** and **Chetan** earn the same salary.

What happens when a parent row is deleted

You declare the behaviour on the foreign key:

ON DELETE RESTRICT	Refuse the delete while children exist. (NO ACTION behaves the same in most engines.)
ON DELETE CASCADE	Delete the children too.
ON DELETE SET NULL	Keep the children, blank out their foreign key. The column must allow NULL.
ON DELETE SET DEFAULT	Keep the children, set the FK to the column default.

Example 6 [Capgemini · pattern]

Given the two tables below and ON DELETE CASCADE, what does t_emp hold after DELETE FROM t_dept WHERE id = 1;?

t_dept: (1, A), (2, B). t_emp: (10, p, 1), (11, q, 1), (12, r, 2).

SOLUTION

CASCADE means “delete the children with the parent”.

Rows 10 and 11 point at department 1, so both go. Row 12 points at department 2, which is untouched.

Real output from running it:

SELECT * FROM t_emp;				TEXT
id	nm	d		
--	--	-		
12	r	2		

Answer: only (12, r, 2) survives

If the rule had been SET NULL, the answer would be (10, p, NULL), (11, q, NULL), (12, r, 2). If it had been RESTRICT, the DELETE itself would fail with a foreign key error and **nothing** would change.

TRAP

DELETE vs TRUNCATE vs DROP — asked in almost every service-company round.

	DELETE	TRUNCATE	DROP
Language	DML	DDL	DDL
WHERE allowed	yes	no	no

Rolls back	yes, inside a transaction	usually no (auto-commits)	usually no
Fires row triggers	yes	no	no
Table stays	yes, empty	yes, empty	no — table is gone
Identity counter	keeps counting	usually resets to 1	gone
Speed on a big table	slow, row by row + log	fast, deallocates pages	fast

One-line answer: “DELETE removes rows and can be filtered and rolled back; TRUNCATE empties the whole table fast as a DDL operation; DROP removes the table itself, structure and all.”

5.4 Relational algebra in ten minutes

Relational algebra is the maths behind SQL. Interviewers ask it because it proves you understand **what** a query computes, separately from the syntax.

Every operator takes one or two relations and returns a relation. That is the whole idea — the output of one operator is the input of the next, so you can chain them.

The six basic operators

Symbol	Name	What it does
$\sigma_c(R)$	selection	keep the rows where condition c is true. Horizontal cut.
$\pi_{a,b}(R)$	projection	keep only columns a and b , and remove duplicate rows . Vertical cut.
$R \times S$	cartesian product	every row of R paired with every row of S . Degree adds, cardinality multiplies.
$R \cup S$	union	all rows of both, duplicates removed. R and S must be union-compatible.
$R - S$	set difference	rows in R that are not in S .
$\rho_X(R)$	rename	give R (or its columns) a new name — needed for a self join.

Union-compatible means: same number of attributes, and matching domains column by column. employee and department are not union-compatible; SELECT dept_id FROM employee and SELECT dept_id FROM department are.

The derived operators

Symbol	Name	Built from the basics as
$R \cap S$	intersection	$R - (R - S)$
$R \bowtie_c S$	theta join	$\sigma_c(R \times S)$ — a product followed by a filter
$R \bowtie S$	natural join	join on all same-named columns, then drop the duplicate column
$R \ltimes S$	left outer join	natural join, plus the unmatched R rows padded with nulls
$R \div S$	division	the R rows that pair with every row of S

Degree and cardinality arithmetic, which is a favourite one-mark question:

- σ : degree unchanged, cardinality $\leq$ original.
- π on k columns: degree = k , cardinality $\leq$ original (duplicates collapse).
- $R \times S$: degree = $\deg(R) + \deg(S)$, cardinality = $|R| \times |S|$.
- $R \bowtie S$ on one common column: degree = $\deg(R) + \deg(S) - 1$; cardinality anywhere from 0 to $|R| \times |S|$.

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 7 [Accenture · pattern]

R has 5 attributes and 300 rows. S has 4 attributes and 200 rows. They share exactly one attribute name. Give the degree and cardinality of (a) $R \times S$, and (b) the degree of the natural join $R \bowtie S$.

SOLUTION

(a) A cartesian product pairs everything.

- Degree: $5 + 4 = 9$.
- Cardinality: $300 \times 200 = 60000$.

(b) A natural join keeps the shared column only once.

- Degree: $5 + 4 - 1 = 8$.
- Cardinality cannot be computed from these numbers alone. It is between 0 (no value matches) and 60000 (every value matches everything). If the shared column were the primary key of S , each R row would match at most one S row, giving at most 300.

Answer: (a) degree 9, cardinality 60000. (b) degree 8, cardinality unknown — between 0 and 60000.

Example 8 [Cognizant · pattern]

Write in relational algebra: “the names of employees in Pune departments earning more than 50000”. Then write the SQL.

SOLUTION

Work outside-in, then write it inside-out.

Step 1 — join the two tables on dept_id:

$\text{employee} \bowtie \text{department}$

Step 2 — filter:

$\sigma_{\text{city} = \text{'Pune'} \wedge \text{salary} > 50000}(\text{employee} \bowtie \text{department})$

Step 3 — project the one column asked for:

$\pi_{\text{emp_name}}(\sigma_{\text{city} = \text{'Pune'} \wedge \text{salary} > 50000}(\text{employee} \bowtie \text{department}))$

SQL:

```
SELECT DISTINCT e.emp_name
FROM employee e JOIN department d ON e.dept_id = d.dept_id
WHERE d.city = 'Pune' AND e.salary > 50000;
```

Pune departments are Engineering (10) and Support (30). Over 50000 there: Asha 90000, Bhavin 62000, Chetan 62000, Farida 52000. Gopal earns 41000 so he is out.

Answer: Asha, Bhavin, Chetan, Farida

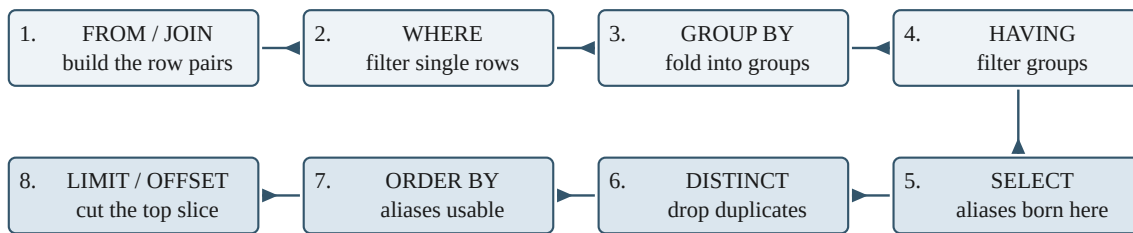
The DISTINCT is there because **projection removes duplicates by definition**, while SQL keeps them. If two employees shared a name, algebra would return one and SQL would return two.

SHORTCUT

Push the selection **inside** the join when you translate: $\sigma_c(R \bowtie S)$ becomes $\sigma_c(R) \bowtie S$ whenever c only mentions R 's columns. Filtering 300 rows down to 10 before joining is far cheaper than joining first. This rewrite is called **selection pushdown**, and every real query optimiser does it. Saying the phrase out loud is worth a mark.

5.5 The logical order of a SELECT

You **write** SELECT first. The database **runs** it almost last. Every confusing SQL error comes from this gap.



The order the database actually evaluates a query in. Follow the arrows, not the text you typed.

Three consequences you can quote directly:

1. A **SELECT** alias cannot be used in **WHERE**. **WHERE** runs at step 2; the alias is not created until step 5.
2. A **SELECT** alias CAN be used in **ORDER BY**. **ORDER BY** runs at step 7, after the alias exists.
3. **WHERE** filters rows, **HAVING** filters groups. **WHERE** cannot see **COUNT(*)** because no group exists yet at step 2.

Example 9 [Wipro · pattern]

Why does the first query fail and the second work?

```
-- (a)
SELECT salary * 12 AS annual FROM employee WHERE annual > 700000;
-- (b)
SELECT salary * 12 AS annual FROM employee ORDER BY annual DESC;
```

SOLUTION

(a) **WHERE** is step 2. At step 2 the only things that exist are the base columns `emp_id`, `emp_name`, `dept_id`, `manager_id`, `salary`, `joined_on`. The name `annual` does not exist yet $\Rightarrow$ “no such column: `annual`”.

Fix: repeat the expression, **WHERE** `salary * 12 > 700000`, or wrap the query:

```
SELECT annual FROM (SELECT salary * 12 AS annual FROM employee) WHERE annual > 700000;
```

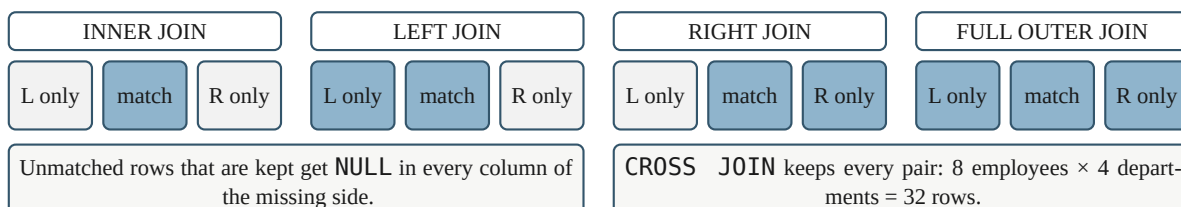
(b) **ORDER BY** is step 7, which is **after** **SELECT** at step 5. By then `annual` is a real column of the result, so it sorts fine. Verified output, top two rows:

```
annual
-----
1080000
936000
```

Answer: (a) the alias does not exist yet at WHERE time; (b) it does exist by ORDER BY time

5.6 Joins

A join answers: for each row on the left, which rows on the right go with it?



Which rows survive each join. 'L only' = a left row with no partner; 'R only' = a right row with no partner. Shaded = kept.

Inner join — only matched rows

Employees with their department name

SQL

```
SELECT e.emp_name, d.dept_name
FROM employee e
JOIN department d ON e.dept_id = d.dept_id
ORDER BY e.emp_id;
```

Real output — 7 rows, not 8:

result

TEXT

emp_name	dept_name
Asha	Engineering
Bhavin	Engineering
Chetan	Engineering
Divya	Sales
Eshan	Sales
Farida	Support
Gopal	Support

Hiral is missing. Her dept_id is NULL, and NULL = 10 is UNKNOWN, never true. So she matched nothing. Research (40) is missing too — no employee points at it.

Left join — keep every left row

Same query, LEFT JOIN

SQL

```
SELECT e.emp_name, d.dept_name
FROM employee e
LEFT JOIN department d ON e.dept_id = d.dept_id
ORDER BY e.emp_id;
```

Now 8 rows. The last one is:

last row of the result

TEXT

Hiral	(blank = NULL)
-------	----------------

Anti-join — the “which ones have none?” pattern

This is asked constantly: **find the departments with no employees.**

Anti-join with LEFT JOIN ... IS NULL

SQL

```
SELECT d.dept_name
FROM department d
LEFT JOIN employee e ON d.dept_id = e.dept_id
WHERE e.emp_id IS NULL;
```

Output: Research. ✓

Read it slowly: the LEFT JOIN keeps every department. A department with no employee gets NULL in all employee columns. e.emp_id is the employee primary key, so it can **only** be NULL when the match failed. Filtering on it keeps exactly the unmatched departments.

TRAP

Test `IS NULL` on a column that can never legitimately be null — a primary key or the join key. If you write `WHERE e.dept_id IS NULL`, you may also catch real rows that happen to store a null, and your answer becomes wrong.

Self join — a table joined to itself

Each employee with their manager's name

SQL

```
SELECT e.emp_name AS emp, m.emp_name AS mgr
FROM employee e
LEFT JOIN employee m ON e.manager_id = m.emp_id
ORDER BY e.emp_id;
```

Real output:

result

TEXT

```
emp      mgr
-----  -
Asha
Bhavin   Asha
Chetan   Asha
Divya
Eshan    Divya
Farida   Divya
Gopal    Farida
Hiral
```

The LEFT matters: with a plain JOIN, Asha, Divya and Hiral (no manager) would vanish.

The LEFT JOIN trap that decides the round

Example 10 [Grab · pattern]

Count employees earning more than 60000 in each department, **including departments where the count is zero**. Explain the difference between putting the salary test in ON and putting it in WHERE.

SOLUTION

Version A — condition in ON:

```
SELECT d.dept_name, COUNT(e.emp_id) AS n
FROM department d
LEFT JOIN employee e ON e.dept_id = d.dept_id AND e.salary > 60000
GROUP BY d.dept_id ORDER BY d.dept_id;
```

Output:

```
dept_name  n
-----  -
Engineering 3
Sales       1
Support     0
Research    0
```

(Engineering has three people over 60000: Asha 90000, Bhavin 62000, Chetan 62000. Sales has one: Divya 78000.)

Version B — condition in WHERE:

```
SELECT d.dept_name, COUNT(e.emp_id) AS n
FROM department d
LEFT JOIN employee e ON e.dept_id = d.dept_id
WHERE e.salary > 60000
GROUP BY d.dept_id ORDER BY d.dept_id;
```

Output:

```
Engineering 3
Sales       1
```

Why. ON runs during step 1 — it decides **which rows pair up**, and unmatched left rows are still padded with NULL and kept. WHERE runs at step 2, **after** the padding. For Support and Research the padded row has e.salary = NULL, and NULL > 60000 is UNKNOWN, so WHERE throws those rows away — and the zero rows disappear.

Answer: ON filters before padding and keeps the zeros; WHERE filters after padding and silently turns the LEFT JOIN back into an inner join

TRAP

“A WHERE clause on the right-hand table of a LEFT JOIN cancels the LEFT.” Learn this sentence. The only safe WHERE test on the right table is IS NULL (the anti-join).

Natural join — why professionals avoid it

NATURAL JOIN joins on **every** column with the same name in both tables, automatically.

Here, employee and department share dept_id, so employee NATURAL JOIN department behaves like the inner join above. But the moment someone adds a city column to employee, the natural join silently starts matching on dept_id **and** city and your results change with no code edit.

Answer: Say: “NATURAL JOIN is convenient but fragile — it depends on column names, so a schema change can silently alter the result. I always write the ON clause.”

5.7 NULL — three-valued logic

NULL is not zero and not an empty string. It means **unknown**.

Expression	Value	Expression	Value
5 = NULL	UNKNOWN	NULL = NULL	UNKNOWN
5 <> NULL	UNKNOWN	NULL IS NULL	TRUE
100 + NULL	NULL	'x' NULL	NULL (most engines)
TRUE OR UNKNOWN	TRUE	FALSE AND UNKNOWN	FALSE
TRUE AND UNKNOWN	UNKNOWN	NOT UNKNOWN	UNKNOWN

Verified: SELECT 100 + NULL, NULL = NULL, NULL IS NULL, 'x' || NULL; returns blank, blank, 1, blank — that is NULL, NULL, true, NULL.

How aggregates treat NULL

Every aggregate except COUNT(*) ignores nulls.

Run on our employee table

SQL

```
SELECT COUNT(*)           AS c_star,      -- 8
       COUNT(dept_id)     AS c_dept,      -- 7 (Hiral's NULL skipped)
       COUNT(DISTINCT dept_id) AS c_distinct -- 3 (10, 20, 30)
FROM employee;
```

Real output: 8 7 3.

AVG divides by the non-null count

SQL

```
SELECT AVG(dept_id), SUM(dept_id), COUNT(*) FROM employee;
-- 18.5714285714286 | 130 | 8
```

Check the arithmetic: the 7 non-null values are 10,10,10,20,20,30,30 $\Rightarrow$ sum = 130. $\frac{130}{7} = 18.571\dots$ — so AVG divided by 7, not by 8. Confirmed.

The NOT IN disaster

Example 11 [Amazon · pattern]

These two queries look equivalent. One returns three rows and one returns nothing. Which, and why?

```
-- (a)
SELECT emp_name FROM employee
WHERE dept_id NOT IN (SELECT dept_id FROM employee WHERE manager_id IS NULL);

-- (b)
SELECT e.emp_name FROM employee e
WHERE NOT EXISTS (SELECT 1 FROM employee x
                  WHERE x.manager_id IS NULL AND x.dept_id = e.dept_id);
```

SOLUTION

First work out the subquery. Employees with `manager_id IS NULL` are Asha (dept 10), Divya (dept 20) and **Hiral (dept NULL)**. So the inner list is (10, 20, NULL).

(a) SQL expands `x NOT IN (10, 20, NULL)` into `x <> 10 AND x <> 20 AND x <> NULL`.

Take Farida, `dept_id = 30`:

- `30 <> 10` $\rightarrow$ TRUE
- `30 <> 20` $\rightarrow$ TRUE
- `30 <> NULL` $\rightarrow$ UNKNOWN
- `TRUE AND TRUE AND UNKNOWN` $\rightarrow$ UNKNOWN

WHERE keeps only TRUE, so Farida is dropped. The same happens to every row. **(a) returns zero rows.** Verified — the query printed nothing.

(b) NOT EXISTS asks a yes/no question: “does a matching row exist?” There is no three-valued trap, because EXISTS is always TRUE or FALSE. Verified output:

```
Farida
Gopal
Hiral
```

Farida and Gopal are in dept 30, which has no manager-less employee. Hiral has `dept_id = NULL`, so `x.dept_id = e.dept_id` is never true for her $\Rightarrow$ NOT EXISTS is true.

Answer: (a) returns nothing because a single NULL in the NOT IN list makes every comparison UNKNOWN; (b) returns Farida, Gopal, Hiral

The fix for (a): add `AND dept_id IS NOT NULL` inside the subquery, or just use NOT EXISTS. Professionals default to NOT EXISTS.

TRAP

IN with nulls is safe-ish; NOT IN with nulls is a bug factory. `x IN (1, 2, NULL)` is TRUE when x is 1 or 2 (because `TRUE OR UNKNOWN = TRUE`). `x NOT IN (1, 2, NULL)` is **never** TRUE (because `TRUE AND UNKNOWN = UNKNOWN`).

Useful null tools

COALESCE(a, b, c)	first non-null argument. COALESCE(dept_id, -1) gave -1 for Hiral.
NULLIF(a, b)	NULL if a = b, else a. Guards divide-by-zero: x / NULLIF(y, 0).
IS DISTINCT FROM	null-safe <>. NULL IS DISTINCT FROM NULL is FALSE. (MySQL: <=> is the null-safe =.)

5.8 Aggregates, GROUP BY and HAVING

Rule: every column in the SELECT list must either be inside an aggregate, or be listed in GROUP BY. A group has many rows; a non-grouped column has no single value to show.

Departments with 2 or more people, richest first

SQL

```
SELECT d.dept_name, COUNT(*) AS headcount, AVG(e.salary) AS avg_sal
FROM employee e
JOIN department d ON e.dept_id = d.dept_id
GROUP BY d.dept_name
HAVING COUNT(*) >= 2
ORDER BY avg_sal DESC;
```

Real output:

result

TEXT

dept_name	headcount	avg_sal
Engineering	3	71333.3333333333
Sales	2	61500.0
Support	2	46500.0

Check Engineering by hand: $\frac{90000+62000+62000}{3} = \frac{214000}{3} = 71333.33$. ✓ Sales: $\frac{78000+45000}{2} = 61500$. ✓ Support: $\frac{52000+41000}{2} = 46500$. ✓

Conditional aggregation — pivot without a PIVOT keyword

Example 12 [Agoda · pattern]

For every department — including empty ones — show how many staff earn at least 60000 (“senior”) and how many earn less (“junior”), in one row per department.

SOLUTION

Put the CASE **inside** the aggregate. That is the whole trick.

```
SELECT d.dept_name,
       SUM(CASE WHEN e.salary >= 60000 THEN 1 ELSE 0 END) AS senior,
       SUM(CASE WHEN e.salary < 60000 THEN 1 ELSE 0 END) AS junior
FROM department d
LEFT JOIN employee e ON e.dept_id = d.dept_id
GROUP BY d.dept_id
ORDER BY d.dept_id;
```

Real output:

dept_name	senior	junior
Engineering	3	0
Sales	1	1
Support	0	2
Research	0	0

Research shows 0 and 0 because LEFT JOIN kept it, and SUM over the single padded NULL row evaluates CASE WHEN NULL >= 60000 → UNKNOWN → ELSE 0.

Answer: one SUM(CASE ...) per bucket, over a LEFT JOIN

SHORTCUT

COUNT(CASE WHEN c THEN 1 END) (with **no** ELSE) works too — the non-matching rows become NULL, and COUNT skips nulls. SUM(CASE ... ELSE 0 END) is safer to read aloud in an interview.

5.9 Subqueries

Kind	Shape and use
Scalar	Returns exactly one value. Can sit anywhere a value can: WHERE salary > (SELECT AVG(salary) FROM employee).
Row-list (IN)	Returns one column, many rows. WHERE dept_id IN (SELECT ...).
EXISTS	Returns a yes/no. The database can stop at the first hit.
Correlated	Mentions a column from the outer query, so it is conceptually re-run per outer row.
Derived table	A subquery in the FROM clause. Must be given an alias in most engines.
CTE (WITH)	A named derived table written before the main query. Easier to read, can be reused.

Correlated subquery, worked

Employees paid above their OWN department's average

SQL

```
SELECT e.emp_name, e.salary
FROM employee e
WHERE e.salary > (SELECT AVG(x.salary)
                  FROM employee x
                  WHERE x.dept_id = e.dept_id);
```

Real output: Asha 90000, Divya 78000, Farida 52000.

Trace it row by row:

- Asha, dept 10: inner average = 71333.33. 90000 > 71333.33 ✓
- Bhavin, dept 10: 62000 > 71333.33 ✗
- Chetan, dept 10: same ✗
- Divya, dept 20: inner average = 61500. 78000 > 61500 ✓
- Eshan, dept 20: 45000 > 61500 ✗
- Farida, dept 30: inner average = 46500. 52000 > 46500 ✓
- Gopal, dept 30: 41000 > 46500 ✗
- Hiral, dept NULL: the inner query matches no row, so AVG over zero rows is NULL, and 58000 > NULL is UNKNOWN ✗

e is the **outer** alias and x the **inner** one. The subquery reads e.dept_id — that is what makes it correlated, and that is why it cannot be run once and cached.

IN vs EXISTS — the standard follow-up

IN (subquery)	EXISTS (subquery)
Builds the whole inner list, then compares.	Stops at the first matching row.
Breaks on NULL when negated (NOT IN).	Null-safe by construction.
Reads better for a short, fixed list.	Reads better when the inner query is correlated.

Good when the inner result is small.

Good when the inner result is large.

Honest modern answer: “On any current optimiser they are usually rewritten to the same semi-join plan, so performance is rarely the deciding factor. The real difference is `NOT IN` versus `NOT EXISTS` with nulls — and there `NOT EXISTS` is correct.”

Division — “did X do ALL of Y?”

Example 13 [Google · pattern]

Find every employee who is assigned to **every** project belonging to department 10.

SOLUTION

Department 10 owns Atlas (100) and Beacon (101). Look at the assignments: Asha has 100 and 101 ✓; Bhavin has only 100 ✗; Chetan has only 101 ✗; Farida has only 100 ✗.

Method 1 — double `NOT EXISTS`. Read it as: “there is no dept-10 project that this employee is not on.”

```
SELECT e.emp_name
FROM employee e
WHERE NOT EXISTS (
  SELECT 1 FROM project p
  WHERE p.dept_id = 10
    AND NOT EXISTS (SELECT 1 FROM assignment a
                     WHERE a.emp_id = e.emp_id AND a.proj_id = p.proj_id)
);
```

Method 2 — count matching. “This employee touches as many distinct dept-10 projects as dept 10 has.”

```
SELECT e.emp_name
FROM employee e
JOIN assignment a ON a.emp_id = e.emp_id
JOIN project p   ON p.proj_id = a.proj_id AND p.dept_id = 10
GROUP BY e.emp_id
HAVING COUNT(DISTINCT p.proj_id) = (SELECT COUNT(*) FROM project WHERE dept_id = 10);
```

Both were run. Both print exactly Asha. ✓

Answer: Asha

Difference worth mentioning: method 1 would also return an employee if department 10 had **zero** projects (the inner set is empty, so “no project is missing” is vacuously true). Method 2 would return nobody in that case, because there would be no rows to group. Say this out loud — it is the follow-up the interviewer is waiting for.

5.10 Set operators

UNION	rows of both, duplicates removed (a sort or hash is needed — slower)
UNION ALL	rows of both, duplicates kept (fastest — prefer it when you know there are no duplicates)
INTERSECT	rows in both
EXCEPT (MINUS in Oracle)	rows in the first and not in the second

Both sides must have the **same number of columns** with **compatible types**. Column names come from the first `SELECT`.

Verified counts on our data:

UNION vs UNION ALL

SQL

```
SELECT COUNT(*) FROM (SELECT dept_id FROM employee UNION
                        SELECT dept_id FROM
                        department); -- 5
```

```
SELECT COUNT(*) FROM (SELECT dept_id FROM employee UNION ALL SELECT dept_id FROM
department); -- 12
```

UNION ALL is $8 + 4 = 12$. UNION collapses to the 5 distinct values {NULL, 10, 20, 30, 40} — note that set operators treat two NULLs as **the same** value, unlike =.

Departments that exist but employ nobody, via EXCEPT

SQL

```
SELECT dept_id FROM department
EXCEPT
SELECT dept_id FROM employee; -- 40
```

Same answer as the LEFT JOIN ... IS NULL anti-join earlier. ✓

5.11 CTEs and recursive queries

A **common table expression** (CTE) is a named subquery written before the main query with WITH. It does not change what the query computes — it changes how readable it is.

The same 'above your department average' query, written with a CTE

SQL

```
WITH dept_avg AS (
  SELECT dept_id, AVG(salary) AS a
  FROM employee
  GROUP BY dept_id
)
SELECT e.emp_name, e.salary, ROUND(d.a, 2) AS dept_avg
FROM employee e
JOIN dept_avg d ON d.dept_id = e.dept_id
WHERE e.salary > d.a;
```

Real output:

result

TEXT

emp_name	salary	dept_avg
-----	-----	-----
Asha	90000	71333.33
Divya	78000	61500.0
Farida	52000	46500.0

Same three people as the correlated subquery earlier — but now the average is computed **once per department** instead of conceptually once per row, and a reader can see the intermediate table.

Recursive CTE — walking a hierarchy

A self-referencing foreign key like manager_id builds a tree. A plain self join only reaches **one** level. To reach the whole subtree you need recursion.

The shape is always the same:

Anchor	The starting row (or rows). Runs once.
UNION ALL	Always UNION ALL, not UNION — see the trap below.
Recursive part	Joins the table back to the CTE's own name. Runs again and again on the rows produced last time.
Stop	Automatic: it stops when a round produces zero new rows.

```
WITH RECURSIVE chain(emp_id, emp_name, lvl) AS (
    SELECT emp_id, emp_name, 0
    FROM employee
    WHERE emp_name = 'Divya'           -- anchor: the root
    UNION ALL
    SELECT e.emp_id, e.emp_name, c.lvl + 1
    FROM employee e
    JOIN chain c ON e.manager_id = c.emp_id -- recursive step
)
SELECT lvl, emp_id, emp_name FROM chain ORDER BY lvl, emp_id;
```

Real output:

result

TEXT

lvl	emp_id	emp_name
0	4	Divya
1	5	Eshan
1	6	Farida
2	7	Gopal

Trace the rounds:

- **Round 0 (anchor):** Divya, level 0.
- **Round 1:** who reports to emp_id 4? Eshan (5) and Farida (6). Level 1.
- **Round 2:** who reports to 5 or 6? Gopal reports to 6. Level 2.
- **Round 3:** who reports to 7? Nobody. Zero new rows ⇒ stop.

TRAP

Two recursive-CTE traps:

1. **UNION instead of UNION ALL.** UNION de-duplicates every round, which is slower and can hide a genuine cycle instead of exposing it.
2. **A cycle in the data** (A manages B, B manages A) makes the query run forever. Guard it by carrying the path and checking it, or by adding `WHERE lvl < 20`. Real engines usually have a recursion-depth setting; SQL Server's default limit is 100 levels.

Example 14 [Sea/Shopee · pattern]

Generate the numbers 1 to 5 with no table at all, then explain one practical use.

SOLUTION

```
WITH RECURSIVE n(k) AS (
    SELECT 1
    UNION ALL
    SELECT k + 1 FROM n WHERE k < 5
)
SELECT group_concat(k) FROM n;
```

Real output: 1,2,3,4,5.

Practical use: generating a complete calendar. Reports must show **every** day of a month, including days with zero orders. A `GROUP BY order_date` can only show days that have rows. So you generate the dates recursively and `LEFT JOIN` the orders onto them — the zero days survive as `NULL`, and `COALESCE(SUM(amount), 0)` turns them into 0.

Answer: a recursive CTE with no FROM table is a number/date generator, used to fill gaps a GROUP BY cannot

Handy aggregate for interviews: GROUP_CONCAT (SQLite/MySQL) or STRING_AGG (PostgreSQL/SQL Server) glues a group's values into one string. SELECT d.dept_name, GROUP_CONCAT(e.emp_name, ', ') FROM department d LEFT JOIN employee e ON e.dept_id = d.dept_id GROUP BY d.dept_id; really prints Engineering | Asha, Bhavin, Chetan and Research | (empty).

5.12 Window functions

A GROUP BY **collapses** rows. A window function **keeps every row** and adds a computed column that looked at a group of neighbours.

	dept_id	emp_name	salary	ROW_NUMBER()	
PARTITION dept_id = 10	10	Asha	90000	1	frame of a running SUM: every row up to here
	10	Bhavin	62000	2	
	10	Chetan	62000	3	
PARTITION dept_id = 20	20	Divya	78000	1	
	20	Eshan	45000	2	

Counter restarts at 1 in every partition — the numbering never crosses a partition boundary.

PARTITION BY splits the rows into independent windows; ORDER BY sets the order inside each one; the frame is the slice the function actually reads.

The three ranking functions

ROW_NUMBER vs RANK vs DENSE_RANK on the same data

SQL

```
SELECT emp_name, salary,  
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn,  
       RANK() OVER (ORDER BY salary DESC) AS rk,  
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dr  
FROM employee;
```

Real output:

result					TEXT
emp_name	salary	rn	rk	dr	
Asha	90000	1	1	1	
Divya	78000	2	2	2	
Bhavin	62000	3	3	3	
Chetan	62000	4	3	3	
Hiral	58000	5	5	4	
Farida	52000	6	6	5	
Eshan	45000	7	7	6	
Gopal	41000	8	8	7	

Watch the tie at 62000 and the row right after it:

ROW_NUMBER	always 1, 2, 3, 4, ... — ties get different numbers, decided arbitrarily unless you add a tie-breaker to ORDER BY.
------------	---

RANK	ties share a rank, then the next value skips : 3, 3, then 5.
DENSE_RANK	ties share a rank, and the next value does not skip: 3, 3, then 4.

TRAP

“ROW_NUMBER is deterministic.” It is not, when the ORDER BY has ties. Bhavin and Chetan both earn 62000; which one gets rn = 3 is up to the engine. Always add a unique tie-breaker: ORDER BY salary DESC, emp_id.

Top-N per group — the pattern you will be asked for

Example 15 [DBS · pattern]

Show the highest-paid employee in each department.

SOLUTION

Number the rows **inside** each department, then keep number 1.

```
SELECT dept_name, emp_name, salary
FROM (
    SELECT d.dept_name, e.emp_name, e.salary,
           ROW_NUMBER() OVER (PARTITION BY e.dept_id
                              ORDER BY e.salary DESC, e.emp_id) AS rn
    FROM employee e
    JOIN department d ON e.dept_id = d.dept_id
) t
WHERE rn = 1;
```

Real output:

Engineering	Asha	90000
Sales	Divya	78000
Support	Farida	52000

Answer: ROW_NUMBER() OVER (PARTITION BY group ORDER BY metric DESC) then filter = 1

Two follow-ups to have ready:

- “What if two people tie for top?” ROW_NUMBER picks one; switch to RANK() = 1 or DENSE_RANK() = 1 to return all of them.
- “Why the subquery?” Window functions are computed at step 5 (SELECT), which is after WHERE at step 2. So you cannot write WHERE rn = 1 in the same block — you must wrap it in a derived table or a CTE.

Running total and LAG

Cumulative salary cost as people joined

SQL

```
SELECT emp_name, joined_on, salary,
       SUM(salary) OVER (ORDER BY joined_on
                         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running
FROM employee
ORDER BY joined_on;
```

Real output:

result

TEXT

emp_name	joined_on	salary	running
Divya	2018-11-05	78000	78000
Asha	2019-03-01	90000	168000

Bhavin	2020-07-15	62000	230000
Chetan	2021-01-10	62000	292000
Farida	2021-09-01	52000	344000
Eshan	2022-02-20	45000	389000
Gopal	2023-04-12	41000	430000
Hiral	2023-06-01	58000	488000

Verify two of them by hand: $78000 + 90000 = 168000$ ✓ and $430000 + 58000 = 488000$ ✓. The last running value equals the total payroll.

LAG(col) reads the previous row, LEAD(col) the next one:

Gap to the next-lower salary	SQL
<pre>SELECT emp_name, salary, LAG(salary) OVER (ORDER BY salary) AS prev, salary - LAG(salary) OVER (ORDER BY salary) AS gap FROM employee;</pre>	

The first row's prev is NULL (no previous row), so its gap is NULL too. Chetan's gap is 0 — he ties with Bhavin. Both verified in the real output.

Nth highest salary — three correct answers

Example 16 [TCS NQT · pattern]

Return the 3rd highest **distinct** salary.

SOLUTION

The distinct salaries, sorted down: 90000, 78000, 62000, 58000, 52000, 45000, 41000. The 3rd is **62000**.

Method A — LIMIT / OFFSET (simplest):

```
SELECT DISTINCT salary FROM employee ORDER BY salary DESC LIMIT 1 OFFSET 2;
```

For the N th, use OFFSET $N-1$. Here $N = 3$ so OFFSET 2.

Method B — DENSE_RANK (works on any engine with windows, and reads best):

```
SELECT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS dr FROM employee
) WHERE dr = 3 LIMIT 1;
```

DENSE_RANK, not RANK — with RANK a tie would make rank 3 disappear.

Method C — correlated count (no window functions, works on very old engines):

```
SELECT e.salary FROM employee e
WHERE 3 = (SELECT COUNT(DISTINCT x.salary) FROM employee x WHERE x.salary >= e.salary)
LIMIT 1;
```

Read it: “exactly 3 distinct salaries are greater than or equal to mine.”

All three were run. All three print 62000. ✓

Answer: 62000

Follow-up to volunteer: “If fewer than N distinct salaries exist, A and B return no rows, which is usually what you want. If the interviewer wants NULL instead, wrap it: SELECT (SELECT ...) AS nth;”

5.13 What a join actually does — in JavaScript

The interviewer sometimes asks “how is a join implemented?” Here are the two answers, written as runnable code so you can see them.

```

const employee = [
  { emp_id: 1, name: 'Asha', dept_id: 10 },
  { emp_id: 2, name: 'Bhavin', dept_id: 10 },
  { emp_id: 4, name: 'Divya', dept_id: 20 },
  { emp_id: 8, name: 'Hiral', dept_id: null },
];

const department = [
  { dept_id: 10, dept_name: 'Engineering' },
  { dept_id: 20, dept_name: 'Sales' },
  { dept_id: 40, dept_name: 'Research' },
];

// 1. NESTED LOOP JOIN - compare every pair. Cost = |E| x |D|.
function nestedLoopInner(left, right) {
  const out = [];
  let comparisons = 0;
  for (const e of left) {
    for (const d of right) {
      comparisons++;
      if (e.dept_id === d.dept_id) out.push({ name: e.name, dept: d.dept_name });
    }
  }
  return { out, comparisons };
}

// 2. HASH JOIN - build a Map on the smaller side, then probe once per row.
function hashInner(left, right) {
  const bucket = new Map();
  for (const d of right) bucket.set(d.dept_id, d);
  const out = [];
  for (const e of left) {
    const d = bucket.get(e.dept_id);
    if (d) out.push({ name: e.name, dept: d.dept_name });
  }
  return { out, comparisons: right.length + left.length };
}

// 3. LEFT JOIN - keep the left row even with no match, pad with null.
function hashLeft(left, right) {
  const bucket = new Map();
  for (const d of right) bucket.set(d.dept_id, d);
  return left.map(e => ({ name: e.name, dept: bucket.get(e.dept_id)?.dept_name ?? null }));
}

console.log('nested loop:', nestedLoopInner(employee, department).comparisons, 'comparisons');
console.log('hash join  :', hashInner(employee, department).comparisons, 'comparisons');
console.log('left join  :', JSON.stringify(hashLeft(employee, department)));

```

Actual output from node:

terminal

TEXT

```
nested loop: 12 comparisons
hash join  : 7 comparisons
left join  : [{"name": "Asha", "dept": "Engineering"},
{"name": "Bhavin", "dept": "Engineering"},
{"name": "Divya", "dept": "Sales"}, {"name": "Hiral", "dept": null}]
```

Time nested loop $O(n \cdot m)$, hash join $O(n+m)$ average **Space** nested loop $O(1)$ extra, hash join $O(m)$ for the hash table

A third strategy, merge join, sorts both sides first: $O(n \log n + m \log m)$, then one linear pass. The optimiser picks whichever is cheapest for the sizes and indexes it has.

TRAP

JavaScript trap hiding in the code above: `bucket.get(null)` returns the department object only if some department really had `dept_id === null`. Using a plain object instead of a `Map` would stringify the key — `obj[null]` becomes `obj["null"]` and `obj[10]` becomes `obj["10"]`, so the number 10 and the string "10" would collide. `Map` keeps the key's type. This is exactly why the language policy says use `Map`.

5.14 Practice

PRACTICE · WARM-UP target: 6 min

1. Give the degree and cardinality of department in this chapter.
2. Is {emp_id, emp_name} a super key of employee? Is it a candidate key?
3. How many rows does `SELECT * FROM employee, department;` return?
4. Write the query for all employees whose name starts with the letter B.
5. What does `SELECT COUNT(manager_id) FROM employee;` return, and why is it not 8?
6. Which is DDL and which is DML: TRUNCATE, UPDATE, ALTER, INSERT?

ANSWER KEY

1. Degree 3, cardinality 4. — 2. Super key yes (it contains emp_id); candidate key no, because emp_name is spare. — 3. $8 \times 4 = 32$. —
4. `SELECT * FROM employee WHERE emp_name LIKE 'B%';` —
5. `COUNT(col)` skips nulls, and Asha, Divya and Hiral have manager_id NULL ($8 - 3 = 5$). — 6. DDL: TRUNCATE, ALTER. DML: UPDATE, INSERT.

PRACTICE · TIER 1 target: 12 min

1. `R(A,B,C,D,E)` has exactly one candidate key, {A,B}. How many super keys?
2. Write a query listing every department name together with its headcount, showing 0 for empty departments.
3. Write a query for the employees who are nobody's manager.
4. What is printed by `SELECT 10 + NULL;` and by `SELECT COUNT(*) FROM employee WHERE dept_id <> 10;`?
5. Write a query for the second-lowest salary in the table.
6. A table `orders(order_id, customer_id, amount)` — write the query for customers whose total order amount is above 5000, highest total first.

ANSWER KEY

1. Every super key must contain both A and B; the other three attributes are free, so $2^3 = 8$.
2. `SELECT d.dept_name, COUNT(e.emp_id) FROM department d LEFT JOIN employee e ON e.dept_id = d.dept_id GROUP BY d.dept_id;`
Use `COUNT(e.emp_id)`, not `COUNT(*)` — `COUNT(*)` would count the padded row and print 1 for an empty department.
3. `SELECT emp_name FROM employee e WHERE NOT EXISTS (SELECT 1 FROM employee m WHERE m.manager_id = e.emp_id);`
That is Bhavin, Chetan, Eshan, Gopal, Hiral.
4. NULL. And 4 — the rows with dept_id 20, 20, 30, 30. Hiral's NULL <> 10 is UNKNOWN, so she is excluded even though “her department is not 10” feels true.
5. `SELECT DISTINCT salary FROM employee ORDER BY salary ASC LIMIT 1 OFFSET 1;` $\Rightarrow$ 45000.
6. `SELECT customer_id, SUM(amount) AS total FROM orders GROUP BY customer_id HAVING SUM(amount) > 5000 ORDER BY total DESC;`
The filter must be HAVING, not WHERE, because it tests an aggregate.

PRACTICE · TIER 2 target: 18 min

1. For each project, show the project name and the total hours booked on it, including projects with no assignments at all.
2. Using the employee table, show each employee's salary and the average salary of their department **side by side on the same row**, without a GROUP BY.
3. A food-delivery table orders(order_id, city, placed_at, amount) — write the query for the top 2 orders by amount in each city.
4. Explain in three lines what happens if you write SELECT dept_id, emp_name, COUNT(*) FROM employee GROUP BY dept_id;
5. Rewrite WHERE dept_id NOT IN (SELECT dept_id FROM department WHERE city = 'Pune') so that it is safe when department.dept_id can be NULL.

ANSWER KEY

1. SELECT p.proj_name, COALESCE(SUM(a.hours), 0) AS total FROM project p LEFT JOIN assignment a ON a.proj_id = p.proj_id GROUP BY p.proj_id;

Delta gets 0 thanks to COALESCE; without it you would print NULL. Atlas = 120 + 80 + 30 = 230, Beacon = 150 + 40 = 190, Comet = 200 + 60 = 260, Delta = 0.

2. A window aggregate: SELECT emp_name, salary, AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg FROM employee;

This is the cleanest demonstration that a window function keeps all 8 rows while GROUP BY would collapse them to 3.

3. SELECT city, order_id, amount FROM (
 SELECT city, order_id, amount,
 ROW_NUMBER() OVER (PARTITION BY city ORDER BY amount DESC, order_id) AS
 rn
 FROM orders) t
 WHERE rn <= 2;

4. emp_name is neither aggregated nor in the GROUP BY. Strict engines (PostgreSQL, SQL Server, MySQL with ONLY_FULL_GROUP_BY) reject it. Lenient ones (SQLite, older MySQL) return **some arbitrary** name from each group — which is worse, because the query runs and quietly gives a different answer each time.

5. WHERE NOT EXISTS (SELECT 1 FROM department d WHERE d.city = 'Pune' AND d.dept_id = employee.dept_id).

Or keep NOT IN but add AND dept_id IS NOT NULL inside the subquery.

PRACTICE · TIER 3 target: 25 min

1. Write one query that returns, for each department, the name of the highest earner **and** the name of the lowest earner, on a single row. Empty departments should still appear.
2. login(user_id, login_date) records one row per login per day. Write a query for the users who logged in on at least 3 **consecutive** days.
3. Explain precisely why SELECT COUNT(*) FROM a LEFT JOIN b ON a.k = b.k; can be larger than SELECT COUNT(*) FROM a; even though a left join “keeps every row of a”.
4. You are told a query returns the right rows but the interviewer asks you to remove the DISTINCT. What structural change to the query usually makes DISTINCT unnecessary?

ANSWER KEY

1. WITH ranked AS (
 SELECT e.dept_id, e.emp_name, e.salary,
 ROW_NUMBER() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC,
 e.emp_id) AS hi,
 ROW_NUMBER() OVER (PARTITION BY e.dept_id ORDER BY e.salary ASC,
 e.emp_id) AS lo

```

FROM employee e)
SELECT d.dept_name,
       MAX(CASE WHEN r.hi = 1 THEN r.emp_name END) AS top_earner,
       MAX(CASE WHEN r.lo = 1 THEN r.emp_name END) AS low_earner
FROM department d LEFT JOIN ranked r ON r.dept_id = d.dept_id
GROUP BY d.dept_id;

```

Two window functions over the same partition, then conditional aggregation to fold the two marked rows into one. On our data: Engineering (Asha, Bhavin — Bhavin wins the tie on `emp_id`), Sales (Divya, Eshan), Support (Farida, Gopal), Research (NULL, NULL).

2. The **gaps-and-islands** pattern. Subtract the row number from the date; inside a run of consecutive days that difference is constant, so it becomes a group key.

```

WITH d AS (
  SELECT user_id, login_date,
         DATE(login_date, '-' || ROW_NUMBER() OVER (PARTITION BY user_id
                                                    ORDER BY login_date) || ' day')
AS grp
  FROM (SELECT DISTINCT user_id, login_date FROM login))
SELECT user_id FROM d GROUP BY user_id, grp HAVING COUNT(*) >= 3;

```

The `DISTINCT` guards against two logins recorded on the same date. (The date arithmetic above is SQLite syntax; PostgreSQL would use `login_date - rn * INTERVAL '1 day'`.)

3. “Keeps every row of `a`” means every row of `a` appears **at least** once, not exactly

once. If one `a` row matches three `b` rows, it appears three times. The count grows whenever `b.k` is not unique. This is the single most common cause of an inflated `SUM` after a join — and the reason people reach for `DISTINCT` instead of fixing the join.

4. Replace the join-that-only-tests-existence with `EXISTS`. A join to a non-unique table

multiplies rows and forces `DISTINCT`; `WHERE EXISTS (...)` tests membership without duplicating anything, so the `DISTINCT` disappears and the plan usually gets cheaper too.

5.15 Rapid fire — 28 one-line answers

Question	Answer
Degree vs cardinality?	Degree = number of columns; cardinality = number of rows.
Super key vs candidate key?	Candidate key is a super key with no spare attribute — remove anything and uniqueness breaks.
Can a primary key be NULL?	No. Entity integrity forbids it, even in one column of a composite key.
Can a unique key be NULL?	Yes, and most engines allow many nulls, because <code>NULL != NULL</code> .
What is referential integrity?	Every foreign key value matches an existing primary key, or is entirely NULL.
Foreign key to the same table?	Allowed — e.g. <code>manager_id</code> referencing <code>emp_id</code> .
Surrogate vs natural key?	Surrogate = generated meaningless id, never changes. Natural = real business data, can change.
DELETE vs TRUNCATE?	DELETE is DML, filterable, rollback-able, fires triggers. TRUNCATE is DDL, all rows, fast, usually no rollback.
TRUNCATE vs DROP?	TRUNCATE empties the table; DROP removes the table itself.
DDL / DML / DCL / TCL?	DDL CREATE ALTER DROP TRUNCATE; DML SELECT INSERT UPDATE DELETE; DCL GRANT REVOKE; TCL COMMIT ROLLBACK SAVEPOINT.
Logical order of a SELECT?	FROM, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY, LIMIT.
Why can't an alias be used in WHERE?	WHERE runs before SELECT, so the alias does not exist yet.
WHERE vs HAVING?	WHERE filters rows before grouping; HAVING filters groups after, and may use aggregates.
COUNT(*) vs COUNT(col)?	COUNT(*) counts rows; COUNT(col) skips nulls in that column.
Result of NULL = NULL?	UNKNOWN. Use IS NULL.
Why does NOT IN break with nulls?	It expands to a chain of ANDs; one UNKNOWN makes the whole condition UNKNOWN, so no row passes.
Safest replacement for NOT IN?	NOT EXISTS — it returns only true or false.
Inner vs left join?	Inner keeps matched rows only; left keeps every left row, padding with NULL.
Rows in a cross join?	$m \times n$ — every pair.
Why avoid NATURAL JOIN?	It matches on every same-named column, so adding a column silently changes the result.
How do you find rows with no match?	Anti-join: LEFT JOIN then WHERE right_pk IS NULL, or NOT EXISTS.
What kills a LEFT JOIN?	A WHERE condition on a right-table column — it drops the padded rows and turns it into an inner join.
Correlated subquery?	A subquery that references a column of the outer query, so it is evaluated per outer row.
UNION vs UNION ALL?	UNION removes duplicates (needs a sort or hash); UNION ALL keeps them and is faster.
RANK vs DENSE_RANK?	After a tie, RANK skips numbers (3,3,5); DENSE_RANK does not (3,3,4).
GROUP BY vs a window function?	GROUP BY collapses rows into one per group; a window function keeps all rows and adds a column.

Why wrap ROW_NUMBER in a subquery?	Window functions are computed after WHERE, so you cannot filter on them in the same block.
Nth highest salary, one line?	SELECT DISTINCT salary FROM t ORDER BY salary DESC LIMIT 1 OFFSET N-1;

ONE-PAGE REVISION CARD

Keys. Super key $\supseteq$ candidate key $\supseteq$ primary key. Candidate = minimal. Primary = chosen + NOT NULL. Alternate = the rest. If one attribute is unique in an n -column table, it sits in 2^{n-1} super keys. Two single-attribute keys give $2^n - 2^{n-2}$.

Constraints. Domain (CHECK), entity (PK not null), referential (FK matches or is null), key (uniqueness). FK delete rules: RESTRICT / CASCADE / SET NULL / SET DEFAULT.

DELETE / TRUNCATE / DROP. Rows + filter + rollback / all rows fast, DDL / table gone.

Order. FROM $\rightarrow$ WHERE $\rightarrow$ GROUP BY $\rightarrow$ HAVING $\rightarrow$ SELECT $\rightarrow$ DISTINCT $\rightarrow$ ORDER BY $\rightarrow$ LIMIT. Aliases are born at SELECT, so WHERE cannot see them and ORDER BY can.

NULL. Unknown, not zero. Any comparison gives UNKNOWN; WHERE keeps only TRUE. Aggregates except COUNT(*) skip nulls. NOT IN + NULL = zero rows. Use NOT EXISTS.

Joins. Inner = matches. Left = all of left + padding. Anti-join = LEFT JOIN + WHERE right_pk IS NULL. A WHERE on the right table cancels the LEFT. Cross = $m \times n$. Self-join for hierarchies.

Grouping. Every selected column must be aggregated or grouped. WHERE before, HAVING after. SUM(CASE WHEN ... THEN 1 ELSE 0 END) pivots without a PIVOT keyword.

Windows. OVER (PARTITION BY g ORDER BY c). ROW_NUMBER = 1,2,3,4. RANK = 3,3,5. DENSE_RANK = 3,3,4. Top-N per group = ROW_NUMBER inside a derived table, filter outside. Running total = SUM(...) OVER (ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW).

Nth highest. LIMIT 1 OFFSET N-1 on DISTINCT, or DENSE_RANK() = N, or N = (SELECT COUNT(DISTINCT x.salary) FROM t x WHERE x.salary >= t.salary).

Join execution. Nested loop $O(nm)$; hash join $O(n + m)$ average, $O(m)$ memory; merge join needs both sides sorted.

DBMS: Normalisation & Schema Design

Functional dependencies, attribute closure, 1NF to BCNF — with every decomposition worked line by line and checked by a program.

Normalisation is the most **mechanical** topic in the whole technical round. There is an algorithm for every question. Once you can compute an attribute closure on paper, you can answer almost anything the interviewer asks: find the candidate keys, name the highest normal form, decompose the table, prove the decomposition is lossless.

The catch is that students learn the **definitions** and never learn the **procedure**. Then they freeze when the board says $R(A, B, C, D, E)$ with four dependencies.

This chapter is the procedure. Every closure, every key list and every decomposition test in these pages was computed by a program that is printed in full in section 6.16 — so the numbers are checked, not remembered.

How to read this chapter. Every worked example carries a coloured tier badge — grey for warm-up mechanics, teal for Tier 1 (TCS NQT, Accenture, Infosys, Wipro, Capgemini), amber for Tier 2 (Grab, Shopee, GIC, DBS, Agoda, SCB) and dark red for Tier 3 (Google, Amazon, Microsoft, Goldman Sachs, D. E. Shaw, Adobe). The practice sets at the end climb the same four steps in order, so do them in order.

WHAT YOU NEED TO KNOW

Functional dependency. $X \rightarrow Y$ means: any two rows that agree on X must agree on Y . Read it as “ X determines Y ”, or “ X is a **determinant** of Y ”.

- **Trivial** if $Y \subseteq X$ (e.g. $AB \rightarrow A$). Always true, tells you nothing.
- **Full** dependency: $X \rightarrow Y$ and no proper subset of X determines Y .
- **Partial** dependency: X is a **proper subset of a candidate key** and $X \rightarrow Y$ where Y is non-prime.
- **Transitive** dependency: $X \rightarrow Y$, $Y \rightarrow Z$, with Y not a super key and Z non-prime.

Armstrong’s axioms (sound and complete):

- **Reflexivity:** if $Y \subseteq X$ then $X \rightarrow Y$.
- **Augmentation:** if $X \rightarrow Y$ then $XZ \rightarrow YZ$.
- **Transitivity:** if $X \rightarrow Y$ and $Y \rightarrow Z$ then $X \rightarrow Z$.

Derived rules (all provable from the three above):

- **Union:** $X \rightarrow Y$, $X \rightarrow Z \Rightarrow X \rightarrow YZ$.
- **Decomposition:** $X \rightarrow YZ \Rightarrow X \rightarrow Y$ and $X \rightarrow Z$.
- **Pseudo-transitivity:** $X \rightarrow Y$, $WY \rightarrow Z \Rightarrow WX \rightarrow Z$.

Attribute closure X^+ : everything X determines. Start with X ; repeatedly, if some $A \rightarrow B$ has $A \subseteq X^+$, add B . Stop when nothing changes.

- X is a **super key** $\Leftrightarrow X^+ = \text{all attributes}$.
- $X \rightarrow Y$ holds $\Leftrightarrow Y \subseteq X^+$.

Prime attribute: appears in at least one candidate key. Otherwise **non-prime**.

The normal forms

1NF	Every value is atomic. No repeating groups, no lists in a cell.
2NF	1NF + no non-prime attribute is partially dependent on a candidate key.
3NF	2NF + for every non-trivial $X \rightarrow A$: X is a super key or A is prime.
BCNF	For every non-trivial $X \rightarrow A$: X is a super key. No exception for prime A .
4NF	BCNF + no non-trivial multivalued dependency unless the left side is a super key.

Decomposition tests. Splitting R into R_1 and R_2 is **lossless** if

$$(R_1 \cap R_2) \rightarrow R_1 \quad \text{or} \quad (R_1 \cap R_2) \rightarrow R_2$$

that is, the shared attributes form a key of at least one fragment. It is **dependency preserving** if the union of the FDs visible inside each fragment still implies every original FD.

The guarantee. 3NF can always be reached losslessly **and** dependency-preservingly. BCNF can always be reached losslessly, but **may lose a dependency**.

6.1 Why normalise — the three anomalies

Here is a table a beginner would design for a college. One row per student per course.

roll	sname	hostel	course_id	course_name	grade
101	Asha	H1	CS01	Databases	A
101	Asha	H1	CS02	Operating Systems	B
102	Bhavin	H2	CS01	Databases	A
103	Chetan	H1	CS03	Networks	C

pink = the same fact stored twice

green = a fact that only one row is holding up

One wide table, three kinds of trouble. The tinted cells are the evidence.

UPDATE anomaly. Asha’s hostel is stored twice (pink). Move her to H3 and you must edit every one of her rows. Miss one and the database now says two different things at once.

DELETE anomaly. Chetan is the only student in CS03 (green). Delete his registration and the fact that “CS03 is called Networks” is gone forever — you lost a course by deleting a registration.

INSERT anomaly. A new course that nobody has registered for yet cannot be stored at all, because `roll` is part of the primary key and a primary key column cannot be NULL.

Every anomaly has the same root cause: **the table stores facts about more than one thing**. It mixes a fact about a student (`roll` → `hostel`) with a fact about a course (`course_id` → `course_name`) with a fact about a registration (`roll`, `course_id` → `grade`).

Normalisation is one rule applied over and over:

SHORTCUT

Every table should describe exactly one kind of thing, and every non-key column should be a fact about the whole key and nothing but the key.

The old exam mnemonic for 3NF: “the key, the whole key, and nothing but the key.”

- **the key** ⇒ 1NF (rows are identifiable)
- **the whole key** ⇒ 2NF (no partial dependency)
- **nothing but the key** ⇒ 3NF (no transitive dependency)

6.2 Functional dependencies

$X \rightarrow Y$ says: **if two rows have the same X , they must have the same Y .**

It does **not** say Y is unique, and it does **not** say anything about $Y \rightarrow X$.

Reading an FD out of a business rule

This is the step students skip. Practise it.

Business rule	FD
---------------	----

Every student has one hostel.	$\text{roll} \rightarrow \text{hostel}$
Every course has one name and one instructor.	$\text{course_id} \rightarrow \text{course_name}, \text{instructor}$
A student gets one grade in a course.	$\text{roll}, \text{course_id} \rightarrow \text{grade}$
A teacher teaches only one subject.	$\text{tutor} \rightarrow \text{subject}$
Two students may share a hostel.	nothing — this is not an FD
Each PIN code lies in exactly one city.	$\text{pin} \rightarrow \text{city}$
A city can have many PIN codes.	$\text{city} \nrightarrow \text{pin}$

TRAP

“The data shows no repeats, so it is an FD.” Wrong direction. Sample data can only **disprove** an FD — find two rows that agree on X and disagree on Y , and $X \rightarrow Y$ is dead. It can never **prove** one. FDs come from the rules of the business, not from today’s rows. Say this sentence in the interview.

Armstrong’s axioms, proved once

You will be asked to **derive** an FD. Use the three axioms.

Example 1 [Infosys · pattern]

Given $A \rightarrow B$ and $BC \rightarrow D$, prove $AC \rightarrow D$.

SOLUTION

1. $A \rightarrow B$ — given.
2. $AC \rightarrow BC$ — **augmentation** of line 1 with C .
3. $BC \rightarrow D$ — given.
4. $AC \rightarrow D$ — **transitivity** of lines 2 and 3. ■

That combination is used so often it has its own name: **pseudo-transitivity**.

Answer: augment, then apply transitivity

Example 2 [TCS NQT · pattern]

Is $A \rightarrow BC$ equivalent to the pair $\{A \rightarrow B, A \rightarrow C\}$? Is $AB \rightarrow C$ equivalent to $\{A \rightarrow C, B \rightarrow C\}$?

SOLUTION

First pair — yes, equivalent.

- Forward: $A \rightarrow BC$ gives $A \rightarrow B$ and $A \rightarrow C$ by **decomposition**.
- Backward: $A \rightarrow B$ and $A \rightarrow C$ give $A \rightarrow BC$ by **union**.

Second pair — no. Decomposition splits the **right** side only, never the left.

- $\{A \rightarrow C, B \rightarrow C\}$ does imply $AB \rightarrow C$ (augment $A \rightarrow C$ with B).
- But $AB \rightarrow C$ does **not** imply $A \rightarrow C$. Counter-example: a table where $(1, x) \rightarrow 5$ and $(1, y) \rightarrow 9$. Here AB fixes C , but A alone does not.

Answer: right sides split; left sides do not

TRAP

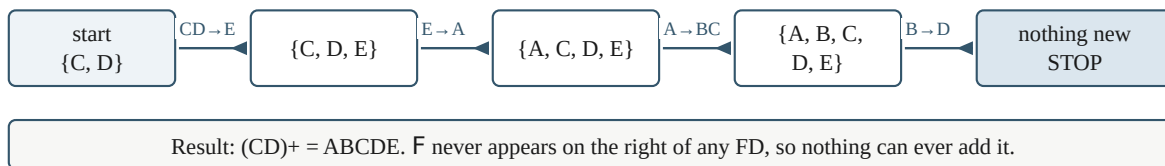
The single most common mistake in this topic: splitting the left-hand side. $AB \rightarrow C$ **never** gives you $A \rightarrow C$.

6.3 Attribute closure — the one algorithm you must own

X^+ is the set of every attribute that X determines.

The procedure

1. Start: $X^+ = X$.
2. Scan every FD. If its left side is fully inside X^+ , add its right side to X^+ .
3. Repeat the scan until one full pass adds nothing new.



Computing $(CD)^+$ for the FD set $A \rightarrow BC$, $CD \rightarrow E$, $B \rightarrow D$, $E \rightarrow A$. Each arrow is one FD firing.

Example 3 [Wipro · pattern]

$R(A, B, C, D, E, F)$ with $F = \{A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow A\}$. Compute A^+ , B^+ and $(AF)^+$.

SOLUTION

A^+ :

- Start $\{A\}$.
- $A \rightarrow BC$ fires (A is in). Now $\{A, B, C\}$.
- $B \rightarrow D$ fires. Now $\{A, B, C, D\}$.
- $CD \rightarrow E$ fires (both C and D are in). Now $\{A, B, C, D, E\}$.
- $E \rightarrow A$ fires but A is already there. Nothing new. Stop.

$A^+ = ABCDE$. Note F is missing.

B^+ :

- Start $\{B\}$.
- $B \rightarrow D$ fires. Now $\{B, D\}$.
- $A \rightarrow BC$? A is not in the set. ✗
- $CD \rightarrow E$? C is not in the set. ✗
- $E \rightarrow A$? E is not in the set. ✗
- Nothing new. Stop.

$B^+ = BD$.

$(AF)^+$:

- Start $\{A, F\}$. Everything that fired for A^+ fires again, and F just rides along.
- $(AF)^+ = ABCDEF$ = all six attributes.

Answer: $A^+ = ABCDE$, $B^+ = BD$, $(AF)^+ = ABCDEF$ — so AF is a super key and A alone is not

SHORTCUT

Three shortcuts that save real time:

1. **Any attribute that never appears on the right of any FD must be in every candidate key.** Nothing can derive it, so it has to be given. In the example above, F is never on a right side $\Rightarrow$ every candidate key contains F .
2. **Any attribute that never appears on the left of any FD is never needed in a key.**
3. Once X^+ reaches all attributes, **stop scanning**. You are done.

6.4 Finding all candidate keys

The exam-safe procedure:

1. Split the attributes into three buckets: **only-left** (L), **only-right** (R), **both** (B), plus **neither** (N).
2. Every attribute in L and in N **must** be in every candidate key. Call this seed S .

3. Compute S^+ . If it is everything, S is the **only** candidate key — stop.
4. Otherwise try S plus one attribute from B , then S plus two, and so on. Test the smallest sets first; once a set is a key, never test a superset of it.

Example 4 [Grab · pattern]

Find every candidate key of $R(A, B, C, D, E, F)$ with $F = \{A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow A\}$.

SOLUTION

Step 1 — bucket the attributes.

Attribute	Appears on the left of	Appears on the right of
A	$A \rightarrow BC$	$E \rightarrow A$
B	$B \rightarrow D$	$A \rightarrow BC$
C	$CD \rightarrow E$	$A \rightarrow BC$
D	$CD \rightarrow E$	$B \rightarrow D$
E	$E \rightarrow A$	$CD \rightarrow E$
F	—	—

F is in bucket N (neither side) $\Rightarrow$ **F is in every candidate key**. Seed $S = \{F\}$.

Step 2 — is the seed enough? $F^+ = F$. Not everything, so we must add attributes.

Step 3 — try one extra attribute at a time.

Try	Closure	Key?
AF	$A^+ = ABCDE$, plus F $\Rightarrow ABCDEF$	yes
BF	$B^+ = BD$, plus F $\Rightarrow BDF$	no
CF	$C^+ = C$, plus F $\Rightarrow CF$	no
DF	$D^+ = D$, plus F $\Rightarrow DF$	no
EF	E^+ : $E \rightarrow A$, $A \rightarrow BC$, $B \rightarrow D$, so $ABCDE$; plus F	yes

Step 4 — try two extra attributes, skipping any set containing AF or EF. So only combinations of B, C, D are left: BCF, BDF, CDF.

Try	Closure	Key?
BCF	$B \rightarrow D$ gives BCD; $CD \rightarrow E$ gives E; $E \rightarrow A$ gives A $\Rightarrow ABCDEF$	yes
BDF	$B \rightarrow D$ adds nothing new. Stuck at BDF	no
CDF	$CD \rightarrow E$ gives E; $E \rightarrow A$ gives A; $A \rightarrow BC$ adds B $\Rightarrow ABCDEF$	yes

Step 5 — three extra attributes? Every remaining 3-subset of $\{B, C, D\}$ is BCD, which contains BC, so BCDF is a superset of the key BCF — not minimal. Done.

Answer: AF, EF, BCF, CDF — four candidate keys

Prime attributes = the union of all four = $\{A, B, C, D, E, F\}$. **Every** attribute is prime here. That fact decides the normal form in section 6.9 — keep it.

The program in section 6.16 was run on this exact relation. It printed `keys = AF|EF|BCF|CDF` and `prime = ABCDEF`. The hand work above matches.

6.5 First normal form

1NF: every value in every cell is atomic, and there are no repeating groups.

Bad:

roll	sname	phones
101	Asha	9000011111, 9000022222
102	Bhavin	9000033333

Also bad (a repeating group spread across columns):

roll	sname	phone1	phone2
101	Asha	9000011111	9000022222
102	Bhavin	9000033333	NULL

Why the second one is still wrong: “how many phone columns is enough?” There is no answer. And `SELECT ... WHERE phone = '9000022222'` now has to search every phone column.

Good — one fact per row:

roll	sname
101	Asha
102	Bhavin

roll	phone
101	9000011111
101	9000022222
102	9000033333

Primary key of the second table: {roll, phone}.

TRAP

“A JSON column or an array column breaks 1NF, so modern databases are not relational.” Careful. The honest answer: “Strictly, a multi-valued column violates 1NF. In practice PostgreSQL arrays and JSONB are used deliberately when the inner values are never queried or joined on individually — it is a conscious trade of purity for speed. The moment you need to filter or join on an element, you should normalise it into rows.” That answer scores far higher than a flat yes or no.

6.6 Second normal form

2NF: 1NF, and no non-prime attribute depends on only PART of a candidate key.

This can only go wrong when the candidate key is **composite**. If every candidate key is a single attribute, a partial dependency is impossible and 2NF is automatic.

Example 5 [Capgemini · pattern]

REG(roll, course_id, sname, hostel, course_name, instructor, grade) with $\text{roll} \rightarrow \text{sname}, \text{hostel}, \text{course_id} \rightarrow \text{course_name}, \text{instructor}, \text{roll}, \text{course_id} \rightarrow \text{grade}$. Find the candidate key, name the highest normal form, and decompose to 2NF.

SOLUTION

Candidate key. roll is never on a right side; neither is course_id. Both are in bucket L, so both are in every key. Seed = {roll, course_id}.

Closure of {roll, course_id}:

- start {roll, course_id}
- $\text{roll} \rightarrow \text{sname}, \text{hostel}$ fires $\Rightarrow$ add both
- $\text{course_id} \rightarrow \text{course_name}, \text{instructor}$ fires $\Rightarrow$ add both
- $\text{roll}, \text{course_id} \rightarrow \text{grade}$ fires $\Rightarrow$ add grade
- all 7 attributes. ✓

So {roll, course_id} is the only candidate key. Prime = {roll, course_id}. Non-prime = sname, hostel, course_name, instructor, grade.

Highest normal form. Look for a partial dependency:

- roll → sname — roll is a **proper subset** of the key, and sname is non-prime. **Partial dependency**. 2NF fails.
- course_id → course_name — same problem.
- roll, course_id → grade — full dependency, fine.

Answer: the relation is in 1NF only

Decomposition. Give every determinant its own table.

Table	Attributes	Key
STUDENT	roll, sname, hostel	roll
COURSE	course_id, course_name, instructor	course_id
REGISTRATION	roll, course_id, grade	{roll, course_id}

Check it. The program in 6.16 confirms: the decomposition is **lossless** and **dependency preserving**, and each of the three fragments is in BCNF — so this one split jumped the table from 1NF all the way to BCNF.

And notice all three anomalies from section 6.1 are gone:

- Asha's hostel now lives in **one** row of STUDENT — no update anomaly.
- Deleting Chetan's registration leaves COURSE untouched — no delete anomaly.
- A new course goes straight into COURSE with no student — no insert anomaly.

TRAP

A famous wrong split: STUDENT(roll, sname, hostel) + REST(course_id, course_name, instructor, grade). It **looks** tidier, but it is **lossy** — the fragments share no attribute at all, so grade can never be reconnected to a student. The program reports lossless: false for it. Always run the lossless test before you accept a split.

6.7 Third normal form

3NF: 2NF, and for every non-trivial $X \rightarrow A$, either X is a super key or A is prime.

The everyday phrasing: **no non-prime attribute depends on another non-prime attribute**. That is a transitive dependency — key → something → something else.

Example 6 [Accenture · pattern]

EMP(emp_id, emp_name, dept_id, dept_name, dept_head) with emp_id → emp_name, dept_id and dept_id → dept_name, dept_head. Name the highest normal form and fix it.

SOLUTION

Candidate key. emp_id is never on a right side ⇒ it is in every key. emp_id⁺: adds emp_name, dept_id, then dept_id fires and adds dept_name, dept_head ⇒ all 5. So {emp_id} is the only candidate key.

Prime = {emp_id}. Non-prime = the other four.

2NF? The key is a single attribute, so no proper subset of it exists (other than the empty set). No partial dependency is possible. **2NF holds.**

3NF? Check dept_id → dept_name:

- Is dept_id a super key? dept_id⁺ = {dept_id, dept_name, dept_head} — no emp_id, so no.
- Is dept_name prime? No.

Both tests fail ⇒ **3NF fails**. The chain is emp_id → dept_id → dept_name: a **transitive dependency**.

Answer: highest normal form is 2NF

Fix — split at the offending determinant.

Table	Attributes	Key
EMPLOYEE	emp_id, emp_name, dept_id	emp_id
DEPARTMENT	dept_id, dept_name, dept_head	dept_id

Lossless? The shared attribute is dept_id, and dept_id⁺ contains all of DEPARTMENT. Shared attributes form a key of one fragment ⇒ **lossless**. ✓

Dependency preserving? emp_id → emp_name, dept_id lives inside EMPLOYEE; dept_id → dept_name, dept_head lives inside DEPARTMENT. Nothing lost. ✓

Both fragments are in BCNF (each has a single-attribute key that determines everything else). Verified by the program in 6.16.

The anomaly this removed: previously, if the last employee of a department left, the department's name and head vanished with them.

2NF failure vs 3NF failure, in one line each.

- 2NF failure: a non-key column depends on **part of** the key. (Only possible with a composite key.)
- 3NF failure: a non-key column depends on **another non-key column**.

6.8 Boyce–Codd normal form

BCNF: for every non-trivial $X \rightarrow A$, X must be a super key. Full stop.

BCNF is 3NF with the escape hatch removed. 3NF forgives a violation when the right-hand attribute happens to be prime; BCNF does not.

Example 7 [Sea/Shopee · pattern]

A tutoring centre records COACH(student, subject, tutor) under two rules:

1. Each tutor teaches exactly one subject.
2. For a given subject, a student is assigned exactly one tutor.

Find the candidate keys, the highest normal form, and decompose if needed.

SOLUTION

FDs from the rules.

- Rule 1: tutor → subject.
- Rule 2: {student, subject} → tutor.

Candidate keys.

- {student, subject}⁺: add tutor by rule 2 ⇒ all three. **Key**. Minimal? student alone determines nothing; subject alone determines nothing. Yes, minimal.
- {student, tutor}⁺: tutor → subject adds subject ⇒ all three. **Key**. Minimal for the same reason.
- {subject, tutor}? Closure is {subject, tutor} — no student. Not a key.

Candidate keys: {student, subject} and {student, tutor}.

Prime attributes = student, subject, tutor — **all three**.

3NF? Check every FD:

- {student, subject} → tutor: left side is a super key ✓
- tutor → subject: tutor is not a super key ✗ — **but** subject is prime ✓

3NF's rule is “super key OR prime right side”, and the second test passes. **The relation is in 3NF.**

BCNF? tutor → subject with tutor not a super key. No escape hatch. **BCNF fails.**

Answer: 3NF, not BCNF

Why it actually matters. Store the row (Asha, Databases, Ravi). The fact “Ravi teaches Databases” is repeated in every row where Ravi appears. If Ravi switches to Networks you must edit them all — an update anomaly **surviving in a 3NF table**. That is exactly why BCNF exists.

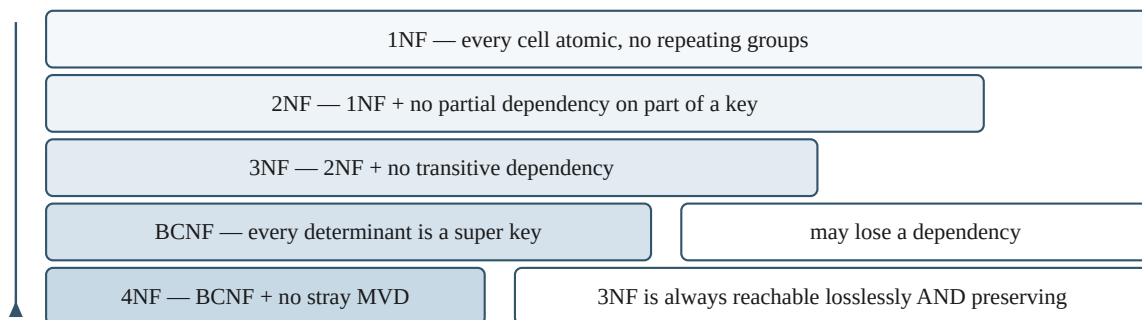
Decomposition. Split on the violating FD $X \rightarrow Y$ into X^+ and $X \cup (R - X^+)$: tutor⁺ = {tutor, subject}.

Table	Attributes	Key
TEACHES	tutor, subject	tutor
ASSIGNED	student, tutor	{student, tutor}

Lossless? Shared attribute is tutor, and $\text{tutor}^+ = \text{all of TEACHES}$. ✓ Verified.

Dependency preserving? TEACHES keeps $\text{tutor} \rightarrow \text{subject}$. But $\{\text{student}, \text{subject}\} \rightarrow \text{tutor}$ is now **split across two tables** — student is in one, subject in the other — so no single table can enforce it. **The FD is lost.** Verified: the program prints dependency preserving: false, and the projected FDs are TEACHES: $\text{tutor} \rightarrow \text{subject}$ and ASSIGNED: (none).

The consequence to say out loud: after this split, the database can no longer stop you inserting (Asha, Ravi) and (Asha, Meera) when Ravi and Meera both teach Databases — which rule 2 forbids. You have traded a redundancy problem for a constraint-checking problem. This is the classic BCNF trade-off.



The normal forms nest. Each step removes one more kind of redundancy — and each step costs you more joins at read time.

6.9 3NF versus BCNF — the case that decides the round

Example 8 [Microsoft · pattern]

$R(A, B, C, D, E, F)$ with $F = \{A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow A\}$. Name the highest normal form, with proof.

SOLUTION

From Example 4: candidate keys are AF, EF, BCF, CDF, and **every** attribute is prime.

Test each FD against the 3NF rule (“ X is a super key OR every attribute on the right is prime”):

FD	Is the left a super key?	Is the right prime?	3NF
$A \rightarrow BC$	$A^+ = ABCDE$, no $F \Rightarrow$ no	B prime ✓, C prime ✓	passes
$CD \rightarrow E$	$(CD)^+ = ABCDE$, no $F \Rightarrow$ no	E prime ✓	passes
$B \rightarrow D$	$B^+ = BD \Rightarrow$ no	D prime ✓	passes
$E \rightarrow A$	$E^+ = ABCDE$, no $F \Rightarrow$ no	A prime ✓	passes

Every FD passes 3NF. And 3NF implies 2NF implies 1NF, so the relation is in 3NF.

Now BCNF. BCNF needs the left side to be a super key, with no prime-attribute excuse. All four left sides fail. **BCNF fails on all four FDs.**

Answer: highest normal form is 3NF

The insight to state. When every attribute is prime, a relation is **automatically** in 3NF — the “right side is prime” clause can never fail. So a question where all attributes are prime is always a 3NF-not-BCNF question. Spotting this in ten seconds is the whole trick.

SHORTCUT

Fast normal-form triage, in order:

1. Compute the candidate keys. Mark prime and non-prime attributes.
2. For each FD $X \rightarrow Y$ (drop the trivial part): is X a super key? If yes, this FD is fine at every level. Move on.
3. If not: is every attribute of Y prime? If yes $\Rightarrow$ 3NF survives, BCNF dies.
4. If some attribute of Y is non-prime: is X a **proper subset of a candidate key**?
 - Yes $\Rightarrow$ 2NF dies (partial dependency). Highest form is 1NF.
 - No $\Rightarrow$ 2NF survives, 3NF dies (transitive dependency). Highest form is 2NF.
5. The answer is the **worst** verdict across all FDs.

6.10 Lossless join and dependency preservation

Any table can be shredded into tiny pieces. The question is whether you can put it back.

The lossless join test (two fragments)

Splitting R into R_1 and R_2 is lossless if and only if

$$(R_1 \cap R_2) \rightarrow R_1 \quad \text{or} \quad (R_1 \cap R_2) \rightarrow R_2$$

In words: **compute the closure of the shared attributes; if it swallows either fragment completely, the split is safe.**

Example 9 [GIC · pattern]

EMP(emp_id, emp_name, dept_id, dept_name, dept_head) with emp_id $\rightarrow$ emp_name, dept_id and dept_id $\rightarrow$ dept_name, dept_head. Test two splits:

(a) (emp_id, emp_name, dept_id) + (dept_id, dept_name, dept_head)

(b) (emp_id, emp_name, dept_name) + (dept_id, dept_name, dept_head)

SOLUTION

(a) Shared attributes = {dept_id}. dept_id⁺ = {dept_id, dept_name, dept_head} — that is exactly fragment 2. Shared $\rightarrow R_2$ ✓ $\Rightarrow$ **lossless**.

(b) Shared attributes = {dept_name}. dept_name⁺ = {dept_name} — dept_name is never on the left of any FD, so its closure adds nothing. It does not cover either fragment. $\Rightarrow$ **lossy**.

Verified by the program: (a) {"ok":true,"common":"D","closure":"DHM"} and (b) {"ok":false,"common":"M","closure":"M"} (letters: E=emp_id, N=emp_name, D=dept_id, M=dept_name, H=dept_head).

Answer: (a) lossless, (b) lossy

What “lossy” actually means. Rejoin (b) on dept_name and you get **spurious tuples** — rows that were never in the original table. If two departments happened to share a name, every employee of one would appear to belong to the other as well. Lossy decomposition does not lose rows; it **invents** them.

TRAP

“Lossy means rows disappear.” No — a lossy natural join gives you **more** rows than you started with, not fewer. The information that is lost is the ability to tell the real rows from the invented ones.

Dependency preservation

A decomposition preserves dependencies if every original FD can still be checked **inside a single fragment**, without a join.

Why it matters: a constraint you can only check with a join is a constraint the database will not enforce for you cheaply. Every INSERT would need a join.

Target	Lossless?	Dependency preserving?
3NF	always achievable	always achievable
BCNF	always achievable	not always

That single row is the answer to “what is the difference between 3NF and BCNF in practice?”

6.11 Minimal cover — the three-step algorithm

A **minimal cover** (or canonical cover) F_c is the smallest FD set that means exactly the same as F . You need it before 3NF synthesis, and it is asked on its own.

1. **Split right sides.** Rewrite every FD so it has exactly one attribute on the right. ($X \rightarrow YZ$ becomes $X \rightarrow Y$ and $X \rightarrow Z$.)
2. **Trim left sides.** For each FD $X \rightarrow A$ with $|X| \geq 2$, try removing each attribute B from X . If $(X - B)^+$ (under the **current** FD set) still contains A , then B was extraneous — drop it.
3. **Drop whole FDs.** For each FD $X \rightarrow A$, remove it temporarily and compute X^+ under the **remaining** FDs. If A is still there, the FD was redundant — delete it for good.

Example 10 [Cognizant · pattern]

Find a minimal cover of $F = \{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C, AC \rightarrow D\}$ over $R(A, B, C, D)$.

SOLUTION

Step 1 — split right sides. $A \rightarrow B, A \rightarrow C, B \rightarrow C, A \rightarrow B$ (duplicate, keep one), $AB \rightarrow C, AC \rightarrow D$.

Working set: $\{A \rightarrow B, A \rightarrow C, B \rightarrow C, AB \rightarrow C, AC \rightarrow D\}$.

Step 2 — trim left sides. Only $AB \rightarrow C$ and $AC \rightarrow D$ have two attributes.

$AB \rightarrow C$:

- Drop A , leaving $B \rightarrow C$. Is $C \in B^+$? $B^+ = \{B, C\}$ using $B \rightarrow C$. **Yes** $\Rightarrow A$ was extraneous. $AB \rightarrow C$ becomes $B \rightarrow C$ (a duplicate of one we already have).

$AC \rightarrow D$:

- Drop C , leaving $A \rightarrow D$. Is $D \in A^+$? A^+ : $A \rightarrow B$ gives B , $A \rightarrow C$ gives C , then $AC \rightarrow D$ fires and gives D . **Yes** $\Rightarrow C$ was extraneous. $AC \rightarrow D$ becomes $A \rightarrow D$.

Working set: $\{A \rightarrow B, A \rightarrow C, B \rightarrow C, B \rightarrow C, A \rightarrow D\}$.

Step 3 — drop redundant FDs.

- $A \rightarrow B$: remove it. Remaining $\{A \rightarrow C, B \rightarrow C, A \rightarrow D\}$. $A^+ = \{A, C, D\}$ — no B . **Keep it.**
- $A \rightarrow C$: remove it. Remaining $\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$. A^+ : $A \rightarrow B$ gives B , $B \rightarrow C$ gives $C \Rightarrow C$ is there. **Redundant — delete.**
- The duplicate $B \rightarrow C$: one copy is obviously redundant. **Delete the copy.**
- $B \rightarrow C$ (the surviving copy): remove it. Remaining $\{A \rightarrow B, A \rightarrow D\}$. $B^+ = \{B\}$ — no C . **Keep it.**
- $A \rightarrow D$: remove it. Remaining $\{A \rightarrow B, B \rightarrow C\}$. $A^+ = \{A, B, C\}$ — no D . **Keep it.**

Answer: $F_c = \{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$

Program trace, printed exactly:

node fd2.js

TEXT

```
after split : A->B, A->C, B->C, AB->C, AC->D
after LHS   : A->B, A->C, B->C, B->C, A->D
after drop  : A->B, B->C, A->D
MINIMAL COVER: A->B, B->C, A->D
keys of R(A,B,C,D): A
```

Sanity check — the cover must still derive everything the original did. Original had $A \rightarrow C$. Under the cover: $A \rightarrow B$ and $B \rightarrow C$ give $A \rightarrow C$ by transitivity. ✓ Original had $AC \rightarrow D$. Under the cover: $A \rightarrow D$, so augmenting gives $AC \rightarrow D$. ✓

TRAP

Order matters in step 3, and that is **allowed**. A minimal cover is not unique — removing FDs in a different order can give a different but equally valid cover. If the interviewer's answer differs from yours, check that each one derives the other; do not assume you are wrong.

Example 11 [SCB · pattern]

Find a minimal cover of $F = \{P \rightarrow Q, PQ \rightarrow R, R \rightarrow S, RS \rightarrow T, T \rightarrow U, PU \rightarrow T\}$ over $R(P, Q, R, S, T, U)$, and give the candidate key.

SOLUTION

Step 1 — split. Every right side is already a single attribute. Nothing to do.

Step 2 — trim left sides.

- $PQ \rightarrow R$: drop Q, leaving $P \rightarrow R$. Is $R \in P^+$? P^+ : $P \rightarrow Q$ gives Q, then $PQ \rightarrow R$ gives R. **Yes** $\Rightarrow$ becomes $P \rightarrow R$.
- $RS \rightarrow T$: drop S, leaving $R \rightarrow T$. Is $T \in R^+$? R^+ : $R \rightarrow S$ gives S, then $RS \rightarrow T$ gives T. **Yes** $\Rightarrow$ becomes $R \rightarrow T$.
- $PU \rightarrow T$: drop U, leaving $P \rightarrow T$. Is $T \in P^+$? $P^+ = \{P, Q, R, S, T, U\}$ ($P \rightarrow Q, P \rightarrow R, R \rightarrow S, R \rightarrow T, T \rightarrow U$). **Yes** $\Rightarrow$ becomes $P \rightarrow T$.

Working set: $\{P \rightarrow Q, P \rightarrow R, R \rightarrow S, R \rightarrow T, T \rightarrow U, P \rightarrow T\}$.

Step 3 — drop redundant FDs.

- $P \rightarrow T$: remove it. Remaining set still gives $P \rightarrow R \rightarrow T$. **Redundant — delete.**
- The other five are each needed (remove any one and the closure of its left side loses that attribute).

Answer: $F_c = \{P \rightarrow Q, P \rightarrow R, R \rightarrow S, R \rightarrow T, T \rightarrow U\}$, candidate key **{P}**

Program output, exactly:

node fd3.js

TEXT

```
after split : P->Q, PQ->R, R->S, RS->T, T->U, PU->T
after LHS   : P->Q, P->R, R->S, R->T, T->U, P->T
after drop  : P->Q, P->R, R->S, R->T, T->U
keys: P
```

6.12 The two decomposition algorithms

3NF synthesis — always lossless, always dependency preserving

1. Compute a minimal cover F_c .
2. Group the FDs of F_c by identical left side. Make one relation per group, containing the left side plus all its right sides.
3. If no fragment contains a whole candidate key, add one more relation that **is** a candidate key.
4. Delete any fragment whose attributes are a subset of another fragment's.

Example 12 [Agoda · pattern]

Synthesise a 3NF schema for $R(P, Q, R, S, T, U)$ using the minimal cover from Example 11.

SOLUTION

Step 1. $F_c = \{P \rightarrow Q, P \rightarrow R, R \rightarrow S, R \rightarrow T, T \rightarrow U\}$.

Step 2 — group by left side.

Left side	FDs in the group	Fragment
P	$P \rightarrow Q, P \rightarrow R$	R1(P, Q, R)
R	$R \rightarrow S, R \rightarrow T$	R2(R, S, T)
T	$T \rightarrow U$	R3(T, U)

Step 3 — does some fragment hold a candidate key? The candidate key is {P}, and R1(P,Q,R) contains P.
✓ No extra relation needed.

Step 4 — any fragment inside another? {P,Q,R}, {R,S,T}, {T,U} — no subset relationships. Nothing to delete.

Answer: R1(P, Q, R), R2(R, S, T), R3(T, U)

Verified by the program:

```
node fd4.js TEXT

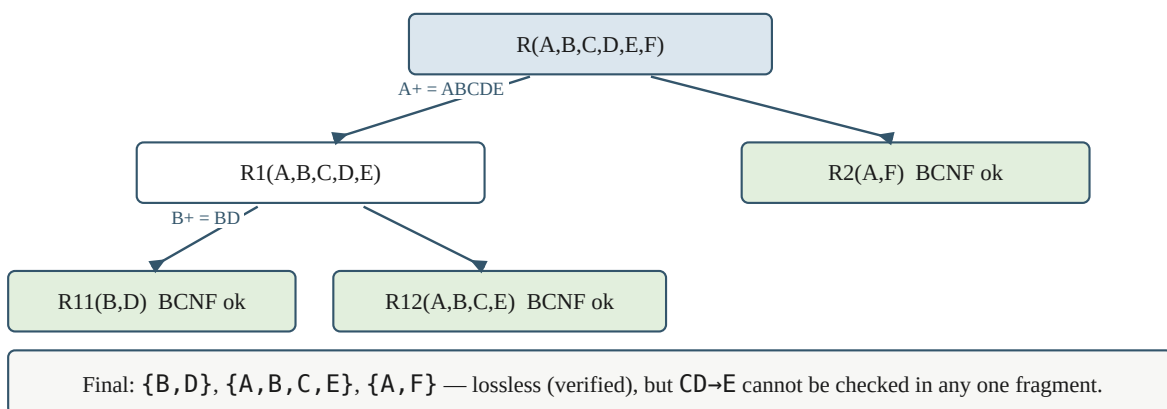
fragments      : PQR , RST , TU
candidate key(s) : P
lossless       : true
dep preserving  : true
frag PQR -> keys P    -> in BCNF
frag RST -> keys R    -> in BCNF
frag TU  -> keys T    -> in BCNF
```

Here 3NF synthesis happened to land in BCNF as well. That is common but not guaranteed.

BCNF decomposition — always lossless, may lose a dependency

Repeat until no violation remains:

1. Find a non-trivial FD $X \rightarrow Y$ inside a fragment where X is not a super key **of that fragment**.
2. Replace the fragment with X^+ and $X \cup (R_i - X^+)$.



BCNF decomposition of $R(A,B,C,D,E,F)$. Lossless at every step — but $CD \rightarrow E$ ends up split across two fragments and is lost.

Example 13 [D. E. Shaw · pattern]

Decompose $R(A, B, C, D, E, F)$, $F = \{A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow A\}$, into BCNF. Then state whether dependencies are preserved.

SOLUTION

Round 1. Pick the violating FD $A \rightarrow BC$. $A^+ = ABCDE$ (Example 3), which is not all of R , so A is not a super key $\Rightarrow$ genuine violation.

- Fragment 1: $A^+ = R_1(A, B, C, D, E)$
- Fragment 2: $A \cup (R - A^+) = \{A\} \cup \{F\} = R_2(A, F)$

Lossless? Shared = {A}, and A^+ covers R_1 entirely. ✓

Is $R_2(A, F)$ in BCNF? The only FDs visible inside it are trivial; its key is {A, F}. BCNF ✓.

Round 2 — inside $R_1(A, B, C, D, E)$. Its candidate keys are A, E, BC, CD (drop the F from Example 4's answer). Check $B \rightarrow D$: $B^+ = BD$, not a super key of $R_1 \Rightarrow$ violation.

- Fragment: $B^+ = R_{11}(B, D)$
- Fragment: $B \cup (R_1 - B^+) = \{B\} \cup \{A, C, E\} = R_{12}(A, B, C, E)$

Lossless? Shared = {B}, and $B^+ = BD$ covers R_{11} . ✓

Is $R_{11}(B, D)$ in BCNF? Key B, and $B \rightarrow D$ has a super-key left side. ✓

Is $R_{12}(A, B, C, E)$ in BCNF? Project the FDs onto these four attributes: $A \rightarrow BCE$ and $E \rightarrow ABC$. So the candidate keys are A, E and BC, and every determinant is a super key. BCNF ✓. (The program prints keys ABCE : A | E | BC and verdict: in BCNF.)

Answer: {B, D}, {A, B, C, E}, {A, F} — all three in BCNF, decomposition lossless

Dependencies preserved? No. Look at $CD \rightarrow E$. After the split, C lives in {A, B, C, E} and D lives in {B, D}. No single fragment holds both, so no fragment can enforce the rule. Program output: dep preserving: false.

The sentence that wins the follow-up: “This is the guaranteed trade-off. BCNF is always reachable losslessly, but not always with dependency preservation. If losing $CD \rightarrow E$ matters to the business, I would stop at 3NF — where both properties are guaranteed — and enforce the redundancy with a trigger or an application check.”

6.13 Beyond BCNF — MVDs, 4NF and 5NF

Multivalued dependency

$X \Rightarrow Y$ (“ X multidetermines Y ”) means: for a fixed X , the set of Y values is **independent** of the other columns. Concrete case. A staff member can hold several skills **and** speak several languages, and the two lists have nothing to do with each other.

emp_id	skill	language
E1	Java	Hindi
E1	Java	Tamil
E1	SQL	Hindi
E1	SQL	Tamil

Four rows to record **two** skills and **two** languages. Add one more language and you must add **two** more rows. That is 2×2 growth for facts that are unrelated.

Here $\text{emp_id} \Rightarrow \text{skill}$ and $\text{emp_id} \Rightarrow \text{language}$. The table is in BCNF — its only key is all three columns together, and there is no non-trivial FD at all — yet the redundancy is obvious.

4NF fix: split into $\text{EMP_SKILL}(\text{emp_id}, \text{skill})$ and $\text{EMP_LANG}(\text{emp_id}, \text{language})$. Two rows and two rows instead of four. Adding a language now costs one row.

Every FD is also an MVD (if $X \rightarrow Y$ then $X \Rightarrow Y$), so 4NF implies BCNF. The nesting is $1\text{NF} \supset 2\text{NF} \supset 3\text{NF} \supset \text{BCNF} \supset 4\text{NF} \supset 5\text{NF}$.

Example 14 [Adobe · pattern]

A college keeps $\text{COURSE_INFO}(\text{course}, \text{instructor}, \text{book})$. Each course is taught by a set of instructors and prescribes a set of textbooks, and the two sets have nothing to do with each other — any instructor of the course may use any of its books.

Course CS05 has instructors {Rao, Iyer} and books {Vol1, Vol2, Vol3}.

(a) How many rows does the table hold for CS05? (b) Is the table in BCNF? (c) What happens when a fourth book is prescribed? (d) Decompose it and show the row count afterwards.

SOLUTION

(a) Because the two sets are independent, every instructor must be paired with every book, or the table would falsely suggest that Rao does not use Vol3. $2 \times 3 = 6$ rows:

course	instructor	book
CS05	Rao	Vol1
CS05	Rao	Vol2
CS05	Rao	Vol3
CS05	Iyer	Vol1
CS05	Iyer	Vol2
CS05	Iyer	Vol3

(b) Look for a non-trivial FD.

- $\text{course} \rightarrow \text{instructor}$? No — CS05 has two instructors.
- $\text{course} \rightarrow \text{book}$? No — three books.
- $\{\text{course}, \text{instructor}\} \rightarrow \text{book}$? No — Rao appears with three books.
- $\{\text{course}, \text{book}\} \rightarrow \text{instructor}$? No — Vol1 appears with two instructors.

There is **no** non-trivial FD at all. So the only candidate key is all three columns together, every attribute is prime, and every determinant is trivially a super key. **The table is in BCNF.** It is even in 3NF, 2NF and 1NF. And yet it is obviously redundant.

(c) Adding Vol4 forces you to insert **two** rows — one per instructor. Hiring a third instructor would force **four** new rows. The cost of one fact is multiplied by the size of an unrelated set. That is the signature of a multivalued dependency: $\text{course} \Rightarrow \text{instructor}$ and $\text{course} \Rightarrow \text{book}$.

(d) 4NF decomposition — one table per independent set.

course	instructor	
CS05	Rao	
CS05	Iyer	

course	book	
CS05	Vol1	
CS05	Vol2	
CS05	Vol3	

$2 + 3 = 5$ rows instead of 6. That looks like a small win here — but the saving is $m + n$ instead of $m \times n$, so at 10 instructors and 40 books it is 50 rows instead of 400. Adding a book now costs exactly **one** row.

Answer: (a) 6 (b) yes, BCNF (c) two new rows for one new fact (d) split into (course, instructor) and (course, book): 5 rows

Lossless? Yes — and this is the **definition** of the MVD. $\text{course} \Rightarrow \text{instructor}$ holds precisely when joining the two projections back on course reproduces the original table exactly, with no spurious rows. If the sets were **not** independent (say Rao only ever used Vol1), the MVD would not hold, the join would invent the row (CS05, Rao, Vol2), and the split would be lossy.

The trap in this question: a candidate who only knows FDs will say “it is in BCNF, so it is fully normalised” and stop. The redundancy is real and visible, and FD-based normal forms cannot see it. That is the entire reason 4NF exists — say so.

5NF in one paragraph

5NF (project-join normal form) deals with a table that cannot be split into **two** pieces losslessly, but **can** be split into three. It comes up when a three-way business rule is genuinely three separate two-way rules — for example “supplier S stocks part P”, “part P is used in project J”, “supplier S supplies project J”, and a row exists only when all three pairs hold. In practice you will almost never meet this. Know the name, know it is about **join dependencies**, and move on.

6.14 When to stop — denormalisation

Normalisation removes redundancy. Redundancy is what makes reads fast. So there is a price.

Normalised	Denormalised
No update anomalies	Must update the same fact in several places
Smaller tables, smaller writes	Bigger rows, more disk
More joins per query	Fewer joins — sometimes none
Constraints enforced by keys	Constraints enforced by application code

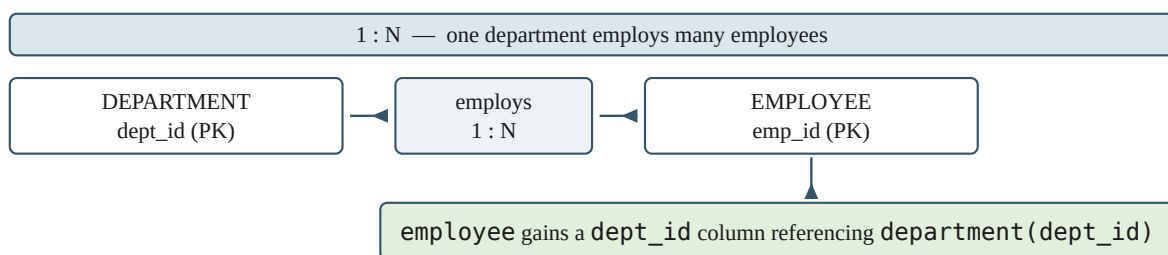
Legitimate reasons to denormalise **on purpose**:

- **Reporting and analytics.** A star schema keeps one big fact table plus wide dimension tables that are deliberately not in 3NF, because the workload is read-heavy and the data is loaded in batches.
- **A stored aggregate.** Keeping `order.total_amount` alongside the line items saves a SUM on every page view. The cost: a trigger or a job must keep it correct.
- **A frozen copy.** An invoice must store the price **as it was at purchase time**. That is not redundancy at all — the current product price is a different fact from the price paid. Getting this distinction right is a strong interview answer.

TRAP

“Always normalise to BCNF.” Wrong as a blanket rule, and interviewers probe it. The real answer: “Normalise to 3NF or BCNF by default, because correctness is the expensive thing to fix later. Denormalise only from a measured read problem, and only where you can name who keeps the duplicate in sync.”

6.15 Turning an ER design into tables



A 1:N relationship needs no third table — the foreign key goes on the 'many' side.

ER construct	Becomes
Strong entity	One table. Its key attribute becomes the primary key.
Multivalued attribute	A separate table: the owner’s key + the value, keyed on both. (This is the 1NF rule.)
Composite attribute	Its leaf parts become columns: <code>address</code> becomes <code>street</code> , <code>city</code> , <code>pin</code> .
Derived attribute	Usually not stored — computed on read, e.g. <code>age</code> from <code>dob</code> .
1 : 1	Put the foreign key on the side that must exist, and mark it UNIQUE . Or merge both entities into one table if they always appear together.
1 : N	Foreign key on the N side. No third table.

M : N	A third table holding both foreign keys. Its primary key is the pair. Relationship attributes (like hours) live here.
Weak entity	A table whose primary key is (owner's key + its own partial key), with ON DELETE CASCADE to the owner.
Total participation	NOT NULL on the foreign key.
Ternary relationship	One table with three foreign keys; the primary key depends on the cardinality rules.

Example 15 [LINE MAN · pattern]

Map this design to tables: a RESTAURANT has many MENU_ITEMS (an item belongs to exactly one restaurant and is identified only by its name within that restaurant); a CUSTOMER places many ORDERS; an ORDER contains many MENU_ITEMS with a quantity.

SOLUTION

Take the relationships one at a time.

RESTAURANT — MENU_ITEM is 1 : N, and MENU_ITEM is a weak entity (its name is only unique inside its restaurant).

```
CREATE TABLE restaurant (
  rest_id  INTEGER PRIMARY KEY,
  rest_name TEXT NOT NULL,
  city     TEXT
);
CREATE TABLE menu_item (
  rest_id  INTEGER NOT NULL REFERENCES restaurant(rest_id) ON DELETE CASCADE,
  item_name TEXT NOT NULL,
  price    INTEGER NOT NULL CHECK (price > 0),
  PRIMARY KEY (rest_id, item_name)          -- weak entity: owner key + partial key
);
```

CUSTOMER — ORDER is 1 : N, so the foreign key goes on order.

```
CREATE TABLE customer (
  cust_id  INTEGER PRIMARY KEY,
  cust_name TEXT NOT NULL,
  phone    TEXT UNIQUE
);
CREATE TABLE cust_order (
  order_id  INTEGER PRIMARY KEY,
  cust_id   INTEGER NOT NULL REFERENCES customer(cust_id),  -- total participation
  placed_at TEXT NOT NULL,
  total_paise INTEGER
);
```

ORDER — MENU_ITEM is M : N with an attribute (quantity), so it needs its own table.

```
CREATE TABLE order_line (
  order_id  INTEGER NOT NULL REFERENCES cust_order(order_id) ON DELETE CASCADE,
  rest_id   INTEGER NOT NULL,
  item_name TEXT NOT NULL,
  quantity  INTEGER NOT NULL CHECK (quantity > 0),
  unit_paise INTEGER NOT NULL,          -- frozen price, see below
  PRIMARY KEY (order_id, rest_id, item_name),
  FOREIGN KEY (rest_id, item_name) REFERENCES menu_item(rest_id, item_name)
);
```

Answer: 5 tables: restaurant, menu_item, customer, cust_order, order_line

Three points to volunteer, because they are what separate a good answer from a passing one:

1. The M : N table's primary key is the **combination** of the two foreign keys. Adding a surrogate `line_id` is allowed, but you then still need a **UNIQUE** constraint on the pair, or you will get duplicate lines.
2. `unit_paise` is stored on the line even though `menu_item.price` exists. That is **not** a normalisation error — the price paid is a different fact from today's price, and it must not change when the restaurant raises its rates.
3. Money is stored as an integer count of paise, never as a float. `0.1 + 0.2` is not `0.3` in binary floating point — in node, `0.1 + 0.2` prints `0.30000000000000004`.

6.16 Schema-design questions they actually ask

Normalisation theory is only half of the design round. The other half is a set of judgement questions with no single right answer — the interviewer wants your **reasoning**, and a rehearsed trade-off sentence.

Surrogate key or natural key?

Surrogate (<code>order_id</code> INTEGER)	Natural (<code>email</code> , <code>isbn</code> , <code>gst_no</code>)
Never changes, so foreign keys never need updating.	Can change — a person changes their email and every child row must follow.
Narrow and uniform, so indexes and joins are cheap.	Often wide text; every foreign key copy pays for that width.
Carries no meaning, so it leaks nothing if exposed.	Leaks business data in URLs and logs.
Needs an extra UNIQUE constraint on the real business key, or duplicates creep in.	Uniqueness is enforced for free.
Two rows can be identical except for the id — a silent duplicate.	Impossible to duplicate.

The answer that scores: “**Surrogate primary key, plus a UNIQUE constraint on the natural key.**” You get stable foreign keys and you still cannot insert the same customer twice.

TRAP

“Auto-increment is always the right primary key.” Two counter-examples to keep ready:

1. A **join table** for an M : N relationship. Its natural key is already the pair of foreign keys, and that pair must be **UNIQUE** anyway. An extra `id` adds a column and an index for no benefit.
2. A **distributed system**. Sequential ids need a central counter. A **UUID** or a time-ordered id (**ULID** / **Snowflake** style) can be generated anywhere — at the cost of a wider key and, for random UUIDs, a badly scattered index. Say “time-ordered id” and you sound like someone who has hit this.

Should this column allow NULL?

NULL means “unknown or not applicable”. If a column is **NULL** for a **different reason** each time, that is a design smell.

Example 16 [Razer · pattern]

A user table has `phone TEXT NULL`. Product says some users sign up with email only. Later, product says a phone can also be “pending verification”. The team starts storing **NULL** for both cases. What is wrong, and what would you do?

SOLUTION

What is wrong: one column is now encoding two different facts — “no phone given” and “phone given but not verified”. You cannot tell them apart, so you cannot count either, and any query that says `WHERE phone IS NULL` answers the wrong question.

Fix: make the second fact its own column.

```
ALTER TABLE users ADD COLUMN phone_verified_at TEXT;  -- NULL = not verified yet
-- now: phone IS NULL                                -> no phone on file
--         phone IS NOT NULL AND phone_verified_at IS NULL -> given, not verified
--         phone_verified_at IS NOT NULL                -> verified
```


Answer: one column must carry one fact; split the second fact into its own column

The general rule to state: “If I need a comment to explain what NULL means in a column, the column is holding more than one fact.”

Hard delete or soft delete?

A **soft delete** marks a row (`deleted_at` `TIMESTAMP`) instead of removing it.

Why teams do it	Undo, audit trails, and foreign keys that would otherwise break.
What it costs	Every single query must remember <code>WHERE deleted_at IS NULL</code> . Forget once and deleted data reappears in a report.
The UNIQUE trap	<code>UNIQUE(email)</code> now blocks a new signup with an email that belongs to a deleted user. Fix with a partial index: <code>CREATE UNIQUE INDEX ... ON users(email) WHERE deleted_at IS NULL;</code>
The honest answer	Soft-delete only where the business genuinely needs recovery or audit; otherwise delete, and keep history in a separate archive table.

The four columns almost every table wants

<code>id</code>	Surrogate primary key.
<code>created_at</code>	Set once, never updated. Makes every table time-sliceable for free.
<code>updated_at</code>	Touched on every write, usually by a trigger so the application cannot forget.
A version or status	Either an optimistic-locking counter, or a small enumerated state column — never a free-text string.

Two more one-line opinions worth having ready:

- **Money** is an integer count of the smallest unit (paise, cents) or a `DECIMAL`, never a float. In node, `0.1 + 0.2` prints `0.30000000000000004`.
- **Timestamps** are stored in UTC and rendered in the user’s zone. Storing local time without a zone is unrecoverable once daylight saving moves.

Star schema — normalisation on purpose broken

A reporting warehouse deliberately stops short of 3NF:

Fact table	Long and narrow. One row per event (<code>sale_id</code> , <code>date_key</code> , <code>product_key</code> , <code>store_key</code> , <code>qty</code> , <code>amount</code>). Billions of rows, mostly foreign keys and numbers.
Dimension tables	Short and wide, and denormalised on purpose : <code>product(product_key, name, category, sub_category, brand, supplier_name, supplier_city).category → supplier_city</code> would be a 3NF violation in an operational schema.
Why it is fine	Dimensions are loaded in controlled batches, not edited by users, so update anomalies cannot happen. Flattening removes a join from every single report.
Snowflake schema	The same design with the dimensions normalised back out. Fewer anomalies, more joins. Most teams choose star.

The sentence: “**In OLTP I normalise for write correctness; in OLAP I denormalise for read speed, and I can do that safely because the write path is a controlled batch load.**”

6.18 The whole chapter, as a program you can run

Everything in this chapter — every closure, every key list, every verdict — came out of this file. Run it yourself and change the inputs.

`nf.js` — run with: `node nf.js`

JS

```

// Attributes are single capital letters. An FD is ['AB', 'C'] meaning AB -> C.

const set = s => new Set([...s]);
const show = s => [...s].sort().join('');

// X+ : keep applying every FD whose left side is already inside the set.
function closure(X, fds) {
  const res = set(X);
  let grew = true;
  while (grew) {
    grew = false;
    for (const [lhs, rhs] of fds) {
      const fires = [...lhs].every(a => res.has(a));
      const adds = [...rhs].some(a => !res.has(a));
      if (fires && adds) { for (const a of rhs) res.add(a); grew = true; }
    }
  }
  return res;
}

const isSuperkey = (X, R, fds) => show(closure(X, fds)) === show(set(R));

// every subset of R, shortest first, so the first super key found is minimal
function subsets(arr) {
  const out = [];
  for (let m = 1; m < (1 << arr.length); m++)
    out.push(arr.filter((_, i) => m & (1 << i)));
  return out.sort((a, b) => a.length - b.length);
}

function candidateKeys(R, fds) {
  const keys = [];
  for (const s of subsets([...set(R)])) {
    if (!isSuperkey(s, R, fds)) continue;
    if (keys.some(k => [...k].every(a => s.includes(a)))) continue; // not minimal
    keys.push(show(s));
  }
  return keys;
}

function verdict(R, fds) {
  const keys = candidateKeys(R, fds);
  const prime = new Set(keys.flatMap(k => [...k]));
  let worst = 'BCNF';
  for (const [lhs, rhs] of fds) {
    const extra = [...rhs].filter(a => !set(lhs).has(a)); // ignore the trivial
    part
    if (!extra.length || isSuperkey(lhs, R, fds)) continue;
    if (extra.every(a => prime.has(a))) { if (worst === 'BCNF') worst = '3NF';
    continue; }
    const partial = keys.some(k => k !== show(set(lhs)) && [...lhs].every(a =>
    k.includes(a)));
    worst = partial ? '1NF' : (worst === '1NF' ? '1NF' : '2NF');
  }
  return { keys, prime: show(prime), highestNormalForm: worst };
}

```

```
const cases = [
  ['course registration', 'RCNHTIG', [['R','NH'], ['C','TI'], ['RC','G']]],
  ['employee-department', 'ENDMH', [['E','ND'], ['D','MH']]],
  ['tutoring', 'SJT', [['SJ','T'], ['T','J']]],
  ['abstract R', 'ABCDEF', [['A','BC'], ['CD','E'], ['B','D'], ['E','A']]],
];
for (const [name, R, fds] of cases) {
  const v = verdict(R, fds);
  console.log(name.padEnd(22), 'keys =', v.keys.join('|').padEnd(14),
    'prime =', v.prime.padEnd(7), 'highest NF =', v.highestNormalForm);
}
```

Actual output from node nf.js:

terminal		TEXT	
course registration	keys = CR	prime = CR	highest NF = 1NF
employee-department	keys = E	prime = E	highest NF = 2NF
tutoring	keys = JS ST	prime = JST	highest NF = 3NF
abstract R	keys = AF EF BCF CDF	prime = ABCDEF	highest NF = 3NF

Read the four lines against the chapter:

- **course registration** (Example 5) — key {course_id, roll}, 2NF fails on the partial dependencies. Highest form 1NF. ✓
- **employee-department** (Example 6) — key {emp_id}, 3NF fails on the transitive dependency. Highest form 2NF. ✓
- **tutoring** (Example 7) — two keys, every attribute prime, BCNF fails on tutor → subject. Highest form 3NF. ✓
- **abstract R** (Examples 4 and 8) — four keys, every attribute prime, BCNF fails everywhere. Highest form 3NF. ✓

Time $O(2^n \cdot |F| \cdot n)$ for candidate keys **Space** $O(n)$ per closure

The subset scan is exponential in the number of attributes, which is fine for the 5-8 attributes an interview uses. A single closure is cheap: each pass is $O(|F| \cdot n)$ and at most n passes can add anything.

TRAP

JavaScript detail that matters here: `new Set([... 'ABC'])` spreads the **string** into characters, giving `{ 'A', 'B', 'C' }`. That is why every attribute in this tool is a single letter. For multi-letter attribute names you must pass arrays instead — `[ 'roll' ]`, `[ 'hostel' ]` — or the spread will split `roll` into `r, o, l, l`.

6.19 Practice

PRACTICE · WARM-UP target: 6 min

1. Is $AB \rightarrow A$ trivial or non-trivial?
2. Given $X \rightarrow Y$, does $Y \rightarrow X$ follow?
3. In $R(A,B,C)$ with the single FD $A \rightarrow B$, what is C^+ ?
4. A relation's only candidate key is $\{A\}$. Can it fail 2NF?
5. Name the normal form that removes repeating groups.
6. Which attributes must be in **every** candidate key?

ANSWER KEY

1. Trivial — the right side is inside the left side. — 2. No. FDs are one-directional.
 $\text{pin} \rightarrow \text{city}$ but a city has many PIN codes. — 3. $C^+ = \{C\}$. C appears on no left side of any FD, so nothing can fire. — 4. No. A partial dependency needs a **proper subset** of a composite key; a single-attribute key has none, so 2NF is automatic. — 5. 1NF. —
6. Every attribute that never appears on the right-hand side of any FD.

PRACTICE · TIER 1 target: 15 min

1. $R(A,B,C,D)$ with $AB \rightarrow C, C \rightarrow D, D \rightarrow A$. Compute $(AB)^+, C^+$ and D^+ .
2. For the same relation, list all candidate keys.
3. For the same relation, name the highest normal form and justify each FD.
4. $\text{SALES}(\text{invoice_no}, \text{line_no}, \text{product_id}, \text{product_name}, \text{qty})$ with $\{\text{invoice_no}, \text{line_no}\} \rightarrow \text{product_id}, \text{qty}$ and $\text{product_id} \rightarrow \text{product_name}$. Find the key, the highest normal form and the fix.
5. Prove from Armstrong's axioms that $X \rightarrow Y$ and $X \rightarrow Z$ give $X \rightarrow YZ$.

ANSWER KEY

1. $(AB)^+$: $AB \rightarrow C$ gives C ; $C \rightarrow D$ gives $D \Rightarrow ABCD$, all of it.
 C^+ : $C \rightarrow D$ gives D ; $D \rightarrow A$ gives A ; now AB ? only A , not B , so $AB \rightarrow C$ cannot fire $\Rightarrow ACD$. D^+ : $D \rightarrow A$ gives $A \Rightarrow AD$.
2. B is never on a right side $\Rightarrow B$ is in every key. $B^+ = B$. Try pairs:
 $(AB)^+ = ABCD \checkmark$ key. $(BC)^+$: $C \rightarrow D, D \rightarrow A$ gives A , then $AB \rightarrow C \Rightarrow ABCD \checkmark$ key. $(BD)^+$: $D \rightarrow A$, then $AB \rightarrow C, C \rightarrow D \Rightarrow ABCD \checkmark$ key. Candidate keys: AB, BC, BD .
3. Prime = $\{A, B, C, D\}$ — every attribute. So every 3NF test passes on the “right side is prime” clause. **3NF holds**. BCNF: $C \rightarrow D$ has $C^+ = ACD$, not a super key $\Rightarrow$ **BCNF fails**. Highest normal form is **3NF**.
4. Key $\{\text{invoice_no}, \text{line_no}\}$ (neither appears on a right side, and their closure is everything). $\text{product_id} \rightarrow \text{product_name}$ has a non-super-key left side and a non-prime right side; product_id is **not** a subset of the key, so it is a **transitive** dependency $\Rightarrow$ 2NF holds, **3NF fails**. Highest form 2NF. Fix: split into $\text{SALES_LINE}(\text{invoice_no}, \text{line_no}, \text{product_id}, \text{qty})$ and $\text{PRODUCT}(\text{product_id}, \text{product_name})$. Lossless (shared product_id is the key of PRODUCT) and dependency preserving.
5. (i) $X \rightarrow Y$ given. (ii) $XX \rightarrow XY$ by augmentation with X ; and $XX = X$, so $X \rightarrow XY$. (iii) $X \rightarrow Z$ given. (iv) $XY \rightarrow YZ$ by augmenting (iii) with Y . (v) $X \rightarrow YZ$ by transitivity of (ii) and (iv). ■

PRACTICE · TIER 2 target: 22 min

1. $R(A, B, C, D, E)$ with $A \rightarrow B$, $BC \rightarrow D$, $D \rightarrow E$, $E \rightarrow A$. Find all candidate keys.
2. For the same relation, find a minimal cover.
3. $BOOKING(hotel_id, room_no, guest_id, checkin, hotel_name, room_type)$ with $hotel_id \rightarrow hotel_name$, $\{hotel_id, room_no\} \rightarrow room_type$, and $\{hotel_id, room_no, checkin\} \rightarrow guest_id$. Give the key, the highest normal form, and a 3NF decomposition. Check it is lossless.
4. A decomposition of $R(A, B, C)$ into (A, B) and (B, C) is given, with the single FD $B \rightarrow C$. Is it lossless? Is it dependency preserving?
5. Explain in three lines why a table in 3NF with only one candidate key is automatically in BCNF when that key is a single attribute and every FD's left side is that key.

ANSWER KEY

1. C is never on a right side $\Rightarrow$ in every key. $C^+ = C$. Try AC : $A \rightarrow B$ gives B , $BC \rightarrow D$ gives D , $D \rightarrow E$ gives $E \Rightarrow ABCDE$ ✓. BC : $BC \rightarrow D$, $D \rightarrow E$, $E \rightarrow A \Rightarrow ABCDE$ ✓. CD : $D \rightarrow E$, $E \rightarrow A$, $A \rightarrow B \Rightarrow ABCDE$ ✓. CE : $E \rightarrow A$, $A \rightarrow B$, $BC \rightarrow D \Rightarrow ABCDE$ ✓. Candidate keys: AC , BC , CD , CE .
2. Split: already single-attribute right sides. Trim left sides: $BC \rightarrow D$ — drop B ? $C^+ = C$, no D . Drop C ? $B^+ = B$, no D . Neither is extraneous. Drop whole FDs: remove $A \rightarrow B$ and $A^+ = A$ — needed. Remove $BC \rightarrow D$ and $(BC)^+ = BC$ — needed. Remove $D \rightarrow E$ and $D^+ = D$ — needed. Remove $E \rightarrow A$ and $E^+ = E$ — needed. The set was already minimal: $F_c = \{A \rightarrow B, BC \rightarrow D, D \rightarrow E, E \rightarrow A\}$.
3. $checkin$ and $room_no$ and $hotel_id$ — none is on a right side except through the listed FDs; the key is $\{hotel_id, room_no, checkin\}$ (its closure gives $guest_id$, then $hotel_name$ and $room_type$). Prime = those three. Now: $hotel_id \rightarrow hotel_name$ — $hotel_id$ is a **proper subset** of the key and $hotel_name$ is non-prime $\Rightarrow$ **partial dependency**. So is $\{hotel_id, room_no\} \rightarrow room_type$, for the same reason. 2NF fails on both. Highest form **1NF**. Decompose: $HOTEL(hotel_id, hotel_name)$, $ROOM(hotel_id, room_no, room_type)$, $STAY(hotel_id, room_no, checkin, guest_id)$. Lossless: $STAY \cap ROOM = \{hotel_id, room_no\}$, which is the key of $ROOM$; then that result $\cap HOTEL = \{hotel_id\}$, the key of $HOTEL$. ✓ Dependency preserving: each FD sits wholly inside one fragment. ✓
4. Shared attribute = $\{B\}$, and $B^+ = \{B, C\}$, which is exactly (B, C) . **Lossless.** ✓
Dependency preserving: the only FD is $B \rightarrow C$ and it lives inside (B, C) . ✓ Both properties hold.
5. If every FD's left side is the single candidate key, then every left side is a super key by definition. BCNF's only requirement is "every determinant is a super key", so it is satisfied automatically. There is nothing left that 3NF could forgive and BCNF could not — the two forms coincide.

PRACTICE • TIER 3 target: 30 min

1. Prove that if a relation has **no** composite candidate key (every candidate key is a single attribute), then 2NF holds automatically. Then give a relation with a single candidate key that is still not in 3NF.
2. Show a relation in 3NF that is not in BCNF **and** whose BCNF decomposition loses a dependency, and explain why no lossless BCNF decomposition of it can preserve that dependency.
3. You inherit a table in BCNF that still shows heavy redundancy. Name two distinct causes and how you would confirm each.
4. A senior engineer proposes storing `order.total_amount` even though it equals `SUM(order_line.qty * order_line.unit_price)`. Argue both sides, then say what you would actually do.

ANSWER KEY

1. A 2NF violation requires a non-prime attribute that depends on a **proper subset** of a candidate key. A single-attribute key $\{A\}$ has exactly one proper subset: the empty set. An FD $\emptyset \rightarrow Y$ would mean Y has the same value in every row — a degenerate case excluded by the definition. So no partial dependency can exist and 2NF holds. ■ A single-key relation that still fails 3NF: $EMP(emp_id, emp_name, dept_id, dept_name, dept_head)$ from Example 6. Key $\{emp_id\}$, so 2NF holds — but $dept_id \rightarrow dept_name$ is transitive and 3NF fails.

2. COACH(student, subject, tutor) from Example 7. Keys {student, subject} and {student, tutor}; all attributes prime, so 3NF holds; tutor → subject kills BCNF. Any BCNF decomposition must separate tutor from at least one of student/subject — because leaving all three together keeps the violation. The FD {student, subject} → tutor spans all three attributes, so once they are split across fragments no single fragment can hold student, subject and tutor together to check it. Hence the loss is structural, not a bad choice of split.

3. (i) **A multivalued dependency** — two independent lists in one table, as in section 6.13. Confirm it by checking whether the row count is the **product** of two independent sets (2 skills × 3 languages = 6 rows). Fix: 4NF split. (ii) **Repeated values that are genuinely one fact per row** — e.g. a city column on a million-row table. That is not a normalisation failure at all; it is just a low-cardinality column. Confirm it by testing whether any FD is violated. If none is, the fix is storage (a lookup table, dictionary encoding), not normalisation.

4. **For:** the aggregate is read on every order page and on every dashboard; recomputing a SUM per read costs a scan of the line items; the total is also the number the customer was actually charged. **Against:** it duplicates derivable data, so it can drift; every write path (add a line, change a quantity, apply a discount, issue a refund) must remember to update it, and one missed path corrupts it silently. **What I would do:** store it, but treat it as a **frozen** fact rather than a cache — the amount charged at checkout, written once when the order is placed, and never recomputed. That removes the drift risk entirely, because after checkout the line items are immutable too. If the lines **can** change, I would keep the total in a generated/computed column or maintain it in a trigger, and add a nightly reconciliation job that reports mismatches.

6.20 Rapid fire — 30 one-line answers

Question	Answer
What is a functional dependency?	$X \rightarrow Y$: any two rows agreeing on X must agree on Y .
Trivial FD?	One where the right side is a subset of the left. Always true.
Armstrong's three axioms?	Reflexivity, augmentation, transitivity.
Can you split the left side of an FD?	No. $AB \rightarrow C$ does not give $A \rightarrow C$. Only right sides split.
What is X^+ ?	The set of all attributes X determines. Computed by firing FDs until nothing changes.
How do you test if X is a super key?	X^+ equals all attributes.
How do you test if $X \rightarrow Y$ holds?	Check $Y \subseteq X^+$.
Which attributes are in every candidate key?	Those that never appear on any right side.
Prime attribute?	One that appears in at least one candidate key.
1NF?	Every cell atomic; no repeating groups, no lists.
2NF?	1NF + no non-prime attribute depends on part of a candidate key.
When is 2NF automatic?	When every candidate key is a single attribute.
3NF?	2NF + for every $X \rightarrow A$: X is a super key or A is prime.
BCNF?	Every determinant is a super key. No prime-attribute exception.
3NF vs BCNF in one line?	BCNF drops 3NF's "right side is prime" escape clause.
When is 3NF automatically satisfied?	When every attribute is prime.
Partial dependency?	A non-prime attribute determined by part of a composite key.
Transitive dependency?	key $\rightarrow$ non-key $\rightarrow$ another non-key.
Lossless join test for a 2-way split?	Closure of the shared attributes must cover at least one fragment.
What does a lossy join actually produce?	Extra spurious rows, not missing ones.
Dependency preserving?	Every original FD can be checked inside one fragment, no join needed.
Which guarantees does 3NF give?	Lossless and dependency preserving, always.
Which guarantee does BCNF lose?	Dependency preservation. Losslessness is still guaranteed.
Minimal cover — the three steps?	Split right sides, trim left sides, drop redundant FDs.
Is a minimal cover unique?	No. Different removal orders give different valid covers.
3NF synthesis, in one line?	Take the minimal cover, one relation per distinct left side, add a candidate key if none is covered.
BCNF decomposition, in one line?	For a violating $X \rightarrow Y$, replace R with X^+ and $X \cup (R - X^+)$; repeat.
MVD and 4NF?	$X \Rightarrow Y$ = two independent lists in one table; 4NF splits them into separate tables.
Why denormalise?	Fewer joins on a read-heavy workload — paid for with update anomalies you must manage yourself.
How is an M : N relationship mapped?	A third table holding both foreign keys, keyed on the pair; relationship attributes live there.

ONE-PAGE REVISION CARD

FD. $X \rightarrow Y$ = same X forces same Y . Comes from business rules, never from sample rows. Right sides split; left sides never do.

Armstrong. Reflexivity, augmentation, transitivity. Derived: union, decomposition, pseudo-transitivity.

Closure. Start with X ; fire any FD whose left side is inside; repeat until stable. Super key $\Leftrightarrow X^+ = \text{everything}$. $X \rightarrow Y \Leftrightarrow Y \subseteq X^+$.

Candidate keys. Attributes never on a right side are in **every** key — that is the seed. Grow the seed one attribute at a time, smallest first, never testing a superset of a key you already found.

The ladder. 1NF atomic $\rightarrow$ 2NF no partial $\rightarrow$ 3NF no transitive $\rightarrow$ BCNF every determinant is a super key $\rightarrow$ 4NF no stray MVD.

Triage a relation in 60 seconds. Find keys. Mark prime. For each FD: left side a super key? fine. Else right side all prime? 3NF lives, BCNF dies. Else left side a proper subset of a key? 2NF dies. Else 3NF dies. Take the worst verdict.

Shortcuts. Every attribute prime $\Rightarrow$ automatically 3NF. Single-attribute key $\Rightarrow$ automatically 2NF.

Decomposition. Lossless: closure of the shared attributes covers one fragment. Lossy joins **invent** rows. 3NF synthesis: minimal cover, group by left side, add a key if missing — lossless AND preserving. BCNF: split on X^+ and $X \cup (R - X^+)$ — lossless but may lose an FD.

Minimal cover. Split right sides $\rightarrow$ trim left sides $\rightarrow$ drop redundant FDs. Not unique.

ER mapping. 1:N $\Rightarrow$ FK on the many side. M:N $\Rightarrow$ third table keyed on the pair. Multivalued attribute $\Rightarrow$ its own table. Weak entity $\Rightarrow$ owner key + partial key, with ON DELETE CASCADE.

Denormalise only from a measured read problem, and name who keeps the copy in sync. A frozen historical value (price at purchase) is not redundancy — it is a different fact.

DBMS: Transactions, Concurrency & Recovery

What the database promises when two people press the button at the same moment

The chapter in one page

A transaction is a group of statements that must happen **all** or **none**. That is the whole idea. Everything else in this chapter is the database keeping that promise while hundreds of other transactions run at the same time, and while the power may cut at any instant.

Three questions get asked again and again in the technical round:

1. **What is ACID?** — say the four words, then give one line of what each one buys you.
2. **What are the isolation levels and which anomaly does each stop?** — the four-by-three table.
3. **How does the database survive a crash?** — write-ahead log, redo the winners, undo the losers.

If you can draw the isolation table from memory and explain write-ahead logging in five sentences, you have cleared most of what a service company asks on this topic.

WHAT YOU NEED TO KNOW

ACID

A — Atomicity	All statements commit, or none do. Delivered by the undo log.
C — Consistency	The database moves from one legal state to another legal state. Constraints, keys and triggers are not violated at commit time.
I — Isolation	Concurrent transactions do not see each other's half-finished work. Delivered by locks or MVCC .
D — Durability	Once COMMIT returns, the change survives a power cut. Delivered by forcing the log to disk , not the data file.

Transaction states: Active → Partially Committed → Committed. A failure at any point sends it to Failed → Aborted (rolled back).

A **schedule** is an interleaving of the operations of several transactions. Two operations **conflict** when they are from different transactions, touch the same data item, and at least one of them is a write. So: R-R never conflicts. R-W, W-R, W-W do.

Serial schedule — transactions run one after another, no overlap. Always correct, always slow.

Conflict serialisable — the schedule can be turned into some serial schedule by swapping non-conflicting neighbouring operations. Test: build the **precedence graph** (a node per transaction; an edge $T_i \rightarrow T_j$ when an operation of T_i conflicts with a **later** operation of T_j). The schedule is conflict serialisable exactly when the graph has **no cycle**.

The four read anomalies

Dirty read	T_1 reads a row that T_2 wrote but has not committed. T_2 then aborts, so T_1 read a value that never existed.
Non-repeatable read	T_1 reads a row twice and gets two different values, because T_2 committed an UPDATE in between.
Phantom read	T_1 runs the same WHERE twice and the second run returns extra rows , because T_2 committed an INSERT that matches.

Lost update	T_1 and T_2 both read a value, both compute from it, both write. One of the two updates silently disappears.
--------------------	--

Isolation levels (the SQL standard)

Level	Dirty read	Non-repeatable read	Phantom
READ UNCOMMITTED	possible	possible	possible
READ COMMITTED	no	possible	possible
REPEATABLE READ	no	no	possible
SERIALIZABLE	no	no	no

Two-phase locking (2PL) — every transaction has a **growing** phase (take locks, release none) then a **shrinking** phase (release locks, take none). 2PL guarantees conflict serialisability. It does **not** prevent deadlock.

Strict 2PL — all locks are held until COMMIT or ROLLBACK. This is what real databases do. It also prevents cascading rollback.

Lock compatibility

	S held	X held
S wanted	yes	no
X wanted	no	no

Write-ahead logging (WAL), the two rules

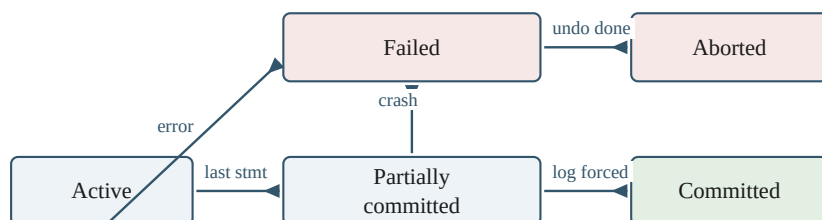
1. **Undo rule:** the log record for a change must reach disk **before** the changed data page does.
2. **Redo rule:** all log records of a transaction must reach disk **before** its COMMIT returns.

Recovery in three passes

1. **Analysis** — read the log from the last checkpoint, decide who committed (**winners**) and who did not (**losers**).
2. **Redo** — replay every logged change forward, winners and losers alike. This brings the disk back to the exact state at the moment of the crash.
3. **Undo** — walk backwards and reverse every change made by a loser.

Deadlock in a DBMS — a cycle in the **wait-for graph**. The database detects the cycle, picks a **victim**, and rolls it back. The application must retry.

The picture of a transaction's life



Transaction states. There is no arrow out of Committed — that is what durability means.

TRAP

“Partially committed” means the last statement has run but the log has **not** been forced to disk yet. A crash here still loses the transaction. COMMIT returns only after the log write returns. That single fsync is the whole of durability.

Warm-up — the mechanics

WARM-UP

build the reflex

Example 1

A bank transfer moves \$ 5000 from A100 to B200. Write it as a transaction and say which ACID letter each part relies on.

SOLUTION

The canonical transaction

SQL

```
BEGIN;
  UPDATE accounts SET balance = balance - 5000 WHERE acc_no = 'A100';
  UPDATE accounts SET balance = balance + 5000 WHERE acc_no = 'B200';
COMMIT;
```

1. **Atomicity** — if the second UPDATE fails, the first is undone. Money never vanishes.
2. **Consistency** — the sum of all balances is the same before and after. A CHECK (balance >= 0) constraint is enforced at commit.
3. **Isolation** — a report running at the same time does not see the money in neither account or in both accounts.
4. **Durability** — after COMMIT returns, a power cut cannot undo the transfer.

Answer: atomicity for the pair, durability for the COMMIT

Example 2

Two ATM sessions read the same balance of 1000, then one withdraws 100 and the other withdraws 200. What is the final balance if nothing stops them, and what is the name of this bug?

SOLUTION

lost-update, run with node

JS

```
let balance = 1000;
const readBalance = () => balance;
const writeBalance = (v) => { balance = v; };

const t1 = readBalance(); // T1 reads 1000
const t2 = readBalance(); // T2 reads 1000
writeBalance(t1 - 100);    // T1 writes 900
writeBalance(t2 - 200);    // T2 writes 800

console.log("final =", balance, " expected = 700");
// output: final = 800 expected = 700
```

T_1 's withdrawal of 100 is gone. The bank lost 100 rupees.

Answer: final balance 800; the bug is a lost update

The fix is one line, and it is the answer the interviewer wants:

let the database do the arithmetic

SQL

```
UPDATE accounts SET balance = balance - 100 WHERE acc_no = 'A100';
```

This takes an exclusive lock on the row and reads-and-writes inside the lock, so the two withdrawals queue up instead of overwriting each other.

Example 3

List the four anomalies in order of how badly they break things, and give the smallest isolation level that stops each one.

SOLUTION

Anomaly	One-line meaning	Stopped from
Dirty read	you read data that was never committed	READ COMMITTED
Lost update	two blind read-modify-writes, one disappears	REPEATABLE READ
Non-repeatable read	same row, same transaction, two different values	REPEATABLE READ
Phantom read	same WHERE, second run has extra rows	SERIALIZABLE

Answer: dirty → lost update → non-repeatable → phantom

Example 4

Which pairs of operations conflict? R1(A) R2(A), R1(A) W2(A), W1(A) W2(A), W1(A) R1(A), W1(A) W2(B).

SOLUTION

The rule has three parts: **different transactions, same item, at least one write.**

Pair	Different txn?	Same item?	A write?	Conflict
R1(A) R2(A)	yes	yes	no	no
R1(A) W2(A)	yes	yes	yes	yes
W1(A) W2(A)	yes	yes	yes	yes
W1(A) R1(A)	no	yes	yes	no
W1(A) W2(B)	yes	no	yes	no

Answer: only R1(A) W2(A) and W1(A) W2(A)

Example 5

Say in one sentence each: what is a shared lock, an exclusive lock, and why is S compatible with S but not with X?

SOLUTION

- **Shared (S)** — “I am reading this; others may read it too, nobody may change it.”
- **Exclusive (X)** — “I am changing this; nobody else may read it or change it.”
- Two readers cannot upset each other, because neither changes anything. A writer must be alone, because a reader who saw the row half-changed would read garbage.

Answer: S-S compatible; every other combination blocks

Example 6

ROLLBACK is issued after a COMMIT. What happens?

SOLUTION

Nothing. The COMMIT ended the transaction. The ROLLBACK starts a new, empty transaction and ends it. There is no way back.

Answer: no effect — a committed transaction cannot be rolled back

TRAP

Students often say “ROLLBACK undoes the last statement”. It undoes **everything since the last BEGIN or COMMIT** — which may be a hundred statements — and nothing before that.

Example 7

What does a SAVEPOINT give you that ROLLBACK does not?

SOLUTION

A partial undo. You can throw away part of a transaction and keep the rest.

keep the order, drop the bad line item

SQL

```
BEGIN;
INSERT INTO orders (order_id, cust_id, amount) VALUES (9001, 42, 1200);
SAVEPOINT after_order;
INSERT INTO order_items (order_id, sku, qty) VALUES (9001, 'SKU-7', -3);
ROLLBACK TO SAVEPOINT after_order; -- the bad item is gone
INSERT INTO order_items (order_id, sku, qty) VALUES (9001, 'SKU-7', 3);
COMMIT; -- order + good item are saved together
```

Answer: a named point inside the transaction you can rewind to, without losing the whole transaction

PRACTICE · WARM-UP target: 8 minutes

1. Name the ACID letter that the **undo** log delivers.
2. Name the ACID letter that the fsync of the log delivers.
3. Does R2(B) R1(B) conflict?
4. A transaction is in the Failed state. Which state comes next?
5. Which isolation level allows a dirty read?
6. True or false: SERIALIZABLE means the transactions actually run one at a time.
7. Two transactions both hold an S lock on row 7. T_1 now asks for X on row 7. What happens?

ANSWER KEY

1. Atomicity. 2. Durability. 3. No — both are reads.
4. Aborted. 5. READ UNCOMMITTED only. 6. False — it means the **result** is the same as some serial order; they still overlap.
7. T_1 waits for T_2 to release its S lock. If T_2 also asks for X, that is a deadlock (this exact pattern is called **lock upgrade deadlock**).

Tier 1 — the questions a service company asks

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 8 [TCS NQT · pattern]

Explain the difference between a **dirty read** and a **non-repeatable read** with a two-session example.

SOLUTION

Dirty read — you read something that was never committed.

dirty read (needs READ UNCOMMITTED)

SQL

```
-- session 2                                -- session 1
BEGIN;
UPDATE accounts SET balance = 9999
  WHERE acc_no = 'A100';
                                SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
                                BEGIN;
                                SELECT balance FROM accounts
                                WHERE acc_no = 'A100';    -- reads 9999
ROLLBACK;    -- the 9999 never existed
                                -- session 1 acted on a fake number
```

Non-repeatable read — you read something twice; both values were committed, but they differ.

non-repeatable read (happens at READ COMMITTED)

SQL

```
-- session 1                                -- session 2
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
BEGIN;
SELECT balance FROM accounts
  WHERE acc_no = 'A100';    -- 5000
                                BEGIN;
                                UPDATE accounts SET balance = 3000
                                WHERE acc_no = 'A100';
                                COMMIT;
SELECT balance FROM accounts
  WHERE acc_no = 'A100';    -- 3000    <-- different!
COMMIT;
```

Answer: dirty read = uncommitted data; non-repeatable read = committed data that changed under you

TRAP

A very common wrong answer: “dirty read means reading old data”. No. Old committed data is perfectly clean — that is exactly what a snapshot gives you. Dirty means **uncommitted**.

Example 9 [Infosys · pattern]

Is this schedule conflict serialisable? Draw the precedence graph.

R1(A) W1(A) R2(A) W2(A) R2(B) W2(B) R1(B) W1(B)

SOLUTION

Step 1 — find conflicting pairs where the **earlier** one belongs to a different transaction.

Earlier op	Later op	Edge
W1(A)	R2(A)	$T_1 \rightarrow T_2$
W1(A)	W2(A)	$T_1 \rightarrow T_2$
R1(A)	W2(A)	$T_1 \rightarrow T_2$
W2(B)	R1(B)	$T_2 \rightarrow T_1$
R2(B)	W1(B)	$T_2 \rightarrow T_1$

Step 2 — the graph has $T_1 \rightarrow T_2$ and $T_2 \rightarrow T_1$. That is a cycle of length 2.

checker, run with node

JS

```
function precedenceGraph(schedule) {
  const edges = new Set();
  for (let i = 0; i < schedule.length; i++) {
    for (let j = i + 1; j < schedule.length; j++) {
      const a = schedule[i], b = schedule[j];
      if (a.txn === b.txn) continue; // same transaction: not a conflict
      if (a.item !== b.item) continue; // different item: not a conflict
      if (a.op === "R" && b.op === "R") continue; // read-read: not a conflict
      edges.add(a.txn + "->" + b.txn);
    }
  }
  return [...edges];
}

function hasCycle(edges) {
  const g = new Map();
  for (const e of edges) {
    const [u, v] = e.split("->");
    if (!g.has(u)) g.set(u, []);
    g.get(u).push(v);
  }
  const state = new Map(); // 1 = on stack, 2 = done
  const dfs = (u) => {
    state.set(u, 1);
    for (const v of g.get(u) ?? []) {
      if (state.get(v) === 1) return true; // back edge -> cycle
      if (!state.has(v) && dfs(v)) return true;
    }
    state.set(u, 2);
    return false;
  };
  for (const u of g.keys()) if (!state.has(u) && dfs(u)) return true;
  return false;
}
```

Running it on this schedule prints S1 edges: T1->T2 , T2->T1 | cycle? true | NOT conflict serialisable, and on the serial order R1(A) W1(A) R1(B) W1(B) R2(A) W2(A) R2(B) W2(B) it prints S2 edges: T1->T2 | cycle? false | conflict serialisable.

Answer: not conflict serialisable — the precedence graph has the cycle $T_1 \rightarrow T_2 \rightarrow T_1$

SHORTCUT

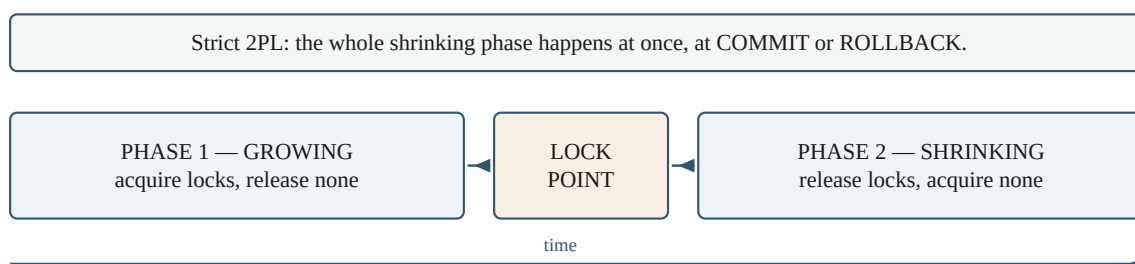
You never need to list every edge. Stop the moment you have both $T_i \rightarrow T_j$ and $T_j \rightarrow T_i$. In a two-transaction schedule that is the only cycle possible, so the test is: **does T_1 beat T_2 on one item and lose to it on another?** If yes, not serialisable.

Example 10 [Wipro · pattern]

State the two-phase locking protocol and show why it forces serialisability. Then show a 2PL schedule that still deadlocks.

| SOLUTION

The protocol. Split every transaction's lifetime at one instant called the **lock point**. Before it, the transaction may only **acquire** locks. After it, it may only **release** them.



Two-phase locking. The lock point is the moment the transaction holds the most locks.

Why it works. Order the transactions by their lock points. If T_i 's lock point comes before T_j 's, then every conflict between them must run T_i first: to conflict, T_j needs a lock that T_i holds, and T_i only lets go after its own lock point. So the lock-point order **is** an equivalent serial order, and the precedence graph cannot have a cycle.

But it deadlocks.

two sessions, opposite order, guaranteed deadlock		SQL
<pre>-- session 1 BEGIN; UPDATE seats SET held = 1 WHERE id = 1; -- now holds X on row 1 UPDATE seats SET held = 1 WHERE id = 2; -- waits for session 2</pre>	<pre>-- session 2 BEGIN; UPDATE seats SET held = 1 WHERE id = 2; -- now holds X on row 2 UPDATE seats SET held = 1 WHERE id = 1; -- waits for session 1 => DEADLOCK</pre>	

Both obeyed 2PL perfectly. 2PL is about **correctness**, not about **liveness**.

Answer: growing then shrinking; it guarantees conflict serialisability but not freedom from deadlock

TRAP

The single most common wrong answer on this topic is “2PL prevents deadlock.” It **causes** deadlocks. What prevents deadlock is a lock **ordering** rule in your application: always touch rows in increasing primary-key order. Rewriting the example above as `WHERE id IN (1,2) ORDER BY id` makes the deadlock impossible.

Example 11 [Capgemini · pattern]

What is **strict** 2PL and what extra problem does it solve?

SOLUTION

Strict 2PL holds all **exclusive** locks until the transaction commits or aborts. Rigorous 2PL holds **all** locks (shared too) until then.

The extra problem it solves is **cascading rollback**.

Without strictness: T_1 writes row A, releases the X lock, keeps running. T_2 reads A. T_3 reads what T_2 wrote. Now T_1 aborts — so T_2 read dirty data and must abort, so T_3 must abort. One failure knocks over a chain.

With strict 2PL, nobody can read T_1 's row until T_1 has committed, so there is nothing to cascade.

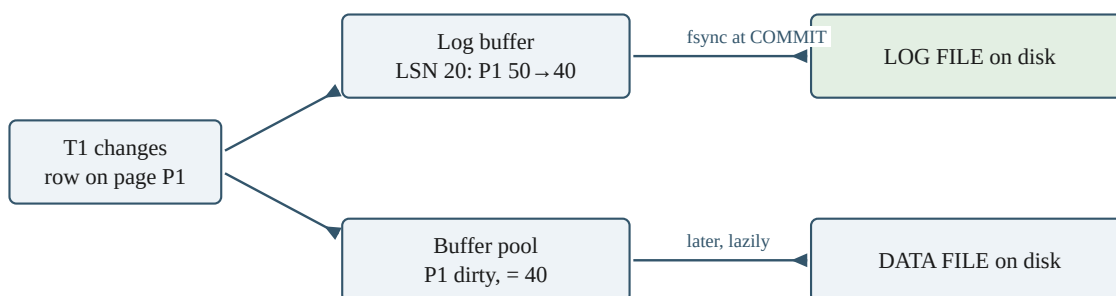
Answer: locks held until commit; it makes the schedule recoverable and cascadeless

Three words you may be asked to order, easiest to strictest: **recoverable** (a transaction commits only after every transaction it read from has committed) → **cascadeless** (you may only read committed data) → **strict** (you may neither read nor overwrite uncommitted data). Strict implies cascadeless implies recoverable.

Example 12 [Accenture · pattern]

Explain write-ahead logging. Why does COMMIT not write the data pages?

SOLUTION



WAL: the log goes to disk at COMMIT; the data page goes whenever the buffer manager feels like it.

The log is a single file written **sequentially**. The data pages are scattered all over the disk. Forcing one sequential 200-byte append is cheap; forcing twenty random 8 KB pages is not. So the database makes durability cheap by writing the **promise** (the log) and letting the **payment** (the data page) happen whenever it is convenient.

The two rules that make this safe:

1. Never write a data page to disk before its log record is on disk. (Otherwise a crash leaves a change on disk with no way to undo it.)
2. Never return from COMMIT before all of that transaction's log records are on disk. (Otherwise a crash leaves a promise you cannot keep.)

Answer: log first, sequentially, and force it at commit; data pages follow lazily

TRAP

“Durability means the row is in the table file on disk.” It is not. Right after COMMIT returns, the data file may still hold the **old** value. The new value exists only in the log and in RAM. Recovery replays the log to put it in the data file. Say **log**, not **data file**.

Example 13 [Cognizant · pattern]

The system crashes. The log since the last checkpoint is below. Which transactions are redone, which are undone, and what is on page P1 afterwards?

LSN	Type	Txn	Page	Before	After
10	BEGIN	T1			
20	UPDATE	T1	P1	50	40
30	BEGIN	T2			
40	UPDATE	T2	P2	70	90
50	COMMIT	T1			
60	BEGIN	T3			
70	UPDATE	T3	P1	40	15

At the moment of the crash the disk happened to hold P1 = 15 and P2 = 90.

SOLUTION

Analysis. Walk forward. A transaction with a COMMIT is a **winner**; anything still open at the end is a **loser**.

- T1 has COMMIT at LSN 50 → winner.

- T2 and T3 have no COMMIT → losers.

Redo. Replay **every** update forward, winners and losers alike. This is called **repeating history**. After redo: P1 = 15, P2 = 90 — the exact state at the crash.

Why redo a loser? Because redo must not have to think. It sets the disk to a known state so that undo's "before" values are guaranteed to be correct.

Undo. Walk the log **backwards**, reversing only the losers.

- LSN 70, T3 (loser): P1 ← 40.
- LSN 40, T2 (loser): P2 ← 70.
- LSN 20, T1 (winner): leave it. P1 stays 40.

the whole recovery in 15 lines, run with node

JS

```
const winners = new Set(), losers = new Set();
for (const r of log) {
  if (r.type === "BEGIN") losers.add(r.txn);
  if (r.type === "COMMIT" || r.type === "ABORT") {
    losers.delete(r.txn); winners.add(r.txn);
  }
}
const page = { P1: 15, P2: 90 }; // disk at restart

for (const r of log)
  if (r.type === "UPDATE") page[r.page] = r.after; // REDO everything

for (const r of [...log].reverse())
  if (r.type === "UPDATE" && losers.has(r.txn)) page[r.page] = r.before; // UNDO
```

It prints winners: T1 | losers: T2,T3, then after REDO {"P1":15,"P2":90}, then after UNDO {"P1":40,"P2":70}.

Answer: redo all of T1, T2, T3; undo T2 and T3; final P1 = 40, P2 = 70

Example 14 [TCS Digital · pattern]

What is a checkpoint and why does recovery get faster with one?

SOLUTION

A checkpoint is a log record that says: **at this instant, here is the list of transactions still running, and here is the list of dirty pages in the buffer pool**. Usually the database also flushes some dirty pages at the same time.

Without a checkpoint, recovery has to read the log from the very beginning of time. With one, it starts at the last checkpoint record.

Sharp checkpoint	Stop accepting new work, finish or roll back everything running, flush every dirty page, then write the record. Simple, but the database freezes for seconds.
Fuzzy checkpoint	Write the record and keep going; flush dirty pages in the background. No freeze. Recovery must start a little earlier in the log — at the oldest LSN that is still dirty. This is what real systems use.

Answer: a restart point in the log; recovery reads from the last checkpoint instead of from the start

Example 15 [Infosys · pattern]

Compare DELETE, TRUNCATE and DROP on transactions, logging and speed.

| SOLUTION

	DELETE FROM t	TRUNCATE TABLE t	DROP TABLE t
Kind	DML	DDL	DDL
What goes	rows	all rows	rows + the table itself
WHERE	yes	no	no
Logging	one log record per row	deallocates pages; almost no row logging	same
Speed on 10M rows	slow	fast	fast
Triggers fire	yes (ON DELETE)	no	no
Identity counter	keeps counting	usually reset to 1	gone with the table
Rollback	always	depends on the engine	depends on the engine

Answer: DELETE is row-by-row DML; TRUNCATE and DROP are DDL that drop storage wholesale

TRAP

The rollback row is where most students lose the mark, because the honest answer is “it depends”, and you should say so:

- **PostgreSQL** — TRUNCATE and even DROP TABLE are transactional. BEGIN; TRUNCATE t; ROLLBACK; brings the rows back.
- **MySQL (InnoDB)** and **Oracle** — DDL causes an **implicit commit**. The rows are gone. There is nothing to roll back to.

Saying “TRUNCATE can never be rolled back” is a half-truth. Say which engine you mean.

Example 16 [Wipro · pattern]

Your Node service does a read, some arithmetic, and a write. Show the wrong way and two right ways to make it safe under concurrency.

SOLUTION

The wrong way — read, compute in the application, write back.

broken: a lost update waiting to happen

SQL

```
SELECT stock FROM inventory WHERE sku = 'SKU-7';    -- app reads 5
-- ... app computes 5 - 1 = 4 ...
UPDATE inventory SET stock = 4 WHERE sku = 'SKU-7';
```

Right way 1 — pessimistic. Take the lock while reading.

SELECT ... FOR UPDATE

SQL

```
BEGIN;
  SELECT stock FROM inventory WHERE sku = 'SKU-7' FOR UPDATE;  -- X lock; others
wait
  UPDATE inventory SET stock = stock - 1 WHERE sku = 'SKU-7' AND stock >= 1;
COMMIT;
```

Right way 2 — optimistic. No lock. Carry a version number and check it on the way out.

compare-and-set on a version column

SQL

```
SELECT stock, version FROM inventory WHERE sku = 'SKU-7';    -- stock 5, version 12
```

```
UPDATE inventory
  SET stock = 4, version = 13
 WHERE sku = 'SKU-7' AND version = 12;    -- 0 rows updated => retry
```

optimistic control, run with node

JS

```
const row = { id: 7, seats: 2, version: 1 };
const read = () => ({ ...row });
const write = (snap, newSeats) => {
  if (row.version !== snap.version) return 0;
  row.seats = newSeats; row.version++; return 1;
};

const t1 = read(); // T1 reads {seats: 2, version: 1}
const t2 = read(); // T2 reads {seats: 2, version: 1}
console.log("T2 rows =", write(t2, t2.seats - 1)); // 1 -> version becomes 2
console.log("T1 rows =", write(t1, t1.seats - 1)); // 0 -> stale, rejected
const t1b = read(); // re-read
console.log("T1 retry rows =", write(t1b, t1b.seats - 1)); // 1
```

Output: T2 rows = 1, T1 rows = 0, T1 retry rows = 1, and the row ends at {"id":7,"seats":0,"version":3}. No update was lost.

	Pessimistic (FOR UPDATE)	Optimistic (version column)
Cost when there is no clash	a lock, still paid for	nothing
Cost when there is a clash	waiting	a full retry
Best for	hot rows, short transactions	rare clashes, long think time
Danger	deadlock, lock convoys	livelock if everyone retries at once

Answer: never compute the new value in the application without either a lock or a version check

PRACTICE · TIER 1 target: 20 minutes

1. Name the isolation level that permits phantoms but not non-repeatable reads.
2. Is $R1(X) \ W2(X) \ W1(X)$ conflict serialisable? Show the graph.
3. Give one reason a database prefers a sequential log write to a random data-page write.
4. A transaction reads a row written by another transaction that has not committed. Which property of the schedule has been broken?
5. Write the SQL that increments a counter safely, in one statement.
6. Which of DELETE, TRUNCATE, DROP fires an ON DELETE trigger?
7. In strict 2PL, when exactly does the shrinking phase happen?
8. A checkpoint record lists 3 active transactions. Two of them appear with COMMIT later in the log. During recovery, how many are losers?

ANSWER KEY

1. REPEATABLE READ.
2. Edges: $R1(X) \text{ before } W2(X) \text{ gives } T_1 \rightarrow T_2; W2(X) \text{ before } W1(X) \text{ gives } T_2 \rightarrow T_1$. Cycle $\rightarrow$ **not** conflict serialisable.
3. One fsync on an append-only file, no seeking; a dirty-page flush may touch dozens of scattered 8 KB pages.
4. It is not **cascadeless** (and possibly not recoverable).

5. UPDATE counters SET n = n + 1 WHERE name = 'hits';
6. DELETE only.
7. All at once, at COMMIT or ROLLBACK.
8. One — plus any transaction that started after the checkpoint and never committed.

Tier 2 — applied, business-flavoured

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 17 [Grab · pattern]

A ride-booking service assigns drivers. Two dispatchers run the same code at REPEATABLE READ:

both sessions, same code, same second

SQL

```
BEGIN;
SELECT count(*) AS free FROM drivers WHERE zone = 'BKK-3' AND state = 'FREE'; -- 2
-- application: "free >= 1, so I may take one"
UPDATE drivers SET state = 'BUSY' WHERE driver_id = ? AND state = 'FREE';
COMMIT;
```

Dispatcher A takes driver 11, dispatcher B takes driver 12. The business rule is “never drop below one free driver in a zone”. It is now broken. Why did REPEATABLE READ not stop this, and what are the three fixes?

SOLUTION

Why it happened. This is **write skew**. Both transactions read the same set of rows, both made a decision from that read, and then each wrote a **different** row. There is no read-write conflict on the same row, so snapshot-based REPEATABLE READ sees nothing wrong. Each transaction on its own is fine; the pair is not.

Lost update	two transactions write the same row
Write skew	two transactions write different rows after reading the same set

Fix 1 — raise the level to SERIALIZABLE. On PostgreSQL this is Serializable Snapshot Isolation: the engine watches the read-write dependencies and aborts one transaction with a serialisation failure. Your code must catch that error and retry.

Fix 2 — materialise the conflict. Give the **zone** a row and lock it, so the two transactions collide on something real.

lock the zone, not the driver

SQL

```
BEGIN;
  SELECT zone_id FROM zones WHERE zone_id = 'BKK-3' FOR UPDATE; -- serialises them
  SELECT count(*) FROM drivers WHERE zone = 'BKK-3' AND state = 'FREE';
  UPDATE drivers SET state = 'BUSY' WHERE driver_id = 11 AND state = 'FREE';
COMMIT;
```

Fix 3 — make the rule a single atomic statement, so there is no gap between the check and the write.

check and write in one statement

SQL

```
UPDATE drivers
  SET state = 'BUSY'
```

```
WHERE driver_id = 11
AND state = 'FREE'
AND (SELECT count(*) FROM drivers WHERE zone = 'BKK-3' AND state = 'FREE') > 1;
-- 0 rows updated => the rule would have been broken; the app sees it and stops
```

Answer: write skew; fix with SERIALIZABLE + retry, a lock on a shared row, or one atomic statement

TRAP

“REPEATABLE READ gives me a consistent snapshot, so my check is safe.” A snapshot makes your **read** consistent. It says nothing about whether the world still agrees with your read at the moment you **write**. Every check-then-act pattern needs a lock, a unique constraint, or a serialisable retry.

Example 18 [Shopee · pattern]

A flash sale has 100 units. 5000 requests arrive in two seconds. The naive code uses `SELECT ... FOR UPDATE` on the single stock row. Latency explodes. Explain why, and give two designs that survive.

SOLUTION

Why it explodes. Every request needs the X lock on **one** row. That row becomes a queue. If each transaction holds the lock for 4 ms (network round trip + commit `fsync`), the row can serve at most 250 requests per second, no matter how many CPUs you have. 5000 requests take 20 seconds. Everything behind the queue times out, and the timeouts trigger retries, which lengthen the queue. This is a **hot row**.

$$\text{max throughput} = \frac{1}{\text{lock hold time}} = \frac{1}{4 \text{ ms}} = 250 \text{ per second}$$

Design 1 — split the row into buckets. Turn one hot row into k warm rows.

sharded counter, $k = 20$ buckets of 5 units each

SQL

```
-- each request picks a random bucket, so the lock traffic spreads over 20 rows
UPDATE stock_bucket
SET remaining = remaining - 1
WHERE sku = 'FLASH-1' AND bucket = ? AND remaining >= 1;
-- 0 rows updated => that bucket is empty; try another bucket, then give up
```

Throughput becomes $20 \times 250 = 5000$ per second. The cost: a bucket can be empty while others are not, so you must sweep a few buckets before declaring “sold out”.

Design 2 — do not use the database for the counter at all. Decrement an atomic counter in Redis (`DECR` returns the new value in one round trip, no transaction, no `fsync`). Only the requests that get a value ≥ 0 go on to write a durable order row. The database now sees 100 inserts instead of 5000 lock waits.

	Bucket split	External counter
Durability of the count	full ACID	weaker; needs reconciliation
Extra system	none	Redis
Failure mode	uneven buckets	counter and orders drift apart

Answer: one row is a serial queue capped at $1/t_{\text{hold}}$; shard the row, or move the counter out

Example 19 [DBS · pattern]

A nightly settlement job runs for 40 minutes at `SERIALIZABLE` and reads three large tables. It aborts almost every night with a serialisation failure. The report it produces does not need to be up to the second. What do you change?

SOLUTION

A long read-only transaction at **SERIALIZABLE** has a huge read set. The chance that **someone** creates a conflicting dependency over 40 minutes is close to 1.

Three changes, cheapest first:

1. **Declare it read-only.** `BEGIN TRANSACTION READ ONLY;` — the engine knows the transaction will never write, so it can skip a large part of the conflict tracking. On PostgreSQL, `READ ONLY DEFERRABLE` at **SERIALIZABLE** waits for a safe snapshot and is then **guaranteed** never to abort.
2. **Drop to REPEATABLE READ.** A read-only transaction at **REPEATABLE READ** gets a single consistent snapshot of all three tables. Write skew cannot happen because it never writes. For a report this is exactly right.
3. **Run it against a read replica** so it competes with nothing. Accept the replication lag — the report already tolerates it.

the fix

SQL

```
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ READ ONLY;
SELECT ... FROM ledger ...;
SELECT ... FROM fees ...;
SELECT ... FROM fx_rate ...; -- all three see the same instant in time
COMMIT;
```

Answer: read-only + REPEATABLE READ snapshot (or a replica); SERIALIZABLE buys nothing for a transaction that never writes

Keep this sentence ready: **the cost of an isolation level is paid in aborts and waiting, and you should buy only the level your invariant actually needs.** A read-only report needs a consistent snapshot. It does not need serialisability.

Example 20 [Agoda · pattern]

A booking API writes to the bookings table and then calls a payment provider over HTTP. The developer puts the HTTP call inside the transaction so that “if payment fails, the booking rolls back”. Criticise this and give the standard alternative.

SOLUTION

What is wrong.

1. The transaction now holds locks for the whole duration of a network call — 200 ms on a good day, 30 s on a bad one. Lock hold time is throughput’s denominator (Example 18).
2. The connection pool drains. Each in-flight payment occupies one database connection doing nothing.
3. It does not even work. If the payment succeeds and the `COMMIT` then fails, you have charged a customer with no booking. Two systems cannot be made atomic by wrapping one inside the other’s transaction.

The standard alternative — the outbox pattern.

transaction 1: local, short, atomic

SQL

```
BEGIN;
INSERT INTO bookings (booking_id, user_id, room_id, state)
VALUES ('B-77', 42, 'R-9', 'PENDING_PAYMENT');
INSERT INTO outbox (id, topic, payload, sent)
VALUES ('B-77', 'charge', '{"booking":"B-77","amount":4200}', false);
COMMIT; -- both rows or neither: one database, real atomicity
```

A separate worker reads unsent outbox rows, calls the payment provider, and then:

transaction 2: also local and short

SQL

```
BEGIN;
UPDATE bookings SET state = 'CONFIRMED' WHERE booking_id = 'B-77';
```

```
UPDATE outbox SET sent = true WHERE id = 'B-77';
COMMIT;
```

The worker may crash after charging and before this update, so it will retry — which means the payment call must be **idempotent**. Send `booking_id` as the provider’s idempotency key and the second charge is a no-op.

Answer: never hold a transaction across a network call; write an outbox row atomically and let an idempotent worker do the remote call

TRAP

Someone will suggest two-phase commit (2PC) here. Name it, then say why it is rarely used: 2PC blocks. If the coordinator dies between “prepare” and “commit”, every participant sits holding locks until a human intervenes. Most teams choose an outbox plus idempotency and accept **eventual** consistency instead.

Example 21 [GIC · pattern]

Explain MVCC. Given the version list below and a transaction whose snapshot is “all transactions with $\text{id} \leq 103$ that have committed”, which rows does it see?

row id	value	xmin (created by)	xmax (deleted by)
1	A	100	105
1	B	105	—
2	C	102	—

SOLUTION

MVCC in one paragraph. An UPDATE does not overwrite. It writes a **new version** of the row and marks the old version as dead from that transaction onwards. Every row version carries `xmin` (the transaction that created it) and `xmax` (the transaction that killed it). A reader takes a **snapshot** — a rule for which transaction ids count as “already committed for me” — and then sees exactly the versions that were alive under that rule. Readers therefore never block writers, and writers never block readers.

Visibility test for a version:

1. `xmin` must be committed, and must be inside my snapshot. Otherwise I cannot see it.
2. If `xmax` is empty, or `xmax` is not committed, or `xmax` is outside my snapshot, the version is still alive for me.
3. Otherwise the version is dead for me.

visibility rule, run with node

JS

```
function visible(v, snapshotXid, running, committed) {
  if (!committed.has(v.xmin)) return false;           // creator never committed
  if (v.xmin > snapshotXid || running.has(v.xmin))
    return false;                                     // created after my snapshot
  if (v.xmax === null) return true;                    // never deleted
  if (!committed.has(v.xmax)) return true;            // deleter did not commit
  if (v.xmax > snapshotXid || running.has(v.xmax))
    return true;                                       // deleted after my snapshot
  return false;
}
```

Running it prints: snapshot `xid <= 103` -> 1:A 2:C and snapshot `xid <= 110` -> 1:B 2:C.

Walking it by hand for snapshot 103:

- (1, A): `xmin 100 ≤ 103`, committed → visible so far. `xmax 105 > 103` → the delete has not happened for me. **Visible**.
- (1, B): `xmin 105 > 103` → created after my snapshot. **Not visible**.

- (2, C): xmin 102 ≤ 103, xmax empty. **Visible.**

Answer: snapshot 103 sees 1: A and 2: C; the update to B is invisible to it

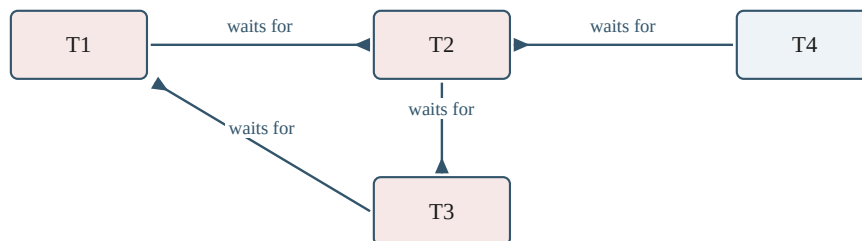
The price of MVCC is **garbage**. Dead versions pile up and must be cleaned (VACUUM in PostgreSQL, the purge thread in InnoDB). A transaction left open for hours pins the oldest snapshot, so nothing can be cleaned, and the table **bloats**. “Long idle transaction” is the number-one cause of a PostgreSQL table growing ten times bigger than its data.

Example 22 [SCB · pattern]

Your application logs show deadlock detected, process 4411 was chosen as the victim a few hundred times a day. Walk through how you would diagnose and fix it.

SOLUTION

Step 1 — draw the wait-for graph from the deadlock report. A node per transaction, an edge $T_i \rightarrow T_j$ meaning “ T_i is waiting for a lock that T_j holds”. A deadlock is a cycle.



Wait-for graph. $T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$ is a cycle, so those three are deadlocked. $T4$ is only a victim of the traffic jam.

cycle finder + victim choice, run with node

JS

```

const waitsFor = { T1: ["T2"], T2: ["T3"], T3: ["T1"], T4: ["T2"] };

function findCycle(g) {
  const state = new Map(), stack = [];
  const dfs = (u) => {
    state.set(u, 1); stack.push(u);
    for (const v of g[u] ?? []) {
      if (state.get(v) === 1) return stack.slice(stack.indexOf(v)); // the cycle
      if (!state.has(v)) { const c = dfs(v); if (c) return c; }
    }
    state.set(u, 2); stack.pop(); return null;
  };
  for (const u of Object.keys(g))
    if (!state.has(u)) { const c = dfs(u); if (c) return c; }
  return null;
}
const cycle = findCycle(waitsFor); // [ 'T1', 'T2', 'T3' ]
const work = { T1: 900, T2: 120, T3: 4000 }; // log records written so far
const victim = cycle.reduce((a, b) => (work[a] <= work[b] ? a : b)); // T2
  
```

Output: cycle: T1 -> T2 -> T3 -> T1 and victim = cheapest to roll back = T2 (120 log records).

Step 2 — find the common shape. Almost always one of these three:

Opposite lock order	Two code paths touch rows A and B in different orders.
---------------------	--

Lock upgrade	Both take an S lock (a plain SELECT under some engines), then both ask to upgrade to X. Neither can, because the other holds S.
Range/gap locks	Two inserts into the same gap of an index under REPEATABLE READ in InnoDB.

Step 3 — fix.

1. Impose a **global lock order**. Sort the keys in the application before touching them: ORDER BY id on the SELECT ... FOR UPDATE, or sort the array in Node before the loop.
2. Replace “read then upgrade” with SELECT ... FOR UPDATE on the first read, so the X lock is taken once and never upgraded.
3. Shorten the transaction. A transaction that holds locks for 2 ms rarely meets another one.
4. Keep a **retry wrapper** anyway. A deadlock is not a bug you can eliminate; it is an error code you must handle.

retry on serialisation / deadlock errors, run with node

JS

```
async function withRetry(run, tries = 4) {
  for (let i = 0; i < tries; i++) {
    try { return await run(); }
    catch (e) {
      // 40001 = serialisation failure, 40P01 = deadlock
      const retryable = e.code === "40001" || e.code === "40P01";
      if (!retryable || i === tries - 1) throw e;
      await new Promise(r => setTimeout(r, 20 * 2 ** i + Math.random() * 20));
    }
  }
}
```

Answer: cycle in the wait-for graph; enforce one lock order, shorten transactions, and always retry

PRACTICE · TIER 2 target: 25 minutes

1. Two transactions read the same 10 rows and each updates a different one. Name the anomaly and the level that stops it.
2. A row is locked for an average of 8 ms. What is the maximum number of transactions per second that row can serve?
3. Give one reason an outbox row and the business row must be inserted in the **same** transaction.
4. Under MVCC, does a SELECT ever block an UPDATE on the same row? Does an UPDATE ever block another UPDATE on the same row?
5. Why does an idle-in-transaction connection make a PostgreSQL table grow?
6. A deadlock report names 3 transactions. How many must be rolled back to break it?
7. You must not sell the same seat twice. Give the **simplest** mechanism, using no explicit locks.

ANSWER KEY

1. Write skew; **SERIALIZABLE**.
2. $1/0.008 = 125$ per second.
3. Otherwise a crash between the two writes leaves either a booking nobody will charge for, or a charge for a booking that does not exist.
4. No, and yes — readers and writers do not block each other, but two writers on the same row do.
5. Its snapshot is the oldest one alive, so no dead row version newer than it may be reclaimed; VACUUM can do nothing and the table bloats.
6. One — removing any single node breaks a cycle.
7. A **UNIQUE** constraint on (show_id, seat_no) in the bookings table. The second insert fails

with a constraint violation. The database does the locking for you.

Tier 3 — the hard version

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 23 [Google · pattern]

“SERIALIZABLE means the transactions run one at a time.” Take that apart. Then explain the difference between **conflict** serialisability, **view** serialisability, and what PostgreSQL’s SSI actually guarantees.

SOLUTION

The claim is wrong in an important way. SERIALIZABLE guarantees the **result** is equal to **some** serial order. It does not say which order, and it does not say the transactions ran one at a time. They overlap freely; the engine only has to prove afterwards that some serial order would have produced the same outcome.

Conflict serialisable. Reachable from a serial schedule by swapping adjacent non-conflicting operations. Test: acyclic precedence graph. Cheap to check — this is what engines implement.

View serialisable. A weaker, larger class. Schedule S is view equivalent to serial S' when

1. every transaction reads the same **initial** values in both,
2. every read in S reads from the same writer as in S' ,
3. the **final** write of each item is by the same transaction in both.

Every conflict serialisable schedule is view serialisable. The reverse is false. The gap is made of **blind writes** — a write with no read before it.

$$S = W_1(A) \ W_2(A) \ W_3(A)$$

The precedence graph has $T_1 \rightarrow T_2$, $T_2 \rightarrow T_3$, $T_1 \rightarrow T_3$ — acyclic, so this one **is** conflict serialisable. Now consider

$$S = R_1(A) \ W_2(A) \ W_1(A) \ W_3(A)$$

Edges: $T_1 \rightarrow T_2$ (read then write), $T_2 \rightarrow T_1$ (write then write), $T_1 \rightarrow T_3$, $T_2 \rightarrow T_3$. The cycle $T_1 \rightarrow T_2 \rightarrow T_1$ makes it **not** conflict serialisable. But it is view equivalent to T_2, T_1, T_3 : T_1 reads the initial A in both, and T_3 writes last in both. So it is view serialisable and not conflict serialisable.

Why nobody implements view serialisability: deciding it is NP-complete. Conflict serialisability is a cycle check on a graph with one node per active transaction — microseconds.

What SSI guarantees. PostgreSQL’s Serializable Snapshot Isolation runs everything on MVCC snapshots, so readers never block. It watches for the one structure that snapshot isolation cannot handle: a transaction with both an incoming and an outgoing read-write dependency — a **dangerous structure**. When it spots one it aborts a transaction with SQLSTATE 40001. So the guarantee is true serialisability, bought with **aborts** rather than with **waiting**, and it can abort transactions that would in fact have been fine (false positives).

Answer: serialisable = equivalent to some serial order, not actually serial; conflict $\subset$ view; engines use conflict because view is NP-complete; SSI achieves it by aborting, not by locking

Example 24 [Amazon · pattern]

Design the locking for a “transfer money between two accounts” API that must run at 5000 transfers per second across 10 million accounts, and prove your design cannot deadlock.

SOLUTION

Step 1 — the shape of the work. Each transfer touches exactly two rows. With 10 million accounts, two random accounts collide rarely. The danger is not volume; it is **ordering**.

Step 2 — the deadlock proof. A deadlock needs a cycle in the wait-for graph. Suppose every transaction acquires its locks in strictly increasing order of `acc_no`. Let T_i wait for T_j . Then T_i is blocked on some account a , and it already holds every account it locked before a — all of which are $< a$. T_j holds a . So the **highest lock held** by T_i is less than the highest lock held by T_j . Follow the cycle $T_1 \rightarrow T_2 \rightarrow \dots \rightarrow T_1$ and this quantity strictly increases all the way round, ending below itself. Contradiction. **No cycle can exist.**

ordered locking — the whole trick is ORDER BY

SQL

```
BEGIN;
-- lock BOTH rows in one statement, in ascending account order
SELECT acc_no, balance
  FROM accounts
 WHERE acc_no IN ('A100', 'B200')
 ORDER BY acc_no
  FOR UPDATE;

UPDATE accounts SET balance = balance - 5000
  WHERE acc_no = 'A100' AND balance >= 5000;
-- the application checks that 1 row was updated; if 0, the balance was too low
UPDATE accounts SET balance = balance + 5000 WHERE acc_no = 'B200';

INSERT INTO ledger (from_acc, to_acc, amount, at)
  VALUES ('A100', 'B200', 5000, now());
COMMIT;
```

TRAP

ORDER BY inside SELECT ... FOR UPDATE is **not** guaranteed to fix the lock acquisition order on every engine — the planner may choose to lock as it scans. The portable version is to sort in the application and lock one row at a time:

portable ordered locking, run with node

JS

```
const ids = ['B200', 'A100'].sort();          // ALWAYS sort -> ['A100', 'B200']
for (const id of ids)
  await tx.query('SELECT 1 FROM accounts WHERE acc_no = $1 FOR UPDATE', [id]);
```

Step 3 — the throughput budget. Lock hold time is everything.

$$\text{hold} = 2 \text{ index lookups} + 3 \text{ writes} + \text{commit fsync} \approx 1 \text{ ms}$$

One account row can then serve about 1000 transfers per second. 5000 transfers per second over 10 million accounts averages $5000 \times 2/10^7 = 0.001$ locks per account per second — no contention at all. The design fails only on **celebrity accounts**: a merchant settlement account receiving 3000 transfers per second exceeds one row's budget. For those, bucket the credit side (Example 18) and sum the buckets when reading the balance.

Step 4 — what else is needed.

1. Group commit, so 200 transactions share one log fsync instead of paying for 200.
2. An idempotency key on the API, because the client will retry a request whose response was lost, and a retried transfer must not move the money twice: `INSERT INTO transfers (idem_key, ...) VALUES (...)` with `UNIQUE(idem_key)`.
3. A retry wrapper anyway, for 40001 from the occasional serialisation failure.

Answer: strict 2PL with locks taken in ascending key order; the highest-lock-held argument proves no cycle; shard hot accounts and use group commit for throughput

Compare **timestamp ordering**, **optimistic concurrency control** and **2PL** on the same workload, and say when each wins. Then run the timestamp-ordering rules on a small schedule.

SOLUTION

Timestamp ordering (T/O). Every transaction gets a timestamp TS when it starts. Every item X carries $RTS(X)$ (largest timestamp that read it) and $WTS(X)$ (largest that wrote it). No locks at all — instead, an operation that would violate timestamp order is **rejected**, and the transaction is restarted with a new timestamp.

read(X) by T	if $TS(T) < WTS(X) \rightarrow$ abort T (it wants a value that has already been overwritten by a younger transaction). Else read, and set $RTS(X) = \max(RTS(X), TS(T))$.
write(X) by T	if $TS(T) < RTS(X) \rightarrow$ abort T (somebody younger already read the old value). If $TS(T) < WTS(X) \rightarrow$ Thomas's write rule : this write is obsolete, so silently ignore it and continue. Else write and set $WTS(X) = TS(T)$.

Worked run. $TS(T_1) = 10$, $TS(T_2) = 20$. Start with $RTS(A) = WTS(A) = 0$.

Step	Operation	Rule check	$RTS(A)$	$WTS(A)$
1	$R_2(A)$	$20 < WTS=0?$ no $\rightarrow$ allow	20	0
2	$W_2(A)$	$20 < RTS=20?$ no. $20 < WTS=0?$ no $\rightarrow$ write	20	20
3	$W_1(A)$	$10 < RTS=20?$ yes	—	—

Step 3 aborts T_1 under the plain rule. Under Thomas's write rule we first ask whether the write is merely obsolete: $10 < WTS=20$ is true, so T_1 's write would be overwritten by T_2 's anyway — but T_1 's timestamp is also below RTS , and the read-check fires first, so T_1 still aborts. Change step 1 to $R_1(A)$ ($RTS = 10$) and step 3 becomes: $10 < RTS=10?$ no. $10 < WTS=20?$ yes $\rightarrow$ **ignore the write, do not abort**. That is exactly what Thomas's rule buys.

The three protocols side by side.

	2PL	Timestamp ordering	OCC
When it checks	before every access	before every access	only at commit
Cost when no conflict	lock/unlock on every row	two comparisons per access	almost zero
Cost on conflict	wait	restart	restart, after all the work is done
Deadlock	possible	impossible	impossible
Starvation	rare	a long transaction can be restarted forever	same
Wins when	contention is real and transactions are short	conflicts are moderate and waiting is worse than restarting	conflicts are rare
Loses when	hot rows, long transactions	long read-write transactions	contention is high — work is repeated again and again

Reading the workload. Let p be the probability a transaction conflicts and W the work it does. OCC's expected cost is roughly $W/(1-p)$ — fine while p is small, and it blows up as $p \rightarrow 1$. 2PL's cost is $W + p \times \text{wait}$, which grows gently. So: low contention $\rightarrow$ OCC; high contention $\rightarrow$ 2PL. Almost every real system is 2PL (SQL Server, InnoDB on writes) or MVCC + 2PL on writers (PostgreSQL, Oracle), with OCC available as a version column for the application to use where it knows conflicts are rare.

Answer: 2PL waits, T/O and OCC restart; use OCC when conflicts are rare, 2PL when they are not

Example 26 [Microsoft · pattern]

Explain exactly how a COMMIT becomes durable on a modern server, including group commit and the operating system page cache. Then explain why fsync failing is such a dangerous bug.

SOLUTION

The path of a commit.

1. The transaction appends log records to the **log buffer** in RAM. Cheap: a memcpy.
2. COMMIT is issued. The log buffer up to this transaction's last LSN must reach **stable storage**.
3. The database calls `write()`. The bytes go into the **operating system page cache** — still RAM, still lost on a power cut.
4. The database calls `fsync()` (or opens the log with `O_DSYNC`). The OS pushes the pages to the device and waits.
5. The device may still hold them in its own **volatile write cache**. A device with a battery or capacitor is safe; a cheap consumer SSD may not be. The database asks for a `FLUSH CACHE / FUA` write to be sure.
6. `fsync` returns. Only now does COMMIT return to the client.

Group commit. Step 4 costs about the same whether it flushes 200 bytes or 200 KB — the cost is the round trip and the flush, not the bytes. So the log writer waits a tiny amount (or simply takes whatever arrived while the previous flush was in flight) and flushes many transactions' records together. 200 transactions then share one `fsync`.

$$\text{throughput} \approx \frac{\text{transactions per flush}}{\text{flush latency}}$$

With a 1 ms flush and no grouping: 1000 commits per second, full stop. With 200 per group: 200000 per second. This is why `innodb_flush_log_at_trx_commit` and `synchronous_commit` exist — turning them off buys throughput by giving up the D in ACID for the last few milliseconds of work.

Why a failing `fsync` is dangerous. On Linux, when a writeback error occurs the kernel may mark the page clean and report the error to **the next** `fsync` on that file descriptor — once. A subsequent `fsync` then returns success even though the data was never written. If the database treated the first failure as “retry later”, it would retry, see success, and report a durable commit for data that is gone. The correct behaviour, which PostgreSQL adopted after this was understood, is brutal and right: **`fsync failed` → panic the server and recover from the log**, because the in-memory state can no longer be trusted to match the disk.

Answer: log buffer → page cache → device → `fsync` returns → COMMIT returns; group commit amortises the flush; a failed `fsync` may not be retried, so the only safe response is to crash and replay the log

TRAP

If you are asked “what makes commits slow”, the wrong answer is “writing the rows to the table”. The right answer is “one `fsync` of the log per commit group, about 0.5–2 ms on a datacentre SSD” — and that is why batching a thousand inserts into one transaction is thousands of times faster than a thousand auto-commit inserts.

Example 27 [D. E. Shaw · pattern]

An INSERT under InnoDB's REPEATABLE READ deadlocks against another INSERT on a different primary key. Explain how that is possible.

SOLUTION

The answer is **gap locks** and **insert intention locks**.

InnoDB's REPEATABLE READ stops phantoms, which the SQL standard does not require. To do it, a locking read does not only lock the rows it found — it locks the **gaps between** index entries, so nobody can insert a new row into the range you looked at. A row lock plus the gap before it is a **next-key lock**.

Now take an index on `order_no` holding the values 10 and 30, and two sessions:

two inserts into the same gap

SQL

```
-- session 1                                -- session 2
BEGIN;                                       BEGIN;
SELECT * FROM t WHERE order_no BETWEEN 10 AND 30
```

```

FOR UPDATE;      -- next-key locks the gap (10,30)
                  SELECT * FROM t
                  WHERE order_no BETWEEN 10 AND 30
                  FOR UPDATE; -- gap locks are SHARED!

INSERT INTO t (order_no) VALUES (20);
-- needs an insert intention lock in (10,30);
-- blocked by session 2's gap lock

INSERT INTO t (order_no) VALUES
(25);
-- blocked by session 1's gap
lock
--
=> DEADLOCK

```

The key facts that make this surprising:

1. Gap locks are **compatible with each other**. Two sessions can both hold a gap lock on (10, 30). Nothing blocks at the SELECT.
2. An INSERT needs an **insert intention** lock in that gap, and that **is** blocked by another session's gap lock.
3. So both sessions pass the SELECT and both block on the INSERT. Cycle. Deadlock.

The same shape appears with INSERT ... ON DUPLICATE KEY UPDATE and with two inserts of the **same** unique value: the loser takes a shared lock on the existing row while waiting, then tries to upgrade.

Fixes.

1. Drop to READ COMMITTED, where InnoDB mostly stops using gap locks. You give up phantom protection — usually acceptable, and it is MySQL's most common production setting.
2. Do not take the range lock at all: INSERT first and handle the duplicate-key error, rather than “check whether it exists, then insert”.
3. Make the ranges disjoint, so two sessions never sit in the same gap.

Answer: gap locks are shared but insert intention locks conflict with them, so two sessions can both hold the gap and then both block trying to insert into it

This is also the reason SELECT ... FOR UPDATE on a column with **no index** is dangerous under InnoDB: with no index to lock gaps in, it locks **every** row it scans — effectively the whole table. Index your locking predicates.

Example 28 [Adobe · pattern]

Your team wants “exactly once” processing of a queue of jobs stored in a table. Many workers, at-least-once delivery, no job run twice. Design it and state precisely what you can and cannot guarantee.

SOLUTION

What is impossible. A worker can crash after finishing its side effect and before recording that it finished. No protocol removes that window, so **exactly once execution** of an arbitrary side effect is not achievable. What you can achieve is **exactly once effect**: at-least-once delivery plus idempotent work.

The claim step. One statement, atomic, no explicit lock.

claim a batch of jobs (PostgreSQL)

SQL

```

UPDATE jobs
  SET state      = 'RUNNING',
      worker_id  = 'w-17',
      claimed_at = now(),
      attempts   = attempts + 1
 WHERE job_id IN (
   SELECT job_id
   FROM jobs
   WHERE state = 'READY'

```

```

        AND run_after <= now()
    ORDER BY run_after
    LIMIT 10
    FOR UPDATE SKIP LOCKED      -- the important line
)
RETURNING job_id, payload;

```

`FOR UPDATE SKIP LOCKED` means: if another worker already holds the lock on that row, do not wait for it — **skip it and take the next one**. Without `SKIP LOCKED`, every worker queues on the same head-of-queue rows and the system serialises to one worker’s speed.

The idempotency step. The side effect carries a key derived from the job, so a repeat is a no-op.

```

record the effect and complete the job together      SQL

BEGIN;
INSERT INTO effects (idem_key, job_id, result)
    VALUES ('job-4412-v1', 4412, '{"charged":true}')
ON CONFLICT (idem_key) DO NOTHING;      -- second run inserts nothing

UPDATE jobs SET state = 'DONE', finished_at = now() WHERE job_id = 4412;
COMMIT;

```

The stuck-worker step. A worker that dies leaves a row in `RUNNING` forever. A reaper returns it:

```

lease expiry      SQL

UPDATE jobs
    SET state = 'READY', worker_id = NULL
WHERE state = 'RUNNING'
    AND claimed_at < now() - interval '5 minutes'
    AND attempts < 5;

```

The lease is what makes this safe **and** the reason the job may run twice: a worker that was merely slow, not dead, may still be running when the lease expires. That is exactly why the side effect must be idempotent.

What you can state precisely:

Guaranteed	every ready job is eventually attempted; no two workers ever claim the same job at the same instant; the effect row and the job’s <code>DONE</code> state flip together, atomically
Not guaranteed	that the side effect runs exactly once — only that repeats are harmless, because of the idempotency key
Cost	<code>SKIP LOCKED</code> gives up FIFO order; a slow job can be overtaken

Answer: claim with `FOR UPDATE SKIP LOCKED`, make the effect idempotent with a unique key, expire leases; the guarantee is at-least-once delivery with exactly-once effect

PRACTICE · TIER 3 target: 35 minutes

1. Give a schedule that is view serialisable but not conflict serialisable, and say which feature of it creates the gap.
2. Prove that acquiring locks in a fixed global order makes deadlock impossible.
3. A workload has a 30% conflict rate. Argue for 2PL over OCC using the expected-cost model.
4. Under Thomas’s write rule, when is a write simply discarded rather than causing an abort?
5. Explain why redo replays the changes of transactions that will immediately be undone.
6. `synchronous_commit = off` — which ACID letter did you sell, and exactly how much of it?

7. Why can `SELECT . . . FOR UPDATE` on an unindexed column lock an entire table in InnoDB?
8. A job queue without `SKIP LOCKED` has 40 workers but the throughput of one. Explain.

ANSWER KEY

1. $R_1(A)W_2(A)W_1(A)W_3(A)$ — the **blind write** $W_3(A)$ (and the blind $W_2(A)$) create the gap; view equivalence only cares who wrote **last**.
2. See Example 24: the highest lock held strictly increases along every wait edge, so a cycle would require a value to be less than itself.
3. OCC's expected cost $\approx W/(1-p) = W/0.7 = 1.43W$, and every restart repeats **all** the work; 2PL pays W plus a wait that is usually shorter than a full restart.
4. When $TS(T) \geq RTS(X)$ (no younger reader) but $TS(T) < WTS(X)$ (a younger writer already won) — the write is invisible to everyone, so dropping it is safe.
5. So that undo's "before" values are known to be correct; redo restores the exact crash state, which is the only state the undo log describes.
6. Durability, for the window between `COMMIT` returning and the next log flush — typically a few hundred milliseconds of committed transactions. Atomicity and consistency are unaffected: you lose whole transactions, never half of one.
7. With no index there are no index entries to place next-key locks on, so InnoDB locks every row it examines — which is all of them.
8. All 40 workers `SELECT . . . FOR UPDATE` the same head rows; 39 wait on the first, so the queue drains at one worker's rate.

Rapid-fire — one-line answers

Cover the right column. Say the answer out loud. These thirty cover most of what a technical round asks on transactions.

Question	Answer
Expand ACID.	Atomicity, Consistency, Isolation, Durability.
Which one does the <code>fsync</code> of the log deliver?	Durability.
What is a transaction?	A unit of work that happens all or not at all.
When do two operations conflict?	Different transactions, same item, at least one is a write.
Test for conflict serialisability?	Precedence graph has no cycle.
What is a dirty read?	Reading a row another transaction wrote but has not committed.
What is a phantom?	The same <code>WHERE</code> returns extra rows the second time.
Lowest level that stops dirty reads?	<code>READ COMMITTED</code> .
Lowest level that stops phantoms?	<code>SERIALIZABLE</code> .
Default level in PostgreSQL? In MySQL InnoDB?	<code>READ COMMITTED</code> ; <code>REPEATABLE READ</code> .
What is write skew?	Two transactions read the same set, then each writes a different row, breaking a rule about the set.
Which level stops write skew?	<code>SERIALIZABLE</code> only.
State two-phase locking.	Growing phase takes locks, shrinking phase releases them, never mixed.
Does 2PL prevent deadlock?	No. It guarantees serialisability and causes deadlocks.
What does strict 2PL add?	Locks held until commit — no cascading rollback.
S with S? S with X?	Compatible; not compatible.

How is a DBMS deadlock detected?	A cycle in the wait-for graph.
How is it resolved?	Roll back a victim, usually the cheapest to undo.
The one-line deadlock prevention rule?	Always take locks in the same global order.
State the write-ahead rule.	The log record reaches disk before the data page, and before COMMIT returns.
The three recovery passes?	Analysis, redo, undo.
Who is redone? Who is undone?	Redo everybody; undo only the losers.
What is a checkpoint for?	So recovery starts there instead of at the start of the log.
What is MVCC?	Writes create new row versions; each reader sees the versions alive in its snapshot.
Under MVCC, do readers block writers?	No. Writers block writers on the same row.
What does SELECT . . . FOR UPDATE do?	Takes an exclusive row lock at read time.
Optimistic concurrency in one line?	Read a version, write with WHERE version = ?, retry if 0 rows changed.
What is group commit?	Many transactions share one log fsync.
Can TRUNCATE be rolled back?	In PostgreSQL yes; in MySQL and Oracle DDL commits implicitly, so no.
What SQLSTATE means “serialisation failure”?	40001 — and the right response is to retry.

ONE-PAGE REVISION CARD

The four words. Atomicity — undo log. Consistency — constraints. Isolation — locks or MVCC. Durability — log fsync.

The table to draw from memory

Level	Dirty	Non-repeatable	Phantom	Write skew
READ UNCOMMITTED	yes	yes	yes	yes
READ COMMITTED	no	yes	yes	yes
REPEATABLE READ	no	no	yes*	yes
SERIALIZABLE	no	no	no	no

*InnoDB's REPEATABLE READ also blocks phantoms, using gap locks. The standard does not require it.

Conflict rule. Different transactions + same item + at least one write. **Serialisability test.** Precedence graph, look for a cycle.

Locking. 2PL = grow then shrink → serialisable, but deadlocks. Strict 2PL = hold until commit → no cascading rollback. This is what engines do. Deadlock = cycle in the wait-for graph → kill a victim → **the application must retry**. Prevent it by locking in a fixed global order.

Recovery. Log first, sequentially. COMMIT returns after fsync. On restart: analysis (winners / losers) → redo **everything** → undo **losers**, backwards. Checkpoint = where recovery starts reading.

MVCC. New version per write, xmin / xmax, snapshot decides visibility. Readers never block. Cost = vacuum, and bloat from long-open transactions.

The five answers that win marks

1. "Durability means the **log** is on disk, not the data file."
2. "2PL prevents non-serialisable schedules; it **causes** deadlock."
3. "SERIALIZABLE means equivalent to **some** serial order, not actually one at a time."
4. "TRUNCATE rollback depends on the engine."
5. "A check-then-act in the application is never safe without a lock, a unique constraint, or a serialisable retry."

The seven traps

1. Dirty read means **uncommitted**, not **old**.
2. ROLLBACK after COMMIT does nothing.
3. REPEATABLE READ does not stop write skew.
4. MVCC does not remove write-write locking.
5. Never hold a transaction across a network call.
6. A long idle transaction bloats the whole table.
7. SELECT ... FOR UPDATE without an index can lock the entire table.

DBMS: Indexing & Query Performance

Why the same query takes 3 milliseconds or 30 seconds

The chapter in one page

An index is a **sorted copy of one or more columns**, plus a pointer back to the row. That is all. Every question in this chapter comes from three consequences of that sentence:

1. Sorted means you can **binary search** it, so a lookup costs $O(\log n)$ page reads instead of reading the whole table.
2. A **copy** means every INSERT, UPDATE and DELETE has to maintain it. Indexes make reads fast and writes slow.
3. Sorted **in a particular order** means an index helps only the queries that match that order. A composite index on (city, age) cannot help WHERE age = 30.

The interviewer's real question is almost always the same: **given this query and this table, would the planner use the index, and should it?**

WHAT YOU NEED TO KNOW

The unit of I/O is the page, not the row. Typical page size 8 KB (PostgreSQL) or 16 KB (InnoDB). The database never reads one row from disk; it reads the page the row sits on.

$$\text{rows per page} = \left\lfloor \frac{\text{page size}}{\text{row size}} \right\rfloor \quad \text{table pages} = \left\lceil \frac{\text{rows}}{\text{rows per page}} \right\rceil$$

B+ tree — the index structure in every relational database.

- All keys live in the **leaf** level. Internal nodes hold only separator keys, so the fanout is huge.
- Every leaf is at the same depth, so every lookup costs the same.
- Leaves are joined in a **linked list**, so a range scan descends once and then walks sideways.

$$\text{fanout } f = \left\lfloor \frac{\text{page size} \times \text{fill factor}}{\text{entry size}} \right\rfloor \quad \text{height } h = \lceil \log_f(\text{number of keys}) \rceil$$

Cost of a lookup = h page reads to reach the leaf, +1 more to fetch the row if the index does not already contain everything the query needs.

Clustered vs secondary

Clustered index	The table is the index. The leaf holds the whole row. One per table. InnoDB always clusters on the primary key.
Secondary index	The leaf holds the key plus a pointer to the row. Many per table. In InnoDB that pointer is the primary key, so a secondary lookup is two descents.
Heap table	No clustering. Rows sit wherever there was room. The pointer is a physical address (ctid in PostgreSQL). This is PostgreSQL's model.

Selectivity — the fraction of the table a predicate keeps.

$$\text{selectivity} = \frac{\text{matching rows}}{\text{total rows}}$$

Low selectivity value (say 0.001) = highly selective = **good for an index**. Selectivity near 1 = the index is useless, a sequential scan is cheaper.

Cost model in one line — a random page read costs several sequential page reads.

$$\text{cost}_{\text{index}} = (\text{height} + \text{matched rows}) \times r \quad \text{cost}_{\text{scan}} = \text{table pages}$$

where r is the random-to-sequential cost ratio (about 4 on an SSD, 20+ on a spinning disk).

The leftmost prefix rule. An index on (c_1, c_2, c_3) can serve a predicate only on a **prefix** of that list: c_1 ; or c_1 and c_2 ; or all three. It cannot serve c_2 alone. Inside the prefix, everything up to the **first range predicate** is used for seeking; after that, the index gives order but not a seek.

Column order rule: equality columns first, then the range column, then the ORDER BY columns. Remember it as **E-R-O**.

Sargable (“Search ARGument able”) — a predicate the index can seek on. $col = value$ is sargable. $f(col) = value$ is not, because the index stores col , not $f(col)$.

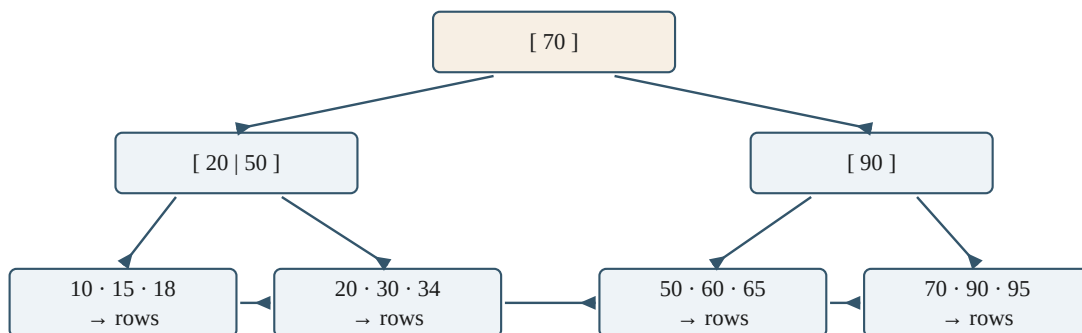
Covering index / index-only scan — the index contains every column the query touches, so the table is never read at all.

Join algorithms

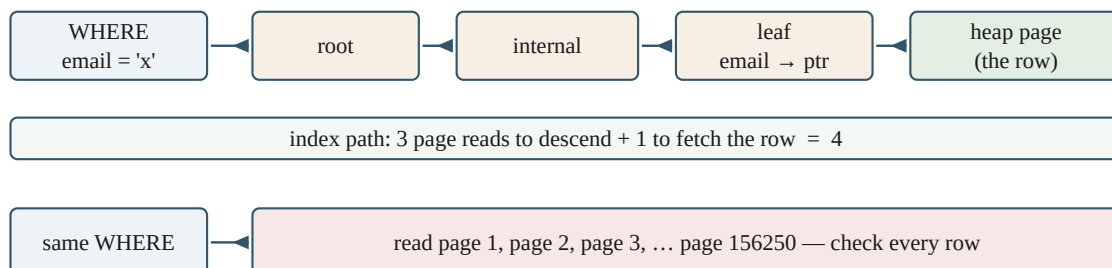
Algorithm	Cost shape	Wins when
Nested loop	$ R \times \text{lookup}$	the outer side is tiny and the inner side has an index
Hash join	$ R + S $	both sides are big and the join is on equality
Sort-merge join	$ R \log R + S \log S $	both sides are already sorted on the join key, or the join is on a range

What kills an index — a function on the column, a leading wildcard, an implicit type cast, OR across different columns, $!=$, low cardinality, or a predicate that matches most of the table.

The structure



A B+ tree. All data is in the leaves; the leaves are a linked list, which is what makes a range scan cheap.



Two ways to answer `WHERE email = 'x'`. The index path reads 4 pages; the scan path reads every page in the table.

Warm-up — the mechanics

Example 1

A table has 10,000,000 rows. Each row is 128 bytes and the page size is 8192 bytes. How many pages does the table occupy, and how big is it in megabytes?

SOLUTION

Step 1 — rows per page.

$$\left\lfloor \frac{8192}{128} \right\rfloor = 64 \text{ rows per page}$$

Step 2 — pages.

$$\left\lceil \frac{10^7}{64} \right\rceil = 156250 \text{ pages}$$

Step 3 — size.

$$156250 \times 8192 = 1.28 \times 10^9 \text{ bytes} = 1220.7 \text{ MB}$$

Answer: 156,250 pages, about 1.22 GB

Remember these three numbers — 64 rows per page, 156,250 pages, 1.2 GB. Every example in this chapter reuses them, so the arithmetic stays out of your way.

Example 2

Build a B+ tree index on a 4-byte integer key over those 10 million rows. Each index entry is 16 bytes (key + pointer + overhead). How many levels does the tree have?

SOLUTION

Step 1 — fanout.

$$f = \left\lfloor \frac{8192}{16} \right\rfloor = 512$$

Step 2 — leaf level. Each leaf holds 512 entries.

$$\left\lceil \frac{10^7}{512} \right\rceil = 19532 \text{ leaves}$$

Step 3 — the level above.

$$\left\lceil \frac{19532}{512} \right\rceil = 39 \text{ nodes}$$

Step 4 — the level above that.

$$\left\lceil \frac{39}{512} \right\rceil = 1 \text{ node — the root}$$

height calculator, run with node

JS

```
function bptHeight(rows, pageBytes, entryBytes, fill = 1.0) {
  const fanout = Math.floor((pageBytes * fill) / entryBytes);
  let nodes = Math.ceil(rows / fanout), levels = 1; // the leaf level
  while (nodes > 1) { nodes = Math.ceil(nodes / fanout); levels++; }
  return { fanout, levels };
}
```

```
console.log(bptHeight(10_000_000, 8192, 16)); // { fanout: 512, levels: 3 }
console.log(bptHeight(10_000_000, 8192, 16, 0.7)); // { fanout: 358, levels: 3 }
console.log(bptHeight(10_000_000, 8192, 64)); // { fanout: 128, levels: 4 }
```

Answer: 3 levels — root, one internal level, leaves. So 3 page reads to reach a leaf.

Notice the third line. A 56-byte text key drops the fanout from 512 to 128 and adds a level. **Fat keys make tall trees.** That is the whole argument for indexing an integer surrogate key rather than a long VARCHAR.

Example 3

Real B+ trees are not packed 100% full. At a 70% fill factor, does the tree from Example 2 get taller?

SOLUTION

$$f = \left\lfloor 8192 \times \frac{0.7}{16} \right\rfloor = 358$$

$$\left\lceil \frac{10^7}{358} \right\rceil = 27933 \text{ leaves} \quad \left\lceil \frac{27933}{358} \right\rceil = 79 \quad \left\lceil \frac{79}{358} \right\rceil = 1$$

Still 3 levels. The tree grew 43% in size but not in height, because height grows like $\log_f n$ — you have to change the fanout enormously to add a level.

Answer: no, still 3 levels; the tree is wider, not taller

SHORTCUT

Height barely moves. With a fanout around 400, a 3-level tree already holds $400^3 = 64$ million keys and a 4-level tree holds 25 **billion**. So the honest answer to “how many I/Os does an index lookup take?” is almost always **“three or four, plus one for the row”** — and the root and the level below it are practically always in the buffer pool, so in practice you pay 1 to 2 real disk reads.

Example 4

Which of these predicates can use an index on orders(order_date)? Say yes or no and why.

SQL

1. WHERE order_date = DATE '2026-01-15'
2. WHERE order_date >= DATE '2026-01-01' AND order_date < DATE '2026-02-01'
3. WHERE YEAR(order_date) = 2026
4. WHERE order_date + INTERVAL '1 day' > now()
5. WHERE order_date IS NULL

SOLUTION

No.	Index?	Why
1	yes	equality on the indexed column — a single descent.
2	yes	a range on the indexed column — descend once, walk the leaves.
3	no	the index stores order_date, not YEAR(order_date). Not sargable.
4	no	same problem: the column is wrapped in an expression.
5	yes	NULLs are stored in the index in PostgreSQL and InnoDB, so this seeks fine.

Fix number 3 by rewriting the predicate so the **column stands alone**:

the rewrite that makes it sargable

SQL

```
WHERE order_date >= DATE '2026-01-01'
AND order_date < DATE '2027-01-01'
```

And number 4 by moving the arithmetic to the other side:

```
WHERE order_date > now() - INTERVAL '1 day'
```

Answer: 1, 2 and 5 yes; 3 and 4 no — a function on the column kills the seek

TRAP

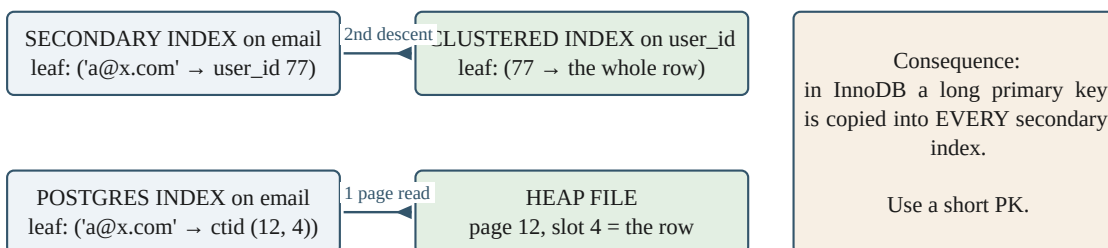
This is the single most common indexing bug in real code and a favourite interview question. The rule to say out loud: **keep the indexed column bare on the left-hand side**. If you truly need the function, build an **expression index**: `CREATE INDEX idx_o_year ON orders ((EXTRACT(YEAR FROM order_date)))`;

Example 5

Explain the difference between a clustered index and a secondary index in one sentence each, and say how many of each a table can have.

SOLUTION

- **Clustered** — the leaf level **is** the table; the rows are physically stored in index order. At most **one** per table, because a row can only be in one place.
- **Secondary** — the leaf holds the key and a pointer to the row. **Many** per table.



InnoDB: a secondary index points at the primary key, so a secondary lookup is two descents.

Answer: one clustered index, many secondary indexes

Example 6

`CREATE INDEX idx ON emp (dept_id, salary);` Which of these can seek on the index?

- `WHERE dept_id = 4`
- `WHERE dept_id = 4 AND salary > 50000`
- `WHERE salary > 50000`
- `WHERE salary = 50000 AND dept_id = 4`

SOLUTION

The index is sorted by `dept_id` first, and only **within one `dept_id`** by salary.

No.	Seek?	Why
-----	-------	-----

a	yes	dept_id is the leftmost column.
b	yes	seek to dept_id = 4, then walk forward from salary > 50000 — both columns used.
c	no	salary is not a prefix. Rows with salary 50001 are scattered through every department. At best the engine does an index-only scan of the whole index, which is cheaper than a table scan but is not a seek.
d	yes	the written order of the AND conditions does not matter. The planner reorders them. Only the index column order matters.

Answer: a, b and d seek; c cannot

TRAP

“I wrote salary first in my WHERE, so I need an index with salary first.” No. AND is commutative and the planner knows it. What matters is the **index** definition order, never the order you typed the conditions.

Example 7

What does EXPLAIN show, and what does EXPLAIN ANALYZE show that EXPLAIN does not?

SOLUTION

- EXPLAIN prints the **plan the planner chose** and its **estimates**. It does not run the query.
- EXPLAIN ANALYZE actually runs the query and prints **real** row counts and **real** timings next to the estimates.

reading a PostgreSQL plan

SQL

```
EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id = 4412;

-- Index Scan using idx_orders_cust on orders
--   (cost=0.43..38.12 rows=11 width=128)
--   (actual time=0.031..0.079 rows=9 loops=1)
--   Index Cond: (customer_id = 4412)
-- Planning Time: 0.121 ms
-- Execution Time: 0.098 ms
```

cost=0.43..38.12	startup cost .. total cost, in arbitrary units where 1.0 is one sequential page read. Not milliseconds.
rows=11	the estimate .
actual ... rows=9	the truth . Compare it with the estimate — this is the whole point.
loops=1	how many times this node ran. Multiply actual time by loops for the real cost.

Answer: EXPLAIN = the plan and estimates; EXPLAIN ANALYZE = the plan plus what actually happened

SHORTCUT

When you read a plan, look at **one number first**: estimated rows versus actual rows. If they differ by 100× or more, the planner is working from bad statistics, and every decision above that node in the tree is built on sand. Fix the statistics before you touch the query.

PRACTICE · WARM-UP target: 10 minutes

- A page is 8 KB and a row is 256 bytes. Rows per page?
- Fanout 200, 8 million keys. How many leaf nodes?

3. Can an index on (a, b, c) serve WHERE a = 1 AND c = 3? Fully or partly?
4. Which is sargable: WHERE UPPER(name) = 'RAVI' or WHERE name = 'Ravi'?
5. How many clustered indexes can one table have?
6. In EXPLAIN, is cost measured in milliseconds?
7. Does WHERE id != 5 use an index on id well?

ANSWER KEY

1. $\lfloor 8192/256 \rfloor = 32$.
2. $\lceil 8000000/200 \rceil = 40000$.
3. Partly: it seeks on a = 1, then **filters** on C while scanning that range. b is missing, so C cannot be used for seeking.
4. WHERE name = 'Ravi'.
5. One.
6. No — arbitrary units, where 1.0 is roughly one sequential page read.
7. No. != matches almost everything, so a sequential scan is cheaper.

Tier 1 — the questions a service company asks

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 8 [TCS NQT · pattern]

“Indexes make queries faster, so index every column.” Give four reasons this is wrong, with numbers.

SOLUTION

Reason 1 — every write pays for every index. An INSERT into a table with k secondary indexes performs $1 + k$ B+ tree insertions.

Secondary indexes	Tree insertions per row	Relative insert cost
0	1	1×
1	2	2×
3	4	4×
6	7	7×

Each of those is a descent plus a possible **page split**, which is a random write.

Reason 2 — space. Our 10 million row index at 16 bytes per entry and a 70% fill factor is $10^7 \times 16/0.7 = 218$ MB. Six such indexes are 1.3 GB — bigger than the 1.22 GB table. They compete with the table for buffer-pool memory, so **adding indexes can make unrelated queries slower** by pushing hot pages out of cache.

Reason 3 — unused indexes are pure loss. An index on a column no query filters on costs writes and memory and returns nothing. Find them:

PostgreSQL: indexes that have never been read

SQL

```
SELECT relname AS table_name, indexrelname AS index_name, idx_scan
FROM pg_stat_user_indexes
WHERE idx_scan = 0
ORDER BY pg_relation_size(indexrelid) DESC;
```

Reason 4 — a bad index can be chosen over a good plan. More indexes means more plans to consider, and with skewed data the planner sometimes picks an index scan that is far worse than a sequential scan (Example 11).

Answer: writes multiply, space can exceed the table, unused indexes are pure cost, and extra choices can mislead the planner

Example 9 [Infosys · pattern]

What is a covering index? Show the difference on a real query.

SOLUTION

A covering index contains **every column the query mentions**, so the engine answers the query from the index alone and never touches the table. PostgreSQL calls this an **index-only scan**.

before: index seek + heap fetch for every row

SQL

```
CREATE INDEX idx_o_cust ON orders (customer_id);

SELECT customer_id, amount
FROM orders
WHERE customer_id = 4412;
-- Index Scan: descend 3 pages, then ONE HEAP READ PER MATCHING ROW to get `amount`
```

after: everything the query needs is in the index

SQL

```
CREATE INDEX idx_o_cust_amt ON orders (customer_id, amount);
-- or, PostgreSQL 11+, keep `amount` as a payload only:
CREATE INDEX idx_o_cust_inc ON orders (customer_id) INCLUDE (amount);

SELECT customer_id, amount
FROM orders
WHERE customer_id = 4412;
-- Index Only Scan: descend 3 pages, read the leaf entries, done. Zero heap reads.
```

The numbers. Say 200 orders match. Before: $3 + 200 = 203$ page reads, of which 200 are **random**. After: $3 + 1 = 4$ page reads, all clustered together in the leaf. Roughly $50\times$ fewer I/Os.

INCLUDE versus adding the column to the key

key column	is sorted, can be searched and can satisfy an ORDER BY; makes the entry bigger at every level of the tree
INCLUDE column	payload only — stored in the leaves, not in internal nodes. Cannot be searched, but keeps the tree's fanout high

Put a column in the key if you filter or sort on it. Put it in INCLUDE if you only SELECT it.

Answer: an index that holds all the columns the query needs, so the table is never read — “index-only scan”

TRAP

In PostgreSQL an index-only scan is **not always** index-only. Because of MVCC, the index entry does not know whether its row version is visible to you. The engine consults the **visibility map**; if the page is not marked all-visible, it must fetch the heap row anyway. A table that has not been vacuumed recently will show Heap Fetches: 198 in EXPLAIN ANALYZE and get none of the benefit. VACUUM the table and the number drops to 0.

Example 10 [Wipro · pattern]

Order the columns for an index that must serve this query well:

```
SELECT order_id, amount
FROM orders
WHERE status = 'SHIPPED'
AND shipped_at >= DATE '2026-01-01'
ORDER BY amount DESC
LIMIT 20;
```

SOLUTION

Apply **E-R-O: Equality** first, then the **Range**, then the **Order by**.

- Equality: status
- Range: shipped_at
- Order by: amount DESC

So the key order is (status, shipped_at, amount).

```
CREATE INDEX idx_o_ship ON orders (status, shipped_at, amount DESC);
```

But look carefully at what this actually buys. Everything **after** the first range predicate is no longer in usable sort order — the index is sorted by amount only **within one exact shipped_at value**. So the engine can seek on status and shipped_at, but it must still collect all matching rows and sort them by amount before the LIMIT 20. The ORDER BY is not free.

The alternative, when the range is broad:

put the sort column second, pay a filter instead

```
CREATE INDEX idx_o_ship_amt ON orders (status, amount DESC);
-- walk the index in amount DESC order inside status='SHIPPED',
-- filter each entry on shipped_at, stop after 20 survivors -> no sort at all
```

This wins when most shipped orders are recent, so the first 20 big amounts are found quickly. It loses badly when the date filter is very selective, because the engine may walk a long way down the amount order before finding 20 rows that pass the date filter.

Answer: (status, shipped_at, amount) by the E-R-O rule; but if the date range is broad, (status, amount) avoids the sort and usually wins

This is a trade-off, not a rule, and saying so is what separates a Tier-1 answer from a Tier-3 one. Two indexes, two plans; the right answer depends on the data. Run both with EXPLAIN ANALYZE and compare **actual** time.

Example 11 [Accenture · pattern]

An index exists on orders(status) but the planner chooses a sequential scan for WHERE status = 'DELIVERED'. Is the planner wrong? Compute.

SOLUTION

Use our standard table: 10 million rows, 64 per page, 156,250 pages. Suppose 88% of orders are DELIVERED, so 8.8 million rows match.

Index path. Descend 3 pages, then one heap read per matching row, scattered randomly:

$$3 + 8800000 \approx 8.8 \text{ million random page reads}$$

Scan path. Read every page once, sequentially:

$$156250 \text{ sequential page reads}$$

The index path does **56 times more I/O**, and each of those I/Os is a **random** read, which costs several times a sequential one. The planner is right by a factor of about 200.

Where is the crossover? Let $r = 4$ (a random read costs 4 sequential ones) and let m be the number of matching rows.

$$4(3 + m) < 156250 \Rightarrow m < 39059.5$$

crossover calculator, run with node

JS

```
const PAGE = 8192, ROW = 128, N = 10_000_000, RANDOM_COST = 4;
const tablePages = Math.ceil(N / Math.floor(PAGE / ROW)); // 156250

const cost = (matched) => {
  const idx = (3 + matched) * RANDOM_COST;
  return { matched, pct: +(matched / N * 100).toFixed(3), index: idx,
    scan: tablePages, winner: idx < tablePages ? "index" : "seq scan" };
};
[100, 1000, 39000, 40000, 100000].forEach(m => console.log(cost(m)));
```

Output:

rows matched	% of table	index cost	scan cost	winner
100	0.001	412	156250	index
1000	0.01	4012	156250	index
39000	0.39	156012	156250	index
40000	0.4	160012	156250	seq scan
100000	1.0	400012	156250	seq scan

Answer: the planner is right; past roughly 0.4% of the table a sequential scan is cheaper

TRAP

The often-quoted “5% rule” is a rough memory aid, not a law. The real crossover depends on the random-to-sequential cost ratio and on the **clustering factor** — how well the table’s physical order matches the index order. If the matching rows happen to sit together (a clustered index, or a naturally time-ordered table), 8.8 million rows occupy only $\lceil 8800000/64 \rceil = 137500$ pages read almost sequentially, and the index wins again. Mention the clustering factor and you have said something most candidates do not.

Example 12 [Capgemini · pattern]

Name six ways a developer accidentally stops the planner from using an index, and give the fix for each.

SOLUTION

Killer	Broken	Fix
Function on the column	WHERE YEAR(d) = 2026	WHERE d >= '2026-01-01' AND d < '2027-01-01' — or an expression index
Leading wildcard	WHERE name LIKE '%ravi'	LIKE 'ravi%' seeks; for the other direction index a reversed copy of the column, or use a trigram index
Implicit type cast	WHERE phone = 9876543210 on a VARCHAR column	quote it: WHERE phone = '9876543210'

OR across columns	WHERE a = 1 OR b = 2	SELECT ... WHERE a = 1 UNION SELECT ... WHERE b = 2 — each branch can use its own index
Negation	WHERE status != 'DONE'	list the values: WHERE status IN ('NEW', 'RUNNING')
Low cardinality	WHERE active = true when 95% are true	a partial index: CREATE INDEX ... WHERE active = false

The implicit-cast trap is worth its own line. If phone is VARCHAR and you compare it to a number, the database must cast **every row's** phone to a number to compare. That is a function on the column, so the index is dead — and the query still returns the right answer, just 1000× slower. It is invisible in code review.

the partial index, the most under-used tool here

SQL

```
-- 20 million rows, only ~4000 are unprocessed at any moment
CREATE INDEX idx_jobs_pending ON jobs (created_at) WHERE state = 'PENDING';

-- this query uses it; the index is a few pages, not gigabytes
SELECT * FROM jobs WHERE state = 'PENDING' ORDER BY created_at LIMIT 100;
```

Answer: functions, leading wildcards, implicit casts, OR, negation, low cardinality

Example 13 [Cognizant · pattern]

Compare a B+ tree index with a hash index. When would you ever choose the hash index?

SOLUTION

	B+ tree	Hash
col = value	$O(\log_f n)$, about 3 reads	$O(1)$, about 1 read
col > value / BETWEEN	yes — descend once, walk leaves	no — hashing destroys order
ORDER BY col	free, the index is sorted	no help at all
LIKE 'abc%'	yes — it is a range	no
MIN / MAX	free — first or last leaf	full scan
Multi-column prefix	yes	only on the whole key
Size	larger	smaller
Worst case	guaranteed $O(\log n)$	$O(n)$ if the hash function is poor

When to choose hash: only when **every** query on the column is an exact-equality lookup, the column is long (so the fixed-size hash saves real space), and you will never sort or range-scan on it. A common real case is a big text or URL column used only as a lookup key.

Why B+ trees dominate anyway: one structure answers equality, range, prefix, sort, MIN, MAX and top-N. A hash index answers exactly one question. The B+ tree's extra 2 page reads are usually served from the buffer pool and cost nothing.

Answer: B+ tree unless every single query is exact equality and you never need order

Other index types worth naming in one line each, because an interviewer may ask “what else is there?”: **Bitmap** — one bit vector per distinct value; superb for low-cardinality columns in a data warehouse, terrible under concurrent writes (a single update locks a whole bit vector). **GiST / GIN** — inverted indexes for full-text search, arrays and JSON: they index the **elements inside** a value. **R-tree** — bounding boxes for geographic queries (“which shops are inside this rectangle”). **LSM**

tree — used by RocksDB and Cassandra: writes go to a memory table and are merged into sorted files later. Great write throughput, reads may have to check several files.

Example 14 [TCS Digital · pattern]

Explain a **composite** index by showing the physical order of the entries. Use (city, age, salary).

SOLUTION

The index is one sorted list. Sorting is by city; ties broken by age; ties broken by salary. Exactly like sorting names in a phone book by surname, then first name.

the actual index order, run with node

JS

index order (city, age, salary):

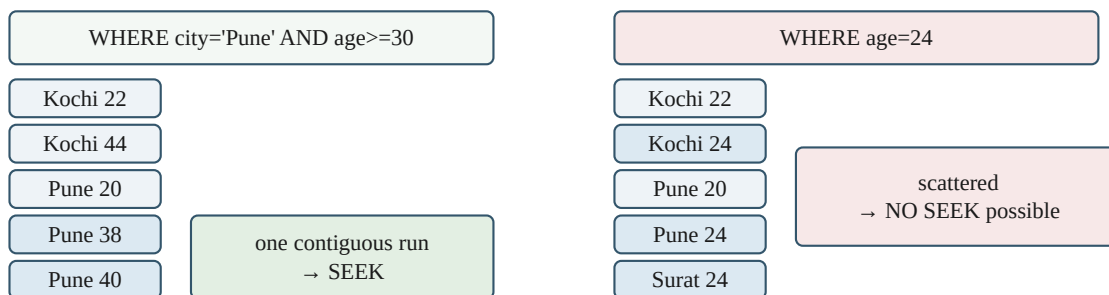
```
Kochi 22 34
Kochi 24 42
Kochi 44 50
Pune 20 20
Pune 24 20
Pune 24 30
Pune 26 50
Pune 38 70
Pune 40 96
Surat 24 42
Surat 46 40
Surat 46 52
```

city='Pune' AND age>=30 -> contiguous run of 2 entries

age=24 alone -> positions 1,4,5,9 (not contiguous -> index useless)

Read the two results carefully.

- city = 'Pune' AND age >= 30 is a **contiguous run**. The engine seeks to the first entry and reads forward until the run ends. That is what a seek **is**.
- age = 24 alone appears at positions 1, 4, 5 and 9 — scattered through the whole index. To find them the engine would have to read **every** entry. There is nothing to seek to.



The leftmost prefix rule, drawn. A seek needs the matching entries to be next to each other.

Answer: sorted by the first column, ties by the second, ties by the third — so only a leftmost prefix produces a contiguous run

SHORTCUT

One index on (a, b, c) already covers (a) and (a, b). So never create all three; create the widest one. Conversely, an index on (a, b) does **not** cover (b) — that needs its own index if WHERE b = ? is a real query.

Example 15 [Infosys · pattern]

Page 5000 of a result set loads in 4 seconds; page 1 loads in 8 milliseconds. The query is `ORDER BY created_at DESC LIMIT 20 OFFSET 99980`. Explain and fix.

SOLUTION

Why. `OFFSET n` does not skip cheaply. The engine must **produce** the first n rows in order and throw them away. `OFFSET 99980` builds 100,000 rows to return 20. The cost is linear in the page number, so the last pages of a listing are always the slowest.

OFFSET	Rows produced and discarded
0	0
1,000	1,000
100,000	100,000
1,000,000	1,000,000

Fix — keyset pagination (also called seek pagination or cursor pagination). Instead of “skip 99,980 rows”, say “start just after the last row I showed you”.

before — cost grows with the page number

SQL

```
SELECT post_id, created_at, title
FROM posts
ORDER BY created_at DESC, post_id DESC
LIMIT 20 OFFSET 99980;
```

after — constant cost, every page

SQL

```
-- the client sends back the (created_at, post_id) of the last row it received
SELECT post_id, created_at, title
FROM posts
WHERE (created_at, post_id) < (TIMESTAMP '2026-03-04 11:20:00', 88213)
ORDER BY created_at DESC, post_id DESC
LIMIT 20;
```

With an index on `(created_at DESC, post_id DESC)` this is a 3-page descent plus one leaf read, **for every page**. Page 5000 costs the same as page 1.

Two details that make it correct:

1. The **row-value comparison** `(a, b) < (x, y)` is the whole trick. Writing `created_at <= x AND post_id < y` is wrong — it drops rows with an earlier timestamp and a larger id.
2. The sort key must be **unique**, which is why `post_id` is appended. With ties on `created_at` alone, rows would be skipped or repeated across pages.

	OFFSET	Keyset
Cost of page k	grows with k	constant
Jump to page 500	yes	no — only next/previous
Stable when rows are inserted	no — rows shift between pages	yes

Answer: OFFSET materialises and discards every skipped row; use keyset pagination with a row-value comparison on a unique sort key

Example 16 [Wipro · pattern]

Explain the three join algorithms and which one the planner picks for each of these: (a) one customer joined to their orders, (b) all 200,000 customers joined to all 2,000,000 orders.

SOLUTION

Table sizes with our page model: $\text{orders} = \lceil 2000000/64 \rceil = 31250$ pages, $\text{customers} = \lceil 200000/64 \rceil = 3125$ pages.

Nested loop join. For each row of the outer side, look up the inner side.

$$\text{cost} = \text{outer pages} + |\text{outer rows}| \times \text{inner lookup cost}$$

Hash join. Build a hash table on the smaller side in memory, then stream the bigger side through it.

$$\text{cost} = \text{pages}_R + \text{pages}_S = 31250 + 3125 = 34375$$

Sort-merge join. Sort both sides on the join key, then walk them in step.

$$\text{cost} \approx 2 \times \text{pages}_R + 2 \times \text{pages}_S + (\text{pages}_R + \text{pages}_S) \approx 103125$$

Case (a) — one customer, about 10 orders. Outer side is a **single row**. Nested loop with an index on `orders(customer_id)`:

$$1 + 1 \times (3 + 10) = 14 \text{ page reads}$$

A hash join would read all 31,250 order pages to build the table. Nested loop wins by 2000×.

Case (b) — everything joined to everything. Nested loop: $3125 + 200000 \times (3 + 10) = 2603125$ page reads, nearly all random. Hash join: 34,375 sequential page reads. Hash join wins by about 76×.

Nested loop	tiny outer side, indexed inner side. Also the only choice for non-equality joins like <code>ON a.x BETWEEN b.lo AND b.hi</code> .
Hash join	both sides large, equality join, and the smaller side fits in the work memory. If it does not fit, it spills to disk in partitions and costs roughly double.
Sort-merge	both sides already sorted on the join key (for example, both come from index scans in that order), or the join is a range, or the output must be sorted anyway.

Answer: (a) nested loop with an index — 14 reads; (b) hash join — 34,375 reads versus 2.6 million

TRAP

“Nested loop is $O(nm)$ so it is always bad.” It is the **fastest** join in an OLTP system, because the outer side is usually one row from a primary-key lookup. What makes nested loop catastrophic is a **wrong row estimate**: the planner thinks the outer side has 3 rows, picks a nested loop, and the outer side actually has 300,000. This is the single most common cause of a query that was fast for a year and is suddenly slow.

PRACTICE · TIER 1 target: 25 minutes

1. A table gains 4 secondary indexes. By roughly what factor does an INSERT get more expensive?
2. Rewrite `WHERE LOWER(email) = 'a@x.com'` so it can use an index (two possible answers).
3. Index on (a, b, c). Which columns does `WHERE a = 1 AND b > 5 AND c = 9` seek on?
4. Why is `LIKE '%son'` unindexable but `LIKE 'son%'` indexable?
5. A query returns 40% of the table. Index scan or sequential scan?
6. Name the join algorithm for: `WHERE o.order_id = 991` joined to `order_items`.
7. What does `Heap Fetches: 0` in an `EXPLAIN ANALYZE` tell you?
8. Give the index that makes `SELECT max(amount) FROM orders WHERE status='NEW'` instant.

ANSWER KEY

1. About 5× — one clustered insertion plus four secondary ones.
2. Either `CREATE INDEX ... ON users (LOWER(email))`, or store the email lower-cased at write time and query `WHERE email = 'a@x.com'`.
3. a and b. c is only a filter, because b is a range and order below a range is lost.

4. The index is sorted left to right, so a known prefix is a contiguous run; a known **suffix** is scattered everywhere.
5. Sequential scan — far past the crossover.
6. Nested loop with an index on `order_items(order_id)`.
7. It was a true index-only scan; the table never had to be read, so the visibility map said every page was all-visible.
8. `CREATE INDEX ON orders (status, amount DESC);` — the answer is the first entry in the `status='NEW'` run.

Tier 2 — applied, business-flavoured

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 17 [Shopee · pattern]

A product search page runs this and takes 2.4 seconds. Redesign the indexes.

SQL

```
SELECT p.product_id, p.title, p.price
FROM products p
WHERE p.category_id = 77
      AND p.is_active = true
      AND p.price BETWEEN 500 AND 2000
ORDER BY p.sold_count DESC
LIMIT 40;
```

Facts: 40 million products; 96% are active; category 77 has 120,000 products; about 18,000 of those are in the price band.

SOLUTION

Step 1 — rank the predicates by selectivity.

Predicate	Rows kept	Selectivity
<code>category_id = 77</code>	120,000	0.003 — very selective
<code>price BETWEEN 500 AND 2000</code>	18,000 of those	0.15 within the category
<code>is_active = true</code>	96%	0.96 — nearly useless

Step 2 — apply E-R-O, but think about the LIMIT. Equality: `category_id`. Range: `price`. Order: `sold_count DESC`.

Candidate A — (`category_id`, `price`, `sold_count DESC`). The engine seeks `category_id = 77` AND `price BETWEEN 500 AND 2000`, reads all 18,000 matching entries, **sorts them** by `sold_count`, and keeps 40. The sort of 18,000 rows is the cost.

Candidate B — (`category_id`, `sold_count DESC`). The engine walks category 77 in `sold_count` order and filters each entry on price, stopping as soon as 40 survive. If roughly 15% of the category is in the price band, it expects to read about $40/0.15 \approx 267$ entries. **No sort at all.**

Candidate B reads 267 entries instead of sorting 18,000. It wins.

Step 3 — make it covering and partial.

the index to build

SQL

```
CREATE INDEX idx_p_cat_sold
ON products (category_id, sold_count DESC)
INCLUDE (price, title)
WHERE is_active = true;
```

- `is_active` moves into a `WHERE` clause on the index: a **partial index**. It stops being a column (which would waste space at 96% one value) and becomes a filter on which rows are indexed at all.
- `price` and `title` are `INCLUDED`, so the filter and the output both come from the index. Zero heap reads.

Step 4 — state the failure mode honestly. Candidate B is a gamble on the price filter not being too selective. If a user searches a price band containing 0.1% of the category, the engine may walk 40,000 entries before finding 40 survivors — slower than candidate A. If your traffic has both shapes, keep both indexes and let the planner choose using the histogram on price.

Answer: (category_id, sold_count DESC) INCLUDE (price, title) WHERE is_active — avoid the sort, make it covering, and move the low-cardinality flag into a partial-index predicate

Example 18 [Grab · pattern]

A query was fast for eight months and became slow overnight. Nothing was deployed. Give five causes and how you would confirm each.

SOLUTION

Cause	Why it happens	How to confirm
Stale statistics	the table grew or its distribution shifted; the planner's row estimates are now wrong and it flipped to a bad plan	<code>EXPLAIN ANALYZE</code> : estimated rows versus actual rows off by 100×. Fix: <code>ANALYZE tablename</code> ;
Crossed the cost threshold	the table grew past the point where the index stopped paying (Example 11) and the planner correctly switched to a sequential scan	the plan changed from <code>Index Scan</code> to <code>Seq Scan</code> and the row count really did grow
Data skew appeared	a single customer now owns 40% of the rows; the average-based estimate is meaningless for that customer's parameter	run the query with the hot parameter and with a normal one and compare <code>EXPLAIN ANALYZE</code>
Index bloat	many updates left dead entries; the index is now three times its useful size and no longer fits in the buffer pool	<code>pg_relation_size</code> on the index versus its expected size; fix with <code>REINDEX CONCURRENTLY</code>
Cache eviction	a new nightly job reads a huge table and evicts the hot index pages, so reads that used to be memory hits are now disk reads	the buffer-cache hit ratio dropped at exactly that hour; the plan is unchanged but timings tripled

The first thing to run, always:

estimate versus reality

SQL

```
EXPLAIN (ANALYZE, BUFFERS) SELECT ... ;
-- look for:  rows=12 ... actual rows=418203      <- statistics are wrong
--           Buffers: shared read=91223          <- it went to disk
--           Rows Removed by Filter: 39000000     <- read far more than returned
```

Answer: stale statistics, a real crossover, new skew, index bloat, or cache eviction — and `EXPLAIN (ANALYZE, BUFFERS)` distinguishes them in one command

SHORTCUT

Rows Removed by Filter is the most under-read number in a plan. A node that returns 40 rows after removing 3.9 million did almost all of its work for nothing — that is exactly where a better index belongs.

Example 19 [Agoda · pattern]

The planner estimates 1 row and the query actually returns 400,000. Explain how the estimate was computed and why it was wrong.

SOLUTION

How the planner estimates. It keeps per-column statistics: the number of distinct values, the most common values with their frequencies, and a histogram of the rest. Then:

1. For an equality predicate with a histogram entry, use that entry's frequency.
2. For one without, assume the remaining values are **uniform**: selectivity = 1/distinct values.
3. For AND of two predicates, assume **independence** and multiply.

the estimator, run with node

JS

```
const stats = { rows: 2_000_000, distinctCity: 400, distinctStatus: 3,
  statusHist: { PENDING: 0.002, SHIPPED: 0.118, DELIVERED: 0.880 } };

const flat = (col) => 1 / stats["distinct" + col];    // no histogram: assume
uniform

console.log("uniform guess for status =", flat("Status").toFixed(4),
  "-> rows", Math.round(stats.rows * flat("Status")));    // 0.3333,
666667
for (const [v, s] of Object.entries(stats.statusHist))
  console.log(" histogram status =", v, s, "-> est rows", stats.rows * s);
    // PENDING 4000 | SHIPPED 236000 | DELIVERED 1760000

const combined = flat("City") * stats.statusHist.PENDING;    // independence
console.log("city AND status='PENDING' -> rows", (stats.rows *
combined).toFixed(2));
    // 5e-6 * 2,000,000 = 10.00
```

Look at the first two lines. Without a histogram the planner guesses 666,667 rows for **every** status value. With a histogram it says 4,000 for PENDING and 1,760,000 for DELIVERED — a 440× difference the uniform model cannot see. This is why ANALYZE matters.

Why the 1-row estimate happened. The independence assumption. Suppose the predicate is WHERE city = 'Pune' AND pincode = '411001'. The planner computes

$$\frac{1}{400} \times \frac{1}{20000} = 1.25 \times 10^{-7} \Rightarrow 2000000 \times 1.25 \times 10^{-7} = 0.25 \rightarrow \text{rounded to } 1$$

But city and pincode are not independent — a pincode **determines** the city. The true selectivity is just the pincode's, 1/20000, giving 100 rows; and if that pincode is a busy one, 400,000.

The consequences and the fixes. The 1-row estimate makes the planner choose a nested loop, expecting one lookup. It performs 400,000 random lookups instead. The plan is not slightly wrong; it is thousands of times wrong.

1. ANALYZE the table — the cheapest fix, and often enough.
2. Raise the histogram resolution on the skewed column: ALTER TABLE t ALTER COLUMN status SET STATISTICS 1000; then ANALYZE t;
3. Tell the planner about the correlation: CREATE STATISTICS s_city_pin (dependencies, ndistinct) ON city, pincode FROM addresses;
4. Restructure so the correlated columns are one column, or one composite index.

Answer: uniform-within-distinct plus independence between predicates; correlated columns break the multiplication and produce estimates that are orders of magnitude too small

Example 20 [DBS · pattern]

A 900-million-row transactions table is queried almost always by date range and account. Queries older than 90 days are rare. Design the physical layout.

SOLUTION

The problem with one giant table. The index on (account_id, txn_date) is $9 \times 10^8 \times 16/0.7 \approx 20$ GB. It will not stay in cache, every lookup is a real disk read, and the nightly delete of old rows is a multi-hour DELETE that bloats the table.

The answer is partitioning. Split the table by month. Each partition is a separate physical table with its own indexes, and the planner **prunes** the ones that cannot match.

range partitioning by month (PostgreSQL)

SQL

```
CREATE TABLE transactions (
  txn_id      bigint,
  account_id  bigint      NOT NULL,
  txn_date    date        NOT NULL,
  amount      numeric(14,2),
  PRIMARY KEY (txn_id, txn_date)      -- partition key must be in every unique key
) PARTITION BY RANGE (txn_date);

CREATE TABLE transactions_2026_01 PARTITION OF transactions
  FOR VALUES FROM ('2026-01-01') TO ('2026-02-01');
CREATE TABLE transactions_2026_02 PARTITION OF transactions
  FOR VALUES FROM ('2026-02-01') TO ('2026-03-01');

CREATE INDEX ON transactions_2026_01 (account_id, txn_date);
CREATE INDEX ON transactions_2026_02 (account_id, txn_date);
```

What this buys.

Partition pruning	a query with <code>txn_date >= '2026-02-01'</code> touches one partition, so the index it searches is 1/36th the size and stays in memory
Instant archiving	<code>ALTER TABLE transactions DETACH PARTITION transactions_2023_01;</code> is metadata only. Compare that with DELETE on 25 million rows.
Cheaper maintenance	VACUUM, ANALYZE and REINDEX run per partition, so each finishes in minutes
Better locality	rows for one month sit together on disk, so a date-range scan is nearly sequential

The rules you must state, because they are where people get caught:

1. **The partition key must appear in the WHERE clause** or the planner cannot prune, and the query touches all 36 partitions. Every query in the application must filter on txn_date.
2. **Every unique constraint must include the partition key.** You cannot have a globally unique txn_id alone; it becomes (txn_id, txn_date).
3. **Do not over-partition.** Planning time grows with the partition count. Monthly for a few years is fine; hourly for ten years is not.

The secondary idea — a covering index for the hot path. Roughly 95% of the traffic is “last 90 days for one account”, so also keep `CREATE INDEX ON transactions_2026_02 (account_id, txn_date DESC) INCLUDE (amount);` so the statement page is answered index-only.

Answer: range-partition by month, index each partition on (account_id, txn_date), always filter on txn_date so pruning works, and drop old data by detaching partitions

Example 21 [LINE MAN · pattern]

Read this plan and say what is wrong.

EXPLAIN ANALYZE output

SQL

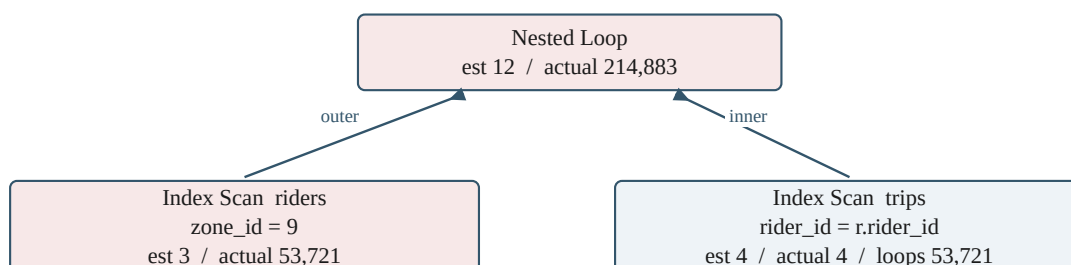
```
Nested Loop (cost=0.86..4821.10 rows=12 width=96)
  (actual time=0.11..9184.22 rows=214883 loops=1)
  -> Index Scan using idx_riders_zone on riders r
    (cost=0.43..18.20 rows=3 width=48)
    (actual time=0.03..14.77 rows=53721 loops=1)
    Index Cond: (zone_id = 9)
  -> Index Scan using idx_trips_rider on trips t
    (cost=0.43..1600.90 rows=4 width=48)
    (actual time=0.10..0.16 rows=4 loops=53721)
    Index Cond: (rider_id = r.rider_id)
Execution Time: 9201.44 ms
```

SOLUTION

Read it bottom-up and compare the two row numbers on every line.

Node	Estimated	Actual	Error
Index Scan on riders	3	53,721	17,907× too low
Index Scan on trips	4 per loop	4 per loop	correct
Nested Loop	12	214,883	17,907× too low

The diagnosis. The inner side is perfect. The **outer** estimate is catastrophically wrong: the planner believed zone 9 had 3 riders, so a nested loop looked cheap — 3 lookups. Zone 9 has 53,721 riders, so the loop ran 53,721 times. `loops=53721` is the smoking gun: `actual` time on an inner node is **per loop**, so the true cost of the inner node is $53721 \times 0.16 \text{ ms} \approx 8600 \text{ ms}$ — essentially the entire query.



The plan tree. The node that lied is the one at the bottom left.

The fixes, in order.

1. `ANALYZE riders;`. If `zone_id` is skewed, also `ALTER TABLE riders ALTER COLUMN zone_id SET STATISTICS 500;` With a correct estimate of 53,721 the planner will choose a **hash join** instead, costing one scan of each side rather than 53,721 index descents.
2. Ask whether the query should return 214,883 rows at all. A join producing a fifth of a million rows for a dashboard usually wants an aggregate or a `LIMIT` pushed down.
3. If the estimate cannot be fixed, a covering index on `trips (rider_id) INCLUDE (...)` at least makes each of the 53,721 loops cheaper.

Answer: the outer row estimate is 17,907× too low, so a nested loop was chosen and ran 53,721 times; fix the statistics so the planner picks a hash join

TRAP

actual time=0.10..0.16 on the inner node looks harmless — a sixth of a millisecond. It is **per loop**. Always multiply by loops before you decide a node is cheap. This one line is where most people misread a plan.

Example 22 [Razer · pattern]

An UPDATE-heavy table has become slow even though no index definition changed. `pg_class` says the index is 11 GB but its data should be 2 GB. Explain and fix without downtime.

SOLUTION

Why an index bloats. Under MVCC an UPDATE writes a **new row version**. If any indexed column changed, a new index entry must be written too, and the old entry stays until vacuum removes it. Vacuum frees the **space inside** a page for reuse, but it does not give pages back to the operating system and it cannot merge half-empty index pages. After millions of updates the tree is a sparse skeleton: 11 GB of pages holding 2 GB of live entries. The damage is not the disk — it is the **buffer pool**. A 2 GB index fits in RAM; an 11 GB one does not, so lookups that were memory hits become disk reads.

How to confirm.

measure before you rebuild

SQL

```
SELECT indexrelname,
       pg_size_pretty(pg_relation_size(indexrelid)) AS size,
       idx_scan
FROM   pg_stat_user_indexes
WHERE  relname = 'sessions'
ORDER BY pg_relation_size(indexrelid) DESC;
```

Fix without downtime.

rebuild online

SQL

```
REINDEX INDEX CONCURRENTLY idx_sessions_user; -- PostgreSQL 12+, no long write lock
```

CONCURRENTLY builds a second copy while writes continue, then swaps. It is slower and needs room for both copies, but it never blocks the application. Plain REINDEX takes an exclusive lock and will stop your service.

Stop it coming back.

1. Make autovacuum more aggressive on that table: `ALTER TABLE sessions SET (autovacuum_vacuum_scale_factor = 0.02);`
2. Avoid updating indexed columns. In PostgreSQL, if **no indexed column changes** and the page has free space, the engine can do a **HOT update** — a new row version on the same page with **no** index entry at all. Leaving `last_seen_at` unindexed can remove almost all the index churn on a session table.
3. Give pages room to work with a lower fill factor: `ALTER TABLE sessions SET (fillfactor = 80);`
4. Hunt for long-running transactions. A transaction open for hours pins the oldest snapshot, so vacuum can remove nothing and bloat grows without limit.

Answer: dead index entries from UPDATES that vacuum cannot compact; REINDEX CONCURRENTLY, then prevent it with aggressive autovacuum, HOT updates and a lower fill factor

PRACTICE · TIER 2 target: 30 minutes

1. 40 million rows, one category holds 120,000. Is `category_id` selective enough to lead an index? Give the number.

2. An EXPLAIN ANALYZE node shows `actual time=0.20..0.30 loops=80000`. What is its real cost?
3. Give two reasons a partial index beats a full index on a boolean column.
4. Why must the partition key appear in every unique constraint of a partitioned table?
5. A plan shows `Rows Removed by Filter: 2900000` above an index scan that returned 50 rows. What index would you build?
6. Name the PostgreSQL feature that avoids writing index entries on an UPDATE, and its precondition.
7. Two predicates each keep 1% of the table. The planner estimates 0.01% together. When is that wrong?

ANSWER KEY

1. $120000/40000000 = 0.003$, that is 0.3% — well inside the crossover, so yes.
2. Up to $80000 \times 0.30 \text{ ms} = 24 \text{ seconds}$.
3. It indexes only the rare value, so it is tiny and stays cached; and it is not maintained for rows that do not match the predicate, so writes to the common case are cheaper.
4. Uniqueness is enforced per partition by a local index; without the partition key in the constraint the engine cannot prove global uniqueness.
5. One that includes the filtered column, so the filter becomes part of the index condition — for example `(status, created_at)` instead of `(status)`.
6. HOT update; it needs **no indexed column to change** and free space on the same heap page.
7. Whenever the two columns are correlated — if one determines the other, the true selectivity is 1%, not 0.01%, and the estimate is 100× too low.

Tier 3 — the hard version

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 23 [Google · pattern]

Derive the cost of an index range scan from first principles, including the clustering factor, and use it to explain why the same index is excellent on one table and useless on another holding identical data.

SOLUTION

Setup. N rows, B rows per page, so $P = \lceil N/B \rceil$ table pages. A predicate matches m rows. The index has height h and fanout f , so the m matching entries occupy about $\lceil m/f \rceil$ leaf pages.

Part 1 — reading the index.

$$C_{\text{index}} = h + \left\lceil \frac{m}{f} \right\rceil - 1$$

Descend once, then walk the leaf chain. With $f = 512$ and $m = 20000$ that is $3 + 40 - 1 = 42$ page reads. The index part is almost never the problem.

Part 2 — fetching the rows. This is where it is decided. Define the **clustering factor** c : the number of **distinct table pages** the m matching rows live on.

Two extremes:

- **Perfectly clustered** — the table is physically stored in index order. The m rows sit together: $c = \lceil m/B \rceil$, and the reads are nearly sequential.
- **Perfectly scattered** — every matching row is on a different page: $c = \min(m, P)$, and every read is random.

$$C_{\text{total}} = \underbrace{h + \left\lceil \frac{m}{f} \right\rceil - 1}_{\text{index}} + \underbrace{c \times r}_{\text{table}}$$

where r is the random-to-sequential cost ratio ($r \approx 1$ when the reads are sequential).

Part 3 — put numbers in. Our table: $N = 10^7$, $B = 64$, $P = 156250$, $h = 3$, $f = 512$, $r = 4$. Let $m = 100000$ (1% of the table).

Table	Clustering factor c	Total cost	Versus scan (156,250)
clustered on this key	$\lceil 100000/64 \rceil = 1563$	$3 + 195 + 1563 \times 1 = 1761$	89× better
scattered	$\min(100000, 156250) = 100000$	$3 + 195 + 100000 \times 4 = 400198$	2.6× worse

The same index, the same data, the same query — and the answer flips from “89 times faster” to “2.6 times slower” purely because of physical row order.

Part 4 — what this means in practice.

1. This is why EXPLAIN on your laptop, where you loaded the data in sorted order, does not predict production, where rows arrived interleaved.
2. It is why a time-series table is nearly free to query by time (rows arrive in time order, so it is naturally clustered) and expensive to query by user_id (one user’s rows are spread across the whole file).
3. The fixes: CLUSTER t USING idx in PostgreSQL (a one-off physical reorder, not maintained); choosing the clustered primary key in InnoDB to match the dominant access pattern; making the index **covering** so the clustering factor becomes irrelevant — you never touch the table at all.

Part 5 — the InnoDB twist. In InnoDB a secondary index leaf holds the **primary key**, not a physical address. So the fetch is a second B+ tree descent, not a single page read:

$$C_{\text{total}} = h_{\text{sec}} + \left\lceil \frac{m}{f} \right\rceil - 1 + m \times h_{\text{pk}}$$

Each fetch costs $h_{\text{pk}} \approx 3$ logical reads rather than 1 — usually cached, but it is why InnoDB benefits even more from covering indexes, and why a wide primary key hurts twice (once in every secondary index’s size, once in every secondary lookup).

Answer: $C = h + \lceil m/f \rceil - 1 + cr$; the clustering factor c ranges from $\lceil m/B \rceil$ to m , a factor of B , and it alone decides whether the index is a 90× win or a loss

Example 24 [Amazon · pattern]

Design the indexes for an order-history API that must serve p99 under 20 ms on a 4-billion-row table, given these three access patterns. Justify every index and state what it costs.

the three queries

SQL

```
-- Q1: the customer's recent orders (92% of traffic)
SELECT order_id, placed_at, status, total
FROM orders WHERE customer_id = ? ORDER BY placed_at DESC LIMIT 20;

-- Q2: one order by its public reference (7%)
SELECT * FROM orders WHERE order_ref = ?;

-- Q3: operations dashboard – stuck orders (1%, but must never be slow)
SELECT order_id, customer_id, placed_at
FROM orders
WHERE status = 'PAYMENT_PENDING' AND placed_at < now() - INTERVAL '1 hour';
```

SOLUTION

First — the physical layout, before any index. 4 billion rows will not sit in one table that anyone can vacuum or reindex. Partition by placed_at, monthly. That also means every query must carry a date bound or it fans out across all partitions — which Q1 does not, so we must handle that.

Q1 — 92% of traffic, and it defines the design.

the index for Q1

SQL

```
CREATE INDEX idx_o_cust_time ON orders (customer_id, placed_at DESC)
    INCLUDE (order_id, status, total);
```

- `customer_id` leads: it is the equality predicate, and it is extremely selective (4 billion rows / 80 million customers = 50 orders per customer).
- `placed_at DESC` second: the index is now **already in the required order** inside one customer, so `LIMIT 20` reads exactly 20 entries. No sort node at all.
- `INCLUDE` the other three columns: an index-only scan, so zero heap reads.

Cost: 3 to 4 page reads, all likely cached. Well under 1 ms. The `INCLUDE` columns make the index perhaps 60 bytes per entry — around 340 GB across all partitions. That is the price, and for 92% of traffic it is worth it.

The partition-pruning problem with Q1. Q1 has no date predicate, so it would search every monthly partition. Two answers:

1. Change the API contract to a rolling window: “recent orders” means the last 24 months, so the query carries `AND placed_at > now() - INTERVAL '24 months'` and prunes to 24 partitions.
2. Better: partition by `HASH(customer_id)` instead of by date. Then Q1 prunes to exactly one partition. Archiving gets harder, which is the trade-off. For a customer-facing API where Q1 is 92% of traffic, hash-partitioning on `customer_id` is the right call.

Q2 — a unique lookup.

the index for Q2

SQL

```
CREATE UNIQUE INDEX idx_o_ref ON orders (order_ref);
```

Two things to say:

1. It must be `UNIQUE` so it also **enforces** the invariant, not merely accelerates it. An index that is really a constraint should be declared as one.
2. Under hash partitioning on `customer_id`, a **global** unique index across partitions is not available in PostgreSQL. So either put `order_ref` in the partition key, or accept a per-partition unique index plus an application-level generator (a UUIDv7 or a Snowflake id) that is unique by construction.

Q3 — 1% of traffic, but it must not degrade.

the index for Q3

SQL

```
CREATE INDEX idx_o_stuck ON orders (placed_at)
    WHERE status = 'PAYMENT_PENDING';
```

A **partial** index. At any instant maybe 3,000 orders are payment-pending out of 4 billion, so this index is a few hundred kilobytes — permanently cached, and almost free to maintain because 99.99% of rows never satisfy the predicate. A full index on `(status, placed_at)` would have been 200 GB to answer a query about 3,000 rows.

There is a subtlety: rows leave the predicate when payment completes, which is a **delete** from the partial index. That is exactly what you want — the index shrinks back on its own.

What all of this costs on the write path.

Clustered insert (the table itself)	1 tree insertion
<code>idx_o_cust_time</code>	1 tree insertion, 60-byte entry
<code>idx_o_ref</code>	1 tree insertion, plus a uniqueness check
<code>idx_o_stuck</code>	1 insertion only for payment-pending rows, and a later delete

About 3.2 tree insertions per order. At 5,000 orders per second that is 16,000 index writes per second — fine on an SSD, and the reason we did **not** add a fourth index “just in case”.

What was deliberately not built.

- No index on `status` alone: three distinct values, no selectivity, and Q3 is already covered by the partial index.
- No index on `total` or on `(customer_id, status)`: no query in the list needs them, and every unused index is pure write cost (Example 8).

Answer: a covering (`customer_id, placed_at DESC`) index with hash partitioning on `customer_id` for Q1; a unique index on `order_ref` for Q2; a tiny partial index for Q3 — three indexes, about 3.2 tree writes per insert

Example 25 [Microsoft · pattern]

Compare a B+ tree with an LSM tree as the storage structure for an index. Give the read, write and space amplification of each and say which database chooses which.

SOLUTION

The three amplifications. Every storage engine trades these against each other.

Write amplification	bytes written to the device / bytes the application logically wrote
Read amplification	device reads performed / logical reads requested
Space amplification	bytes stored on the device / bytes of live data

B+ tree. An update finds the leaf and rewrites it in place.

- **Write:** to change 100 bytes the engine must write a whole 8 KB page, plus the WAL record. Amplification around 80× for small random updates — and worse when a page splits.
- **Read:** h page reads, typically 1 real read after caching. Amplification near 1. **This is the B+ tree’s strength.**
- **Space:** $\frac{1}{\text{fill factor}}$, so about 1.4×, plus bloat between vacuums.

LSM tree. Writes land in an in-memory table; when it fills, it is flushed as an immutable sorted file (an SSTable). Background **compaction** merges files into bigger levels.

- **Write:** the initial flush is one sequential write — very cheap. But each byte is rewritten once per level it is compacted through: with 7 levels and a size ratio of 10, amplification is roughly 10 to 30×. **Lower than a B+ tree for random writes, because every write is sequential**, and sequential bandwidth is 100× random IOPS on a spinning disk and still several times higher on an SSD.
- **Read:** a key may be in the memtable or in any level. Without help, that is one lookup per level. Bloom filters cut it to roughly 1 real read for a point lookup that exists, but a **range scan** must merge across all levels, so amplification stays higher than a B+ tree’s.
- **Space:** obsolete versions live until compaction removes them — typically 1.1× to 2×, with spikes during compaction.

	B+ tree	LSM tree
Random write	slow — read-modify-write a page	fast — append only
Point read	1 I/O	1 I/O with bloom filters
Range scan	excellent — linked leaves	merge across levels
Write amplification	high for small updates	lower, and sequential
Space amplification	1.4× + bloat	1.1–2×, spiky
Latency shape	steady	spiky — compaction stalls
Used by	PostgreSQL, InnoDB, Oracle, SQL Server	RocksDB, LevelDB, Cassandra, HBase, ScyllaDB

How to choose.

- Write-heavy, key-value or wide-column, tolerant of p99 spikes → LSM.
- Read-heavy or range-scan-heavy, needs predictable latency and full SQL → B+ tree.

- MySQL offers both: InnoDB (B+ tree) and MyRocks (LSM). Facebook moved parts of its MySQL fleet to MyRocks for roughly half the space and much lower write amplification on flash.

The honest caveat. The classic “LSM writes are cheap” claim compares the **user-visible** write. Compaction does the work later, in the background, and it competes for the same device bandwidth. LSM does not remove write cost; it **defers and batches** it, converting random writes into sequential ones. That is the real win, and saying it that way shows you understand the mechanism rather than the slogan.

Answer: B+ tree: read amplification 1, high write amplification, steady latency. LSM: sequential writes and lower write amplification, higher read and range amplification, spiky latency from compaction.

Example 26 [Goldman Sachs · pattern]

A report joins four tables and takes 90 seconds. The planner’s join order is clearly wrong. Explain how a cost-based optimiser chooses a join order, why it goes wrong here, and what you can do.

SOLUTION

How the optimiser works.

- Enumerate.** For n tables there are $\frac{(2(n-1))!}{(n-1)!}$ join orders — 12 for 3 tables, 120 for 4, and 17,297,280 for 8. A System-R style optimiser uses dynamic programming over **subsets**: it computes the best plan for every subset of tables, building up from pairs. That is $O(3^n)$ instead of factorial — feasible to about 10 to 12 tables.
- Cost.** For each candidate it estimates the output cardinality of the join and the cost of each physical algorithm (nested loop, hash, merge).
- Prune.** Keep only the cheapest plan per subset — plus the cheapest plan per **interesting sort order**, because a plan that is already sorted on the next join key may win later even though it is more expensive now.
- Beyond roughly 12 tables, the optimiser gives up on exhaustive search and uses a greedy or genetic algorithm (geqo in PostgreSQL).

Why it goes wrong. Join order is chosen entirely from **estimated cardinalities**, and estimation error **compounds multiplicatively** up the tree. If each of three joins is estimated 10× too low, the top of the tree is 1000× off. The optimiser is not choosing badly — it is choosing correctly from wrong numbers.

The three usual sources:

- Correlated predicates (Example 19) — independence assumption multiplies when it should not.
- Join cardinality on non-key columns — estimated as

$$|R| \times |S| \frac{1}{\max(\text{distinct}_R, \text{distinct}_S)}$$

which is badly wrong when the join column’s distribution is skewed.

- A subquery, function or LIKE the planner cannot see through, for which it substitutes a fixed default guess.

What to do, in the order you should try it.

1. Fix the inputs	ANALYZE every table. Raise SET STATISTICS on the skewed join and filter columns. Create extended statistics for correlated pairs.
2. Give the planner room	raise join_collapse_limit and from_collapse_limit so it actually searches, and work_mem so a hash join is allowed to stay in memory.
3. Reduce the problem	materialise a selective part into a CTE or temporary table, ANALYZE it, and join against that. Now the planner has measured cardinalities, not estimates.
4. Force the order	last resort. SET join_collapse_limit = 1 makes PostgreSQL join in written order; MySQL has STRAIGHT_JOIN; SQL Server and Oracle have hints. This freezes the plan against future data changes — you are taking on maintenance forever.

step 3 in practice — the usual real fix

SQL

```
-- 90 s version: four-way join, planner guesses at every step
-- fixed version: reduce first, then join against a small measured set
```

```

CREATE TEMP TABLE hot_accounts AS
SELECT account_id
  FROM transactions
 WHERE txn_date >= DATE '2026-03-01'
 GROUP BY account_id
HAVING sum(amount) > 1000000;           -- a few thousand rows

ANALYZE hot_accounts;                  -- now the planner KNOWS the size

SELECT ...
  FROM hot_accounts h
 JOIN accounts a ON a.account_id = h.account_id
 JOIN customers c ON c.customer_id = a.customer_id
 JOIN regions   r ON r.region_id   = c.region_id;

```

Answer: dynamic programming over subsets, costed from estimated cardinalities that compound multiplicatively; fix the statistics first, materialise a measured intermediate second, and force the order only as a last resort

Example 27 [Adobe · pattern]

You must add an index to a 600 GB table in production. Writes must not stop. Describe exactly what happens, what can go wrong, and how you would verify the result.

SOLUTION

The naive command is a trap. `CREATE INDEX idx ON big_table (col);` takes an ACCESS EXCLUSIVE-equivalent lock in PostgreSQL — it blocks every INSERT, UPDATE and DELETE on the table for the entire build. On 600 GB that is hours. Your application is down.

The right command.

```
CREATE INDEX CONCURRENTLY idx_big_col ON big_table (col);
```

What it actually does — two table scans and a wait.

1. **Pass 1.** Take a snapshot, scan the whole table, build the index from the rows visible in that snapshot. Writes continue throughout.
2. **Wait.** Wait for every transaction that started before pass 1 to finish, so that no writer is still unaware of the new index.
3. **Pass 2.** Take a second snapshot and scan again, adding the rows that were inserted or updated during pass 1.
4. **Wait again,** then mark the index valid and usable by the planner.

What can go wrong — say all of these, because this is the real content of the question.

It can fail and leave a mess	If it fails, the index remains in the catalogue marked <code>indisvalid = false</code> . It is not used by queries but is maintained on every write — the worst of both worlds. Find it with <code>SELECT indexrelid::regclass FROM pg_index WHERE NOT indisvalid;</code> and <code>DROP INDEX</code> it.
It is blocked by long transactions	Both waits block behind the oldest running transaction. One analyst's session left open makes the build hang for hours while still holding resources.
It needs a lot of space	the new index plus temporary sort space. Check free disk first; a 600 GB table's index may be 40 GB and the sort may need as much again.
It is slow and I/O heavy	two full scans plus a sort. It will compete with production traffic for cache and bandwidth. Run it in a low-traffic window and consider lowering the build's I/O priority.

Not allowed inside a transaction	CREATE INDEX CONCURRENTLY cannot run in a transaction block, so most migration frameworks need it marked as non-transactional.
Unique indexes can fail late	CREATE UNIQUE INDEX CONCURRENTLY fails at the end if a duplicate exists — after hours of work. Check for duplicates first.

The procedure I would actually follow.

1. Check for duplicates and for long-running transactions; kill or wait out anything older than a few minutes.
2. Confirm free disk space is at least 3× the expected index size.
3. Run CREATE INDEX CONCURRENTLY in a maintenance window, monitoring pg_stat_progress_create_index.
4. Verify validity: SELECT indisvalid FROM pg_index WHERE indexrelid = 'idx_big_col'::regclass;
5. ANALYZE big_table; — the planner will not use a new index well until statistics are fresh.
6. Verify usefulness, not just existence: run the target query with EXPLAIN (ANALYZE, BUFFERS) and confirm the plan changed and shared read fell.
7. Check idx_scan in pg_stat_user_indexes a day later. If it is still 0, the index is pure cost and should be dropped.

MySQL note. InnoDB’s online DDL does the equivalent with ALTER TABLE ... ADD INDEX ..., ALGORITHM=INPLACE, LOCK=NONE; — it buffers concurrent changes in an online log and applies them at the end. The failure mode is that the log fills (innodb_online_alter_log_max_size) and the whole ALTER fails after hours.

Answer: CREATE INDEX CONCURRENTLY: two scans and two waits, no write lock; watch for invalid leftovers, long transactions, disk space and late unique violations; verify with indisvalid, a fresh ANALYZE, a re-run EXPLAIN ANALYZE, and idx_scan the next day

Example 28 [Uber · pattern]

SELECT count(*) FROM events; takes 40 seconds on a 900-million-row table. Explain why, and give three designs with their exact trade-offs.

SOLUTION

Why it is slow. In PostgreSQL, MVCC means **no single place records how many rows are visible to you.** Different transactions see different counts, so count(*) must examine every row and test its visibility. That is a full scan of all 900 million rows — about $\lceil 9 \times 10^8 / 64 \rceil = 14062500$ pages, roughly 115 GB of I/O.

An index does not help by itself, but an **index-only scan** over the smallest index does reduce the bytes read — it still visits every entry, but the entries are far smaller than the rows. That is why count(*) sometimes uses an index and is still slow.

Design 1 — an approximate count.

milliseconds, accurate to a few percent

SQL

```
SELECT reltuples::bigint AS approx_rows
FROM pg_class
WHERE oid = 'events'::regclass;
```

Cost	one catalogue row read — under a millisecond
Accuracy	as of the last ANALYZE or autovacuum; typically within 1–5%
Use when	a dashboard shows “about 900 million events”. Nobody reads the last six digits.

Design 2 — a maintained counter. A trigger or an application-level increment keeps a running total.

exact, but it creates a hot row

SQL

```
CREATE TABLE event_counts (bucket int PRIMARY KEY, n bigint NOT NULL DEFAULT 0);
INSERT INTO event_counts (bucket) SELECT generate_series(0, 99); -- 100 buckets

-- INSERT path: pick a random bucket so the counter is not one hot row
-- floor(), not a cast: (random() * 100)::int rounds, so it can produce 100
UPDATE event_counts SET n = n + 1 WHERE bucket = floor(random() * 100)::int;

-- read path:
SELECT sum(n) FROM event_counts;
```

Cost	one extra row update per insert; the read is 100 rows
Accuracy	exact
Danger	without bucketing, every insert contends on one row and the insert rate collapses (see the hot-row analysis in Chapter 7). With bucketing you pay a small write cost on every insert forever.
Also	a DELETE path must decrement, or the count drifts. Deletes are the usual source of bugs here.

Design 3 — count only what is asked for. Almost no real product needs the count of the whole table. It needs “events for this user this month” or “is there more than one page of results”.

a bounded count, and a pre-aggregated rollup

SQL

```
-- "are there more than 500?" – stops early, never scans the table
SELECT count(*) FROM (SELECT 1 FROM events WHERE user_id = 42 LIMIT 501) s;

-- a rollup table maintained by a nightly or streaming job
CREATE TABLE event_daily (day date, user_id bigint, n bigint,
                           PRIMARY KEY (day, user_id));
SELECT sum(n) FROM event_daily WHERE user_id = 42 AND day >= DATE '2026-03-01';
```

Cost	an index seek, or a few rollup rows
Accuracy	exact for the bounded count; as fresh as the job for the rollup
Trade-off	you must define the granularity in advance; a new question needs a new rollup

How to choose, said as a rule: ask what the number is **for**. If it is displayed, approximate it. If it drives a decision, bound it (LIMIT 501). If it is a business metric, pre-aggregate it. An exact live count (*) of a 900-million-row table is almost always a requirement nobody actually had.

The InnoDB footnote. MySQL’s count (*) on InnoDB is also a scan, for the same MVCC reason. MyISAM kept an exact row count in its header and answered instantly — but it had no transactions, which is exactly the trade being made.

Answer: MVCC means visibility is per-transaction, so an exact count must scan every row; use reltuples for display, a bucketed counter for exactness, or a bounded/rolled-up count for the question actually being asked

PRACTICE · TIER 3 target: 40 minutes

1. Write the full cost formula for an index range scan and name every symbol.
2. A clustering factor equals the number of matching rows. What does that tell you about the table?
3. Explain why InnoDB benefits more from covering indexes than PostgreSQL does.
4. Give the write, read and space amplification of a B+ tree and of an LSM tree, in one line each.
5. Why does cardinality estimation error compound multiplicatively up a join tree?
6. CREATE INDEX CONCURRENTLY failed overnight. What is now true of your database, and what must you do?

7. A query needs an exact count of a 900-million-row table every 5 seconds. Push back on the requirement — what would you ask?
8. You have (a, b) and (a, b, c) and (a). Which would you drop and why?

ANSWER KEY

1. $C = h + \lceil m/f \rceil - 1 + cr$: h height, m matched rows, f entries per leaf, c clustering factor (distinct table pages touched), r random-to-sequential cost ratio.
2. The matching rows are perfectly scattered — one per page — so the table's physical order has no relationship to this index.
3. In InnoDB the row fetch is a second full B+ tree descent through the clustered index, not a single page read, so avoiding it saves more.
4. B+ tree: write high (a whole page per small change), read 1, space 1.4× plus bloat. LSM: write lower and sequential but repeated per compaction level, read higher (several levels, cut by bloom filters), space 1.1–2× and spiky.
5. The output cardinality of one join is the input to the next, so each level multiplies its own error by the error already carried in.
6. There is an invalid index that queries ignore but every write still maintains. Find it with `pg_index WHERE NOT indisvalid`, drop it, then retry the build.
7. “What decision does the number drive, and what error is acceptable?” — then offer `reltuples` for display, a bucketed counter for exactness, or a rollout.
8. Drop (a) and (a, b). (a, b, c) already serves every leftmost prefix, so both are pure write cost — unless (a, b) is far smaller and is used for an index-only scan on a hot path.

Rapid-fire — one-line answers

Cover the right column. These thirty cover most of what a technical round asks on indexing.

Question	Answer
What is an index, in one sentence?	A sorted copy of some columns plus a pointer to the row.
Why B+ tree and not binary search tree?	Huge fanout means 3–4 levels instead of 30, and the unit of I/O is a page, not a node.
Where does a B+ tree store the data?	Only in the leaves; internal nodes hold separator keys.
Why are B+ tree leaves linked?	So a range scan descends once and then walks sideways.
Typical height for 10 million rows?	3 levels, so 3 page reads to a leaf.
How many clustered indexes per table?	One — the rows can be in only one physical order.
What does an InnoDB secondary index leaf hold?	The indexed key plus the primary key , so a lookup is two descents.
Define selectivity.	Matching rows divided by total rows. Smaller is better for an index.
When is a sequential scan cheaper?	Past roughly 0.4–5% of the table, depending on the random cost ratio and the clustering factor.
What is the clustering factor?	How many distinct table pages the matching rows sit on.
State the leftmost prefix rule.	An index on (a, b, c) serves a, (a, b) or (a, b, c) — never b alone.
Index column order rule?	Equality, then Range, then Order-by. E-R-O.
What happens after the first range column?	Later columns can be filtered but not sought, and their sort order is lost.
What is a covering index?	It holds every column the query needs, so the table is never read.

What is a partial index?	An index with a <code>WHERE</code> clause — only matching rows are indexed.
Define sargable.	The indexed column appears bare on one side of the comparison.
Name four index killers.	Function on the column, leading wildcard, implicit cast, <code>OR</code> across columns.
Why is <code>LIKE '%abc'</code> unindexable?	The index is sorted left to right; a known suffix is scattered everywhere.
<code>EXPLAIN</code> versus <code>EXPLAIN ANALYZE</code> ?	Estimates versus actually running it and reporting the truth.
First thing to check in a plan?	Estimated <code>rows</code> against actual <code>rows</code> .
What does <code>loops=N</code> mean?	The node ran <code>N</code> times; multiply its <code>actual time</code> by <code>N</code> .
What does <code>Rows Removed by Filter</code> tell you?	Work done for nothing — usually a missing index column.
Three join algorithms?	Nested loop, hash join, sort-merge join.
When does nested loop win?	Tiny outer side with an indexed inner side — the OLTP case.
When does hash join win?	Both sides large, equality join, smaller side fits in <code>work_mem</code> .
Why did a fast query suddenly get slow?	Stale statistics, a crossed cost threshold, new skew, index bloat, or cache eviction.
Replacement for <code>OFFSET</code> pagination?	Keyset pagination with a row-value comparison on a unique sort key.
Cost of each extra index on writes?	One more B+ tree insertion per row, plus possible page splits.
How do you add an index without downtime?	<code>CREATE INDEX CONCURRENTLY</code> — two scans, no write lock, cannot run in a transaction.
Hash index versus B+ tree?	Hash is $O(1)$ for equality only; B+ tree also does ranges, prefixes, sorts, <code>MIN</code> and <code>MAX</code> .

ONE-PAGE REVISION CARD

The one sentence. An index is a sorted copy of some columns plus a pointer to the row. Sorted → fast search. Copy → slow writes. Sorted **this** way → helps only **these** queries.

The numbers to carry into the room

Page	8 KB (PostgreSQL), 16 KB (InnoDB)
Rows per page (128-byte row)	64
10 million rows	156,250 pages, 1.2 GB
Fanout (16-byte entry)	512
Tree height for 10 million keys	3
Lookup cost	3 reads to the leaf, +1 for the row
Sequential-scan crossover	about 0.4% of the table at $r = 4$

Formulas

$$f = \left\lfloor \frac{\text{page} \times \text{fill}}{\text{entry}} \right\rfloor \quad h = \lceil \log_f n \rceil \quad C = h + \left\lceil \frac{m}{f} \right\rceil - 1 + c \cdot r$$

$$\text{selectivity} = \frac{m}{N} \quad \text{cost}_{\text{scan}} = \left\lceil \frac{N}{B} \right\rceil$$

Design rules

1. E-R-O: Equality columns, then the Range column, then the ORDER BY columns.
2. One index on (a, b, c) already covers (a) and (a, b). Build the widest, not all three.
3. INCLUDE the columns you only SELECT; put in the key the ones you filter or sort on.
4. Low-cardinality flag → partial index, not a key column.
5. Keep the indexed column bare on the left. No function, no cast, no leading %.
6. A LIMIT with an ORDER BY can often beat a selective filter: walk the sort order and stop early instead of matching then sorting.

Reading a plan, in order

1. Estimated rows versus actual rows. Off by 100×? Fix statistics first.
2. loops — multiply before judging an inner node cheap.
3. Rows Removed by Filter — work done for nothing.
4. Buffers: shared read — how much really came from disk.

The five traps

1. YEAR(col) = 2026 kills the index. Rewrite as a range.
2. A number compared to a VARCHAR column is an invisible cast that kills the index.
3. actual time on an inner node is **per loop**.
4. The “5% rule” is a memory aid; the clustering factor can move the crossover by 60×.
5. Every unused index costs writes, memory and space, and returns nothing. Measure idx_scan.

Networks: Layers, IP & Routing

Where the packet is, who rewrites it, and how to do subnet arithmetic in your head

What this round actually asks

Networking in a technical round is two things glued together.

Half	What it is	How it is scored
Vocabulary	layers, PDU names, device per layer, protocol per layer, DNS, NAT, ARP	instant. You know it or you do not. 10–20 seconds per question.
Arithmetic	subnet mask, network address, broadcast, usable hosts, VLSM, route lookup	you must show the steps. A wrong last octet is a zero.

Service companies (TCS, Infosys, Wipro, Capgemini, Cognizant) ask the vocabulary half rapid-fire and **always** give you one subnetting sum. Product companies ask fewer facts and then push: “why does the router rewrite the MAC but not the IP?”, “what breaks if two networks both use 192.168.1.0/24 and you VPN them together?”

This chapter builds the arithmetic until it is automatic, then the vocabulary hangs off it.

EVERYTHING THIS CHAPTER IS BUILT ON

The two models. OSI has 7 layers and is the exam model. TCP/IP has 4 layers and is the real one.

#	OSI layer	PDU	Address it uses	Lives where
7	Application	data	URL / name	HTTP, DNS, SMTP, FTP, DHCP
6	Presentation	data	–	TLS, encoding, compression
5	Session	data	–	session setup / teardown
4	Transport	segment (TCP) / datagram (UDP)	port number	TCP, UDP
3	Network	packet	IP address	IP, ICMP, routers
2	Data link	frame	MAC address	Ethernet, ARP, switches
1	Physical	bit	–	cable, radio, voltage

Subnetting laws. For a prefix p written as $/p$:

$$\text{addresses in the block} = 2^{32-p}$$

$$\text{usable hosts} = 2^{32-p} - 2 \quad (p \leq 30)$$

$$\text{subnets made by borrowing } b \text{ bits} = 2^b$$

$$\text{block size in the interesting octet} = 256 - \text{that octet of the mask}$$

Network address = the IP with all host bits set to 0.

Broadcast address = the IP with all host bits set to 1.

Usable range = network+1 up to broadcast–1.

Delay on one link.

$$\text{transmission } T_t = \frac{L}{R} \quad (\text{L bits, R bits per second})$$

$$\text{propagation } T_p = \frac{d}{s} \quad (d \text{ metres, } s \text{ about } 2 \times 10^8 \text{ m/s in fibre})$$

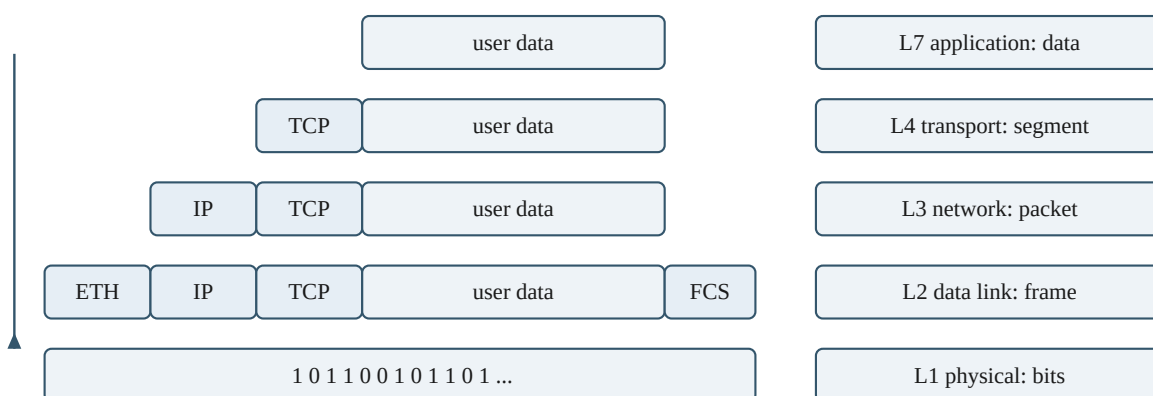
$$\text{total one-way} = T_t + T_p + \text{queuing} + \text{processing}$$

Longest prefix match. A router picks the matching route with the **largest** prefix length. 0.0.0.0/0 matches everything and always loses to anything else.

Part 1 — The layer stack

The picture you must be able to draw

Data goes **down** the stack at the sender. Each layer sticks its own header on the front. At the receiver it goes **up** and each layer peels its own header off. Adding a header is **encapsulation**. Removing it is **decapsulation**.



encapsulation: each layer wraps what the layer above gave it

SHORTCUT

Memorise the stack bottom-up with a sentence you make yourself. One that works: **Please Do Not Throw Sausage Pizza Away**
 = Physical, Data link, Network, Transport, Session, Presentation, Application. Layer 1 is Physical. Layer 7 is Application. If you are asked “what is layer 4”, count up.

TCP/IP: the model that actually runs

TCP/IP layer	Swallows which OSI layers	Protocols
Application	7 + 6 + 5	HTTP, DNS, SMTP, FTP, SSH, DHCP
Transport	4	TCP, UDP
Internet	3	IP, ICMP, ARP ¹
Network access	2 + 1	Ethernet, Wi-Fi, PPP

TRAP

“OSI is used on the internet.” **No.** OSI is a teaching and reference model. The internet runs the TCP/IP model. OSI’s session and presentation layers do not exist as separate pieces in a real stack — TLS and content encoding live inside the application.

¹ARP sits awkwardly. It carries IP addresses but is delivered in a raw Ethernet frame, so different books put it at layer 2 or layer 3. Say: “between 2 and 3 — it maps a layer-3 address to a layer-2 address.” That answer is never marked wrong.

WARM-UP

build the reflex

Example 1 [warm-up]

Name the PDU at layers 4, 3, 2 and 1.

SOLUTION

Layer 4 = segment (TCP) or datagram (UDP). Layer 3 = packet. Layer 2 = frame. Layer 1 = bit.

Answer: segment, packet, frame, bit

Example 2 [warm-up]

Which layer does each device work at: hub, switch, router, NIC?

SOLUTION

Device	Layer	What it looks at
Hub / repeater	1	nothing — copies bits to every other port
Switch / bridge	2	destination MAC address
Router	3	destination IP address
NIC (network card)	1 + 2	puts bits on the wire, owns the MAC

Answer: hub 1, switch 2, router 3, NIC 1–2

Example 3 [warm-up]

A switch has 8 ports, all in one VLAN. How many collision domains and how many broadcast domains? Now answer the same for an 8-port hub.

SOLUTION

Switch. Every port is its own collision domain, because a switch buffers and forwards instead of repeating. So **8 collision domains**. A switch floods broadcasts out of every port, so all 8 ports share **1 broadcast domain**.

Hub. A hub repeats every bit to every port. All 8 ports are one shared wire, so **1 collision domain** and **1 broadcast domain**.

A **router** port is the boundary: a router does not forward broadcasts, so each router interface starts a new broadcast domain.

Answer: switch: 8 collision, 1 broadcast. Hub: 1 collision, 1 broadcast.

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 4 [Infosys · pattern]

A file of 1500 bytes is sent over one 10 Mbps link that is 2000 km long. Signal speed in the fibre is 2×10^8 m/s. Ignore queuing and processing. How long until the last bit arrives?

SOLUTION

Step 1 — length in bits. $L = 1500 \times 8 = 12000$ bits.

Step 2 — transmission delay (time to push the bits out of the card):

$$T_t = \frac{L}{R} = \frac{12000}{10 \times 10^6} = 1.2 \times 10^{-3} \text{ s} = 1.2 \text{ ms}$$

Step 3 — propagation delay (time for a bit to fly down the fibre):

$$T_p = \frac{d}{s} = \frac{2 \times 10^6}{2 \times 10^8} = 10^{-2} \text{ s} = 10 \text{ ms}$$

Step 4 — add them. $1.2 + 10 = 11.2 \text{ ms}$.

Answer: 11.2 ms

SHORTCUT

Transmission delay depends on **how big the file is** and **how fast the link is**. Propagation delay depends on **how far it is** and nothing else. Making the link faster does not shrink propagation delay at all — that is why a ping to another continent never drops below about 100 ms no matter what you pay.

Example 5 [TCS NQT · pattern]

The same 1000-byte packet now crosses **3** links of 2 Mbps each, through 2 store-and-forward routers. Propagation is negligible. Total time?

SOLUTION

Store-and-forward means a router must receive the **whole** packet before it starts sending it on. So the packet is transmitted 3 times, once per link.

$$T_t \text{ per link} = \frac{1000 \times 8}{2 \times 10^6} = \frac{8000}{2 \times 10^6} = 4 \text{ ms}$$

Three links: $3 \times 4 = 12 \text{ ms}$.

Answer: 12 ms

TRAP

Students divide by 3 “because there are 3 links sharing the work”. Wrong direction. Each extra store-and-forward hop **adds** one full transmission time. With n links the answer is $n \times T_t$, not T_t/n .

Part 2 — IPv4 addresses

Dotted decimal is just 32 bits in a costume

An IPv4 address is 32 bits. We print it as 4 decimal numbers (“octets”), each 0–255, separated by dots. Nothing more.

Dotted decimal	The same 32 bits
192.168.10.77	11000000 . 10101000 . 00001010 . 01001101
10.0.0.1	00001010 . 00000000 . 00000000 . 00000001
255.255.255.192	11111111 . 11111111 . 11111111 . 11000000

SHORTCUT

Octet to binary in 5 seconds. Write the place values once: 128 64 32 16 8 4 2 1. Then greedily subtract.

77: is $128 \leq 77$? no $\rightarrow$ 0. $64 \leq 77$? yes $\rightarrow$ 1, left 13. 32 ? no $\rightarrow$ 0. 16 ? no $\rightarrow$ 0. $8 \leq 13$? yes $\rightarrow$ 1, left 5. $4 \leq 5$? yes $\rightarrow$ 1, left 1. 2 ? no $\rightarrow$ 0. 1 ? yes $\rightarrow$ 1, left 0. Result **01001101**. Check: $64 + 8 + 4 + 1 = 77$. Correct.

IPv4 address <-> plain integer (run with node)

JS

```
// IPv4 <-> 32-bit unsigned integer
const toInt = (ip) =>
```

```

ip.split(".").reduce((acc, oct) => acc * 256 + Number(oct), 0);

const toIp = (n) =>
  [24, 16, 8, 0].map((s) => Math.floor(n / 2 ** s) % 256).join(".");

console.log(toInt("192.168.10.77"));
console.log(toIp(3232238157));
console.log(toIp(toInt("10.20.30.40")));
console.log(toInt("255.255.255.255"), toIp(0));

```

output

TEXT

```

3232238157
192.168.10.77
10.20.30.40
4294967295 0.0.0.0

```

Why `2 ** s` and `Math.floor` instead of the bit operators `>>>` and `&`? Because JavaScript bit operators convert to a **signed** 32-bit integer. `toInt("255.255.255.255") | 0` gives `-1`, not `4294967295`. Doing the arithmetic with `*` and `/` keeps every value a safe positive double. This is JS trap number 4 from the language notes: numbers are doubles.

The old classes, and the special addresses

Classes are obsolete in practice (CIDR replaced them in 1993) but they are asked constantly.

Class	First octet	Default prefix	Networks	Hosts per network
A	1 – 126	/8	126	16,777,214
B	128 – 191	/16	16,384	65,534
C	192 – 223	/24	2,097,152	254
D	224 – 239	–	multicast	–
E	240 – 255	–	reserved / experimental	–

SHORTCUT

Read the first octet, top bits first: starts with 0 → A, 10 → B, 110 → C, 1110 → D, 1111 → E. Fast decimal check: under 128 = A, under 192 = B, under 224 = C, under 240 = D, rest = E.

Range	Name	Meaning
10.0.0.0/8	private (class A)	never routed on the public internet
172.16.0.0/12	private (class B)	covers 172.16.x.x to 172.31.x.x
192.168.0.0/16	private (class C)	the home-router range
127.0.0.0/8	loopback	127.0.0.1 = this machine
169.254.0.0/16	link-local / APIPA	self-assigned when DHCP fails
100.64.0.0/10	carrier-grade NAT	used inside ISP networks
224.0.0.0/4	multicast	one sender, a group of receivers
0.0.0.0	unspecified	as a route it means “everywhere”
255.255.255.255	limited broadcast	this link only, never forwarded

TRAP

172.16.0.0/12 is **not** 172.16.0.0 to 172.16.255.255. The /12 means 4 bits of the second octet are fixed, so the range is **172.16.0.0 to 172.31.255.255**. 172.32.5.1 is a public address. Candidates lose this one constantly.

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 6 [Capgemini · pattern]

Classify each address and say whether it is public or private: (a) 172.19.4.8 (b) 172.35.4.8 (c) 192.168.200.1 (d) 127.0.0.5 (e) 169.254.9.9

SOLUTION

Address	Class	Public / private	Why
172.19.4.8	B	private	inside 172.16–172.31
172.35.4.8	B	public	35 is above 31
192.168.200.1	C	private	inside 192.168.0.0/16
127.0.0.5	A (reserved)	neither	loopback, never leaves the host
169.254.9.9	B range	neither	APIPA — DHCP failed

Answer: (a) private B (b) public B (c) private C (d) loopback (e) APIPA / link-local

SHORTCUT

Seeing 169.254.x.x on a machine is a diagnosis, not a configuration. It means **the DHCP server never answered**. Check the cable, the Wi-Fi association, or the DHCP service — not the IP settings.

Part 3 — Subnetting, the method

This is the one calculation you are guaranteed to be given. Learn it as a fixed drill.

The one table to memorise

Prefix	Mask	Block size	Usable hosts	Nickname
/8	255.0.0.0	16,777,216	16,777,214	class A
/16	255.255.0.0	65,536	65,534	class B
/20	255.255.240.0	4,096	4,094	
/22	255.255.252.0	1,024	1,022	
/23	255.255.254.0	512	510	
/24	255.255.255.0	256	254	class C
/25	255.255.255.128	128	126	half a C
/26	255.255.255.192	64	62	quarter
/27	255.255.255.224	32	30	
/28	255.255.255.240	16	14	
/29	255.255.255.248	8	6	
/30	255.255.255.252	4	2	router link

/31	255.255.255.254	2	2 (special)	point-to-point, RFC 3021
/32	255.255.255.255	1	1 (host route)	one exact host

SHORTCUT

You only need to remember the right-hand column of mask values: **128, 192, 224, 240, 248, 252, 254, 255**. Each is the previous one plus the next power of two going down: 128, +64, +32, +16, +8, +4, +2, +1. Block size is always 256 — that value: 128 → 128, 192 → 64, 224 → 32, 240 → 16, 248 → 8, 252 → 4.

The four-step drill

Given any IP/prefix:

Step 1	Find the interesting octet : /1–/8 is octet 1, /9–/16 is octet 2, /17–/24 is octet 3, /25–/32 is octet 4.
Step 2	Block size = 256 — (mask value in that octet). Equivalently $2^{8-(p \bmod 8)}$ when p is not a multiple of 8.
Step 3	Network address: round that octet down to the nearest multiple of the block size. Zero every octet after it.
Step 4	Broadcast = network + block size — 1 in that octet, 255 in every octet after it. Usable range is network+1 to broadcast—1.

WARM-UP

build the reflex

Example 7 [warm-up]

For 192.168.10.77/26 find the mask, network, broadcast, first host, last host and host count.

SOLUTION

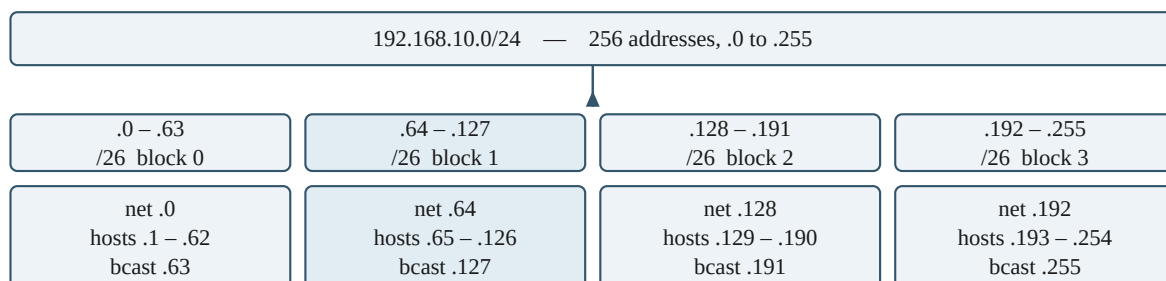
Step 1. /26 is between 25 and 32, so the interesting octet is the **4th** (77).

Step 2. 26 = 24 + 2, so 2 bits are borrowed in octet 4. Mask octet = $256 - 2^{8-2} = 256 - 64 = 192$. Mask = **255.255.255.192**. Block size = **64**.

Step 3. Multiples of 64: 0, 64, 128, 192. Round 77 down → **64**. Network = **192.168.10.64**.

Step 4. Broadcast = 64 + 64 — 1 = 127 → **192.168.10.127**. First host **192.168.10.65**, last host **192.168.10.126**. Hosts = $2^{32-26} - 2 = 64 - 2 = 62$.

Answer: mask 255.255.255.192 • net .64 • bcast .127 • range .65–.126 • 62 hosts



192.168.10.0/24 cut into four /26 blocks of 64 addresses each

The shaded block is where .77 lands. Every subnetting question is “which block, and what are its three edges”.

Example 8 [warm-up]

How many /28 subnets fit inside one /24? How many usable hosts does each have?

SOLUTION

Bits borrowed = $28 - 24 = 4$, so subnets = $2^4 = 16$. Hosts each = $2^{32-28} - 2 = 16 - 2 = 14$. Sanity check: $16 \times 16 = 256$ addresses, which is exactly one /24.

Answer: 16 subnets, 14 usable hosts each

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 9 [TCS NQT · pattern]

Host 172.16.35.200 has mask 255.255.240.0. Find network, broadcast, usable range and host count.

SOLUTION

Step 1. Mask 255.255.240.0 is /20 ($8 + 8 + 4$). Interesting octet is the **3rd** (35).

Step 2. Block size = $256 - 240 = 16$.

Step 3. Multiples of 16: 0, 16, **32**, 48, ... Round 35 down $\rightarrow$ 32. Network = **172.16.32.0**.

Step 4. Broadcast third octet = $32 + 16 - 1 = 47$, last octet 255 $\rightarrow$ **172.16.47.255**. Range **172.16.32.1 – 172.16.47.254**. Hosts = $2^{32-20} - 2 = 4096 - 2 = 4094$.

Answer: net 172.16.32.0 · bcast 172.16.47.255 · 4094 hosts

TRAP

When the interesting octet is the 3rd, the 4th octet is **not** part of the rounding. It goes to 0 in the network address and 255 in the broadcast. A common wrong answer here is “172.16.47.0” for the broadcast.

Example 10 [Wipro · pattern]

Which of these are valid **host** addresses inside 203.0.113.64/27? (a) 203.0.113.64 (b) 203.0.113.65 (c) 203.0.113.94 (d) 203.0.113.95 (e) 203.0.113.96

SOLUTION

/27 $\rightarrow$ block size $256 - 224 = 32$. The block starting at 64 runs 64 to 95.

Address	Role	Usable as a host?
.64	network address (all host bits 0)	no
.65	first host	yes
.94	last host	yes
.95	broadcast (all host bits 1)	no
.96	network address of the next /27	no — different subnet

Answer: (b) and (c) only

Example 11 [Cognizant · pattern]

You own 192.168.5.0/24. You need 6 subnets, each holding at least 25 hosts. What prefix do you use, and list the subnets?

SOLUTION

Step 1 — solve for hosts. Need $2^h - 2 \geq 25$, so $2^h \geq 27$, so $h = 5$ ($2^5 - 2 = 30$, and $h = 4$ gives only 14). Host bits $h = 5 \rightarrow$ prefix = $32 - 5 = /27$.

Step 2 — check the subnet count. Borrowed bits = $27 - 24 = 3 \rightarrow 2^3 = 8$ subnets. We need 6, so 8 is enough (2 spare).

Step 3 — **block size** = $256 - 224 = 32$. Walk in steps of 32:

#	Network	Usable range	Broadcast
1	192.168.5.0/27	.1 – .30	.31
2	192.168.5.32/27	.33 – .62	.63
3	192.168.5.64/27	.65 – .94	.95
4	192.168.5.96/27	.97 – .126	.127
5	192.168.5.128/27	.129 – .158	.159
6	192.168.5.160/27	.161 – .190	.191
7	192.168.5.192/27	.193 – .222	.223
8	192.168.5.224/27	.225 – .254	.255

Answer: /27, mask 255.255.255.224, 8 subnets of 30 hosts; use the first 6

SHORTCUT

Two different questions, two different formulas. “How many **hosts**” → count the **host** bits: $2^h - 2$. “How many **subnets**” → count the **borrowed** bits: 2^b . Decide which one is being asked before you touch the arithmetic.

Example 12 [Accenture · pattern]

A company needs a block big enough for 500 hosts. Which prefix is the smallest that works, and how much space is wasted?

SOLUTION

$2^h - 2 \geq 500 \rightarrow 2^h \geq 502 \rightarrow h = 9$ (since $2^8 - 2 = 254$ is too small and $2^9 - 2 = 510$ fits). Prefix = $32 - 9 = /23$, mask 255.255.254.0. Usable = 510. Waste = $510 - 500 = 10$ usable addresses, plus the 2 reserved ones.

Answer: /23 (255.255.254.0), 510 usable, 10 spare

Doing it with code

a complete subnet calculator (run with node)

JS

```
const toInt = (ip) => ip.split(".").reduce((a, o) => a * 256 + Number(o), 0);
const toIp = (n) => [24, 16, 8, 0].map((s) => Math.floor(n / 2 ** s) % 256).join(".");

function subnet(cidr) {
  const [ip, pStr] = cidr.split("/");
  const p = Number(pStr);
  const size = 2 ** (32 - p); // addresses in the block
  const maskInt = 2 ** 32 - size; // mask as an integer
  const ipInt = toInt(ip);
  const network = Math.floor(ipInt / size) * size; // snap down to a block edge
  const broadcast = network + size - 1;
  return {
    mask: toIp(maskInt),
    wildcard: toIp(size - 1),
    network: toIp(network),
    broadcast: toIp(broadcast),
    firstHost: toIp(network + 1),
  };
}
```

```

    lastHost: toIp(broadcast - 1),
    usableHosts: size - 2,
  };
}

for (const c of ["192.168.10.77/26", "172.16.35.200/20", "203.0.113.77/27"]) {
  console.log(c, subnet(c));
}

```

output — the same numbers we computed by hand

TEXT

```

192.168.10.77/26 {
  mask: '255.255.255.192', wildcard: '0.0.0.63',
  network: '192.168.10.64', broadcast: '192.168.10.127',
  firstHost: '192.168.10.65', lastHost: '192.168.10.126', usableHosts: 62
}
172.16.35.200/20 {
  mask: '255.255.240.0', wildcard: '0.0.15.255',
  network: '172.16.32.0', broadcast: '172.16.47.255',
  firstHost: '172.16.32.1', lastHost: '172.16.47.254', usableHosts: 4094
}
203.0.113.77/27 {
  mask: '255.255.255.224', wildcard: '0.0.0.31',
  network: '203.0.113.64', broadcast: '203.0.113.95',
  firstHost: '203.0.113.65', lastHost: '203.0.113.94', usableHosts: 30
}

```

The line `Math.floor(ipInt / size) * size` is the whole idea of a subnet mask in one expression: **throw away the host part**. A router does the same thing with a bitwise AND.

The **wildcard mask** is the mask with every bit flipped: for /26 it is 0.0.0.63. Access control lists and OSPF configuration use wildcards instead of masks. Block size minus one gives it to you for free.

VLSM — different sizes from one block

Fixed-size subnetting wastes addresses when departments differ in size. VLSM (Variable Length Subnet Masking) cuts blocks of different sizes out of one parent.

SHORTCUT

The VLSM rule: always allocate the biggest requirement first. If you place a small block first you will straddle a boundary and the next big block will not fit anywhere clean.

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 13 [Grab · pattern]

You have 192.168.1.0/24 for one office floor. Requirements:

Segment	Hosts needed
Engineering	60
Sales	28
Operations	12

Router-to-router link	2
-----------------------	---

Allocate with VLSM. Give network, range and broadcast for each.

SOLUTION

Step 1 — turn each requirement into a prefix.

Segment	Need	Smallest $2^h - 2$ that fits	Prefix
Engineering	60	$2^6 - 2 = 62$	/26
Sales	28	$2^5 - 2 = 30$	/27
Operations	12	$2^4 - 2 = 14$	/28
Link	2	$2^3 - 2 = 6$	/29

Step 2 — sort biggest first (already sorted) and lay them down from .0.

Engineering /26, block 64 → starts at 0, ends at 63. Next free address is 64. Sales /27, block 32 → 64 is a multiple of 32, so start at 64, end at 95. Next free is 96. Operations /28, block 16 → 96 is a multiple of 16, start 96, end 111.

Next free is 112. Link /29, block 8 → 112 is a multiple of 8, start 112, end 119.

Step 3 — write the table.

Segment	Network	First – last host	Broadcast	Usable
Engineering	192.168.1.0/26	.1 – .62	.63	62
Sales	192.168.1.64/27	.65 – .94	.95	30
Operations	192.168.1.96/28	.97 – .110	.111	14
Link	192.168.1.112/29	.113 – .118	.119	6

Step 4 — what is left. Used 0 to 119. Free: .120 to .255, which is 136 addresses. That free space can still be cut as 192.168.1.120/29, 192.168.1.128/25, and so on.

Answer: /26, /27, /28, /29 laid down at .0, .64, .96, .112

TRAP

A very common VLSM error is starting Sales at .63 (“the next address after Engineering’s broadcast”). .63 is Engineering’s broadcast. The next usable start is .64. And it must also be a multiple of the new block size — .65 would be illegal for a /27.

Supernetting — gluing blocks together

Route aggregation is VLSM in reverse: many small blocks advertised as one big one, so the routing table stays small.

Example 14 [Shopee · pattern]

A router learns 200.10.4.0/24, 200.10.5.0/24, 200.10.6.0/24 and 200.10.7.0/24. Advertise them as a single route.

SOLUTION

Step 1 — write the varying octet in binary.

4	00000 100
5	00000 101
6	00000 110

7	00000111
---	----------

Step 2 — count the leading bits that are the same. All four start 000001. That is 6 matching bits in the third octet. The first two octets (16 bits) match fully. Total matching = $16 + 6 = 22$.

Step 3 — the aggregate is the lowest network with that prefix: 200.10.4.0/22.

Step 4 — verify. /22 block size in the third octet = $256 - 252 = 4$. Starting at 4 it covers 4, 5, 6, 7. Exactly the four inputs, no extras.

Answer: 200.10.4.0/22, mask 255.255.252.0

TRAP

Aggregation is only safe when the blocks are **contiguous** and start on a correct boundary. 200.10.5.0/24 + 200.10.6.0/24 cannot become a /23: a /23 starting at 5 is illegal because 5 is not a multiple of 2 in that octet. You would have to advertise 200.10.4.0/22, which also claims 4 and 7 — addresses you do not own. That is a black hole.

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 15 [Amazon · pattern]

A VPC is 10.42.0.0/16. Your platform team wants: 3 availability zones, each with one public subnet for 60 hosts and one private subnet for 1000 hosts, and one shared /28 for a NAT/management segment. Design the allocation so that **each zone's whole allocation can be advertised as one route**, and say how much of the /16 is still free.

SOLUTION

Step 1 — size each zone. Per zone: public needs 60 → /26 (62 hosts). Private needs 1000 → $2^{10} - 2 = 1022$, so /22.

Step 2 — the “one route per zone” constraint. A zone contains a /22 and a /26. To aggregate them into a single prefix, the whole zone must sit inside one block that is a power of two and aligned. The /22 dominates the size. A /22 plus a /26 does not fit in a /22, so the zone block must be the next size up: /21 (2048 addresses). Then the /22 and the /26 both sit inside it and the zone is advertised as a single /21.

Step 3 — lay out three /21 zones from 10.42.0.0. /21 block size in the third octet = $256 - 248 = 8$.

Zone	Zone block (one route)	Private /22	Public /26
AZ-a	10.42.0.0/21	10.42.0.0/22	10.42.4.0/26
AZ-b	10.42.8.0/21	10.42.8.0/22	10.42.12.0/26
AZ-c	10.42.16.0/21	10.42.16.0/22	10.42.20.0/26

Step 4 — the shared /28. Put it well clear of the zones so it never blocks a future zone. Take it from the top: 10.42.255.240/28 (block size 16, last block of the /16).

Step 5 — free space. Zones consume 10.42.0.0 through 10.42.23.255 = $3 \times 2048 = 6144$ addresses. The /28 consumes 16. Total in the /16 is 65,536. Free = $65536 - 6144 - 16 = 59,376$ addresses — enough for 28 more /21 zones.

The trade-off to say out loud. Rounding each zone to /21 wastes 962 addresses per zone (2048 used, 1086 needed). You buy back a routing table that has 3 entries instead of 7. In a VPC with route-table entry limits and peering, that trade is worth it. If address space were scarce instead, you would pack tightly and accept the bigger table.

Answer: three /21 zones at .0, .8, .16; shared 10.42.255.240/28; 59,376 addresses free

Part 4 — How a packet actually leaves your machine

The same-subnet test

Before sending anything, a host asks one question: **is the destination in my own subnet?** It ANDs its own IP with its mask, ANDs the destination with the same mask, and compares.

the decision every host makes before it sends (run with node) JS

```
const toInt = (ip) => ip.split(".").reduce((a, o) => a * 256 + Number(o), 0);

// The exact test a host does before sending: same subnet -> ARP for the peer,
// different subnet -> ARP for the default gateway.
function sameSubnet(a, b, prefix) {
  const size = 2 ** (32 - prefix);
  return Math.floor(toInt(a) / size) === Math.floor(toInt(b) / size);
}

const cases = [
  ["192.168.1.10", "192.168.1.200", 24],
  ["192.168.1.10", "192.168.1.200", 25],
  ["10.1.4.9", "10.1.5.9", 23],
  ["10.1.4.9", "10.1.5.9", 24],
  ["172.16.31.1", "172.16.32.1", 20],
];
for (const [a, b, p] of cases)
  console.log(`${a} and ${b} with /${p} -> ${sameSubnet(a, b, p)
    ? "SAME subnet (ARP for peer)" : "DIFFERENT subnet (send to gateway)"}`);
```

output TEXT

```
192.168.1.10 and 192.168.1.200 with /24 -> SAME subnet (ARP for peer)
192.168.1.10 and 192.168.1.200 with /25 -> DIFFERENT subnet (send to gateway)
10.1.4.9 and 10.1.5.9 with /23 -> SAME subnet (ARP for peer)
10.1.4.9 and 10.1.5.9 with /24 -> DIFFERENT subnet (send to gateway)
172.16.31.1 and 172.16.32.1 with /20 -> DIFFERENT subnet (send to gateway)
```

Look at rows 1 and 2. **The same two addresses** are neighbours under /24 and strangers under /25. Nothing about the addresses changed. Only the mask did. That is why “wrong subnet mask” breaks networks in ways that look like magic.

ARP: finding the MAC

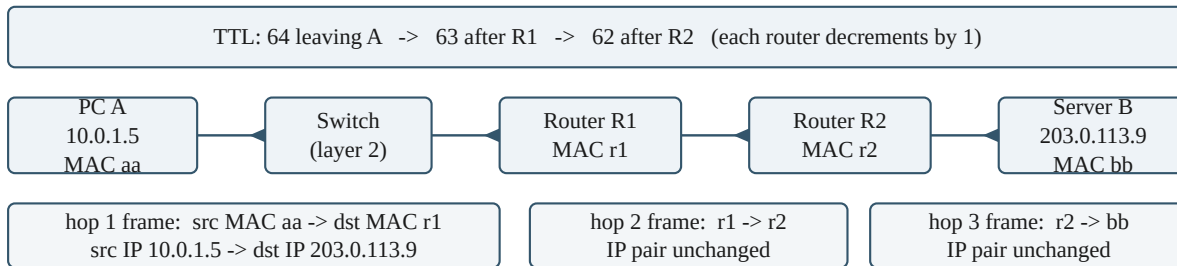
ARP turns an IP address into a MAC address on the local link.

Step	What happens
1	Host needs the MAC for 192.168.1.1.
2	It broadcasts an ARP request to MAC ff:ff:ff:ff:ff:ff: “who has 192.168.1.1?”
3	Every machine on the link sees it. Only the owner replies, unicast , with its MAC.
4	The pair is cached in the ARP table for a few minutes.

TRAP

“ARP finds the MAC of the destination server.” Only if the server is on your own subnet. If it is not, ARP finds the MAC of your **default gateway**. The destination IP stays the server; the destination MAC is the router. Mixing these up is the single most common layer-2/3 mistake.

What changes at each hop



one packet, four hops: the IP pair never changes, the MAC pair changes every hop

Example 16 [Agoda · pattern]

A packet goes from PC A to Server B through 2 routers, as drawn above. Say exactly which of these change per hop and which do not: source IP, destination IP, source MAC, destination MAC, TTL, IP header checksum. Assume no NAT.

SOLUTION

Field	Changes?	Why
Source IP	no	identifies the original sender end to end
Destination IP	no	identifies the final receiver end to end
Source MAC	yes, every hop	the sending interface on this link
Destination MAC	yes, every hop	the next hop on this link
TTL	yes, -1 per router	loop protection
IP header checksum	yes	it must be recomputed because TTL changed

The one-line version to say in an interview: **layer 3 addresses are end-to-end, layer 2 addresses are hop-to-hop.**

If NAT were involved, the source IP **would** change — but only at the NAT box, not at every router.

Answer: MACs, TTL and header checksum change per hop; the IP pair does not (unless NAT)

NAT and PAT

NAT lets many private hosts share one public IP. What real routers do is **PAT** (Port Address Translation), also called NAT overload: the port number is rewritten too, and the port is what makes each flow unique.

Example 17 [Infosys · pattern]

Two laptops behind one home router both open a connection to 203.0.113.9 port 443. The router’s public IP is 49.36.7.2. Show the translation table.

SOLUTION

Inside (private)	Outside (public) after NAT	Destination	Chosen by
192.168.1.10:51000	49.36.7.2:61001	203.0.113.9:443	router

192.168.1.11:51000	49.36.7.2:61002	203.0.113.9:443	router
--------------------	-----------------	-----------------	--------

Both laptops happened to pick the same source port 51000. The router gives each a **different** public port, so the reply “to 49.36.7.2:61002” can only mean laptop .11. The public port number is the whole trick.

Why NAT exists: IPv4 has only $2^{32} \approx 4.3$ billion addresses. NAT lets one address serve a whole household.

What NAT breaks: an outside host cannot start a connection inwards, because there is no table entry yet. That is why peer-to-peer apps need port forwarding, UPnP or STUN/TURN hole-punching.

Answer: same inside port, different outside port; the mapping is keyed on the port

TRAP

“NAT is a firewall.” It is not, but it accidentally behaves like one for **unsolicited inbound** traffic. Say: “NAT is address conservation; the inbound blocking is a side effect, not a security policy.” Do not rely on NAT for security.

DHCP: DORA

A host with no address gets one in four messages.

#	Message	From → to	Contents
1	DISCOVER	client (0.0.0.0) → broadcast	is there a DHCP server?
2	OFFER	server → client	here is an address you may have
3	REQUEST	client → broadcast	I accept that one (broadcast so other servers withdraw their offers)
4	ACK	server → client	confirmed, plus mask, gateway, DNS and lease time

DHCP rides on UDP, server port 67, client port 68. The client has no IP yet, so it sends from 0.0.0.0 to 255.255.255.255.

TRAP

DHCP is a broadcast protocol and routers do not forward broadcasts. So a DHCP server in another subnet is unreachable **unless** the router runs a **DHCP relay agent** (ip helper-address), which converts the broadcast into a unicast to the server. This is a favourite follow-up.

ICMP, ping and traceroute

ICMP is the control and error protocol of layer 3. It carries no user data.

ICMP message	When	Tool that uses it
Echo request / reply	you asked “are you alive”	ping
Time exceeded	TTL hit 0 at a router	traceroute
Destination unreachable	no route, or port closed	error reporting
Fragmentation needed	packet too big, DF bit set	path MTU discovery
Redirect	you used the wrong gateway	route correction

How traceroute works. Send a packet with TTL = 1. The first router decrements it to 0, drops it, and sends back “time exceeded” — which reveals that router’s address. Then TTL = 2 to reveal the second. Keep going until the destination replies. Each row of traceroute output is one more TTL value.

Example 18 [Sea/Shopee · pattern]

ping to a server fails but the website loads in the browser. Give two different explanations.

SOLUTION

Explanation 1 — ICMP is filtered. Many firewalls and cloud security groups drop ICMP echo while allowing TCP 443. Ping tests a different protocol from HTTP. “No ping” is not “no server”.

Explanation 2 — you are not talking to the same machine. The browser resolves the name through DNS, which may return a CDN or load-balancer address, while your ping used a stale cached IP, a different DNS answer, or the origin address directly.

A third, seen in practice: ping uses the A record but the browser used IPv6 (AAAA) and the two paths are configured differently.

Answer: ICMP blocked by policy, or the name resolves to a different address than you pinged

MTU and fragmentation

Ethernet’s standard MTU is 1500 bytes — the biggest IP packet that fits in one frame. A larger packet must be split.

Example 19 [DBS · pattern]

A 4000-byte IP datagram (20-byte header, 3980 bytes of data) must cross a link with MTU 1500. Show every fragment with its size, its offset field and its MF flag.

SOLUTION

Step 1 — how much data fits per fragment. $1500 - 20 = 1480$ bytes of payload. The offset field counts in units of **8 bytes**, so each fragment’s data (except the last) must be a multiple of 8. $1480/8 = 185$ exactly, so 1480 is legal.

Step 2 — split 3980 bytes into pieces of at most 1480. $3980 = 1480 + 1480 + 1020$.

Step 3 — offsets. Offset field = (byte position of this fragment’s data) / 8.

Frag	Data bytes	Total size	Byte range	Offset field	MF flag
1	1480	1500	0 – 1479	0	1 (more coming)
2	1480	1500	1480 – 2959	185	1 (more coming)
3	1020	1040	2960 – 3979	370	0 (last one)

Check: $0/8 = 0$, $1480/8 = 185$, $2960/8 = 370$. All whole numbers, as required. Check total: $1480 + 1480 + 1020 = 3980$. Correct.

Step 4 — who reassembles? Only the **final destination**, never an intermediate router. Every fragment carries the same Identification field so the destination can group them.

Answer: 1500 / 1500 / 1040 bytes; offsets 0, 185, 370; MF = 1, 1, 0

TRAP

The offset field is in 8-byte units, not bytes. Writing “offset 1480” instead of 185 is the standard wrong answer. Also: losing **one** fragment destroys the **whole** datagram, because IP has no per-fragment retransmission. That is why modern stacks avoid fragmentation with Path MTU Discovery (set the DF bit, let ICMP tell you the limit) and why TCP picks an MSS that fits.

The IPv4 header

Field	Size	Why it exists
Version	4 bits	4 or 6
IHL	4 bits	header length in 4-byte words; 5 means the usual 20 bytes
DSCP / ECN	8 bits	quality of service, congestion signalling
Total length	16 bits	header + data, so max 65,535 bytes

Identification	16 bits	groups fragments of one datagram
Flags	3 bits	DF (do not fragment), MF (more fragments)
Fragment offset	13 bits	position of this fragment, in 8-byte units
TTL	8 bits	hop limit; prevents loops
Protocol	8 bits	6 = TCP, 17 = UDP, 1 = ICMP
Header checksum	16 bits	covers the header only , recomputed at every hop
Source IP	32 bits	sender
Destination IP	32 bits	receiver

the IPv4 header checksum, both directions (run with node)

JS

```
// IPv4 header checksum: 16-bit one's complement of the one's complement sum
function checksum16(words) {
  let sum = 0;
  for (const w of words) sum += w;
  while (sum > 0xffff) sum = (sum & 0xffff) + (sum >> 16); // fold the carry back in
  return (~sum) & 0xffff;
}

// A 20-byte header with the checksum field set to 0000
const header = [0x4500, 0x003c, 0x1c46, 0x4000, 0x4006,
                0x0000, 0xac10, 0x0a63, 0xac10, 0x0a0c];

const ck = checksum16(header);
console.log("checksum =", ck.toString(16));

// Receiver side: put it back and the sum must come out 0xffff
header[5] = ck;
let s = 0;
for (const w of header) s += w;
while (s > 0xffff) s = (s & 0xffff) + (s >> 16);
console.log("receiver sum =", s.toString(16), s === 0xffff ? "OK" : "CORRUPT");
```

output

TEXT

```
checksum = b1e6
receiver sum = ffff OK
```

The receiver never has to “compare” anything. It sums **everything including the checksum**. If the result is all ones, the header is intact. That is the whole design.

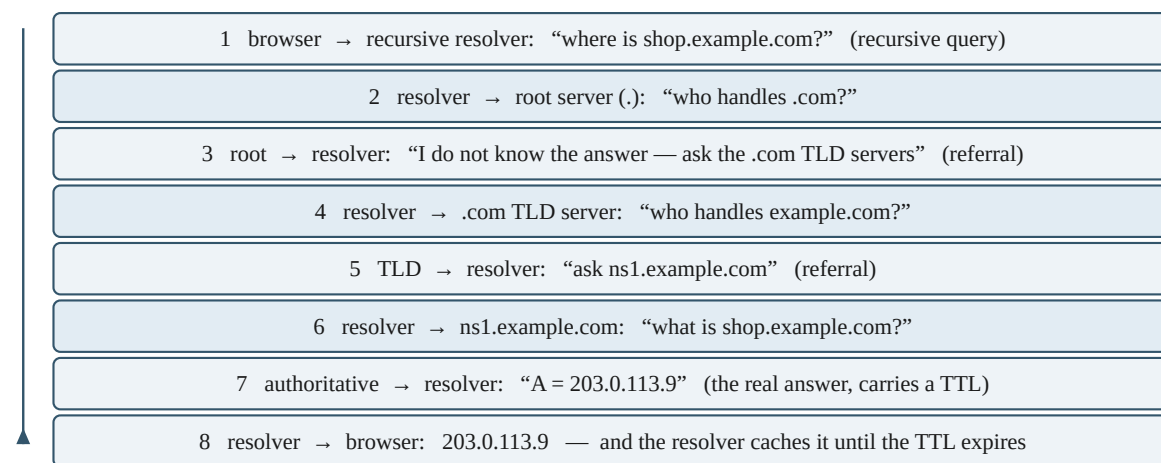
Part 5 — DNS

The record types you must know

Record	Maps	Example use
A	name → IPv4	shop.example → 203.0.113.9
AAAA	name → IPv6	shop.example → 2001:db8::9
CNAME	name → another name	www → shop.example

MX	domain → mail server + priority	mail routing
NS	zone → authoritative name servers	delegation
PTR	IP → name	reverse lookup, spam checks
TXT	name → free text	SPF, domain ownership proofs
SOA	zone → its authority record	serial number, refresh timers

A full resolution, step by step



resolving shop.example.com for the first time — 8 messages, then it is cached

Recursive query	client asks the resolver: “give me the final answer, do the work”
Iterative query	resolver asks a server: “answer, or tell me who to ask next”

Order of caches consulted before any of this happens: browser cache → OS cache → hosts file → recursive resolver cache → the walk above.

Example 20 [Grab · pattern]

A team lowers a record’s TTL from 3600 to 60 one day before moving a service to a new IP. Explain why, and name the cost.

SOLUTION

Why. A cached record is used until its TTL expires. With TTL 3600 (1 hour), resolvers all over the world may keep sending users to the **old** IP for up to an hour after the change. Dropping the TTL to 60 first means that within a few minutes of the switch every cache has re-queried.

The pre-change lowering must happen at least one **old** TTL before the move, so that the long-TTL copies expire and get replaced with short-TTL copies.

The cost. Ten times more DNS queries per hour hitting the authoritative servers, higher lookup latency for users on a cache miss, and a bigger blast radius if the authoritative server is slow or down. So you lower TTL before the change and raise it again after the change is confirmed.

Answer: short TTL = fast cutover; cost is query volume, latency and reduced cache protection

TRAP

“DNS uses UDP.” Mostly, but not only. DNS uses UDP port 53 for normal queries because one packet each way is cheapest. It switches to **TCP** port 53 when the response is too large (the truncated bit is set) and for **zone transfers**. Modern deployments also use DNS over TLS (853) and DNS over HTTPS (443).

Part 6 — Routing

What a routing table is

A routing table is a list of `prefix -> next hop`. For each packet the router finds every row whose prefix contains the destination, and uses the row with the **longest** prefix.

Example 21 [TCS NQT · pattern]

Given this table, where does each destination go?

Prefix	Next hop
0.0.0.0/0	R1 (default)
10.0.0.0/8	R2
10.20.0.0/16	R3
10.20.30.0/24	R4
10.20.30.128/25	R5
192.168.0.0/16	R6

Destinations: 10.20.30.200, 10.20.30.10, 10.20.99.4, 10.5.5.5, 192.168.7.7, 8.8.8.8

SOLUTION

Work one destination at a time. List every row that matches, then keep the longest prefix.

10.20.30.200. Matches /0, /8, /16, /24 and /25 (because $.200 \geq .128$). Longest = /25 → **R5**.

10.20.30.10. Matches /0, /8, /16, /24. Does **not** match /25 — the /25 block starts at .128 and .10 is below it. Longest = /24 → **R4**.

10.20.99.4. Matches /0, /8, /16. Not /24 (third octet is 99, not 30). Longest = /16 → **R3**.

10.5.5.5. Matches /0 and /8 only. → **R2**.

192.168.7.7. Matches /0 and 192.168.0.0/16. → **R6**.

8.8.8.8. Matches only the default. → **R1**.

Destination	Winning route	Next hop
10.20.30.200	10.20.30.128/25	R5
10.20.30.10	10.20.30.0/24	R4
10.20.99.4	10.20.0.0/16	R3
10.5.5.5	10.0.0.0/8	R2
192.168.7.7	192.168.0.0/16	R6
8.8.8.8	0.0.0.0/0	R1

Answer: R5, R4, R3, R2, R6, R1

longest prefix match in 20 lines (run with node)

JS

```
const toInt = (ip) => ip.split(".").reduce((a, o) => a * 256 + Number(o), 0);

const table = [
  { cidr: "0.0.0.0/0",      via: "R1 (default)" },
  { cidr: "10.0.0.0/8",     via: "R2" },
  { cidr: "10.20.0.0/16",   via: "R3" },
  { cidr: "10.20.30.0/24",  via: "R4" },
```

```

{ cidr: "10.20.30.128/25", via: "R5" },
{ cidr: "192.168.0.0/16", via: "R6" },
];

function lookup(dest) {
  const d = toInt(dest);
  let best = null;
  for (const row of table) {
    const [net, pStr] = row.cidr.split("/");
    const p = Number(pStr);
    const size = 2 ** (32 - p);
    if (Math.floor(d / size) * size === toInt(net)) { // dest is inside the block
      if (best === null || p > best.p) best = { p, ...row };
    }
  }
  return best;
}

for (const d of ["10.20.30.200", "10.20.30.10", "10.20.99.4",
                 "10.5.5.5", "192.168.7.7", "8.8.8.8"]) {
  const r = lookup(d);
  console.log(d.padEnd(14), "->", r.cidr.padEnd(16), "via", r.via);
}

```

output — matches the hand-worked table above

TEXT

```

10.20.30.200  -> 10.20.30.128/25  via R5
10.20.30.10   -> 10.20.30.0/24    via R4
10.20.99.4    -> 10.20.0.0/16     via R3
10.5.5.5      -> 10.0.0.0/8        via R2
192.168.7.7   -> 192.168.0.0/16   via R6
8.8.8.8       -> 0.0.0.0/0      via R1 (default)

```

Real routers do not loop over rows; they use a **trie** (a prefix tree) or TCAM hardware so a lookup is $O(\text{prefix length})$ regardless of table size. The internet's default-free routing table is close to a million prefixes.

Distance vector vs link state

	Distance vector	Link state
Idea	tell your neighbours your whole table	tell everyone about your own links
What each router knows	distance and direction only	the full map
Algorithm	Bellman–Ford	Dijkstra
Examples	RIP, EIGRP (hybrid)	OSPF, IS-IS
Convergence	slow; loops possible	fast; loop-free once flooded
Memory / CPU	small	larger
Classic failure	count to infinity	flooding cost in big areas, fixed by splitting into areas

Example 22 [Razer · pattern]

Four routers in a line: A–B–C–D, every link cost 1. Each router starts knowing only its direct neighbours. Show the distance tables after each exchange round until convergence.

| SOLUTION

Write ∞ as 99. At the start each router knows itself (0) and its direct links (1).

Round 0 (start)

	to A	to B	to C	to D
A	0	1	99	99
B	1	0	1	99
C	99	1	0	1
D	99	99	1	0

Round 1. Every router takes $\min(\text{own, cost to neighbour} + \text{neighbour's distance})$. A learns C through B: $1 + 1 = 2$. D learns B through C: $1 + 1 = 2$.

	to A	to B	to C	to D
A	0	1	2	99
B	1	0	1	2
C	2	1	0	1
D	99	2	1	0

Round 2. A learns D through B: B now says 2, so A gets $1 + 2 = 3$. D learns A the same way.

	to A	to B	to C	to D
A	0	1	2	3
B	1	0	1	2
C	2	1	0	1
D	3	2	1	0

Round 3. Nothing changes. Converged.

The pattern: a distance-vector network with a diameter of n hops needs about n rounds. That is why RIP, which exchanges every 30 seconds, can take minutes to settle.

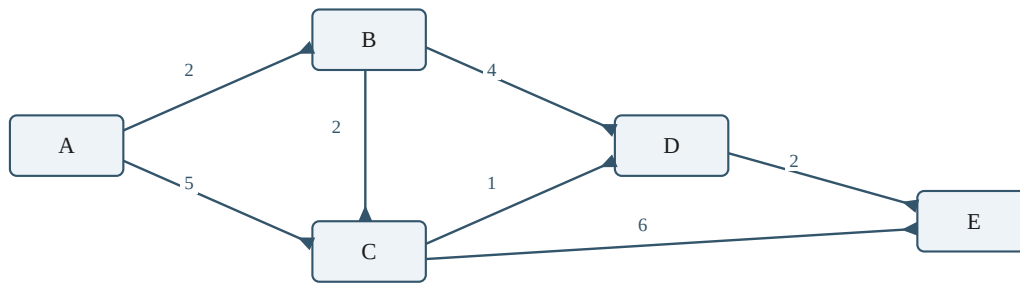
Answer: 3 rounds; final distances A–D = 3

Count to infinity, and the fixes

If link C–D breaks, C sets its distance to D as ∞ . But B still advertises “I can reach D at cost 2”. C believes it, and now thinks D is at cost 3 **through B** — even though B’s path went through C. The two routers bounce the number upwards, one step per round, forever. RIP caps this by defining 16 as infinity, so the nonsense stops after a few rounds.

Fix	What it does
Split horizon	never advertise a route back out of the interface you learned it on
Poison reverse	advertise it back with cost ∞ , which is louder than silence
Hold-down timer	after hearing a route died, ignore better news about it for a while
Triggered updates	send the bad news immediately instead of waiting 30 seconds
Max hop count 15	RIP’s definition of infinity, so the loop is bounded

Link state: Dijkstra by hand



a 5-router area; the number on each link is its OSPF cost

Example 23 [GIC · pattern]

Using the costs drawn above, run Dijkstra from A. Give the shortest cost and path to every router, and the resulting forwarding table at A.

SOLUTION

Links: A–B 2, A–C 5, B–C 2, B–D 4, C–D 1, C–E 6, D–E 2.

Keep a table of “best known cost” and settle one router per step, always the cheapest unsettled one.

Start. A = 0. Others ∞ .

Step 1 — settle A (0). Relax A’s links: B = 2 (via A), C = 5 (via A). Best known: B 2, C 5, D ∞ , E ∞ .

Step 2 — cheapest unsettled is B (2). Settle B. Relax B’s links: C: $2 + 2 = 4 < 5$, so C improves to **4 via B**. D: $2 + 4 = 6$, so D becomes 6 via B. Best known: C 4, D 6, E ∞ .

Step 3 — cheapest unsettled is C (4). Settle C. Relax C’s links: D: $4 + 1 = 5 < 6$, so D improves to **5 via C**. E: $4 + 6 = 10$, so E becomes 10 via C. Best known: D 5, E 10.

Step 4 — cheapest unsettled is D (5). Settle D. Relax D’s links: E: $5 + 2 = 7 < 10$, so E improves to **7 via D**.

Step 5 — settle E (7). Done.

Destination	Cost	Path	First hop from A
B	2	A – B	B
C	4	A – B – C	B
D	5	A – B – C – D	B
E	7	A – B – C – D – E	B

Forwarding table at A: every destination goes out of the interface towards **B**. The direct A–C link at cost 5 is never used, because going the long way round through B costs only 4.

Answer: B 2, C 4, D 5, E 7 — all via B

SHORTCUT

In Dijkstra you never revisit a settled node. The moment a node is settled its cost is final, because all link costs are non-negative and you always settle the cheapest remaining. If an interviewer asks “what if a link had negative cost”, the answer is: Dijkstra breaks, use Bellman–Ford.

The three routing protocols by name

Protocol	Type	Metric	Scope	Notes
RIP	distance vector	hop count, max 15	inside one organisation (IGP)	simple, slow, obsolete
OSPF	link state	cost, from bandwidth	IGP, split into areas	fast, standard in enterprises

IS-IS	link state	cost	IGP	common inside ISPs
BGP	path vector	AS path + policy	between organisations (EGP)	runs the internet

Example 24 [Google · pattern]

Why does the internet use BGP between networks instead of just running OSPF everywhere?

SOLUTION

Reason 1 — scale. OSPF floods the full link-state map and every router runs Dijkstra over it. With a million prefixes and hundreds of thousands of networks, that map cannot be flooded or recomputed. BGP advertises reachability, not topology.

Reason 2 — policy, not shortest path. Between companies the best route is a **business** decision. A transit provider costs money; a peering link is free; a customer’s traffic is revenue. OSPF has only one notion of “best” (lowest cost) and no way to express “never send my customer’s traffic through my competitor”. BGP attributes (local preference, AS path length, MED, communities) exist to encode exactly those rules.

Reason 3 — trust boundaries. OSPF assumes every router in the area is honest and shares a full map. Different companies are not in the same trust domain. BGP deliberately hides internal topology: a network advertises only what it wants others to see.

Reason 4 — loop prevention across administrations. BGP carries the full AS path. A router that sees its own AS number in a path drops the advertisement. This works without any shared map at all.

The trade-off to name. BGP converges slowly and trusts what it is told, which is why route leaks and hijacks happen; RPKI and route filtering are the mitigations.

Answer: scale, policy expressiveness, trust boundaries, and AS-path loop detection

Part 7 — IPv6 in one page

	IPv4	IPv6
Size	32 bits	128 bits
Total addresses	$\approx 4.3 \times 10^9$	$\approx 3.4 \times 10^{38}$
Written as	dotted decimal	8 groups of 4 hex digits, colon separated
Header	variable, 20+ bytes, has a checksum	fixed 40 bytes, no checksum
Fragmentation	routers may fragment	only the sender may; routers never do
Address resolution	ARP (broadcast)	NDP (multicast)
Broadcast	yes	none — multicast and anycast only
Auto-configuration	DHCP	SLAAC, or DHCPv6
NAT	everywhere	rarely needed

Compression rules. Drop leading zeros in each group. Replace **one** run of all-zero groups with ::.

Example 25 [Capgemini · pattern]

Compress 2001:0db8:0000:0000:0000:ff00:0042:8329. Then expand fe80::1.

SOLUTION

Compressing. Step 1 — drop leading zeros in each group: 2001:db8:0:0:0:ff00:42:8329. Step 2 — replace the longest run of zero groups (three of them) with :: **2001:db8::ff00:42:8329**.

Expanding fe80::1. There are 8 groups in total. We are given fe80 and 1, so the :: stands for $8 - 2 = 6$ zero groups: **fe80:0000:0000:0000:0000:0000:0000:0001**.

Answer: 2001:db8::ff00:42:8329 and fe80:0000:0000:0000:0000:0000:0000:0001

TRAP

:: may appear **only once** in an address. 2001::25de::cade is invalid, because you could not tell how many zero groups belong to each ::.

Practice

PRACTICE · TIER 1 target: 18 minutes

1. Convert 10.1.130.7 and 255.255.255.240 to binary.
2. For 192.168.20.130/26: network, broadcast, first host, last host, usable count.
3. How many /30 subnets are inside a /24? How many usable hosts in each?
4. Give the class and the private/public verdict for 172.20.0.5, 172.40.0.5, 192.169.1.1.
5. You need 4 subnets of at least 50 hosts each from 172.16.9.0/24. Give the prefix and the four network addresses.
6. A 2000-byte datagram (20-byte header) crosses an MTU-576 link. How many fragments?
7. Which is the broadcast address of 10.8.0.0/13?
8. A host is 192.168.4.100/22. Is 192.168.7.250 on the same subnet?

ANSWER KEY

1. 10.1.130.7 = 00001010.00000001.10000010.00000111. 255.255.255.240 = 11111111.11111111.11111111.11110000. (130 = 128+2; 240 = 128+64+32+16.)
2. /26 block 64. Multiples: 0, 64, **128**, 192. Network **192.168.20.128**, broadcast **192.168.20.191**, hosts **.129 – .190**, usable **62**.
3. $2^{30-24} = 64$ subnets, each $2^2 - 2 = 2$ usable hosts.
4. 172.20.0.5 class B **private** (16–31). 172.40.0.5 class B **public**. 192.169.1.1 class C **public** — only 192.168 is private.
5. $2^h - 2 \geq 50 \rightarrow h = 6 \rightarrow$ /26 (62 hosts), and $2^2 = 4$ subnets. Networks: 172.16.9.0, 172.16.9.64, 172.16.9.128, 172.16.9.192.
6. Data = 1980. Payload per fragment = $(576 - 20) = 556$, rounded down to a multiple of 8 = **552**. $1980 = 552 + 552 + 552 + 324 \rightarrow$ **4 fragments**. Offsets 0, 69, 138, 207.
7. /13 $\rightarrow$ block size in octet 2 is $256 - 248 = 8$. Block starting at 8 covers 8–15. Broadcast = **10.15.255.255**.
8. /22 block size in octet 3 is 4. 100 is in octet 4, so look at octet 3: multiples of 4 are 0, **4**, 8. The host's block is 192.168.4.0 – 192.168.7.255. 192.168.7.250 is inside. **Yes, same subnet.**

PRACTICE · TIER 2 target: 22 minutes

1. Aggregate 10.6.8.0/24, 10.6.9.0/24, 10.6.10.0/24, 10.6.11.0/24 into one prefix. Prove it covers nothing extra.
2. VLSM 172.16.40.0/24 for 100, 50, 20, 10 and 2 hosts. Show the layout and the leftover.
3. A routing table has 0.0.0.0/0 via R1, 172.16.0.0/16 via R2, 172.16.40.0/21 via R3 and 172.16.40.0/24 via R4. Where do 172.16.40.9, 172.16.44.9 and 172.16.90.9 go?
4. Five routers in a ring, all link costs 1: A–B–C–D–E–A. Run Dijkstra from A.
5. Two offices both use 192.168.1.0/24 and you must join them with a VPN. Name the problem and two fixes.

ANSWER KEY

1. Third octet 8, 9, 10, 11 = 00001000, 00001001, 00001010, 00001011. Six bits match $\rightarrow 16 + 6 = 22$. Aggregate **10.6.8.0/22**. Its block size in octet 3 is 4, so it spans 8, 9, 10, 11 and stops. No extra space is claimed.
2. 100 $\rightarrow$ /25 (126), 50 $\rightarrow$ /26 (62), 20 $\rightarrow$ /27 (30), 10 $\rightarrow$ /28 (14), 2 $\rightarrow$ /30 (2). Biggest first: .0/25 (0–127), .128/26 (128–191), .192/27 (192–223), .224/28 (224–239), .240/30 (240–243). Leftover: .244 – .255 = 12 addresses.
3. 172.16.40.9 matches /0, /16, /21 and /24 $\rightarrow$ longest is /24 $\rightarrow$ **R4**. 172.16.44.9: the /21 starting at 40 covers 40–47, so it matches /0, /16, /21, not the /24 $\rightarrow$ **R3**. 172.16.90.9 matches /0 and /16 only $\rightarrow$ **R2**.

- Ring of 5. From A: B = 1 (direct), E = 1 (direct), C = 2 (A–B–C), D = 2 (A–E–D). Every node is at most 2 hops away. Forwarding: B and C via B; D and E via E.
- Problem:** overlapping address space. A host at 192.168.1.10 in office 1 cannot tell whether 192.168.1.20 means its own neighbour or the remote office — the same-subnet test picks local every time, so remote traffic never reaches the tunnel. **Fix 1:** renumber one office (for example to 192.168.20.0/24). Cleanest, most disruptive. **Fix 2:** NAT the tunnel — each side translates the other's range to a spare block (office 2 appears as 10.99.1.0/24 to office 1). No renumbering, but harder to debug and it breaks protocols that embed IP addresses in their payload.

PRACTICE · TIER 3 target: 25 minutes

- Design addressing for a 4-region service: each region needs 2 subnets of 500 hosts and 4 subnets of 60 hosts, out of 10.80.0.0/16, with one summarisable route per region. State the waste.
- A router has both a /24 static route and a /24 learned by OSPF for the same prefix, with different next hops. Which wins, and what mechanism decides?
- Explain why lowering a DNS TTL does not guarantee a fast cutover. Name two things outside your control.
- Traceroute shows * * * for hops 4 to 7 and then normal output at hop 8. Is the path broken? Explain.

ANSWER KEY

- Per region: 500 hosts → /23 (510), two of them = $2 \times 512 = 1024$. 60 hosts → /26 (62), four of them = $4 \times 64 = 256$. Total needed 1280, so the region block rounds up to **2048 = /21**. Regions: 10.80.0.0/21, 10.80.8.0/21, 10.80.16.0/21, 10.80.24.0/21. Inside region 1: /23s at 10.80.0.0 and 10.80.2.0; /26s at 10.80.4.0, .4.64, .4.128, .4.192. Waste per region = $2048 - 1280 = 768$ addresses; total waste 3072, which is 4.7% of the /16. Bought: 4 routes instead of 24.
- The **static** route wins. The mechanism is **administrative distance** — a trustworthiness ranking applied before the metric is even looked at. Typical values: connected 0, static 1, OSPF 110, RIP 120. Only if two routes have the same administrative distance does the protocol's own metric decide. And prefix length still beats both: a /25 from RIP beats a /24 static, because longest prefix match happens first.
- Lowering the TTL only controls what **well-behaved** caches do from the moment they fetch the new record. Outside your control: (a) resolvers that ignore or floor the TTL — some ISP resolvers pin a minimum of several minutes to cut their own load; (b) application and OS caching — a JVM with the wrong `networkaddress.cache.ttl` can cache a DNS answer for the life of the process, and long-lived HTTP connection pools keep using the **old** socket even after DNS changes. That last one is why cutovers are done at the load balancer, not in DNS, whenever it is possible.
- Not necessarily broken — the final hop answered, so packets are getting through. Those routers are simply **not replying with ICMP time-exceeded**, or their replies are being rate-limited or filtered. ICMP generation is a low-priority, often disabled, control-plane task. Traceroute measures the willingness of routers to answer, not the health of the forwarding path. If hop 8 onwards is fine, the path is fine.

Rapid fire — one-line answers

Question	Answer
Layer of a switch?	2 (data link) — forwards on MAC
Layer of a router?	3 (network) — forwards on IP
PDU at layers 4, 3, 2, 1?	segment, packet, frame, bit
ARP maps what to what?	IP → MAC on the local link; request broadcast, reply unicast
What is 169.254.x.x?	APIPA — DHCP failed, self-assigned link-local
DHCP's four messages and ports?	Discover, Offer, Request, Ack; UDP 67 server, 68 client
The three private ranges?	10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16
Loopback range?	127.0.0.0/8, usually 127.0.0.1
Usable hosts in a /29?	6
Mask for /27?	255.255.255.224

Block size of /26?	64 addresses
Why subtract 2 for usable hosts?	the network address and the broadcast address are reserved
When is a /31 used?	point-to-point router links, where 2 reserved addresses are wasteful
Wildcard mask of /24?	0.0.0.255
Does the source IP change per hop?	no (unless NAT); both MAC addresses change every hop
What decrements TTL, and what happens at 0?	every forwarding router; the packet is dropped and ICMP time-exceeded is sent
Who reassembles IP fragments?	only the final destination, never a router
Fragment offset unit?	8 bytes
Default Ethernet MTU, and IPv4 header size?	1500 bytes; 20 bytes minimum
Protocol number for TCP / UDP / ICMP?	6 / 17 / 1
IPv4 checksum covers what?	the header only, and it is recomputed at every hop
NAT solves what, and how does PAT differ?	IPv4 address exhaustion; PAT also rewrites the port
DNS default transport and port?	UDP 53; TCP 53 for big answers and zone transfers
A vs AAAA vs CNAME?	IPv4 address, IPv6 address, alias to another name
Recursive vs iterative query?	“get me the answer” vs “answer or refer me to the next server”
What is longest prefix match?	the most specific matching route wins
What is 0.0.0.0/0?	the default route — matches everything, always the last resort
RIP vs OSPF vs BGP?	distance vector / link state with Dijkstra / path vector by policy
Administrative distance decides what?	which protocol to believe when two offer the same prefix
IPv6 length, and does it broadcast?	128 bits; no broadcast — multicast and anycast only

ONE-PAGE REVISION CARD

The stack. Physical 1, Data link 2 (frame, MAC, switch), Network 3 (packet, IP, router), Transport 4 (segment, port), Session 5, Presentation 6, Application 7. TCP/IP squashes this to Application / Transport / Internet / Network-access.

Subnet drill.

1. Interesting octet from the prefix (/25→/32 → octet 4, /17→/24 → octet 3).
2. Block size = 256 – mask value in that octet.
3. Network = round that octet down to a multiple of the block size; zero the rest.
4. Broadcast = network + block – 1 in that octet, 255 after it.
5. Hosts = $2^{32-p} - 2$. Subnets from borrowing b bits = 2^b .

Mask values to memorise. 128 192 224 240 248 252 254 255. **Block sizes.** 128 64 32 16 8 4 2 1.

Reserved. 10/8, 172.16/12, 192.168/16 private. 127/8 loopback. 169.254/16 APIPA. 224/4 multicast.

VLSM. Biggest block first. Each block must start on a multiple of its own size. **Aggregation.** Count the matching leading bits; that count is the new prefix.

Per hop. Source and destination IP stay. Source and destination MAC change. TTL – 1. Header checksum recomputed.

ARP finds the MAC of the peer if it is on your subnet, otherwise the MAC of the gateway.

Fragmentation. Payload per fragment is a multiple of 8. Offset counts in 8-byte units. Only the destination reassembles.

DNS. Browser cache → OS cache → hosts file → recursive resolver → root → TLD → authoritative. TTL controls how long an answer is reused.

Routing. Longest prefix match, then administrative distance, then metric. RIP = distance vector, Bellman–Ford, hop count, max 15. OSPF = link state, Dijkstra, cost, areas. BGP = path vector, policy, between organisations.

The two sentences that answer half the follow-ups. “Layer 3 addressing is end-to-end; layer 2 addressing is hop-to-hop.” “A mask does not change the address — it changes who counts as a neighbour.”

Networks: TCP, UDP & HTTP

Handshakes, windows, status codes — and the arithmetic behind all three

What this round actually asks

Chapter 9 got the packet to the right machine. This chapter is about what happens **inside** that machine, and what the two programs at the ends say to each other.

Asked as	Looks like	What is really being tested
Definition	“difference between TCP and UDP”	can you list guarantees, not adjectives
Sequence	“explain the 3-way handshake”	can you say the seq and ack numbers
Arithmetic	“window 64 KB, RTT 40 ms, how fast?”	do you know throughput is window / RTT
Debugging	“why is the connection stuck in <code>TIME_WAIT</code> ?”	do you know the state machine
Web	“difference between 301 and 302”	do you know what a client caches
Depth	“why is HTTP/3 on UDP?”	head-of-line blocking, and where it lives

EVERYTHING THIS CHAPTER IS BUILT ON

TCP vs UDP, as guarantees.

Guarantee	TCP	UDP
Connection set up before data	yes (3-way handshake)	no
Every byte arrives	yes (ACK + retransmit)	no
Arrives in order	yes	no
No duplicates	yes	no
Flow control (do not drown the receiver)	yes	no
Congestion control (do not drown the network)	yes	no
Message boundaries kept	no — it is a byte stream	yes — one datagram, one read
Header size	20 bytes minimum	8 bytes, fixed

Connections are named by a 4-tuple.

(source IP, source port, destination IP, destination port)

Two connections differing in any one of the four are different connections.

Throughput and the pipe.

$$\text{bandwidth-delay product (bytes)} = \frac{\text{link rate in bits/s} \times \text{RTT in s}}{8}$$

$$\text{throughput} \approx \frac{\text{window size in bytes}}{\text{RTT}}$$

$$\text{utilisation with a window of } N \text{ frames} = \min\left(1, \frac{N \times T_t}{T_t + \text{RTT}}\right)$$

$$\text{window needed for 100\%} = 1 + \frac{\text{RTT}}{T_t}$$

Congestion control (TCP Reno).

- Slow start: cwnd **doubles** every RTT, until cwnd reaches ssthresh.
- Congestion avoidance: cwnd grows by **1** every RTT.
- 3 duplicate ACKs (mild loss): ssthresh = cwnd/2, cwnd = ssthresh. Keep going.
- Timeout (severe loss): ssthresh = cwnd/2, cwnd = 1. Back to slow start.
- Sender may send min(cwnd, receiver window) unacknowledged bytes.

Retransmission timer (RFC 6298). With $\alpha = 1/8$ and $\beta = 1/4$:

$$\text{RTTVAR} = (1 - \beta) \times \text{RTTVAR} + \beta \times |\text{SRTT} - \text{sample}|$$

$$\text{SRTT} = (1 - \alpha) \times \text{SRTT} + \alpha \times \text{sample}$$

$$\text{RTO} = \text{SRTT} + 4 \times \text{RTTVAR}$$

HTTP status classes. 1xx informational, 2xx success, 3xx redirect, 4xx **client** made a mistake, 5xx **server** broke.

Part 1 — Ports, sockets and the 4-tuple

An IP address finds the machine. A **port** finds the program on it.

Port	Service	Transport
20, 21	FTP data, FTP control	TCP
22	SSH	TCP
25	SMTP (mail sending)	TCP
53	DNS	UDP, and TCP for big answers
67, 68	DHCP server, DHCP client	UDP
80	HTTP	TCP
110	POP3	TCP
143	IMAP	TCP
443	HTTPS, and HTTP/3 over QUIC	TCP, and UDP for QUIC
3306	MySQL	TCP
5432	PostgreSQL	TCP
6379	Redis	TCP

0 – 1023	well-known ports; on Unix only root may bind them
1024 – 49151	registered ports (databases, application servers)
49152 – 65535	ephemeral ports; the OS hands these to client sockets

WARM-UP

build the reflex

Example 1 [warm-up]

A server listens on port 443. Two browsers on the same laptop both connect to it. How does the server tell the two connections apart?

SOLUTION

By the **4-tuple**. Both connections share three of the four values:

Connection	Src IP	Src port	Dst IP	Dst port
browser 1	10.0.0.5	52001	203.0.113.9	443
browser 2	10.0.0.5	52002	203.0.113.9	443

The OS gave each browser a different **ephemeral source port**. That one number makes the tuples different, so the two connections are different sockets.

Answer: the source port differs, so the 4-tuples differ

TRAP

“A server can handle only 65,535 connections because there are 65,535 ports.” Wrong. The server uses **one** port (443) for all of them. The limit is on the **number of distinct 4-tuples**, and the client’s source port is what varies. A single server port can hold millions of connections; the real limits are file descriptors and memory.

Part 2 — UDP: the honest minimum

The UDP header is 8 bytes. That is the entire protocol.

Field	Size	Purpose
Source port	16 bits	where to send the reply (may be 0 = no reply wanted)
Destination port	16 bits	which program gets it
Length	16 bits	header + data
Checksum	16 bits	optional in IPv4, mandatory in IPv6

UDP keeps message boundaries (run with node)

JS

```
const dgram = require("dgram");

// UDP keeps message boundaries: 3 sends -> 3 receives, each whole.
const server = dgram.createSocket("udp4");
let seen = 0;
server.on("message", (msg, rinfo) => {
  console.log(`datagram ${++seen}: ${JSON.stringify(msg.toString())} ` +
    `(${msg.length} bytes) from port ${rinfo.port}`);
  if (seen === 3) { server.close(); client.close(); }
});

const client = dgram.createSocket("udp4");
server.bind(0, "127.0.0.1", () => {
  const port = server.address().port;
  for (const m of ["PING", "PONG", "DONE"]) client.send(m, port, "127.0.0.1");
});
```

output — three sends, three separate reads

TEXT

```
datagram 1: "PING" (4 bytes) from port 44580
datagram 2: "PONG" (4 bytes) from port 44580
datagram 3: "DONE" (4 bytes) from port 44580
```


Hold on to that output. The very next section shows TCP doing the opposite.

Example 2 [warm-up]

TCP or UDP, and one reason each: (a) downloading a PDF (b) a live video call (c) a DNS lookup (d) sending an email (e) position updates in a shooting game.

SOLUTION

Task	Choice	Reason
Downloading a PDF	TCP	one missing byte ruins the file — you need every byte, in order
Live video call	UDP	a packet that arrives late is useless; skipping it looks better than freezing
DNS lookup	UDP	one small question, one small answer — a handshake costs more than a retry
Sending an email	TCP	SMTP must deliver the message exactly, once
Game position updates	UDP	the next update replaces the lost one within milliseconds

Answer: TCP, UDP, UDP, TCP, UDP

Use UDP when	Because
DNS query	one small question, one small answer; retry is cheaper than a handshake
Voice and video calls	a late packet is useless; better to skip it than wait
Online game state	the next update replaces the lost one anyway
DHCP	the client has no IP yet, so it cannot complete a handshake
QUIC / HTTP/3	the app wants to build its own reliability, better than TCP's

TRAP

“UDP is faster than TCP.” Sloppy. On an idle, lossless link they push bits at the same speed. UDP feels faster because it skips the handshake, never waits for a retransmission, and never backs off for congestion. On a congested link an unthrottled UDP flood can be **slower overall**, because it makes the loss worse. Say “UDP has lower latency and no delivery guarantee”, not “UDP is faster”.

Part 3 — TCP is a byte stream

The thing students get wrong first

three writes, one read (run with node) JS

```
const net = require("net");

// TCP is a BYTE STREAM. Three writes may arrive as one chunk, or split oddly.
const server = net.createServer((sock) => {
  sock.on("data", (buf) => console.log("server got chunk:",
JSON.stringify(buf.toString())));
  sock.on("end", () => server.close());
});

server.listen(0, "127.0.0.1", () => {
  const port = server.address().port;
  const c = net.connect(port, "127.0.0.1", () => {
    c.write("PING");
    c.write("PONG");
```

```

    c.write("DONE");
    c.end();
  });
});

```

output — the three messages have merged

TEXT

```
server got chunk: "PINGPONGDONE"
```

TCP promised that every byte arrives, in order. It never promised that your **messages** stay separate. The application must add its own framing.

length-prefix framing fixes it (run with node)

JS

```

const net = require("net");

// Fix: put a 4-byte length in front of every message, then re-assemble.
function onFramed(sock, handler) {
  let buf = Buffer.alloc(0);
  sock.on("data", (chunk) => {
    buf = Buffer.concat([buf, chunk]);
    while (buf.length >= 4) {
      const len = buf.readUInt32BE(0);
      if (buf.length < 4 + len) break; // message not complete yet
      handler(buf.subarray(4, 4 + len).toString());
      buf = buf.subarray(4 + len);
    }
  });
}

const frame = (s) => {
  const body = Buffer.from(s);
  const head = Buffer.alloc(4);
  head.writeUInt32BE(body.length, 0);
  return Buffer.concat([head, body]);
};

const server = net.createServer((sock) => {
  onFramed(sock, (msg) => console.log("server got message:", msg));
  sock.on("end", () => server.close());
});

server.listen(0, "127.0.0.1", () => {
  const c = net.connect(server.address().port, "127.0.0.1", () => {
    c.write(frame("PING"));
    c.write(frame("PONG"));
    c.write(frame("DONE"));
    c.end();
  });
});

```

output — boundaries restored by the application

TEXT

```

server got message: PING
server got message: PONG
server got message: DONE

```

SHORTCUT

Every text protocol on TCP solves this same problem. HTTP/1.1 uses Content-Length or chunked encoding. Redis uses a length prefix. Line-based protocols use `\n`. If an interviewer asks “how would you design a protocol over TCP”, the first sentence of your answer is **framing**.

The TCP header

Field	Size	Why it exists
Source port / destination port	16 + 16 bits	half of the 4-tuple
Sequence number	32 bits	byte offset of the first byte in this segment
Acknowledgement number	32 bits	the next byte the sender of this ACK expects
Data offset	4 bits	header length in 4-byte words (options make it longer)
Flags	SYN ACK FIN RST PSH URG	see below
Window	16 bits	free space in the receiver’s buffer, in bytes
Checksum	16 bits	covers header + data + a pseudo-header with the IPs
Urgent pointer	16 bits	almost never used
Options	up to 40 bytes	MSS, window scale, SACK permitted, timestamps

SYN	start a connection and synchronise sequence numbers
ACK	the acknowledgement field is valid (set on every segment after the first)
FIN	I have no more data to send (but I will still receive)
RST	abort immediately — no graceful close, no <code>TIME_WAIT</code>
PSH	deliver to the application now, do not sit in the buffer

Example 3 [warm-up]

Match each flag to its job: SYN, ACK, FIN, RST, PSH.

SOLUTION

SYN	open a connection and announce my starting sequence number
ACK	the acknowledgement field is valid — set on every segment after the first
FIN	I have finished sending ; I can still receive
RST	something is wrong — kill this connection immediately
PSH	hand the data to the application now, do not wait for more

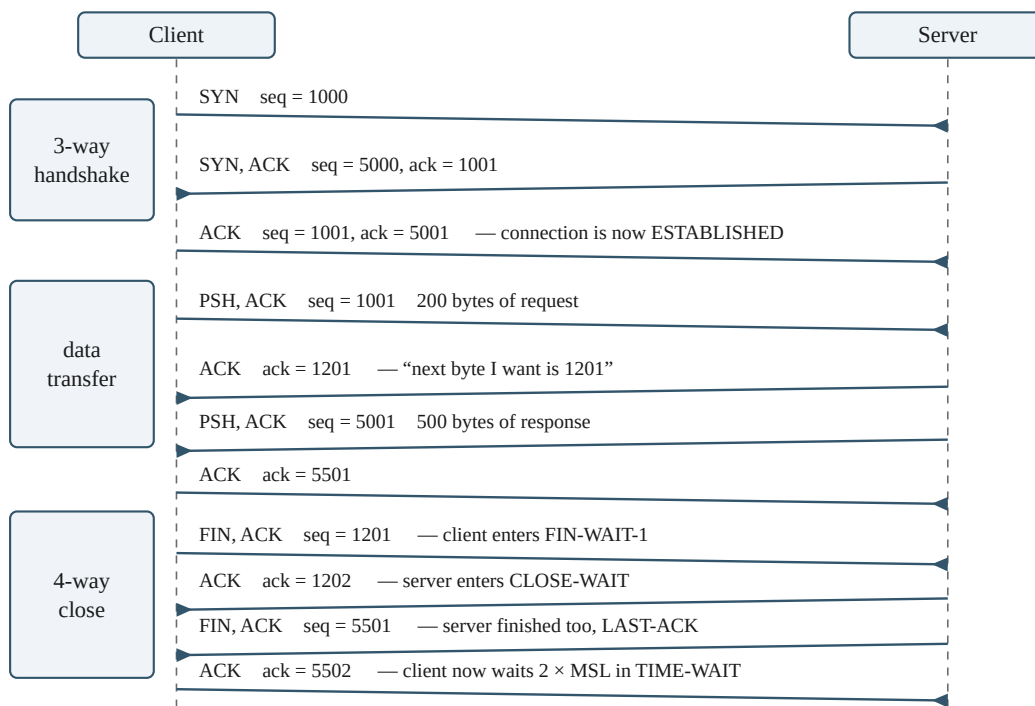
The pair that gets confused is FIN and RST. FIN is a polite goodbye that starts the 4-way close. RST is a slammed door: no close sequence, no `TIME_WAIT`, and the peer’s next read fails with “connection reset”.

Answer: as tabled — FIN is graceful, RST is not

TRAP

The acknowledgement number is **the next byte expected**, not **the last byte received**. If a receiver has bytes 1001 to 1200, it sends `ack = 1201`. Answering “1200” is the standard error. And TCP ACKs are **cumulative**: `ack = 1201` means “everything below 1201 is safe”, not “I got exactly that one segment”.

Part 4 — The handshake, the data, the close



one whole TCP connection: 3 segments to open, 4 to close, and the numbers in between

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 4 [TCS NQT · pattern]

A client picks ISN 1000, a server picks ISN 5000. The client sends 200 bytes, the server replies with 500. Write every seq and ack number, in order.

SOLUTION

Two rules do all the work:

1. **seq** = the byte number of the first byte in this segment.
2. **ack** = the next byte number I expect from you.
3. SYN and FIN each consume **one** sequence number, even though they carry no data.

Dir	Flags	seq	ack	len	Why
C to S	SYN	1000	—	0	the SYN itself counts as 1 byte
S to C	SYN,ACK	5000	1001	0	1000 is consumed, so next is 1001
C to S	ACK	1001	5001	0	handshake done
C to S	PSH,ACK	1001	5001	200	bytes 1001 to 1200
S to C	ACK	5001	1201	0	$1000 + 1 + 200 = 1201$
S to C	PSH,ACK	5001	1201	500	bytes 5001 to 5500
C to S	ACK	1201	5501	0	$5000 + 1 + 500 = 5501$
C to S	FIN,ACK	1201	5501	0	FIN consumes 1
S to C	ACK	5501	1202	0	acknowledging the FIN
S to C	FIN,ACK	5501	1202	0	server's own FIN

C to S	ACK	1202	5502	0	last ACK, then TIME_WAIT
--------	-----	------	------	---	--------------------------

Answer: shown above — the key arithmetic is $1000 + 1 + 200 = 1201$ and $5000 + 1 + 500 = 5501$

the same walk, computed instead of memorised (run with node)

JS

```
// Walk the sequence and acknowledgement numbers of one short TCP conversation.
let c = { isn: 1000 }, s = { isn: 5000 };
const rows = [];
const add = (dir, flags, seq, ack, len, note) =>
  rows.push({ dir, flags, seq, ack, len, note });

add("C->S", "SYN",      c.isn,      "-",      0, "SYN itself counts as 1 byte");
add("S->C", "SYN,ACK",  s.isn,      c.isn + 1, 0, "acks the client's SYN");
add("C->S", "ACK",      c.isn + 1, s.isn + 1, 0, "handshake complete");
add("C->S", "PSH,ACK",  c.isn + 1, s.isn + 1, 200, "request body, bytes 1001..1200");
add("S->C", "ACK",      s.isn + 1, c.isn + 201, 0, "next byte expected = 1201");
add("S->C", "PSH,ACK",  s.isn + 1, c.isn + 201, 500, "response, bytes 5001..5500");
add("C->S", "ACK",      c.isn + 201, s.isn + 501, 0, "next byte expected = 5501");
add("C->S", "FIN,ACK",  c.isn + 201, s.isn + 501, 0, "FIN also counts as 1 byte");
add("S->C", "ACK",      s.isn + 501, c.isn + 202, 0, "");
add("S->C", "FIN,ACK",  s.isn + 501, c.isn + 202, 0, "");
add("C->S", "ACK",      c.isn + 202, s.isn + 502, 0, "then the client waits 2*MSL");

for (const r of rows)
  console.log(`${r.dir}  ${r.flags.padEnd(8)} seq=${String(r.seq).padEnd(5)} ` +
    `ack=${String(r.ack).padEnd(5)} len=${String(r.len).padEnd(4)}
    ${r.note}`);
```

output

TEXT

```
C->S  SYN      seq=1000  ack=-    len=0    SYN itself counts as 1 byte
S->C  SYN,ACK   seq=5000  ack=1001 len=0    acks the client's SYN
C->S  ACK      seq=1001  ack=5001 len=0    handshake complete
C->S  PSH,ACK   seq=1001  ack=5001 len=200  request body, bytes 1001..1200
S->C  ACK      seq=5001  ack=1201 len=0    next byte expected = 1201
S->C  PSH,ACK   seq=5001  ack=1201 len=500  response, bytes 5001..5500
C->S  ACK      seq=1201  ack=5501 len=0    next byte expected = 5501
C->S  FIN,ACK   seq=1201  ack=5501 len=0    FIN also counts as 1 byte
S->C  ACK      seq=5501  ack=1202 len=0
S->C  FIN,ACK   seq=5501  ack=1202 len=0
C->S  ACK      seq=1202  ack=5502 len=0    then the client waits 2*MSL
```

Why three, and not two or four

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 5 [Grab · pattern]

Why does opening need three messages but closing needs four?

| SOLUTION

Opening. Each side must tell the other its starting sequence number, and each must be sure the other heard it. That is 4 logical messages: C’s SYN, S’s ACK, S’s SYN, C’s ACK. The server’s ACK and SYN are **carried in the same segment**, so 4 collapses to 3.

Closing. A TCP connection is two independent one-way streams. The client’s FIN means “I have nothing more to **send**” — it says nothing about what the client still wants to **receive**. The server may still have data in flight. So the server ACKs the FIN immediately, keeps sending, and only later sends its own FIN. Those two cannot be merged, because there is real time between them. Hence 4.

If the server happens to have nothing left to send, its ACK and FIN **can** be combined and the close does take 3 segments. That is a valid follow-up answer.

Answer: open: the server’s ACK and SYN merge. Close: they cannot, because the server may still be sending.

The state machine, and TIME_WAIT

State	Who is in it	Meaning
LISTEN	server	waiting for a SYN
SYN_SENT	client	SYN sent, waiting for SYN-ACK
SYN_RECEIVED	server	SYN-ACK sent, waiting for the final ACK
ESTABLISHED	both	data may flow
FIN_WAIT_1	the side that closed first	FIN sent, not yet acknowledged
FIN_WAIT_2	same side	its FIN was acknowledged; waiting for the peer’s FIN
CLOSE_WAIT	the other side	peer’s FIN received; the application has not closed yet
LAST_ACK	the other side	its own FIN sent, waiting for the final ACK
TIME_WAIT	the side that closed first	waiting $2 \times \text{MSL}$ before the tuple can be reused
CLOSED	both	gone

Example 6 [Sea/Shopee · pattern]

A load-test box shows tens of thousands of sockets in TIME_WAIT, and new outbound connections start failing. Explain the state and give two fixes.

SOLUTION

Why TIME_WAIT exists. Two reasons.

1. **The last ACK might be lost.** If it is, the peer retransmits its FIN. The closing side must still be around to re-answer, otherwise the peer gets an RST and reports an error on a connection that actually finished cleanly.
2. **Stale duplicates.** An old, delayed segment from this connection could arrive after a **new** connection with the same 4-tuple has opened, and be accepted as valid data. Waiting $2 \times \text{MSL}$ (maximum segment lifetime, typically 30–120 s total) guarantees every old segment has died.

Why the failures. The box is the **client**, so it is the side closing first, so it accumulates TIME_WAIT. Each one holds an ephemeral port for the tuple. With roughly 28,000 ephemeral ports, tens of thousands of TIME_WAIT entries exhaust them.

Fix 1 — reuse connections. Keep-alive / connection pooling. If 10,000 requests share 50 connections, there are 50 closes instead of 10,000. This is the real fix, and it also removes the handshake cost.

Fix 2 — let the server close first. Then the **server** holds TIME_WAIT, and a server closing from a fixed port does not consume the client’s ephemeral range.

Fix 3 (tuning, not a fix) — widen the ephemeral port range, or enable tcp_tw_reuse so the kernel may reuse a TIME_WAIT tuple for a new outbound connection when timestamps prove it is safe. Never disable TIME_WAIT outright; you are turning off a correctness guarantee to hide a design problem.

Answer: it protects the final ACK and kills stale duplicates; fix by pooling connections, not by disabling it

TRAP

CLOSE_WAIT and TIME_WAIT are different bugs. **Many TIME_WAIT** = normal for a busy client; tune or pool. **Many CLOSE_WAIT** = **your application code has a bug**. The peer closed, the kernel told you, and your program never called `close()`. Sockets in CLOSE_WAIT never time out on their own. If you see them piling up, look for a missing close in an error path.

Part 5 — Reliability

Mechanism	What it catches
Checksum	corrupted bits — the segment is dropped, not repaired
Sequence number	reordering and duplicates
Cumulative ACK	tells the sender the highest contiguous byte received
Retransmission timeout (RTO)	a segment (or its ACK) vanished entirely
Fast retransmit	a single lost segment, detected by 3 duplicate ACKs, without waiting for the timer
SACK (selective ACK)	lets the receiver say “I have 1–1000 and 2001–3000”, so only the hole is resent

Fast retransmit: why three duplicates

If segment 2 of 1, 2, 3, 4, 5 is lost, the receiver still gets 3, 4 and 5. It cannot ACK them — ACKs are cumulative — so it repeats “I still want 2” each time. The sender sees duplicate ACKs.

One or two duplicate ACKs are ambiguous: they also appear when the network simply **reordered** two segments. Three duplicates is the threshold chosen so that mild reordering does not trigger a pointless retransmission, while real loss is caught in well under one RTO.

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 7 [Capgemini · pattern]

A receiver has bytes 1 to 500, then bytes 501 to 1000 are lost, then bytes 1001 to 1500 and 1501 to 2000 arrive. What acknowledgement numbers does it send? What does SACK add?

SOLUTION

Step 1 — what a plain cumulative ACK can say. An ACK names the next byte expected, and everything below it must be contiguous. The receiver has a contiguous run only up to 500, so the next byte it wants is **501**.

Step 2 — it repeats. Bytes 1001–1500 arrive: the receiver still cannot move past 500, so it sends `ack = 501` again. Bytes 1501–2000 arrive: `ack = 501` again.

Segment received	ACK sent	Kind
1 – 500	<code>ack = 501</code>	normal
1001 – 1500	<code>ack = 501</code>	duplicate ACK 1
1501 – 2000	<code>ack = 501</code>	duplicate ACK 2

Step 3 — what the sender learns. Only “I am still missing byte 501”. Without more information a strict sender may resend **everything** from 501 onwards — including the 1000 bytes the receiver already has.

Step 4 — what SACK adds. With the SACK option the receiver appends “I also have 1001–2000” to each duplicate ACK. Now the sender resends only the 500-byte hole. On a path with several holes this is the difference between one retransmission and a full window of them.

Step 5 — what happens next. One more duplicate ACK (three in total) triggers fast retransmit, so the hole is resent without waiting for the RTO to expire.

Answer: ack = 501 every time; SACK reports the out-of-order block so only the hole is resent

Computing the timeout

the RFC 6298 retransmission timer (run with node)

JS

```
// RFC 6298 retransmission timer
let srtt = null, rttvar = null;
const ALPHA = 1 / 8, BETA = 1 / 4, G = 0; // G = clock granularity, 0 here for clarity

function onSample(m) { // m = measured RTT in ms
  if (srtt === null) { srtt = m; rttvar = m / 2; }
  else {
    rttvar = (1 - BETA) * rttvar + BETA * Math.abs(srtt - m);
    srtt = (1 - ALPHA) * srtt + ALPHA * m;
  }
  const rto = srtt + Math.max(G, 4 * rttvar);
  return { srtt, rttvar, rto: Math.max(1000, rto) }; // RFC floor is 1 second
}

for (const m of [100, 92, 110, 150, 98, 300, 105]) {
  const r = onSample(m);
  console.log(`sample=${String(m).padStart(4)} SRTT=${r.srtt.toFixed(3).padStart(8)}
  +
    ` RTTVAR=${r.rttvar.toFixed(3).padStart(7)} ` +
    ` raw RTO=${(srtt + 4 * rttvar).toFixed(3).padStart(8)} ms`);
}
```

output

TEXT

sample= 100	SRTT= 100.000	RTTVAR= 50.000	raw RTO= 300.000 ms
sample= 92	SRTT= 99.000	RTTVAR= 39.500	raw RTO= 257.000 ms
sample= 110	SRTT= 100.375	RTTVAR= 32.375	raw RTO= 229.875 ms
sample= 150	SRTT= 106.578	RTTVAR= 36.688	raw RTO= 253.328 ms
sample= 98	SRTT= 105.506	RTTVAR= 29.660	raw RTO= 224.146 ms
sample= 300	SRTT= 129.818	RTTVAR= 70.869	raw RTO= 413.292 ms
sample= 105	SRTT= 126.715	RTTVAR= 59.356	raw RTO= 364.139 ms

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 8 [DBS · pattern]

Read the table above. One sample jumped from 98 ms to 300 ms. Why did the RTO nearly double while SRTT moved only from 105.5 to 129.8?

SOLUTION

Two averages with different jobs.

SRTT uses $\alpha = 1/8$, so a new sample moves it by only one eighth of the gap: $105.506 + (1/8)(300 - 105.506) = 105.506 + 24.312 = 129.818$. It is deliberately slow, so one odd measurement does not throw the estimate away.

RTTVAR uses $\beta = 1/4$ and measures **how surprising** the sample was: $0.75 \times 29.660 + 0.25 \times |105.506 - 300| = 22.245 + 48.624 = 70.869$.

The RTO adds **four times** RTTVAR, so a surprise is amplified: $129.818 + 4 \times 70.869 = 413.292$ ms.

That is the design. When the network becomes unpredictable, TCP does not just raise its estimate of the delay — it raises its **margin**, so it does not retransmit segments that were merely slow. Retransmitting a segment that was only late makes congestion worse.

Answer: SRTT smooths the mean slowly; RTTVAR tracks the variation and is multiplied by 4

TRAP

Karn's rule. When a segment is retransmitted and an ACK arrives, you cannot tell whether it acknowledges the original or the copy, so the RTT sample is ambiguous and must be **discarded**. TCP instead doubles the RTO on every retransmission (exponential backoff) until a clean, un-retransmitted sample arrives.

Part 6 — Flow control vs congestion control

Two different problems, two different windows. Confusing them is the classic mistake.

	Flow control	Congestion control
Protects	the receiver's buffer	the network's routers
Signal	the receiver advertises a window	loss, duplicate ACKs, timeouts, delay
Window	rwnd — sent in the header	cwnd — kept privately by the sender
Set by	the receiver	the sender itself
Failure looks like	receiver overflows and drops	routers queue, then drop; everyone slows

The sender may have at most $\min(\text{cwnd}, \text{rwnd})$ unacknowledged bytes in flight. Whichever is smaller wins.

Zero window, and the persist timer

If the receiving application stops reading, its buffer fills and it advertises $\text{window} = 0$. The sender must stop. But the ACK that later reopens the window is a pure ACK, and pure ACKs are not retransmitted — if it is lost, both sides wait forever. So the sender runs a **persist timer** and sends a 1-byte **window probe** to ask “is the window still zero?”. That breaks the deadlock.

Nagle and delayed ACK: the interaction question

Nagle's algorithm	sender-side: if there is unacknowledged data in flight, buffer small writes and send them as one bigger segment. Stops 1-byte segments with 40 bytes of header.
Delayed ACK	receiver-side: wait up to about 200 ms before ACKing, hoping to piggyback the ACK on outgoing data or to ACK two segments at once.

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 9 [Google · pattern]

A request/response service sends its request as two `write()` calls: a small header, then a small body. Latency is fine on the first request and then jumps to about 200 ms per request. Explain, and give the fix.

SOLUTION

The interaction. Write 1 (header) goes out immediately — nothing is in flight. Write 2 (body) is small **and** there is now unacknowledged data, so **Nagle holds it**. The server has received only the header, which is an incomplete request, so it cannot reply — and having no data to piggyback on, **delayed ACK holds its ACK** for up to 200 ms. Nagle waits for the ACK; the ACK waits for data. The deadlock breaks only when the delayed-ACK timer fires. Every request then costs about 200 ms of pure waiting.

Fix 1 — one write. Build header + body into a single buffer and write once. Nagle never engages, because there is nothing to coalesce. This is the correct fix and it is free.

Fix 2 — disable Nagle (`TCP_NODELAY`, which is what `socket.setNoDelay(true)` sets in Node). Standard for latency-sensitive request/response and RPC traffic. The cost is more, smaller segments.

What not to do: disabling delayed ACK. It is a receiver-side setting you usually do not control, and it increases ACK traffic for everyone.

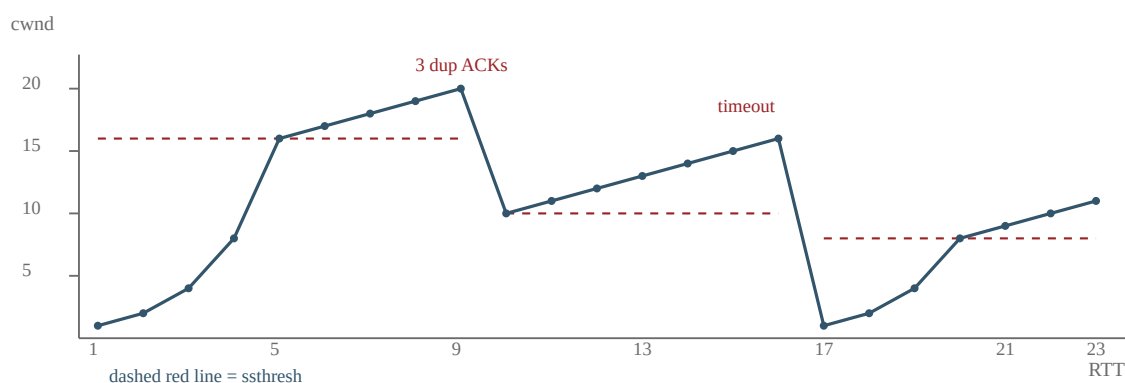
The one-line answer: “Nagle is waiting for an ACK, delayed ACK is waiting for data. Write the whole message in one call, or set `TCP_NODELAY`.”

Answer: Nagle + delayed ACK deadlock; fix with a single write, or `TCP_NODELAY`

Part 7 — Congestion control, with the numbers

The four phases

Phase	Rule	Why
Slow start	cwnd doubles each RTT	find the capacity fast; “slow” refers to the tiny start , not the growth
Congestion avoidance	cwnd grows by 1 each RTT	probe gently once near the limit (additive increase)
Fast retransmit / recovery	on 3 dup ACKs: halve and continue	dup ACKs prove packets are still flowing, so the loss was mild
Timeout	cwnd back to 1, slow start again	silence means something is badly wrong



cwnd over 23 RTTs: ssthresh starts at 16, 3 dup ACKs at RTT 9, a timeout at RTT 16

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 10 [Agoda · pattern]

ssthresh starts at 16 MSS and cwnd at 1. Three duplicate ACKs arrive at the end of RTT 9, and a timeout happens at the end of RTT 16. Give cwnd for RTTs 1 to 23.

SOLUTION

Apply the four rules, one RTT at a time.

RTTs 1–4, slow start ($\text{cwnd} < \text{ssthresh}$, so double): $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16$.

RTT 5: cwnd is now 16, which equals ssthresh . Switch to congestion avoidance: +1 per RTT.

RTTs 5–9: 16, 17, 18, 19, 20.

End of RTT 9 — 3 dup ACKs. $\text{ssthresh} = 20/2 = 10$, and $\text{cwnd} = 10$ (Reno's fast recovery keeps the connection going rather than restarting).

RTTs 10–16, congestion avoidance from 10: 10, 11, 12, 13, 14, 15, 16.

End of RTT 16 — timeout. $\text{ssthresh} = 16/2 = 8$, and $\text{cwnd} = 1$.

RTTs 17–19, slow start again: 1, 2, 4. At 8 it meets the new ssthresh .

RTTs 20–23, congestion avoidance: 8, 9, 10, 11.

RTT	1	2	3	4	5	6	7	8	9	10	11
cwnd	1	2	4	8	16	17	18	19	20	10	11

RTT	12	13	14	15	16	17	18	19	20	21	22	23
cwnd	12	13	14	15	16	1	2	4	8	9	10	11

Answer: 1 2 4 8 16 17 18 19 20 | 10 11 12 13 14 15 16 | 1 2 4 8 9 10 11

the same simulation (run with node)

JS

```
// Reno: slow start -> congestion avoidance; 3 dup ACKs halves, timeout resets to 1.
function simulate(rounds, ssthresh0, events) {
  let cwnd = 1, ssthresh = ssthresh0;
  const log = [];
  for (let r = 1; r <= rounds; r++) {
    const phase = cwnd < ssthresh ? "slow start" : "cong. avoid";
    const ev = events[r] ?? "";
    log.push({ rtt: r, cwnd, ssthresh, phase, ev });
    if (ev === "dup") { ssthresh = Math.floor(cwnd / 2); cwnd = ssthresh; }
    else if (ev === "timeout") { ssthresh = Math.floor(cwnd / 2); cwnd = 1; }
    else if (cwnd < ssthresh) { cwnd = Math.min(cwnd * 2, ssthresh); }
    else { cwnd += 1; }
  }
  return log;
}
const log = simulate(23, 16, { 9: "dup", 16: "timeout" });
for (const r of log)
  console.log(`RTT ${String(r.rtt).padStart(2)} cwnd=${String(r.cwnd).padStart(3)}
  +
    `ssthresh=${String(r.ssthresh).padStart(3)}
  ${r.phase.padEnd(12)}${r.ev}`);
console.log("segments sent in 23 RTTs =", log.reduce((s, r) => s + r.cwnd, 0));
```

output (middle rows trimmed for space)

TEXT

```
RTT 1 cwnd= 1 ssthresh= 16 slow start
RTT 2 cwnd= 2 ssthresh= 16 slow start
RTT 3 cwnd= 4 ssthresh= 16 slow start
RTT 4 cwnd= 8 ssthresh= 16 slow start
RTT 5 cwnd= 16 ssthresh= 16 cong. avoid
```

```

...
RTT 9  cwnd= 20  ssthresh= 16  cong. avoid dup
RTT 10 cwnd= 10  ssthresh= 10  cong. avoid
...
RTT 16 cwnd= 16  ssthresh= 10  cong. avoid timeout
RTT 17 cwnd= 1  ssthresh= 8   slow start
RTT 18 cwnd= 2  ssthresh= 8   slow start
RTT 19 cwnd= 4  ssthresh= 8   slow start
RTT 20 cwnd= 8  ssthresh= 8   cong. avoid
RTT 23 cwnd= 11 ssthresh= 8   cong. avoid
segments sent in 23 RTTs = 241

```

TRAP

“Slow start is slow.” It is the **fastest** growth TCP ever uses — exponential. The name means it **starts** small (1 segment) instead of blasting a full window on a path it knows nothing about. Say “slow start grows exponentially” and you have already separated yourself from most candidates.

Algorithm	Reacts to	Character
Reno	loss	the classic sawtooth; poor on high bandwidth-delay links
CUBIC	loss	Linux default; window grows as a cubic curve, recovers faster after loss
BBR	measured bandwidth and RTT	models the pipe instead of waiting for loss; avoids filling router queues (bufferbloat)

Part 8 — Throughput arithmetic

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 11 [Infosys · pattern]

A 1000-byte frame is sent on a 10 Mbps link with an RTT of 20 ms. (a) Utilisation with stop-and-wait? (b) With a sender window of 7 frames? (c) What window makes utilisation 100%?

SOLUTION

Step 1 — transmission time of one frame.

$$T_t = \frac{1000 \times 8}{10 \times 10^6} = \frac{8000}{10^7} = 0.8 \text{ ms}$$

Step 2 — stop-and-wait. Send one frame, then wait a whole RTT for its ACK. The sender is busy for T_t out of every $T_t + \text{RTT}$:

$$U = \frac{0.8}{0.8 + 20} = \frac{0.8}{20.8} = 0.0385 = 3.85\%$$

Step 3 — window of 7. Seven frames go out back to back before the first ACK returns:

$$U = \frac{7 \times 0.8}{20.8} = \frac{5.6}{20.8} = 0.2692 = 26.92\%$$

Step 4 — window for 100%. Need $N \times T_t \geq T_t + \text{RTT}$:

$$N \geq 1 + \frac{\text{RTT}}{T_t} = 1 + \frac{20}{0.8} = 1 + 25 = 26$$

Answer: (a) 3.85% (b) 26.92% (c) 26 frames

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 12 [GIC · pattern]

A 100 Mbps path has an RTT of 40 ms. The TCP window field is 16 bits. (a) How many bytes fit “in the pipe”? (b) What is the best throughput a 64 KiB window can give? (c) What fixes it?

SOLUTION

Step 1 — bandwidth-delay product. This is how many bytes are in flight when the pipe is full:

$$\text{BDP} = \frac{100 \times 10^6 \times 0.040}{8} = \frac{4 \times 10^6}{8} = 500\{, \}000 \text{ bytes}$$

Step 2 — what the window allows. The 16-bit window field maxes out at 65,535 bytes. The sender may have only that much unacknowledged, then it must stop and wait:

$$\text{throughput} = \frac{65535 \times 8}{0.040} = \frac{524280}{0.040} = 13.107 \text{ Mbps}$$

So a 100 Mbps path delivers 13.1 Mbps. We are using 13% of the link, and the **only** reason is the size of a header field.

Step 3 — how short are we? $500000/65535 = 7.63$. We need about 7.6 windows in flight, and TCP allows 1.

Step 4 — the fix. The **window scale option**, negotiated in the SYN. It declares a shift of up to 14, multiplying the advertised window by up to $2^{14} = 16384$, giving a maximum window of about 1 GB. With scaling on, a 500 KB window fits easily and the link fills.

This situation — fast link, long RTT — is called a **long fat network**, and window scaling is the standard answer.

Answer: (a) 500,000 bytes (b) 13.107 Mbps (c) the window scale option

all of the throughput arithmetic, run (run with node)

JS

```
// 1. Bandwidth-delay product: how many bytes fit "in flight" on the link.
function bdp(bps, rttSeconds) { return (bps * rttSeconds) / 8; }
console.log("BDP 100 Mbps x 40 ms =", bdp(100e6, 0.04), "bytes");

// 2. What a 64 KiB window can actually push at that RTT.
const WIN = 65535;
const thr = (WIN * 8) / 0.04;
console.log("throughput with a 64 KiB window =", (thr / 1e6).toFixed(3), "Mbps");
console.log("windows needed to fill the pipe =", (bdp(100e6, 0.04) /
WIN).toFixed(2));

// 3. Stop-and-wait vs sliding window utilisation.
function util(frameBytes, linkBps, rttSeconds, windowFrames) {
  const Tt = (frameBytes * 8) / linkBps;
  return { TtMs: Tt * 1000, u: Math.min(1, (windowFrames * Tt) / (Tt +
rttSeconds)) };
}
const sw = util(1000, 10e6, 0.020, 1);
console.log("stop-and-wait: Tt =", sw.TtMs.toFixed(3), "ms, utilisation =",
(sw.u * 100).toFixed(2), "%");
console.log("window of 7 : utilisation =",
(util(1000, 10e6, 0.020, 7).u * 100).toFixed(2), "%");
console.log("window needed for 100% =", (1 + 0.020 / (sw.TtMs / 1000)).toFixed(0),
```

```

"frames");

// 4. How long before a 32-bit sequence number wraps.
for (const bps of [100e6, 1e9]) {
  console.log(`seq space wraps in ${ (2 ** 32 / (bps / 8)).toFixed(1)} s at ${bps / 1e6} Mbps`);
}

```

output

TEXT

```

BDP 100 Mbps x 40 ms = 500000 bytes
throughput with a 64 KiB window = 13.107 Mbps
windows needed to fill the pipe = 7.63
stop-and-wait: Tt = 0.800 ms, utilisation = 3.85 %
window of 7 : utilisation = 26.92 %
window needed for 100% = 26 frames
seq space wraps in 343.6 s at 100 Mbps
seq space wraps in 34.4 s at 1000 Mbps

```

That last pair of lines is the reason TCP timestamps exist. At 1 Gbps the 32-bit sequence space repeats every 34 seconds — shorter than the maximum segment lifetime. An old segment could then look like a valid new one. The timestamp option (PAWS, “protect against wrapped sequence numbers”) adds the extra bits needed to tell them apart.

Part 9 — HTTP

A request is just bytes

the literal bytes on the wire (run with node)

JS

```

const http = require("http");
const net = require("net");

const server = http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "text/plain", "Content-Length": "5" });
  res.end("hello");
});

server.listen(0, "127.0.0.1", () => {
  const port = server.address().port;
  const c = net.connect(port, "127.0.0.1", () => {
    // These are the literal bytes an HTTP/1.1 request is made of.
    c.write("GET /hi HTTP/1.1\r\n");
    c.write("Host: example.test\r\n");
    c.write("Connection: close\r\n");
    c.write("\r\n");
  });
  let raw = "";
  c.on("data", (d) => (raw += d));
  c.on("end", () => {
    console.log(JSON.stringify(raw)); // escaped, so you can SEE the \r\n
    console.log("-----");
    console.log(raw);
    server.close();
  });
});

```

```
});  
});
```

output

TEXT

```
"HTTP/1.1 200 OK\r\nContent-Type: text/plain\r\nContent-Length: 5\r\nDate: Tue, 15 Sep 2026 16:36:23 GMT\r\nConnection: close\r\n\r\nhello"  
-----  
HTTP/1.1 200 OK  
Content-Type: text/plain  
Content-Length: 5  
Date: Tue, 15 Sep 2026 16:36:23 GMT  
Connection: close  
  
hello
```

Three rules and you can read any HTTP/1.1 message:

1. The first line is the **start line** (request line, or status line).
2. Then **Name: value** headers, one per line.
3. Then **one blank line**, then the body. The blank line is `\r\n\r\n`.

parsing a request by hand (run with node)

JS

```
// Parse the literal bytes of an HTTP/1.1 request. This is all HTTP/1.1 is.  
const raw =  
  "POST /api/orders HTTP/1.1\r\n" +  
  "Host: shop.example\r\n" +  
  "Content-Type: application/json\r\n" +  
  "Content-Length: 25\r\n" +  
  "Cookie: sid=abc123; theme=dark\r\n" +  
  "\r\n" +  
  '{"item":"kettle","qty":2}\r\n';  
  
const [head, body] = raw.split("\r\n\r\n");  
const [startLine, ...headerLines] = head.split("\r\n");  
const [method, path, version] = startLine.split(" ");  
  
const headers = new Map(  
  headerLines.map((l) => {  
    const i = l.indexOf(":");  
    return [l.slice(0, i).toLowerCase(), l.slice(i + 1).trim()];  
  })  
);  
  
const cookies = Object.fromEntries(  
  (headers.get("cookie") ?? "").split("; ").filter(Boolean).map((c) => c.split("="))  
);  
  
console.log({ method, path, version });  
console.log("host", headers.get("host"));  
console.log("content-length:", headers.get("content-length"));  
console.log("cookies", cookies);  
console.log("body", body.trim());  
console.log("declared length matches?",  
  Number(headers.get("content-length")) === body.trim().length);
```

output

TEXT

```
{ method: 'POST', path: '/api/orders', version: 'HTTP/1.1' }
host      : shop.example
content-length: 25
cookies   : { sid: 'abc123', theme: 'dark' }
body      : {"item":"kettle","qty":2}
declared length matches? true
```

Headers are lower-cased on the way in because HTTP header names are **case-insensitive**. Content-Type, content-type and CONTENT - TYPE are the same header. Header **values** are not case-insensitive.

Methods: safe, idempotent, cacheable

Method	Safe	Idempotent	Cacheable	Meaning
GET	yes	yes	yes	read a resource
HEAD	yes	yes	yes	GET without the body — headers only
OPTIONS	yes	yes	no	what may I do here (used by CORS preflight)
PUT	no	yes	no	replace the resource with this exact content
DELETE	no	yes	no	remove the resource
POST	no	no	rarely	submit; the server decides what it means
PATCH	no	usually no	no	partial update

Safe	does not change server state — a crawler may call it freely
Idempotent	doing it n times leaves the same state as doing it once

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 13 [Accenture · pattern]

The same search form can be submitted with GET (/search?q=kettle&page=2) or with POST (fields in the body). Give four practical differences.

SOLUTION

Difference	GET	POST
Where the data goes	in the URL query string	in the request body
Bookmark and share	yes — the URL holds the state	no — the body is not in the URL
Cacheable and pre-fetchable	yes	effectively no
Shows up in logs, history, Referer	yes — never put a password or token here	no
Length	limited in practice (often about 2000 characters)	effectively unlimited
Repeating it	safe — GET is idempotent	may create a duplicate; browsers warn on refresh

The rule to say: GET for reading, where the URL should describe what you are looking at. POST for changing something, or when the input is large or must not appear in a URL.

Answer: URL vs body; bookmarkable; cacheable; visible in logs; length; safe to repeat

Example 14 [Wipro · pattern]

DELETE is idempotent, yet calling it twice gives 200 the first time and 404 the second. Is that a contradiction?

SOLUTION

No. Idempotence is about the **state of the server**, not about the **response code**.

After the first DELETE the resource is gone. After the second DELETE the resource is still gone. The state is identical, so DELETE is idempotent.

The 404 is just a truthful report about what the second call found.

Same logic for PUT: PUT /user/7 with the same body twice leaves user 7 in exactly one state. POST is **not** idempotent because POST /orders twice creates **two** orders — that is why refreshing a payment page can charge you twice, and why real systems add an **idempotency key** header so the server can recognise a repeat.

Answer: no — idempotence constrains the state, not the status code

Status codes

Code	Name	When you use it
200	OK	normal success with a body
201	Created	POST made something; send a Location header
204	No Content	success, deliberately empty body (a DELETE, a save)
301	Moved Permanently	the new URL is forever; browsers cache this, sometimes hard
302	Found	temporary move; do not cache the new location
304	Not Modified	your cached copy is still good; no body is sent
307 / 308	Temporary / Permanent Redirect	like 302 / 301 but the method must not change
400	Bad Request	the request itself is malformed
401	Unauthorized	not authenticated — who are you?
403	Forbidden	authenticated, but not allowed
404	Not Found	no such resource
405	Method Not Allowed	the path exists, this verb does not
409	Conflict	version clash, duplicate key
415	Unsupported Media Type	wrong Content-Type
429	Too Many Requests	rate limited; send Retry-After
500	Internal Server Error	an unhandled exception on the server
502	Bad Gateway	a proxy got garbage from the upstream server
503	Service Unavailable	overloaded or in maintenance; usually temporary
504	Gateway Timeout	a proxy waited for the upstream and gave up

TRAP

401 vs 403. 401 means “I do not know who you are — send credentials” (and the response should carry **WWW-Authenticate**). 403 means “I know exactly who you are and you still may not”. Swapping these is the most-asked HTTP trick question after 301 vs 302.

301 vs 302. A browser may cache a 301 for a long time — so a wrong 301 in production is very hard to undo, because you cannot reach the clients that cached it. Use 302 (or 307) unless you are certain the move is permanent.

Caching and conditional requests

Cache-Control: max-age=60	reuse for 60 seconds without asking
Cache-Control: no-cache	you may store it, but revalidate before every use
Cache-Control: no-store	never write it to disk or memory (use for private data)
Cache-Control: private	only the browser may cache it, not a shared CDN
ETag: "a1b2"	an opaque version tag for this exact content
If-None-Match: "a1b2"	client: “only send it if the version changed”
Last-Modified / If-Modified-Since	the same idea, using a date instead of a tag

ETag revalidation producing a 304 (run with node)

JS

```
const http = require("http");

const BODY = JSON.stringify({ price: 499 });
const ETAG = '"a1b2"';

const server = http.createServer((req, res) => {
  if (req.headers["if-none-match"] === ETAG) {
    res.writeHead(304, { ETag: ETAG }); // 304 has NO body
    return res.end();
  }
  res.writeHead(200, { ETag: ETAG, "Content-Type": "application/json" });
  res.end(BODY);
});

function ask(port, etag) {
  return new Promise((resolve) => {
    const headers = etag ? { "If-None-Match": etag } : {};
    http.get({ host: "127.0.0.1", port, path: "/price", headers }, (res) => {
      let b = "";
      res.on("data", (d) => (b += d));
      res.on("end", () =>
        resolve({ status: res.statusCode, etag: res.headers.etag, bytes:
b.length }));
    });
  });
}

server.listen(0, "127.0.0.1", async () => {
  const port = server.address().port;
  const first = await ask(port, null);
  console.log("first request :", first);
  const second = await ask(port, first.etag);
  console.log("second request:", second);
  server.close();
});
```

output — the second response carries zero body bytes

TEXT

```
first request : { status: 200, etag: '"a1b2"', bytes: 13 }
second request: { status: 304, etag: '"a1b2"', bytes: 0 }
```

TRAP

A 304 still costs a full round trip — the client asked, the server answered. It only saves **bandwidth**, not **latency**. max-age saves both, because the client does not send a request at all. Interviewers like this distinction: “ETag saves bytes, max-age saves the round trip.”

Part 10 — HTTP/1.1, 2 and 3

The problem each version solves

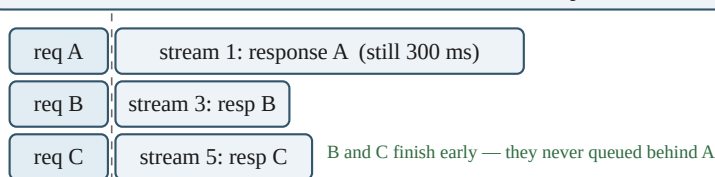
	HTTP/1.1	HTTP/2	HTTP/3
Year	1997	2015	2022
Transport	TCP	TCP	QUIC over UDP
Format	plain text	binary frames	binary frames
Requests per connection	one at a time	many, multiplexed	many, multiplexed
Header compression	none	HPACK	QPACK
Server push	no	yes (now deprecated)	no
Head-of-line blocking	at the HTTP layer	removed at HTTP, remains at TCP	removed at both
Connection setup	TCP + TLS	TCP + TLS	QUIC folds transport and TLS into one

HTTP/1.1 — one connection carries one request at a time; a slow response blocks the queue



B and C were ready at t = 0 but had to wait for A

HTTP/2 — one connection, three independent streams; frames are interleaved



B and C finish early — they never queued behind A

three requests on one connection: serialised in HTTP/1.1, interleaved in HTTP/2

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 15 [Razer · pattern]

A page needs 30 small assets. RTT is 100 ms and the assets themselves are tiny, so only round trips matter. Compare HTTP/1.1 on 1 connection, HTTP/1.1 on 6 connections, and HTTP/2 on 1 connection.

| SOLUTION

HTTP/1.1, 1 connection. One request at a time: 30 round trips. $30 \times 100 = 3000$ ms.

HTTP/1.1, 6 connections. Browsers open up to 6 per host. Six assets per round:

$$\text{rounds} = \left\lceil \frac{30}{6} \right\rceil = 5, \quad 5 \times 100 = 500 \text{ ms}$$

HTTP/2, 1 connection. All 30 requests are sent as streams immediately; all 30 responses come back together:
1 round trip = 100 ms.

Setup	Round trips	Time
HTTP/1.1, 1 connection	30	3000 ms
HTTP/1.1, 6 connections	5	500 ms
HTTP/2, 1 connection	1	100 ms

The follow-up you should volunteer: HTTP/2's win also removes the cost of 6 TCP+TLS handshakes and 6 separate congestion windows that each have to slow-start from scratch. And it makes the old workarounds — sprite sheets, file concatenation, domain sharding — unnecessary or actively harmful.

Answer: 3000 ms, 500 ms, 100 ms

the round-trip arithmetic, run (run with node)

JS

```
// Page load time when the only cost that matters is round trips.
const RTT = 100;           // ms
const ASSETS = 30;

function http11(parallelConns) {
  const rounds = Math.ceil(ASSETS / parallelConns);
  return { rounds, ms: rounds * RTT };
}

function http2() { return { rounds: 1, ms: RTT }; } // all streams multiplexed

console.log("HTTP/1.1, 1 connection :", http11(1));
console.log("HTTP/1.1, 6 connections:", http11(6));
console.log("HTTP/2, 1 connection  :", http2());

// Time before the FIRST request byte can even be sent.
const setup = {
  "HTTP/1.1 over TCP, no TLS": 1, // TCP handshake
  "HTTPS with TLS 1.2": 3, // TCP 1 + TLS 1.2 two round trips
  "HTTPS with TLS 1.3": 2, // TCP 1 + TLS 1.3 one round trip
  "HTTP/3 (QUIC, first visit)": 1, // QUIC folds transport + crypto together
  "HTTP/3 (QUIC, 0-RTT resume)": 0,
};

for (const [name, rtt] of Object.entries(setup))
  console.log(`${name.padEnd(30)} ${rtt} RTT = ${rtt * RTT} ms before the request goes out`);
```

output

TEXT

```
HTTP/1.1, 1 connection : { rounds: 30, ms: 3000 }
HTTP/1.1, 6 connections: { rounds: 5, ms: 500 }
HTTP/2, 1 connection  : { rounds: 1, ms: 100 }
HTTP/1.1 over TCP, no TLS      1 RTT = 100 ms before the request goes out
HTTPS with TLS 1.2             3 RTT = 300 ms before the request goes out
HTTPS with TLS 1.3            2 RTT = 200 ms before the request goes out
```

HTTP/3 (QUIC, first visit)	1 RTT = 100 ms before the request goes out
HTTP/3 (QUIC, 0-RTT resume)	0 RTT = 0 ms before the request goes out

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 16 [Google · pattern]

HTTP/2 already fixed head-of-line blocking. So why did HTTP/3 move to UDP?

SOLUTION

Where the blocking moved to. HTTP/2 removed head-of-line blocking **at the HTTP layer**: ten streams share one TCP connection and their frames interleave. But TCP itself still promises **one ordered byte stream**. If a single TCP segment is lost, the kernel holds back every byte that arrived after it — including bytes belonging to the nine streams that were never affected — until the missing segment is retransmitted. So on a lossy network, HTTP/2 can be **worse** than HTTP/1.1 with 6 connections, because 6 connections mean a loss stalls only one sixth of the page.

Why the fix could not be in TCP. TCP lives in the kernel and in every middlebox (NAT, firewall, load balancer) on the path. A new TCP option takes a decade to deploy, and boxes that do not understand it often drop the traffic. This is **protocol ossification**.

Why UDP. QUIC is built **on top of** UDP in user space. To the network it is ordinary UDP that middleboxes already pass. Inside, QUIC re-implements what TCP gave up:

Per-stream ordering	loss on stream 3 stalls only stream 3; streams 1 and 5 keep delivering
Handshake merged with TLS 1.3	1 RTT to first byte instead of 3, and 0-RTT on resumption
Connection ID instead of the 4-tuple	Wi-Fi to mobile switch keeps the connection alive
Encrypted transport headers	middleboxes cannot inspect or ossify them
Updatable in user space	ships with the browser, not with the kernel

The trade-off to name. QUIC costs more CPU (encryption and packet handling happen in user space, not in NIC offload), UDP is throttled or blocked on some corporate networks, and tooling is younger. Deployments therefore keep HTTP/2 as a fallback via Alt-Svc.

Answer: the blocking that remained was TCP's, and TCP could not be changed — so QUIC rebuilt it over UDP

Part 11 — HTTPS and TLS

0 TCP 3-way handshake completes (1 RTT — TLS has not started yet)

1 client → server ClientHello: TLS version, cipher list, a key share

2 server → client ServerHello: chosen cipher, its own key share

3 server → client Certificate + signature (now encrypted)

4 both sides derive the same symmetric key from the two key shares

5 client checks the certificate chain up to a trusted root CA

6 client → server Finished, then the HTTP request — all symmetric from here

TLS 1.3 handshake: one round trip after TCP, then everything is encrypted

	Asymmetric (public key)	Symmetric
Keys	a public and a private key	one shared secret
Speed	slow	fast
Used in TLS for	proving identity and agreeing on a secret	encrypting all the actual data

That split is the whole design of TLS: **use slow public-key maths once, to agree on a fast shared key.**

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 17 [SCB · pattern]

What does a certificate actually prove, and what stops an attacker from simply copying one?

SOLUTION

What it contains. The domain name, the server’s **public key**, validity dates, the issuing CA, and the CA’s **signature over all of that**.

What it proves. That a certificate authority the client already trusts has vouched that this public key belongs to this domain name. Nothing more — it does not prove the site is honest or the company is good.

Why copying it is useless. The certificate is public; anyone can download it. But the matching **private key** never leaves the real server. During the handshake the server must **sign** handshake data with that private key. An attacker with a copied certificate cannot produce that signature, so the client rejects the connection.

The chain. Browsers ship with root CA public keys. A site certificate is signed by an intermediate, the intermediate by a root. The client verifies each link. A self-signed certificate fails only because no trusted root vouches for it — the encryption itself is identical.

Answer: it binds a domain name to a public key, signed by a trusted CA; the private key cannot be copied

TRAP

“HTTPS encrypts everything.” Not quite. The URL path, headers, cookies and body **are** encrypted. But the **destination IP**, the **port**, the **size and timing** of packets, and — unless Encrypted Client Hello is in use — the **server name** in the TLS SNI field are visible to anyone on the path. DNS lookups too, unless DoH or DoT is used. So an observer often knows **which site** you visited, just not **what you did** there.

Part 12 — Cookies, sessions and CORS

Attribute	Effect	Attack it blunts
HttpOnly	JavaScript cannot read the cookie	XSS stealing the session
Secure	sent only over HTTPS	interception on plain HTTP
SameSite=Lax	not sent on cross-site POSTs; sent on top-level navigations	CSRF
SameSite=Strict	never sent on any cross-site request	CSRF, at the cost of usability
Max-Age / Expires	how long it lives	stale long-lived sessions
Domain / Path	which requests carry it	over-sharing across subdomains

	Server session + cookie	JWT in a header	JWT in a cookie
State lives	on the server	in the token	in the token
Revoke instantly	easy — delete the session	hard — needs a blocklist	hard

Scales across servers	needs shared storage	no shared storage needed	no shared storage
Sent automatically	yes	no — the code must attach it	yes
CSRF exposure	yes, needs SameSite	low	yes, needs SameSite

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 18 [Cognizant · pattern]

A login endpoint replies with Set-Cookie: sid=abc123. Name three attributes that should be added and say what each one stops.

SOLUTION

Add	Effect	Attack it blunts
HttpOnly	document.cookie cannot see it	an XSS script stealing the session
Secure	the cookie is sent only over HTTPS	reading it off a plain-HTTP request
SameSite=Lax	not attached to cross-site POSTs	CSRF — another site posting as the user

Two more worth naming: Max-Age so an abandoned session expires, and a narrow Path or Domain so the cookie is not broadcast to every subdomain.

The final header: Set-Cookie: sid=abc123; HttpOnly; Secure; SameSite=Lax; Max-Age=3600; Path=/.

Answer: HttpOnly (XSS), Secure (plain HTTP), SameSite (CSRF)

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 19 [LINE MAN · pattern]

A browser app on app.example.com calls an API on api.example.com and the browser blocks the response. The same call works from curl. Explain, and list what the server must send.

SOLUTION

Why curl works. CORS is enforced by **browsers**, not by servers or by the network. curl has no same-origin policy, so it never checks.

Why the browser blocks. An origin is (scheme, host, port). app.example.com and api.example.com are different hosts, so they are different origins. The browser sends the request but refuses to hand the response to JavaScript unless the server opts in.

The preflight. For a non-simple request (a custom header such as Authorization, or Content-Type: application/json) the browser first sends an OPTIONS request carrying Origin, Access-Control-Request-Method and Access-Control-Request-Headers.

What the server must answer.

Access-Control-Allow-Origin: https://app.example.com	the exact origin (not * if credentials are used)
Access-Control-Allow-Methods: GET, POST, PUT	allowed verbs
Access-Control-Allow-Headers: Content-Type, Authorization	allowed request headers

Access-Control-Allow-Credentials: true	only if cookies must be sent
Access-Control-Max-Age: 600	cache the preflight for 10 minutes

The trap inside the trap. Access-Control-Allow-Origin: * together with Access-Control-Allow-Credentials: true is rejected by browsers, because it would let **any** site read authenticated responses. With credentials you must echo one exact origin.

Answer: same-origin policy; the server must return the Access-Control-Allow- headers, and answer the OPTIONS preflight

Pushing data to a browser

	Polling	Server-Sent Events	WebSocket
Direction	client asks	server to client only	both ways
Transport	ordinary HTTP	one long-lived HTTP response	HTTP upgrade, then its own framing
Reconnect	trivial	built in	you write it
Good for	rare updates	feeds, notifications, progress	chat, games, live collaboration
Cost	wasted requests	cheap	a held connection per client

Practice

PRACTICE · TIER 1 target: 18 minutes

- List four guarantees TCP gives that UDP does not.
- A client's ISN is 3000 and it sends 400 bytes after the handshake. What ack number does the server return?
- Which is 401 and which is 403: "your token expired", "you are not an admin"?
- A 500-byte frame on a 2 Mbps link with RTT 12 ms. Give T_t and the stop-and-wait utilisation.
- What is the difference between Cache-Control: no-cache and no-store?
- Name the port and transport for DNS, HTTPS, SSH and DHCP.
- Why does a 304 response have no body?
- sssthresh is 8 and cwnd is 1. Give cwnd for the first 6 RTTs with no loss.

ANSWER KEY

- Ordered delivery, retransmission of lost data, duplicate removal, flow control (and congestion control — any four).
- Handshake consumes one number, so data occupies 3001 to 3400. The server acks the **next** byte expected: **3401**.
- Expired token = **401** (not authenticated). Not an admin = **403** (authenticated, not allowed).
- $T_t = (500 \times 8) / (2 \times 10^6) = 4000 / 2000000 = 2 \text{ ms}$. $U = 2 / (2 + 12) = 2 / 14 = 14.29\%$.
- no-cache** = store it, but check with the server before each reuse (a conditional request). **no-store** = never write it down at all.
- DNS 53 UDP (TCP for large answers), HTTPS 443 TCP (UDP for HTTP/3), SSH 22 TCP, DHCP 67/68 UDP.
- Because the client already has the body. 304 means "your cached copy is current" — the whole point is to save the bytes.
- 1, 2, 4, 8 (slow start; at 8 it meets sssthresh), then congestion avoidance: 9, 10.

PRACTICE · TIER 2 target: 22 minutes

- A 1 Gbps link has RTT 100 ms. Give the BDP in bytes, and the window needed to fill it. Is a 16-bit window field enough?
- sssthresh = 32, cwnd = 1, timeout at the end of RTT 8. Give cwnd for RTTs 1 to 12.
- A monitoring dashboard opens a new HTTPS connection per metric, 40 metrics, once a second. RTT is 50 ms. Estimate the wasted time per second and name two fixes.

4. Explain why POST is not idempotent but PUT is, with a shopping-cart example.
5. A service sees thousands of sockets in CLOSE_WAIT. Where is the bug?

ANSWER KEY

1. $BDP = (10^9 \times 0.1)/8 = 12\{, \}500\{, \}000$ bytes = 12.5 MB. A 16-bit field caps at 65,535 bytes, which is **190 times too small** — window scaling is mandatory here. Without it, throughput is $(65535 \times 8)/0.1 = 5.24$ Mbps on a 1 Gbps link.
2. Slow start 1, 2, 4, 8, 16, 32 for RTTs 1–6. At 32 it equals `ssthresh`, so congestion avoidance: RTT 7 = 33, RTT 8 = 34. Timeout at the end of RTT 8: `ssthresh = 34/2 = 17`, `cwnd = 1`. RTTs 9–12: 1, 2, 4, 8.
3. Each new HTTPS connection costs TCP (1 RTT) + TLS 1.3 (1 RTT) = 2 RTT = 100 ms before any request goes out. 40 connections per second → about 4 seconds of handshake work per second of wall clock (spread across parallel sockets, but it is real CPU and latency, and TLS handshakes are expensive). **Fix 1:** one keep-alive connection reused for all 40, or an HTTP/2 connection multiplexing them — handshake cost goes to zero after the first. **Fix 2:** batch the 40 metrics into one request. Either fix also avoids 40 congestion windows restarting from slow start every second.
4. PUT `/cart/item/7` with `{qty: 3}` sets the quantity to 3. Send it five times and the quantity is still 3 — same final state, so idempotent. POST `/cart/items` with `{item: 7, qty: 3}` **appends** a line. Send it five times and the cart holds five lines. Different state each time, so not idempotent.
5. In the **application**, not the network. CLOSE_WAIT means the kernel received the peer's FIN and is waiting for the program to call `close()` on the socket. Thousands of them means a code path — usually an error or timeout branch — returns without closing the socket or releasing it back to the pool. These never expire on their own; the process will run out of file descriptors.

PRACTICE · TIER 3 target: 25 minutes

1. An API behind a CDN serves a 200 with `Cache-Control: max-age=86400` and no Vary header. Users on mobile start receiving the desktop version. What went wrong and what are two fixes?
2. Two datacentres 200 ms apart copy a 10 GB file over one TCP connection and get 8 Mbps. The link is 1 Gbps. Diagnose it and give the numbers.
3. Why does TCP pick a **random** initial sequence number instead of always starting at 0?
4. An HTTP/2 page loads slower than the same page on HTTP/1.1 for users on a train. Explain.

ANSWER KEY

1. **What went wrong.** The CDN caches one object per URL. The origin returned different content for the same URL depending on the **User-Agent**, but did not tell the cache that. The first request (a desktop user) filled the cache, and every mobile user for the next 24 hours got that copy. **Fix 1:** send `Vary: User-Agent` so the cache keys on that header too. It works but fragments the cache badly, because there are thousands of user-agent strings. **Fix 2 (better):** stop varying the response by device. Serve one response and let the client adapt (responsive CSS, client-side logic), or use **different URLs** for genuinely different content. A cache key should be visible in the URL.
2. **Diagnosis.** Throughput is window / RTT, and 8 Mbps at 200 ms RTT implies a window of

$$(8 \times 10^6 \times 0.2)/8 = 200\{, \}000$$

bytes — roughly a 200 KB window, far below what the path needs. **What the path needs:** $BDP = (10^9 \times 0.2)/8 = 25\{, \}000\{, \}000$ bytes = 25 MB. **The gap:** the window is 125 times too small, and the achieved 8 Mbps is 0.8% of the link. **Causes, in the order to check them:** window scaling disabled or clamped by a middlebox; a socket buffer limit capping the window; or loss forcing the congestion window down (at 200 ms RTT, Reno needs a very long time to reopen after each loss — this is exactly what CUBIC and BBR were built for). **Fixes:** enable window scaling and raise the buffer limits, switch the congestion algorithm, or — simplest for a bulk copy — run several parallel TCP streams so the windows add up.

3. Two reasons. **Security.** If the ISN were predictable, an off-path attacker who knows the 4-tuple could guess valid sequence numbers and inject data or a forged RST into an existing connection. A random ISN makes that guess a 1-in- 2^{32} shot. **Correctness.** A delayed segment from a **previous** connection with the same 4-tuple could otherwise land inside the new one with a plausible sequence number and be accepted as real data. Random ISNs plus TIME_WAIT plus timestamps together make that essentially impossible.
4. A train means **packet loss and variable delay**, not just low bandwidth. HTTP/2 puts every stream on **one** TCP connection. TCP guarantees one ordered byte stream, so a single lost segment stalls the delivery of **all** streams until it is retransmitted — one loss, the whole page waits. HTTP/1.1 with 6 connections spreads the risk: a loss stalls only the assets on that one

connection, and the other five keep going. HTTP/3 over QUIC is the real fix, because QUIC tracks loss **per stream**, so only the affected stream waits.

Rapid fire — one-line answers

Question	Answer
What identifies a TCP connection?	the 4-tuple: src IP, src port, dst IP, dst port
TCP vs UDP header size?	20 bytes minimum (up to 60) vs 8 bytes, fixed
Does UDP keep message boundaries?	yes — TCP does not; TCP is a byte stream
Why 3 messages to open but 4 to close?	the server's SYN and ACK merge; the two FINs cannot
What does the ack number mean?	the next byte expected, not the last byte received
Do SYN and FIN consume a sequence number?	yes, one each
What is <code>TIME_WAIT</code> for?	protect the final ACK and let stale duplicates die ($2 \times \text{MSL}$)
Many <code>CLOSE_WAIT</code> sockets means?	an application bug — <code>close()</code> was never called
What is RST?	abort the connection now; no graceful close, no <code>TIME_WAIT</code>
Fast retransmit trigger, and why 3?	3 duplicate ACKs; 1 or 2 also come from harmless reordering
What does SACK add?	the receiver names the blocks it holds, so only the holes are resent
Flow control vs congestion control?	protects the receiver's buffer vs protects the network
How much may the sender have in flight?	$\min(\text{cwnd}, \text{rwnd})$ unacknowledged bytes
Slow start vs congestion avoidance growth?	cwnd doubles per RTT vs cwnd plus 1 per RTT
Reaction to a timeout?	$\text{ssthresh} = \text{cwnd}/2$, $\text{cwnd} = 1$, back to slow start
Reaction to 3 duplicate ACKs?	$\text{ssthresh} = \text{cwnd}/2$, $\text{cwnd} = \text{ssthresh}$, keep going
Nagle plus delayed ACK causes?	a stall of about 200 ms; fix with one write, or <code>TCP_NODELAY</code>
How is a zero window escaped?	the persist timer sends a 1-byte window probe
BDP formula, and throughput from a window?	$\text{rate} \times \text{RTT} \div 8 \text{ bytes}$; $\text{throughput} \approx \text{window} \div \text{RTT}$
Why window scaling?	the 16-bit window field caps at 65,535 bytes
Karn's rule?	never take an RTT sample from a retransmitted segment
When is UDP the right choice?	DNS, voice, video, gaming, DHCP, QUIC
Safe vs idempotent methods?	safe: GET, HEAD, OPTIONS. Idempotent: those plus PUT and DELETE. POST is neither.
301 vs 302?	301 permanent and cached hard; 302 temporary
401 vs 403?	401 = not authenticated; 403 = authenticated but not allowed
304 means?	your cached copy is still valid — no body is sent
no-cache vs no-store?	revalidate before every use vs never write it down at all
HTTP/2's gain, and what blocking remains?	multiplexing plus HPACK; TCP-level head-of-line blocking remains
HTTP/3 runs on what, and why?	QUIC over UDP — loss on one stream no longer stalls the others
Who enforces CORS?	the browser — never the server, never curl

ONE-PAGE REVISION CARD

TCP vs UDP. TCP: connection, ordered, reliable, flow + congestion control, byte stream, 20-byte header. UDP: none of that, message boundaries kept, 8-byte header.

4-tuple. src IP, src port, dst IP, dst port. One server port holds millions of connections because the client's source port varies.

Handshake. SYN (seq=x) → SYN,ACK (seq=y, ack=x+1) → ACK (seq=x+1, ack=y+1). SYN and FIN each consume one sequence number. ack = **next byte expected**.

Close. FIN → ACK → FIN → ACK. Closer waits $2 \times \text{MSL}$ in TIME_WAIT. TIME_WAIT piling up = pool your connections. CLOSE_WAIT piling up = your code forgot close().

Congestion (Reno). Slow start doubles until ssthresh; then +1 per RTT. 3 dup ACKs: halve and continue. Timeout: ssthresh = cwnd/2, cwnd = 1.

Arithmetic. $T_t = L/R$. $U_{\text{stop-and-wait}} = T_t / (T_t + \text{RTT})$. $U_N = NT_t / (T_t + \text{RTT})$. $N_{100\%} = 1 + \text{RTT}/T_t$. BDP = rate $\times$ RTT / 8 bytes. throughput $\approx$ window / RTT. Window field is 16 bits → 65,535 → needs window scaling on fast, long links.

Nagle + delayed ACK deadlock → 200 ms stalls → one write, or TCP_NODELAY.

HTTP methods. Safe: GET, HEAD, OPTIONS. Idempotent: those plus PUT and DELETE. POST is neither.

Status codes. 200 ok, 201 created, 204 empty, 301 permanent, 302 temporary, 304 use your cache, 400 malformed, 401 who are you, 403 not allowed, 404 missing, 409 conflict, 429 slow down, 500 server bug, 502 bad upstream, 503 overloaded, 504 upstream timeout.

Caching. max-age saves the round trip. ETag + If-None-Match → 304 saves only the bytes. Vary adds to the cache key. no-cache = revalidate; no-store = never store.

Versions. HTTP/1.1 = text, one request at a time. HTTP/2 = binary, multiplexed on one TCP connection, HPACK. HTTP/3 = same but over QUIC/UDP, so loss no longer stalls every stream.

TLS. Asymmetric once to agree a key, symmetric for the data. TLS 1.3 = 1 RTT. A certificate binds a domain to a public key, signed by a CA.

Two sentences that answer half the follow-ups. "TCP gives you an ordered byte stream, not messages — framing is your job." "Flow control protects the receiver; congestion control protects the network."

OOP: Principles & Language Mechanics

Four pillars, the words the panel uses, and the truth about JavaScript

This chapter is different from the rest of the book.

Your day job is JavaScript. But the OOP round at a service company is asked in **Java and C++ vocabulary**: virtual functions, abstract classes, interfaces, method overloading, the diamond problem. The interviewer has a checklist and the words on it are Java words.

So every topic here is taught three times:

1. **The idea** — what the concept actually means, in plain language, in no particular language.
2. **The JS reality** — what JavaScript really does, with code you can run.
3. **What the interviewer expects to hear** — the Java/C++ sentence that scores the point.

A student who can only answer in JS fails this round. A student who can do all three passes it and sounds senior. Learn all three.

THE VOCABULARY YOU MUST BE ABLE TO DEFINE IN ONE LINE

Class	A blueprint. Describes what state and behaviour objects of this kind have.
Object	One concrete thing built from a class. Has its own copy of the state.
Encapsulation	Keep data private; expose behaviour. The object guards its own invariants.
Abstraction	Show what a thing does, hide how. The caller sees a contract, not the wiring.
Inheritance	A child class reuses and extends a parent. Models “is-a”.
Polymorphism	One name, many behaviours, chosen by the actual type at call time.
Overloading	Same method name, different parameter list . Chosen at compile time.
Overriding	Same method name, same signature , in a child class. Chosen at run time.
Virtual function	C++: a method the compiler dispatches through a table, so the child version runs.
Abstract class	Cannot be instantiated. May hold state and concrete methods. extends it.
Interface	A pure contract: method names, no state. implements it. Many allowed.
Diamond problem	Class D inherits from B and C which both inherit A. Which A does D get?
Composition	An object has-a part and delegates to it, instead of inheriting from it.
Coupling	How much one class must know about another. Lower is better.
Cohesion	How focused one class is on a single job. Higher is better.

Two sentences that pay for themselves

- Overloading is **compile-time** (static) polymorphism. Overriding is **run-time** (dynamic) polymorphism.
- Inheritance is “is-a”. Composition is “has-a”. Prefer composition unless “is-a” is genuinely true.

11.1 Class and object — the smallest possible start

A class is a cookie cutter. An object is a cookie. The cutter is not edible.

```

class Student {
  constructor(name, marks) { // runs once, when the object is built
    this.name = name;        // state lives on the object
    this.marks = marks;
  }
  grade() {                  // behaviour lives on the class
    if (this.marks >= 80) return "A";
    if (this.marks >= 60) return "B";
    return "C";
  }
}

const a = new Student("Riya", 91);
const b = new Student("Kabir", 64);
console.log(a.name, a.grade()); // Riya A
console.log(b.name, b.grade()); // Kabir B
console.log(a.grade === b.grade); // true -> ONE shared function, two objects

```

Run it: Riya A, Kabir B, true.

That last line matters. Each object has its own name and marks. The method `grade` exists **once**, shared. Memory holds N copies of the data and 1 copy of the code. This is true in Java, C++ and JavaScript — the mechanism differs, the outcome is the same.

WHAT THE INTERVIEWER EXPECTS TO HEAR — JAVA / C++

In Java the same class is:

Java — same class, more ceremony

JAVA

```

class Student {
  private String name;        // private -> encapsulated
  private int marks;
  Student(String name, int marks) { this.name = name; this.marks = marks; }
  public String getName() { return name; }
  public String grade() {
    if (marks >= 80) return "A";
    if (marks >= 60) return "B";
    return "C";
  }
}

```

Say out loud: “name and marks are private instance fields; `grade()` is a public instance method; the constructor initialises the fields.” Those five words — private, instance field, public, instance method, constructor — are the checklist.

11.2 Pillar 1 — Encapsulation

The idea. An object hides its data and only lets you change it through methods it controls. That way it can refuse bad changes. A bank account must never go to a negative balance because somebody typed `acct.balance = -500`.

Real privacy in JS: the `#` field

JS

```

class BankAccount {
  #balance = 0;                // '#' = truly private
  constructor(owner) { this.owner = owner; }
}

```

```

deposit(amt) {
  if (amt <= 0) throw new Error("amount must be positive");
  this.#balance += amt;
  return this.#balance;
}
get balance() { return this.#balance; } // read-only view
}

const a = new BankAccount("Riya");
a.deposit(500);
a.deposit(250);
console.log(a.owner, a.balance);           // Riya 750
console.log(Object.keys(a));               // [ 'owner' ] -- #balance is invisible
console.log(JSON.stringify(a));           // {"owner":"Riya"}
try { a.deposit(-5); } catch (e) { console.log("rejected:", e.message); }

```

Output, verified with node:

```

Riya 750
[ 'owner' ]
{"owner":"Riya"}
rejected: amount must be positive

```

Three things to notice.

1. `#balance` does not appear in `Object.keys` and does not appear in `JSON.stringify`. It is genuinely hidden, not hidden by a naming convention.
2. The only way in is `deposit`, and `deposit` checks the amount. That check is the **invariant**.
3. `get balance()` gives a read path with no write path. That is the classic “getter, no setter”.

TRAP

Writing `a.#balance = 999` from outside the class is not a runtime error you can catch. It is a **SyntaxError** — the file will not even load. Interviewers like this: JS private fields are enforced by the language at parse time, not by convention like the old `_balance` underscore style. The underscore convention (`this._balance`) is a **request**, not a wall. Anyone can write `a._balance = -1`.

WHAT THE INTERVIEWER EXPECTS TO HEAR — JAVA / C++

Java has four access levels. You will be asked to list them in order.

Modifier	Same class	Same package	Subclass, other package	Anywhere
private	yes	no	no	no
(default)	yes	yes	no	no
protected	yes	yes	yes	no
public	yes	yes	yes	yes

C++ has three: `private`, `protected`, `public` — no package level, but it adds `friend`, which lets a named outside class or function reach in.

The model answer: “Encapsulation means the fields are `private` and access goes through public getters and setters, so the class can validate every change and keep its invariants.”

JavaScript has `#private` fields and that is it. No `protected`, no package-private. If the panel asks “what is `protected` in JS?” the correct answer is **“there is none; JS has only `public` and `#private`, and `protected` is simulated by convention or by module scope.”**

11.3 Pillar 2 — Abstraction

The idea. The caller sees **what**, never **how**. You press the brake pedal. You do not know whether it is a disc brake or a drum brake, and your code should not break when the mechanic swaps one for the other.

Encapsulation and abstraction sound the same and get confused in every interview. Hold this distinction:

	Encapsulation	Abstraction
What it hides	the data	the implementation
How	access modifiers, private fields	abstract classes, interfaces, public API
Question it answers	who may touch this field?	what does this thing promise to do?
One-line	bundling data with the methods that guard it	exposing a contract, hiding the mechanism

Abstraction: the caller depends on a contract, not a class

JS

```
const Repository = ["save", "findById", "delete"]; // the contract, as data
function implementsContract(obj, contract) {
  return contract.filter(m => typeof obj[m] !== "function");
}
class MemoryRepo {
  #rows = new Map();
  save(x) { this.#rows.set(x.id, x); return x; }
  findById(id) { return this.#rows.get(id) ?? null; }
  delete(id) { return this.#rows.delete(id); }
}
class BrokenRepo { save(x) { return x; } }

console.log(implementsContract(new MemoryRepo(), Repository)); // [] -> ok
console.log(implementsContract(new BrokenRepo(), Repository)); // missing methods

const r = new MemoryRepo();
r.save({ id: 1, name: "pen" });
console.log(r.findById(1), r.findById(2), r.delete(1), r.findById(1));
```

Verified output:

```
[]
[ 'findById', 'delete' ]
{ id: 1, name: 'pen' } null true null
```

Swap MemoryRepo for a PostgresRepo later and the calling code never changes, because the calling code was written against the three method names, not against the class.

WHAT THE INTERVIEWER EXPECTS TO HEAR — JAVA / C++

In Java you do not check the contract by hand — the compiler does it.

Java — the contract is a language construct

JAVA

```
interface Repository<T> {
  T save(T item);
  T findById(int id);
  boolean delete(int id);
}
class MemoryRepo implements Repository<Item> { /* compiler FORCES all three */ }
```

Say: “Abstraction is achieved through abstract classes and interfaces. In Java an interface is a pure contract — the compiler refuses to build a class that claims to implement it but does not.” In C++ the same thing is an abstract base class with pure virtual functions (`virtual T f() = 0;`).

The JS honest answer: “**JavaScript has no interfaces. It uses duck typing — if the object has the methods, it works. TypeScript adds compile-time interface back.**”

11.4 The JS reality — everything is a prototype

Before inheritance, you need this. In Java, a class is a **type** known to the compiler. In JavaScript, class is **syntax sugar over functions and prototype objects**, and lookups happen at run time by walking a chain.

A JS class is a function; methods live on .prototype

JS

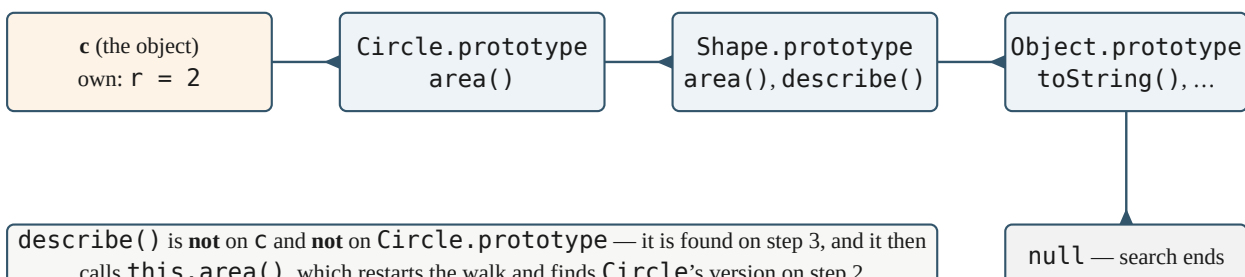
```
class Shape {
  constructor(name) { this.name = name; }
  area() { return 0; }
  describe() { return `${this.name} has area ${this.area()}`; }
}
class Circle extends Shape {
  constructor(r) { super("circle"); this.r = r; }
  area() { return +(Math.PI * this.r * this.r).toFixed(2); }
}

const c = new Circle(2);
console.log(typeof Shape); // function
console.log(Object.getPrototypeOf(c) === Circle.prototype); // true
console.log(Object.getPrototypeOf(Circle.prototype) === Shape.prototype); // true
console.log(c.hasOwnProperty("area")); // false: it is on the
prototype
console.log(Circle.prototype.hasOwnProperty("area")); // true
console.log(c.describe()); // dynamic dispatch
```

Verified output:

```
function
true
true
false
true
circle has area 12.57
```

Check the maths yourself: $\text{area} = \pi r^2 = 3.14159 \times 4 = 12.566\dots$, rounded to 2 places is 12.57. Correct.



How c.describe() is found: the prototype chain is walked left to right

That restart is the whole trick. `describe()` is written in `Shape`, but the `this.area()` inside it resolves against the **actual** object, so it finds `Circle.area`. That is dynamic dispatch, and it is what “polymorphism” means in practice.

TRAP

`hasOwnProperty("area")` on the instance returns **false**. Students expect `true` because `c.area()` works. The property is reachable through the chain but it is not **owned**. This distinction shows up in real bugs: `for...in` walks the chain, `Object.keys` does not.

11.5 Pillar 3 — Inheritance

The idea. `class B extends A` means “every B is also an A”. B gets A’s methods for free and may add or replace some.

extends and superJS

```
class Payment {
  constructor(amt) { this.amt = amt; }
  fee() { return 0; }
  total() { return this.amt + this.fee(); }
}
class Card extends Payment {
  constructor(amt) { super(amt); } // super() MUST run before using 'this'
  fee() { return +(this.amt * 0.02).toFixed(2); }
}
const p = new Card(1000);
console.log(p.fee(), p.total()); // 20 1020
console.log(p instanceof Card, p instanceof Payment); // true true
```

`p instanceof Payment` is true. That is the “is-a” test, and it is the one-line proof of inheritance.

Types of inheritance — the list they want

Type	Shape	Allowed?
Single	B extends A	Java yes · C++ yes · JS yes
Multilevel	C extends B extends A	Java yes · C++ yes · JS yes
Hierarchical	B and C both extend A	Java yes · C++ yes · JS yes
Multiple	D extends B and C (classes)	Java NO · C++ yes · JS NO
Hybrid	any mix of the above	only where multiple is allowed

WHAT THE INTERVIEWER EXPECTS TO HEAR — JAVA / C++

“Java does not support multiple inheritance **of classes** — to avoid the diamond problem — but it does support multiple inheritance **of type** through interfaces: a class may implement any number of interfaces. C++ supports multiple inheritance of classes and solves the diamond with `virtual` inheritance.”

Memorise the shape of that sentence. It answers three questions at once.

In Java the child constructor implicitly calls `super()` first; in C++ base constructors run before the derived constructor body; in JS you must write `super(...)` explicitly before touching `this`, or you get a `ReferenceError`.

Constructor order — a favourite trap

```

class Base {
    Base() { System.out.println("Base ctor"); init(); }
    void init() { System.out.println("Base.init"); }
}
class Derived extends Base {
    int n = 42;
    Derived() { super(); System.out.println("Derived ctor, n=" + n); }
    @Override void init() { System.out.println("Derived.init, n=" + n); }
}
public class Order { public static void main(String[] a){ new Derived(); } }

```

Compiled and run with `javac / java`. Actual output:

```

Base ctor
Derived.init, n=0
Derived ctor, n=42

```

Walk it step by step.

1. `new Derived()` starts. `super()` runs first, so `Base()` begins.
2. `Base()` prints `Base ctor`, then calls `init()`.
3. `init()` is **virtual** — Java methods are virtual by default — so `Derived.init` runs, not `Base.init`.
4. But `Derived`'s field initialisers have not run yet. `n` is still the default 0.
5. `Base()` returns. **Now** `n = 42` runs. Then the `Derived` body prints `n=42`.

TRAP

Never call an overridable method from a constructor. The subclass override runs against a half-built object. This is a real, common production bug and a classic senior-level question. The fix: make the method `final` (Java) / non-virtual (C++), or call it after construction.

11.6 Pillar 4 — Polymorphism

The idea. Write the loop once; let each object decide what to do.

One loop, four types, no if/else on type

JS

```

class Payment { constructor(amt){ this.amt = amt; } fee(){ return 0; } }
class Card extends Payment { fee(){ return +(this.amt * 0.02).toFixed(2); } }
class UPI extends Payment { fee(){ return 0; } }
class Cheque extends Payment { fee(){ return 15; } }

const batch = [new Card(1000), new UPI(1000), new Cheque(1000), new Card(2500)];
let total = 0;
for (const p of batch) total += p.fee(); // no switch on type
console.log(batch.map(p => `${p.constructor.name}:${p.fee()}`).join(" "));
console.log("total fee =", +total.toFixed(2));

```

Verified output:

```

Card:20 UPI:0 Cheque:15 Card:50
total fee = 85

```

Check the arithmetic: $1000 \times 0.02 = 20$, 0, flat 15, $2500 \times 0.02 = 50$. $20 + 0 + 15 + 50 = 85$. Correct.

Now add `NetBanking`. The loop does not change. **That** is the payoff, and it is what you should say when asked “why is polymorphism useful?” — not “one interface, many forms”, which says nothing.

The two kinds — say both names

	Compile-time (static)	Run-time (dynamic)
Also called	early binding, static binding	late binding, dynamic binding
Mechanism	method overloading , operator overloading	method overriding , virtual functions
Decided by	the declared types of the arguments	the actual type of the object
Decided when	compilation	execution
Cost	free	one indirect jump through the vtable
In Java	overloading	every non-static, non-final, non-private method
In C++	overloading, templates	only methods marked <code>virtual</code>
In JS	does not exist	every method — the prototype chain is always walked

11.7 Overloading vs overriding — the question you will definitely be asked

Overloading: same name, different parameter list

Java — three methods, one name

JAVA

```
class Calc {
    int add(int a, int b)        { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
    double add(double a, double b) { return a + b; }
}
public class Overload {
    public static void main(String[] args) {
        Calc k = new Calc();
        System.out.println(k.add(1, 2));        // 3
        System.out.println(k.add(1, 2, 3));      // 6
        System.out.println(k.add(1.5, 2.5));    // 4.0
    }
}
```

Compiled and run. Output: 3, 6, 4.0. Three different methods exist in the class file. The compiler picks one by looking at the argument types at the call site.

Rules that make an overload legal (Java): the parameter list must differ in **number**, **type**, or **order** of parameters. Changing only the return type is **not** an overload — it is a compile error.

Now the JS truth: there is no overloading

JS — the second definition silently wins

JS

```
class Calc {
    add(a, b)        { return a + b; }
    add(a, b, c)     { return a + b + c; } // silently replaces the one above
}
const k = new Calc();
console.log(k.add(1, 2));        // NaN -> c is undefined
console.log(k.add(1, 2, 3));    // 6
```

```
// The honest JS way: one method, inspect the arguments.
class Calc2 {
  add(...nums) { return nums.reduce((s, n) => s + n, 0); }
}
console.log(new Calc2().add(1, 2)); // 3
console.log(new Calc2().add(1, 2, 3, 4)); // 10
```

Verified output: NaN, 6, 3, 10.

Why NaN? `k.add(1, 2)` calls the **surviving** three-parameter version. `c` is undefined, and `1 + 2 + undefined` is NaN. No warning, no error. This bites people.

TRAP

“JavaScript supports method overloading using default parameters” — **wrong answer, say no**. A default parameter changes what one function does when an argument is missing. Overloading means **several distinct functions** chosen by signature. JS objects hold one property per key, so the second `add` overwrites the first. The right phrasing: “JavaScript does not support overloading; it simulates it with rest parameters or by branching on arguments.`length` and `typeof`.”

Overriding: same name, same signature, child class

	Overloading	Overriding
Where	same class (or inherited into one)	child class only
Signature	must differ	must be identical
Return type	may differ freely	same, or a subtype (covariant)
Access	any	may not be more restrictive than the parent’s
static methods	can be overloaded	cannot be overridden — they are hidden
Binding	compile time	run time
Marker	none	@Override (Java), override (C++11)

TRAP

Static methods are not overridden, they are hidden. In Java, if `Base` and `Derived` both declare `void f()`, then `Base b = new Derived(); b.f();` calls **Base.f** — because static dispatch uses the **declared** type. Saying “static methods can be overridden” is an instant loss of a point. Same for `private` and `final` methods: `private` is invisible to the child, and `final` forbids the override outright.

11.8 Virtual functions and the vtable

This is the C++ question. Java makes every method virtual by default, so nobody asks it there; C++ makes you opt in, so everybody asks it there.

C++ — the one experiment that explains everything

CPP

```
#include <iostream>
struct Shape {
  void draw() { std::cout << "Shape::draw\n"; } // NOT virtual
  virtual void print() { std::cout << "Shape::print\n"; } // virtual
  virtual ~Shape() {}
};
struct Circle : Shape {
```

```

void draw()          { std::cout << "Circle::draw\n"; }
void print() override { std::cout << "Circle::print\n"; }
};
int main() {
    Shape* p = new Circle();
    p->draw();    // static binding -> base version
    p->print();    // dynamic binding -> derived version
    delete p;
}

```

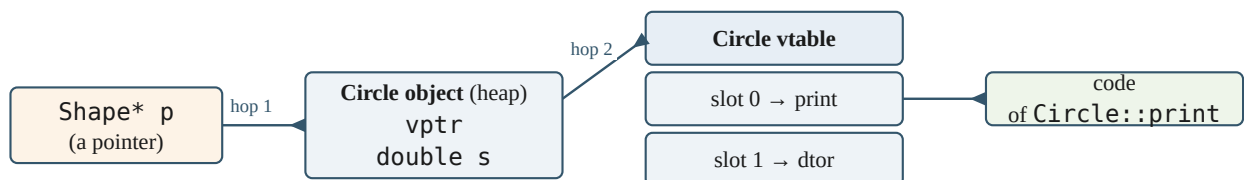
Compiled with `g++ -std=c++17` and run. Actual output:

```

Shape::draw
Circle::print

```

The object is a Circle in both lines. The only difference is the word `virtual`. Without it, the compiler decides from the **pointer type** (`Shape*`) and hard-codes a call to `Shape::draw`. With it, the compiler emits a lookup through a per-class table of function pointers — the **vtable** — and the answer depends on what the object actually is.



Cost: one extra memory read per virtual call, and the target is not known at compile time, so it cannot be inlined. That is the whole price of polymorphism in C++ — and the reason `virtual` is opt-in there.

Dynamic dispatch: `p->print()` is two pointer hops, not a direct call

Proof that the vptr is real

sizeof tells you the hidden pointer exists

CPP

```

#include <iostream>
struct Plain { int x; };
struct Virtual { int x; virtual void f(); virtual ~Virtual() = default; };
void Virtual::f() {}
struct Abstract { virtual double area() = 0; virtual ~Abstract() = default; }; //
pure virtual
struct Sq : Abstract { double s; Sq(double s):s(s){} double area() override { return
s*s; } };
int main() {
    std::cout << "sizeof(Plain)    = " << sizeof(Plain)    << "\n";
    std::cout << "sizeof(Virtual) = " << sizeof(Virtual) << " (int + vptr)\n";
    Abstract* p = new Sq(3);
    std::cout << "area = " << p->area() << "\n";
    delete p;
}

```

Verified output on a 64-bit Linux build:

```

sizeof(Plain)    = 4
sizeof(Virtual) = 16 (int + vptr)
area = 9

```

Read the numbers. Plain is one int = 4 bytes. Virtual is one int **plus one hidden 8-byte pointer**, which the compiler then pads to a multiple of 8, giving 16. The vptr is not imaginary; you can measure it.

TRAP

Always make the destructor of a polymorphic base class virtual. If `Shape::~~Shape` is not virtual and you write `delete p;` where `p` is a `Shape*` pointing at a `Circle`, only `~Shape` runs. `Circle`'s members are never destroyed — a leak, and formally undefined behaviour. One-line answer: **“a base class with any virtual function needs a virtual destructor.”**

WHAT THE INTERVIEWER EXPECTS TO HEAR — JAVA / C++

Say these four sentences and the C++ virtual-function question is closed.

1. “A virtual function is resolved at run time using the object’s actual type, not the pointer’s static type.”
2. “The compiler gives each polymorphic class a vtable of function pointers and gives each object a hidden vptr to it.”
3. “A **pure virtual** function is declared `virtual T f() = 0;`. A class with one is **abstract** and cannot be instantiated.”
4. “The base destructor must be virtual, or deleting through a base pointer is undefined behaviour.”

And for Java: **“In Java every non-static, non-final, non-private method is virtual by default, so there is no virtual keyword. final is how you opt out.”**

JS honest answer: **“JavaScript has no vtable — every method call is a prototype-chain lookup, so every method behaves as if it were virtual.”**

11.9 Abstract class vs interface — the highest-frequency question

The idea. Both say “you must supply this method”. They differ in what else they may carry and how many you may have.

	Abstract class	Interface
Purpose	share partial implementation among related classes	declare a capability any class may have
Relationship	“is-a”	“can-do”
Instance fields / state	yes	no (only <code>public static final</code> constants in Java)
Constructor	yes (runs via <code>super()</code>)	no
Concrete methods	yes	Java 8+: <code>default</code> and <code>static</code> methods only
How many per class	one	many
Keyword	<code>extends</code>	<code>implements</code>
Access of members	any	implicitly <code>public</code>
C++ equivalent	base with some pure virtuals	base with all pure virtuals, no data

Java — both, in one file, compiled and run

JAVA

```
abstract class Shape {
    protected String name;
    Shape(String name) { this.name = name; }    // abstract class CAN have a
constructor                                     // no body -> subclass must supply
    abstract double area();
```

```

one
    public String describe() { return name + " area=" + area(); } // concrete method
}
interface Drawable { void draw(); } // pure contract
interface Serial { default String tag() { return "obj"; } } // Java 8 default
method

class Square extends Shape implements Drawable, Serial {
    private final double s;
    Square(double s) { super("square"); this.s = s; }
    @Override double area() { return s * s; }
    @Override public void draw() { System.out.println("[]"); }
}

public class Shapes {
    public static void main(String[] a) {
        Shape sh = new Square(3); // upcast: reference is Shape, object is
        Square
        System.out.println(sh.describe());
        ((Drawable) sh).draw();
        System.out.println(((Serial) sh).tag());
        // Shape x = new Shape("x"); // ERROR: Shape is abstract
    }
}

```

Compiled with javac, run with java. Output:

square area=9.0

[]

obj

Four facts this proves:

1. An abstract class **can** have a constructor, fields, and fully written methods (describe() here). It just cannot be instantiated.
2. describe() calls area(), which the abstract class never defines. At run time Square.area() runs. That is polymorphism reaching **down** from the abstract layer.
3. One extends, two implements, in the same declaration. That is Java's multiple inheritance of type.
4. A default method in an interface gives a body without state.

When to choose which — the follow-up

- Shared **state** or shared **code**? Abstract class.
- A capability that unrelated classes can each have (Comparable, Serializable, Closeable)? Interface.
- Need it in more than one family at once? Interface — you get only one parent class.
- Adding a method later to an already-shipped type? Interface with a default body, or the existing implementors all break.

And in JavaScript?

JS has neither keyword. You emulate an abstract class with new.target.

Emulating an abstract class in JS

JS

```

class Shape {
    constructor() {
        if (new.target === Shape) throw new Error("Shape is abstract");
    }
    area() { throw new Error(`${this.constructor.name} must implement area()`); }
}

```

```

class Tri extends Shape { constructor(b,h){ super(); this.b=b; this.h=h; } area()
{ return this.b*this.h/2; } }
class Blob extends Shape {}

try { new Shape(); } catch (e) { console.log("1:", e.message); }
console.log("2:", new Tri(6, 4).area());
try { new Blob().area(); } catch (e) { console.log("3:", e.message); }

```

Verified output:

1: Shape is abstract

2: 12

3: Blob must implement area()

Check: triangle area = $\frac{b \times h}{2} = \frac{6 \times 4}{2} = 12$. Correct.

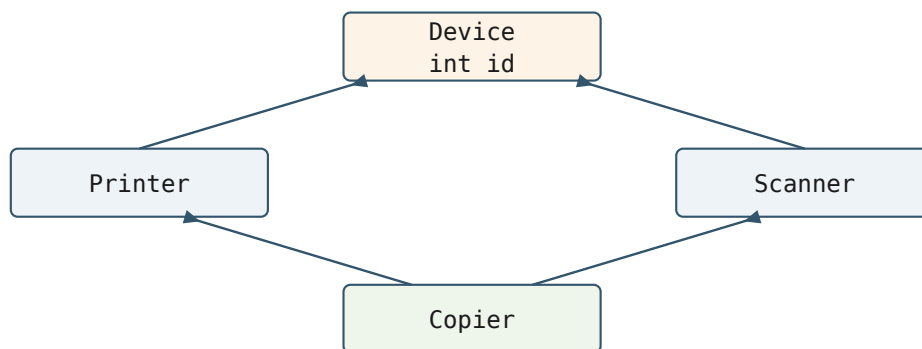
`new.target` is the constructor that `new` was actually called with. When you write `new Tri(...)`, inside `Shape`'s constructor `new.target` is `Tri`, not `Shape`, so the guard passes.

TRAP

The JS version fails **at run time**, and only when that exact line executes. Java fails **at compile time** — the build does not produce a class file at all. When asked “is JS object oriented?”, the sharp answer is: **“Yes, it is object oriented, but it is dynamically typed and prototype-based, so contract violations surface at run time instead of compile time. TypeScript adds the compile-time check back.”**

11.10 The diamond problem

The setup. `Device` has a field `id`. `Printer` and `Scanner` each inherit from `Device`. `Copier` inherits from both. How many `Device` parts does a `Copier` contain — one or two?



Question: does `Copier` hold one `Device` or two? C++ says **two** by default. Java forbids the shape entirely.

The diamond: two inheritance paths from `Copier` reach `Device`

C++ — it really does make two copies

Plain multiple inheritance: two `Device` sub-objects

CPP

```

#include <iostream>
struct Device { int id = 7; Device() { std::cout << "Device ctor\n"; } };
struct Printer : Device {};
struct Scanner : Device {};
struct Copier : Printer, Scanner {};
int main() {
    Copier c;
}

```



```
std::cout << sizeof(Copier) << " bytes\n";
// std::cout << c.id; // ERROR: request for member 'id' is ambiguous
std::cout << c.Printer::id << " " << c.Scanner::id << "\n";
}
```

Compiled and run. Actual output:

```
Device ctor
Device ctor
8 bytes
7 7
```

Read it carefully. Device ctor prints **twice**. sizeof(Copier) is **8** — two ints, not one. And plain c.id does not compile at all; the compiler says the request is ambiguous. You must disambiguate with c.Printer::id.

The C++ fix: virtual inheritance

One shared Device

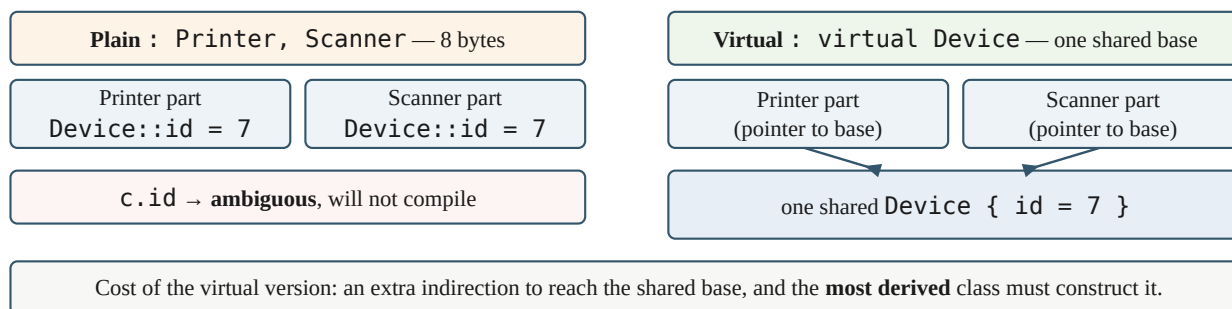
C++

```
#include <iostream>
struct Device { int id = 7; Device() { std::cout << "Device ctor\n"; } };
struct Printer : virtual Device {};
struct Scanner : virtual Device {};
struct Copier : Printer, Scanner {};
int main() {
    Copier c;
    std::cout << c.id << "\n"; // now unambiguous: ONE Device
}
```

Verified output:

```
Device ctor
7
```

One constructor call. One id. c.id compiles.



Copier memory layout: plain multiple inheritance vs virtual inheritance

Java — the shape is banned, but a smaller diamond survives

Java forbids class C extends A, B. So the state diamond cannot happen. But Java 8 added default methods to interfaces, and two interfaces **can** supply the same default.

Java's interface diamond and its fix

JAVA

```
interface A { default String who() { return "A"; } }
interface B { default String who() { return "B"; } }
class C implements A, B {
```

```
// Without this override javac refuses to compile.
@Override public String who() { return A.super.who() + B.super.who(); }
}
public class Dia { public static void main(String[] x){ System.out.println(new
C().who()); } }
```

With the override, output is AB. Remove the override and javac prints, word for word:

error: types A and B are incompatible;

class C inherits unrelated defaults for who() from types A and B

So Java's rule is: **the compiler will not guess — you must override and pick, using `A.super.who()` to name the one you want.**

TRAP

“Java avoids the diamond problem by not allowing multiple inheritance” is only **half** the answer and the panel is waiting for the other half. Full answer: **“Java forbids multiple inheritance of classes, so there is no state diamond. Since Java 8, two interfaces can both supply a default method, which creates a behaviour diamond — the compiler then forces the class to override and disambiguate with `A.super.method()`.”**

JavaScript — mixins, the flat answer

JS also allows only one extends. The idiom is a **mixin**: a function that takes a class and returns a subclass.

Mixins: multiple capabilities, one straight chain

JS

```
const CanPrint = (Base) => class extends Base {
  print() { return `printing ${this.id}`; }
};
const CanScan = (Base) => class extends Base {
  scan() { return `scanning ${this.id}`; }
};

class Device { constructor(id) { this.id = id; } }
class Copier extends CanScan(CanPrint(Device)) {}

const c = new Copier("C-100");
console.log(c.print());
console.log(c.scan());
console.log(c instanceof Device); // true
```

Verified output:

```
printing C-100
scanning C-100
true
```

There is no diamond because there is no branching. The chain is a straight line:

```
Copier -> CanScan(...) -> CanPrint(...) -> Device
```

Whoever is applied **last** is **closest** to the object, so if two mixins defined the same method, the outer one wins. That is a deterministic rule, which is exactly what C++ and Java lack.

11.11 static, final and the modifiers

static belongs to the class, not to any object

JS

```

class Counter {
  static made = 0; // one copy, on the class
  constructor() { Counter.made++; this.id = Counter.made; }
  static reset() { Counter.made = 0; }
  who() { return `instance ${this.id}`; }
}
new Counter(); new Counter(); const t = new Counter();
console.log(Counter.made, t.who()); // 3 instance 3
console.log(typeof t.reset); // undefined - static is NOT on instances
Counter.reset();
console.log(Counter.made); // 0

```

Verified output: 3 instance 3, undefined, 0.

`t.reset` is undefined. Static members live on the class object, not on instances, and they are **not** reachable through the instance in JavaScript (unlike Java, where `t.reset()` compiles with a warning).

Keyword	On a method	On a field / class
static (Java/C++)	no <code>this</code> ; called on the class	one shared copy for all objects
final (Java)	cannot be overridden	field: assign once. Class: cannot be extended.
const (C++)	method promises not to modify the object	value cannot change
abstract (Java)	no body; subclass must supply one	class cannot be instantiated
virtual (C++)	dispatch through the vtable	—

TRAP

`final` on a **reference** field freezes the **reference**, not the object. In Java, `final List<String> xs = new ArrayList<>(); xs.add("a");` is perfectly legal — you cannot reassign `xs`, but you can mutate what it points at. Same story for `const` in JavaScript: `const a = [1]; a.push(2);` works. `const` binds the name, not the value.

11.12 Association, aggregation, composition

Three words for “class A uses class B”, ordered by how tightly.

	Association	Aggregation	Composition
Meaning	uses-a	has-a, weak	has-a, strong / owns
Lifetime	independent	part outlives the whole	part dies with the whole
Example	Teacher — Student	Department — Professor	House — Room
UML line	plain line	hollow diamond	filled diamond
Test question	do they just talk?	can the part be reused elsewhere?	is the part meaningless alone?

Composition: the Copier owns its parts and can swap them

JS

```

class PrintUnit { print(doc) { return `printed ${doc}`; } }
class ScanUnit { scan(doc) { return `scanned ${doc}`; } }

class Copier {
  constructor() { this.printer = new PrintUnit(); this.scanner = new ScanUnit(); }
  print(d) { return this.printer.print(d); } // delegate
  scan(d) { return this.scanner.scan(d); }
}

```

```

    copy(d) { this.scanner.scan(d); return this.printer.print(d); }
  }
  const c = new Copier();
  console.log(c.copy("page1"));
  console.log(c.scan("page2"));
  c.printer = { print: (d) => `faxed ${d}` }; // swap a part at RUN TIME
  console.log(c.copy("page3"));

```

Verified output:

```

printed page1
scanned page2
faxed page3

```

That last line is the argument for composition. You replaced a component while the program was running. With inheritance, the parent is fixed at compile time for ever.

11.13 SOLID — five principles, one worked example each

You will be asked to expand the acronym and give one example. Keep each answer to two lines.

S — Single Responsibility

A class should have one reason to change. If Invoice both computes totals and writes PDFs, a tax-rule change and a layout change both edit the same file. Split into Invoice (numbers) and InvoiceRenderer (output).

O — Open / Closed

Open for extension, closed for modification. Adding a case should not mean editing a switch.

Add a shipping rule without touching cost()

JS

```

const rules = new Map();
const register = (kind, fn) => rules.set(kind, fn);

register("standard", w => 40 + 8 * w);
register("express", w => 90 + 14 * w);

function cost(kind, weightKg) {
  const fn = rules.get(kind);
  if (!fn) throw new Error("unknown kind: " + kind);
  return fn(weightKg);
}
console.log(cost("standard", 3), cost("express", 3));

register("sameday", w => 200 + 20 * w); // extension, no edit to cost()
console.log(cost("sameday", 3));

```

Verified output: 64 132 then 260.

Check each: standard = $40 + 8 \times 3 = 64$. Express = $90 + 14 \times 3 = 132$. Same-day = $200 + 20 \times 3 = 260$. All correct.

L — Liskov Substitution

Anywhere the parent works, the child must work too. The famous counter-example is real, and it compiles.

Square extends Rectangle breaks the caller

JS

```

class Rectangle {
  constructor(w, h) { this.w = w; this.h = h; }
  setWidth(w) { this.w = w; }
  setHeight(h) { this.h = h; }
  area() { return this.w * this.h; }
}
class Square extends Rectangle { // "a square IS-A rectangle" -- in maths,
  yes
  setWidth(w) { this.w = w; this.h = w; }
  setHeight(h) { this.w = h; this.h = h; }
}
function stretch(r) { r.setWidth(5); r.setHeight(4); return r.area(); }
console.log(stretch(new Rectangle(1, 1))); // 20 - expected
console.log(stretch(new Square(1))); // 16 - caller's contract broken

// Fix: stop inheriting. Model the shared CAPABILITY instead.
class Shape2 { area() { throw new Error("not implemented"); } }
class Rect2 extends Shape2 { constructor(w,h){super();this.w=w;this.h=h;} area()
{return this.w*this.h;} }
class Sq2 extends Shape2 { constructor(s){super();this.s=s;} area(){return
this.s*this.s;} }
console.log([new Rect2(5,4), new Sq2(4)].map(s => s.area()));

```

Verified output: 20, 16, [20, 16].

Trace the failure. `stretch` sets width 5 then height 4, and expects $5 \times 4 = 20$. On a Square, `setHeight(4)` also resets the width to 4, so the answer is $4 \times 4 = 16$. The child is not substitutable. **“Is-a” in mathematics is not “is-a” in code** — the deciding question is whether the child honours every promise the parent’s callers rely on.

I — Interface Segregation

No client should be forced to depend on methods it does not use. One fat Machine interface with `print`, `scan`, `fax` forces a plain printer to implement `fax()` with a thrown error. Split into `Printer`, `Scanner`, `Fax` and let a class implement the ones it truly has.

D — Dependency Inversion

Depend on abstractions, not concrete classes. `OrderService` should take a `Repository` (the contract) in its constructor, not construct a `MySQLRepository` inside itself. Then tests pass a fake one. That is exactly the `MemoryRepo` example from section 11.3.

SHORTCUT

Say SOLID in **five short sentences**, one per letter, then offer one example for the letter they ask about. Reciting long definitions for all five wastes the interviewer’s time and they will cut you off. Short first, detail on demand.

11.14 Shallow vs deep copy

The spread operator copies one level only

JS

```

const orig = { name: "cart", items: [{ sku: "A", qty: 1 }] };
const shallow = { ...orig };
shallow.items[0].qty = 99;
console.log("orig after shallow edit:", orig.items[0].qty); // 99 - shared!

const deep = structuredClone(orig);

```

```
deep.items[0].qty = 5;
console.log("orig after deep edit:", orig.items[0].qty); // 99 - untouched
```

Verified output:

```
orig after shallow edit: 99
orig after deep edit: 99
```

The first 99 is the bug: `{...orig}` copied the **reference** to items, so both objects point at the same array. The second 99 is the proof that `structuredClone` worked — after the deep copy was edited to 5, the original still reads 99.

	Shallow copy	Deep copy
Copies	top-level fields only	the whole object graph
Nested objects	shared	duplicated
JS	<code>{...o}</code> , <code>Object.assign</code> , <code>arr.slice()</code>	<code>structuredClone(o)</code>
Java	default <code>Object.clone()</code>	hand-written, or serialise-and-read-back
C++	default copy constructor (copies the pointer)	user-defined copy ctor that copies the target
Cost	$O(1)$ in depth	$O(n)$ in the size of the graph

TRAP

In C++ this is the **rule of three**: if a class manages a raw resource and you do not write your own copy constructor, copy assignment and destructor, the default shallow copy gives two objects holding the same pointer — and the second destructor calls `delete` on already-freed memory. A double free. Modern C++ answer: use `std::unique_ptr` / `std::vector` and the rule of **zero**.

11.15 The `this` problem — a JS-only interview topic

'this' is decided by the CALL, not by the class

JS

```
class Timer {
  constructor() { this.ticks = 0; }
  tick() { this.ticks++; return this.ticks; }
}

const t = new Timer();
const loose = t.tick; // method pulled off the object
try { loose(); } catch (e) { console.log("broken:", e.constructor.name); }

const bound = t.tick.bind(t); // fix 1: bind
console.log(bound(), bound());

class Timer2 { // fix 2: arrow field captures 'this' at
  construction
    ticks = 0;
    tick = () => { this.ticks++; return this.ticks; };
}
const g = new Timer2().tick;
console.log(g(), g());
```

Verified output:

broken: TypeError

1 2

1 2

In Java and C++, this is part of the call — you cannot detach a method from its object. In JS you can, and then `this` is undefined (class bodies are strict mode), so `this.ticks++` throws a `TypeError`. This is why `onClick={this.handleClick}` in old React needed `.bind(this)` in the constructor.

11.16 Worked examples — Warm-up

WARM-UP

build the reflex

Example 1

Name the four pillars of OOP and give each one a three-word description.

SOLUTION

Encapsulation — hide the data.

Abstraction — hide the implementation.

Inheritance — reuse the parent.

Polymorphism — one name, many behaviours.

Answer: Encapsulation, Abstraction, Inheritance, Polymorphism.

Example 2

What is printed?

```
class A { hi() { return "A"; } }  
class B extends A { hi() { return "B"; } }  
console.log(new B().hi());
```

SOLUTION

B overrides hi. Lookup starts on the object, then B.prototype — found there, so the walk stops before reaching A.prototype.

Answer: B

Example 3

True or false: a class with a private constructor can still be instantiated from inside itself.

SOLUTION

True. private restricts access from **outside** the class. A static factory method inside the class can call the private constructor. That is exactly how the singleton pattern works.

Answer: True.

Example 4

In one line: what is the difference between extends and implements in Java?

SOLUTION

extends takes the parent's code and state — one parent only. implements takes only the method signatures — any number.

Answer: extends = inherit implementation (one); implements = accept a contract (many).

Example 5

What does this print, and why?

```
class Box { static count = 0; }  
const b = new Box();  
console.log(Box.count, b.count);
```

SOLUTION

count lives on the class object Box, not on b. Reading b.count walks b, then Box.prototype, then Object.prototype — count is on none of them.

Answer: 0 undefined

11.17 Worked examples — Tier 1

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 6 [TCS NQT · pattern]

Explain the difference between method overloading and method overriding with one example each. State when each is resolved.

SOLUTION

Overloading — same name, **different** parameter list, **same** class, resolved at **compile time**.

```
int add(int a, int b)           { return a + b; }
int add(int a, int b, int c)    { return a + b + c; }
```

Overriding — same name, **same** signature, **child** class, resolved at **run time**.

```
class Shape { double area() { return 0; } }
class Circle extends Shape { @Override double area() { return 3.14 * r * r; } }
```

Overloading is static (compile-time) polymorphism. Overriding is dynamic (run-time) polymorphism.

Answer: Overloading: different signature, same class, compile time. Overriding: same signature, child class, run time.

Example 7 [Infosys · pattern]

Can you override a static method in Java? What actually happens if you declare a method with the same signature static in both parent and child?

SOLUTION

No. That is **method hiding**, not overriding.

```
class P { static String f() { return "P"; } }
class C extends P { static String f() { return "C"; } }
P ref = new C();
System.out.println(ref.f()); // prints P
```

Static dispatch uses the **declared** type of the reference (P), not the object's actual type. @Override on such a method is a compile error, which is the compiler telling you the same thing.

Answer: No — it is hidden, not overridden. The declared type decides which one runs.

Example 8 [Wipro · pattern]

List four differences between an abstract class and an interface, and say when you would choose each.

SOLUTION

1. **Count:** one abstract parent, many interfaces.
2. **State:** an abstract class may hold instance fields; an interface may not.
3. **Constructor:** an abstract class has one; an interface does not.
4. **Relationship:** abstract class means “is-a”; interface means “can-do”.

Choose an **abstract class** when subclasses share code or state — e.g. Employee holding name and id with an abstract calculateSalary().

Choose an **interface** when unrelated classes share a capability — e.g. Comparable, which a Student, an Invoice and a Date may all implement without being related.

Answer: Count, state, constructor, relationship. Abstract class for shared code; interface for shared capability.

Example 9 [Capgemini · pattern]

What is the diamond problem? Which languages have it and how does each deal with it?

SOLUTION

Diamond: D inherits from B and C, both of which inherit from A. If A has a member, D may end up with two copies of it, and a call to `d.m()` is ambiguous.

- **C++** — the problem exists. By default D gets **two** A sub-objects; a plain `d.a` is a compile error and you must write `d.B::a`. The fix is `class B : virtual public A`, which makes all paths share one A.
- **Java** — forbids extends from two classes, so the **state** diamond cannot occur. Since Java 8 a **behaviour** diamond can, when two interfaces supply the same default method; the compiler then forces the class to override and pick with `B.super.m()`.
- **JavaScript** — one extends only. Multiple capabilities come from mixins, which produce a straight chain, so the last mixin applied wins by a clear rule.

Answer: Ambiguity from two inheritance paths. C++: virtual inheritance. Java: banned for classes, override required for default methods. JS: mixins, no branching.

Example 10 [Cognizant · pattern]

A junior writes this and reports “the balance went negative”. Fix it and name the principle.

```
class Account {
  constructor() { this.balance = 0; }
}
const a = new Account();
a.balance -= 900;
```

SOLUTION

The field is public, so any line anywhere can break the invariant “balance is never negative”. The principle is **encapsulation**.

```
class Account {
  #balance = 0;
  get balance() { return this.#balance; }
  deposit(amt) {
    if (amt <= 0) throw new Error("deposit must be positive");
    this.#balance += amt;
  }
  withdraw(amt) {
    if (amt <= 0) throw new Error("withdrawal must be positive");
    if (amt > this.#balance) throw new Error("insufficient funds");
    this.#balance -= amt;
  }
}
```

Now the rule lives in **one** place. There is no path to the field that skips the check.

Answer: Make the field private and route every change through methods that validate. Principle: encapsulation.

Example 11 [Accenture · pattern]

What is a constructor? Can it be private, static, final, or inherited?

SOLUTION

A constructor is a special member that runs when an object is created and initialises its state. It has the class's name and no return type.

private	yes	used for singletons and factory methods
static	no	it acts on the new object, so it needs an implicit <code>this</code>
final	no	<code>final</code> blocks overriding, and constructors are never inherited anyway
inherited	no	but the child's constructor calls the parent's via <code>super()</code>

Answer: Can be private. Cannot be static or final. Never inherited — only called through `super()`.

Example 12 [TCS Digital · pattern]

What is printed?

```
class A {
  constructor() { this.greet(); }
  greet() { console.log("A"); }
}
class B extends A {
  constructor() { super(); this.name = "bee"; }
  greet() { console.log("B, name =", this.name); }
}
new B();
```

SOLUTION

1. `new B()` runs B's constructor. `super()` runs first.
2. A's constructor calls `this.greet()`. Dispatch is dynamic, and the object is a B, so B.greet runs.
3. `this.name` has not been assigned yet — that line comes **after** `super()`.

Output: B, name = undefined.

Same lesson as the Java version in 11.5, with undefined instead of `0` because JS has no field defaults.

Answer: B, name = undefined — never call an overridable method from a constructor.

11.18 Worked examples — Tier 2

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 13 [Grab · pattern]

A ride-hailing app prices trips. Today: Bike, Car, Premium. Next month, marketing wants Pooled and a surge multiplier that applies to some types but not others. The current code is:

```
function fare(type, km) {
  if (type === "bike")    return 15 + 6 * km;
  if (type === "car")     return 40 + 11 * km;
  if (type === "premium") return 90 + 18 * km;
  throw new Error("unknown");
}
```

Redesign it. Name the principles you applied.

SOLUTION

Two problems: every new type edits `fare` (violates Open/Closed), and surge has nowhere to live without more branches.

Make each type an object that knows its own base fare, and make surge a **decorator** that wraps any pricer. That is composition, not inheritance.

```
class Pricer { constructor(base, perKm) { this.base = base; this.perKm = perKm; }
  quote(km) { return this.base + this.perKm * km; } }

class Surge { // wraps ANY pricer
  constructor(inner, mult) { this.inner = inner; this.mult = mult; }
  quote(km) { return +(this.inner.quote(km) * this.mult).toFixed(2); }
}

const catalog = new Map([
  ["bike", new Pricer(15, 6)],
  ["car", new Pricer(40, 11)],
  ["premium", new Pricer(90, 18)],
  ["pooled", new Pricer(25, 7)], // new type: one line
]);

function fare(type, km, mult = 1) {
  const p = catalog.get(type);
  if (!p) throw new Error("unknown type: " + type);
  return mult === 1 ? p.quote(km) : new Surge(p, mult).quote(km);
}
```

Verified with node for a 8 km trip:

type	base + perKm × 8	normal	× 1.5 surge
bike	15 + 6 × 8	63	94.5
car	40 + 11 × 8	128	192
premium	90 + 18 × 8	234	351
pooled	25 + 7 × 8	81	121.5

Check one by hand: premium = 90 + 144 = 234; 234 × 1.5 = 351. Correct.

Principles: Open/Closed (new type = new map entry, fare untouched), Single Responsibility (Pricer computes, Surge multiplies), and composition over inheritance (Surge wraps rather than extends, so it works for types invented later).

Answer: Replace the type branch with a registry of pricer objects; add surge as a wrapper. Open/Closed + Single Responsibility + composition.

Example 14 [Shopee · pattern]

Your codebase has class AdminUser extends User and class GuestUser extends User. A product manager now asks for a user who is an admin **and** can be temporarily downgraded to guest permissions at run time. What is wrong with the inheritance design, and what replaces it?

SOLUTION

What is wrong. Inheritance is fixed when the object is constructed. A JS object's class cannot be changed afterwards in any sane way, so “temporarily downgraded” is impossible. And “admin AND guest” would need multiple inheritance, which JS does not have.

What replaces it. The varying part is not **what the user is**, it is **what the user may do**. Model permissions as a value the user **has**, not a class the user **is**.

```
class User {
  constructor(name, role) { this.name = name; this.role = role; }
  can(action) { return this.role.allows(action); }
}
```

```

class Role {
    constructor(name, actions) { this.name = name; this.actions = new Set(actions); }
    allows(a) { return this.actions.has(a); }
}
const ADMIN = new Role("admin", ["read", "write", "delete", "invite"]);
const GUEST = new Role("guest", ["read"]);

const u = new User("Nadia", ADMIN);
console.log(u.can("delete")); // true
u.role = GUEST; // temporary downgrade, one assignment
console.log(u.can("delete")); // false
u.role = ADMIN;
console.log(u.can("delete")); // true

```

Verified output: true, false, true.

This is the **Strategy** pattern. The general rule: **if a subclass differs only in a value or a behaviour that might change during the object's life, it should be a field, not a subclass.**

Also note the count. Three roles and two flags would need $2^2 \times 3 = 12$ subclasses to cover every combination. With composition it stays at one User class.

Answer: Inheritance fixes the type at construction. Replace AdminUser/GuestUser with a role field holding a Role object — the Strategy pattern.

Example 15 [Agoda · pattern]

Explain what this C++ code prints and why, then say what a Java developer would expect.

```

struct Base { void show() { std::cout << "Base\n"; } };
struct Derived : Base { void show() { std::cout << "Derived\n"; } };
int main() {
    Base* p = new Derived();
    p->show();
    delete p;
}

```

SOLUTION

It prints Base.

show() is not virtual. Without virtual, C++ binds the call at compile time using the **static** type of p, which is Base*. The object really is a Derived, but the compiler never looks.

A Java developer expects Derived, because in Java **every** instance method is virtual by default and dispatch always uses the object's real type. This is the single biggest mental switch between the two languages.

The fix is one word:

```

struct Base { virtual void show() { std::cout << "Base\n"; } virtual ~Base() = default; };
struct Derived : Base { void show() override { std::cout << "Derived\n"; } };

```

Now it prints Derived.

Second bug in the original: ~Base is not virtual either, so delete p runs only ~Base. With real members in Derived, that leaks. Both bugs are fixed by the same habit — mark the base class's overridable methods and its destructor virtual.

Answer: Prints Base — non-virtual calls bind to the static pointer type. Java would print Derived. Fix: virtual, and a virtual destructor.

Example 16 [DBS · pattern]

A payments service has this interface. A new client, “internal transfers”, needs settle but has no card and no refund flow. What principle is violated and how do you fix it?

```
interface PaymentMethod {
  authorize(amount)
  capture(id)
  refund(id, amount)
  tokenizeCard(pan)
  settle(batchId)
}
```

SOLUTION

Violated: Interface Segregation — “no client should be forced to depend on methods it does not use.” An internal-transfer class would have to implement `tokenizeCard` and `refund` with thrown errors, and every caller then has to guess which methods are real.

It also breaks **Liskov**: a method that throws “not supported” is not a valid substitute for one that works.

Fix: split by capability and let each class implement only what it truly has.

```
interface Authorizable { authorize(amount); capture(id) }
interface Refundable   { refund(id, amount) }
interface Cardable     { tokenizeCard(pan) }
interface Settleable   { settle(batchId) }
```

```
CardPayment      implements Authorizable, Refundable, Cardable, Settleable
InternalTransfer implements Authorizable, Settleable
```

Now the type system itself answers “can this method be refunded?” — there is nothing to throw and nothing to check at run time.

Answer: Interface Segregation (and Liskov). Split the fat interface into capability interfaces; each class implements only the ones it supports.

11.19 Worked examples — Tier 3

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 17 [Google · pattern]

Design the class model for a document editor that supports plain text, images and tables, where any element may be nested inside a table cell, and where the editor must support undo of every edit. Name the patterns and justify each inheritance-versus-composition choice.

SOLUTION

Step 1 — the nesting requirement decides the shape. “Any element may be nested inside a table cell” means a table cell holds a **list of elements**, and a table is itself an element. That is the **Composite** pattern: leaves and containers share one interface.

```
class Element { // abstract in spirit
  render() { throw new Error("must implement render()"); }
  size()   { throw new Error("must implement size()"); }
}
class TextRun extends Element {
  constructor(s) { super(); this.s = s; }
  render() { return this.s; }
  size()   { return this.s.length; }
}
class Image extends Element {
  constructor(src, w, h) { super(); Object.assign(this, { src, w, h }); }
  render() { return `img ${this.w}x${this.h}`; }
```

```

    size() { return 1; }
}
class Cell extends Element {
  constructor(children = []) { super(); this.children = children; }
  render() { return this.children.map(c => c.render()).join(""); }
  size() { return this.children.reduce((n, c) => n + c.size(), 0); }
}
class Table extends Element {
  constructor(rows) { super(); this.rows = rows; } // rows: Cell[][]
  render() { return this.rows.map(r => r.map(c => c.render()).join(" | ")).join("\n"); }
  size() { return this.rows.flat().reduce((n, c) => n + c.size(), 0); }
}

```

Inheritance is correct here because a `Table` genuinely **is-a** `Element` — it is used interchangeably wherever an element is expected, and it honours the whole contract. Liskov holds.

Step 2 — undo is a separate axis. Do not put `undo()` on `Element`. Undo is about **edits**, not about **content**. That is the **Command** pattern: each edit is an object that knows how to apply itself and how to reverse itself.

```

class InsertCmd {
  constructor(parent, index, node) { Object.assign(this, { parent, index, node }); }
  do() { this.parent.children.splice(this.index, 0, this.node); }
  undo() { this.parent.children.splice(this.index, 1); }
}
class DeleteCmd {
  constructor(parent, index) { Object.assign(this, { parent, index }); this.removed = null; }
  do() { this.removed = this.parent.children.splice(this.index, 1)[0]; }
  undo() { this.parent.children.splice(this.index, 0, this.removed); }
}
class Editor {
  constructor(root) { this.root = root; this.done = []; this.undone = []; }
  run(cmd) { cmd.do(); this.done.push(cmd); this.undone.length = 0; return this; }
  undo() { const c = this.done.pop(); if (c) { c.undo(); this.undone.push(c); }
return this; }
  redo() { const c = this.undone.pop(); if (c) { c.do(); this.done.push(c); } return this; }
}

```

Verified run:

```

const cell = new Cell([new TextRun("hello ")]);
const doc = new Cell([cell]);
const ed = new Editor(doc);
ed.run(new InsertCmd(cell, 1, new TextRun("world")));
console.log(doc.render(), "| size", doc.size()); // hello world | size 11
ed.undo();
console.log(doc.render(), "| size", doc.size()); // hello | size 6
ed.redo();
console.log(doc.render(), "| size", doc.size()); // hello world | size 11

```

Count it: "hello " is 6 characters, "world" is 5, total 11. After undo, 6. Correct.

Step 3 — the justifications the interviewer is grading.

Decision	Choice	Why
Table vs Element	inheritance	true is-a; used wherever an <code>Element</code> is; Liskov holds
Cell holds children	composition	a cell has elements; the set changes at run time

undo	composition (Command)	undo varies per edit , not per element ; putting it on Element would need a subclass per edit type
Editor holds two stacks	composition	history is state, not identity

Step 4 — the follow-up they will ask: “what breaks at scale?”

- DeleteCmd holds a reference to the removed subtree, so the undo stack pins memory. Cap the stack, or store a serialised patch instead of the live node.
- Deep nesting makes `size()` recursive; a 10,000-level document overflows the JS stack. Cache the size on each container and invalidate upwards on edit.
- Commands hold direct object references, so any edit that replaces a parent invalidates them. In a collaborative editor you would move to positions expressed as paths, plus operational transformation or CRDTs.

Answer: Composite for the element tree (inheritance — true is-a), Command for edits (composition — undo varies per edit, not per element), stacks for history. Scale risks: memory pinned by the undo stack, recursion depth, stale node references.

Example 18 [Amazon · pattern]

You are told: “Our Notification base class has 14 subclasses — EmailNotification, SmsEmailNotification, SmsWhatsappRetryNotification, and so on. Adding a channel means adding seven classes.” Diagnose it and give the refactor, with the class count before and after.

SOLUTION

Diagnosis: a combinatorial class explosion from using inheritance for two independent axes.

The axes are:

- **channel** — email, SMS, WhatsApp, push (4 options)
- **policy** — retry, no retry (2 options)

With inheritance, one class must name a point in the product space, so you need $4 \times 2 = 8$ classes; add “digest vs immediate” and it becomes $4 \times 2 \times 2 = 16$. Each new channel multiplies, it does not add.

Refactor — one axis becomes composition. Channels become interchangeable **senders**. Retry becomes a **decorator** that wraps any sender.

```
class EmailSender { send(m) { return `email:${m}`; } }
class SmsSender { send(m) { return `sms:${m}`; } }
class WhatsappSender { send(m) { return `wa:${m}`; } }
class PushSender { send(m) { return `push:${m}`; } }

class Retrying { // wraps ANY sender
  constructor(inner, attempts) { this.inner = inner; this.attempts = attempts; }
  send(m) {
    let last;
    for (let i = 1; i <= this.attempts; i++) {
      try { return this.inner.send(m); } catch (e) { last = e; }
    }
    throw last;
  }
}

class Fanout { // wraps MANY senders
  constructor(senders) { this.senders = senders; }
  send(m) { return this.senders.map(s => s.send(m)); }
}
```

Now SmsWhatsappRetryNotification is not a class at all — it is an expression:

```
const channel = new Retrying(new Fanout([new SmsSender(), new WhatsappSender()]), 3);
console.log(channel.send("otp 4821")); // [ 'sms:otp 4821', 'wa:otp 4821' ]
```

Verified output: ['sms:otp 4821', 'wa:otp 4821'].

The count.

	Inheritance	Composition
4 channels, retry on/off	8 classes	$4 + 1 = 5$
1. fanout combinations	$2^4 \times 2 = 32$	$4 + 1 + 1 = 6$
1. a 5th channel	64	7
Growth	multiplicative	additive

That table is the answer. **Inheritance composes badly because a class can only be one thing; objects compose well because you can hold as many as you like.**

Follow-up they will ask: “when is the 14-subclass design actually right?” When the axes are **not** independent — when EmailNotification and SmsNotification really do differ in behaviour that cannot be expressed as a parameter, and when there is exactly one axis. With one axis, inheritance is n classes and composition is n objects; the counts match and inheritance is simpler to read.

Answer: Combinatorial explosion from two independent axes on one inheritance hierarchy. Make channel a Sender object and retry/fanout decorators. Class count goes from multiplicative ($4 \times 2 = 8$, then 32, then 64) to additive (5, 6, 7).

Example 19 [Microsoft · pattern]

In C++, `struct A { virtual void f(); };` — explain exactly what happens in memory when you call `p->f()` through an `A*`, what it costs, and give one concrete case where making a method `virtual` is the **wrong** choice.

SOLUTION

What happens, in order.

1. The compiler gives class A one static, read-only table — the **vtable** — holding a pointer per virtual function, in a fixed slot order.
2. Every A object carries a hidden pointer, the **vptr**, set by the constructor to its class’s vtable. That is why `sizeof` grows by one pointer (measured: $4 \rightarrow 16$ bytes for `struct { int x; virtual void f(); }` on 64-bit, after padding).
3. `p->f()` compiles to: load `vptr` from the object; load slot k from the vtable; indirect-call that address. Two dependent loads, then a call whose target is unknown at compile time.

What it costs.

Memory	8 bytes per object (the <code>vptr</code>), plus one small table per class
Time	two dependent loads before the call; if the target varies, a branch-target mispredict
Optimisation	the real cost — the callee cannot be inlined, so constant propagation, loop unrolling and vectorisation all stop at the call boundary

Where virtual is the wrong choice — a concrete case.

An inner loop over a `std::vector<Shape*>` calling `s->area()` millions of times. The virtual call prevents inlining, so a one-line `return w * h;` costs a full call. Three better options:

- **Make the set of types closed and known.** Use `std::variant<Circle, Square>` plus `std::visit`, or a tagged union. The compiler sees all cases and can inline each one.
- **Use CRTP** — static polymorphism: `template <class D> struct Shape { double area(){ return static_cast<D*>(this)->areaImpl(); } };`. Dispatch is resolved at compile time; zero runtime cost.
- **Restructure the data.** If you must call one function per element, sort or bucket the vector by concrete type and run one tight monomorphic loop per bucket. The call becomes predictable and the body inlines.

The honest trade-off sentence to say out loud: “virtual buys extensibility — new types with no change to calling code. It costs a pointer per object and blocks inlining. If the type set is closed and the call is hot, pay with variant or CRTP instead; if the type set is open, the virtual call is the right price.”

Answer: Two dependent loads (object → vptr → slot) then an indirect call; 8 bytes per object; the real cost is lost inlining. Wrong choice in a hot loop over a closed type set — use `std::variant` + `std::visit` or CRTP.

Example 20 [Adobe · pattern]

“JavaScript is not really object oriented — it has no classes.” Argue both sides, then give the answer you would actually give in an interview.

SOLUTION

The case for “not really”.

- There are no classes at run time. `class` is sugar; `typeof Shape` is `"function"` (verified in section 11.4), and the methods live on a plain object called `Shape.prototype`.
- No interfaces, no abstract keyword, no access levels beyond `public` and `#private`, no overloading, no generics enforced at run time.
- No compile-time type checking at all, so “contract” violations are discovered by a customer, not by a build.

The case for “yes it is”.

- OOP is defined by **encapsulation, abstraction, inheritance, polymorphism** — not by the keyword `class`. JS has all four: `#private` fields, duck-typed contracts, `extends` with `super`, and dynamic dispatch on every call.
- Prototypal inheritance is **more** general than class inheritance, not less: a class model can be built on prototypes (and is — that is exactly what `class` does), while the reverse is awkward.
- Objects can be extended, and their behaviour swapped, at run time. Composition and the Strategy pattern are cheaper in JS than in Java, not harder.

The answer to give.

“Yes, JavaScript is object oriented — it is **prototype-based** rather than **class-based**, and it is dynamically typed. It has all four pillars, but three mechanisms Java developers expect are missing: interfaces, method overloading, and compile-time enforcement. In practice teams add the first and third back with TypeScript, and replace the second with rest parameters. So the correct framing is not ‘is it OOP’, it is ‘it is OOP with run-time binding instead of compile-time binding’, and that changes where your errors show up — in tests and production rather than in the build.”

That last clause is what separates a good answer from a memorised one: it names the **consequence**, not just the difference.

Answer: Prototype-based and dynamically typed, but it has all four pillars. Missing: interfaces, overloading, compile-time checks. Consequence: contract errors surface at run time, which is why teams add TypeScript.

11.20 Practice

PRACTICE · WARM-UP target: 6 min

1. Expand: OOP, SOLID.
2. One line each: encapsulation vs abstraction.
3. Can an abstract class have a constructor? Can an interface?
4. Which is compile-time polymorphism — overloading or overriding?
5. In JS, `typeof MyClass` is what?
6. Name the four Java access modifiers, least to most open.
7. True/false: `final` on a Java list field prevents `list.add(...)`.

ANSWER KEY

1. Object Oriented Programming; Single responsibility, Open/closed, Liskov substitution, Interface segregation, Dependency inversion.
2. Encapsulation hides **data** (private fields + accessor methods); abstraction hides **implementation** (expose a contract).
3. Abstract class: yes. Interface: no.
4. Overloading.
5. "function".
6. **private**, default (package-private), **protected**, **public**.
7. False — it prevents **reassigning** the reference, not mutating the object.

PRACTICE · TIER 1 target: 14 min

1. Write a Java Shape hierarchy: abstract Shape with name and abstract `area()`; concrete Rectangle and Circle; a main that stores both in a `Shape[]` and prints each area. Say which line demonstrates run-time polymorphism.
2. Given class `P { void f(int x) {} }` and class `C extends P { void f(String s) {} }` — is `f` overloaded or overridden in `C`? What does `new C().f(3)` call?
3. List five rules that make a Java method override legal.
4. Explain why a **private** method cannot be overridden.
5. In JS, what is the difference between `Object.keys(o)` and a `for...in` loop over `o`?
6. Rewrite this to remove the type branch:

```
function area(s){ if(s.kind=="sq") return s.a*s.a; if(s.kind=="rect") return s.w*s.h; }
```

ANSWER KEY

1.

```
abstract class Shape { protected String name; Shape(String n){name=n;} abstract double area(); };
```


`Rectangle/Circle with @Override double area();` loop for `(Shape s : new Shape[]{ new Rectangle(3,4), new Circle(2) }) System.out.println(s.area());`. The polymorphic line is `s.area()` — `s` is declared `Shape`, the actual type decides.
2. **Overloaded** — the signature differs, so `C` has **both** methods (`f(int)` inherited, `f(String)` new). `new C().f(3)` calls the inherited `P.f(int)`.
3. Same name; same parameter list; return type same or covariant; access not more restrictive; may not throw broader **checked** exceptions. (And the method must not be **static**, **final** or **private**.)
4. A **private** method is not visible to the subclass, so the subclass's same-named method is a brand-new method, not an override. Dispatch for it is static.
5. `Object.keys` returns **own, enumerable, string** keys. `for...in` also walks the **prototype chain**. That is why `for...in` over an array or a class instance can surprise you.

6. Put the behaviour on the object: `const shapes = { sq: s => s.a*s.a, rect: s => s.w*s.h }; function area(s){ return shapes[s.kind](s); }` — or make them classes with an `area()` method.

PRACTICE · TIER 2 target: 18 min

1. A `Bird` class has `fly()`. `Penguin` extends `Bird`. Which principle breaks, and give two different fixes.
2. Design classes for a food-delivery order supporting: base price, optional packaging fee, optional peak-hour multiplier, optional coupon. Combinations are arbitrary. How many classes does inheritance need for 3 optional modifiers? How many does your design need?
3. You must add a `close()` method to an interface that 40 classes already implement, without breaking the build. How?
4. Explain to a Java developer, in three sentences, why this in JavaScript can be undefined inside a method.
5. A `Cache` class has `get`, `put`, `evict`, `serialize`, `renderStats`, `sendMetrics`. Which principle is violated? Split it.

ANSWER KEY

1. **Liskov** — `penguin.fly()` must either lie or throw, so a `Penguin` is not substitutable wherever a `Bird` is used. Fix A: move `fly()` off `Bird` into a `Flying` capability (interface/mixin) that `Penguin` does not take. Fix B: make `Bird` hold a `movement` strategy object and give `Penguin` a `Swimming` strategy.
2. Inheritance needs $2^3 = 8$ classes to cover every on/off combination. With decorators you need `BasePrice` + one class per modifier = 4, and any combination is built by wrapping: `new Coupon(new Peak(new Packaging(base)))`.
3. Give it a `default` body (Java 8+): `default void close() {}` — existing implementors inherit it and still compile; new ones can override it. This is the exact reason `default` methods were added to the language.
4. “In Java `this` is bound by the compiler and can never be missing. In JavaScript `this` is bound by **how the function is called**, so pulling a method off its object (`const f = obj.method`) loses the binding. Class bodies are strict mode, so the lost `this` is `undefined` rather than the global object — hence the `TypeError`.”
5. **Single Responsibility** — it stores, it serialises, it renders, and it talks to a metrics service; four reasons to change. Split into `Cache` (`get/put/evict`), `CacheSerializer`, `StatsFormatter`, `MetricsReporter`, and have the cache emit events the reporter subscribes to.

PRACTICE · TIER 3 target: 25 min

1. Explain object slicing in C++ with a three-line example, and say what the JS equivalent problem is (or why there is none).
2. You are designing a plugin system. Third parties ship classes you never see. Argue for interface-based design and name the exact cost you are paying versus a closed switch.
3. A team proposes `class Money extends Number`. Give three reasons to refuse.
4. Why does Java allow **covariant return types** on an override but not **contravariant parameter types**? Answer in terms of Liskov.

ANSWER KEY

1. `Base b = derivedObject;` — assigning a `Derived` to a `Base` by **value** copies only the `Base` part; the derived fields are “sliced off” and the vptr is reset to `Base`’s, so virtual calls now run the base version. Example: `Circle c(2); Shape s = c; s.area();` calls `Shape::area`. Fix: pass by reference or pointer (`Shape& s = c;`). **There is no JS equivalent** — JS objects are always handled by reference and never copied on assignment, so nothing can be sliced.
2. **For interfaces**: the plugin author compiles against a contract you published, and your dispatch code never changes when a plugin ships — new behaviour with zero edits on your side. **The cost**: (a) the contract is now frozen — adding a method breaks every plugin unless it has a default; (b) you lose exhaustiveness checking, so you can

never prove at compile time that all cases are handled; (c) dispatch is indirect, so it cannot be inlined; (d) a misbehaving plugin can violate your invariants and you must validate at the boundary. A closed `switch` gives you exhaustiveness and inlining but requires editing your own code for every new case — right for a closed set, wrong for third parties.

3. (a) `Number` was not designed for extension — in Java it is abstract with a fixed set of

`xxxValue()` methods, and in JS extending the built-in gives you a broken primitive wrapper whose arithmetic silently converts to `double`. (b) **Liskov fails**: `Money` must refuse `usd.plus(inr)`, but a `Number` freely adds to any number, so `Money` cannot honour its parent's contract. (c) It is **not** an is-a relationship — money **has** an amount and a currency. Use composition: `class Money { #amount; #currency; plus(o){...} }`, with the amount stored in minor units as an integer or `BigInt` to avoid floating-point error.

4. Liskov says the child must accept **everything** the parent accepts and return **nothing broader**

than the parent returns. A **covariant return** (child returns a subtype) is safe: every caller expecting the parent's return type still gets something valid. A **contravariant parameter** (child accepts a supertype) would in principle also be safe — but Java resolves methods by exact signature, so a differing parameter list creates an **overload**, not an override, and the compiler would silently give you two methods instead of one. So the restriction is partly theory (covariance on outputs is sound) and partly Java's dispatch rules (inputs must match exactly to be recognised as an override at all).

11.21 Rapid fire — one-line answers

Cover the right column. Say each answer out loud. Target: under five seconds each.

Question	Answer
Four pillars?	Encapsulation, abstraction, inheritance, polymorphism.
Encapsulation vs abstraction in one line?	Encapsulation hides data ; abstraction hides implementation .
Overloading — resolved when?	Compile time. Static polymorphism.
Overriding — resolved when?	Run time. Dynamic polymorphism.
Can a static method be overridden?	No. It is hidden ; the declared type decides.
Can a private method be overridden?	No — the child cannot see it, so it is a new method.
Constructors: static ? inherited?	Neither. But one can be private — that is how a singleton works.
Abstract class — constructor allowed?	Yes. It just cannot be instantiated.
Interface — instance fields allowed?	No. Java allows only public static final constants.
C++ pure virtual syntax?	<code>virtual T f() = 0;</code> — makes the class abstract.
Why must a polymorphic base destructor be virtual?	Else <code>delete basePtr</code> skips the derived destructor — a leak / UB.
Default virtual-ness: Java vs C++?	Java: all instance methods virtual. C++: only those marked virtual .
What is a vtable?	Per-class array of function pointers; each object holds a vptr to it.
Cost of a virtual call?	8 bytes per object, two loads, and no inlining.
Diamond problem in one line?	Two inheritance paths to one base — which copy does the child get?
C++ fix for the diamond?	virtual inheritance — all paths share one base sub-object.
Java's fix?	No multiple class inheritance; default -method clashes must be overridden with <code>A.super.m()</code> .
JavaScript's answer to multiple inheritance?	Mixins — class factories that build one straight prototype chain.
Does JS support method overloading?	No. The later definition overwrites; use rest params instead.
Does JS have interfaces?	No — duck typing. TypeScript adds interface at compile time.
Is-a vs has-a?	Is-a → inheritance. Has-a → composition. Prefer has-a.
Aggregation vs composition?	Aggregation: part survives the whole. Composition: part dies with it.
Shallow vs deep copy?	Shallow shares nested objects; deep duplicates the whole graph.
Deep copy in JS?	<code>structuredClone(obj)</code> .
C++ rule of three?	Own a resource → write copy ctor, copy assignment, destructor.
Expand SOLID.	Single responsibility, Open/closed, Liskov, Interface segregation, Dependency inversion.
One-line Liskov test?	A child must work everywhere the parent works, with no surprises.

Why never call an overridable method in a constructor?	The override runs on a half-built object; child fields are still default.
Coupling vs cohesion — which do you want high?	Cohesion high, coupling low.
Why is <code>this</code> undefined in a detached JS method?	<code>this</code> is set by the call site; class bodies are strict mode.

ONE-PAGE REVISION CARD

The three-pass rule. Every OOP answer: the idea → the JS reality → the Java/C++ words. Give the Java/C++ words at a service company, then add “in JavaScript this works differently” if asked.

The two sentences that carry the round.

- Overloading is compile-time polymorphism (different signature, same class); overriding is run-time polymorphism (same signature, child class).
- Inheritance is is-a; composition is has-a; prefer composition unless is-a is genuinely true.

Abstract class vs interface. One parent vs many interfaces. State and a constructor vs none. Shared **code** vs shared **capability**. `extends` vs `implements`.

Virtual. Java: every instance method, by default. C++: only with `virtual`, dispatched through a vtable, and the base destructor must be virtual too.

Diamond. C++ gets two base copies (measured: `sizeof(Copier) = 8`, `Device` ctor printed twice) — fixed with `virtual` inheritance. Java bans the class form and forces an override for clashing `default` methods. JS has no diamond because mixins build one straight chain.

What JS genuinely lacks: interfaces, method overloading, `protected`, compile-time checking. Say this plainly; it reads as honesty, not weakness.

The four traps that cost marks

1. “Static methods can be overridden.” — No. Hidden.
2. “JS supports overloading with default parameters.” — No. The later method wins.
3. “Java avoids the diamond by banning multiple inheritance.” — Half an answer; `default` methods bring it back.
4. Calling an overridable method from a constructor — the child override sees default field values (`n=0` in the verified Java run).

SOLID, five short sentences. One reason to change · extend without editing · the child must substitute · no fat interfaces · depend on contracts.

Copying. Shallow shares nested objects; deep duplicates them. `structuredClone` in JS; rule of three in C++.

Compilers, Git & Dev Fundamentals

What happens between your source file and a running process — and how Git really stores it

Every number, every file size and every command output in this chapter was produced by actually running the command on a Linux machine. Nothing here is from memory. Where you see an output block, it is a real transcript, and you can reproduce it.

WHAT YOU NEED TO KNOW

Compiler	Translates a whole program to another language (usually machine code) before it runs.
Interpreter	Reads and executes the program statement by statement, while it runs.
JIT	Starts interpreting, then compiles the hot parts to machine code at run time. V8 does this.
Preprocessor	Text stage. Expands includes and macros. Output is still source code.
Assembler	Assembly text → machine code in an object file . Addresses still unresolved.
Linker	Joins object files and libraries, resolves every symbol, produces an executable.
Loader	Copies the executable into memory, sets up segments, jumps to the entry point.
Token	The smallest meaningful piece of source: a number, a name, an operator.
AST	Abstract syntax tree — the parsed shape of the program, with precedence baked in.
Symbol table	Name → (type, scope, address). Built during semantic analysis.
Static linking	Library code is copied into the executable. Big file, no runtime dependency.
Dynamic linking	Library is loaded at run time from a <code>.so</code> / <code>.dll</code> . Small file, shared in RAM.
Segment	A region of a process's address space: text, rodata, data, bss, heap, stack.
Stack frame	One function call's locals, arguments and return address. Pushed on call, popped on return.
Blob / tree / commit	Git's three object types: file bytes / directory listing / snapshot + parent + message.
HEAD	A file holding which branch you are on. In detached state it holds a raw commit SHA.
Index	Also called the staging area. The list of what the next commit will contain.

Four sentences that carry most of this chapter

- A compiler produces a **file**; an interpreter produces an **answer**.
- The four stages are **preprocess** → **compile** → **assemble** → **link**, and they produce `.i` → `.s` → `.o` → executable.
- The stack grows **down** toward low addresses; the heap grows **up**. They meet in the middle.
- A Git commit is a **snapshot**, not a diff. It stores a tree hash, a parent hash, and a message.

12.1 From source file to executable — the four stages

Here is the program. Six lines.

hello.c

C

```
#include <stdio.h>
#define TIMES 3
int main(void) {
```

```

for (int i = 0; i < TIMES; i++) printf("hi %d\n", i);
return 0;
}

```

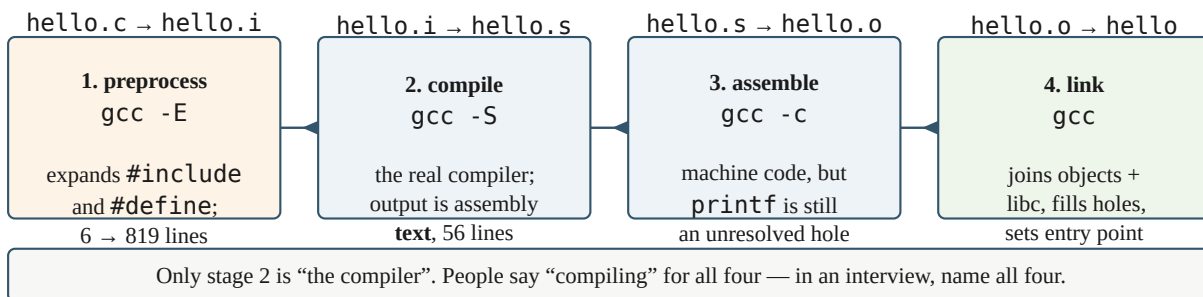
Now run each stage by hand instead of letting gcc do all four silently.

The four stages, one command each	BASH
gcc -E hello.c -o hello.i	# 1. preprocess -> still C source
gcc -S hello.i -o hello.s	# 2. compile -> assembly text
gcc -c hello.s -o hello.o	# 3. assemble -> object file (machine code)
gcc hello.o -o hello	# 4. link -> executable

Real measured results:

File	Lines	Bytes	What it is
hello.c	6	—	what you wrote
hello.i	819	21339	C source with <code>stdio.h</code> pasted in and <code>TIMES</code> replaced by 3
hello.s	56	814	x86-64 assembly, human readable text
hello.o	—	1528	machine code, but <code>printf</code> 's address is still a hole
hello	—	15960	executable: holes filled, entry point set

Six lines became 819. That is `stdio.h` and everything it includes. This is why C compiles slowly: the compiler really does re-read those 800 lines for every `.c` file.



The four stages. All sizes measured on a real build.

Proof that the preprocessor is just text substitution

The last lines of `hello.i`, straight from the file:

```

# 3 "hello.c"
int main(void) {
    for (int i = 0; i < 3; i++) printf("hi %d\n", i);
    return 0;
}

```

`TIMES` is gone. The literal 3 is in its place. The preprocessor does not know C — it does not know that 3 is a number or that `i` is a variable. It matches text and pastes text.

TRAP

Because macros are pure text, `#define SQ(x) x*x` then `SQ(2+3)` expands to `2+3*2+3 = 11`, not 25. The fix is parentheses everywhere: `#define SQ(x) ((x)*(x))`. This is the classic macro question, and the reason C++ says “use inline functions and `constexpr` instead of macros”.

12.2 Compiler vs interpreter vs JIT

	Compiler	Interpreter	JIT
Translates	whole program, once, ahead of time	one statement at a time, every time	hot functions, at run time
Output	a machine-code file	an answer — nothing is saved	machine code in memory
Errors found	at build time, all at once	when that line runs	build-free; run time
Speed of run	fastest	slowest	close to compiled, after warm-up
Startup	slow to build, instant to start	instant	instant, then gets faster
Portability	one binary per platform	runs anywhere the interpreter runs	same as interpreter
Examples	C, C++, Rust, Go	classic Python, shell	Java HotSpot, V8, .NET, PyPy

Measure the difference yourself

Both strategies, same work: evaluate $x * 3 + 7$ three million times.

Interpreting an AST vs 'compiling' it to a closure

JS

```
const ast = { n: "bin", op: "+",
  l: { n: "bin", op: "*", l: { n: "var", v: "x" }, r: { n: "num", v: 3 } },
  r: { n: "num", v: 7 } };

function interpret(node, env) { // walks the tree on EVERY call
  switch (node.n) {
    case "num": return node.v;
    case "var": return env[node.v];
    case "bin": { const a = interpret(node.l, env), b = interpret(node.r, env);
      return node.op === "+" ? a + b : a * b; }
  }
}

function compile(node) { // walks the tree ONCE, returns a
function
  switch (node.n) {
    case "num": { const v = node.v; return () => v; }
    case "var": { const k = node.v; return env => env[k]; }
    case "bin": { const L = compile(node.l), R = compile(node.r);
      return node.op === "+" ? env => L(env) + R(env) : env => L(env) *
R(env); }
  }
}

const N = 3_000_000, env = { x: 5 };
let t = process.hrtime.bigint(); let s = 0;
for (let i = 0; i < N; i++) s += interpret(ast, env);
const ti = Number(process.hrtime.bigint() - t) / 1e6;

const fn = compile(ast); // one-time cost
t = process.hrtime.bigint(); let s2 = 0;
for (let i = 0; i < N; i++) s2 += fn(env);
const tc = Number(process.hrtime.bigint() - t) / 1e6;

console.log("same answer:", s === s2, "value per call =", interpret(ast, env));
```

```
console.log(`interpret : ${ti.toFixed(0)} ms`);
console.log(`compiled  : ${tc.toFixed(0)} ms`);
```

Real output from node (two runs, to show it is stable):

```
same answer: true value per call = 22
interpret : 114 ms
compiled  : 7 ms
```

Check the value: $5 \times 3 + 7 = 22$. Correct.

The compiled version is about **16 times** faster and it is the **same algorithm**. The only change is **when** the decisions are made. The interpreter re-asks “what node type is this?” three million times. The compiled version asked once and baked the answer into a closure.

That is the entire idea behind a JIT, in miniature.

SHORTCUT

When asked “why is C faster than Python?”, do not say “because it is compiled”. Say: “**the work of deciding what each operation means is done once at build time instead of on every execution, and the compiler can then see the whole function and optimise across statements.**” That answer survives the follow-up “but Python has a JIT now”.

12.3 Inside the front end: lexer, parser, semantic analysis

The compiler reads your file in three passes before it emits a single machine instruction.

Pass 1 — the lexer: characters become tokens

A working lexer in 14 lines

JS

```
function lex(src) {
  const toks = [], re = /\s*(\d+|[A-Za-z_]\w*|[-+*/()=;])/g;
  let m, pos = 0;
  while ((m = re.exec(src)) !== null) {
    if (m.index !== pos) throw new Error("bad char at " + pos);
    pos = re.lastIndex;
    const s = m[1];
    if (/^\d/.test(s)) toks.push({ type: "NUM", value: +s });
    else if (/^[A-Za-z_]/.test(s)) toks.push({ type: "ID", value: s });
    else toks.push({ type: "OP", value: s });
  }
  return toks;
}
console.log(lex("total = price * 3 + 12;").map(t => `${t.type}(${t.value})`).join(" "));
```

Real output:

```
ID(total) OP(=) ID(price) OP(*) NUM(3) OP(+) NUM(12) OP(;
```

Notice what the lexer does **not** do. It does not know that `total` is a variable, that `=` is an assignment, or that `*` binds tighter than `+`. It only chops the character stream into labelled pieces and throws away whitespace. Errors it can catch: an illegal character, an unterminated string. That is all.

Pass 2 — the parser: tokens become a tree

This is where precedence lives. $2 + 3 * 4$ must become $2 + (3 \times 4)$, and the grammar’s **shape** is what makes that happen.

```

expr  := term  (('+' | '-') term)*
term  := factor (('*' | '/') factor)*
factor := NUM | '(' expr ')'

```

expr calls term first, so multiplication is grabbed **before** addition gets a chance. Nothing else is needed — no precedence table.

A recursive-descent parser and an evaluator

JS

```

function parse(toks) {
  let i = 0;
  const peek = () => toks[i];
  const eat = v => { if (toks[i]?.value !== v) throw new Error("expected " + v); i++; };
  function factor() {
    const t = peek();
    if (t.type === "NUM") { i++; return { n: "num", v: t.value }; }
    eat("("); const e = expr(); eat(")"); return e;
  }
  function term() {
    let l = factor();
    while (peek() && (peek().value === "*" || peek().value === "/")) {
      const op = toks[i++].value; l = { n: "bin", op, l, r: factor() };
    }
    return l;
  }
  function expr() {
    let l = term();
    while (peek() && (peek().value === "+" || peek().value === "-")) {
      const op = toks[i++].value; l = { n: "bin", op, l, r: term() };
    }
    return l;
  }
  return expr();
}

const lex = s => [...s.matchAll(/\d+|[-+*/()]/g)].map(m =>
  /\d/.test(m[0]) ? { type: "NUM", value: +m[0] } : { type: "OP", value: m[0] });

const show = a => a.n === "num" ? String(a.v) : `(${show(a.l)} ${a.op} ${show(a.r)})`;
const evalAst = a => a.n === "num" ? a.v :
  ({ "+": (x,y)=>x+y, "-": (x,y)=>x-y, "*": (x,y)=>x*y, "/": (x,y)=>x/y })[a.op]
  (evalAst(a.l), evalAst(a.r));

for (const s of ["2 + 3 * 4", "(2 + 3) * 4", "20 - 6 - 4"]) {
  const ast = parse(lex(s));
  console.log(s.padEnd(12), "->", show(ast).padEnd(20), "=", evalAst(ast));
}

```

Real output:

```

2 + 3 * 4    -> (2 + (3 * 4))      = 14
(2 + 3) * 4   -> ((2 + 3) * 4)     = 20
20 - 6 - 4    -> ((20 - 6) - 4)    = 10

```

Read the middle column. The parentheses in the printed tree were **never in the source** of the first line — the parser put them there. And $20 - 6 - 4$ grouped **left**, giving 10 and not $20 - (6 - 4) = 18$, because the while loop in `expr()` keeps folding to the left. That loop **is** left-associativity.

Pass 3 — semantic analysis: does it make sense?

The parser only checks **shape**. `x = "hello" * true`; parses fine. Semantic analysis walks the tree with a **symbol table** and asks: is `x` declared? what type is it? is `*` defined for those types?

JavaScript does a small version of this before running each script, and you can see it:

```
Scope resolution happens before line 1 executes JS

console.log(typeof hoistedFn); // "function" -> whole function lifted
console.log(typeof varX);      // "undefined" -> name lifted, value not
try { console.log(letY); }     // name is known, but in the "temporal dead zone"
catch (e) { console.log(e.constructor.name + ":", e.message); }

function hoistedFn() {}
var varX = 1;
let letY = 2;
```

Real output:

```
function
undefined
```

ReferenceError: Cannot access 'letY' before initialization

Three different behaviours for three declaration forms, and all three are decided **before the first line runs**. The engine built a scope record for the whole script first. That is a symbol table.

Stage	Catches	Example error
Lexical	illegal characters, unterminated literals	<code>let x = "abc;</code>
Syntactic	wrong shape	<code>if (x { } — missing)</code>
Semantic	undeclared names, type mismatch, wrong argument count	<code>int n = "hello";</code>
Link	a name that no file defines	undefined reference to 'helper'
Run	everything the compiler could not know	divide by zero, null dereference

12.4 The back end: intermediate code, optimisation, code generation

After the front end, the compiler builds an **intermediate representation** — a simple, machine-independent form. Then it optimises the IR, then it turns it into instructions for the target CPU.

You can watch optimisation happen.

```
opt2.c — two obvious wastes C

int compute(void) {
    int a = 6 * 7;      /* constant folding      */
    int unused = 999;   /* dead code elimination */
    return a + 8;
}
```

Compile twice and look at the assembly gcc produced.

```
gcc -O0 -S opt2.c vs gcc -O2 -S opt2.c BASH

# ---- gcc -O0 (no optimisation) ----
compute:
    endbr64
```

```
pushq    %rbp
movq     %rsp, %rbp
movl     $42, -8(%rbp)      # a = 42          (6*7 already folded)
movl     $999, -4(%rbp)     # unused = 999    (still stored!)
movl     -8(%rbp), %eax
addl     $8, %eax
popq     %rbp
ret

# ---- gcc -O2 ----
compute:
    endbr64
    movl    $50, %eax        # the WHOLE function is one constant
    ret
```

That is a real transcript. Eight instructions became two.

Follow the two optimisations:

- 1. **Constant folding** — $6 * 7$ is computed at build time. This happened even at `-O0` (`movl $42`), because it is done in the front end.
- 2. **Dead code elimination** — `unused` is written at `-O0` and **absent** at `-O2`. Nothing reads it, so the store is removed, and with it the local variable.
- 3. **Constant propagation** — knowing `a` is 42, `a + 8` becomes 50 at build time. The function no longer computes anything.

Optimisation	What it does
Constant folding	Evaluate expressions with known values at build time: $6*7 \rightarrow 42$.
Constant propagation	Replace a variable by its known value, then fold again.
Dead code elimination	Delete computations whose result nothing reads.
Common subexpression elimination	Compute $a*b$ once when it appears twice.
Loop-invariant code motion	Move a computation that does not change out of the loop.
Strength reduction	Replace an expensive op with a cheap one: $x*8 \rightarrow x \ll 3$.
Inlining	Paste a small function’s body into the caller, then optimise across the join.
Register allocation	Keep hot variables in CPU registers instead of memory.

TRAP

“Optimisation makes my code faster, so `-O2` is always safe” is not quite true, and the follow-up is about **undefined behaviour**. If your C program has UB — signed overflow, reading uninitialised memory, a data race — the optimiser is allowed to assume it never happens and may delete the code that handles it. Programs that “work at `-O0` and break at `-O2`” almost always have UB. The fix is to remove the UB, not to lower the optimisation level.

12.5 Linking: where “undefined reference” comes from

u.c — a function that is declared but never definedC

```
int helper(int);          /* declared, never defined */
int main(void) { return helper(1); }
```

Real transcript:

```
$ gcc -c u.c -o u.o
u.o built <- compiling ALONE succeeds
```

```
$ gcc u.o -o u
/usr/bin/ld: u.o: in function `main':
u.c:(.text+0xe): undefined reference to `helper'
collect2: error: ld returned 1 exit status
```

Read what this tells you. The **compiler** was happy: it saw a declaration, so it knew the name, the return type and the argument types — enough to emit a call instruction with a **hole** where the address goes. Only the **linker** has to find a real definition, and there is none.

This is the single most useful diagnostic skill in C and C++:

Message	It means
error: 'helper' was not declared	Compiler error. You forgot the header / the declaration.
undefined reference to 'helper'	Linker error. Declaration found, definition missing. You forgot to compile or link that .c file, or forgot -lm / -lpthread.
multiple definition of 'helper'	Linker error. Two files define it. Usually a function body put in a header.
ld: cannot find -lfoo	The library file libfoo.so / libfoo.a is not on the search path.

Static vs dynamic linking — measured

```
$ gcc      hello.c -o dyn
$ gcc -static hello.c -o sta
$ ls -l dyn sta
dyn      15960
sta      785328

$ ldd dyn
    linux-vdso.so.1
    libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
    /lib64/ld-linux-x86-64.so.2
```

```
$ ldd sta
not a dynamic executable
```

The static build is **49 times larger** ($\frac{785328}{15960} = 49.2$) because the parts of libc it uses are copied inside it.

	Static (.a)	Dynamic (.so / .dll)
When joined	at link time	at load time (and later, with dlopen)
Executable size	large — measured 785 KB vs 16 KB	small
RAM with 50 copies running	50 copies of the library	one shared copy
Security patch in libc	rebuild and reship every program	replace the .so; everything is fixed
Startup	faster — nothing to resolve	slower — the dynamic loader must resolve symbols
Deployment	one file, no dependencies	needs the right library version present
Failure mode	none at run time	error while loading shared libraries / version hell

12.6 The memory layout of a running program

This is asked constantly, and most students recite the diagram without ever having checked it. Let us check it.


```

#include <stdio.h>
#include <stdlib.h>
int g_init = 5; /* .data - initialised global */
int g_zero; /* .bss - uninitialised global */
const char *msg = "hi"; /* pointer in .data, "hi" in .rodata */
void fn(void) {} /* .text */
int main(void) {
    int local = 1; /* stack */
    int *heap = malloc(sizeof(int)); /* heap */
    printf("text (fn) %p\n", (void*)fn);
    printf("rodata(\"hi\") %p\n", (void*)msg);
    printf("data (g_init) %p\n", (void*)&g_init);
    printf("bss (g_zero) %p\n", (void*)&g_zero);
    printf("heap (malloc) %p\n", (void*)heap);
    printf("stack (local) %p\n", (void*)&local);
    free(heap);
    return 0;
}

```

Built with `gcc -no-pie layout.c -o layout` (the `-no-pie` just keeps the addresses fixed so they are easy to read). Real output:

```

text (fn)      0x401196
rodata("hi")   0x402004
data (g_init)  0x404030
bss (g_zero)   0x404044
heap (malloc)  0x17dca2a0
stack (local)  0x7fffe2b52bcc

```

Every address is larger than the one above it. The textbook picture is correct, and now you have seen it.

stack — locals, arguments, return addresses, saved registers. Grows DOWN ↓	0x7fffe2b52bcc
free gap — the two ends grow toward each other	
heap — <code>malloc</code> / <code>new</code> . Grows UP ↑. Freed by you, never automatically (in C).	0x17dca2a0
.bss — globals/statics with no initialiser. Takes zero bytes in the file; zero-filled at load.	0x404044
.data — globals/statics with an initialiser. The values are stored in the file.	0x404030
.rodata — string literals and <code>const</code> data. Read-only; writing here segfaults.	0x402004
.text — the machine code itself. Read-only and shareable between processes.	0x401196

Process address space, low address at the bottom. Addresses are a real run of layout.c

Segment	Holds	Lifetime	Freed by
.text	code	whole program	the OS, at exit
.rodata	literals, <code>const</code>	whole program	the OS, at exit
.data	initialised globals/statics	whole program	the OS, at exit

.bss	zero-initialised globals/statics	whole program	the OS, at exit
heap	malloc / new	until you free it	you
stack	locals, parameters, return address	until the function returns	automatically

TRAP

.bss takes no space in the executable file. `int big[1000000];` as a global adds 4 MB to the **process** but roughly **nothing** to the file on disk, because the file only records “reserve 4 MB and zero it”. Change it to `int big[1000000] = {1};` and it moves to `.data`, and now 4 MB of mostly-zeros really is written into the file. This is a favourite follow-up: “why is my binary suddenly 4 MB?”

12.7 The stack frame and why recursion has a limit

Each function call pushes a **frame**: the return address, saved registers, the parameters and the locals. Return pops it.

frame.c — print the address of a local at each depth

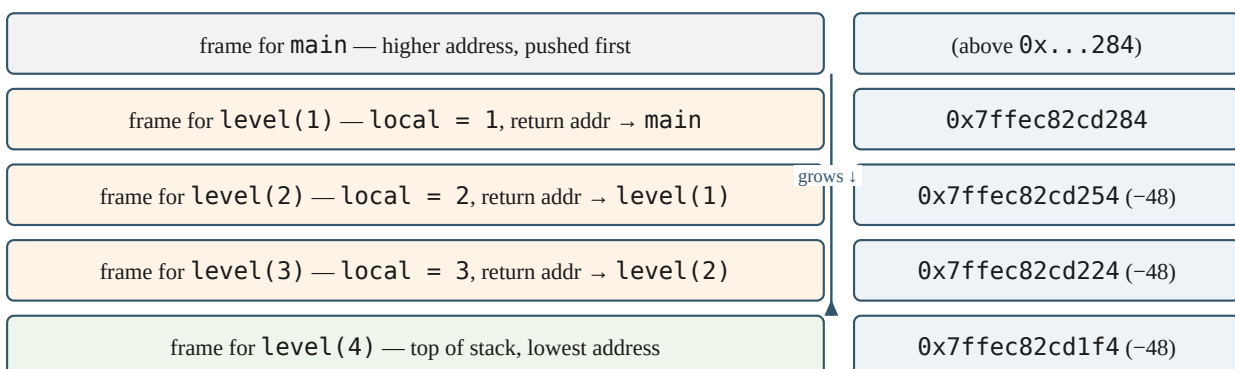
C

```
#include <stdio.h>
void level(int n) {
    int local = n;           /* one per frame */
    printf("depth %d frame at %p\n", n, (void*)&local);
    if (n < 4) level(n + 1);
}
int main(void) { level(1); return 0; }
```

Real output:

```
depth 1 frame at 0x7ffec82cd284
depth 2 frame at 0x7ffec82cd254
depth 3 frame at 0x7ffec82cd224
depth 4 frame at 0x7ffec82cd1f4
```

Subtract: $0x284 - 0x254 = 0x30 = 48$ bytes, and the gap is the same every time. So on this build each frame is 48 bytes and the addresses go **down**. Both facts are now measured, not assumed.



Stack frames from the real run. Each call moves the stack pointer DOWN by 48 bytes.

The same limit, in JavaScript

How deep can you go, and what to do instead

JS

```

let depth = 0;
function dive() { depth++; dive(); }
try { dive(); } catch (e) { console.log(e.constructor.name, "at depth ~", depth); }

// Same job, heap-allocated explicit stack -> no limit problem.
function sumTo(n) {
  let total = 0n;
  const stack = [n];
  while (stack.length) { const k = stack.pop(); if (k > 0) { total += BigInt(k);
stack.push(k - 1); } }
  return total;
}
console.log("sumTo(200000) =", sumTo(200000).toString());
console.log("check n(n+1)/2 =", (200000n * 200001n / 2n).toString());

```

Real output:

```

RangeError at depth ~ 12546
sumTo(200000) = 20000100000
check n(n+1)/2 = 20000100000

```

Two lessons here.

First, the JS call stack died at about **12,500** frames. So any recursion that could go deeper than roughly 10^4 needs an explicit stack held on the heap, exactly as the loop version does. The loop handled 200,000 levels without complaint.

Second, the arithmetic checks out. $\frac{n(n+1)}{2} = \frac{200000 \times 200001}{2} = 20\{, \}000\{, \}100\{, \}000$, and that is what the program printed. BigInt was used so the check stays exact for any n ; for this particular n a plain Number would also have been fine, since 2×10^{10} is far below the safe-integer limit $2^{53} - 1 \approx 9.007 \times 10^{15}$.

TRAP

Stack overflow and heap exhaustion are different failures. Deep recursion overflows the **stack** (fixed size, typically 1–8 MB, set at thread creation). Allocating too many objects exhausts the **heap** (grows until the OS refuses). In Node the first is `RangeError: Maximum call stack size exceeded`; the second is `JavaScript heap out of memory`. Naming the right one is the whole point of the question.

12.8 Git — the three trees

Everything in Git is easier once you hold this picture. There are three places a file can be, and every command moves files between them.



`git status` shows two diffs at once: **working** ↔ **index** ("not staged") and **index** ↔ **HEAD** ("to be committed").

`git diff` = working vs index. `git diff --cached` = index vs HEAD. `git diff HEAD` = working vs HEAD.

The three trees. Every everyday Git command is an arrow on this picture.

Here is the index, printed. It is a real list of file paths and content hashes:

```
$ git ls-files -s
100644 ce013625030ba8dba906f756967f9e9ca394464a 0 a.txt
100644 ce013625030ba8dba906f756967f9e9ca394464a 0 b.txt
```

Note that `a.txt` and `b.txt` have the **same hash**. They have the same content, so Git stores the bytes **once**. That is not a special feature; it falls out of how the objects are named.

12.9 Git's object model — and the hash, computed by hand

Git stores four object types. Three matter for interviews.

Object	Contains	Named by
blob	the bytes of one file — no name, no path, no permissions	SHA-1 of its content
tree	a directory listing: mode, type, hash, name — one row per entry	SHA-1 of that listing
commit	tree hash, parent hash(es), author, committer, message	SHA-1 of all of that
tag	an annotated tag: object hash, tagger, message	SHA-1 of that

Make a repo with one file and look inside.

One commit, three objects

BASH

```
$ git init -q -b main && printf 'hello\n' > a.txt && git add a.txt && git commit -q -m "first"
```

```
$ find .git/objects -type f
ce/013625030ba8dba906f756967f9e9ca394464a
8e/b30367d95cd1f9d226d1eeacb0e3439d20c794
2e/81171448eb9f2ee3821e3d447aa6b2fe3ddb1
```

```
$ git cat-file -t ce01362 ; git cat-file -t 2e81171 ; git cat-file -t 8eb3036
blob
```

```
tree
commit
```

One six-byte file produced **three** objects: the blob (the bytes), the tree (the directory listing saying “a.txt is that blob”), and the commit (a pointer to the tree plus the message).

Print the tree and the commit:

```
$ git cat-file -p HEAD^{tree}
100644 blob ce013625030ba8dba906f756967f9e9ca394464a  a.txt

$ git cat-file -p HEAD
tree 2e81171448eb9f2ee3821e3d447aa6b2fe3ddb1
author Student <s@example.com> 1789490157 +0530
committer Student <s@example.com> 1789490157 +0530
```

first

Compute the blob hash yourself

Git’s rule for a blob is exactly this: take the literal string blob, a space, the content length in decimal, a **zero byte**, then the content — and SHA-1 the whole thing.

Reproducing Git’s hash in six lines of Python

PYTHON

```
import hashlib
content = b'hello\n'
store = b'blob ' + str(len(content)).encode() + b'\x00' + content
print('header+body =', store)
print('sha1      =', hashlib.sh1(store).hexdigest())
```

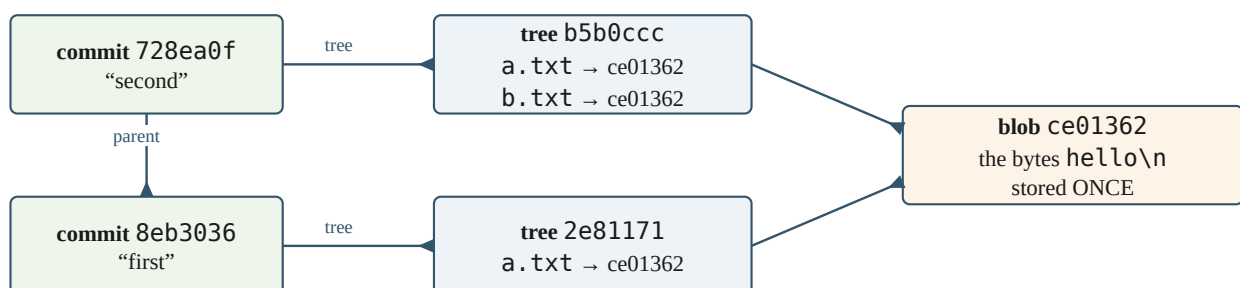
Real output:

```
header+body = b'blob 6\x00hello\n'
sha1      = ce013625030ba8dba906f756967f9e9ca394464a
```

That is **character for character** the hash Git reported above. You have just reimplemented `git hash-object`.

The tree works the same way, with a body of <mode> <name>\0<20 raw bytes of the hash>:

```
blob = bytes.fromhex('ce013625030ba8dba906f756967f9e9ca394464a')
body = b'100644 a.txt\x00' + blob
store = b'tree ' + str(len(body)).encode() + b'\x00' + body
print(hashlib.sh1(store).hexdigest())
# 2e81171448eb9f2ee3821e3d447aa6b2fe3ddb1  <- matches the tree above
```



Two commits. The blob is stored once and shared, because its hash is its name.

TRAP

A Git commit is a full snapshot, not a diff. Subversion stored diffs; Git stores a complete tree per commit and relies on content addressing so that unchanged files cost nothing extra. Deltas do exist — but only later, inside **pack files**, as a storage optimisation. Measured: after committing a 50,000-byte random file and then changing **one character**, the two loose blobs occupy 74,573 bytes in `.git/objects`; after `git gc` packs them with deltas, the whole directory is 39,387 bytes. Snapshots first, deltas as a compression step.

12.10 Branches, HEAD and the reflog are just files

This is the demo that makes Git stop being magic.

```
$ cat .git/HEAD
ref: refs/heads/main
```

```
$ cat .git/refs/heads/main
7be546fae23440745fc098dd8c1aff50ee304a4d
```

A **branch** is a 41-byte text file containing one commit hash. HEAD is a one-line file saying which branch you are on. That is all. Creating a branch writes 41 bytes — which is why it is instant, and why Git encourages branching where older tools did not.

Now check out an old commit directly:

```
$ git checkout HEAD~1
$ cat .git/HEAD
8eb30367d95cd1f9d226d1eeacb0e3439d20c794
```

```
$ git status --short --branch
## HEAD (no branch)
```

That is “detached HEAD”: HEAD now holds a raw SHA instead of `ref: refs/heads/...`. Commits you make here belong to no branch, so nothing points at them and they are eventually garbage collected. The fix is to give them a name before you leave: `git switch -c rescue`.

And the safety net:

```
$ git reflog
7be546f HEAD@{0}: commit: C3
5636034 HEAD@{1}: commit: C2 (bad)
3a6b71b HEAD@{2}: commit (initial): C1
```

The reflog records **every** position HEAD has held on this machine, including ones no branch points to any more. After a bad `git reset --hard`, `git reset --hard HEAD@{1}` gets you back. Say this in an interview and it reads as experience.

12.11 Merge vs rebase

Build the classic situation: main and feature both moved after they split.

```
$ git log --oneline --graph --all
* e4dd9cb D      <- feature
* 0939c0b C
| * b9e3e0e E    <- main
|/
* 1a4b9c1 B
* 9de8a76 A
```

git merge

```
$ git merge --no-ff feature
* a01ac63 merge feature
|\
| * e4dd9cb D
```

```
| * 0939c0b C
* | b9e3e0e E
|/
* 1a4b9c1 B
* 9de8a76 A
```

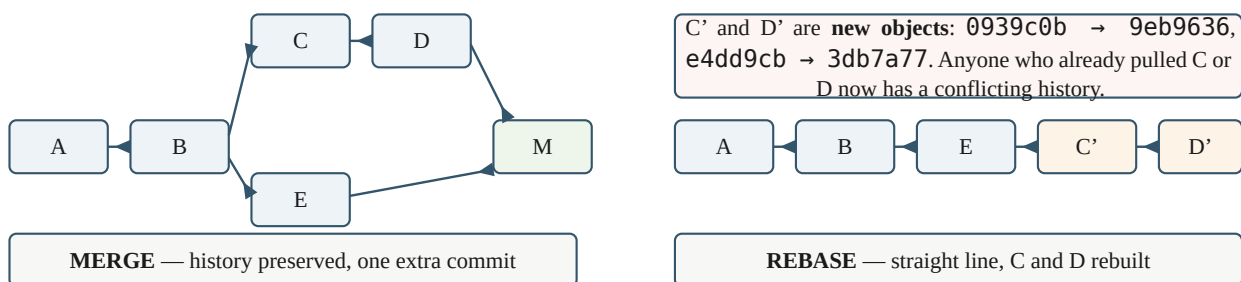
A **new commit with two parents** is created. C and D keep their original hashes — 0939c0b and e4dd9cb, exactly as before. Nothing was rewritten.

git rebase

Same starting point, different command:

```
$ git rebase main
* 3db7a77 D      <- NEW hash
* 9eb9636 C      <- NEW hash
* b9e3e0e E
* 1a4b9c1 B
* 9de8a76 A
```

The graph is a straight line. But look carefully: C went from 0939c0b to 9eb9636 and D from e4dd9cb to 3db7a77. **Those are not the same commits.** Rebase replayed the changes on top of E, which means it built brand-new commits with a different parent — and a commit’s hash covers its parent, so the hash had to change.



Same starting graph, two results. Rebase gives C and D new hashes.

	merge	rebase
History	true — shows that a branch existed	linear — looks like it was written in order
Commit hashes	unchanged	rewritten for every replayed commit
Extra commit	yes, a merge commit with 2 parents	none
Conflicts	resolved once	possibly once per replayed commit
Safe on a shared branch	yes	no — never rebase commits others have pulled
Good for	integrating a finished feature into main	tidying your own branch before a PR

TRAP

The golden rule of rebase: never rebase a branch that other people have already pulled. Their clone still points at 0939c0b; yours now says 9eb9636. Their next `git pull` sees two histories containing the same change and either creates a duplicate or forces a painful merge. Rebase your private branch; merge everything else. “It rewrites history” is the phrase the panel listens for.

Fast-forward: the merge that is not a merge

If main did **not** move while you worked, there is nothing to combine — Git just slides the branch pointer forward.

```
$ git merge feat
Updating d3fdaba..5d8e775
Fast-forward
 b.txt | 1 +
```

```
$ git log --oneline --graph
* 5d8e775 B
* d3fdaba A      <- no merge commit at all
```

With `--no-ff` you force a merge commit anyway, which keeps a visible record that a feature branch existed:

```
$ git merge --no-ff feat -m "merge feat"
* e28c951 merge feat
|\
| * 5d8e775 B
|/
* d3fdaba A
```

Many teams require `--no-ff` on `main` for exactly that reason: the merge commit is the audit trail of the pull request.

12.12 reset vs revert vs checkout/restore

The three commands students confuse. Here is the difference, measured.

Setup: two commits, C1 writing v1 to `f.txt` and C2 writing v2. Then run each reset mode on a fresh copy.

Command	HEAD	Index	Working file	git status shows
<code>reset --soft HEAD~1</code>	C1	still v2	v2	<code>f.txt</code> staged, ready to re-commit
<code>reset --mixed HEAD~1</code>	C1	reset to C1	v2	<code>f.txt</code> modified, not staged
<code>reset --hard HEAD~1</code>	C1	reset to C1	v1	nothing — your edit is gone

That table is a real transcript, not a recollection. `--mixed` is the default when you type `git reset HEAD~1` with no flag.

	git reset	git revert
What it does	moves the branch pointer backwards	creates a new commit that undoes an old one
History	rewritten — commits disappear from the branch	preserved — the bad commit is still there
Safe on a shared branch	no	yes
Use when	the commits are still local and private	the commit is already pushed

Real `git revert` transcript. Three commits; C2 added `bad.txt`.

```
$ ls
a.txt  bad.txt  c.txt
```

```
$ git revert --no-edit HEAD~1
$ ls
a.txt  c.txt
```

```
$ git log --oneline
523ec46 Revert "C2 add bad.txt"
6f49bfc C3
12968d2 C2 add bad.txt      <- still in history
f9ba0fa C1
```

The file is gone from the working tree; the commit that added it is **still in the log**, and a new commit records the undo. That is what “does not rewrite history” means in practice.

If the reverted commit and later commits touched the **same lines**, `git revert` will stop with a merge conflict and ask you to resolve it, exactly like a merge. That is normal; fix the file, `git add`, then `git revert --continue`.

Command	Effect
<code>git restore f.txt</code>	throw away working-directory changes to <code>f.txt</code> (take the index version)
<code>git restore --staged f.txt</code>	unstage <code>f.txt</code> , keep the edit in the working directory
<code>git checkout <commit></code>	move HEAD to that commit — detached HEAD
<code>git switch <branch></code>	move to a branch (the modern, unambiguous spelling)
<code>git reset --hard <commit></code>	move the branch and wipe index and working tree to match

SHORTCUT

`git checkout` did two unrelated jobs — switch branches **and** discard file changes — which is why it confused everybody. Git 2.23 split it: `git switch` for branches, `git restore` for files. Use those two names in an interview; it signals you have used Git recently.

12.13 cherry-pick, stash, and .gitignore

cherry-pick — copy one commit onto another branch

```
$ git log fix --oneline
5936873 wip
806a058 hotfix          <- we want only this one
d446f83 base

$ git switch main
$ git cherry-pick 806a058
$ git log --oneline
f888b61 hotfix          <- SAME change, DIFFERENT hash
d4e9ef2 main moves on
d446f83 base
```

The hash changed from `806a058` to `f888b61` for the same reason rebase changes hashes: the parent is different, and the parent is part of what gets hashed. So `cherry-pick` **copies a change**; it does not move a commit.

stash — park uncommitted work

```
$ git status --short
M f.txt
?? new.txt

$ git stash push -m "wip"
$ git status --short
?? new.txt          <- untracked file was NOT stashed
$ cat f.txt
v1                  <- tracked edit is gone from the working tree

$ git stash list
stash@{0}: On main: wip

$ git stash pop
$ git status --short
M f.txt             <- edit is back
?? new.txt
```

Two things that catch people out, both visible above: `git stash` by default takes only **tracked** modifications — untracked files stay behind (use `-u` to include them) — and `pop` applies **and deletes** the stash while `apply` keeps it.

.gitignore does not untrack anything

```
$ git add .env && git commit -m "oops committed .env"
$ echo ".env" > .gitignore && git commit -am "add gitignore"
$ echo changed > .env
$ git status --short
M .env                                <- STILL TRACKED. .gitignore did nothing.

$ git rm --cached .env
$ git commit -m "untrack .env"
$ git status --short
<- clean, and .env is still on disk

$ git log --oneline --all -- .env
7903c18 untrack .env
e624687 oops committed .env    <- the secret is STILL in history
```

TRAP

.gitignore only affects files Git is not already tracking. Adding a path to it does nothing for a file already in the index — you must `git rm --cached <file>`. And even then, **the old content stays in every historical commit**. If it was a credential, it is compromised: rotate the key. Removing it from history needs `git filter-repo` (or BFG) **and** a force-push **and** every collaborator re-cloning. Say all three parts; the “rotate the key” part is what a security-aware answer sounds like.

12.14 Branching workflows

Workflow	Shape	Fits
Feature branch	branch off main, PR, review, merge back	almost everyone; the default
Git flow	long-lived <code>develop</code> , <code>release/*</code> , <code>hotfix/*</code> , <code>main</code> is releases only	versioned products with scheduled releases
Trunk-based	everyone commits to <code>main</code> many times a day behind feature flags	continuous deployment, strong test suite
Forking	contributors fork, PR from their fork	open source, untrusted contributors

A pull request that gets approved quickly:

1. One branch, one purpose. `fix/login-redirect`, not `misc-changes`.
2. Small. Under 400 changed lines is reviewable; 2,000 gets rubber-stamped, which is worse than not being reviewed.
3. Commit messages in the imperative: “Add retry to payment webhook”, not “added stuff”.
4. Rebase **your own** branch on `main` before opening it, so the diff is only your work.
5. Never commit generated files, `node_modules`, `.env`, or a lockfile you did not mean to change.

12.15 Versioning and dependencies

Semantic versioning: MAJOR.MINOR.PATCH.

- **MAJOR** — you broke the public API. Callers must change.
- **MINOR** — you added something, backwards compatible.
- **PATCH** — you fixed a bug, nothing else changed.

The npm range operators follow from that.

```
function satisfies(range, v) {
  const [M, m, p] = v.split(".").map(Number);
  const op = range[0] === "^" || range[0] === "~" ? range[0] : "=";
  const [RM, Rm, Rp] = range.replace(/^[\^~]/, "").split(".").map(Number);
  if (op === "=") return M === RM && m === Rm && p === Rp;
  if (op === "^") return M === RM && (m > Rm || (m === Rm && p >= Rp)); // MAJOR
  if (op === "~") return M === RM && m === Rm && p >= Rp; // ~ locks
  MINOR too
}
const tests = [
  ["^2.3.1", "2.4.0"], ["^2.3.1", "3.0.0"], ["^2.3.1", "2.3.0"],
  ["~2.3.1", "2.3.9"], ["~2.3.1", "2.4.0"], ["2.3.1", "2.3.1"]];
for (const [r, v] of tests) console.log(`${r.padEnd(7)} allows ${v.padEnd(7)} -> ${satisfies(r, v)}`);
```

Real output:

```
^2.3.1 allows 2.4.0 -> true
^2.3.1 allows 3.0.0 -> false
^2.3.1 allows 2.3.0 -> false
~2.3.1 allows 2.3.9 -> true
~2.3.1 allows 2.4.0 -> false
2.3.1 allows 2.3.1 -> true
```

Read the pattern: ^ lets MINOR and PATCH move but freezes MAJOR. ~ lets only PATCH move. A bare version pins exactly.

package.json	What you asked for : ranges like ^2.3.1. Human-edited.
package-lock.json	What you got : the exact version and integrity hash of every package in the whole tree. Machine-generated.
npm install	May update the lockfile to satisfy package.json.
npm ci	Installs exactly the lockfile, deletes node_modules first, fails if the two disagree. Use this in CI.

TRAP

Commit the lockfile for an application; do not rely on it for a library. Without it, two developers running `npm install` a week apart can get different transitive versions and one of them sees a bug the other cannot reproduce. In CI always use `npm ci`, not `npm install` — `ci` is reproducible and fails loudly when `package.json` and the lockfile have drifted apart.

12.16 Static vs dynamic typing — and floating point

One more front-end topic that gets asked as a “dev fundamentals” question.

```
function add(a, b) { return a + b; }
console.log(add(2, 3)); // 5
console.log(add("2", 3)); // "23"
console.log(add([], {})); // "[object Object]"
console.log(add(0.1, 0.2)); // 0.30000000000000004
```

```
console.log(0.1 + 0.2 === 0.3);
console.log((0.1 + 0.2).toFixed(20));
```

Real output:

```
5
23
[object Object]
0.30000000000000004
false
0.30000000000000004441
```

The last two lines are **not** a JavaScript bug. 0.1 and 0.2 are not representable in binary floating point (IEEE-754 double), just as $\frac{1}{3}$ is not representable in finite decimal. The stored value for $0.1 + 0.2$ is genuinely a hair above 0.3 , and `.toFixed(20)` shows it: `...04441`.

Every language using doubles does this — C, Java, Python. The rules:

- Never compare floats with `===`. Compare `Math.abs(a - b) < 1e-9`.
- Never store money in a float. Store **minor units** as an integer: 1099 paise, not 10.99.
- For exact integers beyond $2^{53} - 1 = 9007199254740991$, use `BigInt`.

	Static typing	Dynamic typing
Types checked	at compile time	at run time
Errors found	before shipping	when that line executes
Speed	faster — the compiler knows the layout	slower — types checked on every operation
Flexibility	less	more
Languages	C, C++, Java, Go, Rust, TypeScript	JavaScript, Python, Ruby
Note	strong/weak is a different axis	JS is dynamic and weak; Python is dynamic and strong

TRAP

Static vs dynamic and strong vs weak are two different axes, and interviewers test whether you know that. Static/dynamic = **when** types are checked. Strong/weak = whether the language **implicitly converts** between unrelated types. JavaScript is dynamic and weak ("`2`" + `3` is "`23`"). Python is dynamic and strong ("`2`" + `3` raises `TypeError`). Java is static and relatively strong. Saying "JS is weakly typed" alone is an incomplete answer.

12.17 Worked examples — Warm-up

WARM-UP

build the reflex

Example 1

Name the four stages of building a C program and the file each one produces.

SOLUTION

1. Preprocess	gcc -E	.i — still C source
2. Compile	gcc -S	.s — assembly text
3. Assemble	gcc -c	.o — object file
4. Link	gcc	executable

Answer: Preprocess → compile → assemble → link, giving .i, .s, .o, executable.

Example 2

Which direction does the stack grow, and which way does the heap grow?

SOLUTION

The stack grows **down**, toward lower addresses — measured: each call moved the local from 0x...284 to 0x...254, a drop of 48 bytes. The heap grows **up**. They approach each other across the free gap.

Answer: Stack down, heap up.

Example 3

What are the three Git object types you must be able to name, and what does each hold?

SOLUTION

blob — the bytes of one file, with no name attached.

tree — a directory listing: mode, type, hash, filename per row.

commit — one tree hash, the parent hash(es), author, committer, message.

Answer: blob (file bytes), tree (directory listing), commit (snapshot + parent + message).

Example 4

In one line each: git diff, git diff --cached, git diff HEAD.

SOLUTION

git diff — working directory vs **index**.

git diff --cached — index vs **HEAD**.

git diff HEAD — working directory vs **HEAD** (both of the above together).

Answer: Working ↔ index, index ↔ HEAD, working ↔ HEAD.

Example 5

int big[1000000]; as a global. Roughly how much does this add to the **file** on disk, and to the **process** in memory?

SOLUTION

It has no initialiser, so it lives in .bss. .bss records only “reserve this much and zero it”, so the **file** grows by almost nothing. The **process** grows by 4 MB (1000000 × 4 bytes).

Change it to `int big[1000000] = {1};` and it moves to `.data`, and now 4 MB of mostly zeros is really written into the file.

Answer: File: 0 bytes. Process: 4 MB. It is in `.bss`.

Example 6

What does `git stash` do with your **untracked** files by default?

SOLUTION

Nothing — it leaves them in the working directory. Verified: after `git stash push`, `git status --short` still showed `?? new.txt`. Use `git stash -u` to include untracked files.

Answer: Leaves them alone. Use `-u` to include them.

12.18 Worked examples — Tier 1

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 7 [TCS NQT · pattern]

Difference between a compiler and an interpreter. Give two examples of each and say which finds errors earlier.

SOLUTION

A **compiler** translates the whole program to machine code **before** it runs and writes the result to a file. An **interpreter** reads and executes the program one statement at a time, every time it runs, and produces no file.

	Compiler	Interpreter
Translates	whole program, once	one statement, every run
Output	an executable file	just the result
Errors	all syntax and type errors at build time	only when that line is reached
Speed	faster to run, slower to start building	slower to run, instant to start
Examples	C, C++, Go, Rust	classic Python, Bash

The compiler finds errors earlier — before a single user runs the program. A modern extra: a **JIT** (Java HotSpot, V8) starts by interpreting and then compiles the hot functions to machine code at run time, getting most of the speed with none of the build step.

Answer: Compiler: whole program ahead of time, produces a file, errors at build time. Interpreter: statement by statement at run time, errors when reached. JIT is the hybrid.

Example 8 [Infosys · pattern]

You get `undefined reference to 'compute'` when building a C project. Is this a compiler error or a linker error? Give three causes.

SOLUTION

Linker error. The compiler was satisfied — it found a **declaration** (a prototype), which is all it needs to emit a `call` instruction with a placeholder address. The linker then has to find the **definition**, and there is none.

Verified transcript: `gcc -c u.c -o u.o` **succeeds**; `gcc u.o -o u` fails with `undefined reference to 'helper'`.

Three causes:

1. You wrote the prototype in a header but never wrote the function body anywhere.
2. The body exists in `compute.c` but you did not compile or link it: you need `gcc main.c compute.c -o app`.

3. The function is in a library you did not link: e.g. using `sqrt` without `-lm`, or `pthread_create` without `-lpthread`.

A fourth, C++-only cause worth naming: **name mangling**. Calling a C function from C++ without wrapping its declaration in `extern "C" { ... }` makes the C++ compiler look for a mangled name the C object file does not contain.

Answer: Linker error. Missing definition, unlinked .c file, or missing -l<library> (plus extern "C" in C++).

Example 9 [Wipro · pattern]

Draw the memory layout of a running C program and say what lives in each part.

SOLUTION

Low address at the bottom:

stack	locals, parameters, return addresses; grows down	0x7fffe2b52bcc
(gap)	free space the two ends grow into	
heap	<code>malloc/new</code> ; grows up ; freed by you	0x17dca2a0
.bss	globals/statics with no initialiser, zero-filled at load	0x404044
.data	globals/statics with an initialiser	0x404030
.rodata	string literals and <code>const</code> data, read-only	0x402004
.text	the machine code, read-only	0x401196

The addresses on the right are from a real run of a program that printed the address of one object in each segment, so the ordering is measured rather than assumed.

Two facts to add, because they are the usual follow-ups: `.bss` costs nothing in the **file**, only in the **process**; and `.text` is read-only and can be **shared** between several processes running the same binary.

Answer: Bottom to top: .text, .rodata, .data, .bss, heap (grows up), gap, stack (grows down).

Example 10 [Capgemini · pattern]

Difference between `git merge` and `git rebase`. When would you use each?

SOLUTION

Merge creates one new commit with **two parents**. Existing commits keep their hashes.

Rebase replays your commits one by one on top of the target branch. Each replayed commit gets a **new hash**, because a commit's hash covers its parent.

Verified: after `git rebase main`, commit C went from 0939c0b to 9eb9636 and D from e4dd9cb to 3db7a77.

	merge	rebase
Result graph	branched, with a merge commit	one straight line
Hashes	unchanged	rewritten
Conflicts	once	possibly once per commit
Shared branch	safe	unsafe

Use **merge** to bring a finished feature into `main`, and whenever the branch is shared. Use **rebase** to tidy up **your own, unpushed** branch before opening a pull request, so reviewers see only your changes and not a merge commit from `main`.

The golden rule: never rebase commits that other people have already pulled.

Answer: Merge joins with a two-parent commit and preserves hashes; rebase replays and rewrites hashes. Merge to integrate, rebase to tidy your own private branch.

Example 11 [Accenture · pattern]

git reset vs git revert. Which is safe on a branch you have already pushed, and why?

SOLUTION

git reset moves the branch pointer backwards. The commits after it vanish from the branch — history is **rewritten**.

git revert creates a **new** commit whose content is the inverse of an old one. Nothing is removed; history is **preserved**.

Verified: after `git revert --no-edit HEAD~1`, the file `bad.txt` was gone from the working tree, but `git log` still showed C2 `add bad.txt`, with a new Revert "C2 `add bad.txt`" on top.

revert is the safe one on a pushed branch, because everyone else's clone already contains the old commits. Reset would give you a history that contradicts theirs and require a force-push, which destroys any commit they pushed in the meantime.

Bonus — the three reset modes, all measured on a repo where C2 changed `f.txt` from `v1` to `v2`:

Mode	HEAD	Index	Working file
--soft	C1	still v2 (staged)	v2
--mixed	C1	reset to C1	v2 (unstaged edit)
--hard	C1	reset to C1	v1 — edit destroyed

Answer: Reset rewrites history (unsafe once pushed); revert adds an inverse commit (safe). --soft keeps the change staged, --mixed unstages it, --hard destroys it.

Example 12 [Cognizant · pattern]

What is `#define SQ(x) x*x` going to compute for `SQ(2+3)`, and why? Fix it.

SOLUTION

The preprocessor does **text** substitution. It does not know that `2+3` is one value.

`SQ(2+3)` becomes the text `2+3*2+3`. C then applies normal precedence: $2 + (3 \times 2) + 3 = 2 + 6 + 3 = 11$.

Expected 25, got 11.

Fix — parenthesise every parameter and the whole body:

```
#define SQ(x) ((x)*(x))
```

Now `SQ(2+3)` becomes `((2+3)*(2+3)) = 5 × 5 = 25`.

There is still a second bug even with the fix: `SQ(i++)` expands to `((i++)*(i++))`, which increments `i` twice and is undefined behaviour. The real fix in C++ is `inline int sq(int x) { return x*x; }` or `constexpr`, which evaluates the argument once and type-checks it.

Answer: 11, because the macro pastes text: `2+3*2+3`. Fix with `((x)*(x))`; better, use an inline function.

12.19 Worked examples — Tier 2

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 13 [Grab · pattern]

A teammate says: “Our repo is 4 GB. Git must be storing every version of every file in full. We should switch to a system that stores diffs.” Is the diagnosis right? What would you actually check and do?

SOLUTION

The premise is half right and the conclusion is wrong.

Git **does** store a full snapshot per commit at the object level — a commit points at a tree, the tree points at blobs, and each blob is a complete file. But Git then packs loose objects into **pack files** and delta-compresses them, so on disk you do not pay full size per version.

Measured proof. Commit a 50,000-byte random file, change **one character**, commit again:

Two loose blobs in <code>.git/objects</code>	74,573 bytes
After <code>git gc</code> (packed with deltas)	39,387 bytes

So the storage model is “snapshots logically, deltas physically”. Switching version control systems would gain nothing here.

What to actually check.

1. `git count-objects -vH` — how much is loose vs packed. If loose is large, `git gc` has not run.
2. Find the big objects: list the pack index sorted by size and map hashes to paths. Nine times out of ten the answer is a handful of binaries — a 300 MB video, a committed `node_modules`, a vendored SDK, or build output.
3. `git log --all --diff-filter=A -- <path>` to see when each big file was added.

What to do.

- Add the offending paths to `.gitignore` **and** `git rm --cached` them, so they stop growing.
- Move large binaries to **Git LFS**, which stores a pointer in the repo and the bytes elsewhere.
- Only if history size is genuinely blocking people: rewrite history with `git filter-repo` to drop those blobs. This changes every commit hash after the rewrite, needs a force-push, and every collaborator must re-clone. It is a last resort, agreed with the whole team.
- For everyday relief without rewriting: `git clone --filter=blob:none` (partial clone) or `--depth=1` (shallow clone) for CI machines that do not need history.

Answer: Diagnosis is wrong: Git stores snapshots logically but delta-compresses in pack files (measured 74,573 → 39,387 bytes). The real cause is almost always large binaries. Fix with `.gitignore` + `git rm --cached`, Git LFS, shallow/partial clones, and `filter-repo` only as a last resort.

Example 14 [Shopee · pattern]

A build works on your laptop and fails in CI with a “cannot find module” error from a package nobody changed. Walk through the likely causes and how you would make the build reproducible.

SOLUTION

Read the symptom precisely: the code did not change, the **dependency tree** did. Something in the install step is not pinned.

Cause 1 — `npm install` instead of `npm ci` in CI. `npm install` is allowed to resolve ranges freshly and update the lockfile. A `^2.3.1` range accepts `2.4.0`, and that release may have dropped an export.

Verified range behaviour:

Range	2.4.0	3.0.0
<code>^2.3.1</code>	allowed	blocked
<code>~2.3.1</code>	blocked	blocked

So `^` will happily take a new MINOR, and a MINOR is where an accidental breaking change usually hides.

Cause 2 — the lockfile is not committed, or is in `.gitignore`. Then CI has nothing to install **from** and resolves ranges from scratch.

Cause 3 — a transitive dependency. Your direct deps are pinned but a dependency of a dependency is not, and only the lockfile records those.

Cause 4 — different Node versions. A native module compiled for Node 18 on your laptop, Node 22 in CI.

Cause 5 — case sensitivity. `require("./Utils")` works on macOS/Windows (case-insensitive filesystems) and fails on Linux CI. Very common and very confusing.

How to make it reproducible.

1. Commit `package-lock.json`. It records the exact version **and integrity hash** of every package in the whole tree.
2. In CI use `npm ci` — it deletes `node_modules`, installs exactly the lockfile, and **fails** if the lockfile and `package.json` disagree. `npm install` does none of that.
3. Pin the runtime: an `.nvmrc` / `engines` field, and the same version in the CI config.
4. Cache by **lockfile hash**, not by branch, so a changed lockfile always means a fresh install.
5. Build once, promote the same artefact through environments; do not rebuild per environment.

Answer: Unpinned transitive dependency resolved differently in CI. Commit the lockfile, use `npm ci` not `npm install`, pin the Node version, key the cache on the lockfile hash, and check for case-sensitivity differences on Linux.

Example 15 [Agoda · pattern]

You are on a feature branch with 9 messy commits (“wip”, “fix typo”, “wip again”). `main` has moved ahead by 12 commits. Describe exactly how you prepare this for review, and what you must check before you push.

SOLUTION

Goal: the reviewer should see a small number of meaningful commits, and a diff that contains only your work — no noise from `main`.

Step 1 — make sure nothing is uncommitted.

```
git status
git stash -u          # if anything is in progress
```

Step 2 — get the real `main`.

```
git fetch origin
```

Fetch, not pull. `fetch` updates `origin/main` without touching your branch.

Step 3 — rebase onto it.

```
git rebase origin/main
```

Your 9 commits are replayed on top of the 12 new ones. Every one of them gets a new hash. If a conflict stops the rebase, fix the files, `git add`, then `git rebase --continue`. To bail out at any point: `git rebase --abort`.

Step 4 — squash the noise.

```
git rebase -i origin/main
```

In the editor, keep pick on the commits that are real units of work, change the rest to `squash` (or `fixup` to also throw away their messages). Nine commits become, say, two: “Add retry to payment webhook” and “Add tests for webhook retry”.

Step 5 — check before pushing.

1. `git log --oneline origin/main...HEAD` — is the list exactly your commits, in a sensible order?
2. `git diff origin/main...HEAD` — does the diff contain **only** your change? Anything from `main` appearing here means the rebase went wrong.
3. Run the tests. Rebasing replays each commit but does not compile them; a silent semantic conflict is possible even with zero textual conflicts.
4. Confirm **nobody else is working on this branch**. Rebasing rewrites hashes, so if a colleague has pulled it, you will break their clone.

Step 6 — push.

```
git push --force-with-lease
```

--force-with-lease, never plain --force. Plain force overwrites the remote branch unconditionally, including a commit somebody pushed while you were rebasing. `--force-with-lease` refuses unless the remote is still where you last saw it.

Answer: fetch → `git rebase origin/main` → `git rebase -i` to squash → verify with `git log origin/main...HEAD` and `git diff origin/main...HEAD` → run tests → `git push --force-with-lease`. Only safe because the branch is yours alone.

Example 16 [DBS · pattern]

A C service crashes only in production, only under load, and only in a build compiled with `-O2`. The same build at `-O0` never crashes. What is your hypothesis and how do you test it?

SOLUTION

Hypothesis: the program has undefined behaviour, and the optimiser is exposing it.

The optimiser is allowed to assume UB never occurs. If your code relies on signed overflow wrapping, on reading uninitialised memory, on a strict-aliasing violation, or on unsynchronised access from two threads, then `-O2` may reorder, cache in a register, or delete the code around it — and only then does the bug appear. “Works at `-O0`” is **evidence of UB**, not evidence that `-O2` is broken.

Load makes it worse because a data race needs two threads to interleave badly, and higher load means more interleavings per second.

How to test it, in order of cost.

1. **Turn on the warnings you are not using.** `-Wall -Wextra -Wuninitialized -Wstrict-aliasing=2`. Free, and often enough.
2. **UndefinedBehaviorSanitizer:** rebuild with `-fsanitize=undefined -fno-omit-frame-pointer` and run the load test. It prints file and line when UB happens.
3. **ThreadSanitizer:** `-fsanitize=thread`. This is the one for “only under load” — it detects data races even when the race did not actually corrupt anything on that run.
4. **AddressSanitizer:** `-fsanitize=address` for use-after-free and buffer overruns. (Run it separately from TSan; they do not combine.)
5. **Bisect the optimisation.** If a sanitiser finds nothing, try `-O2 -fno-strict-aliasing`. If that fixes it, you have an aliasing bug — code reading the same memory through two incompatible pointer types.
6. **Bisect the code.** `git bisect run ./loadtest.sh` to find the commit that introduced it.

What is NOT the fix: shipping at `-O0`, or adding `volatile` until the symptom goes away. Both hide the bug for the current compiler version and it returns after the next upgrade. Fix the UB.

Answer: Undefined behaviour exposed by the optimiser — most likely a data race or strict-aliasing violation. Test with `-fsanitize=thread`, then `undefined`, then `address`; try `-fno-strict-aliasing` as a diagnostic; `git bisect` to find the commit. Do not ship at `-O0`.

12.20 Worked examples — Tier 3

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 17 [Google · pattern]

Explain precisely why a Git commit hash changes when you rebase, what security property the hash chain gives you, and what breaks if SHA-1 collisions become cheap.

SOLUTION

Why the hash changes.

A commit object's serialised body is, literally:

```
tree <tree-sha>
parent <parent-sha>
author <name> <email> <timestamp> <tz>
committer <name> <email> <timestamp> <tz>
```

<message>

and its name is `SHA-1("commit " + len + "\0" + body)`. The **parent hash is inside the body**. A rebase replays your change onto a different base, so the parent field is different, so the body is different, so the hash is different. There is no way to keep the hash and change the parent — that is the whole design.

Measured: rebasing C onto E changed it from 0939c0b to 9eb9636; D followed, from e4dd9cb to 3db7a77. D's hash had to change **even though its content did not**, because its parent C changed. That cascade is the point.

Note also that committer includes a **timestamp**, so cherry-picking the same change at a different moment also yields a different hash — verified: 806a058 on the branch became f888b61 on main.

The security property: a Merkle tree (hash chain).

Each commit names its tree; each tree names its blobs and sub-trees; each commit names its parent. So one commit hash transitively covers **every byte of every file in every ancestor commit**.

Consequences:

- If anyone alters a file in a commit from 2019, that blob's hash changes, so its tree's hash changes, so that commit's hash changes, so every descendant's hash changes. `git fsck` sees it immediately.
- Saying "my HEAD is 9eb9636" to a colleague is a complete, verifiable statement about the entire history. This is why signing **one** tag with GPG effectively vouches for everything beneath it.
- It also means history is **append-only in practice**: any edit to the past is visible as a change of identity, not a silent modification.

What breaks if SHA-1 collisions get cheap.

A **collision** (two inputs, same hash, attacker chooses both) has been demonstrated — SHAttered, 2017. What Git actually needs to fear is a **second-preimage** attack (given a commit that already exists, find different content with the same hash), which is much harder and has not been demonstrated.

If collisions became cheap enough to be useful, the attack shape is: get a benign blob accepted into the repo, then serve a malicious blob with the same hash. Everyone verifying by hash accepts it. Signed tags would not help, because the signature covers the hash and the hash still matches.

Git's actual mitigations:

1. **Collision detection on every hash.** Git ships `sha1dc`, which detects the specific bit patterns used by known collision attacks and aborts rather than accepting the object.
2. **SHA-256 repositories.** Supported since Git 2.29 (`git init --object-format=sha256`). Adoption is slow because interoperability with SHA-1 remotes is the hard part.
3. **Practical friction:** the attacker must get the benign half committed, and Git objects are length-prefixed and typed, which constrains what a colliding pair can look like.

Answer: The parent hash is part of the commit body, so changing the parent changes the hash, and the change cascades to every descendant. That makes Git a Merkle tree: one hash covers all history, so tampering is detectable. Cheap collisions would let a malicious blob impersonate a benign one; Git mitigates with `sha1dc` collision detection and optional SHA-256 repositories.

Example 18 [Amazon · pattern]

A service written in a JIT-compiled language has fast median latency but a terrible 99th percentile, and the worst latencies cluster in the first minutes after deployment. Explain the mechanism and give four mitigations with their costs.

SOLUTION

The mechanism, in order.

1. **Cold start.** A JIT begins by **interpreting**. Interpretation is roughly an order of magnitude slower — measured in section 12.2, the interpreted tree walk took 114 ms against 7 ms for the same work compiled to closures, a factor of about 16. Every request in the first seconds pays that.
2. **Profiling overhead.** While interpreting, the runtime counts call sites and loop iterations to decide what is hot. That counting is not free either.
3. **Compilation pauses.** When a function becomes hot, a compiler thread optimises it. On a busy machine that thread competes for CPU with request handling.
4. **Deoptimisation.** This is the one that produces **late** spikes, not just early ones. The optimiser speculates — “this call site always receives the same object shape”, “this value is always a small integer” — and compiles code that assumes it. The first request that breaks the assumption triggers a **deopt**: throw away the optimised code, fall back to the interpreter, re-profile, recompile. A rare input path can therefore cost hundreds of times the median.
5. **Garbage collection.** Warm-up allocates heavily (profiling data, compiled code, caches). Early GC pauses land on top of everything above.

Deploy all instances at once and every one of them is cold simultaneously, so the p99 for the whole fleet degrades together.

Four mitigations, with honest costs.

Mitigation	What it does	Cost
Warm-up before serving	Send synthetic traffic through the real code paths, and only then mark the instance healthy in the load balancer.	Slower deploys; the synthetic traffic must exercise the real paths or you warm the wrong code.
Rolling deploy with surge	Replace a few instances at a time, keeping warm ones serving.	Longer deploys; two versions live at once, so the code must be backwards compatible.
Ahead-of-time compilation	Compile before shipping (GraalVM native image, CRaC/AppCDS for Java, V8 code cache/snapshots).	Loses peak throughput — AOT cannot use run-time profiles; longer builds; some dynamic features stop working.
Make the code less speculative	Keep object shapes stable (same fields, same order, same types), avoid megamorphic call sites, avoid mixing types in hot arrays.	Constrains how you write code; needs profiling tools to find the megamorphic sites.

How to prove which one it is before changing anything.

- Plot latency **against instance age**. If the spike is confined to the first N seconds, it is warm-up. If it recurs later, suspect deopt or GC.
- Read the runtime’s own logs: `--trace-deopt` / `--trace-opt` in V8, JIT compilation logs in the JVM. A deopt storm is unmistakable.
- Separate GC time from CPU time in the metrics. If p99 tracks GC pause duration, warm-up is not your problem.
- Check whether p99 improves when you **slow the deploy down**. If it does, it is a fleet-wide cold start and rolling deploys are the cheapest fix.

The trade-off sentence: a JIT buys peak throughput by deciding late; the price is paid as variance early and as deoptimisation whenever a late assumption is broken. AOT buys predictability and gives up the peak.

Answer: Cold start: the JIT interprets first (measured 16× slower), profiles, then compiles, and later deoptimises when a speculation is violated; GC pauses stack on top. Mitigate with pre-serving warm-up, rolling deploys, AOT compilation, and less speculative code — each trading peak throughput or deploy speed for predictability.

Example 19 [Microsoft · pattern]

Design the Git strategy for a repository with 200 engineers, releases every two weeks, and a requirement that any release can be patched within one hour. Justify each choice and name what you are giving up.

| SOLUTION

Step 1 — pin down what the constraints actually force.

- **200 engineers** means long-lived branches are lethal: the more branches live for days, the more merge conflicts grow superlinearly with the number of concurrent editors.
- **Two-week releases** means there must be **something stable to ship**, so a pure “main is always the release” model is not enough on its own.
- **Patch within one hour** means you must be able to build a fix on the exact code that is in production, without dragging in anything merged since.

That third constraint is the decisive one: it forces a **release branch**, because you need a place to apply a fix that is not main.

Step 2 — the model.

main	Always green, always deployable. Every merge is gated by CI. Nobody commits directly.
feature/*	Short-lived — target under two days . Branch from main, rebase on main, PR, squash-merge back, delete.
release/1.14	Cut from main at the start of the release window. Tagged v1.14.0 when shipped. Kept alive for the support period.
hotfix/*	Branched from the release branch , not from main. Fix, test, merge into the release branch, tag v1.14.1, deploy. Then cherry-pick into main .

Step 3 — the one-hour patch path, concretely.

```
git switch release/1.14
git switch -c hotfix/order-total-rounding
# ...fix, test...
git switch release/1.14 && git merge --no-ff hotfix/order-total-rounding
git tag -s v1.14.1 && git push origin release/1.14 --tags
git switch main && git cherry-pick <the fix commit>
```

The cherry-pick is the step teams forget, and the failure it causes is the classic **regression on the next release**: 1.14.1 fixed the bug, 1.15 shipped without the fix, the bug came back. Automate it — a bot that opens the cherry-pick PR the moment a hotfix lands on a release branch.

Step 4 — the rules that make it survive 200 people.

1. **Squash-merge PRs into main**. One feature, one commit. It makes `git bisect` meaningful and makes cherry-picking a hotfix a single-commit operation.
2. **Merge with --no-ff from release branches** so the release history shows discrete units.
3. **Feature flags for anything that spans more than two days**. Merge incomplete work behind a flag rather than keeping a branch open. This is what actually keeps branches short.
4. **Protected branches**: no force-push to main or release/*, required reviews, required CI. Force-push only ever with `--force-with-lease`, and only on personal branches.
5. **Signed, annotated tags** for releases. The hash chain then means one signature vouches for the whole tree.
6. **Merge queue**. At 200 engineers, “CI passed on my branch” is not “CI passes on main” — something else merged in between. A merge queue re-tests each PR against the current main before landing it.
7. **CODEOWNERS** so reviews route automatically instead of by asking in chat.

Step 5 — what you are giving up. Name this; it is the part that distinguishes a senior answer.

- **Not trunk-based**. Release branches add ceremony and a permanent risk of divergence between main and release/*. You accept that cost to buy the one-hour patch guarantee.
- **Squash-merging destroys intermediate commits**. You lose fine-grained history and per-commit authorship inside a feature. You accept that to get a bisectable main.
- **Feature flags are technical debt**. Every flag is a branch in the code and a combination to test. You need an owner and an expiry date per flag, and a quarterly sweep, or they accumulate.
- **A merge queue adds latency**. Time from approval to landing grows, and the queue itself needs capacity and monitoring.

- **Supporting multiple release branches multiplies backport work.** Cap it — support the last two releases, not five.

Answer: Trunk-ish main plus short-lived feature branches, cut a **release/*** branch per release, hotfix from the release branch then cherry-pick back to main. Squash-merge, protected branches, merge queue, feature flags, signed tags. **Giving up:** pure trunk-based simplicity, fine-grained history, flag debt, merge-queue latency, and backport cost.

12.21 Practice

PRACTICE · WARM-UP target: 6 min

1. Which stage turns `.s` into `.o`?
2. Which segment holds a string literal `"hello"`?
3. Which segment holds an uninitialised global?
4. What is inside `.git/refs/heads/main`?
5. `git diff --cached` compares what to what?
6. Does a commit store a diff or a snapshot?
7. Which command is safe on an already-pushed branch — `reset` or `revert`?

ANSWER KEY

1. The assembler (`gcc -c`). 2. `.rodata` — read-only. 3. `.bss`. 4. A single 40-character commit SHA. 5. The index versus HEAD. 6. A snapshot (a tree hash); deltas happen later in pack files. 7. `revert`.

PRACTICE · TIER 1 target: 14 min

1. Write the four `gcc` commands for the four stages and name each output file.
2. Explain in three lines why `gcc -c` succeeds but `gcc` fails with “undefined reference”.
3. Give three differences between static and dynamic linking, with a number for at least one.
4. What does `git cat-file -p HEAD` print? Name every field.
5. Explain the three `git reset` modes in terms of HEAD, index and working directory.
6. A colleague force-pushed and your commits vanished. How do you recover them?
7. What does `.bss` cost in the file, and what does it cost in memory?

ANSWER KEY

1. `gcc -E f.c -o f.i`; `gcc -S f.i -o f.s`; `gcc -c f.s -o f.o`; `gcc f.o -o f`.
2. `-c` stops after assembling. The compiler only needs a **declaration** to emit a `call` with a placeholder address. The linker needs a **definition** and cannot find one.
3. Size (measured: 785,328 bytes static vs 15,960 dynamic — about 49×); RAM sharing (one copy of a `.so` serves every process); patching (replace the `.so` once vs rebuild every program). Plus: static starts faster, dynamic can fail at load time.
4. `tree <sha>`, `parent <sha>` (absent on the first commit, twice on a merge), `author name/email/timestamp/tz`, `committer` (same fields), blank line, then the message.
5. `--soft`: moves HEAD only — the change stays **staged**. `--mixed` (default): moves HEAD and resets the index — the change stays in the **working directory**, unstaged. `--hard`: moves all three — **the change is destroyed**.
6. `git reflog` on **your** machine still lists every position HEAD held. Find the hash, then `git switch -c rescue <hash>` (or `git reset --hard <hash>`). The reflog is local, so this works only on a machine that had the commits.
7. Nearly zero bytes in the file (it records only “reserve this much, zero-filled”). Full size in memory once the process starts.

PRACTICE · TIER 2 target: 18 min

1. Your branch has 6 commits; `main` moved by 20. Give the exact command sequence to prepare a clean PR, and the one flag that makes the final push safe.
2. Explain why `0.1 + 0.2 != 0.3` and give the correct way to store a price.

3. CI passes, production fails, same commit. Give five hypotheses.
4. Why is `npm ci` preferred over `npm install` in CI? What exactly does it do differently?
5. A file was committed with a password in it three months ago. It is no longer in the working tree. Is the secret safe? What do you do?

ANSWER KEY

1. `git fetch origin → git rebase origin/main → git rebase -i origin/main` (squash the noise) → verify with `git log --oneline origin/main...HEAD` and `git diff origin/main...HEAD` → run tests → `git push --force-with-lease`. The safe flag is `--force-with-lease`, which refuses if the remote moved since you last fetched.
2. Neither `0.1` nor `0.2` is exactly representable in binary IEEE-754, so the sum is a hair high — `(0.1+0.2).toFixed(20)` really prints `0.30000000000000004441`. Store money as an **integer in minor units** (1099 paise) or use a decimal library; compare floats with a tolerance, never `===`.
3. (a) Different environment variables or config. (b) Different data — production has rows CI's fixtures do not. (c) Different dependency versions (unpinned lockfile). (d) Concurrency and load — races only appear with real traffic. (e) Different OS or filesystem (case sensitivity, path separators, locale). Also: CI runs a fresh process; production has long-lived state and caches.
4. `npm ci` deletes `node_modules`, installs **exactly** what `package-lock.json` says, never updates the lockfile, and **fails** if `package.json` and the lockfile disagree. `npm install` resolves ranges and may silently change the lockfile, so two CI runs on the same commit can install different trees.
5. **It is not safe.** Removing a file from the working tree does not remove it from history — `git log --all -- <path>` still finds the blob, and anyone with a clone has it. Order of operations: (a) **rotate the credential immediately** — this is the only step that actually fixes the exposure; (b) remove it from the current tree and add it to `.gitignore`; (c) if the repo is private and the team agrees, rewrite history with `git filter-repo`, force-push, and have everyone re-clone; (d) add a secret-scanning pre-commit hook so it cannot happen again.

PRACTICE · TIER 3 target: 22 min

1. Explain why `git rebase` can produce a build that fails even though every individual commit merged with zero textual conflicts.
2. You have a 40-minute CI pipeline and 200 engineers merging. Describe how “CI green on my branch” can still break `main`, and what fixes it.
3. Why can `-O2` break a correct-looking multithreaded C program that works at `-O0`? Name the two language-level tools that make it correct.
4. Given only a commit hash, explain exactly what you can and cannot prove about the repository.

ANSWER KEY

1. Git merges **text**, not **meaning**. A **semantic conflict**: your branch adds a call to `helper(a, b);` `main` independently changed `helper` to take three parameters. Neither change touches the other's lines, so there is no textual conflict, and the rebase is silent — but the build is broken. Only running the build and tests after the rebase catches it. This is why “rebased cleanly” is never the same as “it works”.
2. Your branch was tested against `main` as it was when you branched. By the time you merge, 40 minutes of other merges have landed. Two PRs can each pass alone and conflict semantically together. Fixes: a **merge queue** that re-tests each PR against the current `main` immediately before landing; required “branch up to date before merge”; splitting the pipeline so a fast subset gates the merge and the slow suite runs post-merge with automatic revert on failure; and reducing batch size so fewer changes are in flight at once.
3. Without synchronisation, the compiler may keep a shared variable in a register across a loop, reorder stores, or delete a read it believes is redundant — all legal, because a data race is **undefined behaviour** and the optimiser may assume there is none. At `-O0` the compiler reloads from memory every time, which accidentally hides the bug. The two correct tools: a **mutex** (`pthread_mutex_t` / `std::mutex`) for anything non-trivial, and **atomics** with a defined memory order (`_Atomic` / `std::atomic`) for single variables. `volatile` is **not** one of them — it prevents compiler caching but gives no ordering or atomicity guarantees against other threads.

4. **Can prove:** that the entire history reachable from that commit — every file, every tree, every ancestor commit, every author line and timestamp — is exactly what it was, because each object's hash covers its contents and each commit's hash covers its parent and tree. Anyone presenting different bytes under that hash is detectable. **Cannot prove:** who wrote it (the author field is free text; only a GPG/SSH signature binds an identity), that it is the **latest** commit on any branch (a hash names a point, not a tip), that any branch still points at it (refs move; the commit may be unreachable and pending garbage collection), or anything about commits that are not its ancestors.

12.22 Rapid fire — one-line answers

Cover the right column. Say each answer out loud.

Question	Answer
Four stages of building a C program?	Preprocess → compile → assemble → link.
What does the preprocessor actually do?	Text substitution: expand <code>#include</code> and <code>#define</code> , strip comments.
Compiler vs interpreter in one line?	Compiler makes a file ahead of time; interpreter produces an answer as it goes.
What is a JIT?	Interprets first, compiles the hot code at run time. V8, HotSpot.
undefined reference vs “was not declared”?	Linker vs compiler. One found the declaration but no definition; the other found neither.
Which segment: <code>"hi" · int g; · int g = 5;?</code>	<code>· rodata · .bss</code> (free on disk) <code>· .data</code> .
Stack direction? Heap direction?	Stack grows down, heap grows up.
What is in a stack frame?	Return address, saved registers, parameters, locals.
Stack overflow vs heap exhaustion?	Deep recursion vs too many allocations. Different limits, different errors.
JS recursion depth limit, roughly?	Measured 12,500 frames — so use an explicit stack past 10^4 .
Three optimisations by name?	Constant folding, dead code elimination, loop-invariant code motion.
Why can -O2 break working code?	The program has undefined behaviour; the optimiser assumes it cannot happen.
Git’s three object types?	blob (file bytes), tree (directory listing), commit (snapshot + parent + message).
How is a blob named?	<code>SHA-1("blob " + length + "\\0" + content)</code> — verified by hand.
Is a commit a diff or a snapshot?	A snapshot. Deltas exist only inside pack files.
What is a branch, physically?	A file holding one 40-character commit SHA.
What is HEAD? What is detached HEAD?	A one-line file naming your branch. Detached = it holds a raw SHA, so new commits belong to no branch.
<code>git diff</code> vs <code>--cached</code> vs HEAD?	Working ↔ index, index ↔ HEAD, working ↔ HEAD.
merge vs rebase, and why do hashes change?	Merge adds a two-parent commit. Rebase replays onto a new parent — and the parent SHA is inside the hashed body, so every hash is rewritten.
The golden rule of rebase?	Never rebase commits that others have already pulled.
<code>reset --soft</code> / <code>--mixed</code> / <code>--hard</code> ?	Staged / unstaged / destroyed.
reset vs revert?	Reset rewrites history; revert adds an inverse commit. Revert is safe once pushed.
Does <code>git stash</code> take untracked files?	No, not by default. Use <code>-u</code> .
Does <code>.gitignore</code> untrack a tracked file?	No. You need <code>git rm --cached</code> , and history still has it.
How do you recover from a bad <code>reset --hard</code> ?	<code>git reflog</code> , find the hash, <code>git reset --hard HEAD@{1}</code> .
<code>--force</code> vs <code>--force-with-lease</code> ?	Lease refuses if the remote moved since your last fetch. Always use lease.

npm install vs npm ci?	ci installs the lockfile exactly, wipes node_modules, and fails on drift.
What do ^2.3.1 and ~2.3.1 allow?	^: any 2.x ≥ 2.3.1. ~: any 2.3.x ≥ 2.3.1.
Why is 0.1 + 0.2 != 0.3, and how do you store money?	Neither is exactly representable in binary IEEE-754. Store money as an integer in minor units.
Static/dynamic vs strong/weak typing?	When types are checked vs whether implicit conversion happens. Two axes.

ONE-PAGE REVISION CARD

Build pipeline. Preprocess (-E, text) → compile (-S, assembly) → assemble (-c, object) → link (executable). Measured: 6 lines → 819 → 56 asm lines → 1528 B → 15960 B.

Error, at which stage? “not declared” = compiler. “undefined reference” = linker. “multiple definition” = linker. Works at -O0, breaks at -O2 = undefined behaviour in your code.

Linking. Static: one big file (785 KB vs 16 KB measured), no runtime dependency, rebuild to patch. Dynamic: small, one shared copy in RAM, patch the .so once.

Memory layout, low to high: .text 0x401196 → .rodata 0x402004 → .data 0x404030 → .bss 0x404044 → heap 0x17dca2a0 (grows ↑) → gap → stack 0x7fffe2b52bcc (grows ↓). .bss is free on disk, not free in RAM.

Stack frames. Measured 48 bytes each, addresses decreasing. JS dies at 12,500 frames — past 10^4 , use an explicit stack.

Compiler front end. Lexer → tokens. Parser → AST (precedence is the grammar’s shape). Semantic analysis → symbol table, type checks.

Git objects. blob = file bytes; tree = directory listing; commit = tree + parent + author + message. Blob name = SHA-1("blob " + len + "\\0" + content) — verified by hand against git hash-object. Commits are **snapshots**; deltas live in pack files.

Git is files. A branch is 41 bytes holding a SHA. HEAD is one line. Detached HEAD = a raw SHA in HEAD. git reflog remembers every position and is the undo button.

The four commands people confuse.

1. reset --soft keeps the change **staged**; --mixed **unstaged**; --hard **destroys** it.
2. revert adds an inverse commit — the only safe undo on a pushed branch.
3. rebase rewrites hashes (verified 0939c0b → 9eb9636). Never on a shared branch.
4. cherry-pick copies a change and gives it a new hash.

Dependencies. package.json = what you asked for; the lockfile = what you got. npm ci in CI, always. ^ frees MINOR+PATCH, ~ frees PATCH only.

Floating point. $0.1 + 0.2 = 0.30000000000000004$. Money goes in integer minor units. Compare floats with a tolerance. Past $2^{53} - 1$, use BigInt.

Quick Reference

The whole book compressed. Read this in the last week, and again in the last hour.

Everything below is already explained somewhere in this book. Nothing here is new. If a line does not make sense, that is your revision list — go back to the chapter, not to the internet.

A • The formula sheet

EVERY NUMBER YOU MAY HAVE TO COMPUTE

Scheduling	Turnaround = completion — arrival. Waiting = turnaround — burst. Response = first-run start — arrival. Average = sum ÷ number of processes.
Throughput	processes finished ÷ total time. CPU utilisation = busy time ÷ total time.
Effective access (TLB)	$EAT = h \times (t_{TLB} + t_{mem}) + (1 - h) \times (t_{TLB} + 2 \times t_{mem})$, with h the hit ratio. Two memory hits on a miss because the page table is in memory too.
Effective access (paging)	$EAT = (1 - p) \times t_{mem} + p \times t_{fault}$, where p is the page-fault rate. t_{fault} is milliseconds against nanoseconds — that is why p must be tiny.
Page/frame split	page size $2^n \Rightarrow$ offset is n bits. Virtual address m bits $\Rightarrow$ page number is $m - n$ bits $\Rightarrow 2^{m-n}$ entries in a one-level page table.
Address translation	page number = address ÷ page size; offset = address mod page size. Physical = frame $\times$ page size + offset.
Disk time	access = seek + rotational latency + transfer. Average rotational latency = half a rotation = $\frac{60 \div RPM}{2}$.
Disk scheduling	score = total head movement in cylinders. Add every hop, including the walk to the end of the disk in SCAN and the wrap in C-SCAN.
Subnetting	$/p \Rightarrow$ hosts per subnet = $2^{32-p} - 2$; subnets from a $/q$ block = 2^{p-q} ; block size in the interesting octet = $256 - \text{mask octet}$.
TCP throughput	window ÷ RTT. Bandwidth-delay product = bandwidth $\times$ RTT — the window must be at least this to fill the pipe.
B+ tree height	with order n , each node holds up to n pointers, so height $\approx \log_n(\text{rows})$. Disk reads = height, plus one more if the index does not cover the query.
Index selectivity	distinct values ÷ total rows. Near 1 is excellent; near 0 (a two-value flag) means the planner will ignore the index and scan.

SHORTCUT

In any Gantt question, build the **chart first and completely**, then fill one table with columns arrival / burst / completion / turnaround / waiting. Every mark is in that table. Doing arithmetic while drawing the chart is where the slips happen.

B • Operating systems — Chapters 1 to 4

B1 • The comparisons they ask by name

Pair	Left	Right
Process vs thread	Own address space. Creation is heavy. Crash is contained. IPC needed to talk.	Shares code, data and heap; own stack, registers, PC . Cheap switch. One crash kills all.

Context switch vs mode switch	Between two processes: save PCB, swap page tables, flush or tag the TLB. Expensive.	User mode to kernel mode inside the same process, via a system call. Much cheaper.
Preemptive vs non-preemptive	OS can take the CPU back on a timer. Better response, needs locking.	A process holds the CPU until it blocks or exits. Simple, but one long job stalls everyone.
Mutex vs semaphore	Ownership: the locker must unlock. Binary. A lock .	A counter, no owner; any thread may signal. A signalling device.
Binary semaphore vs mutex	Counter capped at 1, still no ownership, no priority inheritance.	Owned, and typically supports priority inheritance — the correct choice for mutual exclusion.
Deadlock vs starvation vs livelock	Circular wait; nobody moves, ever.	Starvation: work is possible but this one never gets picked. Livelock: all busy, none progressing.
Paging vs segmentation	Fixed-size pages. Internal fragmentation. Invisible to the programmer.	Variable-size logical units. External fragmentation. Visible and meaningful.
Internal vs external fragmentation	Wasted space inside an allocated block (last page half empty).	Free space exists but is scattered — no single hole is big enough.
Logical vs physical address	What the CPU emits. Also called virtual. Bounded by the address width.	What the memory bus sees. The MMU converts one to the other, at every access.
Hard link vs soft link	Another name for the same inode . Same filesystem only. Original can be deleted.	A file holding a path string . Crosses filesystems. Breaks when the target moves.
Thrashing vs a page fault	Faulting so often that CPU utilisation collapses. Fix: fewer processes, or more frames.	One normal, expected event. Demand paging requires page faults to work at all.

B2 · Algorithms in one line each

FCFS	Queue order. Simple, fair-looking, suffers the convoy effect — one long job delays every short one.
SJF	Shortest burst first. Provably minimal average waiting time, but burst length must be estimated , and long jobs starve.
SRTF	Preemptive SJF. Even lower averages, more context switches, worse starvation.
Round Robin	Quantum q . Great response time, poor waiting time. q too big → FCFS; q too small → all switching overhead.
Priority	Highest priority first. Starvation is the flaw; aging (raise priority while waiting) is the fix.
Multilevel feedback	Several queues, a job that uses its whole quantum drops a level. Approximates SJF without knowing burst lengths.
FIFO (pages)	Oldest page out. The only one that suffers Belady's anomaly — more frames can mean more faults.
Optimal (OPT)	Evict the page used furthest in the future. Unimplementable; it is the benchmark you compare against.
LRU	Evict least recently used. Good, but needs a stack or timestamps. Stack algorithm, so no Belady's anomaly.
Clock / second chance	Circular scan of reference bits. The practical approximation of LRU, and what real kernels ship.
FCFS (disk)	Requests in arrival order. Fair, lots of wasted seeking.
SSTF	Nearest cylinder next. Good throughput, starves the far edges.
SCAN / LOOK	Sweep one way then reverse. LOOK turns at the last request instead of the disk edge.
C-SCAN / C-LOOK	Sweep one way, jump back, sweep again. More uniform waiting than SCAN.
Banker's	Grant a request only if a safe sequence still exists afterwards. Needs maximum demand declared up front. Avoidance, not detection.

B3 · Rapid fire — OS

1. **Five process states?** New, Ready, Running, Waiting/Blocked, Terminated.
2. **Ready to Waiting — possible?** No. Only Running can block. Waiting always returns to Ready.
3. **What is in the PCB?** PID, state, program counter, registers, scheduling info, memory maps, open-file table, accounting.
4. **Zombie?** Finished child whose exit status the parent has not reaped with `wait()`. It holds only a PCB entry.

5. **Orphan?** Parent died first; `init/systemd` adopts it and reaps it.
6. **What does `fork()` return?** 0 in the child, the child's PID in the parent, `-1` on failure.
7. **Copy-on-write?** Parent and child share physical pages read-only; the first **write** triggers the copy. Makes `fork` cheap.
8. **Thread stack shared?** No — each thread has its own stack, registers and program counter.
9. **User vs kernel threads?** User: fast switching, one blocking call blocks all. Kernel: the OS schedules them, real parallelism.
10. **Critical section requirements?** Mutual exclusion, progress, bounded waiting.
11. **Peterson's solution needs?** Two variables — `flag[]` and `turn` — and it assumes no instruction reordering.
12. **Why is `test-and-set` special?** It is **atomic** in hardware: read and write happen with no gap to interleave into.
13. **Spinlock — when?** Only when the wait is shorter than a context switch, and never on a single CPU with preemption off.
14. **Busy waiting cost?** Burns CPU doing nothing. Semaphores with a wait queue block instead.
15. **Four deadlock conditions?** Mutual exclusion, hold-and-wait, no preemption, circular wait — **all four at once**.
16. **Easiest one to break?** Circular wait: impose a global lock ordering. It is the standard real-world fix.
17. **Avoidance vs prevention vs detection?** Prevention breaks a condition structurally. Avoidance (Banker's) checks each request. Detection lets it happen, then recovers.
18. **Cycle in a wait-for graph?** With one instance per resource, a cycle **is** deadlock. With multiple instances, a cycle is only a possibility.
19. **Priority inversion?** A low-priority thread holds a lock a high-priority thread needs. Fix: priority inheritance.
20. **Why virtual memory?** Run programs larger than RAM, isolate processes, and share pages without copying.
21. **What is in a page-table entry?** Frame number plus valid, protection, reference, dirty and caching bits.
22. **Why multi-level page tables?** A flat table for a 32-bit space is megabytes **per process**. Levels let unused ranges stay unallocated.
23. **Inverted page table?** One entry per **frame**, not per page. Tiny, but lookup needs a hash.
24. **Demand paging?** Load a page only on its first touch. Start-up is fast; the cost is paid per fault.
25. **Belady's anomaly — which algorithms?** FIFO, and not LRU or OPT, because those are stack algorithms.
26. **Working set?** The pages a process touched in the last Δ references. Frames below the working set means thrashing.
27. **Inode holds?** Size, permissions, owner, timestamps, link count and block pointers — **not** the filename.
28. **Where is the filename?** In the directory entry, which maps name to inode number.
29. **Why does `rm` on a big open file free nothing?** The link count reaches 0 but the **open count** has not; blocks are freed at the last close.
30. **Contiguous vs linked vs indexed allocation?** Contiguous: fast random access, external fragmentation. Linked: no fragmentation, no random access. Indexed (inode): both, at the cost of an index block.
31. **Journaling gives you?** Crash consistency — write the intent to a log first, so recovery replays or discards a half-done operation.
32. **What does `fsync` cost?** It forces the page cache to durable storage and waits. Correct, and slow, which is why databases batch it.
33. **RAID 0 / 1 / 5 / 10?** 0 stripes (speed, no safety); 1 mirrors (safety, half capacity); 5 stripes with distributed parity (one disk may fail); 10 mirrors then stripes (fast and safe, half capacity).
34. **Why are SSD rules different?** No seek time, but erase happens in large blocks — hence wear levelling, the FTL, and why TRIM matters.

C · Databases — Chapters 5 to 8

C1 · SQL you must produce without thinking

Logical order	FROM → ON → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. This is why a SELECT alias cannot be used in WHERE, but can be in ORDER BY.
WHERE vs HAVING	WHERE filters rows before grouping; HAVING filters groups after . An aggregate can only appear in HAVING.
JOIN types	INNER keeps matches only. LEFT keeps all left rows, NULL-filling the right. FULL keeps both sides. CROSS is every pair.
The LEFT JOIN trap	Putting the right table's condition in WHERE silently turns it back into an INNER JOIN. Put it in the ON clause instead.
NULL logic	NULL = NULL is unknown, not true. Use IS NULL. COUNT(col) skips NULLs; COUNT(*) does not.
NOT IN trap	If the subquery returns a single NULL, NOT IN returns no rows at all . Use NOT EXISTS.
UNION vs UNION ALL	UNION removes duplicates (a sort or hash, so slower). UNION ALL just concatenates. Default to ALL when you know there are no duplicates.
DELETE / TRUNCATE / DROP	DELETE is DML, row-by-row, logged, WHERE-able, rollback-able, fires triggers. TRUNCATE is DDL, deallocates pages, no WHERE, resets identity. DROP removes the table itself.

Window vs GROUP BY	GROUP BY collapses rows into one per group. A window function keeps every row and adds the computed column beside it.
ROW_NUMBER / RANK / DENSE_RANK	On a tie: 1,2,3 — 1,1,3 — 1,1,2.
Correlated subquery	It references the outer row, so it runs per row . Usually rewritable as a join, usually faster that way.
CTE	WITH x AS (. . .) — a named result for readability. WITH RECURSIVE walks hierarchies: anchor, UNION ALL, recursive step.

C2 · Keys, normal forms and transactions

Super key	Any attribute set that identifies a row uniquely. Every candidate key is a super key.
Candidate key	A minimal super key — remove any attribute and uniqueness breaks.
Primary key	The candidate key you picked. Unique and NOT NULL. One per table.
Unique key	Also unique, but may allow a NULL (one, in most engines). Any number per table.
Foreign key	Points at a candidate key of another table. Enforces referential integrity; may be NULL.
Prime attribute	An attribute belonging to some candidate key. Everything else is non-prime.
Closure X^+	Start with X ; keep adding right-hand sides of every FD whose left side is already inside. If X^+ is all attributes, X is a super key.

Form	Rule	It removes
1NF	Every value is atomic; no repeating groups or lists in a cell.	Lists stuffed into one column.
2NF	1NF, and no partial dependency — no non-prime attribute depends on part of a composite key.	Duplication tied to half the key.
3NF	2NF, and no transitive dependency — non-prime attributes must not determine other non-prime attributes.	Insert/update/delete anomalies.
BCNF	For every FD $X \rightarrow Y$, X must be a super key.	The last anomalies — but may lose dependency preservation.
4NF	BCNF, and no non-trivial multivalued dependency unless the left side is a super key.	The Cartesian blow-up of two independent lists.

3NF vs BCNF is the question that decides the round. 3NF allows $X \rightarrow Y$ where X is not a super key **provided** Y is prime. BCNF forbids it outright. A lossless, dependency-preserving 3NF decomposition always exists; a BCNF one does not always. That trade-off is the answer.

ACID	Atomicity — all or nothing (undo log). Consistency — constraints hold before and after. Isolation — concurrent looks serial. Durability — committed survives a crash (redo log, WAL).
WAL rule	The log record reaches durable storage before the data page does. That single rule is what makes both undo and redo possible.
Dirty read	Reading a row another transaction wrote but has not committed.
Non-repeatable read	Reading the same row twice and getting different values.
Phantom read	Running the same range query twice and getting different row counts.
Lost update	Two read-modify-writes overlap; one overwrites the other's result.
2PL	Growing phase acquires locks, shrinking phase releases them. Guarantees serialisability. Strict 2PL holds every exclusive lock to commit — that is what prevents cascading rollback.
MVCC	Readers see a snapshot, writers create new versions. Readers never block writers. The cost is version storage and vacuuming.

Isolation level	Dirty read	Non-repeatable	Phantom
READ UNCOMMITTED	possible	possible	possible
READ COMMITTED	prevented	possible	possible
REPEATABLE READ	prevented	prevented	possible
SERIALIZABLE	prevented	prevented	prevented

C3 · Indexing

Why B+ and not B?	All data lives in the leaves and the leaves are linked, so a range scan is one descent plus a walk. Internal nodes hold only keys, so fan-out is higher and the tree is shorter.
Clustered index	Defines the physical row order. One per table. The leaf is the row — no second lookup.
Non-clustered	A separate structure whose leaf holds a pointer. Many per table. Costs an extra fetch unless the index covers the query.
Covering index	Contains every column the query needs, so the table is never touched. This is the single biggest easy win.
Composite index order	Leftmost-prefix rule: an index on (a, b, c) serves a, (a,b) and (a,b,c) — never b alone. Put equality columns first, the range column last.
Hash index	$O(1)$ equality, useless for ranges and for ORDER BY. B+ tree is the safe default.
When an index hurts	Every INSERT/UPDATE/DELETE must maintain it; low selectivity makes the planner scan anyway; and a wrapped column (WHERE YEAR(d) = 2024) disables it entirely — rewrite as a range.
Why is my index ignored?	Function on the column, leading wildcard LIKE '%x', type mismatch forcing a cast, stale statistics, or the query genuinely touching most of the table.
Reading EXPLAIN	Look for the access type (seek vs scan), the rows examined versus rows returned, and any sort or temporary table. A large gap between examined and returned is the bug.

C4 • Rapid fire — DBMS

1. **DDL / DML / DCL / TCL?** CREATE,ALTER,DROP,TRUNCATE / SELECT,INSERT,UPDATE,DELETE / GRANT,REVOKE / COMMIT,ROLLBACK,SAVEPOINT.
2. **View?** A stored query. Simplifies and restricts access. A **materialised** view stores the result and must be refreshed.
3. **Can you update a view?** Only a simple one — no aggregate, no DISTINCT, no GROUP BY, one base table.
4. **Trigger vs stored procedure?** A trigger fires automatically on a table event; a procedure is called explicitly.
5. **ER: cardinality?** 1:1, 1:N, M:N. M:N always becomes a third **junction** table with two foreign keys.
6. **Weak entity?** No key of its own; identified by its owner's key plus a partial key. Drawn with a double rectangle.
7. **Surrogate vs natural key?** Surrogate is a meaningless generated id (stable, joins fast). Natural carries meaning (readable, but meanings change).
8. **Denormalise when?** Reads dominate, the join is proven to be the bottleneck, and you accept writing the duplicate consistently.
9. **Schedule is conflict serialisable?** Draw the precedence graph. No cycle $\Rightarrow$ yes.
10. **Recoverable schedule?** A transaction commits only after every transaction it read from has committed.
11. **Cascading rollback?** One abort forces others to abort because they read its uncommitted data. Strict 2PL prevents it.
12. **Deadlock in a database?** Two transactions each hold a lock the other wants. The engine detects it and kills a victim — your code must be ready to **retry**.
13. **Shared vs exclusive lock?** Many readers together (S), one writer alone (X). S and X conflict.
14. **Checkpoint?** A marker in the log saying “everything before this is on disk”, so recovery does not replay from the beginning.
15. **OLTP vs OLAP?** Many small writes, normalised, row storage / few huge reads, denormalised star schema, column storage.
16. **SQL vs NoSQL?** Fixed schema, joins and strong transactions / flexible documents, horizontal scaling, application-side joins.
17. **CAP?** During a network **partition** you choose consistency or availability. It is not a menu for normal operation.
18. **Sharding vs partitioning?** Sharding splits across **machines**; partitioning splits within **one** database.
19. **CHAR vs VARCHAR?** Fixed width, padded, slightly faster / variable width, stores a length, saves space.
20. **DISTINCT cost?** It forces a sort or a hash over the whole result. If you need DISTINCT, suspect a join that is duplicating rows.

D • Networks — Chapters 9 and 10

D1 • The layers, and who does what

Layer	Name	Job	Unit • examples
7	Application	What the user's program speaks.	data • HTTP, DNS, SMTP
6	Presentation	Encoding, compression, encryption.	data • TLS, JPEG
5	Session	Opening, syncing and closing a dialogue.	data • RPC
4	Transport	Process-to-process; ports, reliability, ordering.	segment • TCP, UDP
3	Network	Host-to-host across networks; logical addressing.	packet • IP, ICMP
2	Data link	Hop-to-hop on one link; MAC, framing, error check.	frame • Ethernet, ARP
1	Physical	Bits as voltage, light or radio.	bit • cable, Wi-Fi

TCP/IP collapses this to four: **Application** (OSI 5–7), **Transport**, **Internet**, **Link**. Repeater/hub = layer 1. Switch/bridge = layer 2. Router = layer 3. Firewall and load balancer = layer 3, 4 or 7 depending on what it inspects.

D2 · Addressing and subnetting

Private ranges	10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16. Loopback 127.0.0.0/8. Link-local 169.254.0.0/16 — seeing that address means DHCP failed.
Mask shorthand	/24 = 255.255.255.0 · /25 = .128 · /26 = .192 · /27 = .224 · /28 = .240 · /29 = .248 · /30 = .252.
Block size	256 — the interesting mask octet. A /26 has block size 64, so networks start at .0, .64, .128, .192.
Usable hosts	$2^{32-p} - 2$ — subtract the network address and the broadcast address. A /30 gives 2 usable, which is exactly one router link.
Broadcast address	The last address in the block — one less than the next network address.
Public vs private	NAT rewrites a private source address and port to the router's public pair, and keeps a table to reverse it on the way back.
IPv6	128 bits, hexadecimal, :: collapses one run of zeros. No broadcast, no NAT needed, no fragmentation by routers, and SLAAC for autoconfiguration.

D3 · TCP, UDP and HTTP

Aspect	TCP	UDP
Connection	Three-way handshake first.	None — just send.
Guarantees	Ordered, reliable, de-duplicated, or it reports failure.	Best effort. May be lost, duplicated or reordered.
Control	Flow control and congestion control.	Neither. The application must cope.
Header	20 bytes minimum.	8 bytes.
Use when	Web, mail, file transfer, database connections.	DNS, video and voice, gaming, QUIC's foundation.

Handshake	SYN → SYN-ACK → ACK. It synchronises the initial sequence numbers in both directions — that is the real purpose, not “saying hello”.
Close	FIN → ACK → FIN → ACK. Four steps, because each direction closes independently (half-close).
TIME_WAIT	The active closer waits 2×MSL so late duplicates die and the final ACK can be resent. It is correct behaviour, not a leak.
Flow vs congestion control	Flow control protects the receiver (its advertised window). Congestion control protects the network (the congestion window). Effective window is the smaller of the two.
Slow start	cwnd doubles every RTT until ssthresh, then congestion avoidance adds one MSS per RTT (additive increase).
On loss	Triple duplicate ACK ⇒ fast retransmit and fast recovery, halving cwnd. Timeout ⇒ the network is badly congested: cwnd back to 1 and slow start again.
Nagle vs delayed ACK	Nagle batches small writes; delayed ACK holds acknowledgements. Together they can add avoidable latency — the reason interactive apps disable Nagle.
HTTP is stateless	Every request stands alone. Cookies, tokens and sessions are what add the memory on top.
Idempotent methods	GET, PUT, DELETE, HEAD — repeating them is harmless. POST is not. GET must also be safe : no side effects.
PUT vs PATCH	PUT replaces the whole resource; PATCH applies a partial change.
Status classes	1xx info · 2xx success · 3xx redirect · 4xx you made the mistake · 5xx the server did. Know 200, 201, 204, 301, 302, 304, 400, 401, 403, 404, 409, 429, 500, 502, 503.
401 vs 403	401 = not authenticated (who are you?). 403 = authenticated but not allowed.
HTTP/1.1 → 2 → 3	1.1 adds keep-alive and pipelining but suffers head-of-line blocking. 2 adds binary framing, multiplexing and header compression — but TCP-level head-of-line blocking remains. 3 moves to QUIC over UDP, which removes it and gives 0-RTT resumption.
HTTPS handshake	Asymmetric keys authenticate the server and agree a symmetric session key; the rest of the traffic uses that faster symmetric key. The certificate chain is what proves identity.
Cookie flags	HttpOnly hides it from JavaScript, Secure means HTTPS only, SameSite blocks cross-site sending (the CSRF defence).
CORS	A browser rule, not a server firewall. A non-simple request is preflighted with OPTIONS, and the server must answer with the matching Access-Control-Allow-* headers.

D4 · Rapid fire — Networks

1. **What happens when you type a URL?** DNS resolve → TCP handshake → TLS handshake → HTTP request → response → parse → sub-resource requests → render.
2. **DNS resolution order?** Browser cache → OS cache → hosts file → resolver → root → TLD → authoritative.
3. **DNS record types?** A (IPv4), AAAA (IPv6), CNAME (alias), MX (mail), NS (nameserver), TXT (verification), PTR (reverse).
4. **ARP?** Maps an IP to a MAC address **on the local link**, by broadcast. Layer 2 and 3 meeting.
5. **DHCP?** DORA — Discover, Offer, Request, Acknowledge. Leases an address, mask, gateway and DNS server.
6. **ICMP?** Control and error messages. **ping** uses echo request/reply; **traceroute** walks the TTL up from 1.
7. **MTU and fragmentation?** Ethernet's MTU is 1500 bytes. IPv4 routers may fragment; IPv6 routers never do — the sender must discover the path MTU.
8. **TTL?** A hop counter, decremented by each router. At 0 the packet dies and ICMP reports it. It stops routing loops.
9. **Port ranges?** 0–1023 well-known, 1024–49151 registered, 49152–65535 ephemeral. Know 20/21, 22, 25, 53, 80, 443, 3306, 5432, 6379.
10. **What identifies a connection?** The 4-tuple: source IP, source port, destination IP, destination port.
11. **Switch vs router?** A switch forwards by MAC inside one network; a router forwards by IP between networks.
12. **Distance vector vs link state?** DV (RIP) tells neighbours about everyone, converges slowly, count-to-infinity. LS (OSPF) tells everyone about neighbours, then each router runs Dijkstra locally.
13. **Longest prefix match?** Of all matching routes, the most specific mask wins. A /28 route beats a /24 route.
14. **Default gateway?** The route of last resort, `0.0.0.0/0`, used when nothing more specific matches.
15. **Latency vs bandwidth vs jitter?** Delay of one packet / volume per second / the **variation** in delay, which is what breaks voice calls.
16. **Why is UDP used for video?** A late frame is worthless, so retransmission would hurt more than a dropped frame does.
17. **Sticky session?** The load balancer pins a client to one server. It works, but it makes that server's failure visible to the user — external session storage is the better answer.

E · OOP and dev fundamentals — Chapters 11 and 12

Encapsulation	Bundle the data with the methods and hide the internals. Private fields plus a controlled interface. JS: <code>#field</code> . Java/C++: <code>private</code> .
Abstraction	Expose what it does, hide how . An abstract class or interface names the contract; the implementation is somebody else's problem.
Inheritance	An “is-a” relationship that reuses a base class. Prefer composition (“has-a”) when the relationship is not genuinely is-a.
Polymorphism	One interface, many implementations. Compile-time = overloading. Run-time = overriding through a base reference.
Overloading vs overriding	Overloading: same name, different parameter list , same class, resolved at compile time — and JavaScript does not have it. Overriding: same signature, subclass , resolved at run time via the vtable.
Abstract class vs interface	Abstract class: may hold state and implemented methods, single inheritance, use for “is-a” with shared code. Interface: a pure contract, many may be implemented, use for capability. Java 8+ default methods blur the line but state still separates them.
Virtual function	A method resolved by the object's real type, not the reference type. C++ needs the virtual keyword; Java methods are virtual by default; JavaScript is always dynamic.
vtable	A per-class array of function pointers; each object stores a hidden vptr. That indirection is the entire run-time cost of polymorphism.
Why a virtual destructor?	Deleting a derived object through a base pointer without one skips the derived destructor and leaks. C++ only.
Diamond problem	Two parents inherit from one grandparent — whose copy does the child get? C++ answers with virtual inheritance; Java forbids multiple class inheritance and uses interfaces; JavaScript uses a single linear prototype chain plus mixins.
Association / aggregation / composition	Just knows about / has-a with independent lifetime (a team and its players) / has-a with shared lifetime (a house and its rooms — destroy one, destroy both).
SOLID	S one reason to change · O extend without editing · L a subtype must be substitutable for its base · I many small interfaces beat one fat one · D depend on abstractions, not concretions.
Shallow vs deep copy	Shallow copies references, so nested objects stay shared. Deep copies everything, recursively — <code>structuredClone</code> in modern JS.

static	Belongs to the class, not the instance. One copy for all objects. It cannot see this .
JS this	Decided by the call site , not by where the function was written. Arrow functions have no this of their own, which is exactly why they are safe as callbacks.
Prototype chain	JS class is syntax over prototypes. Lookup walks __proto__ upward until it finds the property or hits null .

E1 · From source to running program

Four stages	Preprocess (expand includes and macros) → Compile (to assembly) → Assemble (to an object file) → Link (resolve symbols into an executable).
Compiler vs interpreter vs JIT	Translate everything first, then run fast / translate and run line by line, easier to debug / interpret, then compile the hot paths at run time — what V8 does.
Front end	Lexer (characters → tokens) → Parser (tokens → syntax tree) → Semantic analysis (types, scopes, the symbol table).
Back end	Intermediate representation → optimisation → target code generation → register allocation.
“Undefined reference”	A link error: something was declared and used but never defined, or the library was not passed to the linker. Not a compile error.
Static vs dynamic linking	Static copies the library in: bigger file, no runtime surprises. Dynamic shares one .so/.dll : smaller, patchable, but the right version must be present.
Memory layout	Low to high: text (code, read-only) · data (initialised globals) · BSS (uninitialised globals, zeroed) · heap , growing up · stack , growing down.
Stack vs heap	Stack: automatic, LIFO, fast, small, freed on return. Heap: manual or garbage-collected, large, slower, can fragment.
Stack overflow	Recursion without a base case, or too deep. Each call frame holds return address, saved registers, parameters and locals.
Segfault	Touching memory you do not own: a null or dangling pointer, or running off the end of an array.
Floating point	$0.1 + 0.2 \neq 0.3$ because binary cannot represent 0.1 exactly. Compare with a tolerance; use integers or a decimal type for money.

E2 · Git

Three trees	Working directory → staging area (index) → repository. add moves left to middle, commit moves middle to right.
Object model	Four objects: blob (file contents), tree (a directory listing), commit (a tree plus parents plus a message), tag . Each is addressed by the SHA-1 of its content.
A branch is	A 41-byte file holding one commit hash. That is why branching is instant.
HEAD	A pointer to the current branch. Detached HEAD means it points straight at a commit instead.
Merge vs rebase	Merge keeps real history and adds a merge commit. Rebase replays your commits on top of the target, producing a straight line but new hashes . Never rebase a branch others have pulled.
reset vs revert	reset moves the branch pointer — history is rewritten (--soft keeps changes staged, --mixed unstages, --hard destroys them). revert adds a new commit that undoes an old one, and is the only safe choice on a shared branch.
fetch vs pull	fetch downloads. pull is fetch plus merge (or rebase with --rebase).
cherry-pick	Copy one commit's change onto the current branch as a new commit with a new hash.
stash	Park uncommitted work, clean the tree, restore it later with stash pop .
Recovering a lost commit	git reflog — it records every move of HEAD, so almost nothing is truly lost before garbage collection.
SemVer	MAJOR.MINOR.PATCH — breaking / backward-compatible feature / backward-compatible fix. The lock file is what makes a build reproducible.

IF YOU READ NOTHING ELSE, READ THIS

The twelve traps that cost the most marks.

1. Threads share code, data and heap — **not the stack**, not registers, not the program counter.
2. Deadlock needs **all four** conditions at the same time. Any one of them alone is not deadlock.
3. Belady's anomaly hits **FIFO only**. LRU and Optimal are stack algorithms and cannot suffer it.
4. Internal fragmentation belongs to **paging**; external fragmentation belongs to **segmentation** and contiguous allocation.
5. A hard link shares the **inode**; a soft link stores a **path**. Deleting the original breaks only the soft one.
6. **WHERE** filters rows, **HAVING** filters groups. An aggregate cannot appear in **WHERE**.
7. A right-table condition in the **WHERE** clause turns a **LEFT JOIN** into an **INNER JOIN**.
8. **TRUNCATE** is DDL: no **WHERE**, no per-row logging, and in most engines you cannot roll it back.
9. BCNF may cost you dependency preservation; 3NF never does. That is the whole 3NF-vs-BCNF answer.
10. An index is not free — it slows every write, and on a low-selectivity column the planner will ignore it.
11. The handshake exists to **synchronise sequence numbers**, and **TIME_WAIT** is correct behaviour, not a bug.
12. Overloading is **compile**-time and needs a different parameter list; overriding is **run**-time and needs the same signature.

And the four sentences that win the round. “The trade-off here is...” · “It depends on whether reads or writes dominate.”
· “I would measure before I optimise.” · “I have not used that, but from first principles it would have to work like this...”